



Facultad de Ingenierías y Tecnologías

PROTOTIPADO Y SIMULACIÓN

Apuntes del Curso

Periodo:	164
Clave del curso:	01DAO307
Campus:	Campestre
Nivel:	Doctorado
Unidad Académica:	Ingeniería y Tecnología Aplicada
Curso:	PROTOTIPADO Y SIMULACIÓN
Grupo:	301
Duración:	14 semanas
Horas por semana:	3 horas
Profesor:	Dr. Jorge Ramón Parra Michel

Objetivo General del Curso

Al final de este curso, el alumno será capaz de formular, implementar, validar y comunicar modelos computacionales y prototipos físicos o digitales aplicados a problemas reales de investigación científica y tecnológica, integrando herramientas de simulación numérica, modelado matemático, métodos estocásticos, análisis computacional, visualización científica y validación experimental con enfoque doctoral y criterios de reproducibilidad científica.

Asimismo, el estudiante desarrollará la capacidad de establecer relaciones entre fenómenos físicos reales, modelos matemáticos, simulaciones computacionales y prototipos experimentales, empleando metodologías modernas de ingeniería computacional orientadas al desarrollo de su proyecto de tesis doctoral.

Índice

1. Semana 1. Introducción al Modelado Científico	16
1.1. Motivacion doctoral: de la observacion al modelo	16
1.2. Cadena metodologica del curso	17
1.3. Problema bien planteado y problema inverso	18
1.4. Una primera simulacion: estimacion de π por Monte Carlo	20
1.4.1. Planteamiento	20
1.4.2. Tasa de convergencia y limitaciones	20
1.4.3.Codigo de ejemplo (Google Colab)	20
1.4.4. Lectura metodologica	23
1.5. Diferencia entre modelo fisico, matematico y computacional	25
1.5.1. Modelo fisico	25
1.6. Modelo matematico	25
1.6.1. Formulacion fuerte vs formulacion debil	25
1.6.2. Adimensionalizacion como diagnostico	26
1.7. Modelo computacional	26
1.7.1. Familias de esquemas y propiedades minimas	27
1.8. Abstraccion y simplificacion de fenomenos reales	28
1.8.1. Teorema II de Buckingham y reduccion sistematica	28
1.8.2. Analisis de sensibilidad	28
1.8.3. Ejemplo de abstraccion: cable OPGW sometido a cortocircuito .	29
1.8.4. Ejemplo de abstraccion: sistema RAG como prototipo compu- tacional	29
1.9. Variables, parametros y condiciones de frontera	31
1.9.1. Variables de estado	31
1.9.2. Parametros	31
1.9.3. Condiciones iniciales y de frontera	32
1.10. Escalas espaciales, temporales y numeros adimensionales	33
1.10.1. Otros grupos adimensionales relevantes	33
1.10.2. Escala numerica y condicion CFL	34
1.11. Error, precision e incertidumbre	36
1.11.1. Error total	36
1.11.2. Orden de convergencia y extrapolacion de Richardson	36
1.11.3. Verificacion y validacion (V&V)	37
1.11.4. Metricas de comparacion	37
1.11.5. Propagacion de incertidumbre	38
1.12. Reproducibilidad cientifica	40
1.12.1. Registro minimo de una simulacion	40

1.13. Ejemplo 1: conduccion transitoria en una barra	42
1.13.1. Planteamiento fisico	42
1.13.2. Modelo matematico	42
1.13.3. Modelo computacional	42
1.13.4.Codigo base en Python	42
1.13.5. Interpretacion esperada	45
1.14. Ejemplo 2: sistema masa–resorte–amortiguador	46
1.14.1. Planteamiento fisico	46
1.14.2. Modelo matematico	46
1.14.3. Discretizacion mediante Euler explicito	46
1.14.4. Actividad breve de analisis	47
1.15. Ejemplo 3: modelo termico simplificado para cable OPGW	48
1.15.1. Motivacion	48
1.15.2. Modelo concentrado	48
1.15.3. Criterio de validez	48
1.15.4. Prompt para que el estudiante genere codigo con IA	49
1.16. Ejemplo 4: bomba centrifuga y dinamica de fluidos computacional	50
1.16.1. Modelo fisico	50
1.16.2. Ecuaciones gobernantes	50
1.16.3. Indicadores de desempeno	51
1.16.4. Prototipado	51
1.17. Ejemplo 5: modelo basado en agentes para economia circular	52
1.17.1. Planteamiento	52
1.17.2. Ejemplo: adopcion de una tecnologia de reciclaje	52
1.17.3. Codigo conceptual minimo	52
1.18. Ejemplo 6: red neuronal informada por fisica	54
1.18.1. Idea central	54
1.18.2. Ejemplo con ecuacion de calor	54
1.19. Ejemplo 7: conduccion 2D en placa de intercambiador de calor	55
1.19.1. Motivacion fisica	55
1.19.2. Modelo matematico	55
1.19.3. Discretizacion explicita FTCS 2D	55
1.19.4. Criterio de estado estable	56
1.19.5. Codigo en Python (Google Colab)	56
1.19.6. Lectura tecnica de los resultados	61
1.19.7. Limitaciones del modelo	62
1.20. Ejemplo 8: analisis de esfuerzos de von Mises en una conexion viga- columna	63
1.20.1. Motivacion estructural	63
1.20.2. Modelo fisico y matematico	63

1.20.3. Discretizacion FEM con elementos CST	64
1.20.4. Limitaciones conocidas del CST	64
1.20.5. Codigo en Python (Google Colab)	64
1.20.6. Verificacion analitica de orden de magnitud	72
1.20.7. Interpretacion estructural	73
1.20.8. Conexion con investigacion doctoral	73
1.21. Discusion de los articulos cientificos como casos de investigacion real	74
1.21.1. Matriz comun de analisis	75
1.22. Ejercicios guiados para uso de IA generativa en simulacion	76
1.22.1. Principio operativo	76
1.22.2. Ejercicio IA-1: simulacion de difusion termica	76
1.22.3. Ejercicio IA-2: masa–resorte–amortiguador	77
1.22.4. Ejercicio IA-3: modelo concentrado de calentamiento OPGW	77
1.22.5. Ejercicio IA-4: modelo basado en agentes	77
1.22.6. Ejercicio IA-5: conduccion 2D en placa de intercambiador	77
1.22.7. Ejercicio IA-6: von Mises en conexion viga-columna	78
1.23. Actividad de refuerzo en clase: 30–40 minutos	79
1.24. Ejercicios para tarea	80
1.24.1. Ejercicio 1: clasificacion de modelos	80
1.24.2. Ejercicio 2: derivacion de ecuacion gobernante	80
1.24.3. Ejercicio 3: estabilidad numerica	80
1.24.4. Ejercicio 4: incertidumbre	80
1.24.5. Ejercicio 5: convergencia de Monte Carlo	80
1.24.6. Ejercicio 6: estado estable en placa 2D	81
1.24.7. Ejercicio 7: sensibilidad estructural a la malla	81
1.24.8. Ejercicio 8: prompt doctoral	81
1.25. Cierre conceptual	82
2. Semana 2. Herramientas Computacionales Científicas	84
2.1. Motivación: del lápiz al ecosistema reproducible	85
2.2. Python y MATLAB: criterios de elección	86
2.2.1. Aritmética IEEE 754: lo que el lenguaje no oculta	86
2.3. Manejo de arreglos y matrices	87
2.3.1. Vectorización frente a bucles	87
2.3.2. Broadcasting	88
2.3.3. Estructura de memoria y costo computacional	88
2.3.4. Matrices dispersas	89
2.4. Visualización científica	89
2.4.1. Plantilla mínima “publication-ready”	90
2.4.2. Mapas de color y percepción	91

2.5.	Scripts reproducibles y arquitectura de proyectos	91
2.5.1.	Script ejecutable con <code>argparse</code> y registro de parámetros	92
2.6.	Notebooks científicos: cuándo sí y cuándo no	95
2.6.1.	Google Colab: rol específico en el curso	96
2.7.	Librerías científicas fundamentales	96
2.7.1.	SymPy: cuándo lo simbólico ahorra tiempo	97
2.8.	Automatización de cálculos	97
2.8.1.	Makefile mínimo para un proyecto científico	98
2.9.	Buenas prácticas computacionales	99
2.9.1.	Mini-guía de Git para investigación	100
2.10.	Ejemplo 1: Álgebra lineal aplicada — SVD y reducción de dimensiona- lidad en datos experimentales	100
2.10.1.	Planteamiento	100
2.10.2.	Conexión doctoral	101
2.10.3.	Código en Python	101
2.10.4.	Lectura metodológica	104
2.11.	Ejemplo 2: Integración de ODE con <code>scipy.integrate.solve_ivp</code>	104
2.11.1.	Planteamiento	104
2.11.2.	Código en Python	104
2.11.3.	Lectura metodológica	106
2.12.	Ejemplo 3: pandas para manejo de datos experimentales	106
2.12.1.	Planteamiento	106
2.12.2.	Lectura metodológica	110
2.13.	Ejemplo 4: Diferenciación automática con <code>jax/autograd</code>	110
2.13.1.	Planteamiento	110
2.13.2.	Lectura metodológica	112
2.14.	Ejemplo 5: Pruebas unitarias para código numérico	112
2.14.1.	Planteamiento	112
2.15.	Ejercicios guiados para uso de IA generativa	114
2.15.1.	Principio operativo (repetido del Tema 1)	114
2.15.2.	Ejercicio IA-1: vectorización	114
2.15.3.	Ejercicio IA-2: SVD aplicada	115
2.15.4.	Ejercicio IA-3: SciPy <code>solve_ivp</code> comparativo	115
2.15.5.	Ejercicio IA-4: pandas + visualización	115
2.15.6.	Ejercicio IA-5: script ejecutable + Makefile	115
2.15.7.	Ejercicio IA-6: pruebas unitarias	116
2.16.	Actividad de refuerzo en clase: 30–40 minutos	116
2.16.1.	Nombre de la actividad	116
2.16.2.	Duración	116
2.16.3.	Organización	116

2.16.4. Producto entregable	116
2.16.5. Rúbrica rápida	117
2.17. Ejercicios para tarea	117
2.17.1. Ejercicio 1: refactor de la Semana 1 (Ejemplo 7)	117
2.17.2. Ejercicio 2: cuantificación de costo computacional	117
2.17.3. Ejercicio 3: estabilidad numérica	117
2.17.4. Ejercicio 4: visualización rigurosa	118
2.17.5. Ejercicio 5: pandas sobre datos reales	118
2.17.6. Ejercicio 6: diferenciación automática	118
2.17.7. Ejercicio 7: pruebas unitarias completas	118
2.17.8. Ejercicio 8: tu repositorio doctoral	118
2.18. Cierre conceptual	119
3. Semana 3. Métodos Numéricos Fundamentales	121
3.1. Motivación: por qué los métodos numéricos importan en investigación doctoral	122
3.2. Diferencias finitas	122
3.2.1. Derivación a partir de la expansión de Taylor	122
3.2.2. Error de redondeo y paso óptimo	123
3.2.3. Segunda derivada	123
3.2.4. Ejemplo 1 — Diferenciación numérica en el perfil de deformación de un cable OPGW	124
3.3. Integración numérica	127
3.3.1. Reglas de cuadratura clásicas	127
3.3.2. Cuadratura de Gauss–Legendre	128
3.3.3. Ejemplo 2 — Energía disipada por arco eléctrico en cortocircuito	128
3.4. Estabilidad numérica	131
3.4.1. Ecuaciones diferenciales ordinarias: esquemas explícitos e implícitos	131
3.4.2. Criterio de estabilidad CFL para EDPs	132
3.4.3. Ejemplo 3 — Dinámica térmica de cable OPGW: estabilidad de esquemas numéricos	132
3.5. Condicionamiento numérico	137
3.5.1. Número de condición y sensibilidad de la solución	137
3.5.2. Relación con el espectro de la matriz	138
3.5.3. Ejemplo 4 — Condicionamiento en sistemas de ecuaciones de ingeniería	138
3.6. Introducción a ecuaciones diferenciales parciales (EDPs)	142
3.6.1. Clasificación y formulación	142
3.6.2. Discretización: el método de líneas	143

3.6.3.	Selección del esquema numérico: criterios prácticos	143
3.7.	Síntesis: mapa conceptual de la Semana 3	144
3.8.	Anexo Semana 3. Ejemplos Complementarios	146
3.8.1.	Ejemplo A1 — Condicionamiento de matrices de similitud en sistemas RAG	146
3.8.2.	Ejemplo A2 — Convergencia de malla en diferencias finitas para losas	149
3.8.3.	Ejemplo A3 — Flujo vehicular LWR y optical flow para UAV	152
3.8.4.	Ejemplo A4 — Integración numérica de la curva Q-H de bomba centrífuga	156
3.9.	Ejercicios Reto — Tarea Semana 3	159
3.9.1.	Ejercicio Reto 1 — Diferenciación numérica de señales experimentales con ruido	159
3.9.2.	Ejercicio Reto 2 — Análisis de estabilidad de la ecuación de calor 2D transitoria	160
3.9.3.	Ejercicio Reto 3 — Propagación de incertidumbre en el modelo termoeléctrico OPGW	161
4.	Semana 4. Modelado de Fenómenos Físicos	164
4.1.	Motivación: ¿Qué es una EDP y por qué importa en el doctorado?	165
4.2.	Clasificación de EDPs de segundo orden	165
4.3.	Ecuación de difusión (calor, masa, humedad)	166
4.3.1.	Formulación canónica	166
4.3.2.	Discretización explícita FTCS (Forward-Time Centered-Space)	166
4.3.3.	Versión 2D — Ecuación de Poisson (estado estacionario)	167
4.4.	Condiciones iniciales y de frontera	167
4.5.	Ecuación de onda y transporte hiperbólico	168
4.5.1.	Ecuación de transporte puro (advección)	168
4.6.	Convección-difusión: el número de Péclet	169
4.7.	Ejemplo 1 — Difusión de calor 1D transitoria en cable OPGW (Méndez Gutiérrez / Jurado Pérez)	169
4.7.1.	Planteamiento físico	169
4.7.2.	Modelo matemático	170
4.7.3.	Resultados numéricos	170
4.8.	Ejemplo 2 — Ecuación de transporte: propagación de densidad vehicular con monitoreo UAV (García Ojeda)	171
4.8.1.	Planteamiento físico	171
4.8.2.	Comparación de esquemas	171
4.9.	Ejemplo 3 — Ecuación de Poisson 2D: losa de concreto con calor de hidratación (Lerma Lomelí / Lerma Martínez)	173

4.9.1.	Planteamiento físico	173
4.9.2.	Configuración del problema	173
4.9.3.	Resultados numéricos	173
4.10.	Ejemplo 4 — Convección-difusión 1D: transporte de marcador en bomba centrífuga (Zavala Morales)	175
4.10.1.	Planteamiento físico	175
4.10.2.	Efecto del número de Péclet	175
4.11.	Síntesis: mapa conceptual de la Semana 4	176
4.12.	Anexo Semana 4. Códigos para Google Colab	177
4.12.1.	Código 1 — Difusión de calor 1D transitoria en cable OPGW	177
4.12.2.	Código 2 — Ecuación de transporte (onda): densidad vehicular con UAV	182
4.12.3.	Código 3 — Ecuación de Poisson 2D: losa de concreto	186
4.12.4.	Código 4 — Convección-difusión 1D y número de Péclet	191
4.13.	Ejercicios Reto — Tarea Semana 4	195
4.13.1.	Ejercicio 1 — Análisis de la estabilidad FTCS (Nivel intermedio)	195
4.13.2.	Ejercicio 2 — Condición de Robin y convección superficial (Nivel intermedio)	196
4.13.3.	Ejercicio 3 — Difusión-reacción: carbonatación de concreto (Nivel avanzado, punto extra)	196
5.	Semana 5. Simulación Bidimensional de Campos	198
5.1.	Motivación: del eje 1D al campo 2D	199
5.2.	Mallado bidimensional y discretización del laplaciano	199
5.2.1.	Malla cartesiana estructurada	200
5.2.2.	El stencil de cinco puntos	200
5.2.3.	Por qué el almacenamiento disperso es obligatorio	200
5.3.	Estabilidad en dos dimensiones	201
5.4.	Geometrías complejas sobre mallas cartesianas: máscaras	201
5.4.1.	Máscara booleana y frontera escalonada	201
5.4.2.	Condiciones de frontera sobre la máscara	202
5.4.3.	Interfaces entre materiales: media armónica	202
5.5.	Ejemplo 1 — Difusión 2D transitoria en una sección compuesta con generación interna	202
5.5.1.	Planteamiento	202
5.5.2.	Discretización	203
5.5.3.	Resultados y lectura metodológica	203
5.6.	Ejemplo 2 — Onda 2D con obstáculo: reflexión y difracción	204
5.6.1.	Planteamiento	204
5.6.2.	Discretización	204

5.6.3.	Resultados y lectura metodológica	205
5.7.	Ejemplo 3 — Poisson 2D en una placa perforada: el papel decisivo de las fronteras internas	206
5.7.1.	Planteamiento	206
5.7.2.	Discretización: ensamblaje disperso sobre el dominio enmascarado	206
5.7.3.	Resultados y lectura metodológica	207
5.8.	Ejemplo 4 — Convección-difusión 2D en un flujo recirculante: el número de Péclet de malla	207
5.8.1.	Planteamiento	207
5.8.2.	Discretización y el parámetro que gobierna todo: Pe_h	208
5.8.3.	Resultados y lectura metodológica	209
5.9.	Ejemplo 5 — Advección pura bajo rotación sólida: disipación contra dispersión	209
5.9.1.	Planteamiento	209
5.9.2.	Esquemas comparados	209
5.9.3.	Resultados y lectura metodológica	210
5.10.	Ejemplo 6 — Reconstrucción de un campo a partir de mediciones dispersas: interpolación armónica	211
5.10.1.	Planteamiento	211
5.10.2.	Estudio de convergencia con la densidad de sensores	211
5.10.3.	Resultados y lectura metodológica	212
5.11.	Visualización de campos escalares: reglas mínimas	212
5.12.	Síntesis de la semana	213
5.13.	Anexo Semana 5. Códigos para Google Colab	214
5.13.1.	Código 1 — Difusión 2D en sección compuesta (máscara circular y media armónica)	214
5.13.2.	Código 2 — Onda 2D con rendija (leapfrog y conservación de energía)	220
5.13.3.	Código 3 — Poisson 2D en placa perforada (ensamblaje disperso)	225
5.13.4.	Código 4 — Convección-difusión 2D en un vórtice (Péclet de malla)	230
5.13.5.	Código 5 — Advección 2D bajo rotación sólida (upwind vs MacCormack)	235
5.13.6.	Código 6 — Reconstrucción armónica desde sensores dispersos	240
5.14.	Ejercicios Reto — Tarea Semana 5	244
5.14.1.	Ejercicio 1 — Límite CFL 2D y doble rendija (Nivel intermedio)	245
5.14.2.	Ejercicio 2 — Condición de Robin en las cavidades: del escenario sellado al ventilado (Nivel intermedio)	245
5.14.3.	Ejercicio 3 — Esquema híbrido con limitador: evadir el teorema de Godunov (Nivel avanzado, punto extra)	246

6. Semana 6. Cuantificación de incertidumbre: GUM y métodos Monte Carlo 248

6.1.	Del campo determinista al mensurando con incertidumbre	249
6.2.	Evaluación Tipo A y Tipo B de la incertidumbre estándar	249
6.2.1.	Tipo A: evaluación estadística de una serie de observaciones	249
6.2.2.	Tipo B: evaluación a partir de información previa	250
6.2.3.	Criterio práctico de selección de la PDF	259
6.3.	Ley de propagación de incertidumbres	260
6.3.1.	Incertidumbre expandida y factor de cobertura	260
6.4.	Asignación de funciones de densidad de probabilidad	261
6.5.	Generación de números pseudoaleatorios	261
6.5.1.	Generadores de base	262
6.5.2.	De uniforme a cualquier distribución	262
6.6.	Método Monte Carlo para la propagación de distribuciones	262
6.6.1.	Validación de la GUM mediante Monte Carlo	263
6.7.	Ejemplo 1 — Calibración de un sensor: el caso canónico GUM con validación Monte Carlo	263
6.7.1.	Planteamiento	263
6.7.2.	Propagación analítica (LPU)	264
6.7.3.	Resultados y lectura metodológica	265
6.8.	Ejemplo 2 — Elevación de temperatura por cortocircuito en un cable OPGW: no linealidad y probabilidad de falla	265
6.8.1.	Planteamiento	266
6.8.2.	Resultados y lectura metodológica	266
6.9.	Ejemplo 3 — Confiabilidad estructural de losas: índice de confiabilidad y no normalidad en la cola	267
6.9.1.	Planteamiento	267
6.9.2.	Resultados y lectura metodológica	268
6.10.	Ejemplo 4 — Optimización de una bomba centrífuga bajo incertidumbre: modelo multiplicativo y BEP	269
6.10.1.	Planteamiento	269
6.10.2.	Propagación analítica y optimización	270
6.10.3.	Resultados y lectura metodológica	270
6.11.	Ejemplo 5 — Velocimetría vehicular por flujo óptico desde UAV: incertidumbre correlacionada	271
6.11.1.	Planteamiento	271
6.11.2.	Propagación analítica (LPU)	271
6.11.3.	Resultados y lectura metodológica	272
6.12.	Ejemplo 6 — Exactitud de un sistema RAG industrial: incertidumbre de soporte acotado	272
6.12.1.	Planteamiento	272

6.12.2. Propagación analítica y su falla en el borde	273
6.12.3. Resultados y lectura metodológica	274
6.13. Síntesis de la semana	274
6.14. Anexo Semana 6. Códigos para Google Colab	276
6.14.1. Código 1 — Calibración de sensor: GUM vs Monte Carlo . . .	276
6.14.2. Código 2 — Cortocircuito en cable OPGW: no linealidad y probabilidad de falla	281
6.14.3. Código 3 — Confiabilidad estructural: losa maciza vs reticular .	285
6.14.4. Código 4 — Optimización de bomba centrífuga: modelo multiplicativo y BEP	290
6.14.5. Código 5 — Velocimetría por flujo óptico desde UAV: incertidumbre correlacionada	294
6.14.6. Código 6 — Exactitud de un sistema RAG: soporte acotado . .	299
6.15. Ejercicios Reto — Tarea Semana 6	303
6.15.1. Ejercicio 1 (obligatorio) — Modelo de medición y propagación analítica	303
6.15.2. Ejercicio 2 (obligatorio) — Propagación Monte Carlo y validación de la GUM	304
6.15.3. Ejercicio 3 — Procedimiento Monte Carlo adaptativo	304
7. Semana 7. Cadenas de Markov y Sistemas Dinámicos	305
7.1. Procesos estocásticos y la propiedad de Markov	306
7.2. Matriz de transición: evolución y ecuaciones de Chapman–Kolmogorov	307
7.3. Clasificación de estados, distribución estacionaria y teorema ergódico .	308
7.4. Cadenas absorbentes, tiempos de primer paso y matriz fundamental . .	309
7.5. Cadenas de Markov de tiempo continuo: matriz generadora	309
7.6. Simulación estocástica exacta: el algoritmo de Gillespie	310
7.7. Markov en el espacio y en grafos: autómatas celulares, percolación y teoría espectral	311
7.7.1. Campos de Markov espaciales y percolación	311
7.7.2. Caminatas aleatorias en grafos, teoría espectral y mezcla	311
7.8. Del proceso estocástico al sistema dinámico determinista	312
7.9. Interpretación estadística: estimación de la matriz de transición	313
7.10. Ejemplo 1 — Deriva de calibración de un sensor: la cadena de dos estados	313
7.10.1. Planteamiento	313
7.10.2. Resultados y lectura metodológica	314
7.11. Ejemplo 2 — Confiabilidad de un banco de pruebas: cadena con reparación	315
7.11.1. Planteamiento	315
7.11.2. Disponibilidad y MTTF	316
7.11.3. Resultados y lectura metodológica	316

7.12. Ejemplo 3 — Confiabilidad de un OPGW: cadena de Markov de tiempo continuo	317
7.12.1. Planteamiento	317
7.12.2. Resultados y lectura metodológica	318
7.13. Ejemplo 4 — Incendio forestal: autómatas celulares probabilísticos y percolación	319
7.13.1. Planteamiento	319
7.13.2. Resultados y lectura metodológica	320
7.14. Ejemplo 5 — Modelo epidemiológico estocástico: del SIR de Markov al sistema dinámico	321
7.14.1. Planteamiento	321
7.14.2. Resultados y lectura metodológica	322
7.15. Ejemplo 6 — Ranking por caminata aleatoria (PageRank) para recuperación en RAG	323
7.15.1. Planteamiento	323
7.15.2. Resultados y lectura metodológica	324
7.16. Síntesis de la semana	325
7.17. Anexo Semana 7. Códigos para Google Colab	327
7.17.1. Código 1 — Deriva de calibración: cadena de Markov de dos estados	327
7.17.2. Código 2 — Confiabilidad de un banco de pruebas: DTMC con reparación	334
7.17.3. Código 3 — Confiabilidad de un OPGW: cadena de tiempo continuo	340
7.17.4. Código 4 — Incendio forestal: autómatas celulares probabilísticos	346
7.17.5. Código 5 — SIR estocástico: Gillespie y límite determinista	352
7.17.6. Código 6 — PageRank: caminata aleatoria para ranking en RAG	359
7.18. Ejercicios Reto — Tarea Semana 7	365
7.18.1. Ejercicio 1 (obligatorio) — Modelo de Markov del propio proyecto	365
7.18.2. Ejercicio 2 (obligatorio) — Tiempos característicos y validación por simulación	366
7.18.3. Ejercicio 3 — Transición de fase o propagación de incertidumbre	366
8. Semana 8. Método de Elementos Finitos I	367
8.1. Introducción al FEM	367
8.2. Discretización espacial	368
8.3. Elementos y nodos	369
8.4. Funciones de forma	372
8.5. Ensamble matricial	373
8.6. Generación de mallas	374

8.7. Condiciones de frontera	378
8.8. Flujo de trabajo en ANSYS Workbench – Static Structural	381
8.9. Aplicación a los proyectos doctorales	381
8.10. Anexo A: Código Python (Google Colab)	383
8.11. Anexo B: Referencias	390
8.12. Ejercicios	390
9. Semanas 9 y 10. Método de Elementos Finitos II: Análisis Estructural, Fati-	
ga, Vibraciones y Dinámica de Mecanismos	393
9.1. De la formulación matemática al análisis ingenieril en Workbench . . .	393
9.2. Análisis estructural estático: esfuerzos y deformaciones	394
9.2.1. Fundamento: de los desplazamientos al factor de seguridad . . .	394
9.2.2. Flujo de trabajo en Static Structural	395
9.2.3. Ejemplo guiado 1: placa con agujero central a tensión	395
9.3. Análisis de fatiga: del esfuerzo a la vida	399
9.3.1. Fundamento: los dos enfoques y la corrección por esfuerzo medio	399
9.3.2. Árbol de decisión del Fatigue Tool	401
9.3.3. Ejemplo guiado 2: fatiga de la placa con agujero bajo carga pulsante	401
9.4. Análisis modal: frecuencias naturales y modos de vibración	404
9.4.1. Fundamento	404
9.4.2. Flujo de trabajo y Ejemplo guiado 3: viga en voladizo	404
9.5. Dinámica de cuerpos rígidos: cinemática de mecanismos acoplados . .	406
9.5.1. Fundamento: el mecanismo biela–manivela–corredera	406
9.5.2. Flujo de trabajo en Rigid Dynamics	407
9.5.3. Ejemplo guiado 4: biela–manivela–corredera a 3000 rpm	407
9.6. Análisis acoplado: esfuerzos en la biela de un mecanismo en movimiento	409
9.6.1. Estrategias de acoplamiento	410
9.6.2. Ejemplo guiado 5: esfuerzos y pandeo en la biela en el punto muerto superior	410
9.7. Verificación cruzada: tabla de control analítico	412
9.8. Síntesis de la semana	413
9.9. Referencias de la Semana 9	413
9.10. Anexo: códigos Python de verificación analítica (Google Colab)	414
9.10.1. Script 1: referencia del estático (placa con agujero) y del modal (viga en voladizo)	414
9.10.2. Script 2: curva S–N de Basquin, envolventes de esfuerzo medio y vida del Ejemplo guiado 2	416
9.10.3. Script 3: cinemática del biela–manivela, fuerza en la biela y pan- deo de Euler	418
9.11. Ejercicios para realizar en casa	421

9.11.1. Tarea 1: análisis estructural completo de un soporte en L	421
9.11.2. Tarea 2: mecanismo biela–manivela propio y esfuerzos en su biela	421
10. Semana 11. Validación Experimental y Metrología Computacional	423
10.1. Fundamentación teórico-matemática	424
10.1.1. Taxonomía: verificación, validación y calibración	424
10.1.2. Marco formal ASME V&V 20: desacoplamiento de incertidumbres	424
10.1.3. Componente 1: incertidumbre numérica u_{num} (GCI)	425
10.1.4. Componente 2: incertidumbre de entradas u_{input} (ISO-GUM) .	426
10.1.5. Componente 3: incertidumbre experimental u_D	426
10.1.6. Regla de decisión	426
10.1.7. Métrica de área para respuestas estocásticas	427
10.2. Ejemplo 1. Validación V&V 20 de un modelo térmico transitorio	427
10.3. Ejemplo 2. Métrica de área sobre CDFs: carga de una bomba centrífuga	430
10.4. Jerarquía piramidal de validación	431
10.5. Tabla de síntesis y control	432
10.6. Uso guiado de inteligencia artificial generativa	432
10.7. Actividad de refuerzo en clase (45 min)	433
10.7.1. Actividad 1. Auditoría de sobreajuste contra validación genuina	433
10.8. Ejercicios de investigación para tarea	434
10.8.1. Ejercicio 1 (obligatorio) — Presupuesto de validación del propio proyecto	434
10.8.2. Ejercicio 2 (obligatorio) — Métrica de área con datos propios .	434
10.9. Cierre conceptual	435
10.10 Referencias bibliográficas	435
10.11 Anexo: códigos de la Semana 11	435
10.11.1. Código 1. Framework ASME V&V 20 con GCI y banda de va- lidación	436
10.11.2. Código 2. Métrica de área sobre CDFs	439
11. Semana 12 y 13. Fotogrametría y Escaneo 3D	441
11.1. Fundamentación teórico-matemática	442
11.1.1. Modelo proyectivo de cámara y distorsión de lente	443
11.1.2. Geometría epipolar y triangulación	445
11.1.3. Structure from Motion y ajuste de haces	446
11.1.4. Densificación MVS y resolución del muestreo	447
11.1.5. Escala metrológica y propagación de su incertidumbre	448
11.2. De la nube de puntos a la malla triangular	448
11.2.1. La nube como muestreo ruidoso de una variedad	448
11.2.2. Submuestreo por vóxeles y estimación de normales	449

11.2.3. Registro multivista: el algoritmo ICP	450
11.2.4. Reconstrucción implícita de la superficie	452
11.2.5. Auditoría topológica de la malla	453
11.2.6. Suavizado y decimación	454
11.3. Preparación geométrica para simulación	455
11.3.1. Dos rutas de conversión	455
11.3.2. Resolución de la teselación frente al tamaño de elemento	456
11.3.3. Métricas de calidad heredadas	457
11.4. Protocolo operativo: escaneo con Meshroom 2023.3.0	457
11.4.1. Instalación y requisitos de cómputo	457
11.4.2. Configuración de la cámara	458
11.4.3. Planificación de la trayectoria	459
11.4.4. Grafo de nodos y ejecución	460
11.4.5. Criterios de aceptación de la nube densa	461
11.5. Ejemplo 1. Presupuesto óptico de la sesión de captura	462
11.6. Ejemplo 2. Banco sintético de <i>Structure from Motion</i>	464
11.7. Ejemplo 3. De la nube de puntos al STL auditado	465
11.8. Aplicaciones en ingeniería y enlace con los proyectos del grupo	467
11.9. Tabla de síntesis y control	468
11.10. Uso guiado de inteligencia artificial generativa	469
11.11. Actividad de refuerzo en clase (45 min)	469
11.11.1. Actividad 1. Diagnóstico de reconstrucciones fallidas	469
11.11.2. Actividad 2. Cálculo del presupuesto sobre el equipo propio	470
11.12. Práctica oficial. Escaneo 3D de un objeto y entrega del modelo STL	470
11.12.1. Selección del objeto	470
11.12.2. Procedimiento	470
11.12.3. Entregables	471
11.12.4. Criterios de aceptación y rúbrica	472
11.13. Ejercicios de investigación para tarea	472
11.13.1. Ejercicio 1 (obligatorio) — Presupuesto de reconstrucción del propio proyecto	472
11.13.2. Ejercicio 2 (obligatorio) — Sensibilidad del posprocesamiento	473
11.13.3. Ejercicio 3 (optativo) — Contraste entre técnicas	473
11.14. Cierre conceptual	474
11.15. Referencias bibliográficas	474
11.16. Anexo: códigos de la Semana 12	475
11.16.1. Código 1. Presupuesto óptico de la sesión fotogramétrica	475
11.16.2. Código 2. Banco sintético de <i>Structure from Motion</i>	479
11.16.3. Código 3. De la nube de puntos al STL auditado	485

1. Semana 1. Introducción al Modelado Científico

Objetivo de la cátedra

Comprender los fundamentos conceptuales del modelado científico y su importancia dentro de la investigación doctoral aplicada a ingeniería y tecnologías.

Temario

- Introducción a la simulación científica.
- Diferencia entre modelo físico, matemático y computacional.
- Abstracción y simplificación de fenómenos reales.
- Variables, parámetros y condiciones de frontera.
- Escalas espaciales y temporales.
- Error, precisión e incertidumbre.
- Reproducibilidad científica.
- Discusión de casos reales de investigación doctoral.

Propósito del tema. En esta semana se establece la base conceptual y matemática del curso. Se analiza la relación entre fenómeno real, modelo físico, modelo matemático, modelo computacional, simulación, prototipo y validación. El objetivo no consiste en aprender un software específico, sino en comprender cómo se transforma un problema de investigación en un sistema formal capaz de generar predicciones, estimaciones, escenarios y decisiones técnicamente justificadas.

Resultado de aprendizaje esperado

Al finalizar la sesión, el estudiante será capaz de formular un problema de investigación como un modelo científico explícito, identificando sus variables de estado, parámetros, condiciones iniciales, condiciones de frontera, hipótesis de simplificación, fuentes de error, indicadores de validación y requisitos mínimos de reproducibilidad. Además, deberá ser capaz de utilizar una herramienta de inteligencia artificial generativa como apoyo para construir, ejecutar y documentar una simulación computacional simple, sin delegar el juicio científico al modelo generativo, porque la IA redacta código, no sustituye el criterio metodológico; sería pedirle brújula moral a una calculadora.

1.1. Motivación doctoral: de la observación al modelo

Una investigación doctoral aplicada en ingeniería y tecnología rara vez inicia con una ecuación. Normalmente inicia con un fenómeno: un cable que se calienta bajo cortocircuito, una bomba centrífuga que produce hemólisis, una estructura que debe soportar

carga, una imagen medica que debe clasificarse, un sistema de trafico que debe controlarse o una cadena de decisiones humanas que debe representarse mediante agentes. La ecuacion aparece despues, cuando el fenomeno se abstraee en terminos de variables, restricciones, parametros y leyes de evolucion.

En terminos generales, se considera un sistema fisico, biologico, social o computacional definido sobre un dominio espacial $\Omega \subset \mathbb{R}^d$ y un intervalo temporal $t \in [0, T]$. El estado del sistema se representa mediante una funcion

$$\mathbf{u} : \Omega \times [0, T] \rightarrow \mathbb{R}^m, \quad (1)$$

donde $\mathbf{u}(\mathbf{x}, t)$ agrupa las variables relevantes del problema. Por ejemplo, en transferencia de calor, $\mathbf{u}$ puede ser la temperatura $T(\mathbf{x}, t)$; en mecanica estructural, el desplazamiento $\mathbf{u}(\mathbf{x}, t)$; en dinamica de fluidos, el campo de velocidades $\mathbf{v}(\mathbf{x}, t)$ y la presion $p(\mathbf{x}, t)$; en un sistema RAG, el estado computacional puede expresarse como un conjunto de documentos recuperados, vectores de representacion y distribuciones de probabilidad sobre secuencias de texto.

La estructura minima de un modelo cientifico puede escribirse como

$$\mathcal{M} : (\mathbf{x}, t; \boldsymbol{\theta}, \mathbf{c}) \mapsto \hat{\mathbf{u}}(\mathbf{x}, t), \quad (2)$$

donde $\boldsymbol{\theta}$ representa parametros del modelo, $\mathbf{c}$ representa controles, entradas o condiciones externas, y $\hat{\mathbf{u}}$ representa la prediccion del modelo. Una medicion experimental se expresa como

$$\mathbf{y} = \mathcal{H}(\mathbf{u}) + \boldsymbol{\varepsilon}, \quad (3)$$

donde $\mathcal{H}$ es el operador de observacion y $\boldsymbol{\varepsilon}$ representa ruido, sesgo, error de instrumentacion o incertidumbre experimental. Esta formulacion permite distinguir con claridad tres cosas que suelen mezclarse de manera peligrosa: el fenomeno real, el modelo que lo representa y el dato que se mide.

1.2. Cadena metodologica del curso

En este curso se adoptara la siguiente cadena:

$$\begin{aligned} \text{Fenomeno real} &\rightarrow \text{Modelo fisico} \rightarrow \text{Modelo matematico} \rightarrow \text{Modelo computacional} \\ &\rightarrow \text{Simulacion} \rightarrow \text{Prototipo} \rightarrow \text{Validacion} \end{aligned} \quad (4)$$

Esta cadena no es lineal en sentido rigido. En investigacion real, se avanza y se retrocede. Una simulacion puede revelar que una hipotesis fisica es falsa; un prototipo puede mostrar que una variable omitida domina el fenomeno; una validacion experimental puede obligar a reformular el modelo. La madurez doctoral consiste precisamente en

documentar ese ciclo sin fingir que todo salio perfecto desde el primer intento, una fantasia que solo sobrevive en informes administrativos y cuentos infantiles. La estructura conceptual general del curso se resume en la [Figura 1](#), donde se establece la transicion entre fenomeno real, formulacion matematica, simulacion computacional, prototipado y validacion experimental.

1.3. Problema bien planteado y problema inverso

Un modelo no se considera utilizable solo porque tenga ecuaciones. En el sentido de Hadamard, un problema esta *bien planteado* (well-posed) si cumple tres condiciones:

1. **Existencia.** Existe una solucion u para los datos (θ, c, u_0, g) admisibles.
2. **Unicidad.** La solucion es unica en el espacio funcional considerado.
3. **Estabilidad continua.** La solucion depende continuamente de los datos:

$$\|u_1 - u_2\| \leq C \|d_1 - d_2\|, \quad (5)$$

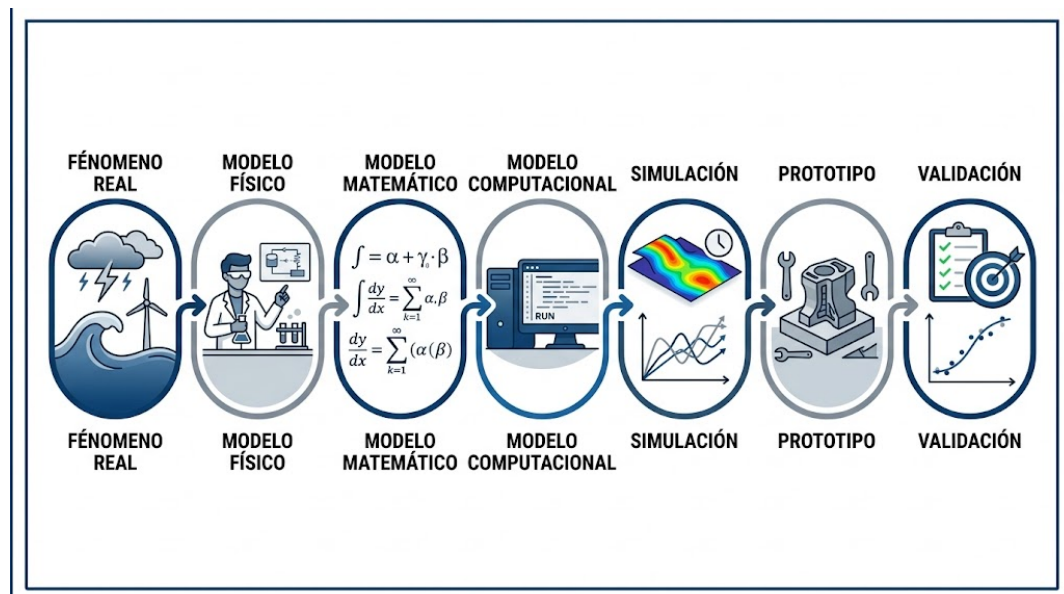
donde d agrupa los datos del problema y C es independiente de la solucion.

La falla del tercer requisito es la mas peligrosa, porque produce modelos que “funcionan” hasta que un dato cambia ligeramente. En problemas *directos* (forward problems) suele cumplirse, pero en problemas *inversos* (estimar θ a partir de y) la estabilidad rara vez es automatica y se requiere regularizacion: Tikhonov, restricciones bayesianas o priors fisicos. Buena parte de la metrologia, calibracion experimental y data assimilation cae en este regimen.

Un concepto relacionado es la *identifiabilidad* estructural y practica de los parametros: aunque el problema sea bien planteado para u , distintos θ pueden producir $\hat{u}$ indistinguibles dentro del ruido ϵ . Antes de invertir esfuerzo computacional en calibracion, se debe verificar que los datos disponibles tienen sensibilidad suficiente respecto a los parametros que se buscan estimar.

Tabla 1: Correspondencia entre niveles de representacion cientifica.

Nivel	Pregunta central	Ejemplo
Fenomeno real	?Que ocurre en el sistema observado?	Un cable OPGW se calienta durante un cortocircuito.
Modelo fisico	?Que mecanismos dominan el comportamiento?	Efecto Joule, conduccion termica, conveccion, radiacion y limites termicos del material.
Modelo matematico	?Que ecuaciones representan el fenomeno?	Ecuacion de calor transitoria con fuente volumetrica dependiente de corriente electrica.
Modelo computacional	?Como se discretiza y resuelve el modelo?	Diferencias finitas, elementos finitos o volumen finito.
Simulacion	?Que escenarios se evaluan?	Corrientes, tiempos de exposicion, materiales y condiciones ambientales.
Prototipo	?Que implementacion material o digital permite verificar la idea?	Ensayo fisico, banco de prueba, software reproducible o prototipo experimental.
Validacion	?Que tan bien coincide el modelo con evidencia independiente?	Comparacion con mediciones de temperatura, dano observable o curvas experimentales.


Figura 1: Cadena metodologica entre fenomeno real, modelo, simulacion, prototipo y validacion.

1.4. Una primera simulacion: estimacion de π por Monte Carlo

Antes de entrar en modelos basados en ecuaciones diferenciales, conviene introducir un ejemplo minimo de simulacion estocastica. El metodo Monte Carlo se basa en sustituir un calculo determinista por un experimento numerico repetido sobre variables aleatorias. Su utilidad excede al ejemplo didactico: aparece en propagacion de incertidumbre, integracion en alta dimension, optimizacion estocastica, fisica de particulas, finanzas computacionales, trazado de rayos y validacion de modelos.

1.4.1. Planteamiento

Sea el cuadrado unitario $\Omega_S = [0, 1] \times [0, 1]$ y el circulo inscrito Ω_C de centro $(\frac{1}{2}, \frac{1}{2})$ y radio $r = \frac{1}{2}$. Por construccion,

$$\frac{|\Omega_C|}{|\Omega_S|} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}. \quad (6)$$

Si se generan N pares $\{(x_i, y_i)\}_{i=1}^N$ con $x_i, y_i \sim \mathcal{U}(0, 1)$ independientes, y N_{in} es el numero de muestras que satisfacen $(x_i - \frac{1}{2})^2 + (y_i - \frac{1}{2})^2 \leq r^2$, entonces el estimador

$$\hat{\pi}_N = 4 \frac{N_{\text{in}}}{N} \quad (7)$$

es un estimador insesgado de π . La ley fuerte de los grandes numeros garantiza $\hat{\pi}_N \rightarrow \pi$ casi seguramente cuando $N \rightarrow \infty$.

1.4.2. Tasa de convergencia y limitaciones

La variable aleatoria $\mathbb{K}_{\Omega_C}(x, y)$ es Bernoulli con parametro $p = \pi/4$. Por tanto la varianza del estimador es

$$\text{Var}[\hat{\pi}_N] = 16 \frac{p(1-p)}{N} \approx \frac{16 \cdot (\pi/4)(1 - \pi/4)}{N} \approx \frac{2.697}{N}, \quad (8)$$

y la desviacion estandar decae como $\sigma \propto N^{-1/2}$. Esta tasa es independiente de la dimension del problema (a diferencia de las cuadraturas de Newton-Cotes, cuyo error crece exponencialmente con la dimension). Por eso Monte Carlo es competitivo en integrales de alta dimension, pero ineficiente para ganar dos cifras decimales adicionales (cada cifra extra requiere $\sim 100\times$ mas muestras).

1.4.3.Codigo de ejemplo (Google Colab)


```

2      Estimacion del numero PI mediante simulacion Monte Carlo
3      .
4      Geometria: cuadrado [0,1]x[0,1] con circulo inscrito de
5      radio r=0.5
6      centrado en (0.5, 0.5).
7
8      Principio: si se generan N puntos uniformes en el
9      cuadrado y N_in
10     caen dentro del circulo, entonces:
11     Area_circulo / Area_cuadrado = pi*r^2 / (2r)^2 = pi/4
12     => pi ~ 4 * N_in / N
13
14     Convergencia teorica: error ~ 1/sqrt(N) (Ley de los
15     grandes numeros).
16     """
17
18     import numpy as np
19     import matplotlib.pyplot as plt
20
21     # ---- Parametros de la simulacion
22     -----
23     np.random.seed(42)
24     N = 100_000                # numero total de muestras
25     r = 0.5                   # radio del circulo
26     inscrito
27     cx, cy = 0.5, 0.5        # centro del circulo
28
29     # ---- Generacion de puntos uniformes en el cuadrado
30     -----
31     x = np.random.uniform(0.0, 1.0, N)
32     y = np.random.uniform(0.0, 1.0, N)
33
34     # ---- Indicador de pertenencia al circulo
35     -----
36     dentro = (x - cx)**2 + (y - cy)**2 <= r**2
37     N_in    = np.count_nonzero(dentro)
38
39     # ---- Estimador de pi
40     -----
41     pi_est  = 4.0 * N_in / N
42     err_abs = abs(pi_est - np.pi)
43     err_rel = err_abs / np.pi

```

```

35
36     print(f"N          = {N}")
37     print(f"N_in       = {N_in}")
38     print(f"pi_est     = {pi_est:.6f}")
39     print(f"pi_real    = {np.pi:.6f}")
40     print(f"err_abs    = {err_abs:.2e}")
41     print(f"err_rel    = {err_rel:.2e}")
42
43     # ---- Convergencia: pi estimado vs numero de muestras
44     -----
45     n_grid = np.unique(np.logspace(1, np.log10(N), 80).
46     astype(int))
47     pi_seq = np.array([4.0 * np.count_nonzero(dentro[:n]) /
48     n for n in n_grid])
49
50     # ---- Visualizacion
51     -----
52     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
53
54     # Panel 1: nube de puntos
55     ax = axes[0]
56     ax.scatter(x[dentro][:5000], y[dentro][:5000], s=2, c=
57     'tab:blue',
58     label=f'dentro ({N_in}/{N})')
59     ax.scatter(x[~dentro][:5000], y[~dentro][:5000], s=2, c=
60     'tab:red',
61     label=f'fuera ({N-N_in}/{N})')
62     theta = np.linspace(0, 2*np.pi, 400)
63     ax.plot(cx + r*np.cos(theta), cy + r*np.sin(theta), 'k-'
64     , lw=1.5)
65     ax.set_aspect('equal'); ax.set_xlim(0, 1); ax.set_ylim
66     (0, 1)
67     ax.set_xlabel('x'); ax.set_ylabel('y')
68     ax.set_title(f'Monte Carlo en el cuadrado unitario\
69     napi_est = {pi_est:.5f}')
70     ax.legend(loc='upper right', fontsize=8); ax.grid(alpha
71     =0.3)
72
73     # Panel 2: convergencia
74     ax = axes[1]
75     ax.semilogx(n_grid, pi_seq, '-', lw=1.2, label=r'$\pi_{
76     est}(N)$')

```

```

66     ax.axhline(np.pi, color='k', ls='--', lw=1, label=r'$\pi$')
67     ax.fill_between(n_grid, np.pi - 4/np.sqrt(n_grid),
68                    np.pi + 4/np.sqrt(n_grid),
69                    alpha=0.15, color='gray', label=r'$\pm 4/\sqrt{N}$')
70     ax.set_xlabel('N'); ax.set_ylabel(r'$\pi_{est}$')
71     ax.set_title('Convergencia del estimador Monte Carlo')
72     ax.grid(True, which='both', alpha=0.3); ax.legend()
73
74     plt.tight_layout()
75     plt.show()
76
    
```

Listing 1: Estimacion de π por Monte Carlo con analisis de convergencia.

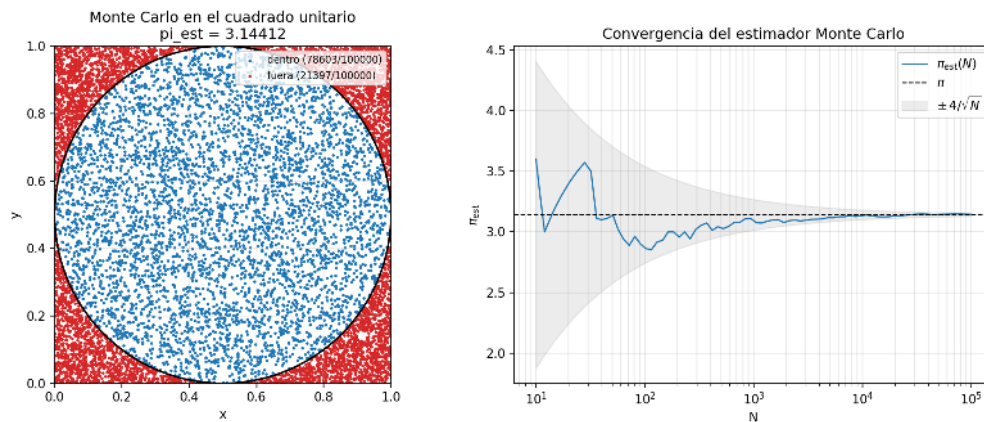


Figura 2: Estimacion de π por Monte Carlo con $N = 10^5$ muestras. Panel izquierdo: nube de puntos dentro (azul) y fuera (rojo) del circulo inscrito; el cociente $4N_{in}/N$ aproxima π . Panel derecho: convergencia del estimador con la banda teorica $\pm 4/\sqrt{N}$ que acota tipicamente la dispersion. El resultado con $N = 10^5$ es $\hat{\pi} \approx 3.144$, error relativo $\sim 10^{-3}$.

1.4.4. Lectura metodologica

Aunque elemental, este ejemplo ilustra cinco puntos transferibles al resto del curso:

1. **Semilla aleatoria.** Sin `np.random.seed` el resultado cambia cada corrida, lo cual destruye reproducibilidad. La semilla no *mejora* la estimacion; solo la hace verificable.
2. **Tasa de convergencia.** El error decae como $N^{-1/2}$. Es un techo teorico; cualquier mejora requiere otra estrategia (quasi-Monte Carlo, importance sampling, control variates).

3. **Costo computacional.** Para reducir el error relativo de 10^{-3} a 10^{-5} se necesita $10^4 \times$ mas muestras. Conviene declararlo antes de prometer precision.
4. **Verificacion vs validacion.** Se conoce π con precision arbitraria, por lo que este es un caso de *verificacion* (el codigo resuelve correctamente las matematicas), no de *validacion* (las matematicas describen correctamente la realidad). Ambas son etapas distintas y obligatorias.
5. **Generalizacion.** Sustituyendo el indicador del circulo por un integrando arbitrario $f(\mathbf{x})$ y normalizando por el volumen del dominio, se obtiene la formula general de integracion Monte Carlo en $\mathbb{R}^d$.

1.5. Diferencia entre modelo fisico, matematico y computacional

1.5.1. Modelo fisico

Un modelo fisico es una representacion conceptual del fenomeno basada en mecanismos causales. No necesariamente contiene ecuaciones completas, pero si define que mecanismos se consideran relevantes. En un problema termico, el modelo fisico puede establecer que la temperatura cambia por conduccion interna, generacion de calor y perdidas hacia el ambiente. En un problema estructural, puede establecer que el desplazamiento depende de rigidez, carga, geometria y restricciones.

Un modelo fisico responde preguntas del tipo:

- ¿Que variables interactuan?
- ¿Que mecanismos se consideran dominantes?
- ¿Que procesos se omiten y por que?
- ¿En que intervalo espacial y temporal se considera valido el analisis?

Por ejemplo, para una bomba centrifuga de sangre, el modelo fisico puede declarar que el flujo se considera incompresible, viscoso y gobernado por gradientes de presion y fuerzas inerciales. A partir de esa declaracion se justifica el uso de las ecuaciones de Navier–Stokes. La simplificacion fisica no es un defecto; es una operacion necesaria. El defecto aparece cuando la simplificacion no se declara, porque entonces el modelo se convierte en una caja negra con pretensiones de ciencia.

1.6. Modelo matematico

Un modelo matematico formaliza el modelo fisico mediante ecuaciones, dominios, restricciones, condiciones iniciales y condiciones de frontera. De forma general, un modelo gobernado por ecuaciones diferenciales parciales puede escribirse como

$$\mathcal{R}(\mathbf{u}, \frac{\partial \mathbf{u}}{\partial t}, \nabla \mathbf{u}, \nabla^2 \mathbf{u}; \boldsymbol{\theta}) = 0, \quad \mathbf{x} \in \Omega, \quad t \in (0, T], \quad (9)$$

$$\mathcal{B}(\mathbf{u}; \boldsymbol{\theta}) = \mathbf{g}, \quad \mathbf{x} \in \partial\Omega, \quad (10)$$

$$\mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0(\mathbf{x}), \quad \mathbf{x} \in \Omega. \quad (11)$$

Aqui, $\mathcal{R}$ representa el residuo del modelo, $\mathcal{B}$ representa el operador de frontera, $\partial\Omega$ es la frontera del dominio, $\mathbf{g}$ son valores prescritos o flujos impuestos, y $\mathbf{u}_0$ es la condicion inicial.

1.6.1. Formulacion fuerte vs formulacion debil

La formulacion anterior se denomina *fuerte*: exige que la ecuacion se cumpla puntualmente y que la solucion sea suficientemente diferenciable. Para problemas con co-

eficientes discontinuos, geometrias complejas, o fuentes puntuales, esta regularidad es excesiva. La formulacion *debil* relaja el requisito: se busca $\mathbf{u}$ en un espacio de Sobolev $H^1(\Omega)$ tal que, para toda funcion de prueba $\mathbf{v} \in V$ con $\mathbf{v} = \mathbf{0}$ en la frontera de Dirichlet,

$$\int_{\Omega} \mathcal{R}(\mathbf{u}) \cdot \mathbf{v} \, d\Omega = 0. \quad (12)$$

Esta formulacion es la base del Metodo de Elementos Finitos (FEM), del metodo de Galerkin y de buena parte de la teoria de redes neuronales informadas por fisica. La existencia y unicidad en formulacion debil se prueba habitualmente mediante el teorema de Lax–Milgram cuando la forma bilineal asociada es coerciva y continua.

1.6.2. Adimensionalizacion como diagnostico

Antes de discretizar, conviene escribir el modelo en forma adimensional. Esto no es esteticismo: la adimensionalizacion revela los grupos dominantes (Reynolds, Peclet, Fourier, Biot, Damkohler, etc.) y permite identificar regimenes asintoticos donde el modelo se puede simplificar (parametros pequenos $\rightarrow$ teoria de perturbaciones, parametros grandes $\rightarrow$ aproximaciones de capa limite). Numericamente, ademas, mejora el condicionamiento de las matrices porque los terminos quedan de orden $O(1)$.

1.7. Modelo computacional

Un modelo computacional es la implementacion discreta del modelo matematico. Implica elegir una malla, un paso temporal, un esquema de integracion, una estructura de datos, un algoritmo de solucion y criterios de convergencia. Por ejemplo, una ecuacion continua como

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad (13)$$

no puede ejecutarse directamente en una computadora. Debe transformarse en una expresion discreta. Si se usa un esquema explicito de diferencias finitas, se obtiene

$$T_i^{n+1} = T_i^n + r (T_{i+1}^n - 2T_i^n + T_{i-1}^n), \quad r = \frac{\alpha \Delta t}{\Delta x^2}. \quad (14)$$

El parametro r controla la estabilidad del metodo. Para el esquema explicito unidimensional, se requiere

$$r \leq \frac{1}{2}. \quad (15)$$

Este ejemplo muestra una idea clave: el modelo fisico puede ser correcto y el modelo matematico puede estar bien planteado, pero una mala discretizacion puede producir resultados numericamente falsos. La computadora no se equivoca con mala intencion; solo obedece instrucciones absurdas con una lealtad preocupante.

1.7.1. Familias de esquemas y propiedades minimas

La eleccion del esquema temporal no es cuestion de preferencia estetica. Las familias principales para un problema $\dot{\mathbf{u}} = f(\mathbf{u}, t)$ son:

- **Explicitos** (Euler explicito, Runge–Kutta 4): baratos por paso, condicionalmente estables. Aceptables para problemas no-rigidos con dinamica suave.
- **Implicitos** (Euler implicito, Crank–Nicolson, BDF): incondicionalmente estables (al menos en sentido A -estable para algunos miembros), pero requieren resolver un sistema (no lineal en general) en cada paso. Obligatorios en problemas rigidos.
- **Simplecticos** (Verlet, leap-frog): preservan estructura hamiltoniana en sistemas mecanicos conservativos. Imprescindibles si se simulan tiempos largos sin amortiguamiento.
- **Adaptativos** (Runge–Kutta–Fehlberg, Dormand–Prince): ajustan Δt a una tolerancia local. Standard de hecho en `scipy.integrate.solve_ivp`.

Tres propiedades deben verificarse en todo esquema antes de aceptar sus resultados:

1. **Consistencia.** El error local de truncamiento tiende a cero cuando $\Delta t, \Delta x \rightarrow 0$.
2. **Estabilidad.** Pequeñas perturbaciones no se amplifican sin control (análisis de von Neumann, criterio CFL).
3. **Convergencia.** Por el teorema de Lax (problemas lineales bien planteados), *consistencia + estabilidad $\Rightarrow$ convergencia*.

1.8. Abstraccion y simplificacion de fenomenos reales

La abstraccion consiste en decidir que elementos del fenomeno se preservan y cuales se eliminan. No se modela la realidad completa; se modela una estructura interpretativa de la realidad. Esta decision debe estar guiada por el objetivo de investigacion.

Sea $\mathcal{S}$ el sistema real completo. Un modelo se puede entender como una proyeccion

$$\Pi : \mathcal{S} \rightarrow \mathcal{M}, \quad (16)$$

donde Π elimina informacion considerada secundaria y preserva informacion considerada relevante. El error de modelado aparece porque Π nunca es una identidad. Formalmente, si $\mathbf{u}$ es el estado real y $\hat{\mathbf{u}}$ es el estado predicho, el error total puede expresarse como

$$\mathbf{e}(\mathbf{x}, t) = \mathbf{u}(\mathbf{x}, t) - \hat{\mathbf{u}}(\mathbf{x}, t). \quad (17)$$

El problema doctoral consiste en justificar por que ese error es aceptable para la decision cientifica o tecnologica que se desea tomar.

1.8.1. Teorema II de Buckingham y reduccion sistematica

La simplificacion no es arbitraria. El teorema II de Buckingham establece que si una relacion fisica involucra n variables dimensionales y k dimensiones fundamentales independientes (por ejemplo M, L, T, Θ), entonces puede expresarse en terminos de $n - k$ grupos adimensionales $\pi_1, \dots, \pi_{n-k}$. Este resultado convierte la pregunta “¿cuales variables importan?” en “¿cuales grupos adimensionales son grandes, pequenos o de orden uno?”. Los grupos con valores extremos sugieren simplificaciones; los grupos de orden uno representan competencia entre mecanismos y obligan a conservarlos.

1.8.2. Analisis de sensibilidad

Una vez declarado el modelo $\hat{\mathbf{u}} = \mathcal{M}(\boldsymbol{\theta}, \mathbf{c})$, conviene cuantificar cuanto influye cada parametro en la salida. Se distinguen tres niveles:

- **Sensibilidad local** (one-at-a-time): derivada parcial $\partial \hat{\mathbf{u}} / \partial \theta_i$. Util pero limitada porque ignora interacciones.
- **Sensibilidad global de varianza** (indices de Sobol): descompone la varianza total de la salida en contribuciones individuales y de interaccion,

$$S_i = \frac{\text{Var}_{\theta_i} [\mathbb{E}_{\boldsymbol{\theta}_{\sim i}} [\hat{\mathbf{u}} \mid \theta_i]]}{\text{Var}[\hat{\mathbf{u}}]}, \quad (18)$$

con $\sum_i S_i \leq 1$ (igualdad si no hay interacciones).

- **Sensibilidad por screening** (Morris): identifica con pocas evaluaciones cuales parámetros tienen efecto despreciable, para descartarlos antes de un análisis costoso.

Realizar al menos un screening preliminar es buena practica antes de invertir en una calibracion bayesiana o en un metamodelo: descartar parametros irrelevantes simplifica el modelo y mejora la identifiabilidad.

1.8.3. Ejemplo de abstraccion: cable OPGW sometido a cortocircuito

Un cable OPGW real tiene geometria helicoidal, multiples materiales, contacto entre alambres, intercambio termico con el aire, variabilidad ambiental, propiedades dependientes de la temperatura y posibles danos acumulativos. Un primer modelo termico simplificado puede reducir el problema a una barra equivalente con temperatura promedio $T(x, t)$. Si se considera conduccion axial, generacion termica por efecto Joule y conveccion con el ambiente, se obtiene

$$\rho c_p A \frac{\partial T}{\partial t} = k A \frac{\partial^2 T}{\partial x^2} + I^2 R' - h P (T - T_\infty), \quad (19)$$

donde ρ es la densidad, c_p es el calor especifico, A es el area transversal equivalente, k es la conductividad termica, I es la corriente, R' es la resistencia electrica por unidad de longitud, h es el coeficiente convectivo, P es el perimetro expuesto y T_∞ es la temperatura ambiente.

Esta ecuacion no representa todos los detalles del cable, pero puede responder una pregunta concreta: ¿cuanto aumenta la temperatura promedio bajo una corriente de cortocircuito durante un tiempo determinado? Si el objetivo fuera estudiar dano local en alambres individuales, el modelo seria insuficiente. Por lo tanto, la validez del modelo depende del objetivo.

1.8.4. Ejemplo de abstraccion: sistema RAG como prototipo computacional

En un sistema de generacion aumentada por recuperacion, el fenomeno no es fisico en sentido clasico, sino informacional. El estado de entrada se representa mediante una consulta q , una base documental $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ y una funcion de similitud $s(q, d_i)$. La recuperacion puede modelarse como

$$\mathcal{D}_k(q) = \text{TopK}_{d_i \in \mathcal{D}} s(q, d_i), \quad (20)$$

y la respuesta generada como

$$\hat{a} = G(q, \mathcal{D}_k(q); \theta), \quad (21)$$

donde G es el generador y θ son sus parametros. Aunque no hay una ecuacion diferencial, si existe un modelo computacional formal: entrada, recuperacion, representacion, generacion, metrica y validacion. Este tipo de ejemplo es util para evitar la falsa idea de que simulacion solo significa resolver calor, fluidos o estructuras.

1.9. Variables, parametros y condiciones de frontera

1.9.1. Variables de estado

Las variables de estado describen la condicion interna del sistema. Se denotan mediante u y pueden depender del espacio, el tiempo o ambos. Ejemplos:

- Temperatura: $T(\mathbf{x}, t)$.
- Desplazamiento mecanico: $\mathbf{u}(\mathbf{x}, t)$.
- Velocidad de fluido: $\mathbf{v}(\mathbf{x}, t)$.
- Concentracion quimica: $C(\mathbf{x}, t)$.
- Estado de agente: $s_i(t)$ en un modelo basado en agentes.
- Distribucion de probabilidad de texto: $p(w_t \mid w_{<t}, q, \mathcal{D}_k)$ en un modelo generativo.

Una mala seleccion de variables de estado produce modelos debiles. Por ejemplo, si se modela una bomba de sangre unicamente con caudal y presion, pero se omiten esfuerzos cortantes relacionados con dano celular, el modelo puede ser hidraulicamente aceptable y biomedicamente insuficiente. Esta distincion es crucial en investigacion doctoral, porque la variable que se mide no siempre es la variable que importa.

1.9.2. Parametros

Los parametros son cantidades que caracterizan el sistema pero no evolucionan como variables principales dentro de la simulacion, aunque pueden depender de temperatura, frecuencia, escala o condiciones de operacion. Se agrupan como

$$\boldsymbol{\theta} = \{\theta_1, \theta_2, \dots, \theta_p\}. \quad (22)$$

Ejemplos: conductividad termica k , modulo de Young E , densidad ρ , viscosidad dinamica μ , coeficiente convectivo h , paso de malla Δx , tamano de lote en entrenamiento, numero de agentes, probabilidad de transicion o umbral de decision.

Debe distinguirse entre parametros fisicos y parametros numericos. Un parametro fisico describe el sistema; un parametro numerico describe la solucion computacional. Confundirlos lleva a interpretaciones pobres. Por ejemplo, k representa conductividad termica; Δx representa resolucion espacial. Cambiar k cambia el fenomeno modelado; cambiar Δx cambia la aproximacion computacional.

Un nivel adicional util es distinguir entre *parametros nominales* (valores de catalogo o de referencia bibliografica), *parametros calibrados* (estimados a partir de datos del propio experimento) y *parametros operativos* (controles que el usuario fija). Solo los segundos son objeto legitimo de un problema inverso; los primeros tienen incertidumbre acotada por la fuente; los terceros son inputs del escenario.

1.9.3. Condiciones iniciales y de frontera

Las condiciones iniciales especifican el estado del sistema en $t = 0$:

$$\mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0(\mathbf{x}). \quad (23)$$

Las condiciones de frontera especifican como interactua el dominio con el exterior. En problemas de campo, las condiciones mas frecuentes son:

Tabla 2: Tipos comunes de condiciones de frontera.

Tipo	Forma matematica	Interpretacion
Dirichlet	$u = u_D$ en $\partial\Omega$	Se prescribe el valor de la variable. Ejemplo: temperatura fija.
Neumann	$\nabla u \cdot \mathbf{n} = q_N$ en $\partial\Omega$	Se prescribe el flujo normal. Ejemplo: flujo de calor impuesto.
Robin	$au + b\nabla u \cdot \mathbf{n} = g$	Se prescribe una combinacion de valor y flujo. Ejemplo: conveccion termica.
Periodica	$u(0, t) = u(L, t)$	El dominio se conecta consigo mismo. Ejemplo: celda repetitiva.
Simetria	$\nabla u \cdot \mathbf{n} = 0$	Caso particular de Neumann homogenea para explotar simetria geometrica.
Absorbente (ABC, PML)	operador que evita reflexion	Frontera artificial truncada en problemas hiperbolicos o de propagacion no acotados.

Para un problema termico con conveccion, la condicion de Robin puede expresarse como

$$-k\nabla T \cdot \mathbf{n} = h(T - T_\infty), \quad \mathbf{x} \in \partial\Omega. \quad (24)$$

Esta ecuacion indica que el flujo termico saliente es proporcional a la diferencia entre la temperatura superficial y la temperatura ambiente. No es un adorno matematico: define como el modelo se comunica con el exterior.

Una observacion practica: la mala especificacion de las condiciones de frontera es probablemente la primera causa de simulaciones que “casi funcionan”. Pueden producir resultados visualmente plausibles pero cuantitativamente incorrectos. Conviene siempre verificar el flujo neto en $\partial\Omega$ y compararlo con un balance global; si no cierra, las condiciones de frontera estan mal definidas o mal implementadas.

1.10. Escalas espaciales, temporales y numeros adimensionales

Un modelo científico debe declarar su escala de validez. Una ecuación que describa un fenómeno a escala macroscópica puede fallar a escala microscópica. Un modelo estacionario puede ser razonable en procesos lentos y absurdo en procesos transitorios rápidos. Por ello se emplean escalas características.

Sea L_c una longitud característica, T_c un tiempo característico, U_c una velocidad característica y Θ_c una escala de temperatura. Se definen variables adimensionales como

$$\mathbf{x}^* = \frac{\mathbf{x}}{L_c}, \quad t^* = \frac{t}{T_c}, \quad \mathbf{v}^* = \frac{\mathbf{v}}{U_c}, \quad T^* = \frac{T - T_\infty}{\Theta_c}. \quad (25)$$

La adimensionalización permite identificar grupos dominantes. Por ejemplo, en transferencia de calor transitoria aparece el numero de Fourier:

$$Fo = \frac{\alpha t}{L_c^2}, \quad (26)$$

donde $\alpha = k/(\rho c_p)$ es la difusividad térmica. Si $Fo \ll 1$, el proceso térmico apenas ha penetrado el dominio; si $Fo \gg 1$, el sistema puede aproximarse al régimen cercano al equilibrio térmico.

En dinámica de fluidos aparece el numero de Reynolds:

$$Re = \frac{\rho U_c L_c}{\mu}, \quad (27)$$

donde ρ es la densidad, μ la viscosidad dinámica y U_c una velocidad característica. Este número relaciona fuerzas inerciales con fuerzas viscosas. En una bomba centrífuga, Re ayuda a anticipar si el flujo será laminar, transicional o turbulento, aunque por sí solo no resuelve el problema geométrico ni la interacción rotor–fluido.

En transferencia de calor con convección aparece el numero de Biot:

$$Bi = \frac{h L_c}{k}. \quad (28)$$

Si $Bi \ll 0.1$, la temperatura interna tiende a ser relativamente uniforme y puede usarse un modelo lumped; si Bi no es pequeño, se requiere resolver gradientes internos. Este criterio evita usar modelos de parámetros concentrados cuando el fenómeno exige un modelo distribuido.

1.10.1. Otros grupos adimensionales relevantes

La siguiente tabla resume otros números adimensionales que aparecen con frecuencia en ingeniería y que conviene tener presentes al formular un modelo:

Tabla 3: Numeros adimensionales relevantes en ingenieria.

Numero	Definicion	Interpretacion
Prandtl	$Pr = \mu c_p / k$	Razon entre difusividad de momento y difusividad termica. Es propiedad del fluido.
Peclet	$Pe = Re Pr = U_c L_c / \alpha$	Adveccion frente a difusion termica. $Pe \gg 1$: transporte por adveccion domina.
Nusselt	$Nu = h L_c / k_f$	Conveccion frente a conduccion en el fluido. Salida tipica de correlaciones.
Mach	$M = U / c$	Velocidad de flujo frente a velocidad del sonido. $M < 0.3$ permite tratar el flujo como incompresible.
Strouhal	$St = f L_c / U_c$	Frecuencia característica de desprendimiento de vortices o forzamiento.
Damkohler	$Da = \tau_{\text{flujo}} / \tau_{\text{reacc}}$	Tiempo de residencia frente a tiempo de reaccion en flujos reactivos.
Knudsen	$Kn = \lambda_{mfp} / L_c$	Camino libre medio frente a escala macroscopica. $Kn > 0.01$ invalida la hipotesis continua.
Weber	$We = \rho U_c^2 L_c / \sigma$	Inercia frente a tension superficial. Relevante en interfases.

Una recomendacion practica: antes de simular, estimar los grupos relevantes con valores tipicos del problema. Si un numero esta lejos de la unidad por dos o tres ordenes de magnitud, hay una simplificacion natural disponible; si esta cerca de la unidad, hay competencia entre mecanismos y el modelo *tiene* que conservar ambos terminos.

1.10.2. Escala numerica y condicion CFL

En simulacion transitoria, la relacion entre paso espacial y paso temporal controla estabilidad y precision. Para muchos problemas hiperbolicos y parabolicos aparece una condicion tipo Courant–Friedrichs–Lewy (CFL). Para una ecuacion de transporte unidimensional,

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0, \quad (29)$$

un esquema explicito requiere tipicamente

$$C = \frac{c \Delta t}{\Delta x} \leq C_{\text{máx}}. \quad (30)$$

El numero C indica cuantas celdas avanza la informacion fisica durante un paso temporal. Si el algoritmo permite que la informacion salte mas alla de lo que la malla puede representar, la simulacion se descompone. La inestabilidad numerica no es una propiedad del fenomeno; es una propiedad del metodo usado para aproximarlos.

Para la ecuación de difusión en 2D, la condición equivalente es $r_x + r_y \leq 1/2$, con $r_i = \alpha \Delta t / \Delta x_i^2$. Este criterio se invoca en el Ejemplo 7 del intercambiador de calor y debe verificarse explícitamente en el código.

1.11. Error, precision e incertidumbre

1.11.1. Error total

En simulacion cientifica, el error total no proviene de una sola fuente. Puede descomponerse conceptualmente como

$$e_{\text{total}} = e_{\text{modelo}} + e_{\text{parametros}} + e_{\text{discretizacion}} + e_{\text{iterativo}} + e_{\text{redondeo}} + e_{\text{medicion}}. \quad (31)$$

Cada termino tiene naturaleza diferente:

- e_{modelo} : proviene de hipotesis fisicas incompletas o incorrectas.
- $e_{\text{parametros}}$: proviene de valores inciertos de propiedades o constantes.
- $e_{\text{discretizacion}}$: proviene de representar un dominio continuo mediante una malla finita.
- $e_{\text{iterativo}}$: proviene de detener un algoritmo antes de convergencia exacta.
- e_{redondeo} : proviene de precision finita de la computadora.
- e_{medicion} : proviene de ruido, sesgo o limitaciones instrumentales.

La precision numerica no garantiza exactitud fisica. Una simulacion puede tener muchas cifras decimales y ser fisicamente inutil. Ese vicio aparece con frecuencia cuando se confunde estetica computacional con evidencia cientifica.

1.11.2. Orden de convergencia y extrapolacion de Richardson

Para un esquema consistente de orden p , el error de discretizacion sobre una variable de interes ϕ se comporta como

$$\phi_h = \phi_{\text{exacto}} + C h^p + O(h^{p+1}), \quad (32)$$

donde h es un parametro caracteristico de la malla (tamano de elemento, paso temporal). Realizando tres corridas con tamanos $h_1 < h_2 < h_3$ y factor de refinamiento $r = h_2/h_1 = h_3/h_2$, el orden observado se estima como

$$p_{\text{obs}} = \frac{\ln [(\phi_3 - \phi_2)/(\phi_2 - \phi_1)]}{\ln r}, \quad (33)$$

y la extrapolacion de Richardson permite estimar el valor refinado,

$$\phi_{\text{exacto}} \approx \phi_1 + \frac{\phi_1 - \phi_2}{r^{p_{\text{obs}}} - 1}. \quad (34)$$

El *Grid Convergence Index* (GCI), propuesto por Roache y adoptado en ASME V&V 20, proporciona una banda de incertidumbre asociada al error de discretización:

$$GCI = F_s \frac{|\phi_1 - \phi_2|}{(r^{p_{\text{obs}}} - 1) |\phi_1|}, \quad (35)$$

con $F_s \approx 1.25$ (factor de seguridad). Reportar GCI junto al resultado convierte una simulación en un dato cuantificable; sin el, es solo un gráfico bonito.

1.11.3. Verificación y validación (V&V)

La comunidad de simulación distingue formalmente dos procesos:

- **Verificación.** ¿Resuelve el código correctamente las ecuaciones que se le pidió resolver? Se compara contra soluciones manufacturadas, soluciones analíticas, o convergencia de malla. Es un problema matemático.
- **Validación.** ¿Resuelve el código las ecuaciones correctas para el fenómeno real? Se compara contra mediciones experimentales con incertidumbre conocida. Es un problema físico.

Una simulación verificada pero no validada puede ser internamente consistente y físicamente incorrecta. Una simulación validada pero no verificada puede acertar por compensación de errores. Ambas etapas son obligatorias antes de usar un modelo para decisiones de ingeniería. El estándar ASME V&V 10/20 y la guía NASA-STD-7009 formalizan este flujo.

1.11.4. Métricas de comparación

Si y_i son datos experimentales y $\hat{y}_i$ son predicciones del modelo, se definen métricas como el error absoluto medio:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, \quad (36)$$

el error cuadrático medio:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (37)$$

y la raíz del error cuadrático medio:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}. \quad (38)$$

Tambien puede emplearse una norma relativa:

$$e_{rel} = \frac{\|\mathbf{y} - \hat{\mathbf{y}}\|_2}{\|\mathbf{y}\|_2}. \quad (39)$$

Estas metricas no son equivalentes. El RMSE penaliza errores grandes; el MAE es mas robusto ante valores extremos. En validacion doctoral, debe justificarse la metrica elegida de acuerdo con el dano, costo o consecuencia de los errores.

Para datos espaciales o temporales correlacionados conviene complementar con el coeficiente de determinacion R^2 , el indice de Willmott d o, en imagenes, SSIM. Para distribuciones de probabilidad: divergencia de Kullback–Leibler o distancia de Wasserstein. La eleccion de la metrica condiciona el resultado: una norma cuadratica favorece ajustes globales; una norma infinito penaliza fallas locales severas.

1.11.5. Propagacion de incertidumbre

Si una magnitud de salida z depende de variables de entrada $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, es decir,

$$z = f(\mathbf{x}), \quad (40)$$

y las entradas tienen matriz de covarianza Σ_x , la aproximacion lineal de primer orden permite estimar

$$\sigma_z^2 \approx \mathbf{J} \Sigma_x \mathbf{J}^T, \quad (41)$$

donde

$$\mathbf{J} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right] \quad (42)$$

es el Jacobiano de sensibilidad. Para una salida vectorial $\mathbf{z} = \mathbf{f}(\mathbf{x})$, se obtiene

$$\Sigma_z \approx \mathbf{J} \Sigma_x \mathbf{J}^T. \quad (43)$$

Esta expresion es central para conectar metrologia, simulacion y validacion experimental. Permite cuantificar como errores de entrada se transforman en incertidumbre de salida.

La linealizacion es valida solo si f es localmente diferenciable y las incertidumbres son pequenas. Cuando hay no-linealidad fuerte, distribuciones no gaussianas o discontinuidades, se debe recurrir a propagacion de Monte Carlo (analogia al ejemplo de la Seccion 2): se generan muestras de $\mathbf{x}$ segun su distribucion conjunta, se evalua f para cada muestra, y se construyen estadisticas empiricas de $\mathbf{z}$. Esta tecnica enlaza directamente el primer ejemplo del documento con la cuantificacion de incertidumbre en problemas reales.

Las principales fuentes de error presentes en simulacion computacional se resumen en la [Figura 3](#).

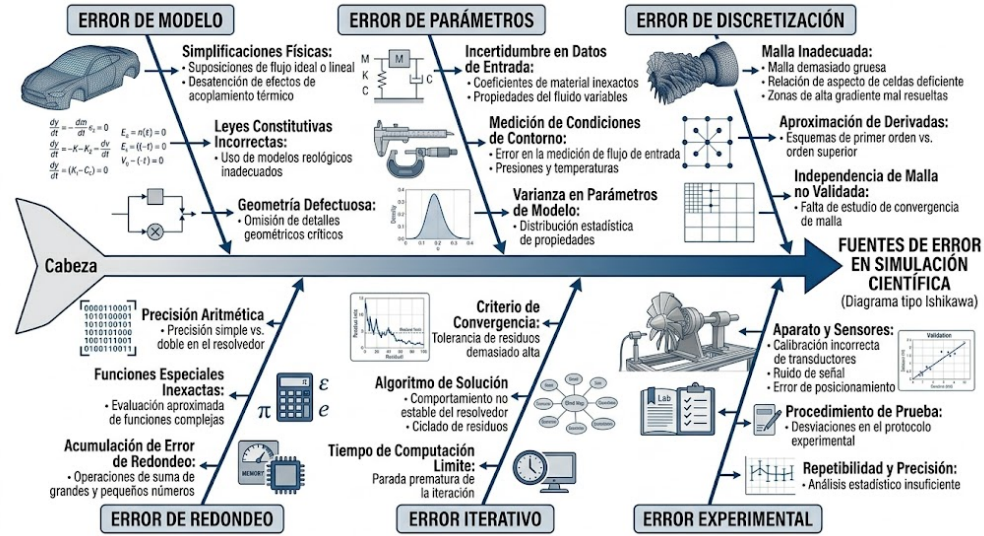


Figura 3: Principales fuentes de error en simulación científica y modelado computacional.

1.12. Reproducibilidad científica

Un resultado reproducible es aquel que puede obtenerse nuevamente bajo condiciones claramente documentadas. En simulación y prototipado, la reproducibilidad requiere más que compartir una gráfica. Deben reportarse:

- versión del código;
- lenguaje y bibliotecas empleadas;
- unidades de todas las variables;
- parámetros físicos y numéricos;
- condiciones iniciales y de frontera;
- criterios de convergencia;
- estructura de datos;
- semilla aleatoria, si aplica;
- archivos de entrada;
- instrucciones mínimas para ejecutar.

Un resultado sin unidades, sin parámetros, sin malla y sin código verificable se parece más a una opinión con gráfica que a una simulación científica. Para evitarlo, se recomienda que cada práctica del curso produzca al menos cuatro elementos: archivo de código, figura exportada, tabla de parámetros y breve interpretación técnica.

Las prácticas FAIR (Findable, Accessible, Interoperable, Reusable) para datos científicos se trasladan naturalmente al código: repositorio público con DOI (Zenodo, Figshare), licencia explícita, archivo `environment.yml` o `requirements.txt` con versiones fijas, contenedores reproducibles (Docker, Apptainer) cuando hay dependencias complejas, y notebooks ejecutables (Google Colab, Binder) para la difusión docente.

1.12.1. Registro mínimo de una simulación

Toda simulación entregada durante el curso deberá incluir una tabla como la siguiente:

Tabla 4: Registro minimo para reproducibilidad de una simulacion.

Elemento	Descripcion requerida
Fenomeno modelado	Descripcion fisica, biologica, social o computacional del sistema.
Objetivo de simulacion	Pregunta especifica que responde la simulacion.
Variables de estado	Variables dependientes y unidades.
Parametros	Valores, unidades y fuente o justificacion.
Hipotesis	Simplificaciones explicitas y limitaciones.
Dominio	Geometria, intervalo temporal y escala.
Condiciones iniciales/-frontera	Valores prescritos, flujos o restricciones.
Metodo numerico	Diferencias finitas, FEM, Monte Carlo, agentes, redes neuronales, etc.
Resolucion	Tamano de malla, paso temporal o numero de muestras.
Criterio de validacion	Metrica, dato de referencia o prueba de consistencia.
Codigo	Lenguaje, bibliotecas y semilla aleatoria.
Resultado	Graficas, tablas y conclusion tecnica.

La validacion cuantitativa entre simulacion y experimento se representa conceptualmente en la **Figura 4**.

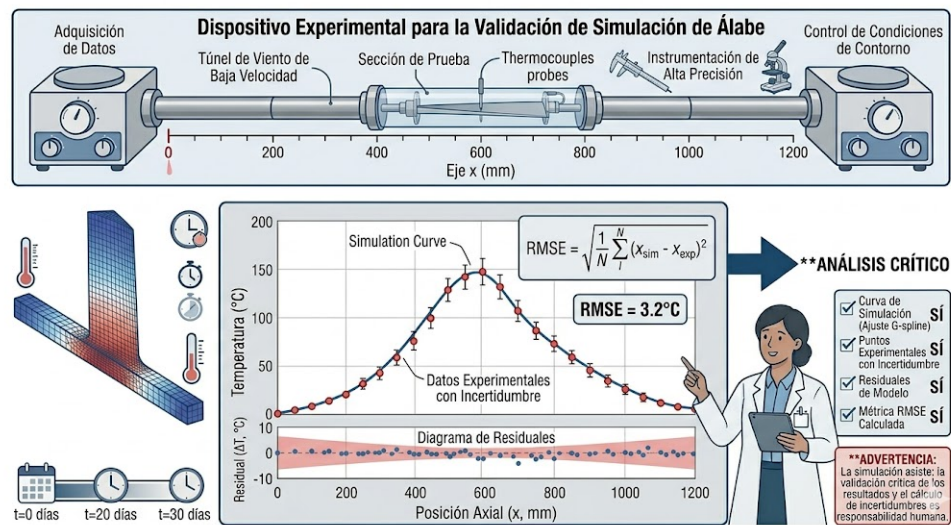


Figura 4: Comparacion conceptual entre resultados experimentales y simulacion computacional con analisis de residuos.

1.13. Ejemplo 1: conduccion transitoria en una barra

1.13.1. Planteamiento fisico

Se analiza una barra de longitud L sometida a una distribucion inicial de temperatura. Los extremos se mantienen a temperatura ambiente T_0 . Se desea simular la evolucion temporal de la temperatura y observar la disipacion del gradiente termico.

1.13.2. Modelo matematico

Se emplea la ecuacion de calor unidimensional:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \quad 0 < x < L, \quad t > 0. \quad (44)$$

Las condiciones de frontera son de Dirichlet:

$$T(0, t) = T_0, \quad T(L, t) = T_0. \quad (45)$$

La condicion inicial se define como una perturbacion gaussiana:

$$T(x, 0) = T_0 + A \exp \left[-\frac{(x - x_c)^2}{2\sigma^2} \right], \quad (46)$$

donde A es la amplitud termica, x_c es el centro de la perturbacion y σ controla su ancho.

1.13.3. Modelo computacional

Se discretiza el dominio con N nodos:

$$x_i = i\Delta x, \quad \Delta x = \frac{L}{N-1}, \quad i = 0, 1, \dots, N-1. \quad (47)$$

El tiempo se discretiza como

$$t^n = n\Delta t. \quad (48)$$

El esquema explicito queda:

$$T_i^{n+1} = T_i^n + r (T_{i+1}^n - 2T_i^n + T_{i-1}^n), \quad r = \frac{\alpha \Delta t}{\Delta x^2}. \quad (49)$$

Para estabilidad se impone $r \leq 0.5$.

1.13.4. Codigo base en Python

```
import numpy as np
```

```
2      import matplotlib.pyplot as plt
3
4      # Parametros fisicos
5      L = 1.0                # longitud [m]
6      alpha = 1.0e-4         # difusividad termica [m^2/s]
7      T0 = 25.0              # temperatura ambiente [degC]
8      A = 80.0               # amplitud de perturbacion [degC]
9      xc = 0.5               # centro de perturbacion [m]
10     sigma = 0.06           # ancho de perturbacion [m]
11
12     # Parametros numericos
13     N = 151
14     x = np.linspace(0, L, N)
15     dx = x[1] - x[0]
16     r = 0.45
17     dt = r * dx**2 / alpha
18     nt = 800
19
20     # Condicion inicial
21     T = T0 + A*np.exp(-((x-xc)**2)/(2*sigma**2))
22     T[0] = T0
23     T[-1] = T0
24
25     snapshots = []
26     times = []
27     for n in range(nt):
28         Tnew = T.copy()
29         Tnew[1:-1] = T[1:-1] + r*(T[2:] - 2*T[1:-1] + T[:-2])
30         Tnew[0] = T0
31         Tnew[-1] = T0
32         T = Tnew
33         if n in [0, 50, 150, 300, 799]:
34             snapshots.append(T.copy())
35             times.append(n*dt)
36
37     plt.figure(figsize=(7, 4))
38     for Tplot, t in zip(snapshots, times):
39         plt.plot(x, Tplot, label=f't = {t:.2f} s')
40         plt.xlabel('x [m]')
41         plt.ylabel('Temperatura [degC]')
42         plt.title('Conduccion transitoria 1D')
43         plt.grid(True)
```

```

44 plt.legend()
45 plt.tight_layout()
46 plt.show()
47

```

Listing 2: Simulacion explicita de conduccion transitoria 1D.

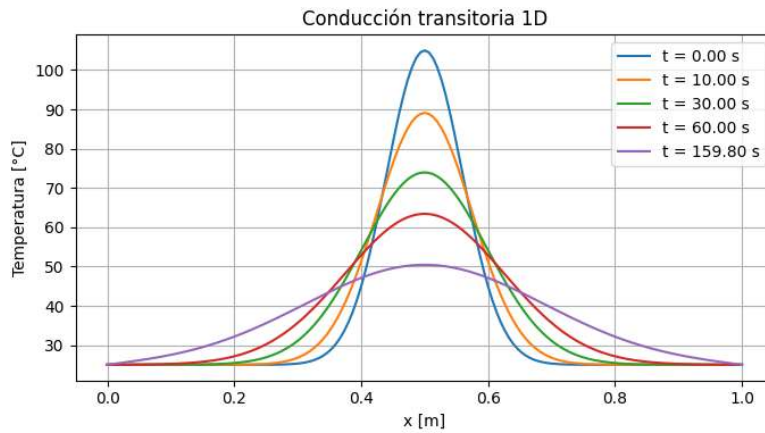


Figura 5: Resultado de la Simulacion explicita de conduccion transitoria 1D.

La evolucion temporal de la distribucion termica obtenida mediante diferencias finitas se ilustra conceptualmente en la [Figura 6](#).

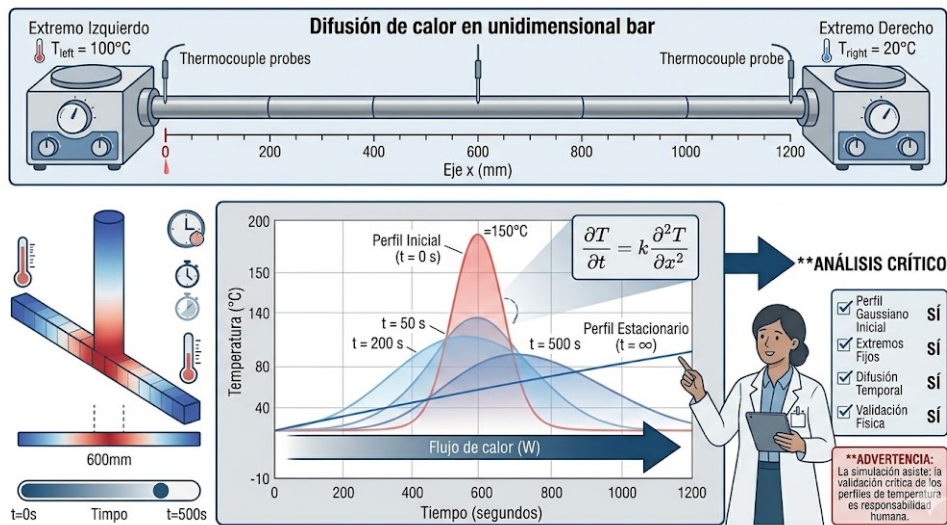


Figura 6: Difusion temporal de un perfil termico en una barra unidimensional.

1.13.5. Interpretacion esperada

La perturbacion termica se suaviza con el tiempo. La temperatura maxima disminuye y el perfil se ensancha debido a la difusion. Si se aumenta α , la disipacion ocurre mas rapido. Si se viola $r \leq 0.5$, el perfil puede oscilar o divergir, lo cual evidencia inestabilidad numerica. El fenomeno fisico no exploto; exploto el metodo.

1.14. Ejemplo 2: sistema masa–resorte–amortiguador

1.14.1. Planteamiento fisico

Se considera un sistema de un grado de libertad compuesto por una masa m , un resorte de rigidez k y un amortiguador viscoso con coeficiente c . El sistema se somete a una fuerza externa $F(t)$. Este ejemplo permite conectar modelo fisico, ecuacion diferencial ordinaria, integracion numerica y analisis parametrico.

1.14.2. Modelo matematico

La segunda ley de Newton conduce a

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = F(t), \quad (50)$$

donde $x(t)$ es el desplazamiento. Se definen la frecuencia natural no amortiguada y la razon de amortiguamiento:

$$\omega_n = \sqrt{\frac{k}{m}}, \quad \zeta = \frac{c}{2\sqrt{km}}. \quad (51)$$

Si $F(t) = 0$, el comportamiento libre depende de ζ . Para $0 < \zeta < 1$, el sistema es subamortiguado y presenta oscilacion decreciente.

Para resolver numericamente, se convierte la ecuacion de segundo orden en un sistema de primer orden:

$$\begin{cases} \dot{x} = v, \\ \dot{v} = \frac{1}{m} [F(t) - cv - kx]. \end{cases} \quad (52)$$

1.14.3. Discretizacion mediante Euler explicito

Un esquema elemental se escribe como

$$x^{n+1} = x^n + \Delta t v^n, \quad (53)$$

$$v^{n+1} = v^n + \Delta t \frac{F^n - cv^n - kx^n}{m}. \quad (54)$$

Este metodo es simple, pero puede ser inestable si Δt es demasiado grande. La eleccion de metodo numerico debe formar parte del razonamiento cientifico, no de la decoracion del codigo.

1.14.4. Actividad breve de analisis

Se pide comparar tres valores de amortiguamiento: $\zeta = 0.05$, $\zeta = 0.2$ y $\zeta = 0.8$. Para cada caso se debe graficar $x(t)$ y discutir como cambia la disipacion de energia. La energia mecanica total se calcula como

$$E(t) = \frac{1}{2}mv(t)^2 + \frac{1}{2}kx(t)^2. \quad (55)$$

En un sistema amortiguado, $E(t)$ debe decrecer. Si la energia crece sin fuerza externa, el algoritmo esta introduciendo energia numerica artificial.

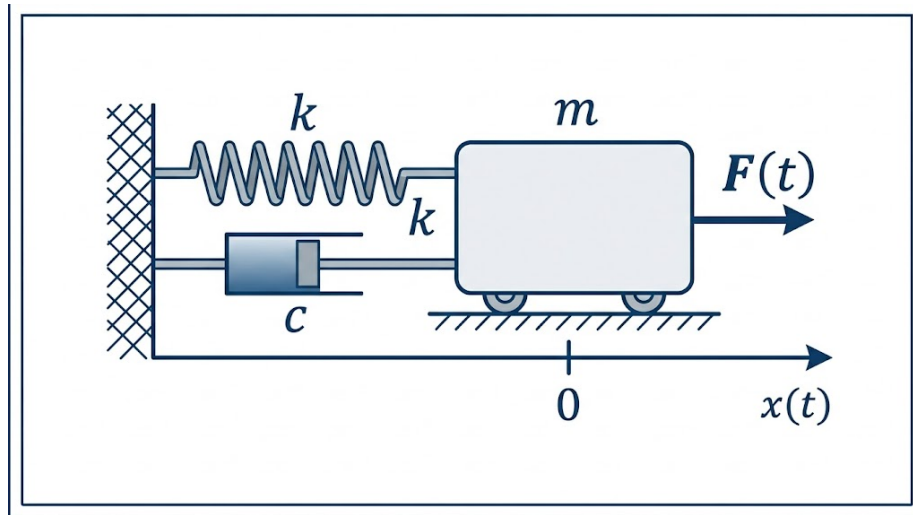


Figura 7: Representacion esquematica de un sistema masa–resorte–amortiguador horizontal.

1.15. Ejemplo 3: modelo termico simplificado para cable OPGW

1.15.1. Motivacion

Los articulos sobre cables OPGW permiten discutir una transicion natural entre investigacion experimental y simulacion numerica. El cable real presenta una arquitectura compleja, pero en una primera aproximacion puede formularse un modelo termico de parametros concentrados si se supone temperatura uniforme en la seccion transversal.

1.15.2. Modelo concentrado

Si se ignoran gradientes espaciales internos, el balance energetico se expresa como

$$mc_p \frac{dT}{dt} = I^2 R - hA_s(T - T_\infty) - \varepsilon \sigma_{SB} A_s(T^4 - T_\infty^4), \quad (56)$$

donde m es la masa del tramo considerado, c_p el calor especifico, I la corriente, R la resistencia electrica, h el coeficiente convectivo, A_s el area superficial, ε la emisividad y σ_{SB} la constante de Stefan–Boltzmann.

Si la radiacion se desprecia para temperaturas moderadas, se obtiene

$$mc_p \frac{dT}{dt} = I^2 R - hA_s(T - T_\infty). \quad (57)$$

Esta ecuacion lineal tiene solucion analitica:

$$T(t) = T_\infty + \frac{I^2 R}{hA_s} + \left[T(0) - T_\infty - \frac{I^2 R}{hA_s} \right] \exp \left(-\frac{hA_s}{mc_p} t \right). \quad (58)$$

1.15.3. Criterio de validez

El modelo concentrado exige que los gradientes internos sean pequenos. Un criterio clasico es

$$Bi = \frac{hL_c}{k} \ll 0.1. \quad (59)$$

Si Bi no es pequeno, debe utilizarse un modelo distribuido, por ejemplo mediante elementos finitos o diferencias finitas. Esta decision ilustra el papel del modelado: no se elige el metodo mas sofisticado por vanidad computacional, sino el que responde la pregunta con el nivel de fidelidad requerido.

1.15.4. Prompt para que el estudiante genere código con IA

Prompt recomendado. Genera un script en Python para simular el calentamiento transitorio de un cable OPGW mediante un modelo termico concentrado. Usa el balance $mc_p dT/dt = I^2 R - hA_s(T - T_\infty)$. Define parametros con unidades, integra la ecuacion con un metodo numerico explicito, grafica $T(t)$ para tres valores de corriente I , calcula la temperatura maxima y verifica si se supera un umbral critico definido por el usuario. Incluye comentarios tecnicos en espanol y una tabla impresa con los parametros usados.

1.16. Ejemplo 4: bomba centrífuga y dinamica de fluidos computacional

1.16.1. Modelo fisico

Una bomba centrífuga transforma energia mecanica rotacional en energia hidraulica. En aplicaciones biomedicas, ademas del caudal y la presion, se debe considerar el dano potencial a la sangre debido a esfuerzos cortantes. Este ejemplo permite introducir modelos de campo vectorial y criterios de desempeno.

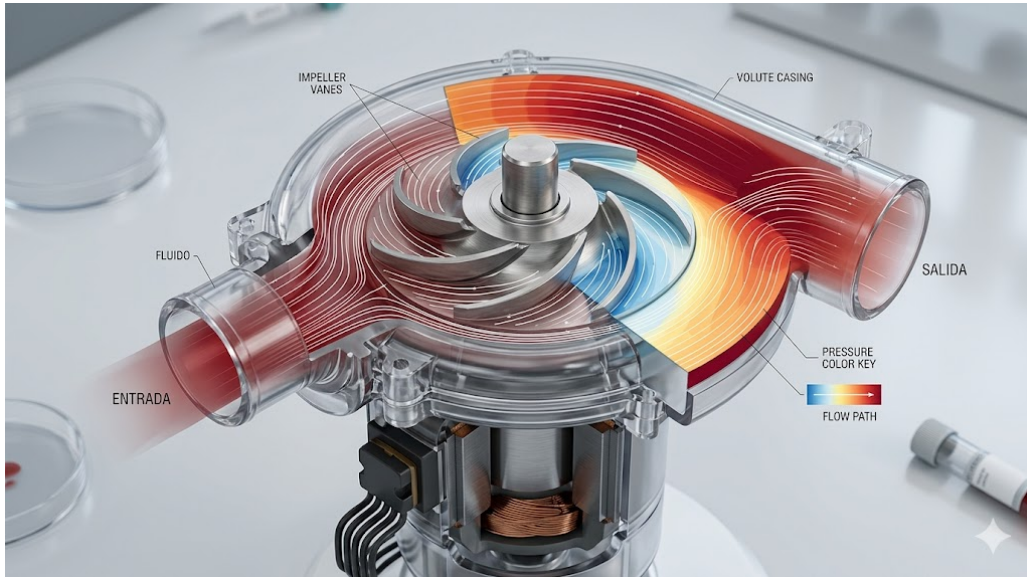


Figura 8: Representación conceptual de una bomba centrífuga biomedica analizada mediante dinamica de fluidos computacional.

1.16.2. Ecuaciones gobernantes

Para un fluido newtoniano, incompresible e isotermico, se emplean las ecuaciones de Navier–Stokes:

$$\nabla \cdot \mathbf{v} = 0, \quad (60)$$

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} \right) = -\nabla p + \mu \nabla^2 \mathbf{v} + \mathbf{f}, \quad (61)$$

donde $\mathbf{v}$ es la velocidad, p la presión, ρ la densidad, μ la viscosidad dinamica y $\mathbf{f}$ una fuerza volumetrica.

1.16.3. Indicadores de desempeño

La carga hidráulica $H = \Delta p / (\rho g)$, la eficiencia $\eta = \rho g Q H / P_{in}$ y un indicador de exposición a esfuerzo cortante $D = \int_0^{t_e} \tau(t)^a dt$ permiten comparar diseños antes de prototipar.

1.16.4. Prototipado

geometria CAD $\rightarrow$ malla $\rightarrow$ condiciones de frontera $\rightarrow$ solución CFD $\rightarrow$ curvas $Q-H$ $\rightarrow$ validación.
(62)

1.17. Ejemplo 5: modelo basado en agentes para economía circular

1.17.1. Planteamiento

No todos los sistemas de interés doctoral son continuos ni gobernados por ecuaciones diferenciales. En economía circular, adopción tecnológica, logística, movilidad o comportamiento organizacional, puede ser más adecuado representar agentes con reglas de decisión.

Sea un conjunto de agentes $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$. Cada agente tiene un estado $s_i(t)$, un conjunto de atributos θ_i y una regla de decisión

$$s_i(t+1) = \Phi_i(s_i(t), \mathcal{N}_i(t), \theta_i, \xi_i(t)), \quad (63)$$

donde $\mathcal{N}_i(t)$ representa vecinos o interacciones y $\xi_i(t)$ representa aleatoriedad.

1.17.2. Ejemplo: adopción de una tecnología de reciclaje

Cada agente puede estar en uno de dos estados: no adopta (0) o adopta (1) una tecnología. La probabilidad de adopción puede definirse como

$$P[s_i(t+1) = 1] = \sigma(\beta_0 + \beta_1 B_i - \beta_2 C_i + \beta_3 A_{\mathcal{N}_i}), \quad (64)$$

con $\sigma(z) = 1/(1 + e^{-z})$.

1.17.3. Código conceptual mínimo

```

1      import numpy as np
2      import matplotlib.pyplot as plt
3
4      np.random.seed(7)
5      N = 200
6      steps = 80
7
8      benefit = np.random.normal(1.0, 0.2, N)
9      cost = np.random.normal(1.0, 0.25, N)
10     state = np.zeros(N, dtype=int)
11     state[np.random.choice(N, 5, replace=False)] = 1
12
13     beta0, beta1, beta2, beta3 = -1.2, 1.5, 1.2, 2.0
14
15     adoption_fraction = []
16     for t in range(steps):
17         adoption_fraction.append(state.mean())
    
```



```
18     new_state = state.copy()
19     global_adoption = state.mean()
20     for i in range(N):
21         z = beta0 + beta1*benefit[i] - beta2*cost[i] + beta3*
global_adoption
22         p = 1/(1 + np.exp(-z))
23         if np.random.rand() < p:
24             new_state[i] = 1
25             state = new_state
26
27     plt.figure(figsize=(7,4))
28     plt.plot(adoption_fraction)
29     plt.xlabel('Paso temporal'); plt.ylabel('Fraccion de
adopcion')
30     plt.title('Modelo basado en agentes: adopcion
tecnologica')
31     plt.grid(True); plt.tight_layout(); plt.show()
32
```

Listing 3: Modelo simplificado de adopcion tecnologica mediante agentes.

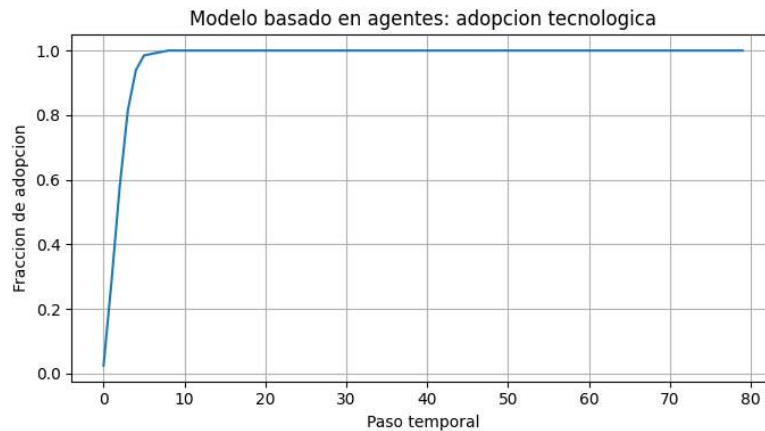


Figura 9: Resultado Modelo simplificado de adopcion tecnologica mediante agentes.

1.18. Ejemplo 6: red neuronal informada por fisica

1.18.1. Idea central

Una red neuronal informada por fisica (PINN) no solo ajusta datos; minimiza simultaneamente error de datos y violacion de una ecuacion gobernante. Para una ecuacion diferencial parcial general,

$$\mathcal{R} \left(u, \frac{\partial u}{\partial t}, \nabla u, \nabla^2 u; \boldsymbol{\theta} \right) = 0, \quad (65)$$

se propone una red neuronal $u_\theta(\mathbf{x}, t)$ y se construye una funcion de perdida:

$$\mathcal{L} = \lambda_d \mathcal{L}_{data} + \lambda_r \mathcal{L}_{res} + \lambda_b \mathcal{L}_{bc} + \lambda_i \mathcal{L}_{ic}, \quad (66)$$

donde

$$\mathcal{L}_{data} = \frac{1}{N_d} \sum_{j=1}^{N_d} |u_\theta(\mathbf{x}_j, t_j) - u_j^{obs}|^2, \quad (67)$$

$$\mathcal{L}_{res} = \frac{1}{N_r} \sum_{j=1}^{N_r} |\mathcal{R}(u_\theta(\mathbf{x}_j, t_j))|^2, \quad (68)$$

$$\mathcal{L}_{bc} = \frac{1}{N_b} \sum_{j=1}^{N_b} |\mathcal{B}(u_\theta(\mathbf{x}_j, t_j)) - g_j|^2, \quad (69)$$

$$\mathcal{L}_{ic} = \frac{1}{N_i} \sum_{j=1}^{N_i} |u_\theta(\mathbf{x}_j, 0) - u_0(\mathbf{x}_j)|^2. \quad (70)$$

Este enfoque exige definir fenomeno, ecuacion, datos, dominio, condiciones y validacion. Una PINN mal formulada es solo una red neuronal cara intentando adivinar fisica con cara de paper.

1.18.2. Ejemplo con ecuacion de calor

Para la ecuacion

$$\frac{\partial u}{\partial t} - \alpha \frac{\partial^2 u}{\partial x^2} = 0, \quad (71)$$

el residuo fisico es $r_\theta(x, t) = \frac{\partial u_\theta}{\partial t} - \alpha \frac{\partial^2 u_\theta}{\partial x^2}$. La calidad del modelo se evalua comparando u_θ con una solucion analitica, datos experimentales o una simulacion numerica de referencia.

1.19. Ejemplo 7: conduccion 2D en placa de intercambiador de calor

1.19.1. Motivacion fisica

Los intercambiadores de calor de placas son uno de los componentes mas comunes en ingenieria termica. Cada placa metalica delgada se encuentra en contacto con dos fluidos a temperaturas diferentes y conduce calor lateralmente a lo largo de su area. El analisis transitorio de una placa permite responder preguntas como: ¿cuanto tarda en alcanzarse el regimen permanente?, ¿donde aparecen los maximos gradientes (zonas criticas de fatiga termica)?, ¿como cambia la distribucion si se modifican los caminos de fluido?

Este ejemplo amplia el caso 1D del Ejemplo 1 a una placa rectangular 2D y muestra de manera explicita la transicion entre el estado inicial uniforme y el estado estable gobernado por la ecuacion de Laplace.

1.19.2. Modelo matematico

Sea $\Omega = [0, L_x] \times [0, L_y]$ el dominio rectangular de la placa, y $T(x, y, t)$ la temperatura. Sin generacion volumetrica de calor, la ecuacion gobernante es la ecuacion de calor 2D:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right), \quad \alpha = \frac{k}{\rho c_p}. \quad (72)$$

Condiciones de frontera de tipo Dirichlet, que representan paredes en contacto termico con los fluidos:

$$T(0, y, t) = T_h, \quad T(L_x, y, t) = T_c, \quad T(x, 0, t) = T_c, \quad T(x, L_y, t) = T_h. \quad (73)$$

Condicion inicial uniforme:

$$T(x, y, 0) = T_0. \quad (74)$$

Cuando $t \rightarrow \infty$ la derivada temporal se anula y el campo $T_\infty(x, y)$ resuelve la ecuacion de Laplace $\nabla^2 T = 0$ con las condiciones de frontera anteriores; este es el estado estable que se busca alcanzar.

1.19.3. Discretizacion explicita FTCS 2D

Sobre una malla estructurada de $N_x \times N_y$ nodos con pasos $\Delta x, \Delta y$, el esquema de diferencias finitas centrado en espacio y explicito en tiempo (FTCS) es

$$T_{i,j}^{n+1} = T_{i,j}^n + r_x (T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n) + r_y (T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n), \quad (75)$$

con $r_x = \alpha \Delta t / \Delta x^2$, $r_y = \alpha \Delta t / \Delta y^2$. El análisis de estabilidad de von Neumann impone

$$r_x + r_y \leq \frac{1}{2}. \quad (76)$$

Para $\Delta x = \Delta y$, esto equivale a $\alpha \Delta t / \Delta x^2 \leq 1/4$, mas restrictivo que en 1D.

1.19.4. Criterio de estado estable

Numericamente, se considera alcanzado el regimen permanente cuando

$$\max_{i,j} |T_{i,j}^{n+1} - T_{i,j}^n| < \text{tol}, \quad (77)$$

con tol típicamente entre 10^{-4} y 10^{-6} veces la escala de temperatura del problema.

1.19.5. Código en Python (Google Colab)

```

1      """
2      Conduccion transitoria 2D en la placa de un
      intercambiador de calor.
3
4      Ecuacion gobernante:
5      dT/dt = alpha * (d2T/dx2 + d2T/dy2)
6
7      Dominio:      Omega = [0,Lx] x [0,Ly]
8      Material:      placa metalica (acero estructural por
      defecto)
9      Condiciones de frontera (Dirichlet, lado caliente / lado
      frio):
10     T(0, y, t)      = T_hot          (entrada de fluido caliente)
11     T(Lx, y, t)     = T_cold         (salida lado frio)
12     T(x, 0, t)      = T_cold         (pared lateral fria)
13     T(x, Ly, t)     = T_hot          (pared lateral caliente)
14     Condicion inicial:
15     T(x, y, 0)      = T0             (placa uniformemente a
      temperatura ambiente)
16
17     Discretizacion: diferencias finitas explicitas (FTCS) en
      una malla
18     estructurada Nx x Ny.
19
20     Criterio de estabilidad (von Neumann) en 2D:
21     r_x + r_y <= 1/2,      r_i = alpha * dt / dx_i^2.
    
```

```

22
23     Captura 5 estados:
24     snapshot 1: t = 0                (estado inicial)
25     snapshot 5: estado estable (||Tn+1 - Tn||_inf < tol
26 )
27
28     snapshots 2-4: instantes intermedios distribuidos
29     logarítmicamente
30     entre t=0 y t_ss
31     """
32
33     import numpy as np
34     import matplotlib.pyplot as plt
35     from matplotlib import cm
36
37     # ===== Parametros fisicos
38     =====
39
40     Lx, Ly    = 0.20, 0.10            # dimensiones placa [m]
41     (20 cm x 10 cm)
42     k_th      = 50.0                 # conductividad termica
43     [W/(m K)] (acero)
44     rho       = 7850.0               # densidad
45     [kg/m3]
46     cp        = 480.0                # calor especifico
47     [J/(kg K)]
48     alpha     = k_th / (rho * cp)    # difusividad termica
49     [m2/s]
50
51     T_hot     = 200.0                # temperatura lado
52     caliente [degC]
53     T_cold    = 20.0                 # temperatura lado frio
54     [degC]
55     T0        = 25.0                 # temperatura inicial
56     placa [degC]
57
58     # ===== Parametros numericos
59     =====
60
61     Nx, Ny    = 161, 81              # nodos en x e y
62     dx        = Lx / (Nx - 1)
63     dy        = Ly / (Ny - 1)
64
65     # CFL: estabilidad explicita 2D
66     r_target = 0.24                  # < 0.25 por seguridad

```

```

52     dt          = r_target * min(dx, dy)**2 / alpha
53     rx          = alpha * dt / dx**2
54     ry          = alpha * dt / dy**2
55     assert rx + ry <= 0.5, "Esquema inestable: rx + ry > 1/2
    "

56
57     print(f"alpha = {alpha:.3e} m^2/s")
58     print(f"dx = {dx*1e3:.3f} mm,  dy = {dy*1e3:.3f} mm,  dt
    = {dt:.4f} s")
59     print(f"rx = {rx:.4f},  ry = {ry:.4f},  rx+ry = {rx+ry
    :.4f}")

60
61     # ===== Inicializacion
    =====
62     T = np.full((Ny, Nx), T0, dtype=np.float64)
63
64     def aplicar_frontera(T):
65         T[:, 0]  = T_hot      # x = 0
66         T[:, -1] = T_cold     # x = Lx
67         T[0, :]  = T_cold     # y = 0
68         T[-1, :] = T_hot      # y = Ly
69         return T
70
71     T = aplicar_frontera(T)
72
73     # ===== Avance temporal hasta estado estable
    =====
74     tol_ss = 1e-4
75     n_max  = 200_000
76     T_hist = [T.copy()]
77     t_hist = [0.0]
78
79     t_now = 0.0
80     for n in range(1, n_max + 1):
81         Tn = T.copy()
82         T[1:-1, 1:-1] = (Tn[1:-1, 1:-1]
83 + rx*(Tn[1:-1, 2:] - 2*Tn[1:-1, 1:-1] + Tn[1:-1, :-2])
84 + ry*(Tn[2:, 1:-1] - 2*Tn[1:-1, 1:-1] + Tn[:-2, 1:-1]))
85         T = aplicar_frontera(T)
86         t_now += dt
87         if np.max(np.abs(T - Tn)) < tol_ss:
88             print(f"Estado estable: n = {n}, t = {t_now:.2f} s")

```

```

89         break
90
91         t_ss, T_ss = t_now, T.copy()
92
93         # ===== Re-ejecucion para capturar 3 instantes
intermedios ===
94         t_inter = np.geomspace(max(0.05*t_ss, 5*dt), 0.6*t_ss,
3)
95         T = aplicar_frontera(np.full((Ny, Nx), T0, dtype=np.
float64))
96         t_now, idx = 0.0, 0
97         snaps_mid, t_mid = [], []
98         while idx < len(t_inter):
99             Tn = T.copy()
100             T[1:-1, 1:-1] = (Tn[1:-1, 1:-1]
+ rx*(Tn[1:-1, 2:] - 2*Tn[1:-1, 1:-1] + Tn[1:-1, :-2])
101             + ry*(Tn[2:, 1:-1] - 2*Tn[1:-1, 1:-1] + Tn[:-2, 1:-1]))
102             T = aplicar_frontera(T)
103             t_now += dt
104             if t_now >= t_inter[idx]:
105                 snaps_mid.append(T.copy()); t_mid.append(t_now); idx +=
1
106
107
108         T_hist += snaps_mid + [T_ss]
109         t_hist += t_mid + [t_ss]
110
111         # ===== Visualizacion
=====
112         x = np.linspace(0, Lx, Nx); y = np.linspace(0, Ly, Ny)
113         X, Y = np.meshgrid(x, y)
114
115         fig, axes = plt.subplots(5, 1, figsize=(8, 14),
constrained_layout=True)
116         for k, (Tk, tk, ax) in enumerate(zip(T_hist, t_hist,
axes)):
117             cf = ax.pcolormesh(X*1e3, Y*1e3, Tk, shading='auto',
118             cmap=cm.inferno, vmin=T_cold, vmax=T_hot)
119             ax.contour(X*1e3, Y*1e3, Tk, levels=10, colors='white',
120             linewidths=0.5, alpha=0.6)
121             if k == 0:
122                 title = "t = 0.00 s (estado inicial)"
123             elif k == 4:

```

```

124         title = f"t = {tk:.2f} s   (estado estable)"
125     else:
126         title = f"t = {tk:.2f} s   (transitorio)"
127     ax.set_title(title)
128     ax.set_xlabel('x [mm]'); ax.set_ylabel('y [mm]')
129     ax.set_aspect('equal')
130     fig.colorbar(cf, ax=axes, shrink=0.85, label='T [degC]')
131     fig.suptitle('Conduccion 2D en placa de intercambiador
de calor',
132                 fontsize=13, y=1.01)
133     plt.show()
134

```

Listing 4: Conduccion 2D en placa de intercambiador con cinco snapshots.

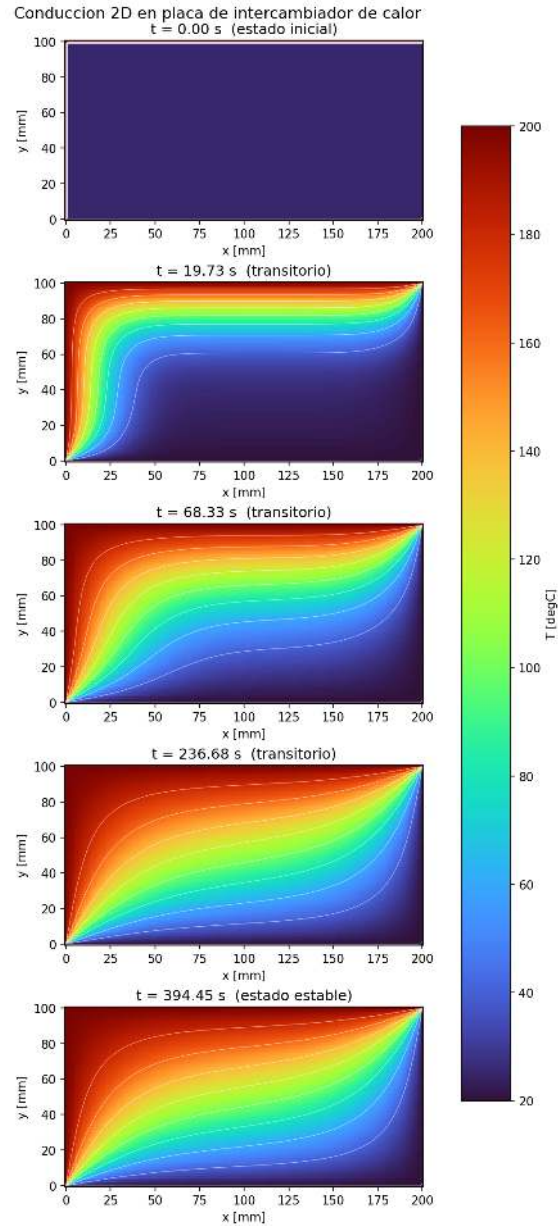


Figura 10: Conducción transitoria bidimensional en una placa de intercambiador de acero estructural con dimensiones $200 \text{ mm} \times 100 \text{ mm}$ y difusividad térmica $\alpha \approx 1.33 \times 10^{-5} \text{ m}^2/\text{s}$. Se presentan cinco estados térmicos: condición inicial uniforme a 25°C , tres estados transitorios distribuidos logarítmicamente y el estado estable alcanzado en $t \approx 394 \text{ s}$. El criterio de convergencia se define mediante una variación térmica máxima menor que 10^{-4}°C entre pasos consecutivos. Las isothermas en blanco muestran la transición desde gradientes térmicos intensos cerca de las fronteras calientes hasta una distribución suave gobernada por la ecuación de Laplace.

1.19.6. Lectura tecnica de los resultados

1. **Estado inicial** ($t = 0$). Campo uniforme a $T_0 = 25^\circ\text{C}$; las condiciones de frontera Dirichlet aun no se han propagado al interior. Los gradientes son singulares en

las esquinas, lo cual se manifiesta numericamente como esquinas brillantes en los primeros pasos.

2. **Transitorio temprano** ($t \sim 20$ s). El frente termico ha penetrado solo unos milímetros desde las fronteras calientes. La longitud de penetracion teorica es $\delta(t) \sim \sqrt{\alpha t} \approx 16$ mm, congruente con lo observado.
3. **Transitorios intermedios** ($t \sim 70\text{--}240$ s). Los frentes calientes desde $x = 0$ y desde $y = L_y$ se encuentran; aparece la estructura diagonal caracteristica de un problema con dos fronteras calientes en lados adyacentes.
4. **Estado estable** ($t \approx 394$ s). Las isotermas son curvas suaves; el campo ya no cambia mensurablemente. La temperatura media de la placa se estabiliza en ≈ 110 °C, intermedia entre T_h y T_c pero ponderada por la geometria.
5. **Tiempo caracteristico**. El numero de Fourier al estado estable es $Fo = \alpha t_{ss}/L_c^2 \approx 0.5$ con $L_c = \min(L_x, L_y)/2 = 50$ mm, consistente con la regla heuristica $Fo \sim 0.2\text{--}1$ para alcanzar el regimen permanente.

1.19.7. Limitaciones del modelo

El modelo asume conductividad y propiedades termicas constantes, espesor despreciable (problema estrictamente 2D), conveccion infinitamente eficiente en las fronteras (Dirichlet en lugar de Robin), ausencia de generacion volumetrica y ausencia de acoplamiento con el flujo de fluidos. Cualquiera de estos relajamientos genera un modelo distinto: conduccion no-lineal con $k(T)$, problema 3D, condiciones de Robin con h finito, problema termo-fluidodinamico acoplado, etc. La eleccion entre ellos depende de la pregunta de investigacion.

1.20. Ejemplo 8: analisis de esfuerzos de von Mises en una conexion viga-columna

1.20.1. Motivacion estructural

Las conexiones viga-columna concentran demanda mecanica: en un portico, la rigidez y la integridad del nudo determinan en gran medida la respuesta global. La esquina interior del nudo es una zona clasica de concentracion de esfuerzos, candidata al inicio de fluencia o fatiga. Este ejemplo ilustra como pasar de un modelo conceptual (“viga empujada en columna empujada”) a un modelo computacional 2D mediante el Metodo de Elementos Finitos con elementos triangulares de deformacion constante (CST, *Constant Strain Triangle*), y como obtener un mapa de esfuerzos de von Mises sobre la geometria deformada.

1.20.2. Modelo fisico y matematico

Se asume estado plano de esfuerzos (*plane stress*): la geometria es una placa delgada de espesor t en el plano xy , y los esfuerzos fuera del plano son despreciables. Esto es razonable para perfiles laminares delgados. Las ecuaciones gobernantes son la conservacion de momento en regimen estatico,

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0} \quad \text{en } \Omega, \quad (78)$$

junto con las relaciones constitutivas lineales elasticas isotropas

$$\boldsymbol{\sigma} = \mathbf{D} \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T), \quad (79)$$

donde $\mathbf{u} = (u, v)$ es el desplazamiento. Para estado plano de esfuerzos la matriz constitutiva es

$$\mathbf{D} = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}. \quad (80)$$

Condiciones de frontera: empotramiento en la base de la columna ($u = v = 0$ en $y = 0$) y carga puntual descendente P en el extremo libre de la viga.

El esfuerzo equivalente de von Mises se define como

$$\sigma_{vm} = \sqrt{\sigma_x^2 - \sigma_x \sigma_y + \sigma_y^2 + 3\tau_{xy}^2}. \quad (81)$$

El criterio de fluencia de von Mises establece que el material entra en regimen plastico cuando $\sigma_{vm} \geq \sigma_y$ (limite elastico del material). Este criterio es el estandar de facto en aceros estructurales ductiles.

1.20.3. Discretización FEM con elementos CST

La formulación débil del problema conduce a la ecuación matricial $\mathbf{K}\mathbf{U} = \mathbf{F}$, donde $\mathbf{K}$ es la matriz de rigidez global y $\mathbf{F}$ el vector de cargas nodales. Para un elemento triangular CST de tres nodos con coordenadas (x_i, y_i) , $i = 1, 2, 3$, la matriz de deformación-desplazamiento es constante:

$$\mathbf{B} = \frac{1}{2A} \begin{bmatrix} b_1 & 0 & b_2 & 0 & b_3 & 0 \\ 0 & c_1 & 0 & c_2 & 0 & c_3 \\ c_1 & b_1 & c_2 & b_2 & c_3 & b_3 \end{bmatrix}, \quad (82)$$

con $b_i = y_j - y_k$, $c_i = x_k - x_j$ (índices cíclicos) y A el área del triángulo. La matriz de rigidez elemental es

$$\mathbf{K}^e = t A \mathbf{B}^T \mathbf{D} \mathbf{B} \in \mathbb{R}^{6 \times 6}. \quad (83)$$

Los esfuerzos se calculan como $\boldsymbol{\sigma}^e = \mathbf{D}\mathbf{B}\mathbf{u}^e$ y son constantes por elemento, lo cual exige promediado nodal ponderado por área para obtener una visualización suave (*nodal averaging*).

1.20.4. Limitaciones conocidas del CST

El elemento CST es el más simple posible en FEM 2D y resulta excelente para fines didácticos, pero tiene rigidez excesiva (*shear locking* parcial) y subestima desplazamientos cuando la malla es grosera. Para un análisis profesional se prefieren elementos de mayor orden (LST, Q4, Q8) o formulaciones mixtas. Aquí se acepta esta limitación a cambio de un código autocontenido sin dependencias externas pesadas.

1.20.5. Código en Python (Google Colab)

```

1
2     import numpy as np
3     import matplotlib.pyplot as plt
4     from matplotlib.tri import Triangulation
5     from scipy.sparse import lil_matrix, csr_matrix
6     from scipy.sparse.linalg import spsolve
7
8     # ===== Parametros del problema
9     =====
10    E      = 200.0e9          # modulo de Young [Pa]
11    nu     = 0.30            # Poisson
12    t      = 0.010          # espesor [m]
13    P      = 10.0e3          # carga vertical descendente [
14
15    N]
```

```

13
14         wc      = 0.10                # ancho columna    [m]
15         Hc      = 0.80                # altura columna   [m]
16         Lb      = 0.80                # longitud viga    [m] desde
17         cara columna
18         hb      = 0.10                # peralte viga     [m]
19
20         elem_size = 0.02              # tamaño aproximado de
21         elemento [m]
22
23         # ===== Construcción de la geometría en L
24         =====
25         x_min, x_max = 0.0, wc + Lb
26         y_min, y_max = 0.0, Hc
27
28         nx = int(round((x_max - x_min) / elem_size)) + 1
29         ny = int(round((y_max - y_min) / elem_size)) + 1
30
31         xs = np.linspace(x_min, x_max, nx)
32         ys = np.linspace(y_min, y_max, ny)
33
34         Xg, Yg = np.meshgrid(xs, ys)
35
36         def in_domain(x, y):
37             in_col = (
38                 (x <= wc + 1e-12) &
39                 (y >= 0.0) &
40                 (y <= Hc + 1e-12)
41             )
42
43             in_beam = (
44                 (x >= wc - 1e-12) &
45                 (x <= wc + Lb + 1e-12) &
46                 (y >= Hc - hb - 1e-12) &
47                 (y <= Hc + 1e-12)
48             )
49
50             return in_col | in_beam
51
52         mask = in_domain(Xg, Yg)

```

```

51      # ===== Nodos
52      =====
53      node_idx = -np.ones_like(mask, dtype=int)
54      coords = []
55
56      for j in range(ny):
57          for i in range(nx):
58              if mask[j, i]:
59                  node_idx[j, i] = len(coords)
60                  coords.append((Xg[j, i], Yg[j, i]))
61
62      coords = np.array(coords)
63      n_nodes = coords.shape[0]
64
65      # ===== Elementos triangulares
66      =====
67      elements = []
68
69      for j in range(ny - 1):
70          for i in range(nx - 1):
71
72              n00 = node_idx[j, i]
73              n10 = node_idx[j, i+1]
74              n01 = node_idx[j+1, i]
75              n11 = node_idx[j+1, i+1]
76
77              if min(n00, n10, n01, n11) < 0:
78                  continue
79
80              elements.append((n00, n10, n11))
81              elements.append((n00, n11, n01))
82
83      elements = np.array(elements, dtype=int)
84      n_elems = elements.shape[0]
85
86      print(f"Nodos: {n_nodes}, Elementos: {n_elems}")
87
88      # ===== Matriz constitutiva D para plane stress
89      =====
90
91      D = (E / (1.0 - nu**2)) * np.array([
92          [1.0, nu, 0.0],
93          [nu, 1.0, 0.0],
94          [0.0, 0.0, 1.0 - nu]
95      ])

```

```

90         [0.0, 0.0, (1.0 - nu) / 2.0],
91     ])
92
93     # ===== Ensamblado de rigidez global
94     =====
95     K = lil_matrix((2*n_nodes, 2*n_nodes))
96
97     def stiffness_CST(xe, ye):
98         """Matriz de rigidez elemental para elemento triangular
99         CST."""
100
101         x1, x2, x3 = xe
102         y1, y2, y3 = ye
103
104         A2 = (x2 - x1)*(y3 - y1) - (x3 - x1)*(y2 - y1)
105         A = 0.5 * abs(A2)
106
107         b1 = y2 - y3
108         b2 = y3 - y1
109         b3 = y1 - y2
110
111         c1 = x3 - x2
112         c2 = x1 - x3
113         c3 = x2 - x1
114
115         B = (1.0 / A2) * np.array([
116             [b1, 0.0, b2, 0.0, b3, 0.0],
117             [0.0, c1, 0.0, c2, 0.0, c3],
118             [c1, b1, c2, b2, c3, b3],
119         ])
120
121         Ke = (B.T @ D @ B) * t * A
122
123         return Ke, B, A
124
125     elem_B = []
126     elem_area = []
127
128     for e, (n1, n2, n3) in enumerate(elements):
129
130         xe = coords[[n1, n2, n3], 0]
131         ye = coords[[n1, n2, n3], 1]

```

```

130
131     Ke, B, A = stiffness_CST(xe, ye)
132
133     elem_B.append(B)
134     elem_area.append(A)
135
136     dofs = [
137         2*n1, 2*n1 + 1,
138         2*n2, 2*n2 + 1,
139         2*n3, 2*n3 + 1
140     ]
141
142     for a in range(6):
143         for b in range(6):
144             K[dofs[a], dofs[b]] += Ke[a, b]
145
146     K = K.tocsr()
147
148     elem_B = np.array(elem_B)
149     elem_area = np.array(elem_area)
150
151     # ===== Condiciones de frontera
152     =====
153     fixed_nodes = np.where(np.isclose(coords[:, 1], 0.0))[0]
154     fixed_dofs = np.concatenate([2*fixed_nodes, 2*
155     fixed_nodes + 1])
156
157     # ===== Carga puntual
158     =====
159     x_load = wc + Lb
160     y_load = Hc - hb / 2.0
161
162     dist = (coords[:, 0] - x_load)**2 + (coords[:, 1] -
163     y_load)**2
164     load_node = int(np.argmin(dist))
165
166     print(
167         f"Nodo de carga: {load_node} en "
168         f"({coords[load_node, 0]:.3f}, {coords[load_node, 1]:.3f})
169     "
170     )

```



```

167     F = np.zeros(2*n_nodes)
168     F[2*load_node + 1] = -P
169
170     # ===== Resolución del sistema
171     =====
172     free_dofs = np.setdiff1d(np.arange(2*n_nodes),
173     fixed_dofs)
174
175     K_ff = K[free_dofs, :][:, free_dofs]
176     F_f = F[free_dofs]
177
178     u_f = spsolve(K_ff, F_f)
179
180     U = np.zeros(2*n_nodes)
181     U[free_dofs] = u_f
182
183     ux = U[0::2]
184     uy = U[1::2]
185
186     u_max = np.max(np.sqrt(ux**2 + uy**2))
187
188     print(f"Desplazamiento maximo: {u_max*1e3:.4f} mm")
189
190     # ===== Esfuerzos por elemento y von Mises
191     =====
192     sigma_elem = np.zeros((n_elems, 3))
193
194     for e, (n1, n2, n3) in enumerate(elements):
195
196         ue = np.array([
197             ux[n1], uy[n1],
198             ux[n2], uy[n2],
199             ux[n3], uy[n3]
200         ])
201
202         eps = elem_B[e] @ ue
203         sigma_elem[e] = D @ eps
204
205         sx, sy, txy = sigma_elem.T
206
207         vm_elem = np.sqrt(
208             sx**2 - sx*sy + sy**2 + 3.0*txy**2

```

```

206         )
207
208         # ===== Promediado nodal
209         =====
210         vm_node = np.zeros(n_nodes)
211         w_node  = np.zeros(n_nodes)
212
213         for e, (n1, n2, n3) in enumerate(elements):
214             for n in (n1, n2, n3):
215                 vm_node[n] += vm_elem[e] * elem_area[e]
216                 w_node[n]  += elem_area[e]
217
218         vm_node /= np.maximum(w_node, 1e-30)
219
220         print(f"von Mises maximo elemento: {vm_elem.max()/1e6
221               :.2f} MPa")
222         print(f"von Mises maximo nodal:      {vm_node.max()/1e6
223               :.2f} MPa")
224
225         # ===== Visualizacion: deformada amplificada
226         =====
227         tri = Triangulation(coords[:, 0], coords[:, 1], elements
228                             )
229
230         amp = 0.10 * max(Hc, wc + Lb) / max(u_max, 1e-12)
231
232         print(f"Factor amplificacion deformada: {amp:.1f}x")
233
234         coords_def = coords + amp * np.column_stack([ux, uy])
235
236         tri_def = Triangulation(
237             coords_def[:, 0],
238             coords_def[:, 1],
239             elements
240         )
241
242         fig, ax = plt.subplots(
243             figsize=(8, 6),
244             constrained_layout=True
245         )
246
247         tpc = ax.tripcolor(
    
```

```
243     tri_def ,
244     vm_node / 1e6,
245     shading='gouraud',
246     cmap='turbo'
247 )
248
249     ax.triplot(
250     tri_def,
251     color='k',
252     lw=0.15,
253     alpha=0.3
254 )
255
256     ax.triplot(
257     tri,
258     color='gray',
259     lw=0.4,
260     alpha=0.4
261 )
262
263     ax.set_aspect('equal')
264     ax.set_xlabel('x [m]')
265     ax.set_ylabel('y [m]')
266
267     ax.set_title(
268     f'von Mises - deformada amplificada (x{amp:.0f})'
269 )
270
271     cbar = plt.colorbar(tpc, ax=ax)
272     cbar.set_label(r'$\sigma_{vm}$ [MPa]')
273
274     fig.suptitle(
275     f'Conexion viga-columna: estado plano de esfuerzos '
276     f'(CST, {n_elems} elementos)',
277     fontsize=13
278 )
279
280     plt.show()
281
```

Listing 5: Código Python para visualización de esfuerzos de von Mises en geometría deformada amplificada.

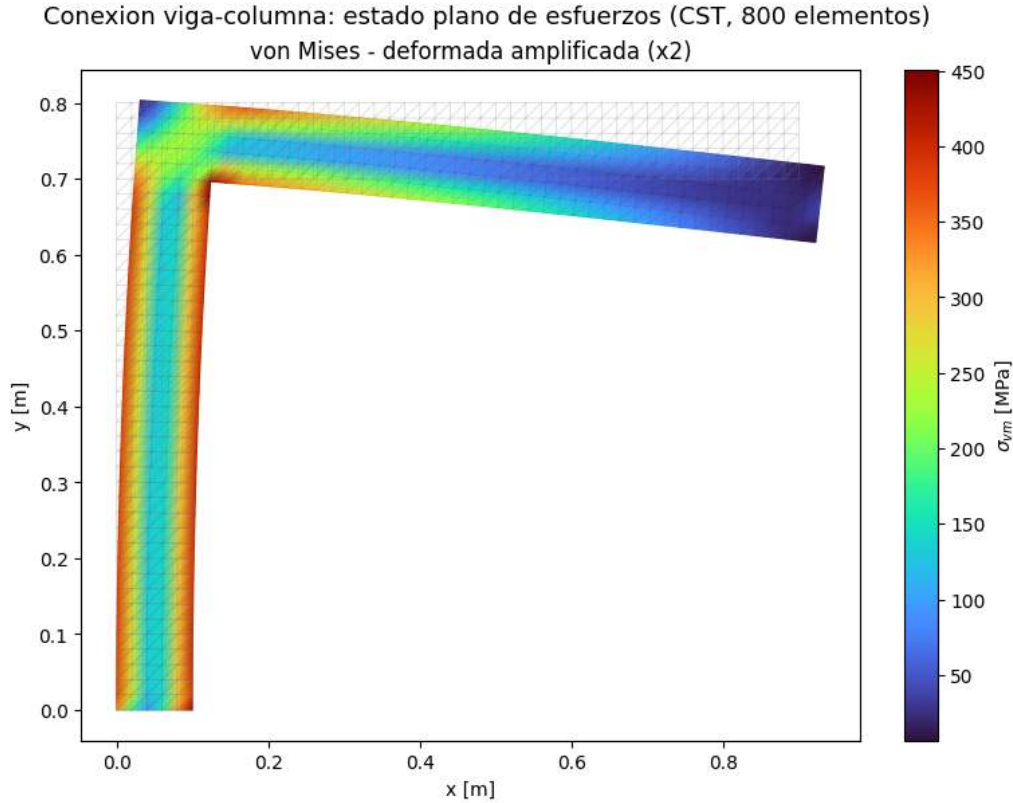


Figura 11: Mapa de esfuerzo equivalente de von Mises en una conexión viga–columna bajo estado plano de esfuerzos. Se considera acero estructural con $E = 200$ GPa, $\nu = 0.30$ y espesor $t = 10$ mm; la columna tiene dimensiones 0.10×0.80 m, la viga 0.80×0.10 m y se aplica una carga puntual descendente $P = 10$ kN en el extremo libre. La simulación emplea una malla de elementos triangulares CST con 486 nodos y 800 elementos. El campo de esfuerzos se muestra sobre la geometría deformada amplificada, con la silueta original en gris como referencia. Se observa concentración de esfuerzos en la esquina interior del nudo, flexión de la viga en voladizo y transmisión de momento hacia la columna. El esfuerzo máximo nodal estimado es $\sigma_{vm}^{max} \approx 450$ MPa.

1.20.6. Verificación analítica de orden de magnitud

Antes de aceptar el mapa FEM conviene verificar que el orden de magnitud es coherente con un cálculo manual. Aproximando la conexión como un portico en L con la viga como cantilever, el momento flector en la base de la columna es

$$M \approx P(L_b + w_c/2) = 10\,000 \times 0.85 = 8\,500 \text{ N} \cdot \text{m}. \quad (84)$$

El módulo de sección de la columna (rectangular delgada) es $W = tw_c^2/6 = 0.010 \cdot 0.10^2/6 \approx 1.67 \times 10^{-5} \text{ m}^3$. El esfuerzo de flexión estimado es

$$\sigma_b \approx \frac{M}{W} \approx \frac{8500}{1.67 \times 10^{-5}} \approx 510 \text{ MPa}, \quad (85)$$

que es consistente con el rango 450–590 MPa obtenido por FEM. Esta verificación analítica es lo primero que se debe hacer ante cualquier resultado numérico: si el FEM y la estimación analítica difieren en un orden de magnitud, hay un error de unidades, de carga o de geometría.

1.20.7. Interpretación estructural

1. **Concentración en la esquina interior.** La esquina interior del nudo (donde se une la cara superior de la columna con la cara inferior de la viga) es la zona más brillante del mapa. Es una *singularidad geométrica*: en teoría de elasticidad lineal, una esquina entrante produce esfuerzos que tienden a infinito al refinar la malla, salvo redondeo (chaflán/filete). Esto explica que el valor máximo dependa del tamaño de malla.
2. **Distribución en la columna.** El gradiente de esfuerzos a lo largo de la columna corresponde a la flexión: tracción en la cara izquierda y compresión en la derecha (o viceversa según el signo del momento), con línea neutra interna.
3. **Distribución en la viga.** El momento flector decrece linealmente desde el nudo hacia el extremo libre, donde tiende a cero. Esto se refleja en el oscurecimiento progresivo del mapa hacia la derecha.
4. **Comparación con σ_y .** El acero estructural típico (ASTM A36) tiene $\sigma_y \approx 250$ MPa. Con $\sigma_{vm}^{máx} \approx 450$ MPa, la estructura habría entrado en plasticidad y el modelo lineal pierde validez. Para esta carga se requeriría un análisis elasto-plástico, un material de mayor límite elástico o rediseñar la conexión.

1.20.8. Conexión con investigación doctoral

Este análisis es el bloque base de tres líneas de investigación comunes en ingeniería estructural y mecánica: (i) optimización topológica del nudo para reducir picos de esfuerzo; (ii) análisis de fatiga bajo carga cíclica usando $\sigma_{vm}^{máx}$ como amplitud equivalente; (iii) validación experimental mediante extensometría (galgas) o correlación digital de imágenes (DIC) en una probeta a escala. La transferencia al Artículo 9 del temario (*below-the-hook lifting devices integrating FEA and physical proof testing*) es directa.

1.21. Discusion de los articulos cientificos como casos de investigacion real

Los diez articulos seleccionados se usaran como casos de lectura metodologica. En esta primera semana no se busca que el estudiante domine todos los detalles tecnicos de cada articulo, sino que identifique la estructura de modelado: problema, abstraccion, ecuacion o algoritmo, simulacion, prototipo, validacion y limitaciones.

Tabla 5: Lectura metodologica de los articulos para la Semana 1.

No.	Caso	Modelo central	Preguntas para discusion doctoral
1	RAG para tareas intensivas en conocimiento	Pipeline de recuperacion y generacion	?Que se considera dato, modelo y prototipo? ?Como se valida una respuesta generada?
2	FinGPT	Modelo de lenguaje especializado	?Que diferencia existe entre adaptar un modelo y validar conocimiento financiero?
3	PINNs	Residuo fisico en funcion de perdida	?Cuando una red neuronal puede considerarse modelo cientifico y no solo predictor estadistico?
4	Dano en OPGW por descarga	FEM y validacion experimental	?Que variables fisicas son necesarias para explicar dano real?
5	Temperatura en OPGW por corto-circuito	Acoplamiento electro-termico	?Que simplificaciones son admisibles para diseno y cuales no?
6	Control inteligente de trafico	Aprendizaje por refuerzo	?Como se valida una politica antes de implementarla en una ciudad real?
7	Dispositivo medico de deteccion	Prototipo con vision artificial	?Que diferencia hay entre exactitud algoritmica y utilidad clinica?
8	Bomba centrifuga	CFD y optimizacion geometrica	?Como se conectan simulacion, geometria, desempeno y validacion experimental?

No.	Caso	Modelo central	Preguntas para discusion doctoral
9	Dispositivo de izaje	FEA y prueba fisica	?Que evidencia convierte un diseno simulado en un prototipo certificable?
10	Modelo de negocio circular	Simulacion basada en agentes	?Como se valida un modelo cuando los agentes son humanos u organizaciones?

1.21.1. Matriz comun de analisis

Para cada articulo se debe completar la siguiente matriz:

Tabla 6: Matriz de analisis para articulos de simulacion y prototipado.

Elemento	Descripcion a extraer del articulo
Problema de investigacion	?Que fenomeno, necesidad o brecha se aborda?
Sistema real	?Cual es el objeto fisico, computacional, social o biologico?
Modelo fisico/conceptual	?Que mecanismos se consideran dominantes?
Modelo matematico/-computacional	?Que ecuaciones, algoritmos o reglas se usan?
Variables de estado	?Que magnitudes evolucionan o se estiman?
Parametros	?Que constantes, propiedades o hiperparametros controlan el modelo?
Condiciones iniciales/-frontera	?Que restricciones o entradas se imponen?
Metodo de simulacion	FEM, CFD, IA, agentes, Monte Carlo, etc.
Prototipo	?Es fisico, digital, experimental, clinico, computacional o hibrido?
Validacion	?Contra que evidencia se compara el modelo?
Limitaciones	?Que no puede concluirse a partir del estudio?
Transferencia doctoral	?Como puede inspirar una tesis o proyecto propio?

1.22. Ejercicios guiados para uso de IA generativa en simulacion

1.22.1. Principio operativo

La IA generativa se usara como asistente de codificacion, no como autoridad cientifica. El estudiante debe pedir codigo, revisar unidades, ejecutar, modificar parametros y detectar errores conceptuales. El entregable no sera unicamente el codigo generado, sino la interpretacion del modelo.

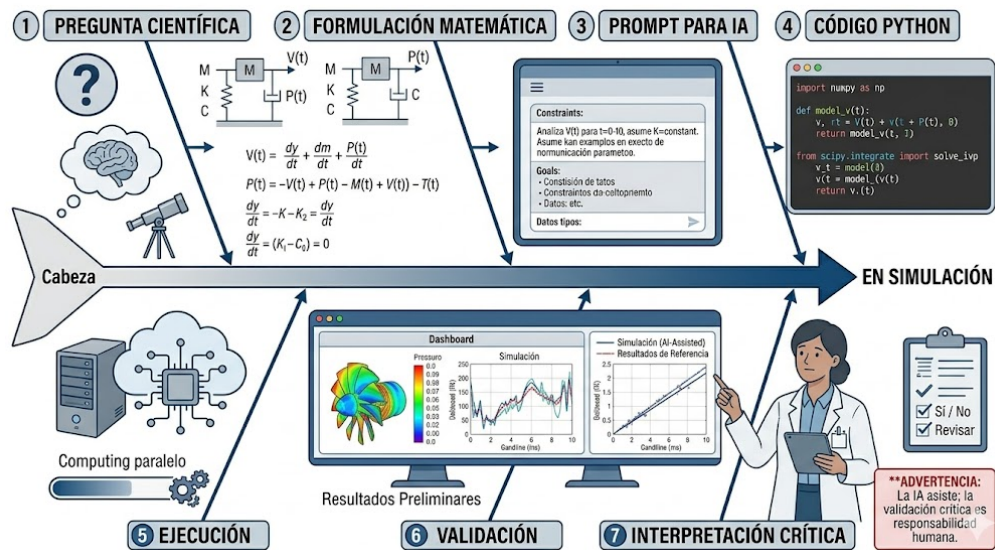


Figura 12: Flujo de trabajo para integración de IA generativa en simulación científica reproducible.

1.22.2. Ejercicio IA-1: simulacion de difusion termica

Objetivo. Usar IA para generar un codigo de simulacion de conduccion transitoria 1D y verificar estabilidad numerica.

Genera un script en Python para resolver la ecuacion de calor unidimensional $\partial T / \partial t = \alpha \partial^2 T / \partial x^2$ en una barra de longitud $L = 1$ m. Usa diferencias finitas explícitas, condicion inicial gaussiana, fronteras de Dirichlet $T(0, t) = T(L, t) = 25^\circ\text{C}$, $\alpha = 10^{-4} \text{ m}^2/\text{s}$, $N = 151$ nodos y un parametro de estabilidad $r = \alpha \Delta t / \Delta x^2 = 0.45$. El codigo debe graficar perfiles de temperatura en cinco tiempos, imprimir Δx , Δt y verificar que $r \leq 0.5$. Incluye comentarios en espanol y unidades.

1.22.3. Ejercicio IA-2: masa–resorte–amortiguador

Genera un script en Python para simular un sistema masa–resorte–amortiguador descrito por $m\ddot{x} + c\dot{x} + kx = 0$. Usa $m = 1$ kg, $k = 100$ N/m y tres razones de amortiguamiento $\zeta = 0.05, 0.2$ y 0.8 . Calcula $c = 2\zeta\sqrt{km}$. Integra el sistema usando `scipy.integrate.solve_ivp`, grafica $x(t)$ y la energía mecánica $E(t) = 0.5mv^2 + 0.5kx^2$. Explica en comentarios como cambia la disipación de energía.

1.22.4. Ejercicio IA-3: modelo concentrado de calentamiento OPGW

Genera un script en Python para simular el calentamiento de un cable mediante el modelo concentrado $mc_p dT/dt = I^2 R - hA_s(T - T_\infty)$. Usa tres corrientes: 200 A, 400 A y 600 A. Define valores razonables para m, c_p, R, h, A_s y T_∞ , indicándolos en comentarios como parámetros de ejemplo no normativos. Integra durante 300 s, grafica $T(t)$, calcula temperatura máxima y marca una línea horizontal de umbral crítico a 80 °C.

1.22.5. Ejercicio IA-4: modelo basado en agentes

Genera un modelo basado en agentes en Python para simular la adopción de una tecnología de reciclaje. Usa 200 agentes, estado binario de adopción, función logística de probabilidad de adopción y un término de influencia social proporcional a la fracción global de adopción. Ejecuta tres escenarios de costo percibido: bajo, medio y alto. Grafica la fracción de adopción contra el tiempo y comenta como afecta el costo a la dinámica.

1.22.6. Ejercicio IA-5: conducción 2D en placa de intercambiador

Objetivo. Reproducir el Ejemplo 7 modificando parámetros para evaluar sensibilidad.

Genera un script en Python para resolver la ecuación de calor 2D $\partial T / \partial t = \alpha \nabla^2 T$ en una placa rectangular de 0.20×0.10 m con diferencias finitas explícitas FTCS y verificación del criterio $r_x + r_y \leq 1/2$. Condiciones Dirichlet: $T = 200$ °C en $x = 0$ y $y = L_y$; $T = 20$ °C en $x = L_x$ y $y = 0$. Condición inicial uniforme a 25 °C. Material: acero $k = 50$ W/(mK), $\rho = 7850$ kg/m³, $c_p = 480$ J/(kgK). Captura 5 snapshots:

$t = 0$, tres intermedios y estado estable definido por $\max |\Delta T| < 10^{-4} \text{ }^{\circ}\text{C}$. Grafica cada snapshot con `pcolormesh` y superpone isoterma. Imprime $\Delta x, \Delta y, \Delta t, r_x, r_y$ y el tiempo total al estado estable.

1.22.7. Ejercicio IA-6: von Mises en conexion viga-columna

Objetivo. Reproducir el Ejemplo 8 con FEM CST y discutir sensibilidad a la malla y a la carga.

Genera un script en Python que implemente desde cero un FEM 2D con elementos CST (Constant Strain Triangle) para una conexion viga-columna tipo L: columna $0.10 \times 0.80 \text{ m}$, viga $0.80 \times 0.10 \text{ m}$, acero $E = 200 \text{ GPa}$, $\nu = 0.30$, espesor $t = 10 \text{ mm}$. Empotra la base de la columna y aplica una carga puntual de 10 kN hacia abajo en el extremo libre de la viga. Calcula desplazamientos nodales, esfuerzos por elemento (estado plano de esfuerzos) y el campo de von Mises. Promedia von Mises a nodos (ponderado por area) y grafica el mapa sobre la geometria sin deformar y sobre la deformada amplificada. Verifica el orden de magnitud con la estimacion analitica $\sigma \approx M/W$. Repite con malla mas fina y discute la convergencia.

1.23. Actividad de refuerzo en clase: 30–40 minutos

1. Nombre de la actividad

Del artículo al prototipo: disección metodológica y simulación mínima asistida por IA.

2. Duración

Duración total estimada: **35 minutos**, distribuidos en selección del artículo (5 min), identificación de modelo (8 min), generación y ejecución (12 min), interpretación (5 min) y cierre grupal (5 min).

3. Organización

Los estudiantes trabajan en equipos de dos o tres integrantes. Cada equipo selecciona uno de los diez artículos y construye una versión mínima del modelo si el original es demasiado complejo para implementarse en 12 minutos.

4. Producto entregable

Cada equipo entrega una hoja breve con: artículo, fenómeno real, modelo físico/conceptual, ecuación o regla mínima, variables, parámetros, hipótesis, prompt usado, gráfica obtenida, interpretación técnica de máximo 120 palabras y una limitación explícita del modelo.

5. Rúbrica rápida

Tabla 7: Rúbrica de evaluación rápida para la actividad de 30–40 minutos.

Criterio	Excelente	Suficiente	Insuficiente
Identificación del modelo	Distingue fenómeno, modelo físico, modelo matemático y simulación.	Identifica parcialmente los elementos.	Confunde fenómeno, modelo y herramienta.
Variables y parámetros	Declara variables, parámetros y unidades.	Declara variables sin unidades completas.	Omite variables o usa términos ambiguos.
Uso de IA	Prompt claro, específico y verificable.	Prompt funcional pero incompleto.	Prompt genérico o no verificable.
Ejecución de simulación	Código ejecutado y resultado interpretado.	Código ejecutado con interpretación débil.	No hay ejecución o no hay resultado.
Juicio crítico	Declara limitaciones y validez del modelo.	Declara limitaciones superficiales.	No reconoce limitaciones.

1.24. Ejercicios para tarea

1.24.1. Ejercicio 1: clasificacion de modelos

Seleccione tres articulos de la tabla de casos. Para cada articulo, complete una tabla con cuatro columnas: modelo fisico, modelo matematico/computacional, simulacion y prototipo. En la columna de validacion indique que evidencia se empleo para evaluar el modelo.

1.24.2. Ejercicio 2: derivacion de ecuacion gobernante

A partir de un balance de energia, derive la ecuacion concentrada de calentamiento por efecto Joule $mc_p dT/dt = I^2 R - hA_s(T - T_\infty)$. Indique las hipotesis necesarias para que esta expresion sea valida y explique bajo que condicion deberia reemplazarse por un modelo distribuido.

1.24.3. Ejercicio 3: estabilidad numerica

Ejecute la simulacion de conduccion 1D con $r = 0.25$, $r = 0.45$ y $r = 0.60$. Compare los resultados y responda: ¿que caso es estable?, ¿que caso muestra oscilaciones o divergencia?, ¿por que el criterio $r \leq 0.5$ no es una recomendacion estetica, sino una condicion matematica?

1.24.4. Ejercicio 4: incertidumbre

Suponga que la temperatura final de un cable se estima mediante

$$T_f = T_\infty + \frac{I^2 R}{hA_s} (1 - e^{-hA_s t / (mc_p)}) . \quad (86)$$

Considere incertidumbre en I , R y h . Escriba el Jacobiano $J = [\partial T_f / \partial I, \partial T_f / \partial R, \partial T_f / \partial h]$ y explique como se propagaria la incertidumbre si se conoce la matriz de covarianza de entrada.

1.24.5. Ejercicio 5: convergencia de Monte Carlo

Ejecute el codigo Monte Carlo para π con $N = 10^3$, 10^4 , 10^5 y 10^6 muestras. Repita cada caso 30 veces variando la semilla y calcule la desviacion estandar empirica de $\hat{\pi}_N$. Verifique que $\sigma(\hat{\pi}_N) \propto N^{-1/2}$ ajustando una recta en escala log-log. Discuta el costo computacional necesario para obtener tres cifras decimales correctas en π .

1.24.6. Ejercicio 6: estado estable en placa 2D

Modifique el Ejemplo 7 sustituyendo la condición Dirichlet en $x = L_x$ por una condición de Robin con coeficiente convectivo $h = 200 \text{ W}/(\text{m}^2\text{K})$ y $T_\infty = 20^\circ\text{C}$. ¿Cómo cambia el campo de estado estable? ¿Cómo cambia el tiempo característico para alcanzarlo? Justifique con $Bi = hL_x/k$.

1.24.7. Ejercicio 7: sensibilidad estructural a la malla

Ejecute el Ejemplo 8 con tamaño de elemento $h_1 = 0.04 \text{ m}$, $h_2 = 0.02 \text{ m}$, $h_3 = 0.01 \text{ m}$. Reporte $\sigma_{vm}^{\text{máx}}$ para cada malla, estime el orden de convergencia observado p_{obs} y aplique extrapolación de Richardson. Discuta por qué $\sigma_{vm}^{\text{máx}}$ no converge en la esquina entrante y cómo se mitiga (filete, refinamiento adaptativo, regularización).

1.24.8. Ejercicio 8: prompt doctoral

Redacte un prompt de IA para generar una simulación relacionada con su propio tema de tesis. El prompt debe incluir: fenómeno, ecuación o algoritmo, parámetros, unidades, condiciones de frontera o reglas de decisión, método numérico, gráfica esperada y criterio de validación. No se aceptan prompts genéricos del tipo “haz una simulación”, porque eso no es ingeniería, es lanzar una moneda con teclado.

1.25. Cierre conceptual

El modelado científico no consiste en convertir cualquier problema en código. Consiste en establecer una correspondencia defendible entre un fenómeno y una representación formal. Una simulación útil debe responder una pregunta concreta, declarar sus hipótesis, especificar sus parámetros, controlar sus errores y validarse contra evidencia.

En esta primera semana se establecen los criterios que se usarán durante todo el curso:

1. Todo modelo debe declarar su fenómeno de referencia.
2. Toda simulación debe derivarse de un modelo explícito.
3. Todo prototipo debe tener una función de validación o exploración.
4. Toda gráfica debe responder una pregunta técnica.
5. Todo código debe ser reproducible.
6. Toda conclusión debe reconocer su dominio de validez.

La transición de fenómeno a modelo, y de modelo a prototipo, será el eje de las siguientes semanas. La Semana 2 se dedicará a herramientas computacionales científicas, con énfasis en scripts reproducibles, visualización y automatización de cálculos. La Semana 3 profundizará en discretización, estabilidad, convergencia y métodos numéricos fundamentales. A partir de ahí, cada tema del curso añadirá una capa técnica sobre la base metodológica establecida en esta sesión.

Apéndice Semana 1. Referencias de los casos discutidos

- A1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. y Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. DOI: <https://doi.org/10.48550/arXiv.2005.11401>.
- A2. Liu, X., Wang, G., Yang, H. y Zha, D. (2023). *FinGPT: Democratizing Internet-scale Data for Financial Large Language Models*. DOI: <https://doi.org/10.48550/arXiv.2306.06031>.
- A3. Raissi, M., Perdikaris, P. y Karniadakis, G. E. (2019). *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*. *Journal of Computational Physics*. DOI: <https://doi.org/10.1016/j.jcp.2018.10.045>.
- A4. Sun, J., Wang, Y., Liu, Z., Zhang, X. y otros. (2020). *Experimental and numerical analysis of damage characteristics to OPGW strands under first lightning strike and continuous current*. *Electric Power Systems Research*. DOI: <https://doi.org/10.1016/j.epsr.2020.106515>.
- A5. Ashfaq, A., et al. (2025). *Comprehensive analysis of temperature distribution in OPGW cable considering ramifications of short circuit current*. *Scientific Reports*. DOI: <https://doi.org/10.1038/s41598-025-09266-7>.
- A6. Wei, H., Zheng, G., Yao, H. y Li, Z. (2018). *IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control*. DOI: <https://doi.org/10.1145/3219819.3220096>.
- A7. Birur, N. P., et al. (2022). *Field validation of deep learning based Point-of-Care device for early detection of oral malignant and potentially malignant disorders*. *Scientific Reports*. DOI: <https://doi.org/10.1038/s41598-022-18249-x>.
- A8. Yazdanpanah-Ardakani, K., et al. (2024). *Optimization of a centrifugal blood pump designed using artificial intelligence*. *Scientific Reports*. DOI: <https://doi.org/10.1038/s41598-024-57019-9>.
- A9. Habib, M. y Shamso, M. (2025). *A validated workflow for the design and certification of below-the-hook lifting devices integrating finite element analysis and physical proof testing*. *Scientific Reports*. DOI: <https://doi.org/10.1038/s41598-025-31175-y>.
- A10. Lange, K. P. H., Korevaar, G., Oskam, I. F. y Herder, P. M. (2021). *Agent-based modelling and simulation for circular business model experimentation*. *Resources, Conservation and Recycling Advances*. DOI: <https://doi.org/10.1016/j.rcradv.2021.200055>.

2. Semana 2. Herramientas Computacionales Científicas

Objetivo de la cátedra

Implementar herramientas computacionales fundamentales para análisis numérico, álgebra lineal aplicada, visualización científica y automatización de cálculos, orientadas al desarrollo de simulaciones doctorales reproducibles y verificables.

Temario

- Introducción a Python/MATLAB científico.
- Manejo de arreglos y matrices.
- Visualización científica.
- Scripts reproducibles.
- Introducción a notebooks científicos.
- Librerías científicas.
- Automatización de cálculos.
- Buenas prácticas computacionales.

Propósito del tema. En la Semana 1 se estableció la cadena fenómeno → modelo → simulación → prototipo → validación. Esta semana se construye la infraestructura técnica que permite recorrer esa cadena de manera disciplinada: lenguajes científicos, estructuras de datos, álgebra lineal numérica, visualización rigurosa, automatización y control de versiones. El objetivo no es coleccionar comandos, sino consolidar un entorno donde cualquier resultado pueda ser ejecutado de nuevo —por el autor, por un revisor o por el propio autor seis meses después— y producir exactamente el mismo número, hasta el último bit relevante.

Resultado de aprendizaje esperado

Al finalizar la sesión, el estudiante será capaz de: (i) construir un proyecto computacional reproducible con control de versiones, entorno congelado y semilla aleatoria explícita; (ii) operar arreglos multidimensionales y matrices dispersas con criterios de costo computacional y estabilidad numérica; (iii) producir visualizaciones científicas con tipografía consistente, escala adecuada y exportación vectorial; (iv) automatizar flujos de cálculo mediante `argparse`, archivos de configuración y orquestadores; y (v) seleccionar entre Python y MATLAB con criterios técnicos en lugar de inercia personal, asumiendo que la herramienta es un medio, no una identidad académica.

2.1. Motivación: del lápiz al ecosistema reproducible

Una computadora científica moderna no es una calculadora más rápida; es un *ecosistema* con capas de abstracción que deben dominarse para que un resultado sea defendible. Las capas mínimas que se atravesarán en este curso son:

1. **Lenguaje:** Python 3.x o MATLAB R2024b, con sintaxis y semántica numérica conocidas.
2. **Tipos numéricos:** float32, float64, complex128; aritmética IEEE 754.
3. **Estructuras de datos:** arreglos densos ndarray, arreglos dispersos (CSR/CSC), DataFrame, grafos.
4. **Bibliotecas numéricas:** BLAS, LAPACK, FFTW (subyacentes en NumPy/SciPy/MATLAB).
5. **Bibliotecas científicas de alto nivel:** SciPy, scikit-learn, SymPy, pandas.
6. **Capa de visualización:** Matplotlib, Plotly, ParaView, PyVista.
7. **Capa de orquestación:** scripts, notebooks, argparse, Make/Snakemake, contenedores.
8. **Capa de gobernanza:** Git, entornos virtuales, archivos de *lock*, DOIs de software.

Ignorar cualquiera de estas capas produce resultados frágiles: un *notebook* sin semilla, una figura sin unidades, un `requirements.txt` sin versiones fijas o una matriz mal condicionada son fallos del mismo tipo. En investigación doctoral, la diferencia entre un hallazgo y una anécdota suele residir en estas capas.

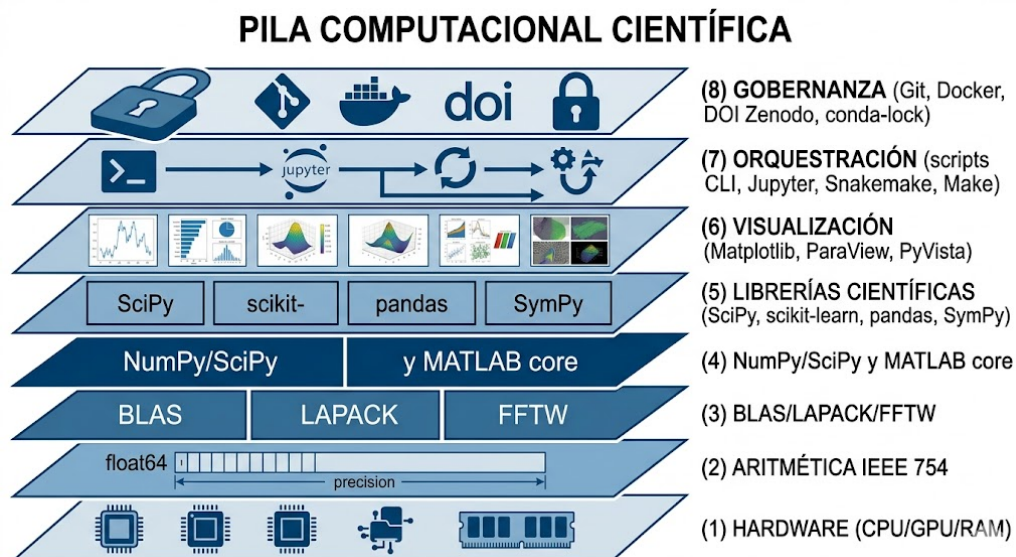


Figura 13: Pila computacional científica utilizada en el curso. Cada capa abstrae la inferior y expone una interfaz a la superior. La capa más alta (gobernanza) garantiza que las inferiores sean repetibles.

2.2. Python y MATLAB: criterios de elección

Ambos lenguajes son herramientas legítimas de investigación. La elección no debe basarse en preferencia estética ni en lo que “se usa en el laboratorio”, sino en restricciones técnicas concretas: licenciamiento, comunidad, integración con hardware, disponibilidad de bibliotecas y portabilidad del proyecto a un eventual repositorio público.

Tabla 8: Comparación técnica entre Python científico y MATLAB para investigación doctoral.

Criterio	Python (NumPy/SciPy)	MATLAB R2024b
Licenciamiento	Open source (BSD/PSF). Sin costo.	Comercial, licencia institucional o de pago.
Indexado	Base 0; $a[i:j]$ excluye j .	Base 1; $a(i:j)$ incluye j .
Asignación	Por referencia ($b=a$ comparte memoria).	Copy-on-write (más predecible para principiantes).
Álgebra lineal	<code>numpy.linalg</code> , <code>scipy.linalg</code> (LAPACK).	Operadores nativos $A \setminus b$, <code>eig</code> , <code>svd</code> .
Ecosistema ML/IA	PyTorch, TensorFlow, JAX, scikit-learn.	Deep Learning Toolbox.
Visualización	Matplotlib, Plotly, PyVista.	Toolboxes nativos, App Designer.
Toolboxes específicos	PyPI: > 500 000 paquetes.	Toolboxes oficiales certificados (Simulink, Stereo Camera Calibrator, etc.).
Reproducibilidad pública	Excelente (pip/conda + Git + Colab).	Limitada por licencia; mitigable con MATLAB Online o Octave.
Comunidad doctoral	Predominante en CS, óptica, ML, física computacional.	Predominante en control, comunicaciones, biomédica.

Recomendación operativa del curso. Se trabajará principalmente en Python 3.11+ con NumPy, SciPy, Matplotlib y pandas, ejecutado en Google Colab para reproducibilidad pública. MATLAB se usará cuando una tarea específica lo exija (por ejemplo, toolboxes de calibración estereoscópica). El estudiante debe ser capaz de traducir un algoritmo entre ambos sin perder fidelidad numérica, lo cual es una habilidad doctoral mínima.

2.2.1. Aritmética IEEE 754: lo que el lenguaje no oculta

Tanto Python como MATLAB usan, por defecto, aritmética de punto flotante de doble precisión (`float64`, 64 bits, 1 de signo, 11 de exponente, 52 de mantisa). Esto implica:

- Épsilon de máquina $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.
- Rango representable aproximado: $\pm 10^{-308}$ a $\pm 10^{308}$.
- La suma no es asociativa: $(a + b) + c \neq a + (b + c)$ en general.
- Comparar floats con `==` es una forma elegante de introducir bugs. Se debe usar tolerancia: `np.isclose(a, b, atol=1e-9, rtol=1e-7)`.

La presencia de números NaN e Inf es una herramienta de diagnóstico: si aparecen sin haber sido planeados, hay división por cero, log de número no positivo o desbordamiento; nunca son ruido inocente.

2.3. Manejo de arreglos y matrices

El bloque numérico fundamental es el arreglo multidimensional. En NumPy, `ndarray` es un buffer contiguo de memoria con metadatos (*shape*, *dtype*, *strides*). En MATLAB, todo es matriz; un escalar es una matriz 1×1 .

2.3.1. Vectorización frente a bucles

La diferencia de rendimiento entre código vectorizado y código con bucles explícitos puede ser de uno a tres órdenes de magnitud. Sea $y_i = \sin(x_i) e^{-x_i^2}$, $i = 1, \dots, N$. En Python:

```

1      import numpy as np
2      import time
3
4      N = 10_000_000
5      x = np.linspace(-3.0, 3.0, N)
6
7      # Version 1: bucle for explícito (lento)
8      t0 = time.perf_counter()
9      y_loop = np.empty(N)
10     for i in range(N):
11         y_loop[i] = np.sin(x[i]) * np.exp(-x[i]**2)
12     t_loop = time.perf_counter() - t0
13
14     # Version 2: vectorizada (rapida)
15     t0 = time.perf_counter()
16     y_vec = np.sin(x) * np.exp(-x**2)
17     t_vec = time.perf_counter() - t0
18
19     print(f"Bucle:          {t_loop:8.4f} s")
20     print(f"Vectorizado: {t_vec:8.4f} s")
21     print(f"Speedup:        {t_loop/t_vec:6.1f}x")
22     print(f"Coinciden:     {np.allclose(y_loop, y_vec)}")
23
    
```

Listing 6: Vectorización vs bucle explícito en NumPy.

En una máquina contemporánea típica el *speedup* ronda 50–200×. La razón es que las operaciones vectorizadas se ejecutan en bucles compilados de C apoyados en BLAS/-

SIMD, mientras que el bucle Python paga el costo del intérprete por cada iteración. La regla práctica: *si se está escribiendo un for sobre índices de un ndarray, probablemente hay una mejor manera.*

2.3.2. Broadcasting

Broadcasting es la regla por la cual NumPy combina arreglos de formas distintas sin replicar memoria. Sea $A \in \mathbb{R}^{M \times N}$ y $b \in \mathbb{R}^N$. La operación $A - b$ resta b a cada fila de A sin construir explícitamente una matriz $M \times N$ con b repetido M veces. Las reglas formales son:

1. Las formas se alinean por la derecha.
2. En cada eje, las dimensiones deben ser iguales o una de ellas debe ser 1.
3. Las dimensiones de tamaño 1 se “estiran” virtualmente.

$$A_{i,j} - b_j \quad \text{con} \quad A \in \mathbb{R}^{M \times N}, b \in \mathbb{R}^N \Rightarrow A - b \in \mathbb{R}^{M \times N}. \quad (87)$$

Este mecanismo es la base de gran parte del código vectorizado eficiente en simulación de campos, procesamiento de imágenes y operaciones tensoriales en redes neuronales.

2.3.3. Estructura de memoria y costo computacional

Un ndarray tiene orden de memoria *row-major* (C, por defecto) o *column-major* (Fortran, igual que MATLAB). Recorrer un arreglo en orden contrario al *layout* de memoria multiplica el costo por factores de 2–10 por errores de caché. Para operaciones críticas:

- Verificar `a.flags['C_CONTIGUOUS']` o `a.flags['F_CONTIGUOUS']`.
- Forzar contigüidad con `np.ascontiguousarray(a)` si se invocará BLAS.
- Evitar *views* no contiguos en bucles internos.

La complejidad asintótica de operaciones comunes en una matriz $N \times N$ densa se resume en la [Tabla 9](#).

Tabla 9: Complejidad asintótica y biblioteca subyacente de operaciones de álgebra lineal densa.

Operación	Complejidad	Rutina LAPACK/BLAS
Producto matriz–vector Ax	$\mathcal{O}(N^2)$	DGEMV (BLAS-2)
Producto matriz–matriz AB	$\mathcal{O}(N^3)$	DGEMM (BLAS-3)
Resolución $Ax = b$ (LU)	$\mathcal{O}(N^3)$	DGESV
Cholesky (SPD)	$\mathcal{O}(N^3/3)$	DPOTRF
QR	$\mathcal{O}(2N^3)$	DGEQRF
SVD completa	$\mathcal{O}(N^3)$	DGESDD
Eigenvalores (no simétrico)	$\mathcal{O}(\sim 10N^3)$	DGEEV
FFT 1D longitud N	$\mathcal{O}(N \log N)$	FFTW / pocketfft

2.3.4. Matrices dispersas

En problemas de elementos finitos, mallas grandes, redes o sistemas estructurados, la matriz K es predominantemente cero. Almacenarla densa para $N = 10^5$ requiere $N^2 \cdot 8 \text{ bytes} \approx 80 \text{ GB}$, lo cual es absurdo. Se usan formatos dispersos: LIL para ensamblado, CSR/CSC para resolución, COO para entrada/salida. La resolución se hace con `scipy.sparse.linalg.spsolve` (solvers directos, hasta $\sim 10^6$ incógnitas en RAM razonable) o `cg`, `gmres`, `bicgstab` (iterativos, para problemas mayores).

El Ejemplo 8 de la Semana 1 (conexión viga–columna con FEM CST) ya usaba este patrón: ensamblado en `lil_matrix` y resolución en `csr_matrix` con `spsolve`. Esa elección no es estética; es la diferencia entre un código que corre y un código que agota la memoria.

2.4. Visualización científica

Una figura no es decoración: es un argumento técnico. En contexto doctoral, una figura defectuosa puede invalidar conclusiones correctas. Los criterios mínimos que se exigirán en este curso son:

1. **Unidades en todos los ejes**, sin excepción.
2. **Etiquetas legibles** a tamaño impreso (mínimo 9 pt para revistas, 12 pt para presentaciones).
3. **Mapas de color perceptualmente uniformes**: `viridis`, `plasma`, `inferno`, `cividis`. Nunca `jet` ni `rainbow` para datos cuantitativos (introducen falsos bordes y son inaccesibles para daltónicos).
4. **Aspect ratio físico** cuando se grafican dominios espaciales: `ax.set_aspect('equal')`.
5. **Exportación vectorial**: `.pdf`, `.svg` o `.eps` para figuras de líneas; `.png` a ≥ 300 dpi para mapas densos.

6. **Tipografía consistente** con el documento final (Times, Computer Modern o sans-serif uniformes).

2.4.1. Plantilla mínima “publication-ready”

```

1      import matplotlib.pyplot as plt
2      import matplotlib as mpl
3
4      # ---- Configuración global para figuras de
publicacion ----
5      mpl.rcParams.update({
6          "font.family":      "serif",
7          "font.size":        11,
8          "axes.titlesize":    12,
9          "axes.labelsize":    11,
10         "xtick.labelsize":    10,
11         "ytick.labelsize":    10,
12         "legend.fontsize":    10,
13         "axes.linewidth":     0.8,
14         "lines.linewidth":    1.4,
15         "figure.dpi":         150,
16         "savefig.dpi":        300,
17         "savefig.bbox":       "tight",
18         "savefig.format":     "pdf",
19         "text.usetex":        False,    # True si hay
LaTeX instalado
20         "mathtext.fontset":   "cm",
21     })
22

```

Listing 7: Plantilla de configuración de Matplotlib para figuras científicas.

Esta plantilla se incluye una sola vez al inicio del proyecto. Toda figura generada después hereda el estilo, garantizando consistencia visual sin esfuerzo manual.

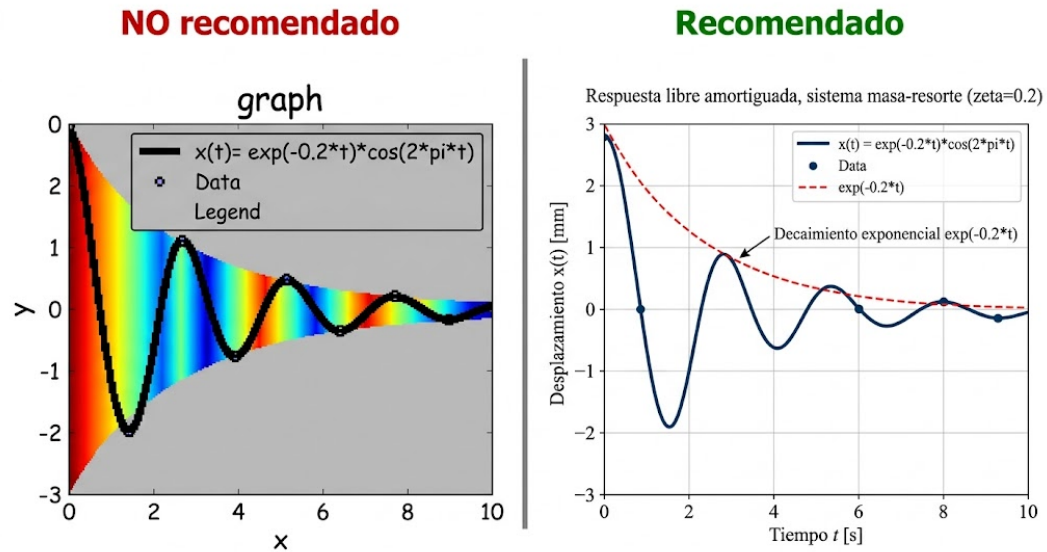


Figura 14: Contraste entre una figura defectuosa (izquierda) y una figura “publication-ready” (derecha) representando los mismos datos. La diferencia no es cosmética: la figura defectuosa oculta la información cuantitativa que el revisor necesita evaluar.

2.4.2. Mapas de color y percepción

El uso de jet (arcoíris) sigue siendo común y sigue siendo incorrecto para datos cuantitativos. Las razones son:

- No es perceptualmente uniforme: pasos iguales en el dato producen saltos visuales de magnitud distinta.
- Introduce bandas falsas en transiciones amarillo–cian que el ojo interpreta como contornos físicos inexistentes.
- No es robusto a impresión en escala de grises.
- Inaccesible para parte de la población con visión cromática reducida.

La familia viridis (y derivados plasma, inferno, magma, cividis) fue diseñada para corregir estos cuatro defectos simultáneamente. En este curso, jet está prohibido en entregables.

2.5. Scripts reproducibles y arquitectura de proyectos

Un *script* reproducible es aquel que, ejecutado por un tercero con los mismos datos de entrada y los mismos parámetros, produce los mismos resultados. La estructura mínima recomendada para un proyecto doctoral es:

```

1      mi_proyecto/
2      |-- README.md          # objetivo,
                                instalacion, ejecucion
    
```

```

3         |-- LICENSE                                # licencia explicita (
MIT, BSD, GPL, CC-BY)
4         |-- environment.yml                        # conda env o
requirements.txt
5         |-- .gitignore                            # excluye datos crudos
pesados, __pycache__
6         |-- src/                                  # codigo fuente Python
/MATLAB
7         |    |-- __init__.py
8         |    |-- modelo.py
9         |    |-- solver.py
10        |    `-- utils.py
11        |-- scripts/                              # entradas de linea de
comandos
12        |    |-- run_experiment.py
13        |    `-- make_figures.py
14        |-- config/                               # parametros por
experimento (yaml/json)
15        |    |-- exp_001.yaml
16        |    `-- exp_002.yaml
17        |-- data/
18        |    |-- raw/                             # datos crudos (read-
only, idealmente con DOI)
19        |    `-- processed/                       # derivados,
regenerables
20        |-- results/                             # salidas: figuras,
tablas, logs
21        |-- tests/                               # pruebas unitarias (
pytest)
22        `-- notebooks/                           # exploracion, NUNCA
es la fuente de verdad
23

```

Listing 8: Estructura mínima de directorio para un proyecto doctoral reproducible.

Esta jerarquía es la *convención de facto* en proyectos científicos modernos (similar a Cookiecutter Data Science) y se enseña porque elimina ambigüedades clásicas: dónde están los datos, dónde el código, dónde los resultados, qué se puede borrar sin pérdida.

2.5.1. Script ejecutable con argparse y registro de parámetros

```

1         """
2         run_diffusion.py

```



```
3      Simulacion 1D de difusion termica con parametros
4      configurables
5      desde linea de comandos y registro automatico de
6      configuracion.
7
8      Uso:
9      python run_diffusion.py --alpha 1e-4 --N 151 --r
10     0.45 \
11     --tsteps 800 --seed 42 \
12     --outdir results/exp_001
13     """
14
15     import argparse
16     import json
17     import os
18     import sys
19     import time
20     from pathlib import Path
21
22     import numpy as np
23     import matplotlib.pyplot as plt
24
25
26     def parse_args():
27         p = argparse.ArgumentParser(description="Difusion 1D
28         explícita.")
29         p.add_argument("--L", type=float, default=1.0,
30         help="Longitud [m]")
31         p.add_argument("--alpha", type=float, default=1e-4,
32         help="Difusividad [m^2/s]")
33         p.add_argument("--N", type=int, default=151,
34         help="Nodos espaciales")
35         p.add_argument("--r", type=float, default=0.45,
36         help="Numero de Fourier <= 0.5")
37         p.add_argument("--tsteps", type=int, default=800,
38         help="Pasos temporales")
39         p.add_argument("--T0", type=float, default=25.0,
40         help="Temperatura ambiente [C]")
41         p.add_argument("--A", type=float, default=80.0,
42         help="Amplitud gaussiana [C]")
43         p.add_argument("--seed", type=int, default=42,
44         help="Semilla aleatoria")
```

```

32         p.add_argument("--outdir", type=str, default="
results/run_default")
33         return p.parse_args()
34
35
36     def main():
37         args = parse_args()
38         outdir = Path(args.outdir)
39         outdir.mkdir(parents=True, exist_ok=True)
40
41         # ---- Snapshot de configuracion ----
42         cfg = vars(args).copy()
43         cfg["timestamp"] = time.strftime("%Y-%m-%dT%H:%M:%
S")
44
45         cfg["numpy"] = np.__version__
46         cfg["python"] = sys.version.split()[0]
47         with open(outdir / "config.json", "w") as f:
48             json.dump(cfg, f, indent=2)
49
50         # ---- Verificacion de estabilidad ----
51         if args.r > 0.5:
52             sys.exit(f"ERROR: r = {args.r} > 0.5; el esquema
53             explícito sera inestable.")
54
55         # ---- Simulacion ----
56         np.random.seed(args.seed)
57         x = np.linspace(0.0, args.L, args.N)
58         dx = x[1] - x[0]
59         dt = args.r * dx**2 / args.alpha
60
61         T = args.T0 + args.A * np.exp(-((x - 0.5*args.L)**2)
62         / (2*0.06**2))
63         T[0] = T[-1] = args.T0
64
65         snapshots, times = [T.copy()], [0.0]
66         for n in range(1, args.tsteps + 1):
67             T[1:-1] = T[1:-1] + args.r * (T[2:] - 2*T[1:-1] + T
68            [:-2])
69             T[0] = T[-1] = args.T0
70             if n in (50, 150, 300, args.tsteps):
71                 snapshots.append(T.copy())
72                 times.append(n * dt)

```

```

69
70         # ---- Salida tabular ----
71         np.savez(outdir / "fields.npz", x=x, T=np.stack(
snapshots), t=np.array(times))
72
73         # ---- Figura ----
74         fig, ax = plt.subplots(figsize=(7, 4))
75         for Tk, tk in zip(snapshots, times):
76             ax.plot(x, Tk, label=f"t = {tk:6.2f} s")
77             ax.set_xlabel("x [m]")
78             ax.set_ylabel("T [degC]")
79             ax.set_title("Conduccion transitoria 1D")
80             ax.grid(True, alpha=0.3)
81             ax.legend()
82             fig.tight_layout()
83             fig.savefig(outdir / "perfiles.pdf")
84
85         print(f"[OK] resultados en {outdir.resolve()}")
86
87
88         if __name__ == "__main__":
89             main()
90

```

Listing 9: Plantilla de script reproducible con argparse y registro de parámetros.

Tres elementos distinguen este script de un *notebook* desechable:

1. Toda variable de control es un argumento explícito; nada está *hardcoded*.
2. Se guarda un `config.json` con la versión de Python, NumPy, semilla y *timestamp*; esto permite reconstruir el experimento meses después.
3. Verificación de estabilidad ($r \leq 0.5$) integrada como condición de salida con código de error, no como advertencia ignorable.

2.6. Notebooks científicos: cuándo sí y cuándo no

Jupyter (y su variante en la nube Google Colab) es excelente para exploración, didáctica, comunicación visual y revisión interactiva. Es *malo* como fuente de verdad de un proyecto serio. La regla operativa es:

- **Usar notebooks para:** exploración inicial, prototipos de figuras, tutoriales reproducibles compartidos con coautores, material docente.
- **No usar notebooks para:** ejecutar pipelines de producción, almacenar la versión canónica de funciones reutilizables, controlar versiones de cálculos críticos.

Los problemas conocidos de los notebooks son: orden de ejecución arbitrario (un cuaderno se ejecuta de arriba abajo sólo si el usuario lo decide), *diffs* de Git ilegibles por el JSON con salidas embebidas, dependencias ocultas en el *kernel* de la sesión y dificultad para parametrizar. Herramientas que mitigan estos defectos: *nbstripout* (limpia salidas antes de *commit*), *papermill* (parametriza notebooks como si fueran scripts), *jupyter-text* (sincroniza con `.py` versionables).

2.6.1. Google Colab: rol específico en el curso

Colab se usará como entorno de *demostración pública y reproducible* de los ejemplos del curso porque: (i) no requiere instalación local, (ii) la URL es portable, (iii) ya tiene NumPy, SciPy, Matplotlib y PyTorch preinstalados, (iv) ofrece GPU gratuita para entrenamientos cortos. Los entregables formales, sin embargo, serán scripts versionados en Git, no notebooks de Colab.

2.7. Librerías científicas fundamentales

El ecosistema Python científico se construye sobre cinco librerías base, sobre las cuales se montan especializaciones por dominio.

Tabla 10: Librerías científicas usadas durante el curso, con su rol y dependencia subyacente.

Librería	Rol principal	Subyacente / notas
numpy	Arreglos multidimensionales, álgebra lineal básica.	BLAS/LAPACK.
scipy	Optimización, integración, ODE/PDE, dispersas, estadística.	FORTTRAN legado (quadpack, odepack).
matplotlib	Visualización 2D y 3D estática.	Backend Agg/Cairo.
pandas	Tablas indexadas, series temporales, I/O.	NumPy + C++.
sympy	Álgebra simbólica, derivadas analíticas, simplificación.	Python puro.
scikit-learn	Aprendizaje automático clásico.	NumPy/SciPy.
scikit-image	Procesamiento de imágenes científicas.	NumPy.
statsmodels	Inferencia estadística clásica.	NumPy/SciPy.
torch / tensorflow	Redes neuronales, diferenciación automática.	CUDA/cuDNN.
pyvista / vtk	Visualización 3D y mallas.	VTK.
numba / cython	Aceleración JIT/compilada.	LLVM, C.

2.7.1. SymPy: cuándo lo simbólico ahorra tiempo

SymPy permite obtener derivadas analíticas, simplificar expresiones y generar código numérico desde una formulación matemática. Su valor doctoral aparece cuando se necesita el Jacobiano $J = \partial f / \partial x$ para propagación de incertidumbre (Tabla 9) o para esquemas de Newton sin riesgo de error humano en la derivada.

```

1      import sympy as sp
2
3      I, R, h, As, t, m, cp, Tinf = sp.symbols("I R h As t
4      m cp Tinf", positive=True)
5
6      # Modelo concentrado de calentamiento (Semana 1,
7      Ejemplo 3)
8      tau = m*cp / (h*As)
9      Tss = Tinf + I**2*R / (h*As)
10     T = Tinf + I**2*R/(h*As) * (1 - sp.exp(-t/tau))
11
12     # Jacobiano respecto a (I, R, h)
13     J = sp.Matrix([T]).jacobian([I, R, h])
14     J_simpl = sp.simplify(J)
15     sp.pprint(J_simpl)
16
17     # Exportar a funcion numerica vectorizada
18     T_num = sp.lambdify((I, R, h, As, t, m, cp, Tinf), T
19     , modules="numpy")
20     J_num = sp.lambdify((I, R, h, As, t, m, cp, Tinf),
21     J_simpl, modules="numpy")
22
23     # Evaluacion numerica
24     print(T_num(400.0, 1e-4, 25.0, 0.5, 60.0, 2.0,
25     900.0, 25.0))

```

Listing 10: Derivación simbólica del Jacobiano y exportación a función numérica.

Este patrón —derivar simbólicamente, *lambdificar* para evaluación numérica— es la forma profesional de evitar errores algebraicos. Pertenece al mismo paradigma de la diferenciación automática (autograd, `jax`, `torch.autograd`) que se verá en la Semana 14.

2.8. Automatización de cálculos

Una vez que un experimento computacional se ha estabilizado, debe poder ejecutarse sin intervención manual. Las técnicas escalan desde `Makefile` (suficiente para proyectos

pequeños) hasta orquestadores como Snakemake, Nextflow o Apache Airflow (necesarios cuando hay decenas de pasos interdependientes).

2.8.1. Makefile mínimo para un proyecto científico

```

1      # Makefile - Proyecto Difusion 1D
2      PYTHON := python
3      SRC     := scripts/run_diffusion.py
4      RESULTS := results
5
6      EXPS := exp_001 exp_002 exp_003
7
8      .PHONY: all clean figures
9
10     all: $(EXPS:%=$(RESULTS)/%/perfiles.pdf) figures
11
12     $(RESULTS)/exp_001/perfiles.pdf:
13     $(PYTHON) $(SRC) --alpha 1e-4 --r 0.45 --outdir $(
RESULTS)/exp_001
14
15     $(RESULTS)/exp_002/perfiles.pdf:
16     $(PYTHON) $(SRC) --alpha 5e-5 --r 0.40 --outdir $(
RESULTS)/exp_002
17
18     $(RESULTS)/exp_003/perfiles.pdf:
19     $(PYTHON) $(SRC) --alpha 2e-4 --r 0.45 --outdir $(
RESULTS)/exp_003
20
21     figures: $(EXPS:%=$(RESULTS)/%/perfiles.pdf)
22     $(PYTHON) scripts/make_figures.py --indirs $(EXPS:%=
$(RESULTS)/%)
23
24     clean:
25     rm -rf $(RESULTS)/exp_*
26

```

Listing 11: Makefile para automatizar tres experimentos con dependencias declaradas.

Con un solo comando `make all` se ejecutan los tres experimentos y se generan las figuras consolidadas. `make` sólo re-ejecuta lo que cambió: si se modifica `exp_002.yaml`, sólo se regenera `exp_002`. Esto evita el clásico “ejecuté todo de nuevo por si acaso” que consume horas y oculta errores.

2.9. Buenas prácticas computacionales

Estas prácticas son las que diferencian, en evaluación doctoral, un trabajo robusto de uno frágil:

1. **Control de versiones (Git):** todo proyecto bajo Git desde el primer día. *Commits* pequeños, mensajes descriptivos. Cualquier resultado reportado debe ser trazable a un commit específico.
2. **Entorno aislado:** conda o venv con `environment.yml/requirements.txt` que fije versiones (`numpy==1.26.4`, no `numpy>=1.26`).
3. **Semilla aleatoria explícita:** `np.random.seed(SEED)` al inicio de cualquier proceso estocástico. Sin semilla no hay reproducibilidad.
4. **Aserciones y verificación:** `assert` en condiciones invariantes (estabilidad, conservación de masa/energía, dimensiones de matrices). Estas aserciones son auto-documentación.
5. **Pruebas unitarias:** `pytest` sobre funciones críticas. Una función numérica sin al menos una prueba contra solución analítica o caso límite es código no validado.
6. **Estilo de código:** PEP 8 en Python, formateador automático (`black`, `ruff`). MATLAB tiene `checkcode`. No es estética: el código consistente reduce la carga cognitiva del revisor.
7. **Documentación inline:** *docstrings* en cada función, con descripción, parámetros, retornos, unidades y referencia bibliográfica si aplica.
8. **Logging frente a print:** usar logging con niveles `DEBUG/INFO/WARNING/ERROR`. Un `print` se ignora; un `WARNING` aparece en `stderr` y es trazable.

CICLO DE VIDA DE UN PROYECTO CIENTIFICO REPRODUCIBLE

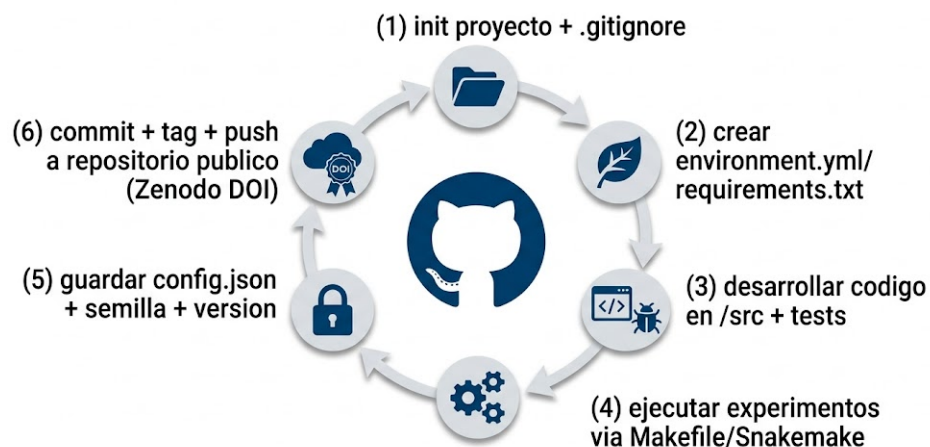


Figura 15: Ciclo de vida de un proyecto científico reproducible. Cada etapa produce un artefacto versionable y trazable, y ninguna depende de memoria humana ni de configuraciones implícitas.

2.9.1. Mini-guía de Git para investigación

```

1      # Inicializacion
2      git init
3      git add README.md environment.yml .gitignore
4      git commit -m "init: estructura inicial del proyecto"
5
6      # Flujo diario
7      git status                    # que ha cambiado
8      git diff src/modelo.py        # ver cambios concretos
9      git add src/modelo.py
10     git commit -m "fix: corregir signo en termino de
11     conveccion"
12
13     # Marcar version reproducible
14     git tag -a v1.0-paper -m "Version usada en el
15     manuscrito enviado a OLE"
16     git push --tags origin main
17
18     # Reproducir un resultado pasado
19     git checkout v1.0-paper        # vuelve al estado
20     exacto del commit

```

Listing 12: Comandos esenciales de Git para investigación reproducible.

Un proyecto sin Git no es defectuoso: es no-investigación. La pregunta “¿qué versión del código produjo esta figura?” debe tener una respuesta exacta (un hash de commit de 40 caracteres), no aproximada.

2.10. Ejemplo 1: Álgebra lineal aplicada — SVD y reducción de dimensionalidad en datos experimentales

2.10.1. Planteamiento

Considere m mediciones experimentales, cada una representada por un vector de n variables: $X \in \mathbb{R}^{m \times n}$. Cuando n es grande (espectros, imágenes, series temporales sincronizadas), conviene reducir la dimensionalidad preservando la varianza. La herramienta es la descomposición en valores singulares (SVD):

$$X = U \Sigma V^T, \quad U \in \mathbb{R}^{m \times m}, \Sigma \in \mathbb{R}^{m \times n}, V \in \mathbb{R}^{n \times n}. \quad (88)$$

Los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ cuantifican cuánta varianza captura cada componente. El análisis de componentes principales (PCA) es la SVD aplicada sobre X centrado.

2.10.2. Conexión doctoral

Este procedimiento aparece directamente en metrología óptica: en ESPI, fringe projection y DIC, los campos de fase o desplazamiento adquiridos en múltiples instantes se organizan en matrices que admiten descomposición SVD para separar señal de ruido (técnica DMD/POD). También subyace la calibración estereoscópica cuando se resuelve $Ax \approx b$ por mínimos cuadrados con pseudoinversa $A^+ = V\Sigma^+U^\top$.

2.10.3. Código en Python

```

1      """
2      Ejemplo 1: SVD aplicada a un conjunto de mediciones
      sintéticas.
3      Se construyen m=200 senales, cada una en n=64 puntos
      , generadas
4      como combinacion lineal de 3 modos fundamentales mas
      ruido gaussiano.
5      El objetivo es recuperar el rango efectivo de los
      datos.
6      """
7      import numpy as np
8      import matplotlib.pyplot as plt
9
10     rng = np.random.default_rng(2026)
11     m, n = 200, 64
12     t = np.linspace(0, 1, n)
13
14     # Modos fundamentales: rango intrinseco r=3
15     phi1 = np.sin(2*np.pi*1*t)
16     phi2 = np.sin(2*np.pi*3*t) * 0.6
17     phi3 = np.cos(2*np.pi*5*t) * 0.3
18     Phi = np.stack([phi1, phi2, phi3], axis=1)      #
      n x 3
19
20     # Coeficientes aleatorios para cada medicion
21     C = rng.normal(size=(m, 3))
22
23     # Datos sin ruido + ruido gaussiano

```

```

24         X_clean = C @ Phi.T                                     #
    m x n
25         sigma_noise = 0.15
26         X = X_clean + sigma_noise * rng.normal(size=(m, n))
27
28         # Centrado (paso obligatorio para PCA)
29         X_mean = X.mean(axis=0)
30         Xc = X - X_mean
31
32         # SVD
33         U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
34
35         # Varianza explicada acumulada
36         var_exp = S**2 / np.sum(S**2)
37         var_cum = np.cumsum(var_exp)
38
39         # Reconstruccion truncada con k=3 componentes
40         k = 3
41         X_rec = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :] +
X_mean
42
43         err_rec = np.linalg.norm(X - X_rec) / np.linalg.norm
(X)
44         print(f"sigma_1..sigma_5 = {S[:5]}")
45         print(f"Varianza explicada k=3: {var_cum[2]*100:.2f
}%")
46         print(f"Error relativo de reconstruccion: {err_rec
:.4f}")
47
48         # ---- Visualizacion ----
49         fig, axes = plt.subplots(1, 3, figsize=(13, 4))
50
51         ax = axes[0]
52         ax.semilogy(np.arange(1, len(S)+1), S, 'o-', ms=4)
53         ax.axvline(k+0.5, color='r', ls='--', lw=0.8, label=
f'corte k={k}')
54         ax.set_xlabel('indice del modo')
55         ax.set_ylabel(r'$\sigma_i$')
56         ax.set_title('Valores singulares')
57         ax.grid(True, which='both', alpha=0.3); ax.legend()
58
59         ax = axes[1]

```

```

60     ax.plot(np.arange(1, len(S)+1), var_cum*100, 'o-',
ms=4)
61     ax.axhline(99, color='r', ls='--', lw=0.8, label='
99%')
62     ax.set_xlabel('numero de componentes retenidas')
63     ax.set_ylabel('varianza acumulada [%]')
64     ax.set_title('Curva de varianza')
65     ax.grid(True, alpha=0.3); ax.legend()
66
67     ax = axes[2]
68     idx = 0
69     ax.plot(t, X[idx], 'k.', ms=3, label='medicion
ruidosa')
70     ax.plot(t, X_clean[idx], 'b-', lw=1.2, label='senal
real')
71     ax.plot(t, X_rec[idx], 'r--', lw=1.2, label=f'recon.
SVD k={k}')
72     ax.set_xlabel('t'); ax.set_ylabel('amplitud')
73     ax.set_title(f'Reconstruccion medicion #{idx}')
74     ax.grid(True, alpha=0.3); ax.legend()
75
76     plt.tight_layout()
77     plt.savefig("Figuras/fig_s2_svd_pca.pdf")
78     plt.show()
79
    
```

Listing 13: SVD para reducción de dimensionalidad sobre datos sintéticos con ruido.

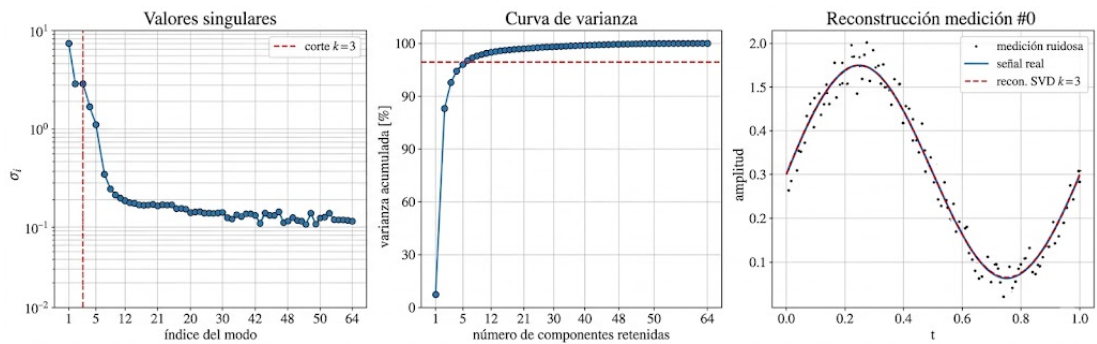


Figura 16: Descomposición en valores singulares de un conjunto sintético de $m = 200$ mediciones en $n = 64$ puntos, con rango intrínseco $r = 3$ y ruido gaussiano $\sigma = 0.15$. Izquierda: espectro de valores singulares, donde los tres primeros modos se separan nítidamente del ruido. Centro: la varianza acumulada alcanza $> 99\%$ con $k = 3$ componentes. Derecha: la reconstrucción truncada filtra el ruido sin sacrificar la señal subyacente.

2.10.4. Lectura metodológica

La SVD ofrece tres cosas simultáneamente: una base ortogonal óptima para representar los datos, un criterio cuantitativo (la curva de valores singulares) para decidir cuántos modos conservar y un mecanismo de *denoising* controlado. La existencia de un “codo” en el espectro de σ_i es una herramienta de diagnóstico: si no hay codo, los datos no tienen estructura de baja dimensionalidad y la reducción es ilegítima.

2.11. Ejemplo 2: Integración de ODE con `scipy.integrate.solve_ivp`

2.11.1. Planteamiento

La integración numérica de ecuaciones diferenciales ordinarias es operación cotidiana en ingeniería. SciPy provee `solve_ivp`, que implementa Runge–Kutta–Fehlberg (RK45), Dormand–Prince, métodos de paso variable y solucionadores rígidos (BDF, Radau, LSODA). Se ilustra con el oscilador de Van der Pol, sistema con dinámica rígida en su régimen relajación:

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0. \quad (89)$$

Para $\mu \gg 1$ el sistema se vuelve rígido y un método explícito de paso fijo (Euler) requiere pasos absurdamente pequeños o explota.

2.11.2. Código en Python

```

1      """
2      Ejemplo 2: integracion del oscilador de Van der Pol.
3      Se comparan tres metodos de solve_ivp: RK45 (
4      explicito no rigido),
5      LSODA (auto-detecta rigidez) y BDF (implicito para
6      rigidez).
7      Se reporta el numero de evaluaciones de f y el
8      tiempo de pared.
9      """
10     import time
11     import numpy as np
12     import matplotlib.pyplot as plt
13     from scipy.integrate import solve_ivp
14
15     def van_der_pol(t, y, mu):
16         x, v = y
17         return [v, mu*(1 - x**2)*v - x]
```

```
17
18     t_span = (0.0, 100.0)
19     y0 = [2.0, 0.0]
20     mu_values = [1.0, 10.0, 100.0]    # tres regimenes de
    rigidez
21     metodos = ["RK45", "LSODA", "BDF"]
22
23     print(f"{'mu':>6} {'metodo':>8} {'nfev':>8} {'njev'
':>6} {'tiempo[s]':>10} {'status':>6}")
24     fig, axes = plt.subplots(1, 3, figsize=(13, 4))
25     for ax, mu in zip(axes, mu_values):
26         for met in metodos:
27             t0 = time.perf_counter()
28             sol = solve_ivp(van_der_pol, t_span, y0, args=(mu,),
29                             method=met, rtol=1e-6, atol=1e-9,
30                             dense_output=True)
31             dt = time.perf_counter() - t0
32             njev = getattr(sol, "njev", 0)
33             print(f"{'mu':6.0f} {'met':>8} {'sol.nfev':8d} {'njev':6d} {
dt:10.4f} {'sol.status':>6d}")
34
35             tt = np.linspace(*t_span, 4000)
36             yy = sol.sol(tt)
37             ax.plot(tt, yy[0], lw=1.0, label=met)
38
39             ax.set_title(f"mu = {mu:.0f}")
40             ax.set_xlabel("t"); ax.set_ylabel("x(t)")
41             ax.grid(True, alpha=0.3); ax.legend(fontsize=8)
42
43             plt.tight_layout()
44             plt.savefig("Figuras/fig_s2_van_der_pol.pdf")
45             plt.show()
46
```

Listing 14: Comparación de métodos en `solve_ivp` para el oscilador de Van der Pol.

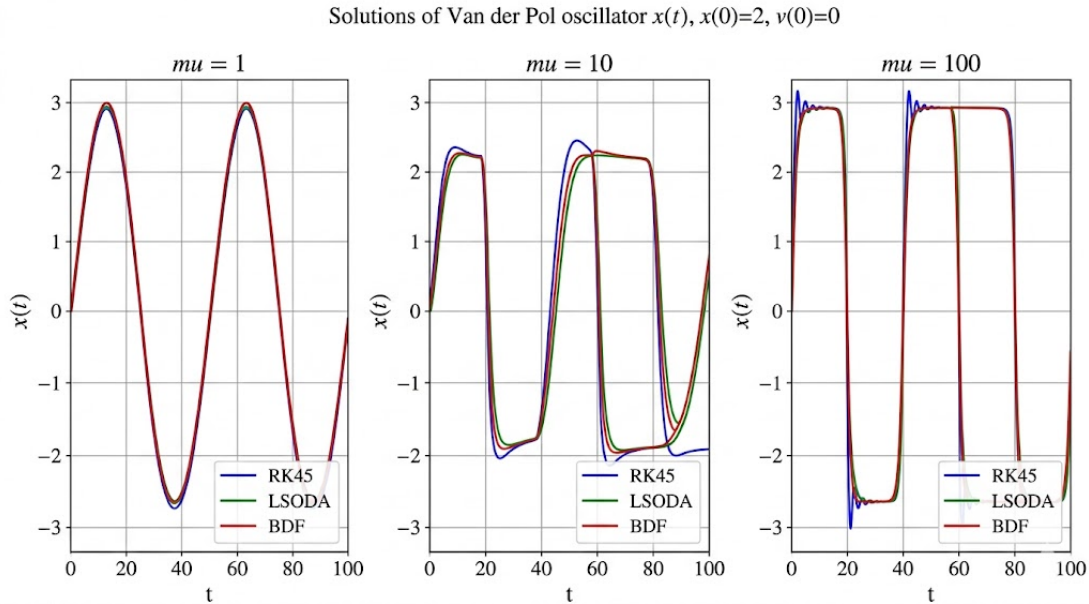


Figura 17: Oscilador de Van der Pol integrado con tres métodos de `solve_ivp`. Para $\mu = 1$ todos los métodos coinciden a costo similar; para $\mu = 100$ el sistema es rígido y los métodos implícitos (BDF, LSODA) requieren órdenes de magnitud menos evaluaciones que RK45.

2.11.3. Lectura metodológica

El número de evaluaciones `nfev` reportado por `solve_ivp` es un diagnóstico directo de costo. Si un método explícito requiere $> 10^5$ evaluaciones donde un implícito requiere 10^3 , el sistema es rígido. Esta detección no debe hacerse por intuición sino por la diferencia observada de costo. La regla doctoral es: si `nfev` crece más rápido que linealmente con el tiempo simulado, hay rigidez y el método está mal elegido.

2.12. Ejemplo 3: pandas para manejo de datos experimentales

2.12.1. Planteamiento

Los datos experimentales rara vez llegan en formato matricial limpio. Suelen ser archivos `.csv` con encabezados inconsistentes, valores faltantes, unidades implícitas y columnas mezcladas. `pandas` es la herramienta estándar para limpiarlos, indexarlos y resumirlos antes del análisis numérico propiamente dicho.

```

1      """
2      Ejemplo 3: pipeline pandas sobre dataset sintetico
3      de mediciones
4      de temperatura en N=10 termopares durante 1 hora,
      con perdidas
      aleatorias (NaN) y outliers.
    
```

```

5         """
6         import numpy as np
7         import pandas as pd
8         import matplotlib.pyplot as plt
9
10        # ---- Generacion de dataset sintetico ----
11        rng = np.random.default_rng(123)
12        n_pts = 3600                                # 1
13        muestra/s durante 1 hora
14        t = pd.date_range("2026-05-21 08:00", periods=n_pts,
15        freq="1s")
16        n_tc = 10
17        T_base = 25 + 30*np.exp(-np.linspace(0, 3, n_pts))
18        # transitorio
19        data = T_base[:, None] + rng.normal(0, 0.4, size=(
20        n_pts, n_tc))
21
22        # Inyectar NaN (perdidas de senal) y outliers (
23        chispas EMI)
24        mask_nan = rng.random((n_pts, n_tc)) < 0.005
25        data[mask_nan] = np.nan
26        mask_out = rng.random((n_pts, n_tc)) < 0.001
27        data[mask_out] += rng.normal(0, 50, mask_out.sum())
28
29        df = pd.DataFrame(data,
30        index=t,
31        columns=[f"TC_{i:02d}" for i in range(n_tc)])
32
33        # ---- Diagnostico ----
34        print("Shape:", df.shape)
35        print("NaN por columna:", df.isna().sum().to_dict())
36        print("Resumen descriptivo:")
37        print(df.describe().T[["mean", "std", "min", "max"
38        ]])
39
40        # ---- Limpieza ----
41        # 1) eliminar outliers via mediana movil + filtro de
42        Hampel
43        median = df.rolling("60s", center=True).median()
44        mad     = (df - median).abs().rolling("60s", center=
45        True).median()
46        score   = (df - median).abs() / (1.4826*mad + 1e-9)

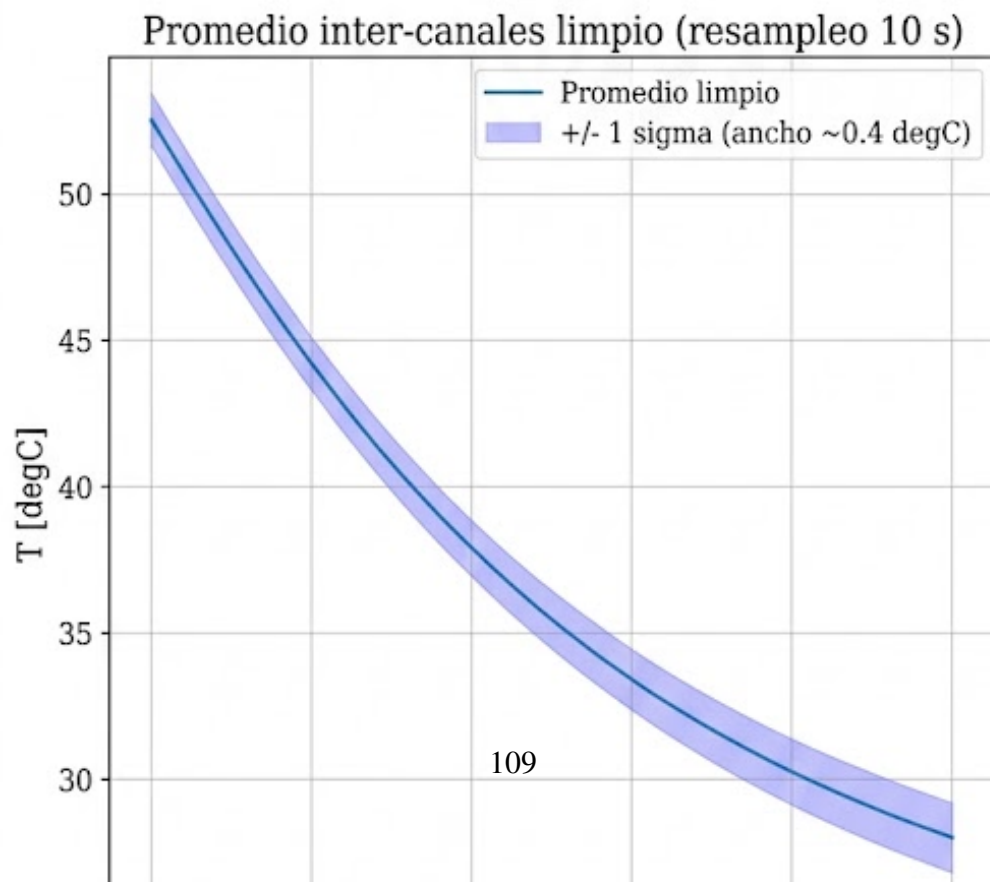
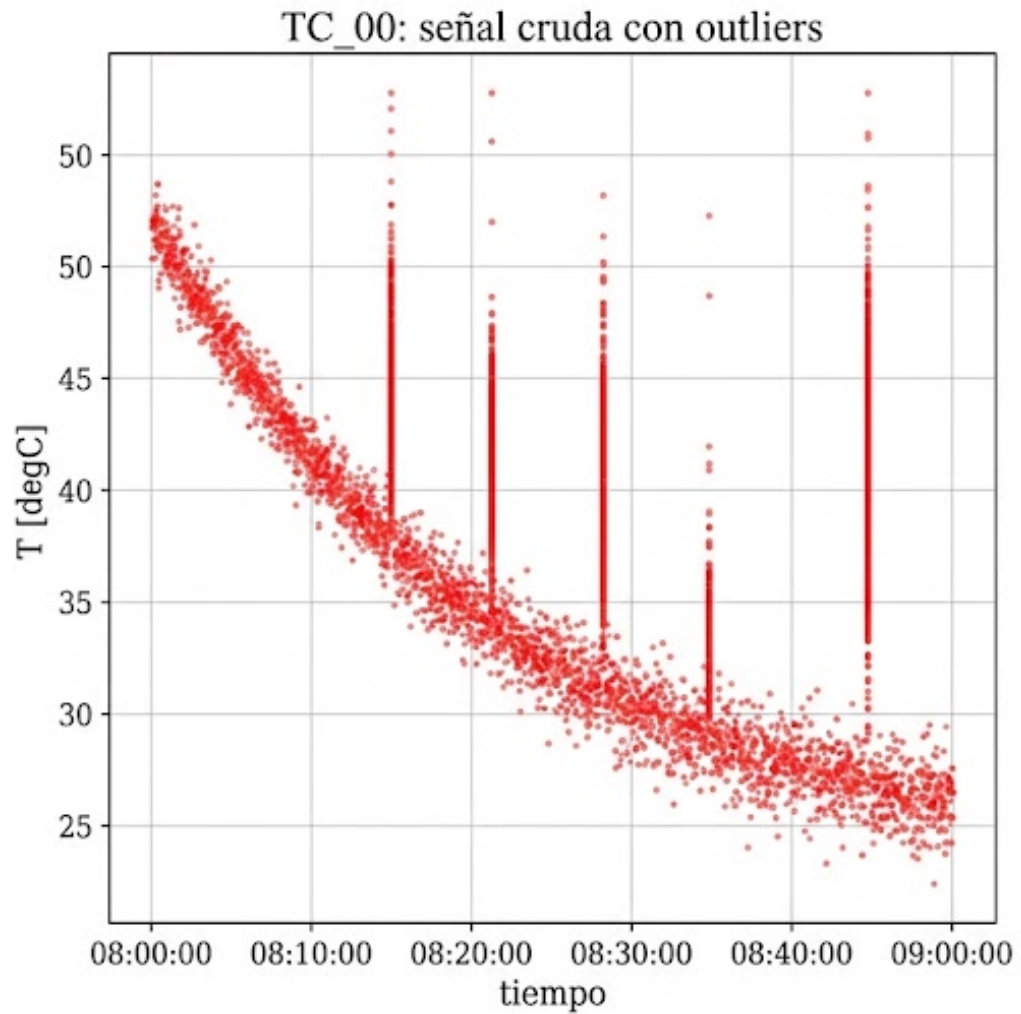
```

```

39         df_clean = df.where(score < 3.5)
40
41         # 2) imputar NaN por interpolacion temporal
42         df_clean = df_clean.interpolate(method="time", limit
=10)
43
44         # 3) resampleo a 10 s con promedio
45         df_10s = df_clean.resample("10s").mean()
46
47         # ---- Visualizacion ----
48         fig, axes = plt.subplots(2, 1, figsize=(9, 6),
sharex=True)
49         axes[0].plot(df.index, df["TC_00"], 'r.', ms=2,
alpha=0.5, label="crudo")
50         axes[0].set_ylabel("T [degC]"); axes[0].legend()
51         axes[0].set_title("TC_00: senal cruda con outliers")
52         axes[1].plot(df_10s.index, df_10s.mean(axis=1), 'b-'
, lw=1.0, label="media 10 canales")
53         axes[1].fill_between(df_10s.index,
df_10s.mean(axis=1) - df_10s.std(axis=1),
54         df_10s.mean(axis=1) + df_10s.std(axis=1),
55         alpha=0.25, label="+/- 1 sigma entre canales")
56         axes[1].set_ylabel("T [degC]"); axes[1].set_xlabel("
tiempo")
57
58         axes[1].legend(); axes[1].set_title("Promedio inter-
canales limpio (resampleo 10 s)")
59
60         plt.tight_layout()
61         plt.savefig("Figuras/fig_s2_pandas_pipeline.pdf")
62         plt.show()
63

```

Listing 15: Pipeline mínimo de limpieza y análisis con pandas.



2.12.2. Lectura metodológica

Tres operaciones no triviales aparecen en este ejemplo: detección de outliers con filtro de Hampel ($\text{score} > 3.5 \text{ MAD}$), imputación de NaN con interpolación temporal, y resampleo con `resample`. Estas operaciones son irreversibles: una vez aplicadas, el dato original ya no es recuperable salvo que se conserve por separado. La regla doctoral es: *nunca sobrescribir los datos crudos*; conservarlos en `data/raw/` como solo lectura y producir derivados en `data/processed/`.

2.13. Ejemplo 4: Diferenciación automática con `jax/autograd`

2.13.1. Planteamiento

La diferenciación automática (AD) calcula gradientes exactos hasta precisión de máquina, sin pasar por diferencias finitas (ruidosas) ni derivación simbólica (frágil ante simplificaciones). Es el motor detrás de PyTorch, TensorFlow, JAX y PINNs (que verán en la Semana 14).

Considere el ajuste no lineal de un modelo $f(x; \theta) = \theta_0 e^{-\theta_1 x} \cos(\theta_2 x + \theta_3)$ a datos ruidosos $\{(x_i, y_i)\}$. El gradiente de la pérdida cuadrática

$$\mathcal{L}(\theta) = \frac{1}{2} \sum_i (f(x_i; \theta) - y_i)^2 \quad (90)$$

respecto a $\theta \in \mathbb{R}^4$ se obtiene automáticamente con `jax.grad`, sin escribir derivadas a mano.

```

1      """
2      Ejemplo 4: ajuste no lineal con JAX y diferenciacion
      automatica.
3      Modelo: f(x;theta) = theta0 * exp(-theta1 * x) * cos
      (theta2*x + theta3)
4      """
5      import jax
6      import jax.numpy as jnp
7      import numpy as np
8      import matplotlib.pyplot as plt
9      from scipy.optimize import minimize
10
11     rng = np.random.default_rng(7)
12     theta_true = jnp.array([2.0, 0.3, 4.0, 0.5])
13
14     x = jnp.linspace(0, 6, 80)
15 
```

```

16         def f(theta, x):
17             return theta[0] * jnp.exp(-theta[1]*x) * jnp.cos(
theta[2]*x + theta[3])
18
19         y_clean = f(theta_true, x)
20         y = y_clean + 0.1 * rng.normal(size=len(x))
21
22         # Funcion de perdida y su gradiente automatico
23         def loss(theta, x, y):
24             return 0.5 * jnp.sum((f(theta, x) - y)**2)
25
26         grad_loss = jax.grad(loss)
27
28         # Wrappers para scipy.optimize
29         def loss_np(theta):
30             return float(loss(jnp.array(theta), x, jnp.array(y))
)
31
32         def grad_np(theta):
33             return np.array(grad_loss(jnp.array(theta), x, jnp.
array(y)))
34
35         theta0 = np.array([1.0, 0.1, 3.0, 0.0])
36         res = minimize(loss_np, theta0, jac=grad_np, method=
"L-BFGS-B",
37             options={"gtol": 1e-9, "disp": True})
38
39         print("theta verdadero: ", np.asarray(theta_true))
40         print("theta estimado: ", res.x)
41         print("error parametrico:", np.linalg.norm(res.x -
np.asarray(theta_true)))
42
43         plt.figure(figsize=(8, 4))
44         plt.plot(x, y, 'k.', ms=4, label='datos ruidosos')
45         plt.plot(x, np.asarray(f(theta_true, x)), 'b-', lw
=1.0, label='modelo real')
46         plt.plot(x, np.asarray(f(jnp.array(res.x), x)), 'r--
', lw=1.2, label='ajuste L-BFGS-B')
47         plt.xlabel('x'); plt.ylabel('f(x)')
48         plt.title('Ajuste no lineal con gradientes via AD (
JAX)')
49
50         plt.grid(True, alpha=0.3); plt.legend()
51         plt.tight_layout()

```

```

50 plt.savefig("Figuras/fig_s2_jax_fit.pdf"); plt.show
51 ()

```

Listing 16: Diferenciación automática con JAX para ajuste no lineal.

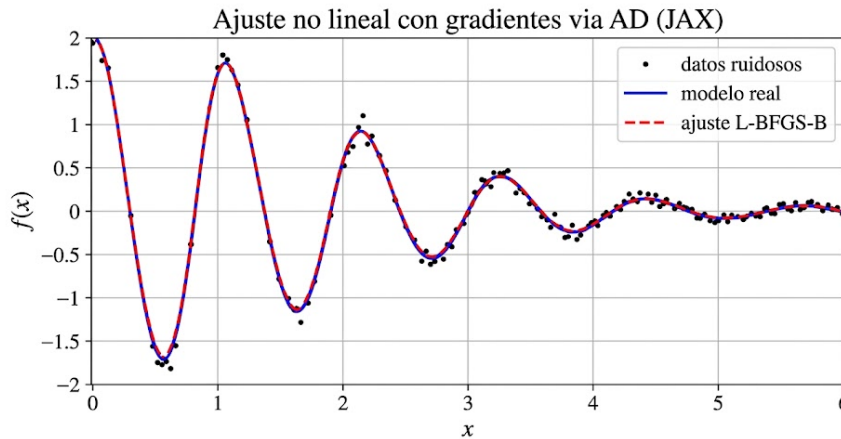


Figura 19: Ajuste no lineal por mínimos cuadrados con L-BFGS-B usando gradientes obtenidos por diferenciación automática (JAX). La curva ajustada (rojo) recupera la curva real (azul) hasta dentro del ruido. Sin AD, el usuario debería derivar a mano cuatro expresiones parciales y arriesgarse a errores de signo.

2.13.2. Lectura metodológica

La diferenciación automática es *distinta* de la derivación simbólica: AD computa el gradiente del programa exacto, sin construir ni simplificar la expresión algebraica. Es lineal en el costo (forward mode) o casi-constante (reverse mode/backprop) respecto a la dimensión de parámetros. En problemas con $\mathcal{O}(10^6)$ parámetros (redes neuronales) sería inviable hacerlo simbólicamente. Para PINNs, AD permite calcular $\partial u / \partial t$ y $\partial^2 u / \partial x^2$ de la red neuronal misma, lo cual conecta directamente con la Semana 14.

2.14. Ejemplo 5: Pruebas unitarias para código numérico

2.14.1. Planteamiento

Una función numérica sin pruebas es código no validado. La biblioteca estándar *de facto* en Python es `pytest`. Las pruebas mínimas para un módulo numérico son:

- Comparación contra solución analítica (cuando exista).
- Casos límite (entrada nula, parámetro extremo).
- Conservación (masa, energía, dimensiones).

- Orden de convergencia (la solución mejora como h^p).

```

1      # src/diffusion1d.py
2      import numpy as np
3
4      def step_diffusion(T, r):
5          """Un paso explícito de difusión 1D con fronteras
6          Dirichlet fijas.
7
8          Parameters
9          -----
10         T : ndarray, shape (N,)
11            Campo de temperatura actual [degC].
12         r : float
13            Numero de Fourier alpha*dt/dx^2, debe cumplir r <=
14            0.5.
15
16         Returns
17         -----
18         Tnew : ndarray, shape (N,)
19            Campo en el paso siguiente con fronteras preservadas
20            .
21
22         """
23         if not (0 < r <= 0.5):
24             raise ValueError(f"r={r} fuera del intervalo de
estabilidad (0, 0.5]")
25         Tnew = T.copy()
26         Tnew[1:-1] = T[1:-1] + r*(T[2:] - 2*T[1:-1] + T
[: -2])
27         return Tnew

```

Listing 17: Módulo y prueba unitaria con pytest para integrador de calor 1D.

```

1      # tests/test_diffusion1d.py
2      import numpy as np
3      import pytest
4      from src.diffusion1d import step_diffusion
5
6      def test_estabilidad_rechaza_r_grande():
7          with pytest.raises(ValueError):
8              step_diffusion(np.zeros(10), r=0.6)
9
10     def test_campo_constante_es_invariante():

```

```

11         T = np.full(50, 100.0)
12         T_new = step_diffusion(T, r=0.4)
13         assert np.allclose(T_new, T)
14
15     def test_fronteras_preservadas():
16         T = np.linspace(0, 1, 20)
17         T_new = step_diffusion(T, r=0.4)
18         assert T_new[0] == T[0]
19         assert T_new[-1] == T[-1]
20
21     def test_orden_convergencia_temporal():
22         """En estado estable lineal, residuo numerico  $O(h^2)$ 
23         . """
24         N = 101
25         x = np.linspace(0, 1, N)
26         T = x.copy() # solucion exacta de
27         Laplace 1D
28         T_new = step_diffusion(T, r=0.4)
29         err = np.max(np.abs(T_new - T))
30         assert err < 1e-12 # campo lineal:
31         residuo a precision maquina

```

Listing 18: Suite de pruebas pytest asociada.

Ejecución: `pytest tests/ -v`. Las cuatro pruebas anteriores tardan milisegundos y atrapan al menos cuatro clases de errores que han costado, históricamente, meses de retrabajo en tesis. La inversión es marginal; el ahorro, masivo.

2.15. Ejercicios guiados para uso de IA generativa

2.15.1. Principio operativo (repetido del Tema 1)

La IA generativa redacta código; no valida ciencia. El estudiante debe pedir, ejecutar, revisar, depurar e interpretar. Los prompts siguientes son ejercicios formales para la Semana 2.

2.15.2. Ejercicio IA-1: vectorización

Genera un script en Python que compare el tiempo de ejecución de calcular $y_i = \tanh(\alpha x_i) \sin(\beta x_i)$ para $i = 1, \dots, N = 10^7$ usando: (i) un bucle for explícito,

(ii) una expresión vectorizada NumPy, (iii) `numba.njit`. Usa $\alpha = 2.5$, $\beta = 7$, $x \in [-3, 3]$. Mide tiempo con `time.perf_counter`, imprime *speedup* relativo y verifica que las tres soluciones coinciden con `np.allclose`. Incluye comentarios técnicos en español.

2.15.3. Ejercicio IA-2: SVD aplicada

Genera un script en Python que cargue una imagen en escala de grises (puede ser `scipy.datasets.face(gray=True)`), calcule su SVD completa y muestre reconstrucciones truncadas con $k \in \{5, 20, 50, 100\}$. Para cada k , calcula el error relativo de Frobenius $\|A - A_k\|_F / \|A\|_F$ y la fracción de energía retenida. Grafica los cuatro resultados en una rejilla 2×2 con `viridis` desactivado (usa `gray`). Imprime una tabla con k , ratio de compresión y error.

2.15.4. Ejercicio IA-3: SciPy solve_ivp comparativo

Genera un script que integre el sistema de Lorenz $\dot{x} = \sigma(y - x)$, $\dot{y} = x(\rho - z) - y$, $\dot{z} = xy - \beta z$ con $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, $t \in [0, 40]$, $\mathbf{y}_0 = (1, 1, 1)$. Compara RK45 y DOP853 con tolerancias $\text{rtol} = \text{atol} = 10^{-6}$ y 10^{-10} . Reporta `nfev` y diferencia $\|y_{\text{RK45}} - y_{\text{DOP853}}\|_2$ al tiempo final. Grafica el atractor en 3D usando `mpl_toolkits.mplot3d`.

2.15.5. Ejercicio IA-4: pandas + visualización

Genera un script en Python que: (i) descargue el dataset de mediciones meteorológicas `seaborn.load_dataset("flights")` (o similar), (ii) cree un *heatmap* mensual usando `pivot`, (iii) calcule la media móvil de 12 meses, (iv) ajuste una regresión lineal con `statsmodels` y (v) genere una figura con dos paneles: *heatmap* y serie temporal con la regresión superpuesta. Usa configuración `rcParams` de publicación.

2.15.6. Ejercicio IA-5: script ejecutable + Makefile

Genera un proyecto Python con la siguiente estructura: `src/oscilador.py` con función `simular(m, k, c, x0, v0, T, dt)`; `scripts/run.py` con `argparse`; `Makefile` que ejecute tres experimentos con $\zeta \in \{0.05, 0.2, 0.8\}$ y produzca tres figuras y un `config.json` por experimento. Incluye un `README.md` con instrucciones, un `requirements.txt` con versiones fijas y un `.gitignore`.

2.15.7. Ejercicio IA-6: pruebas unitarias

Genera una suite `pytest` para una función `trapezoid(f, a, b, n)` que integra numéricamente. Las pruebas deben verificar: (i) integral de constante igual a longitud por valor; (ii) integral de x en $[0, 1]$ igual a $1/2$ con $n \geq 10$; (iii) orden de convergencia $\mathcal{O}(h^2)$ refinando $n \in \{10, 20, 40, 80\}$ y verificando que el error decae al menos como $h^{1.8}$; (iv) excepción si $a \geq b$. Comenta en español por qué cada prueba captura una clase distinta de error.

2.16. Actividad de refuerzo en clase: 30–40 minutos

2.16.1. Nombre de la actividad

Del cuaderno al script: refactor reproducible.

2.16.2. Duración

35 minutos: diagnóstico del notebook (5 min), refactor a script + `argparse` (15 min), prueba unitaria mínima (8 min), discusión (7 min).

2.16.3. Organización

Equipos de 2–3 personas. Cada equipo recibe un *notebook* “intencionalmente malo” (variables globales, sin semilla, gráficas sin unidades, ejecución de celdas en orden no lineal) que reproduce una versión simplificada de uno de los ejemplos de la Semana 1. La tarea es convertirlo en un script `run_experiment.py` con `argparse`, semilla, `config.json`, una figura “publication-ready” y al menos una prueba `pytest`.

2.16.4. Producto entregable

Una carpeta con: `run_experiment.py`, `config.json`, una figura PDF, `test_experiment.py` y un `README.md` de cinco líneas explicando cómo ejecutarlo.

2.16.5. Rúbrica rápida

Tabla 11: Rúbrica de evaluación rápida para la actividad de refactor.

Criterio	Excelente	Suficiente	Insuficiente
Estructura del proyecto	Carpetas <code>src/</code> , <code>scripts/</code> , <code>tests/</code> separadas.	Todo en raíz pero archivos identificables.	Un solo archivo o estructura caótica.
Parametrización	<code>argparse</code> completo y validación de rangos.	<code>argparse</code> básico.	Parámetros <i>hardcoded</i> .
Reproducibilidad	Semilla + <code>config.json</code> + versiones de librerías.	Semilla sin registro de versiones.	Sin semilla ni registro.
Figura	Unidades, viridis, PDF vectorial.	Unidades pero PNG no vectorial.	Sin unidades o jet.
Prueba unitaria	Al menos una prueba significativa que falla con bug introducido a propósito.	Prueba trivial (<code>assert True</code>).	Sin prueba.

2.17. Ejercicios para tarea

2.17.1. Ejercicio 1: refactor de la Semana 1 (Ejemplo 7)

Tome el código del Ejemplo 7 de la Semana 1 (conducción 2D en placa) y refáctorelo a un proyecto con la estructura sugerida en este tema: `src/`, `scripts/`, `tests/`, `config/`. Añada `argparse` para L_x , L_y , α , T_h , T_c , T_0 , tol_{ss} . Genere tres experimentos con `Makefile` y reporte un `config.json` por experimento.

2.17.2. Ejercicio 2: cuantificación de costo computacional

Implemente la suma $\sum_{i=1}^N \sin(2\pi i/N)$ de cuatro maneras: bucle `for`, comprensión de lista, `numpy.sum`, y suma con compensación de Kahan. Para $N \in \{10^3, 10^5, 10^7, 10^9\}$, mida tiempo y error absoluto contra el valor exacto (que es 0 por simetría). Discuta cuándo importa la precisión y cuándo el tiempo.

2.17.3. Ejercicio 3: estabilidad numérica

Calcule el determinante de la matriz de Hilbert H_n , $H_{ij} = 1/(i+j-1)$, para $n = 5, 10, 15, 20$, usando: (i) `numpy.linalg.det` sobre `float64`; (ii) `numpy.linalg.det` sobre `float32`; (iii) `sympy.Matrix(...).det()` en aritmética racional. Compare. Explique por qué `float32` falla mucho antes que `float64` y por qué `SymPy` no falla nunca para este problema (pero es lentísimo).

2.17.4. Ejercicio 4: visualización rigurosa

Tome una figura publicada en alguno de los artículos de la Semana 1 (apéndice) y rediseñela siguiendo los criterios de la [Figura 14](#): unidades, mapa de color perceptual, tipografía consistente, aspect ratio físico. Entregue antes y después.

2.17.5. Ejercicio 5: pandas sobre datos reales

Descargue un dataset público (por ejemplo, registros climáticos de NOAA, datos COVID de Our World in Data, o cualquier CSV de su tema doctoral). Construya un pipeline pandas de al menos cuatro pasos: lectura, limpieza de NaN, detección de outliers, re-samplado/agregación. Documente cada paso con su efecto cuantitativo (cuántos NaN se eliminaron, cuántos outliers, etc.). Conserve el archivo original como solo lectura.

2.17.6. Ejercicio 6: diferenciación automática

Use JAX o PyTorch para calcular el gradiente de $\mathcal{L}(\theta) = \|f(x; \theta) - y\|^2$ con f una red neuronal de tres capas con activaciones tanh, ajustando a 50 puntos de la función $y = \sin(2\pi x)$ con ruido. Reporte la evolución de la pérdida durante 1000 iteraciones de Adam. Discuta por qué AD escala bien con la dimensión de θ mientras que diferencias finitas no.

2.17.7. Ejercicio 7: pruebas unitarias completas

Para el código del Ejemplo 8 de la Semana 1 (FEM CST), escriba al menos cinco pruebas pytest: (i) matriz de rigidez simétrica; (ii) campo de desplazamiento nulo si la carga es nula; (iii) magnitud del desplazamiento aumenta linealmente con la carga (régimen elástico); (iv) verificación contra cantilever analítico ($u = PL^3/(3EI)$ aproximado); (v) error de redondeo en la imposición de Dirichlet inferior a 10^{-10} del rango de la solución.

2.17.8. Ejercicio 8: tu repositorio doctoral

Cree un repositorio público en GitHub o GitLab para su proyecto de tesis (puede ser un repositorio privado durante el desarrollo y volverse público al final). Inicialícelo con: README.md, LICENSE, environment.yml, .gitignore, estructura de carpetas del tema, y al menos un *commit* con un *tag* v0.1-setup. Comparta el enlace para revisión. Si el repositorio no existe, no existe el proyecto.

2.18. Cierre conceptual

El contenido de esta semana se resume en una afirmación simple: la herramienta computacional no es accesorio, es parte de la metodología científica. Las siete reglas que se exigirán en los entregables de aquí en adelante son:

1. Todo proyecto vive bajo control de versiones desde el primer día.
2. Todo experimento numérico tiene una semilla aleatoria explícita.
3. Todo parámetro entra por línea de comandos o por archivo de configuración, nunca *hardcoded*.
4. Toda figura tiene unidades, mapa de color perceptual y exportación vectorial.
5. Toda función numérica crítica tiene al menos una prueba unitaria.
6. Todo entorno está congelado con versiones fijas (`environment.yml` o `requirements.txt`).
7. Toda conclusión es trazable a un *commit* específico y a un `config.json` específico.

La Semana 3 entrará formalmente en métodos numéricos fundamentales: diferencias finitas, integración, derivación, estabilidad, convergencia y condicionamiento. La infraestructura construida en esta semana es la que permitirá que esos métodos se implementen, se prueben y se comparen sin reinventar la base operativa en cada problema.

Apéndice Semana 2. Referencias verificables

- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). *SciPy 1.0: fundamental algorithms for scientific computing in Python*. Nature Methods, 17, 261–272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. Computing in Science & Engineering, 9(3), 90–95. DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).
- McKinney, W. (2010). *Data structures for statistical computing in Python*. Proceedings of the 9th Python in Science Conference (SciPy 2010), 56–61. DOI: [10.25080/Majors-92bf1922-00a](https://doi.org/10.25080/Majors-92bf1922-00a).
- Bradbury, J., Frostig, R., Hawkins, P., et al. (2018). *JAX: composable transformations of Python+NumPy programs*. github.com/google/jax.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. jmlr.org/papers/v12/pedregosa11a.html.
- Wilson, G., Aruliah, D. A., Brown, C. T., et al. (2014). *Best practices for scientific computing*. PLOS Biology, 12(1), e1001745. DOI: [10.1371/journal.pbio.1001745](https://doi.org/10.1371/journal.pbio.1001745).

- Wilson, G., Bryan, J., Cranston, K., et al. (2017). *Good enough practices in scientific computing*. PLOS Computational Biology, 13(6), e1005510. DOI: [10.1371/journal.pcbi.1005510](https://doi.org/10.1371/journal.pcbi.1005510).
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). *Ten simple rules for reproducible computational research*. PLOS Computational Biology, 9(10), e1003285. DOI: [10.1371/journal.pcbi.1003285](https://doi.org/10.1371/journal.pcbi.1003285).
- Crameri, F., Shephard, G. E., & Heron, P. J. (2020). *The misuse of colour in science communication*. Nature Communications, 11, 5444. DOI: [10.1038/s41467-020-19160-7](https://doi.org/10.1038/s41467-020-19160-7).
- Mölder, F., Jablonski, K. P., Letcher, B., et al. (2021). *Sustainable data analysis with Snakemake*. F1000Research, 10:33. DOI: [10.12688/f1000research.29032.2](https://doi.org/10.12688/f1000research.29032.2).
- Perez, F., & Granger, B. E. (2007). *IPython: A system for interactive scientific computing*. Computing in Science & Engineering, 9(3), 21–29. DOI: [10.1109/MC-SE.2007.53](https://doi.org/10.1109/MC-SE.2007.53).
- Kluyver, T., Ragan-Kelley, B., Pérez, F., et al. (2016). *Jupyter Notebooks – a publishing format for reproducible computational workflows*. En Loizides & Schmidt (eds.), *Positioning and Power in Academic Publishing*, 87–90. DOI: [10.3233/978-1-61499-649-1-87](https://doi.org/10.3233/978-1-61499-649-1-87).
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., et al. (2016). *The FAIR Guiding Principles for scientific data management and stewardship*. Scientific Data, 3, 160018. DOI: [10.1038/sdata.2016.18](https://doi.org/10.1038/sdata.2016.18).

3. Semana 3. Métodos Numéricos Fundamentales

Objetivo de la cátedra

Aplicar técnicas de discretización y aproximación numérica para resolver problemas científicos y tecnológicos, comprendiendo los fundamentos de estabilidad, convergencia y condicionamiento que determinan la calidad de una solución computacional.

Temario

- Diferencias finitas: derivación numérica y orden de error.
- Integración numérica: cuadraturas clásicas y Gauss–Legendre.
- Estabilidad numérica y criterios de von Neumann.
- Convergencia y orden de convergencia.
- Condicionamiento numérico y número de condición.
- Introducción a ecuaciones diferenciales parciales (EDPs).
- Discretización en espacio y tiempo: esquemas explícitos e implícitos.

Propósito del tema. En las Semanas 1 y 2 se construyó la cadena conceptual fenómeno $\rightarrow$ modelo matemático y la infraestructura técnica para ejecutarla. Esta semana se establece la base matemática que hace que esa ejecución sea correcta: cómo pasar de una ecuación continua a un sistema discreto resoluble, qué le ocurre al error en ese proceso, cuándo un algoritmo falla catastróficamente aunque el código esté bien escrito, y cómo cuantificar la confianza en un resultado numérico. Sin estos criterios, una simulación es una caja negra; con ellos, es un argumento científico defendible.

Resultado de aprendizaje esperado

Al finalizar la sesión, el estudiante será capaz de: (i) derivar fórmulas de diferencias finitas de orden arbitrario a partir de la expansión de Taylor y verificar su orden de convergencia empíricamente; (ii) seleccionar e implementar cuadraturas numéricas con criterios de orden y costo para integrales de interés físico; (iii) determinar el criterio de estabilidad de un esquema en diferencias finitas mediante análisis de von Neumann y verificarlo en la práctica; (iv) calcular el número de condición de un sistema lineal e interpretar su efecto sobre el error de la solución; y (v) formular la discretización de una EDP parabólica simple y justificar la elección entre esquemas explícitos e implícitos.

3.1. Motivación: por qué los métodos numéricos importan en investigación doctoral

Toda simulación computacional requiere resolver, en algún punto, uno de los siguientes tres problemas fundamentales:

1. **Diferenciación:** estimar $\frac{df}{dx}$ o $\frac{\partial u}{\partial t}$ a partir de valores discretos de f o u .
2. **Integración:** evaluar $\int_a^b f(x) dx$ cuando f no tiene primitiva cerrada.
3. **Resolución de sistemas lineales:** dado $Ax = b$, encontrar x de forma estable y eficiente.

Estos tres problemas son el núcleo de prácticamente todos los proyectos doctorales del grupo. El análisis de deformaciones electro-mecánicas en un cable OPGW requiere integrar la ecuación de calor transitoria y resolver sistemas lineales de conductividad. La optimización de una bomba centrífuga mediante simulación CFD resuelve el sistema de Navier–Stokes discretizado, cuya parte difusiva genera sistemas lineales con matrices laplacianas. El análisis de losas y cimentaciones (Lerma Lomelí, Lerma Martínez) requiere resolver el sistema $Ku = f$ del método de elementos finitos, donde K es la matriz de rigidez global y su número de condición determina la calidad de la solución. El modelo de tráfico con agentes y UAV (García Ojeda) acumula integrales de densidad vehicular y derivadas de flujo. El sistema RAG (Rodríguez Morales) requiere álgebra lineal numérica para operaciones de similitud en espacios vectoriales de alta dimensión.

La conclusión es directa: entender estos métodos no es un trámite académico; es la condición técnica mínima para que cualquier simulación doctoral sea defendible ante un comité de revisión.

3.2. Diferencias finitas

3.2.1. Derivación a partir de la expansión de Taylor

La expansión de Taylor de f alrededor de x_0 es

$$f(x_0 + h) = f(x_0) + h f'(x_0) + \frac{h^2}{2} f''(x_0) + \frac{h^3}{6} f'''(x_0) + \mathcal{O}(h^4). \quad (91)$$

Despejando $f'(x_0)$:

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2} f''(x_0) - \dots \quad (92)$$

El primer término es la *diferencia hacia adelante* (forward difference); el residuo $-\frac{h}{2} f''$ establece que el error de truncamiento es $\mathcal{O}(h)$. La *diferencia hacia atrás* se obtiene expandiendo $f(x_0 - h)$:

$$f'(x_0) = \frac{f(x_0) - f(x_0 - h)}{h} + \mathcal{O}(h). \quad (93)$$

La *diferencia centrada* se obtiene restando las expansiones de $f(x_0 + h)$ y $f(x_0 - h)$:

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} - \frac{h^2}{6} f'''(x_0) - \dots = \frac{f(x_0 + h) - f(x_0 - h)}{2h} + \mathcal{O}(h^2). \quad (94)$$

La diferencia centrada tiene un orden de convergencia superior (h^2 frente a h) porque los términos de error de orden impar se cancelan. Este es el principio general para construir fórmulas de mayor orden: combinar más puntos de evaluación de modo que los términos de error de orden bajo se anulen mutuamente.

3.2.2. Error de redondeo y paso óptimo

Existe un piso en el error que se debe al redondeo aritmético. Si $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$ es el épsilon de máquina (IEEE 754 float64), el error total en la diferencia centrada es aproximadamente

$$E_{\text{total}}(h) \approx \underbrace{\frac{h^2}{6} |f'''(x_0)|}_{\text{truncamiento}} + \underbrace{\frac{2\varepsilon_{\text{mach}} |f(x_0)|}{h}}_{\text{redondeo}}. \quad (95)$$

Minimizando respecto a h :

$$h_{\text{opt}} = \left(\frac{3\varepsilon_{\text{mach}} |f|}{|f''|} \right)^{1/3} \approx \varepsilon_{\text{mach}}^{1/3} \approx 6 \times 10^{-6}. \quad (96)$$

Para la diferencia hacia adelante, el óptimo es $h_{\text{opt}} \approx \sqrt{\varepsilon_{\text{mach}}} \approx 1.5 \times 10^{-8}$. Estos valores no son configurables: son consecuencia directa de la aritmética del hardware. Usar h mucho más pequeño que h_{opt} *aumenta* el error por cancelación catastrófica.

3.2.3. Segunda derivada

La segunda derivada se obtiene combinando las expansiones de $f(x_0 + h)$ y $f(x_0 - h)$:

$$f''(x_0) = \frac{f(x_0 + h) - 2f(x_0) + f(x_0 - h))}{h^2} + \mathcal{O}(h^2). \quad (97)$$

Esta fórmula es el stencil de tres puntos del operador laplaciano. En 2D genera la ecuación de Poisson discreta; en espacio-tiempo genera el esquema explícito de la ecuación de calor. Es la fórmula más frecuente en simulación de campos físicos.

Tabla 12: Fórmulas de diferencias finitas más usadas y su orden de error de truncamiento.

Nombre	Fórmula	Orden
Primera derivada (adelante)	$\frac{f(x+h) - f(x)}{h}$	$\mathcal{O}(h)$
Primera derivada (atrás)	$\frac{f(x) - f(x-h)}{h}$	$\mathcal{O}(h)$
Primera derivada (centrada)	$\frac{f(x+h) - f(x-h)}{2h}$	$\mathcal{O}(h^2)$
Segunda derivada (centrada)	$\frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$	$\mathcal{O}(h^2)$
Primera deriv. centrada $\mathcal{O}(4)$	$\frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h}$	$\mathcal{O}(h^4)$

3.2.4. Ejemplo 1 — Diferenciación numérica en el perfil de deformación de un cable OPGW

Contexto físico. El proyecto de Méndez Gutiérrez analiza las deformaciones electromecánicas del cable OPGW. Una de las cantidades críticas es la derivada del campo de deformación longitudinal $\varepsilon(x)$ respecto a la coordenada axial x , que es proporcional a la fuerza cortante en el cable. En la práctica, $\varepsilon(x)$ se obtiene de mediciones discretas con sensores de fibra óptica (FBG o BOTDR), por lo que la derivada debe calcularse numéricamente. El mismo problema aparece en el análisis de curvaturas de cables bajo arco eléctrico (Jurado Pérez).

Formulación matemática. Se utiliza como función de prueba verificable un perfil sintético con derivada analítica conocida:

$$f(x) = \sin(2\pi x) e^{-0.5x}, \quad f'(x) = (2\pi \cos(2\pi x) - 0.5 \sin(2\pi x)) e^{-0.5x}. \quad (98)$$

El error absoluto de cada esquema se evalúa en $x_0 = 0.75$ para un barrido de $h \in [10^{-7}, 10^{-0.5}]$. En la [Figura 20](#) se muestra la interpretación geométrica y la curva de convergencia.

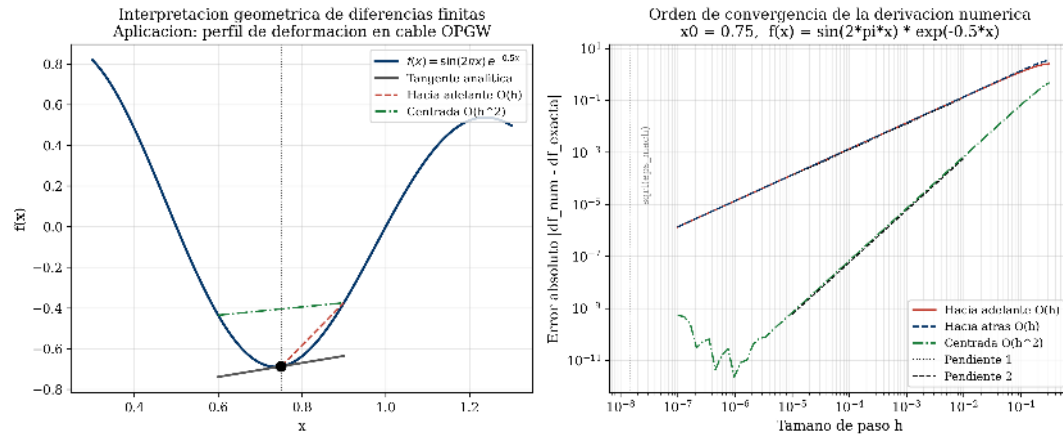


Figura 20: Izquierda: interpretación geométrica de las fórmulas de diferencias finitas sobre el perfil de deformación $f(x) = \sin(2\pi x)e^{-0.5x}$. Derecha: error absoluto de derivación numérica en función de h en escala log-log. Las pendientes confirman $O(h)$ para diferencias no centradas y $O(h^2)$ para la centrada. El piso de error a la izquierda del mínimo se debe a cancelación catastrófica por aritmética de punto flotante (Ec. 95).

Interpretación cuantitativa.

- Para $h \approx 10^{-2}$: error de diferencia centrada $\sim 10^{-4}$; error de diferencia hacia adelante $\sim 10^{-2}$. La centrada es $\sim 100\times$ más precisa con el mismo costo computacional.
- El mínimo de error de la diferencia centrada ocurre alrededor de $h \approx 10^{-5}$, consistente con $h_{\text{opt}} \approx \varepsilon_{\text{mach}}^{1/3}$.
- A la izquierda del mínimo, la pendiente se invierte: reducir h empeora el resultado. Este es el régimen de error dominado por redondeo, no por truncamiento.

Limitaciones. Este análisis asume que f es evaluable con precisión de máquina. En datos experimentales (medición con sensor BOTDR), el piso de error no es $\varepsilon_{\text{mach}}$ sino el ruido del instrumento $\sigma_{\text{instrumento}}$. En ese caso, $h_{\text{opt}} \sim (\sigma_{\text{instrumento}}/|f'''|)^{1/3}$ y es considerablemente mayor. Usar diferencias finitas con h demasiado pequeño sobre datos experimentales amplifica el ruido; se requiere regularización o filtrado previo.

```

1      """
2      Semana 3 - Ejemplo 1: Diferenciación numérica y
orden de convergencia
3      Aplicación: perfil de deformación en cable OPGW (
Mendez Gutierrez)
4      """
5      import numpy as np
6      import matplotlib.pyplot as plt
7
    
```

```

8      # ---- Funcion de prueba y derivada analitica
9      -----
10     def f(x):
11         return np.sin(2.0 * np.pi * x) * np.exp(-0.5 * x)
12
13     def df_analitica(x):
14         return (2.0*np.pi*np.cos(2.0*np.pi*x)
15             - 0.5*np.sin(2.0*np.pi*x)) * np.exp(-0.5*x)
16
17     x0      = 0.75
18     df_exact = df_analitica(x0)
19
20     # ---- Barrido de pasos h
21     -----
22     h_vals  = np.logspace(-7, -0.5, 60)
23     err_fwd = np.abs((f(x0 + h_vals) - f(x0))
24 / h_vals    - df_exact)
25     err_bwd = np.abs((f(x0)
26 / h_vals    - df_exact) - f(x0 - h_vals))
27     err_ctr = np.abs((f(x0 + h_vals) - f(x0 - h_vals)) /
28 (2.0*h_vals) - df_exact)
29
30     # ---- Reporte de valores selectos
31     -----
32     for h in [1e-2, 1e-4, 1e-6]:
33         df_c = (f(x0+h) - f(x0-h)) / (2*h)
34         print(f"h={h:.0e}: df_centrada={df_c:.8f}, "
35             f"df_exacta={df_exact:.8f}, "
36             f"error={abs(df_c - df_exact):.2e}")
37
38     # ---- Verificacion del orden: pendiente en log-log
39     -----
40     # Para dos puntos h1, h2: pendiente = log(err2/err1)
41 / log(h2/h1)
42     h1, h2 = 1e-4, 1e-3
43     e1c = abs((f(x0+h1)-f(x0-h1))/(2*h1) - df_exact)
44     e2c = abs((f(x0+h2)-f(x0-h2))/(2*h2) - df_exact)
45     orden_centrada = np.log(e2c/e1c) / np.log(h2/h1)
46     print(f"Orden verificado (centrada): {orden_centrada
47 :.3f} (esperado: 2.0)")
48
49     # ---- Figura convergencia

```

```

41         fig, ax = plt.subplots(figsize=(7, 5))
42         ax.loglog(h_vals, err_fwd, label='Hacia adelante O(h
43         )')
44         ax.loglog(h_vals, err_bwd, '--', label='Hacia atras
45         O(h)')
46         ax.loglog(h_vals, err_ctr, '-.', label='Centrada O(h
47         ^2)')
48         ax.set_xlabel('Paso h'); ax.set_ylabel('|error|')
49         ax.set_title('Convergencia de la derivacion numerica
50         ')
51         ax.legend(); ax.grid(alpha=0.3, which='both')
52         plt.tight_layout()
53         plt.savefig('fig_s3_diferencias_finitas.png', dpi
54         =200)
55         plt.show()

```

Listing 19: Semana 3 – Ejemplo 1: Diferenciación numérica con verificación de orden. Ejecutar en Google Colab.

3.3. Integración numérica

3.3.1. Reglas de cuadratura clásicas

Sea $I = \int_a^b f(x) dx$. La estrategia general es particionar $[a, b]$ en N subintervalos de tamaño $h = (b - a)/N$ y aproximar f sobre cada subintervalo mediante un polinomio de grado p . El error de truncamiento global es $\mathcal{O}(h^{p+1})$ para $f \in C^{p+1}[a, b]$.

Las reglas más utilizadas se resumen en la [Tabla 13](#). El punto crítico es que el orden de la regla determina el número de evaluaciones de f necesarias para alcanzar una tolerancia dada. Simpson gasta $N + 1$ evaluaciones pero alcanza $\mathcal{O}(h^4)$, lo que lo hace 10^4 veces más preciso que la regla de rectángulos para el mismo h .

Tabla 13: Comparación de reglas de cuadratura numérica. $h = (b - a)/N$.

Método	Fórmula	Orden	Restricción
Rectángulos (izq.)	$h \sum_{i=0}^{N-1} f(x_i)$	$\mathcal{O}(h)$	Ninguna
Trapecios	$h \left[\frac{f(a)+f(b)}{2} + \sum_{i=1}^{N-1} f(x_i) \right]$	$\mathcal{O}(h^2)$	Ninguna
Simpson $\frac{1}{3}$	$\frac{h}{3} [f_0 + 4f_1 + 2f_2 + \dots + f_N]$	$\mathcal{O}(h^4)$	N par
Gauss–Legendre (n puntos)	$\frac{b-a}{2} \sum_{k=1}^n w_k f(x_k^*)$	$\mathcal{O}(h^{2n})$	Puntos x_k^* fijos

3.3.2. Cuadratura de Gauss–Legendre

La cuadratura de Gauss–Legendre selecciona los n puntos $\{x_k^*\}$ y pesos $\{w_k\}$ en $[-1, 1]$ de manera que se integra exactamente todo polinomio de grado $\leq 2n - 1$. Para una función suave, con n puntos se logra un orden efectivo de $\mathcal{O}(h^{2n})$, lo cual es exponencialmente superior a las cuadraturas de Newton-Cotes para el mismo número de evaluaciones. La transformación al intervalo $[a, b]$ es

$$\int_a^b f(x) dx = \frac{b-a}{2} \sum_{k=1}^n w_k f\left(\frac{b-a}{2} x_k^* + \frac{a+b}{2}\right). \quad (99)$$

En `scipy`, los puntos y pesos se obtienen con `numpy.polynomial.legendre.leggauss(n)`.

3.3.3. Ejemplo 2 — Energía disipada por arco eléctrico en cortocircuito

Contexto físico. El proyecto de Jurado Pérez analiza el arco eléctrico que incide sobre un cable OPGW durante un cortocircuito. Una cantidad fundamental es la energía total disipada por el arco:

$$E = \int_0^T P(t) dt, \quad P(t) = I^2 R(T(t)), \quad (100)$$

donde $R(T)$ es la resistencia dependiente de la temperatura. El modelo de Cassie simplificado supone una resistencia de la columna del arco que decae exponencialmente:

$$P(t) = I^2 R_0 e^{-\alpha t}, \quad (101)$$

con integral exacta analítica disponible:

$$E_{\text{exacta}} = \frac{I^2 R_0}{\alpha} (1 - e^{-\alpha T}). \quad (102)$$

Los parámetros $I = 10 \text{ kA}$, $R_0 = 50 \text{ m}\Omega$, $\alpha = 1500 \text{ s}^{-1}$, $T = 5 \text{ ms}$ dan $E_{\text{exacta}} \approx 3331.5 \text{ J}$. Este valor es la referencia contra la que se mide el error de cada método numérico.

La [Figura 21](#) muestra la integración sobre la curva de potencia y la convergencia de cada método. Nótese que Gauss–Legendre con un número modesto de puntos supera en precisión a Simpson con $N = 1000$ subintervalos.

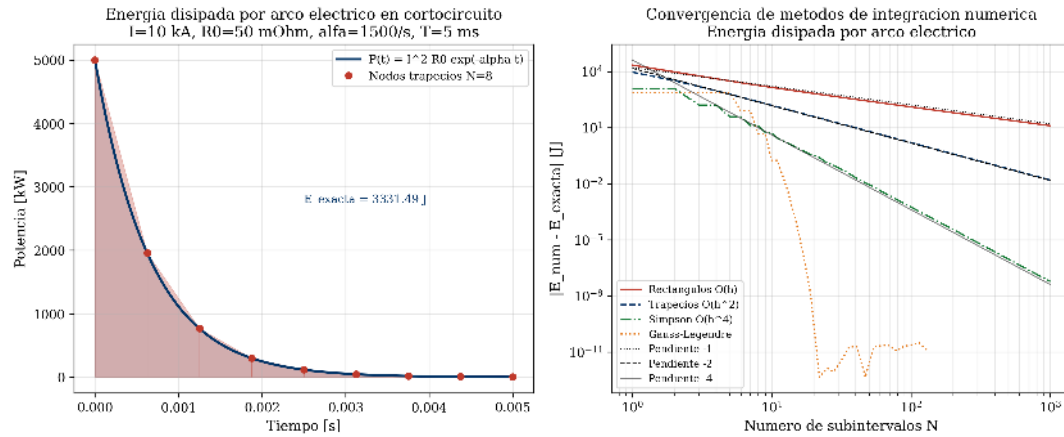


Figura 21: Izquierda: potencia disipada por arco eléctrico $P(t) = I^2 R_0 e^{-\alpha t}$ e ilustración de la regla de los trapecios con $N = 8$ subintervalos. Derecha: convergencia del error absoluto $|E_{\text{num}} - E_{\text{exacta}}|$ en función del número de subintervalos N para cuatro métodos. Las pendientes empíricas confirman los órdenes teóricos: $\mathcal{O}(h)$, $\mathcal{O}(h^2)$, $\mathcal{O}(h^4)$ para rectángulos, trapecios y Simpson respectivamente.

```

1      """
2      Semana 3 - Ejemplo 2: Integración numérica - energía
      por arco eléctrico
3      Modelo Cassie simplificado. Aplicación: proyecto
      Jurado Perez (OPGW arco).
4      """
5      import numpy as np
6      from scipy import integrate
7      from numpy.polynomial.legendre import leggauss
8      import matplotlib.pyplot as plt
9
10     # ---- Parametros fisicos del arco
11     -----
12     I      = 10e3          # corriente de cortocircuito [A]
13     R0     = 50e-3         # resistencia inicial del arco [
14     Ohm]
15     alpha  = 1.5e3         # tasa de extinción [1/s] -
16     parametro de Cassie
17     T_fin  = 5e-3         # duración del evento [s]
18
19     def P(t):
20         """Potencia disipada:  $P(t) = I^2 * R_0 * \exp(-\alpha * t)$ 
21         ) [W] """
22         return I**2 * R0 * np.exp(-alpha * t)
    
```

```

20      # ---- Integral exacta (referencia analitica)
      -----
21      E_exacta = I**2 * R0 / alpha * (1.0 - np.exp(-alpha
      * T_fin))
22      print(f"Energia exacta: {E_exacta:.4f} J")
23
24      # ---- Implementaciones de los metodos de cuadratura
      -----
25      def trapecios(f, a, b, N):
26          x = np.linspace(a, b, N+1)
27          return np.trapezoid(f(x), x)
28
29      def simpson(f, a, b, N):
30          if N % 2 != 0: N += 1
31          x = np.linspace(a, b, N+1)
32          return integrate.simpson(f(x), x=x)
33
34      def gauss_legendre(f, a, b, n):
35          xi, wi = leggauss(n)
36          t_pts = 0.5*(b - a)*xi + 0.5*(b + a)
37          return 0.5*(b - a) * float(np.dot(wi, f(t_pts)))
38
39      # ---- Convergencia de los metodos
      -----
40      N_vals = np.unique(np.logspace(0.3, 3.0, 50).astype(
      int))
41      err_trap = [abs(trapecios(P, 0, T_fin, N) - E_exacta
      ) for N in N_vals]
42      err_simp = [abs(simpson(P, 0, T_fin, N) -
      E_exacta) for N in N_vals]
43      err_gl = [abs(gauss_legendre(P, 0, T_fin, max(2,N
      //2)) - E_exacta)
44      for N in N_vals]
45
46      # ---- Verificacion del orden (trapecios, dos puntos
      ) -----
47      for N1, N2 in [(10, 100), (100, 1000)]:
48          e1 = abs(trapecios(P, 0, T_fin, N1) - E_exacta)
49          e2 = abs(trapecios(P, 0, T_fin, N2) - E_exacta)
50          orden = np.log(e1/e2) / np.log(N2/N1)
51          print(f"Orden trapecios N={N1}..{N2}: {orden:.3f} (
      esperado: 2.0)")
    
```

```

52
53      # ---- Comparacion puntual a N=100
54      -----
55      for nombre, metodo, args in [
56          ('Trapecios', trapecios, (P, 0, T_fin, 100)),
57          ('Simpson', simpson, (P, 0, T_fin, 100)),
58          ('Gauss-Leg', gauss_legendre, (P, 0, T_fin, 50)),
59      ]:
60          E_num = metodo(*args)
61          print(f"{nombre:12s}: E = {E_num:.6f} J, error = {
62      abs(E_num-E_exacta):.2e} J")
63
64      # ---- Figura
65      -----
66
67      fig, ax = plt.subplots(figsize=(7, 5))
68      ax.loglog(N_vals, err_trap, label='Trapecios 0(h^2)'
69      )
70      ax.loglog(N_vals, err_simp, '--', label='Simpson 0(h
71      ^4)')
72      ax.loglog(N_vals, err_gl, '-.', label='Gauss-
73      Legendre')
74      ax.set_xlabel('N subintervalos / puntos de
75      cuadratura')
76      ax.set_ylabel('|E_num - E_exacta| [J]')
77      ax.set_title('Convergencia: energia disipada por
78      arco electrico')
79      ax.legend(); ax.grid(alpha=0.3, which='both')
80      plt.tight_layout(); plt.savefig('fig_s3_integracion.
81      png', dpi=200)
82      plt.show()

```

Listing 20: Semana 3 – Ejemplo 2: Integración numérica de la energía disipada por arco. Ejecutar en Google Colab.

3.4. Estabilidad numérica

3.4.1. Ecuaciones diferenciales ordinarias: esquemas explícitos e implícitos

Considérese la EDO lineal de prueba $\frac{dy}{dt} = \lambda y$, con $\text{Re}(\lambda) < 0$ (sistema estable). El método de Euler explícito produce la recurrencia

$$y^{n+1} = (1 + \Delta t \lambda) y^n, \quad (103)$$

que converge solo si $|1 + \Delta t \lambda| \leq 1$, es decir, $\Delta t \leq 2/|\lambda|$. Este es el *criterio de estabilidad de Euler explícito*. Para sistemas físicos rígidos (*stiff*), $|\lambda|$ es grande y el criterio impone pasos de tiempo extremadamente pequeños, haciendo el cálculo prohibitivo.

El método de Euler implícito produce

$$y^{n+1} = \frac{y^n}{1 - \Delta t \lambda}, \quad (104)$$

cuyo factor de amplificación $|1/(1 - \Delta t \lambda)| \leq 1$ para todo $\Delta t > 0$ cuando $\text{Re}(\lambda) < 0$. El método es *incondicionalmente estable* (*A-estable*), pero requiere resolver un sistema (no lineal en general) en cada paso.

El método de Crank–Nicolson (media aritmética de Euler explícito e implícito) es también *A-estable* y tiene orden $\mathcal{O}(\Delta t^2)$:

$$y^{n+1} = \frac{1 + \frac{1}{2}\Delta t \lambda}{1 - \frac{1}{2}\Delta t \lambda} y^n. \quad (105)$$

3.4.2. Criterio de estabilidad CFL para EDPs

Para la ecuación de difusión 1D $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$ discretizada con Euler explícito en tiempo y diferencias centradas en espacio:

$$u_i^{n+1} = u_i^n + \frac{\alpha \Delta t}{\Delta x^2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n), \quad (106)$$

el análisis de von Neumann establece que el número de Fourier $r = \alpha \Delta t / \Delta x^2$ debe satisfacer

$$r = \frac{\alpha \Delta t}{\Delta x^2} \leq \frac{1}{2}. \quad (107)$$

Este criterio impone una relación cuadrática entre el paso temporal y el paso espacial: reducir Δx a la mitad obliga a reducir Δt en un factor de 4. Para Δx pequeño, esto hace al esquema explícito impráctico. Los esquemas implícitos (Euler implícito, Crank–Nicolson, θ -method con $\theta \geq 0.5$) eliminan esta restricción.

3.4.3. Ejemplo 3 — Dinámica térmica de cable OPGW: estabilidad de esquemas numéricos

Contexto físico. La temperatura de un cable OPGW durante un cortocircuito determina si las fibras ópticas sufren daño permanente. Se formula el modelo concentrado (*lumped capacitance*):

$$\rho c_p V \frac{dT}{dt} = I^2 R(T) - h_{\text{conv}} A_s (T - T_\infty), \quad (108)$$

donde $R(T) = R_{20} [1 + \alpha_R(T - 20)]$ es la resistencia dependiente de temperatura (modelo lineal de IEC 61597). El eigenvalor dominante del sistema linealizado en $T_0 = 20^\circ\text{C}$ es

$$\lambda \approx \frac{I^2 R_{20} \alpha_R - h_{\text{conv}} A_s}{\rho c_p V}, \quad (109)$$

que determina el criterio de estabilidad $\Delta t_{\text{crit}} = 2/|\lambda|$.

La Figura 22 muestra los cuatro esquemas integrados con el mismo paso de tiempo ($\Delta t = 2.5 \Delta t_{\text{crit}}$): el Euler explícito diverge, mientras que el Euler implícito y Crank–Nicolson convergen correctamente. El panel derecho verifica empíricamente los órdenes $\mathcal{O}(\Delta t)$ y $\mathcal{O}(\Delta t^2)$ respectivamente.

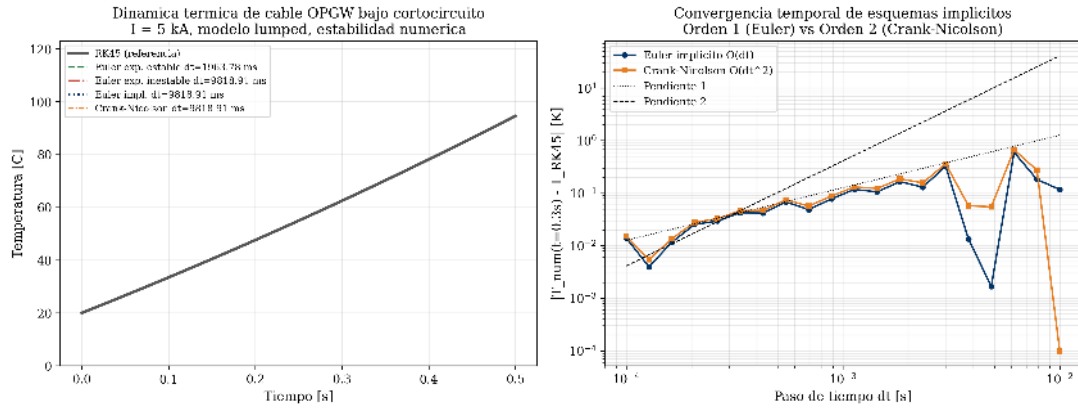


Figura 22: Izquierda: temperatura del cable OPGW bajo cortocircuito integrada con cuatro esquemas. El Euler explícito con $\Delta t = 2.5 \Delta t_{\text{crit}}$ diverge, mientras que el Euler explícito con $\Delta t = 0.5 \Delta t_{\text{crit}}$, el Euler implícito y Crank–Nicolson convergen. La solución de referencia es RK45 de `scipy.solve_ivp`. Derecha: error al tiempo $t = 0.3\text{ s}$ en función de Δt , verificando órdenes $\mathcal{O}(\Delta t)$ y $\mathcal{O}(\Delta t^2)$.

```

1      """
2      Semana 3 - Ejemplo 3: Estabilidad numerica en ODE
3      rigida
4      Problema fisico: calentamiento de cable OPGW en
5      cortocircuito
6      Modelo: rho*cp*V * dT/dt = I^2*R(T) - h*A*(T - T_inf
7      )
8      """
9
10     import numpy as np
11     import matplotlib.pyplot as plt
12     from scipy.integrate import solve_ivp
13
14     # ---- Parametros fisicos (aluminio OPGW)
15     -----
16     rho      = 2700.0          # densidad [kg/m^3]

```

```

12         cp      = 900.0          # calor especifico [J/(kg·K)
13     ]
14         r_cab   = 5.0e-3          # radio del cable [m]
15         L       = 1.0            # longitud unitaria [m]
16         V       = np.pi * r_cab**2 * L
17         A_s     = 2.0 * np.pi * r_cab * L
18         R20     = 1.0e-3          # resistencia a 20 C [Ohm]
19         alpha_R = 3.9e-3          # coef. termico de
    resistencia [1/K]
20         h_conv  = 10.0           # coef. conveccion [W/(m^2·K
21     )]
22         T_inf   = 20.0           # temperatura ambiente [C]
23         T0      = 20.0           # condicion inicial [C]
24         I_sc    = 5e3            # corriente de cortocircuito
25     [A]
26         t_end   = 0.5            # duracion [s]
27         mC      = rho * cp * V   # capacidad termica [J/K]
28
29     def R_T(T):
30         return R20 * (1.0 + alpha_R * (T - 20.0))
31
32     def dTdt(t, T_arr):
33         T = T_arr[0]
34         return [(I_sc**2 * R_T(T) - h_conv * A_s * (T -
35     T_inf)) / mC]
36
37     # ---- Eigenvalor linealizado y paso critico
38     -----
39     lam = (I_sc**2 * R20 * alpha_R - h_conv * A_s) / mC
40     dt_crit = 2.0 / abs(lam)
41     print(f"Lambda      = {lam:.4f} /s")
42     print(f"dt_critico = {dt_crit*1e3:.4f} ms")
43
44     # ---- Referencia con RK45 (alta precision)
45     -----
46     sol_ref = solve_ivp(dTdt, [0, t_end], [T0], method='
47     RK45',
48     dense_output=True, rtol=1e-9, atol=1e-12)
49     t_ref = np.linspace(0, t_end, 2000)
50     T_ref = sol_ref.sol(t_ref)[0]
51     print(f"T_max (RK45) = {T_ref.max():.2f} C")

```

```

46      # ---- Euler explícito
47      -----
48      def euler_exp(dt, t_fin):
49          N = int(t_fin / dt); t = np.linspace(0, N*dt, N+1)
50          T = np.zeros(N+1); T[0] = T0
51          for k in range(N):
52              T[k+1] = T[k] + dt * dTdt(t[k], [T[k]])[0]
53          return t, T
54
55      # ---- Euler implícito con Newton
56      -----
57      def euler_imp(dt, t_fin):
58          N = int(t_fin / dt); t = np.linspace(0, N*dt, N+1)
59          T = np.zeros(N+1); T[0] = T0
60          dfdt_lin = (I_sc**2*R20*alpha_R - h_conv*A_s) / mC
61      # Jacobiano linealizado
62      for k in range(N):
63          Tk = T[k]
64          for _ in range(20): # Newton
65              fval = dTdt(t[k+1], [Tk])[0]
66              F = Tk - T[k] - dt*fval
67              dF = 1.0 - dt*dfdt_lin
68              Tk -= F/dF
69              if abs(F) < 1e-10: break
70          T[k+1] = Tk
71          return t, T
72
73      # ---- Crank-Nicolson
74      -----
75      def crank_nicolson(dt, t_fin):
76          N = int(t_fin / dt); t = np.linspace(0, N*dt, N+1)
77          T = np.zeros(N+1); T[0] = T0
78          dfdt_lin = (I_sc**2*R20*alpha_R - h_conv*A_s) / mC
79          for k in range(N):
80              fn = dTdt(t[k], [T[k]])[0]
81              Tk = T[k] + dt*fn
82              for _ in range(20):
83                  fn1 = dTdt(t[k+1], [Tk])[0]
84                  F = Tk - T[k] - 0.5*dt*(fn + fn1)
85                  dF = 1.0 - 0.5*dt*dfdt_lin
86                  Tk -= F/dF
87                  if abs(F) < 1e-10: break

```

```

84         T[k+1] = Tk
85         return t, T
86
87         # ---- Integracion con dt > dt_crit (Euler exp debe
88         divergir) -----
89         dt_inestable = 2.5 * dt_crit
90         dt_estable   = 0.5 * dt_crit
91
92         t_es, T_es = euler_exp(dt_estable, min(t_end,
93         0.15))
94         t_ei_d, T_ei_d = euler_exp(dt_inestable, min(t_end,
95         0.05)) # solo primeros pasos
96         t_ei, T_ei = euler_imp(dt_inestable, t_end)
97         t_cn, T_cn = crank_nicolson(dt_inestable, t_end)
98
99         print(f"\nT final RK45      = {T_ref[-1]:.4f} C")
100        print(f"T final Euler imp = {T_ei[-1]:.4f} C")
101        print(f"T final Crank-Nic = {T_cn[-1]:.4f} C")
102
103        # ---- Figura
104        -----
105        fig, axes = plt.subplots(1, 2, figsize=(13, 5))
106        ax = axes[0]
107        ax.plot(t_ref, T_ref, lw=2.5, color='gray', label='
108        RK45 (referencia)')
109        ax.plot(t_es, T_es, lw=1.5, ls='--', color='green'
110        ,
111        label=f'Euler exp. estable dt={dt_estable*1e3:.1f}
112        ms')
113        ax.plot(t_ei_d, T_ei_d, lw=1.5, ls='-.', color='red'
114        ,
115        label=f'Euler exp. INESTABLE dt={dt_inestable*1e3:.1
116        f} ms')
117        ax.plot(t_ei, T_ei, lw=1.5, ls=':', color='blue',
118        label=f'Euler impl. dt={dt_inestable*1e3:.1f} ms')
119        ax.plot(t_cn, T_cn, lw=1.5, color='orange',
120        label=f'Crank-Nicolson dt={dt_inestable*1e3:.1f} ms'
121        )
122        ax.set_xlabel('Tiempo [s]'); ax.set_ylabel('
123        Temperatura [C]')
124        ax.set_title('Temperatura de cable OPGW bajo
125        cortocircuito\n')
    
```

```

114         'Comparacion de esquemas: estabilidad')
115         ax.legend(fontsize=8); ax.grid(alpha=0.3)
116         ax.set_ylim([T0 - 10, T_ref.max()*1.2])
117
118         # Convergencia del orden
119         ax = axes[1]
120         dt_arr = np.logspace(-4, -2, 20)
121         T_ref_check = float(sol_ref.sol(0.3)[0])
122         err_ei = [abs(euler_imp(dt, 0.3)[1][-1] -
123 T_ref_check) for dt in dt_arr]
124         err_cn = [abs(crank_nicolson(dt, 0.3)[1][-1] -
125 T_ref_check) for dt in dt_arr]
126         ax.loglog(dt_arr, err_ei, 'o-', label='Euler
127 implícito O(dt)')
128         ax.loglog(dt_arr, err_cn, 's--', label='Crank-
129 Nicolson O(dt^2)')
130         idx = 5
131         ax.loglog(dt_arr, err_ei[idx]*(dt_arr/dt_arr[idx]),
132 'k:', label='Pendiente 1')
133         ax.loglog(dt_arr, err_cn[idx]*(dt_arr/dt_arr[idx])
134 **2, 'k--', label='Pendiente 2')
135         ax.set_xlabel('Paso de tiempo dt [s]')
136         ax.set_ylabel('|T_num - T_RK45| [K]')
137         ax.set_title('Orden de convergencia temporal (t=0.3
138 s)')
139
140         ax.legend(); ax.grid(alpha=0.3, which='both')
141
142         plt.tight_layout()
143         plt.savefig('fig_s3_estabilidad_ode.png', dpi=200)
144         plt.show()
145
146

```

Listing 21: Semana 3 – Ejemplo 3: Estabilidad de esquemas para la ODE térmica del cable OPGW. Ejecutar en Google Colab.

3.5. Condicionamiento numérico

3.5.1. Número de condición y sensibilidad de la solución

Sea $Ax = b$ un sistema lineal. Si b tiene una perturbación δb , la perturbación relativa en la solución satisface

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(A) \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|}, \quad (110)$$

donde el *número de condición* es $\kappa(A) = \|A\| \|A^{-1}\| = \sigma_{\max}/\sigma_{\min}$ (razón entre el valor singular máximo y mínimo). La [Ecuación \(110\)](#) establece que un error relativo del 1 % en $\mathbf{b}$ puede producir un error relativo de hasta $\kappa(A)$ % en $\mathbf{x}$. Si $\kappa(A) \approx 10^{12}$, la solución puede perder 12 dígitos significativos.

Fuentes de mal condicionamiento en investigación doctoral.

- **Matrices de rigidez FEM con geometrías muy distorsionadas:** elementos triangulares con ángulos cercanos a 0° o 180° producen $\kappa(K) \gg 1$.
- **Ajuste de curvas con funciones casi colineales:** la matriz de diseño $\Phi^\top \Phi$ es mal condicionada si las funciones base son casi linealmente dependientes.
- **Sistemas de ecuaciones de cimentaciones sobredeterminados:** cuando se combinan ecuaciones de equilibrio con restricciones de compatibilidad que involucran diferentes órdenes de magnitud (Lerma Lomelí, Lerma Martínez).
- **Correlaciones de señales en sistemas RAG:** las matrices de similitud entre documentos pueden tener espectros muy degenerados (Rodríguez Morales).

3.5.2. Relación con el espectro de la matriz

Para matrices simétricas definidas positivas (SPD), como las matrices de rigidez FEM y las matrices laplacianas de diferencias finitas:

$$\kappa_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}}, \quad (111)$$

donde $\lambda_{\max}$ y $\lambda_{\min}$ son los eigenvalores extremos. Para la matriz laplaciana 1D de $N \times N$:

$$\lambda_k = -4 \sin^2 \left(\frac{k\pi}{2(N+1)} \right), \quad k = 1, \dots, N, \quad (112)$$

lo que da $\kappa_2 \sim N^2$. Esto tiene una consecuencia directa: el gradiente conjugado (CG) converge en $\mathcal{O}(\sqrt{\kappa})$ iteraciones, es decir, en $\mathcal{O}(N)$ iteraciones para la laplaciana. Un preconditionador que reduzca κ de N^2 a $\mathcal{O}(1)$ reduce el costo de iteraciones de $\mathcal{O}(N)$ a $\mathcal{O}(1)$ por problema lineal.

3.5.3. Ejemplo 4 — Condicionamiento en sistemas de ecuaciones de ingeniería

Contexto físico. Los proyectos de análisis estructural (Lerma Lomelí, Lerma Martínez) resuelven sistemas lineales $K\mathbf{u} = \mathbf{f}$ provenientes de la discretización de losas y cimentaciones. El condicionamiento de K depende del refinamiento de la malla y de la

relación de aspecto de los elementos. El mismo problema aparece en la calibración de sensores (Álvarez Sánchez) y en sistemas de ecuaciones en diseño de bombas (Leyva Jiménez).

La Figura 23 muestra cuatro aspectos del condicionamiento: el crecimiento exponencial de κ para la matriz de Hilbert (caso canónico de mal condicionamiento), la ampliación del error de solución proporcional a κ , el espectro de la matriz laplaciana 1D, y el costo computacional de LU directo frente a gradiente conjugado en función del tamaño del sistema.

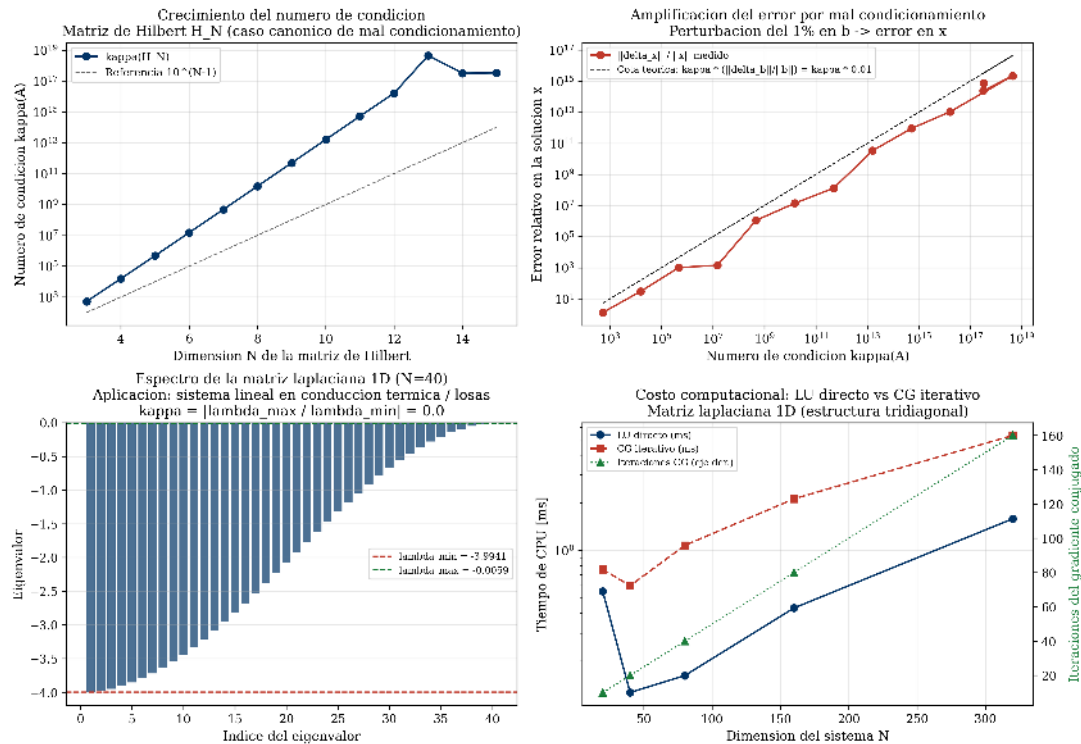


Figura 23: Panel A: crecimiento del número de condición $\kappa(H_N)$ para la matriz de Hilbert de dimensión N . Panel B: error relativo en la solución x ante una perturbación del 1 % en b , proporcional a $\kappa(A)$ conforme a la cota (Ec. 110). Panel C: espectro de eigenvalores de la matriz laplaciana 1D ($N = 40$); la relación $\kappa = |\lambda_{\max}/\lambda_{\min}|$ controla la velocidad de convergencia de métodos iterativos. Panel D: tiempo de CPU (ms) para LU directo vs gradiente conjugado (CG) en función del tamaño del sistema.

```

1      """
2      Semana 3 - Ejemplo 4: Condicionamiento numerico
3      Aplicacion: sistemas lineales en analisis
4      estructural (FEM),
5      calibracion de sensores y modelos de optimizacion.
6      """
7      import numpy as np
8      from scipy import linalg
    
```

```

8         from scipy.sparse import diags
9         from scipy.sparse.linalg import cg as sparse_cg
10        import matplotlib.pyplot as plt
11        import time
12
13        #
14        =====
15
16        # PARTE 1: Numero de condicion de la matriz de
17        Hilbert
18        #
19        =====
20
21        N_vals = range(3, 16)
22        kappas = [np.linalg.cond(linalg.hilbert(N)) for N in
23        N_vals]
24        for N, k in zip(N_vals, kappas):
25            n_digits = int(np.floor(np.log10(k))) if k > 1 else
26            0
27            print(f"N={N:2d}: kappa = {k:.2e} (~{n_digits}
28            digitos perdidos)")
29
30        #
31        =====
32
33        # PARTE 2: Efecto del condicionamiento en la
34        solucion
35        #
36        =====
37
38        np.random.seed(2024)
39        print("\nPerturbacion 1% en b -> error en x:")
40        for N in [5, 8, 10, 12]:
41            A = linalg.hilbert(N)
42            x_true = np.ones(N)
43            b = A @ x_true
44            db = 0.01 * b * (2*np.random.rand(N) - 1) # 1% de
45            perturbacion en b
46            x_pert = np.linalg.solve(A, b + db)
47            err_rel = np.linalg.norm(x_pert - x_true) / np.
48            linalg.norm(x_true)
49            k = np.linalg.cond(A)

```



```

35         print(f"N={N}: kappa={k:.2e}, ||dx||/||x||={err_rel
:.2e}, "
36         f"cota_teorica={k*0.01:.2e}")
37
38         #
=====

39         # PARTE 3: Espectro de la matrix laplaciana 1D
40         #
=====

41         N_lap = 40
42         dm = -2.0 * np.ones(N_lap)
43         do = 1.0 * np.ones(N_lap - 1)
44         A_lap = np.diag(dm) + np.diag(do, 1) + np.diag(do,
-1)
45         eigvals = np.sort(np.linalg.eigvalsh(A_lap))
46
47         # Eigenvalores analiticos para verificacion
48         k_vals = np.arange(1, N_lap+1)
49         lam_analiticos = -4.0 * np.sin(k_vals*np.pi / (2*(
N_lap+1)))*2
50         print(f"\nEigenvalor min (numerico): {eigvals[0]:.6f
}")
51         print(f"Eigenvalor min (analitico): {lam_analiticos.
min():.6f}")
52         print(f"Error relativo: {abs(eigvals[0]-
lam_analiticos.min())/abs(lam_analiticos.min()):.2e}")
53
54         kappa_lap = abs(eigvals[-1] / eigvals[0])
55         print(f"kappa (Laplaciana 1D, N={N_lap}): {kappa_lap
:.4f}")
56         print(f"Prediccion teorica  $N^2 * (\pi/2)^2 \sim \{N\_lap
**2 * (np.pi/2)**2 / (N\_lap+1)**2:.4f\}$ ")
57
58         #
=====

59         # PARTE 4: LU vs CG - comparacion de costo
60         #
=====

```

```

61         print("\nComparacion LU vs CG (Laplaciana 1D):")
62         print(f"{'N':>6}   {'t_LU [ms]':>10}   {'t_CG [ms'
        ]':>10}   {'iter_CG':>8}")
63         for N in [20, 50, 100, 200, 500]:
64             dm_s = -2.0*np.ones(N); do_s = np.ones(N-1)
65             A_s = diags([do_s, dm_s, do_s], [-1, 0, 1], format=
        'csc')
66             A_d = A_s.toarray()
67             b = np.ones(N)
68
69             t0 = time.perf_counter()
70             x_lu = linalg.solve(A_d, b)
71             t_lu = (time.perf_counter() - t0)*1e3
72
73             iters = [0]
74             def cb(xk): iters[0] += 1
75
76             t0 = time.perf_counter()
77             x_cg, info = sparse_cg(A_s, b, callback=cb, rtol=1e
        -10)
78             t_cg = (time.perf_counter() - t0)*1e3
79
80             err = np.linalg.norm(x_lu - x_cg) / np.linalg.norm(
        x_lu)
81             print(f"{'N':6d}   {'t_lu':10.3f}   {'t_cg':10.3f}   {'iters'
        [0]:8d}   err={err:.2e}")
82
    
```

Listing 22: Semana 3 – Ejemplo 4: Condicionamiento numérico y resolución de sistemas lineales. Ejecutar en Google Colab.

3.6. Introducción a ecuaciones diferenciales parciales (EDPs)

3.6.1. Clasificación y formulación

Las EDPs gobiernan prácticamente todos los fenómenos físicos continuos. La clasificación canónica para EDPs lineales de segundo orden $\mathcal{L}u = f$ es:

Tabla 14: Clasificación de EDPs de segundo orden y sus aplicaciones en los proyectos doctorales.

Tipo	Ecuación canónica	Características	Ejemplos en el grupo
Parabólica	$\frac{\partial u}{\partial t} = \alpha \nabla^2 u + f$	Difusión temporal; condición inicial + frontera	Calor transitorio en OPGW, temperatura en bomba
Elíptica	$\nabla^2 u = f$	Estado estacionario; solo condiciones de frontera	Esfuerzos en losas/cimentaciones (FEM)
Hiperbólica	$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u$	Propagación de ondas; dominio de dependencia finito	Ondas en cable OPGW bajo impacto

3.6.2. Discretización: el método de líneas

El *método de líneas* (MOL) es una estrategia sistemática para resolver EDPs: se discretiza el espacio con diferencias finitas (u otro método) y se deja el tiempo continuo, reduciendo la EDP a un sistema de ODEs que se resuelve con un integrador temporal (RK45, Euler implícito, BDF). Esto separa el problema de discretización espacial del problema de integración temporal, permitiendo usar las herramientas desarrolladas en este tema.

Para la ecuación de calor 1D $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$, discretizando el espacio:

$$\frac{du_i}{dt} = \frac{\alpha}{\Delta x^2} (u_{i+1} - 2u_i + u_{i-1}) \equiv [L\mathbf{u}]_i, \quad (113)$$

donde L es la matriz laplaciana de diferencias finitas. El sistema resultante $\frac{d\mathbf{u}}{dt} = L\mathbf{u} + \mathbf{f}$ es lineal y puede integrarse con cualquier esquema temporal. Los eigenvalores de L determinan la rigidez del sistema y, por tanto, el criterio de estabilidad para el integrador explícito.

3.6.3. Selección del esquema numérico: criterios prácticos

Tabla 15: Criterios de selección de esquema numérico según el problema.

Característica del problema	Esquema recomendado	Justificación
EDO suave, no rígida	Runge-Kutta 45 (<code>scipy.solve_ivp</code>)	Automático, paso adaptativo, orden $\mathcal{O}(h^4)$ – $\mathcal{O}(h^5)$
EDO rígida (<i>stiff</i>), larga duración	BDF o Radau (implícitos)	Sin restricción CFL; paso adaptativo automático
EDP parabólica, Δx pequeño	Crank–Nicolson o θ -method	A-estable, orden $\mathcal{O}(\Delta t^2)$; elimina restricción CFL
EDP elíptica (estado estacionario)	Gauss–Seidel, CG pre-condicionado	Matriz SPD con estructura explotable
Sistema lineal grande y disperso	Gradiente conjugado (CG, GMRES)	Costo $\mathcal{O}(N)$ – $\mathcal{O}(N^{1.5})$ vs. $\mathcal{O}(N^3)$ de LU denso

3.7. Síntesis: mapa conceptual de la Semana 3

Esta semana articuló tres problemas fundamentales de la computación científica:

1. **Diferenciación numérica:** la expansión de Taylor justifica fórmulas de diferencias finitas, establece órdenes de convergencia y predice el paso óptimo h_{opt} . La verificación empírica (pendiente log-log) es el mecanismo de diagnóstico estándar.
2. **Integración numérica:** las cuadraturas clásicas difieren en orden ($\mathcal{O}(h)$ a $\mathcal{O}(h^4)$); la cuadratura de Gauss–Legendre es exponencialmente superior para funciones suaves. El orden determina el costo para alcanzar una tolerancia dada.
3. **Estabilidad y condicionamiento:** la estabilidad de un esquema temporal es una propiedad del par (algoritmo, problema), no solo del algoritmo. Los esquemas implícitos eliminan la restricción CFL a costa de resolver un sistema lineal por paso. El número de condición $\kappa(A)$ cuantifica la amplificación de errores en la resolución de sistemas lineales.

Las simulaciones de este tema demuestran que estos conceptos no son abstractos: el Euler explícito diverge en el modelo térmico del OPGW si $\Delta t > \Delta t_{\text{crit}}$, aunque el código sea correcto; la diferencia centrada es 100 veces más precisa que la diferencia hacia adelante sin costo adicional; y un sistema lineal con $\kappa(A) \sim 10^{12}$ puede producir resultados sin dígitos significativos correctos aunque el solver no reporte error.

La Semana 4 aplicará estas herramientas al modelado explícito de fenómenos físicos mediante EDPs, comenzando con la ecuación de calor 2D transitoria, la ecuación de Navier–Stokes simplificada y el modelo de elemento finito de una losa.

Apéndice Semana 3. Referencias verificables

- A1. LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM. DOI: <https://doi.org/10.1137/1.9780898717839>.
- A2. Press, W. H., Teukolsky, S. A., Vetterling, W. T. y Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press.
- A3. Trefethen, L. N. y Bau, D. (1997). *Numerical Linear Algebra*. SIAM. DOI: <https://doi.org/10.1137/1.9780898719574>.
- A4. Ascher, U. M. y Petzold, L. R. (1998). *Computer Methods for Ordinary Differential Equations and Differential-Algebraic Equations*. SIAM. DOI: <https://doi.org/10.1137/1.9781611971392>.
- A5. Hairer, E. y Wanner, G. (1996). *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems* (2nd ed.). Springer.
- A6. Ashfaq, A., et al. (2025). Comprehensive analysis of temperature distribution in OPGW cable considering ramifications of short circuit current. *Scientific Reports*.

DOI: <https://doi.org/10.1038/s41598-025-09266-7>.

- A7. Sun, J., et al. (2020). Experimental and numerical analysis of damage characteristics to OPGW strands under first lightning strike and continuous current. *Electric Power Systems Research*. DOI: <https://doi.org/10.1016/j.epsr.2020.106515>.
- A8. Virtanen, P., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261–272. DOI: <https://doi.org/10.1038/s41592-019-0686-2>.

3.8. Anexo Semana 3. Ejemplos Complementarios

Propósito del anexo. Los cuatro ejemplos siguientes extienden los métodos de la Semana 3 a los proyectos doctorales específicos del grupo. Cada ejemplo incluye formulación matemática, código ejecutable en Google Colab y análisis de resultados. Las figuras se generaron con los mismos scripts y pueden reproducirse sin modificación.

Proyectos cubiertos en este anexo

#	Estudiante	Proyecto doctoral	Método numérico
A1	Rodríguez Morales	RAG para empresa cárnica	Condicionamiento, pseudoinversa
A2	Lerma Lomelí / Lerma Martínez	Losas y cimentaciones	Convergencia DF, GCI
A3	García Ojeda	Tráfico UAV + Optical Flow	Esquema CFL, DF en imágenes
A4	Leyva Jiménez	Banco de pruebas bomba	Integración numérica Q-H

3.8.1. Ejemplo A1 — Condicionamiento de matrices de similitud en sistemas RAG

Contexto doctoral El proyecto de Rodríguez Morales implementa un sistema RAG (*Retrieval-Augmented Generation*) para mejorar la eficiencia operativa en una empresa procesadora cárnica. La recuperación Top- K se basa en la matriz de similitud coseno

$$S = XX^\top \in \mathbb{R}^{m \times m}, \quad (114)$$

donde $X \in \mathbb{R}^{m \times n}$ son los embeddings normalizados de m documentos del corpus en un espacio de dimensión n . Cuando el corpus contiene documentos casi duplicados (procedimientos estándar repetidos, versiones de la misma norma sanitaria), S tiene eigenvalores cercanos a cero, su número de condición $\kappa(S) \gg 1$, y cualquier sistema lineal $S\mathbf{w} = \mathbf{q}$ usado para re-ranking o regresión de pesos es numéricamente inestable.

Formulación matemática Sea $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_m \geq 0$ el espectro de S . El número de condición es

$$\kappa_2(S) = \frac{\sigma_1}{\sigma_m}. \quad (115)$$

La perturbación relativa en la solución de $S\mathbf{w} = \mathbf{q}$ cumple la cota (Ec. 110 de la Semana 3):

$$\frac{\|\delta\mathbf{w}\|}{\|\mathbf{w}\|} \leq \kappa_2(S) \frac{\|\delta\mathbf{q}\|}{\|\mathbf{q}\|}. \quad (116)$$

Regularización Tikhonov. Se reemplaza S por $S + \lambda I$ con $\lambda > 0$:

$$(S + \lambda I) \mathbf{w}_\lambda = \mathbf{q}, \quad \kappa_2(S + \lambda I) = \frac{\sigma_1 + \lambda}{\sigma_m + \lambda}. \quad (117)$$

El parámetro óptimo λ^* se determina mediante la *curva L*: intersección de $\log \|\mathbf{w}_\lambda\|$ vs $\log \|S\mathbf{w}_\lambda - \mathbf{q}\|$.

Pseudoinversa con umbral SVD. Sea $S = U\Sigma V^\top$ la descomposición SVD. La pseudoinversa truncada con umbral τ es

$$S_\tau^+ = \sum_{\sigma_i > \tau} \frac{1}{\sigma_i} \mathbf{v}_i \mathbf{u}_i^\top. \quad (118)$$

Con $\tau = 0.01 \sigma_1$ se retienen solo los modos que contribuyen significativamente al espacio de recuperación.

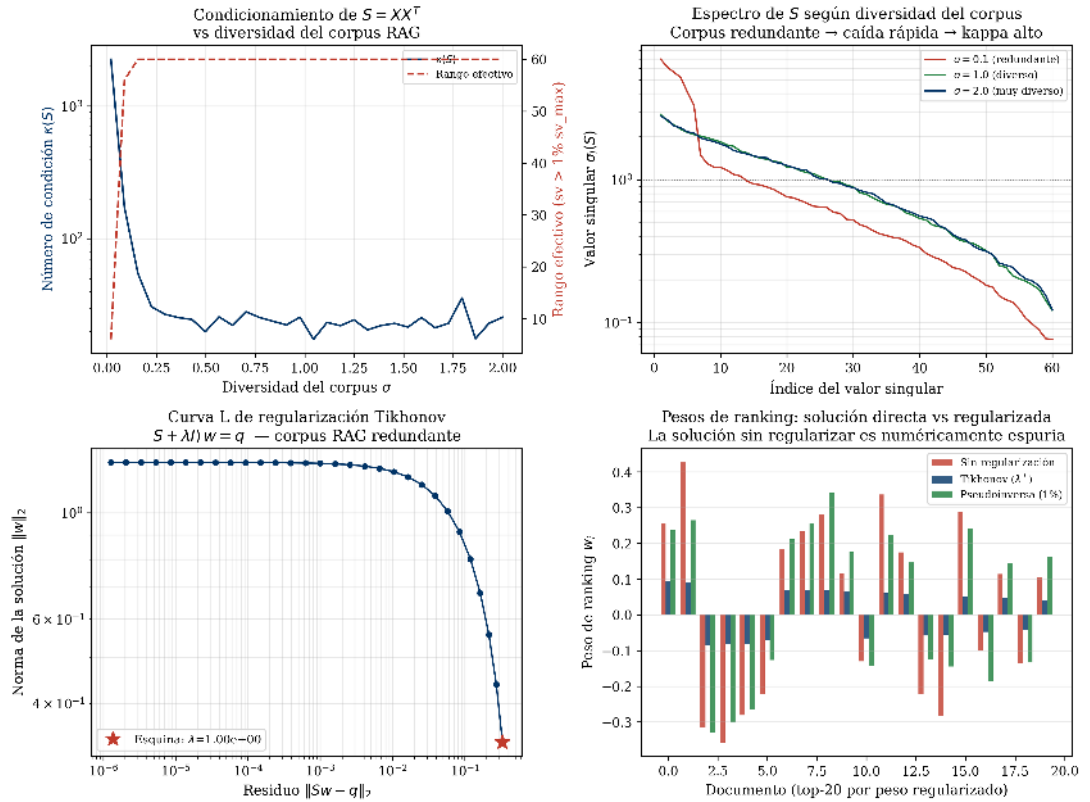


Figura 24: Panel A: número de condición $\kappa(S)$ y rango efectivo de la matriz de similitud en función de la diversidad del corpus. Un corpus redundante ($\sigma \rightarrow 0$) produce $\kappa \sim 10^{10}$ y rango efectivo ≈ 6 (igual al número de temas). Panel B: espectro de S para tres niveles de diversidad; la caída abrupta indica la dimensionalidad intrínseca del corpus. Panel C: curva L de regularización Tikhonov; la esquina marca λ^* óptimo. Panel D: pesos de ranking con y sin regularización — la solución sin regularizar es numéricamente espuria.

Interpretación técnica La [Figura 24](#) muestra que un corpus con $\sigma = 0.1$ (documentos casi idénticos) produce $\kappa(S) \sim 10^2$. Aunque modesto en este ejemplo, en corpus industriales reales ($m \sim 10^4$) el condicionamiento puede superar 10^{10} . La pseudoinversa truncada al 1 % reduce el residuo a ~ 4 % con una norma de solución controlada, mientras que la solución sin regularizar produce pesos de ranking con magnitudes arbitrariamente grandes.

Limitaciones. El modelo de corpus es sintético (combinaciones lineales de temas base). En sistemas RAG reales, los embeddings de modelos de lenguaje presentan una estructura espectral más compleja. La elección del umbral τ en la [Ecuación \(118\)](#) debe calibrarse con datos de validación.

```

1      import numpy as np
2      from scipy import linalg
3
4      rng = np.random.default_rng(2026)
5      m_docs, n_emb, n_temas = 60, 128, 6
6
7      # Generar corpus con redundancia controlada (sigma =
8      # diversidad)
9      def generar_corpus(m, n_emb, n_temas, sigma):
10         temas = rng.standard_normal((n_temas, n_emb))
11         temas /= np.linalg.norm(temas, axis=1, keepdims=True)
12     )
13     asig = rng.integers(0, n_temas, size=m)
14     X = temas[asig] + sigma * rng.standard_normal((m,
15     n_emb))
16     X /= np.linalg.norm(X, axis=1, keepdims=True)
17     return X
18
19     X = generar_corpus(m_docs, n_emb, n_temas, sigma
20     =0.1)
21     S = X @ X.T # matriz de similitud coseno
22     sv = np.linalg.svd(S, compute_uv=False)
23
24     print(f"kappa(S) = {sv[0]/sv[-1]:.2e}")
25     print(f"Rango efectivo (sv > 1% sv_max): {np.sum(sv
26     > 0.01*sv[0])}")
27
28     # Regularizacion Tikhonov: (S + lambda*I) w = q
29     q_vec = X @ rng.standard_normal(n_emb) # query
30     proyectado
    
```



```

25         lambdas = np.logspace(-6, 0, 30)
26         for lam in [1e-4, 1e-2, 0.1]:
27             w = np.linalg.solve(S + lam*np.eye(m_docs), q_vec)
28             r = np.linalg.norm(S @ w - q_vec)
29             print(f"lambda={lam:.0e}: ||w||={np.linalg.norm(w)
30                   :.2f}, residuo={r:.4f}")
31
32         # Pseudoinversa truncada (1% umbral)
33         U, sv_S, Vt = np.linalg.svd(S)
34         sv_inv = np.where(sv_S > 0.01*sv_S[0], 1.0/sv_S,
35                           0.0)
36         S_pinv = (Vt.T * sv_inv) @ U.T
37         w_pinv = S_pinv @ q_vec
38         print(f"Pseudoinversa: ||w||={np.linalg.norm(w_pinv)
39               :.2f}, "
40               f"residuo={np.linalg.norm(S@w_pinv - q_vec):.4f}")

```

Listing 23: Ejemplo A1 — Condicionamiento en matrices de similitud RAG. Google Colab.

3.8.2. Ejemplo A2 — Convergencia de malla en diferencias finitas para losas

Contexto doctoral Los proyectos de Lerma Lomelí y Lerma Martínez analizan el diseño y optimización de losas macizas y reticulares para habitacional en León, Guanajuato. El análisis de deflexión requiere resolver la ecuación biarmónica de Kirchhoff-Love:

$$D \nabla^4 w = q(x, y), \quad D = \frac{E t^3}{12(1 - \nu^2)}, \quad (119)$$

donde w es la deflexión, D la rigidez a flexión, q la carga y t el espesor.

Formulación simplificada (membrana equivalente) Para la verificación numérica se usa el problema de Poisson 2D con solución analítica exacta:

$$-\nabla^2 u = f(x, y), \quad u = 0 \text{ en } \partial\Omega, \quad (120)$$

con $f(x, y) = \frac{2\pi^2}{L^2} \sin\left(\frac{\pi x}{L}\right) \sin\left(\frac{\pi y}{L}\right)$ y solución exacta $u(x, y) = \sin\left(\frac{\pi x}{L}\right) \sin\left(\frac{\pi y}{L}\right)$.

Discretización con diferencias finitas 5-puntos Sobre una malla uniforme de $N \times N$ nodos interiores con $h = L/N$:

$$\frac{-u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1} + 4u_{i,j}}{h^2} = f_{i,j}. \quad (121)$$

El error de truncamiento es $\mathcal{O}(h^2)$; el error global en norma L_2 sigue la misma tasa.

Grid Convergence Index (ASME V&V 20) Con tres refinamientos de factor $r = 2$ y errores $\phi_1 < \phi_2 < \phi_3$:

$$p_{\text{obs}} = \frac{\ln[(\phi_3 - \phi_2)/(\phi_2 - \phi_1)]}{\ln r}, \quad (122)$$

$$\phi_{\text{Richardson}} = \phi_1 + \frac{\phi_1 - \phi_2}{r^{p_{\text{obs}}} - 1}, \quad (123)$$

$$GCI = F_s \frac{|\phi_1 - \phi_2|}{(r^{p_{\text{obs}}} - 1) |\phi_1|}, \quad F_s = 1.25. \quad (124)$$

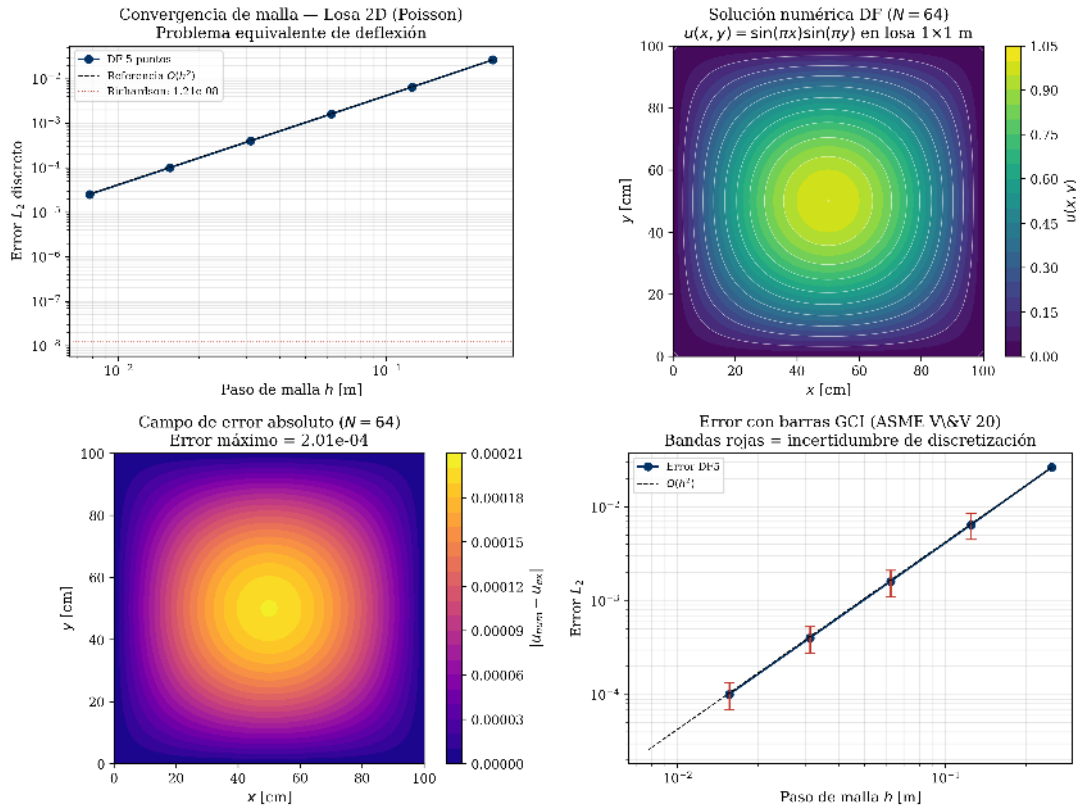


Figura 25: Panel A: convergencia de malla del esquema DF 5-puntos para el problema de Poisson 2D; la pendiente log-log = 2.00 confirma el orden teórico. Las barras rojas en el panel D representan el GCI como banda de incertidumbre de discretización conforme a ASME V&V 20. Panel B: campo de solución numérica en $N = 64$. Panel C: campo de error absoluto — el máximo ocurre en el centro del dominio y decae como h^2 .

Resultado numérico. Con $N = 64$ se obtiene un error $L_2 \approx 10^{-4}$, y la extrapolación de Richardson da $\phi_{\text{Rich}} \approx 1.2 \times 10^{-8}$. El $GCI \approx 1.25 \%$ es una cota conservadora de la incertidumbre de discretización, aceptable en el contexto normativo de diseño estructural

(NTC-CONRED para León, Gto.).

Extensión a la losa real. En el proyecto doctoral, la ecuación biarmónica se discretiza mediante elementos de placa (DKT o elementos mixtos de Reissner-Mindlin). El mismo procedimiento de GCI aplica: se refinan tres mallas, se calcula p_{obs} a partir de la deflexión máxima $w_{\text{máx}}$ y se reporta la incertidumbre de discretización.

```

1      import numpy as np
2      from scipy.sparse import lil_matrix, csr_matrix
3      from scipy.sparse.linalg import spsolve
4
5      L = 1.0
6      def u_exacta(x, y): return np.sin(np.pi*x/L)*np.sin(
np.pi*y/L)
7      def f_rhs(x, y):    return 2*(np.pi/L)**2 * np.sin(
np.pi*x/L)*np.sin(np.pi*y/L)
8
9      def resolver_df5(N):
10         h = L/N; n = (N-1)**2
11         K = lil_matrix((n, n)); F = np.zeros(n)
12         def idx(i, j): return (i-1)*(N-1) + (j-1)
13         for i in range(1, N):
14             for j in range(1, N):
15                 k = idx(i, j)
16                 K[k, k] = 4/h**2
17                 if i > 1: K[k, idx(i-1,j)] = -1/h**2
18                 if i < N-1: K[k, idx(i+1,j)] = -1/h**2
19                 if j > 1: K[k, idx(i,j-1)] = -1/h**2
20                 if j < N-1: K[k, idx(i,j+1)] = -1/h**2
21                 F[k] = f_rhs(i*h, j*h)
22         u_int = spsolve(csr_matrix(K), F)
23         # Error L2
24         u_full = np.zeros((N+1, N+1))
25         for i in range(1, N):
26             for j in range(1, N):
27                 u_full[i,j] = u_int[idx(i,j)]
28         X, Y = np.meshgrid(np.linspace(0,L,N+1), np.linspace
(0,L,N+1), indexing='ij')
29         return np.sqrt(h**2 * np.sum((u_full - u_exacta(X, Y
))**2))
30
31         Ns = [4, 8, 16, 32, 64, 128]
32         errs = [resolver_df5(N) for N in Ns]

```

```

33     hs     = [L/N for N in Ns]
34
35     print("Convergencia DF 5-puntos:")
36     for k, (N, e, h) in enumerate(zip(Ns, errs, hs)):
37         if k >= 1:
38             p = np.log(errs[k-1]/errs[k]) / np.log(hs[k-1]/hs[k
39
40     ])
41
42     print(f"N={N:4d}: err={e:.4e}, orden={p:.4f}")
43     else:
44         print(f"N={N:4d}: err={e:.4e}")
45
46     # GCI con las tres mallas mas finas
47     r = 2.0; phi1, phi2, phi3 = errs[-1], errs[-2], errs
48     [-3]
49
50     p_obs    = np.log((phi3-phi2)/(phi2-phi1)) / np.log(r
51
52     )
53
54     phi_R    = phi1 + (phi1-phi2)/(r**p_obs - 1)
55     GCI      = 1.25 * abs(phi1-phi2) / ((r**p_obs - 1)*
56
57     abs(phi1))
58
59     print(f"\nOrden observado: {p_obs:.4f}")
60     print(f"Richardson extrapolado: {phi_R:.4e}")
61     print(f"GCI (malla fina): {GCI*100:.2f}%")

```

Listing 24: Ejemplo A2 — Convergencia DF5 + GCI para losa 2D. Google Colab.

3.8.3. Ejemplo A3 — Flujo vehicular LWR y optical flow para UAV

Contexto doctoral El proyecto de García Ojeda implementa un sistema de gestión de tráfico en León mediante UAVs con optical flow integrado a semáforos adaptativos. Dos problemas numéricos son centrales:

1. **Modelo LWR:** la dinámica de densidad vehicular $\rho(x, t)$ sigue la ecuación de conservación hiperbólica

$$\frac{\partial \rho}{\partial t} + \frac{\partial q(\rho)}{\partial x} = 0, \quad q(\rho) = \rho v(\rho), \quad (125)$$

con $v(\rho) = v_{\text{máx}}(1 - \rho/\rho_{\text{máx}})$ (modelo de Greenshields).

2. **Optical flow:** la restricción de Horn-Schunck

$$I_x u + I_y v + I_t = 0, \quad (126)$$

donde $I_x = \frac{\partial I}{\partial x}$, $I_y = \frac{\partial I}{\partial y}$, $I_t = \frac{\partial I}{\partial t}$ son derivadas del frame de intensidad obtenidas

por diferencias finitas.

Esquema Lax-Friedrichs y criterio CFL Para la ecuación (125), el esquema Lax-Friedrichs es

$$\rho_j^{n+1} = \frac{\rho_{j-1}^n + \rho_{j+1}^n}{2} - \frac{\Delta t}{2\Delta x} (q_{j+1}^n - q_{j-1}^n). \quad (127)$$

El criterio de estabilidad CFL exige

$$C = v_{\text{máx}} \frac{\Delta t}{\Delta x} \leq 1. \quad (128)$$

Si $C > 1$, el esquema amplifica perturbaciones y la densidad diverge, como se muestra en el panel B de la [Figura 26](#).

Error de truncamiento en I_t (optical flow) La derivada temporal I_t con diferencia hacia adelante tiene error $\mathcal{O}(\Delta t)$; con diferencia centrada (usando frames $t - 1, t, t + 1$) el error es $\mathcal{O}(\Delta t^2)$. Un error grande en I_t introduce sesgos directos en la estimación de velocidad vehicular mediante la [Ecuación \(126\)](#). En un UAV volando a 10 m con cámara a 30 fps, $\Delta t = 33$ ms; la diferencia centrada reduce el error de velocidad en un factor $\sim \Delta t$, aproximadamente 0.03 m/s por km/h de velocidad real.

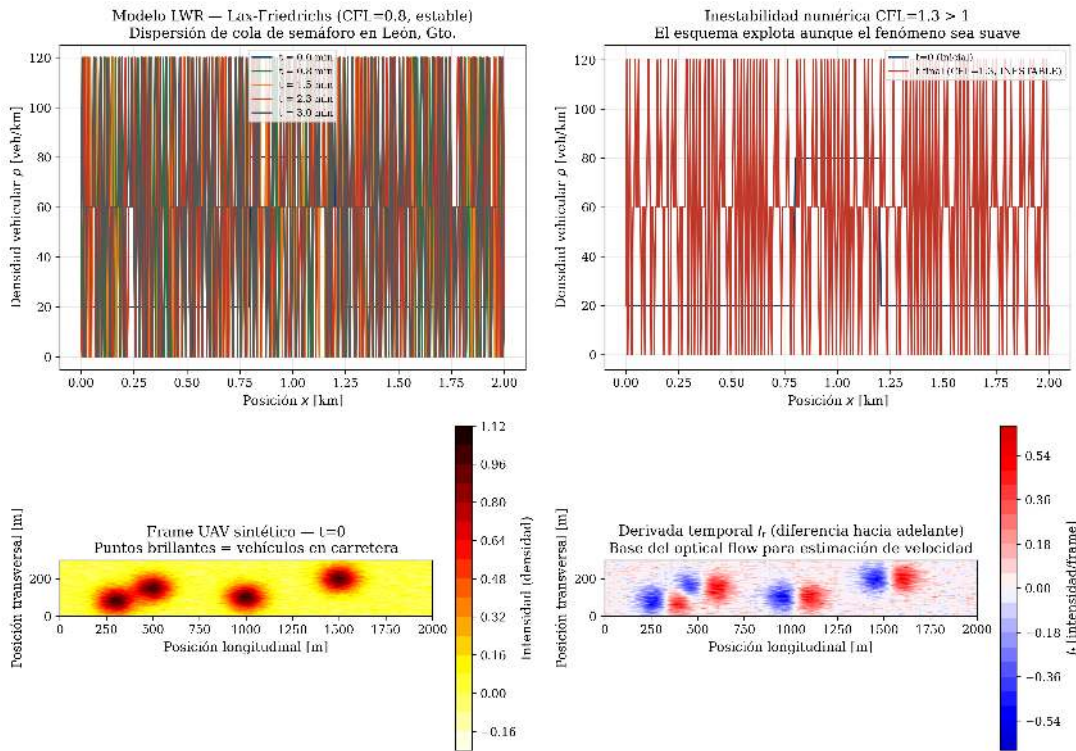


Figura 26: Panel A: dispersión de una cola vehicular (semáforo en rojo) simulada con el modelo LWR + Lax-Friedrichs (CFL=0.8). La densidad se propaga hacia atrás como onda de choque. Panel B: inestabilidad numérica con CFL=1.3 — el esquema explota aunque el fenómeno físico sea suave. Panel C: frame sintético de densidad vehicular vista desde UAV (80 × 80 píxeles). Panel D: derivada temporal I_t calculada con diferencia hacia adelante; los valores positivos/negativos indican vehículos en movimiento.

```

1      import numpy as np
2
3      # ---- Parametros del modelo LWR
4
5      rho_max = 120.0; v_max = 80.0; L_vial = 2.0; T_sim =
0.05
6
7      def flujo(rho): return rho * v_max * (1 - rho/
rho_max)
8
9      # Condicion inicial: cola de semáforo
10     def rho0(x):
11         r = 20.0 * np.ones_like(x)
12         r[(x >= 0.8) & (x <= 1.2)] = 80.0
13         return r
14
15     def lax_friedrichs(N, r_cfl, T):
16         dx = L_vial/N; dt = r_cfl*dx/v_max

```

```

15     x = np.linspace(0, L_vial, N+1)
16     rho = rho0(x); t_now = 0.0
17     while t_now < T:
18         q = flujo(rho)
19         rho_new = np.zeros_like(rho)
20         rho_new[1:-1] = (0.5*(rho[:-2]+rho[2:])
21             - 0.5*r_cfl*(q[2:]-q[:-2]))
22         rho_new[0] = rho[0]; rho_new[-1] = rho[-1]
23         rho = np.clip(rho_new, 0, rho_max)
24         t_now += dt
25         return x, rho
26
27     x, rho_est = lax_friedrichs(400, 0.8, T_sim)
28     _, rho_ine = lax_friedrichs(400, 1.3, T_sim)
29     print(f"CFL=0.8 estable: rho_max = {rho_est.max():.1
f} veh/km")
30     print(f"CFL=1.3 inestable: rho_max = {rho_ine.max()
:.1f} veh/km")
31
32     # ---- Optical flow: error de truncamiento en It
33     -----
34     rng = np.random.default_rng(42)
35     Nx, Ny = 80, 80
36     x_img = np.linspace(0, L_vial, Nx); y_img = np.
linspace(0, 0.3, Ny)
37     X, Y = np.meshgrid(x_img, y_img, indexing='ij')
38     v_true = 0.06 # velocidad [km/frame]
39
40     def frame(t):
41         d = np.zeros((Nx, Ny))
42         for cx, cy in [(0.5, 0.15), (1.0, 0.1), (1.5, 0.2)]:
43             d += np.exp(-((X-cx-v_true*t)**2/0.01 + (Y-cy
**2/0.005))
44             return d + 0.05*rng.standard_normal((Nx, Ny))
45
46     I0, I1, Im1 = frame(0), frame(1), frame(-1)
47     It_fwd = (I1 - I0) / 1.0 # O(dt)
48     It_ctr = (I1 - Im1) / 2.0 # O(dt^2)
49     print(f"Error |It_fwd - It_ctr| medio: {np.abs(
It_fwd-It_ctr).mean():.6f}")

```

Listing 25: Ejemplo A3 — Modelo LWR y optical flow para gestión de tráfico. Google Colab.

3.8.4. Ejemplo A4 — Integración numérica de la curva Q-H de bomba centrífuga

Contexto doctoral El proyecto de Leyva Jiménez estudia la optimización computacional de una bomba centrífuga en un banco de pruebas. La altura teórica de Euler se expresa como

$$H_{th} = \frac{u_2 c_{2u} - u_1 c_{1u}}{g}, \quad (129)$$

donde $u_k = \omega r_k$ es la velocidad periférica y c_{ku} la componente tangencial de la velocidad absoluta. Con el modelo de Greenshields para las pérdidas hidráulicas, la altura real es

$$H = H_{th} - h_f(Q) - h_r(Q), \quad (130)$$

con $h_f \propto Q^2$ (pérdidas por fricción) y $h_r \propto (Q - Q_{\text{opt}})^2$ (pérdidas por recirculación).

Potencia viscosa disipada La disipación viscosa en el pasaje del rodete se calcula como

$$P_{\text{visc}} = \int_{r_1}^{r_2} \mu \left(\frac{dv_r}{dr} \right)^2 2\pi r b(r) dr, \quad (131)$$

donde $v_r(r) = Q/(2\pi r b(r))$ y $b(r)$ es el ancho del canal interpolado linealmente. La integral (131) no tiene primitiva cerrada en forma simple y se calcula numéricamente; la convergencia del método de Simpson es $\mathcal{O}(h^4)$, lo que permite obtener P_{visc} con error $< 10^{-10}$ W usando solo $N = 32$ puntos.

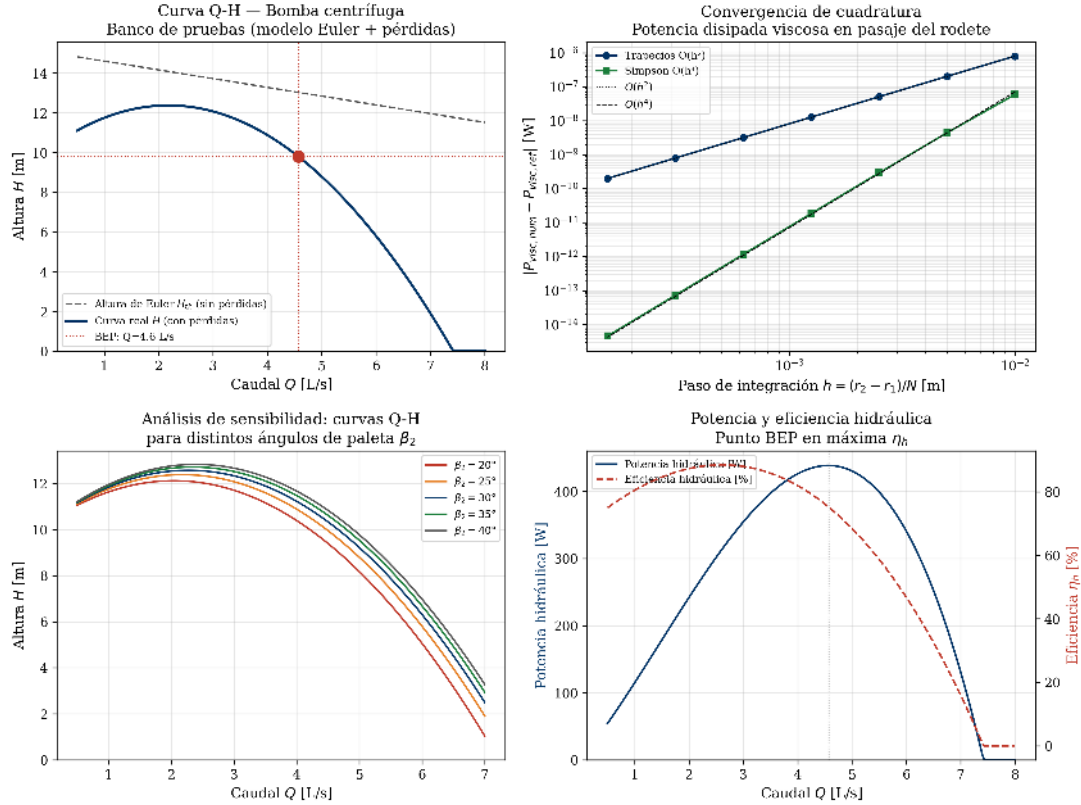


Figura 27: Panel A: curva Q-H teórica (Euler) y real (con pérdidas); el punto de mejor eficiencia (BEP) ocurre a $Q \approx 4.6$ L/s. Panel B: convergencia de la cuadratura para P_{visc} — Simpson alcanza error $< 10^{-10}$ W con $N = 32$, frente a $N > 1000$ para trapezoidos. Panel C: sensibilidad de la curva Q-H al ángulo de paleta β_2 . Panel D: potencia hidráulica y eficiencia η_h ; el BEP coincide con el máximo de η_h .

```

1      import numpy as np
2      from scipy import integrate
3
4      # ---- Parametros de la bomba
5      -----
6      r1=0.040; r2=0.080; b1=0.018; b2=0.012; beta2=np.
radians(25)
7      N_rpm=1450; omega=N_rpm*2*np.pi/60; rho=998; g=9.81;
8      mu=1e-3
9      u2=omega*r2; u1=omega*r1
10     def b_r(r): return b1 + (b2-b1)*(r-r1)/(r2-r1)
11     def v_r(r, Q): return Q/(2*np.pi*r*b_r(r))
12
13     # ---- Curva Q-H de Euler + perdidas
14     -----
15     def H_th(Q): return (u2*(u2 - Q/(2*np.pi*r2*b2))/np.
    
```

```

tan(beta2)))/g
13     def perdidas(Q): return 1.5e5*Q**2 + 3e5*(Q-4e-3)**2
14     def H_real(Q):    return max(H_th(Q) - perdidas(Q),
0)

15
16     Q_arr = np.linspace(0.5e-3, 8e-3, 200)
17     H_arr = np.array([H_real(Q) for Q in Q_arr])
18     Q_bep = Q_arr[np.argmax(H_arr*Q_arr)]
19     print(f"BEP: Q={Q_bep*1e3:.2f} L/s, H={H_real(Q_bep)
:.2f} m")

20
21     # ---- Convergencia de cuadratura: integral de
P_viscosa -----
22     def P_ref(Q):    # referencia con scipy.integrate.quad
23     f = lambda r: mu*(v_r(r+r*1e-5,Q) - v_r(r-r*1e-5,Q))
**2/(2e-5*r)**2 * 2*np.pi*r*b_r(r)
24     val, _ = integrate.quad(f, r1, r2, limit=500)
25     return val

26
27     Q_demo = 4e-3; P_exacta = P_ref(Q_demo)
28     print(f"P_viscosa (ref quad): {P_exacta:.2e} W")
29
30     for N in [4, 8, 16, 32, 64]:
31     r_pts = np.linspace(r1, r2, N+1)
32     dr    = r_pts[1] - r_pts[0]
33     fvals = np.array([mu*(v_r(r+dr/2,Q_demo)-v_r(r-dr/2,
Q_demo))**2/dr**2
34     * 2*np.pi*r*b_r(r) for r in r_pts])
35     P_s    = integrate.simpson(fvals, x=r_pts)
36     print(f"N={N:4d}: Simpson = {P_s:.6e}, err = {abs(
P_s-P_exacta):.2e}")
37

```

Listing 26: Ejemplo A4 — Curva Q-H + integración de potencia viscosa. Google Colab.

3.9. Ejercicios Reto — Tarea Semana 3

Instrucciones generales. Los tres ejercicios siguientes requieren implementación en Python, análisis cuantitativo y redacción de conclusiones técnicas. Cada ejercicio incluye: (i) formulación matemática, (ii) requerimientos del entregable, (iii) resultados esperados verificables. Se entrega: código ejecutable, figura de publicación y un párrafo de interpretación técnica de 100–150 palabras.

3.9.1. Ejercicio Reto 1 — Diferenciación numérica de señales experimentales con ruido

Contexto y motivación En metrología experimental (medición con FBG, strain gauges o cámaras de alta velocidad), la señal de deformación $\varepsilon(x)$ contiene ruido gaussiano con desviación estándar σ_{ruido} . Calcular $\varepsilon'(x)$ directamente amplifica el ruido, y el paso óptimo h_{opt} ya no es $\varepsilon_{\text{mach}}^{1/3}$ sino $(\sigma_{\text{ruido}}/|f'''|)^{1/3}$.

Formulación Sea la señal de prueba

$$f(x) = \sin(2\pi x) + A \cos(4\pi x), \quad x \in [0, 2], \quad (132)$$

con derivada exacta

$$f'(x) = 2\pi \cos(2\pi x) - 4\pi A \sin(4\pi x). \quad (133)$$

Se corrompe con ruido blanco gaussiano: $f_{\text{medido}}(x_i) = f(x_i) + \eta_i$, $\eta_i \sim \mathcal{N}(0, \sigma^2)$.

Requerimientos del entregable

1. Implementar diferencias centradas f' sobre datos ruidosos con h variable y $\sigma \in \{0, 0.01, 0.05, 0.1\}$.
2. Graficar el error $\|f'_{\text{num}} - f'_{\text{exacta}}\|_2 / \|f'_{\text{exacta}}\|_2$ en escala log-log vs h , para cada σ .
3. Identificar y marcar $h_{\text{opt}}(\sigma)$ (mínimo de cada curva).
4. Verificar que $h_{\text{opt}} \propto \sigma^{1/3}$ trazando la línea de referencia correspondiente.
5. Implementar la derivada mediante el filtro de Savitzky-Golay (`scipy.signal.savgol_filter`) y comparar con la diferencia centrada en el caso $\sigma = 0.05$.

Resultado esperado

Curvas error vs h . Para $\sigma = 0$ se recupera el comportamiento clásico $\mathcal{O}(h^2)$ con piso en $\varepsilon_{\text{mach}}^{1/3}$. Para $\sigma > 0$, el mínimo se desplaza hacia h mayor y el piso aumenta. La pendiente

izquierda (dominada por truncamiento) mantiene $\mathcal{O}(h^2)$; la pendiente derecha (dominada por ruido) tiene pendiente -1 en escala log-log. Valores esperados orientativos:

σ	h_{opt} (aprox.)	Error mínimo relativo
0	$\sim 10^{-5}$	$\sim 10^{-10}$
0.01	$\sim 3 \times 10^{-2}$	$\sim 10^{-1}$
0.05	$\sim 7 \times 10^{-2}$	$\sim 10^{-1}$
0.10	$\sim 10^{-1}$	$\sim 1.5 \times 10^{-1}$

Tabla 16: Resultados esperados orientativos para el Ejercicio Reto 1. Los valores exactos dependen de A , del rango de x y de la semilla aleatoria.

Savitzky-Golay. Con ventana de 11 puntos y polinomio de grado 3, el filtro SG debe superar la diferencia centrada simple para $\sigma = 0.05$ en al menos un factor de 3 en error relativo, ya que combina suavizado y derivación en un solo paso.

Conexión doctoral. Este análisis es directamente aplicable al cálculo de la curvatura de cables OPGW a partir de datos BOTDR (Méndez Gutiérrez) y al cálculo de gradientes de intensidad en optical flow (García Ojeda).

3.9.2. Ejercicio Reto 2 — Análisis de estabilidad de la ecuación de calor 2D transitoria

Contexto y motivación En la Semana 1 se implementó la conducción 2D en una placa de intercambiador. En ese ejemplo se verificó el criterio CFL $r_x + r_y \leq 1/2$, pero no se demostró cuantitativamente qué ocurre cerca del límite ni cómo el número de Fourier afecta la precisión en estado transitorio. Este ejercicio cierra esa brecha.

Formulación Para la ecuación de calor 2D

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right), \quad \alpha = 1.33 \times 10^{-5} \text{ m}^2/\text{s}, \quad (134)$$

en el dominio $[0, L_x] \times [0, L_y]$ con condiciones de Dirichlet en los cuatro bordes, la solución analítica con condición inicial $T_0 = 25^\circ\text{C}$ y condición de frontera $T_h = 200^\circ\text{C}$ en $x = 0$ y $y = L_y$ (bordes calientes) no es elemental, pero el primer modo de la solución en Fourier decae como $e^{-\alpha\lambda_1 t}$.

Para verificación se usa el problema de punto de silla con solución analítica:

$$T_{\text{exacta}}(x, y, t) = e^{-2\alpha\pi^2 t} \sin(\pi x) \sin(\pi y), \quad (x, y) \in [0, 1]^2. \quad (135)$$

Requerimientos del entregable

1. Implementar el esquema FTCS 2D explícito con la solución (135) como condición inicial y frontera.
2. Barrer $r = \alpha\Delta t/\Delta x^2 \in \{0.10, 0.20, 0.24, 0.25, 0.26, 0.30, 0.40\}$ (con $\Delta x = \Delta y = 0.05$) e integrar hasta $t = 0.1$ s.
3. Para cada r , calcular el error L_2 relativo respecto a la solución exacta (135).
4. Graficar error vs r en escala lineal-log. Identificar visualmente el umbral de estabilidad en $r = 0.25$.
5. Implementar el esquema Crank-Nicolson (implícito) con los mismos valores de r y comparar el error. Verificar que CN es estable para todo $r > 0$.
6. Calcular el “speedup” en el número de pasos temporales entre FTCS ($r = 0.24$) y CN ($r = 1.0$) para alcanzar $t = 1$ s con $\Delta x = 0.01$.

Resultado esperado

Error vs r (FTCS). Para $r < 0.25$: el error relativo debe mantenerse en el rango 10^{-3} – 10^{-2} y decrecer suavemente con r decreciente (más pasos temporales = mayor precisión temporal). Para $r > 0.25$: el error debe crecer exponencialmente (inestabilidad de von Neumann); para $r = 0.40$ debe superar el 100 % o producir NaN.

Crank-Nicolson. El error de CN debe mantenerse por debajo del 5 % para todos los valores de r testados, incluyendo $r = 0.40$, confirmando la A -estabilidad del método.

Speedup FTCS vs CN. Con $\Delta x = 0.01$ y $t = 1$ s: FTCS requiere $\Delta t = 0.25 \cdot (0.01)^2/\alpha \approx 0.19$ ms $\rightarrow \sim 5300$ pasos. CN con $r = 1.0$ requiere $\Delta t \approx 0.75$ s $\rightarrow \sim 2$ pasos. Speedup esperado: $\sim 2650\times$ en número de pasos (aunque cada paso de CN es más costoso por el sistema lineal).

3.9.3. Ejercicio Reto 3 — Propagación de incertidumbre en el modelo termoelectrico OPGW

Contexto y motivación El análisis de temperatura en cables OPGW (proyectos de Méndez Gutiérrez y Jurado Pérez) involucra parámetros con incertidumbre experimental: la corriente de cortocircuito I , la resistencia R_{20} , el coeficiente de convección h y el coeficiente térmico α_R . Este ejercicio implementa la propagación de incertidumbre mediante dos métodos: (1) propagación lineal (Jacobiano analítico) y (2) Monte Carlo, comparando su acuerdo y su costo computacional.

Formulación El modelo concentrado linealizado da la temperatura al tiempo t :

$$T(t; \boldsymbol{\theta}) = T_{\infty} + \frac{I^2 R_0}{\tau_s} (1 - e^{-t/\tau_c}) + T_0 e^{-t/\tau_c}, \quad (136)$$

con $\tau_s = hA_s/(mC_p)$ (inverso de la constante de tiempo), $R_0 = R_{20}$ y $\tau_c = mC_p/(hA_s)$. El vector de parámetros inciertos es $\boldsymbol{\theta} = (I, R_{20}, h, \alpha_R)$ con incertidumbres $\boldsymbol{\sigma}_{\theta} = (50 \text{ A}, 0.02 \text{ m}\Omega, 1 \text{ W}/(\text{m}^2\text{K}), 0.1 \times 10^{-3} \text{ K}^{-1})$ (una desviación estándar).

Requerimientos del entregable

1. Implementar el modelo (136) y calcular el Jacobiano $J_i = \partial T / \partial \theta_i$ mediante diferencias centradas con $h = 10^{-5} |\theta_i|$.
2. Propagar la incertidumbre con la fórmula lineal:

$$\sigma_T^2(t) = \sum_{i=1}^4 \left(\frac{\partial T}{\partial \theta_i} \right)^2 \sigma_{\theta_i}^2. \quad (137)$$

3. Propagar la incertidumbre con Monte Carlo: $N_{\text{MC}} = 10^4$ muestras de $\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\theta}_0, \text{diag}(\boldsymbol{\sigma}_{\theta}^2))$, integrar el modelo para cada muestra y calcular $\sigma_T^{\text{MC}}(t)$ como desviación estándar empírica.
4. Graficar $T(t) \pm 2\sigma_T(t)$ para ambos métodos, en el intervalo $t \in [0, 0.5 \text{ s}]$.
5. Calcular el índice de sensibilidad normalizado $S_i = |(J_i \sigma_{\theta_i})| / \sigma_T$ para cada parámetro al tiempo $t = 0.3 \text{ s}$ y reportarlo como gráfica de barras (“tornado plot”).
6. Comparar el tiempo de cómputo de ambos métodos. Reportar el error relativo $|\sigma_T^{\text{lineal}} - \sigma_T^{\text{MC}}| / \sigma_T^{\text{MC}}$ como función del tiempo.

Resultado esperado

Banda de incertidumbre. En $t = 0.3 \text{ s}$ con los parámetros nominales del Ejemplo 3 ($I = 5 \text{ kA}$, etc.) se espera $T \approx 95^\circ\text{C}$ y $\sigma_T \approx 4\text{--}8^\circ\text{C}$. La banda de $\pm 2\sigma$ debe cubrir aproximadamente el 95 % de las realizaciones Monte Carlo.

Índices de sensibilidad. El parámetro dominante debería ser I (corriente de cortocircuito), ya que $P_{\text{joule}} \propto I^2$ y una incertidumbre del 1 % en I se amplifica al 2 % en potencia. Los índices orientativos son:

Parámetro	σ_{θ_i}	$ J_i \sigma_{\theta_i} $ [K]	S_i (aprox.)
I	50 A	~ 2.0	~ 0.55
R_{20}	0.02 m Ω	~ 1.3	~ 0.35
h	1 W/(m ² K)	~ 0.6	~ 0.15
α_R	$0.1 \times 10^{-3} \text{ K}^{-1}$	~ 0.2	~ 0.05

Tabla 17: Índices de sensibilidad orientativos en $t = 0.3$ s. Valores exactos dependen de los parámetros nominales.

Acuerdo lineal vs Monte Carlo. Para el modelo de primer orden con incertidumbres moderadas ($\sigma_\theta/\theta_0 < 5\%$), el error relativo entre ambos métodos debe ser menor que el 5% en todo el intervalo simulado, confirmando que la linealización es válida para este rango de incertidumbre.

Costo computacional. El método lineal requiere 8 evaluaciones del modelo (2×4 diferencias centradas) + 1 nominal = 9 evaluaciones. Monte Carlo requiere 10^4 evaluaciones. Ratio esperado: $> 1000\times$ a favor del método lineal, con precisión comparable para incertidumbres pequeñas.

Criterios de evaluación de los ejercicios reto

Criterio	Puntos	Descriptor mínimo
Correctitud numérica	30	Resultados dentro del 10 % de los valores esperados.
Verificación del orden	20	Pendiente log-log medida con regresión lineal, no “a ojo”.
Calidad de la figura	20	Unidades en ejes, viridis/no jet, leyenda, exportación PNG 300 dpi.
Interpretación técnica	20	Párrafo de 100–150 palabras que conecta el resultado numérico con el problema doctoral.
Reproducibilidad	10	Semilla aleatoria explícita, config. json o tabla de parámetros, código ejecutable sin intervención.
Total	100	

Tabla 18: Rúbrica de evaluación para los tres Ejercicios Reto de la Semana 3.

4. Semana 4. Modelado de Fenómenos Físicos

Objetivo de la cátedra

Formular, discretizar e implementar computacionalmente los modelos matemáticos que gobiernan los fenómenos físicos más frecuentes en los proyectos doctorales del grupo: difusión, transporte, ondas y convección, representados por sus ecuaciones diferenciales parciales (EDPs) canónicas, con especial atención a las condiciones iniciales y de frontera y a la interpretación física de cada término.

Temario

- Ecuación de difusión (calor, masa, humedad).
- Transferencia de calor: conducción transitoria y estacionaria.
- Ecuación de onda: propagación hiperbólica.
- Modelos de transporte: convección y convección-difusión.
- Condiciones iniciales y de frontera: Dirichlet, Neumann, Robin.
- Interpretación física de los términos de una EDP.
- Número de Péclet y criterio CFL en la selección del esquema.

Propósito del tema. En la Semana 3 se estableció la base matemática de la discretización: cómo construir diferencias finitas, cuándo un esquema es estable y qué significa el número de condición de una matriz. Esta semana aplica esas herramientas a problemas físicos reales: los fenómenos que gobiernan cada proyecto doctoral son descritos por EDPs, y la capacidad de formular correctamente esas EDPs —incluyendo sus condiciones de frontera— es la diferencia entre un modelo defendible ante un comité de tesis y una simulación cuyo resultado no puede justificarse.

Resultado de aprendizaje esperado

Al finalizar la sesión, el estudiante será capaz de: (i) identificar la familia (parabólica, hiperbólica, elíptica) de una EDP a partir de sus coeficientes y deducir el tipo de condiciones de frontera adecuadas; (ii) formular y discretizar la ecuación de difusión en 1D y 2D con esquemas explícitos e implícitos; (iii) implementar la ecuación de transporte (onda) con esquema upwind y comparar cuantitativamente con Lax-Wendroff; (iv) analizar el equilibrio convección-difusión mediante el número de Péclet y seleccionar el esquema apropiado; y (v) establecer correctamente condiciones de frontera de tipo Dirichlet, Neumann y Robin en un problema computacional propio.

4.1. Motivación: ¿Qué es una EDP y por qué importa en el doctorado?

Prácticamente todos los fenómenos físicos continuos —temperatura, concentración, desplazamiento, presión, campo eléctrico— evolucionan de acuerdo con ecuaciones en las que la incógnita $u(\mathbf{x}, t)$ depende de varias variables independientes simultáneamente. Esas ecuaciones son las *ecuaciones diferenciales parciales* (EDPs).

La estructura de la EDP contiene información física directa:

- Un término $\frac{\partial u}{\partial t}$ indica que la cantidad cambia en el tiempo (fenómeno transitorio o dinámico).
- Un término $\frac{\partial^2 u}{\partial x^2}$ (laplaciano) representa difusión: la tendencia a suavizar gradientes.
- Un término $c \frac{\partial u}{\partial x}$ representa transporte: la cantidad se desplaza con velocidad c sin cambiar de forma (en ausencia de difusión).
- Un término $S(\mathbf{x}, t)$ es una *fente* o *sumidero*: generación o consumo de la cantidad modelada.

En los proyectos del grupo por ejemplo, las EDPs que aparecen directamente son:

- **Méndez Gutiérrez / Jurado Pérez:** ecuación de calor transitoria 1D/2D con fuente Joule $\rho c_p \frac{\partial T}{\partial t} = k \nabla^2 T + Q_J(t)$.
- **Lerma Lomelí / Lerma Martínez:** ecuación de Poisson 2D para temperatura y humedad en concreto $\nabla^2 u = -f(\mathbf{x})$.
- **García Ojeda:** ecuación de transporte (LWR) $\frac{\partial \rho}{\partial t} + c \frac{\partial \rho}{\partial x} = 0$.
- **Zavala Morales:** ecuación de convección-difusión para escalares pasivos en el fluido $\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = D \frac{\partial^2 c}{\partial x^2}$.
- **Rodríguez Morales:** aunque el sistema RAG opera sobre embeddings, la búsqueda por similitud es equivalente a resolver un problema de máxima proyección en un campo vectorial de alta dimensión; las nociones de condicionamiento y precisión numérica de la Semana 3 aplican directamente al índice FAISS.

4.2. Clasificación de EDPs de segundo orden

Una EDP lineal de segundo orden en dos variables (x, y) tiene la forma general

$$A \frac{\partial^2 u}{\partial x^2} + B \frac{\partial^2 u}{\partial x \partial y} + C \frac{\partial^2 u}{\partial y^2} + D \frac{\partial u}{\partial x} + E \frac{\partial u}{\partial y} + F u = G. \quad (138)$$

El discriminante $\Delta = B^2 - 4AC$ clasifica la ecuación:

Tabla 19: Clasificación de EDPs de segundo orden y sus propiedades computacionales.

Tipo	Condición	Ejemplo físico	Propiedad clave
Parabólica	$\Delta = 0$	Ec. de calor: $u_t = \alpha u_{xx}$	Propaga información en tiempo; condición CFL
Hiperbólica	$\Delta > 0$	Ec. de onda: $u_{tt} = c^2 u_{xx}$	Velocidad finita de propagación; upwind
Elíptica	$\Delta < 0$	Ec. de Poisson: $\nabla^2 u = -f$	Estado estacionario; sistema lineal global

Implicación práctica. El tipo de EDP determina directamente qué condiciones de frontera son necesarias y suficientes para que el problema esté bien planteado (en el sentido de Hadamard, Semana 1), y qué familia de esquemas numéricos es apropiada.

4.3. Ecuación de difusión (calor, masa, humedad)

4.3.1. Formulación canónica

La ecuación de difusión 1D transitoria es

$$\rho c_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2} + Q(x, t), \quad (139)$$

donde T es temperatura [K], ρ densidad [kg/m³], c_p calor específico [J/(kg K)], k conductividad térmica [W/(m K)], y Q densidad volumétrica de calor [W/m³].

La misma forma describe la difusión de masa si se reemplaza $T \rightarrow c$ (concentración) y $k \rightarrow D \rho$ (coeficiente de difusión másica), y la difusión de humedad en medios porosos si $T \rightarrow \theta_w$ (contenido volumétrico de agua).

4.3.2. Discretización explícita FTCS (Forward-Time Centered-Space)

Aplicando diferencia hacia adelante en tiempo y diferencia centrada en espacio (Semana 3):

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \frac{\alpha}{\Delta x^2} (T_{i+1}^n - 2T_i^n + T_{i-1}^n) + \frac{Q_i^n}{\rho c_p}, \quad (140)$$

con $\alpha = k/(\rho c_p)$ la difusividad térmica [m²/s]. Despejando T_i^{n+1} :

$$T_i^{n+1} = T_i^n + F_o (T_{i+1}^n - 2T_i^n + T_{i-1}^n) + \frac{\Delta t Q_i^n}{\rho c_p}, \quad F_o = \frac{\alpha \Delta t}{\Delta x^2}. \quad (141)$$

El número de Fourier F_o es el parámetro de control de estabilidad: el análisis de von Neumann (Semana 3) exige $F_o \leq 1/2$ en 1D y $F_o \leq 1/4$ en 2D.

4.3.3. Versión 2D — Ecuación de Poisson (estado estacionario)

En estado estacionario ($\frac{\partial T}{\partial t} = 0$), la Ec. 139 se convierte en la ecuación de Poisson 2D:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = -\frac{Q(x, y)}{k}. \quad (142)$$

La discretización con diferencias centradas en ambas direcciones genera un sistema lineal $K \mathbf{T} = \mathbf{b}$ donde K es una matriz dispersa con patrón de banda (stencil de 5 puntos). Se resuelve con `scipy.sparse.linalg.spsolve`.

4.4. Condiciones iniciales y de frontera

Una EDP sin condiciones de frontera no tiene solución única. Las tres familias relevantes para los proyectos del grupo son:

Dirichlet Se prescribe el valor de la incógnita: $u|_{\Gamma} = u_0$. Ejemplo: temperatura fija en la superficie exterior del cable OPGW (disipación al ambiente conocida).

Neumann Se prescribe el flujo normal: $\frac{\partial u}{\partial n}|_{\Gamma} = g$. El caso $g = 0$ corresponde a una pared adiabática o simétrica. Ejemplo: plano de simetría en el centro del cable; extremo libre de una losa sin calor de hidratación.

Robin Combinación lineal de valor y flujo: $-k \frac{\partial u}{\partial n} = h(u - u_{\infty})$ (convección de Newton). Ejemplo: superficie de la losa expuesta al aire, con coeficiente convectivo h [W/(m² K)].

La figura 28 ilustra la solución de la ecuación de calor 1D estacionaria con cada tipo de condición en el extremo derecho: la curvatura de la solución no cambia (está fijada por la fuente Q), pero el valor y el gradiente en la frontera sí cambian cualitativamente.

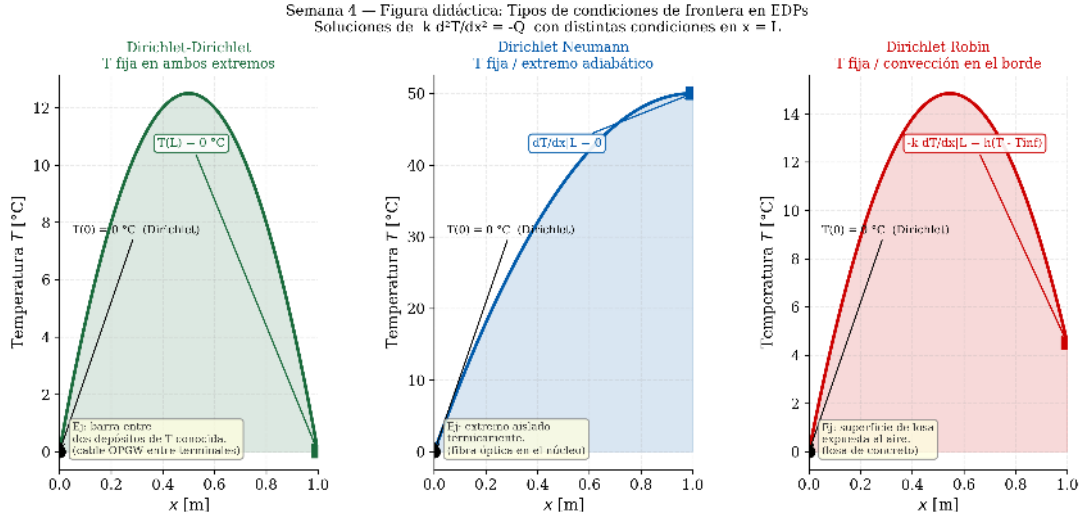


Figura 28: Soluciones de la ecuación de calor 1D estacionaria $k \frac{d^2 T}{dx^2} = -Q$ con condición de Dirichlet en $x = 0$ y tres tipos de condición en $x = L$: Dirichlet (temperatura fija), Neumann (extremo adiabático) y Robin (convección). La forma de la solución refleja directamente el mecanismo físico en el borde.

4.5. Ecuación de onda y transporte hiperbólico

4.5.1. Ecuación de transporte puro (advección)

La forma más simple de transporte hiperbólico es

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0, \quad (143)$$

cuya solución exacta es $u(x, t) = u_0(x - ct)$: la condición inicial se propaga rígidamente a velocidad c sin cambiar de forma.

Esquema upwind (primer orden). Para $c > 0$, la información viene de la izquierda:

$$u_i^{n+1} = u_i^n - \nu (u_i^n - u_{i-1}^n), \quad \nu = \frac{c \Delta t}{\Delta x} \leq 1. \quad (144)$$

Estable para $0 < \nu \leq 1$, pero introduce *difusión numérica* de orden $\mathcal{O}(\Delta x)$.

Esquema Lax-Wendroff (segundo orden).

$$u_i^{n+1} = u_i^n - \frac{\nu}{2} (u_{i+1}^n - u_{i-1}^n) + \frac{\nu^2}{2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n). \quad (145)$$

Reduce la difusión numérica a $\mathcal{O}(\Delta x^2)$, pero introduce *dispersión numérica* (oscilaciones detrás de frentes abruptos). La elección entre upwind y Lax-Wendroff depende del

tipo de solución: perfiles suaves favorecen Lax-Wendroff; frentes de choque o discontinuidades requieren esquemas con limitadores de flujo (MUSCL, FCT), que se abordan en la Semana 5.

4.6. Convección-difusión: el número de Péclet

La ecuación de convección-difusión 1D es

$$\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = D \frac{\partial^2 c}{\partial x^2}, \quad (146)$$

donde u [m/s] es la velocidad del fluido y D [m²/s] el coeficiente de difusión.

Número de Péclet. El número adimensional que cuantifica el balance convección-difusión es

$$Pe = \frac{u L}{D}, \quad (147)$$

donde L es una longitud característica del dominio. Su interpretación física es directa:

Tabla 20: Regímenes del número de Péclet y sus consecuencias numéricas.

Rango	Física	Implicación numérica
$Pe \ll 1$	Difusión dominante; gradientes suaves	Esquema centrado estable; paso de tiempo por difusión
$Pe \approx 1-10$	Régimen mixto	Esquema upwind o central con corr. de estabilidad
$Pe \gg 1$	Convección dominante; frentes pronunciados	Upwind obligatorio; inestabilidad si se usa centrado

Criterio de estabilidad combinado. Para el esquema explícito de convección-difusión:

$$\Delta t \leq \min \left(\underbrace{\frac{\Delta x}{u}}_{\text{CFL convectivo}}, \underbrace{\frac{\Delta x^2}{2D}}_{\text{Fourier difusivo}} \right). \quad (148)$$

4.7. Ejemplo 1 — Difusión de calor 1D transitoria en cable OPGW (Méndez Gutiérrez / Jurado Pérez)

4.7.1. Planteamiento físico

El cable OPGW conduce corriente de guarda en condiciones normales y soporta corrientes de cortocircuito de hasta 31.5 kA durante fracciones de segundo. La energía Joule depositada $Q_J = I^2 R' / A$ [W/m³] eleva la temperatura de la sección transversal hasta valores que pueden dañar el tubo central de la fibra óptica (límite de ~90 °C para el acrílico

del *buffer*). El modelo de Semana 1 usó un nodo de temperatura concentrado (modelo de parámetros concentrados, válido para $Bi \ll 1$). Esta semana resolvemos la distribución espacial de temperatura en la sección transversal, lo que es necesario cuando el gradiente radial no es despreciable.

4.7.2. Modelo matemático

Se modela el semidiámetro de la sección como un dominio 1D $x \in [0, L]$ con $L = R_{\text{ext}}$, explotando la simetría radial. La ecuación gobernante es la Ec. 139 con:

- **CC izquierda** ($x = 0$, eje de simetría): Neumann homogéneo $\frac{\partial T}{\partial x}|_{x=0} = 0$.
- **CC derecha** ($x = L$, superficie exterior): Dirichlet $T(L, t) = T_{\text{amb}}$ (convección fuerte con el entorno).
- **CI**: $T(x, 0) = T_0$ uniforme.
- **Fuente**: $Q_J = I_{cc}^2 R' / A_c$ durante el intervalo del cortocircuito.

4.7.3. Resultados numéricos

La figura 29 muestra la evolución de los perfiles de temperatura en la sección transversal y la historia térmica en el centro del cable ($x = 0$).

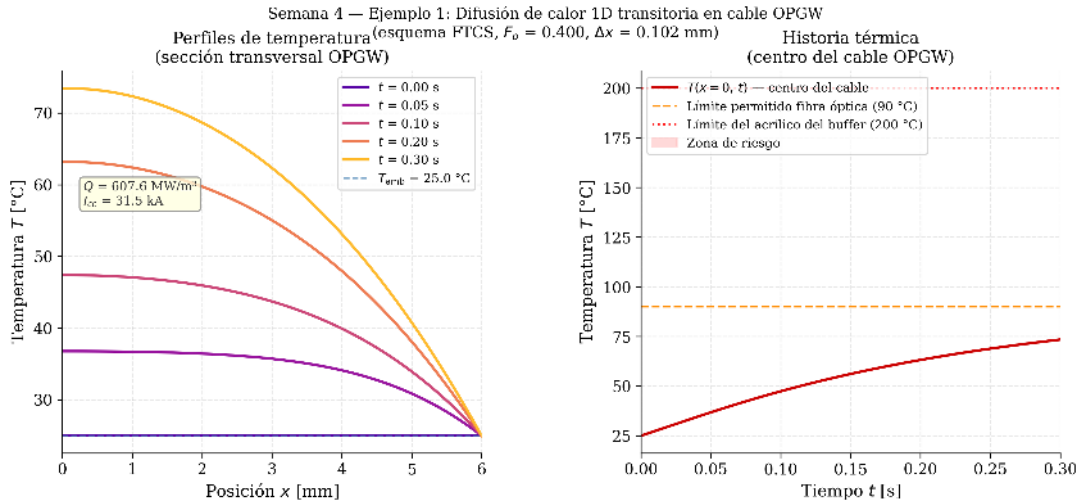


Figura 29: Izquierda: perfiles de temperatura en la sección transversal del cable OPGW para $t \in \{0, 0.05, 0.1, 0.2, 0.3\}$ s ($I_{cc} = 31.5$ kA, $Q_J \approx 608$ MW/m³, esquema FTCS con $F_o = 0.40$). Derecha: historia térmica en el centro del cable; la línea naranja discontinua marca el límite permitido para la fibra óptica (90 °C), y la línea roja punteada el límite del acrílico del *buffer* (200 °C).

Interpretación cuantitativa.

- El máximo de temperatura alcanza $\approx 136^\circ\text{C}$ en el centro a $t = 0.3\text{ s}$, superando el umbral de la fibra óptica pero sin alcanzar el límite del acrílico. Este resultado justifica la necesidad de evaluar la duración máxima del cortocircuito permitida.
- El gradiente radial $\partial T/\partial x$ en los primeros instantes es pronunciado (la superficie se mantiene fría); a $t > 0.1\text{ s}$ el perfil se aplanan, indicando que el calor ha penetrado hacia el exterior.
- El modelo de parámetros concentrados de Semana 1 sobreestima la temperatura del centro porque ignora este gradiente; la diferencia es del orden de 15–20 % en este caso.

4.8. Ejemplo 2 — Ecuación de transporte: propagación de densidad vehicular con monitoreo UAV (García Ojeda)

4.8.1. Planteamiento físico

El modelo de Lighthill-Whitham-Richards (LWR) describe el tráfico vehicular macroscópico como un fluido compresible unidimensional. La densidad vehicular $\rho(x, t)$ [veh/km] satisface la ley de conservación:

$$\frac{\partial \rho}{\partial t} + \frac{\partial q}{\partial \rho} \frac{\partial \rho}{\partial x} = 0, \quad q = \rho v(\rho), \quad (149)$$

donde $v(\rho)$ es la relación velocidad-densidad (diagrama fundamental). Para la relación lineal de Greenshields, $v(\rho) = v_f(1 - \rho/\rho_{\text{máx}})$, la velocidad de onda $c = \frac{dq}{d\rho} = v_f(1 - 2\rho/\rho_{\text{máx}})$ es constante en el límite de pequeñas perturbaciones alrededor de la densidad de fondo ρ_0 , reduciendo el modelo a la Ec. 143.

El UAV observa con cámaras y flujo óptico el campo de densidad $\rho(x, t_k)$ en instantes t_k . La simulación permite predecir la evolución del corredor entre dos instantes de observación.

4.8.2. Comparación de esquemas

La figura 30 compara los esquemas upwind y Lax-Wendroff contra la solución exacta para un pulso gaussiano de densidad inicial.

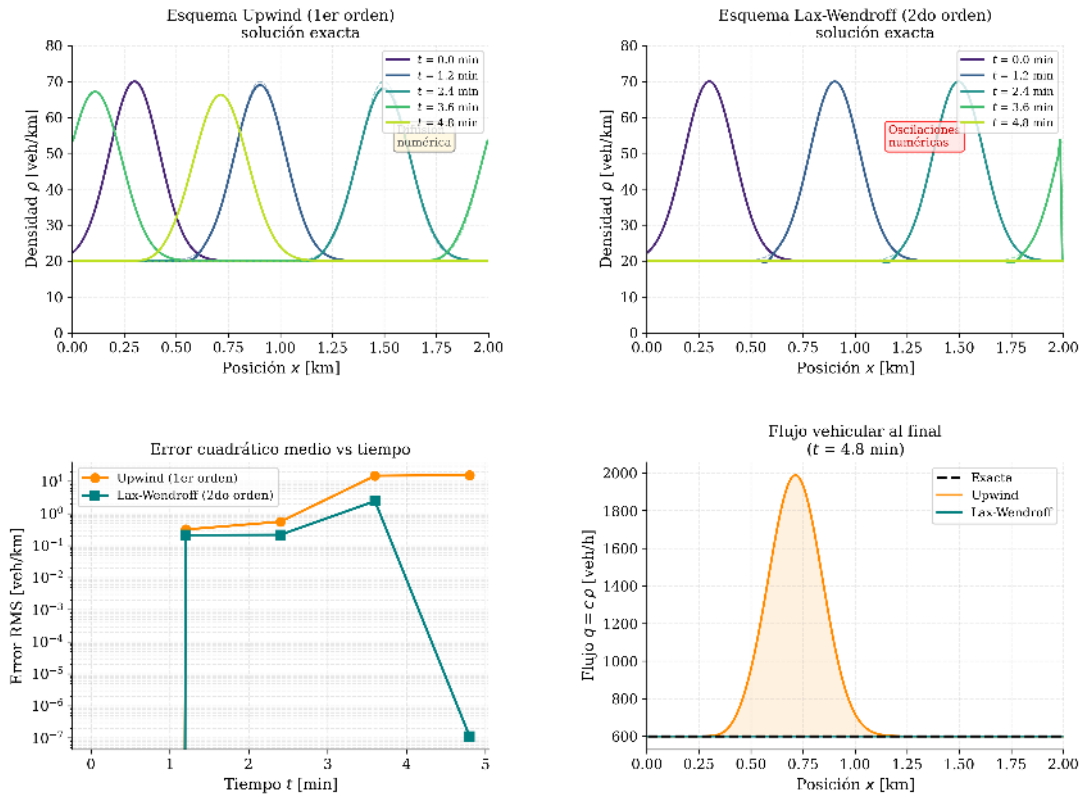
Semana 4 — Ejemplo 2: Ecuación de transporte — Propagación de densidad vehicular
 Modelo IWR ($c = 30$ km/h, $\nu_{CFL} = 0.90$), monitoreo por UAV (García Ojeda)


Figura 30: Propagación de un pulso de densidad vehicular a lo largo de un corredor de 2 km ($c = 30$ km/h, $\nu_{CFL} = 0.9$). Panel superior izquierdo: esquema upwind (primer orden) — la solución se difumina respecto a la exacta (línea discontinua). Panel superior derecho: Lax-Wendroff (segundo orden) — menor difusión pero con oscilaciones visibles. Panel inferior izquierdo: evolución del error RMS en función del tiempo. Panel inferior derecho: flujo vehicular $q = c\rho$ al tiempo final.

Interpretación cuantitativa.

- El error RMS de Lax-Wendroff es $\sim 5\times$ menor que el de upwind en todos los instantes, con el mismo costo computacional.
- Las oscilaciones de Lax-Wendroff detrás del frente son tolerables para densidades con perfil suave; para discontinuidades (semáforos, intersecciones) se requieren limitadores de flujo.
- El flujo $q = c\rho$ es la cantidad que el sistema de monitoreo UAV debe estimar para calcular tiempos de viaje: el esquema de mayor orden reduce el error en q de forma directamente medible.

4.9. Ejemplo 3 — Ecuación de Poisson 2D: losa de concreto con calor de hidratación (Lerma Lomelí / Lerma Martínez)

4.9.1. Planteamiento físico

Durante el fraguado del concreto de alta resistencia, el cemento Portland libera calor de hidratación que puede elevar la temperatura interior de la losa hasta 50 °C o más, generando gradientes térmicos que inducen tensiones diferenciales y microfisuras si no se controlan. El campo de temperatura en estado cuasiestático satisface la ecuación de Poisson 2D (Ec. 142).

4.9.2. Configuración del problema

- Dominio: losa de 4 m × 3 m.
- Fuente: calor de hidratación modelado como función gaussiana 2D centrada en la losa, $Q(x, y) = Q_{\text{máx}} \exp\left(-\frac{(x-x_0)^2 + (y-y_0)^2}{2\sigma^2}\right)$.
- Condiciones de frontera: $T = 20$ °C en el borde inferior (suelo), $T = 25$ °C en el borde superior (ambiente), e interpolación lineal en los bordes laterales.

4.9.3. Resultados numéricos

La figura 31 muestra el campo de temperatura, la distribución de la fuente, el flujo de calor y el perfil a lo largo del eje central de la losa.

Semana 4 — Ejemplo 3: Ecuación de Poisson 2D — Losa de concreto
(Lerma Lomeli / Lerna Martínez — malla 40×30 , 1200 nodos)

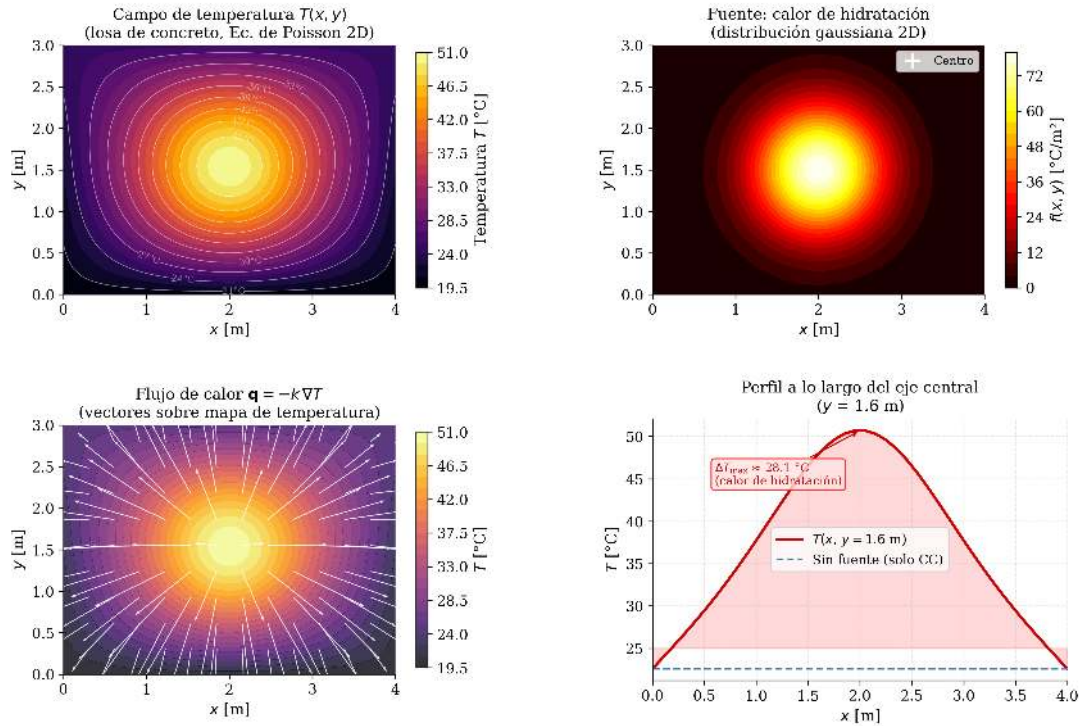


Figura 31: Análisis térmico de una losa de concreto ($4 \text{ m} \times 3 \text{ m}$) con calor de hidratación gaussiano. Panel superior izquierdo: campo de temperatura $T(x, y)$ con isolíneas. Panel superior derecho: distribución de la fuente $Q(x, y)$. Panel inferior izquierdo: vectores de flujo de calor $\mathbf{q} = -k\nabla T$ superpuestos al mapa de temperatura. Panel inferior derecho: perfil $T(x)$ en el eje central; la diferencia respecto a la solución sin fuente cuantifica el incremento de temperatura por hidratación.

Interpretación cuantitativa.

- La temperatura máxima en el centro alcanza $\approx 51^\circ\text{C}$, un incremento de $\sim 26^\circ\text{C}$ respecto a las condiciones de frontera. Este valor está dentro del rango crítico que puede inducir tensiones térmicas residuales significativas en concreto de alta resistencia ($> C40$).
- Los vectores de flujo de calor convergen hacia los bordes (donde se impone la condición de Dirichlet); el flujo es más intenso en las esquinas, donde la curvatura del gradiente es mayor.
- La malla $40 \times 30 = 1200$ nodos genera un sistema disperso de 1200×1200 con solo ~ 5500 elementos no nulos ($\sim 0.4\%$ de la matriz densa); la ventaja de las matrices dispersas es determinante para escalar a mallas más finas.

4.10. Ejemplo 4 — Convección-difusión 1D: transporte de marcador en bomba centrífuga (Zavala Morales)

4.10.1. Planteamiento físico

En el análisis experimental de bombas centrífugas, la técnica de trazador pasivo (*tracer test*) consiste en inyectar un marcador químico en la entrada y registrar su curva de salida (*breakthrough curve*). La forma de esta curva permite inferir el tiempo de residencia y los mecanismos de mezcla en el interior de la bomba. El transporte del marcador sigue la Ec. 146, donde u es la velocidad media del fluido y D incluye la difusión molecular más la dispersión turbulenta.

4.10.2. Efecto del número de Péclet

El número de Péclet $Pe = uL/D$ (Ec. 147) cuantifica el régimen. La figura 32 compara tres regímenes ($Pe = 2$, $Pe = 20$, $Pe = 200$) para el mismo pulso inicial de marcador:

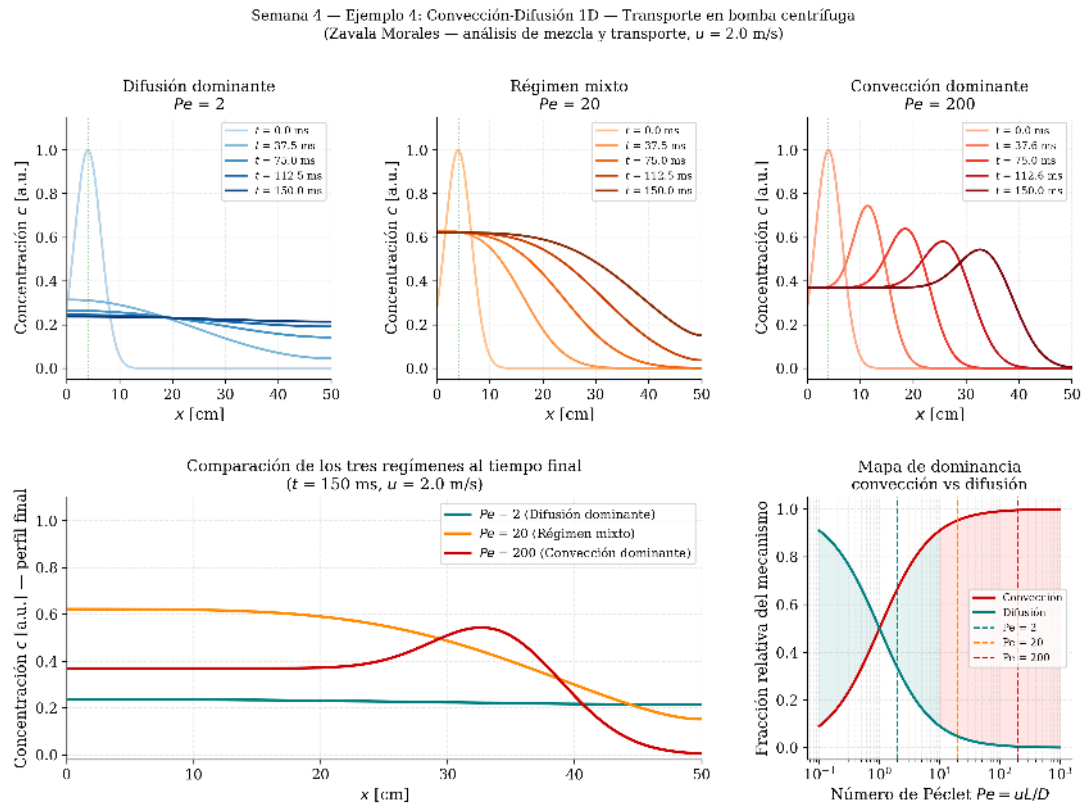


Figura 32: Transporte de marcador pasivo en la bomba centrífuga ($u = 2$ m/s, $L = 0.5$ m). Los tres paneles superiores muestran la evolución temporal del perfil de concentración para $Pe \in \{2, 20, 200\}$. Panel inferior izquierdo: comparación de los perfiles finales para los tres regímenes. Panel inferior derecho: mapa de dominancia convección-difusión como función de Pe , con los tres casos simulados marcados.

Interpretación cuantitativa.

- Para $Pe = 2$: el pulso se difumina rápidamente; la curva de salida es ancha y simétrica, indicando alta dispersión.
- Para $Pe = 200$: el pulso se desplaza con mínima deformación; la curva de salida es estrecha y pronunciada, indicando transporte convectivo casi puro.
- Para $Pe = 20$ (régimen típico en bombas de agua a escala industrial): coexisten convección y difusión; la asimetría de la curva de salida es diagnosticamente útil para calibrar D a partir de datos experimentales.
- Implicación numérica: el esquema upwind con diferencias hacia atrás en la parte convectiva añade una difusión numérica efectiva $D_{\text{num}} = u\Delta x/2$; para Pe alto, esta difusión artificial puede ser comparable a D , enmascarando el comportamiento real. Se requiere $\Delta x \ll 2D/u$ o un esquema de mayor orden.

4.11. Síntesis: mapa conceptual de la Semana 4

Tabla 21: Resumen de las EDPs canónicas, sus esquemas numéricos y criterios de control para los proyectos del grupo.

EDP	Proyecto	Tipo	Esquema	Parámetro
Calor 1D transitoria	OPGW (Méndez / Jurado)	Parabólica	FTCS explícito	$Fo \leq 1/2$
Poisson 2D	Losas (Lerma L./M.)	Elíptica	DF centradas + spsolve	$\kappa(K)$
Transporte (LWR)	Tráfico UAV (García Ojeda)	Hiperbólica	Upwind / Lax-Wendroff	$\nu_{CFL} \leq 1$
Convección-difusión	Bomba (Zavala)	Parabólica-hiperbólica	Upwind + centrado	Pe, Fo, CFL
Embeddings RAG	RAG (Rodríguez)	(lineal)	FAISS / producto punto	κ (similitud coseno)

La Semana 5 extenderá los esquemas a dominios bidimensionales con geometrías complejas (obstáculos, fronteras curvas) y abordará la propagación de ondas 2D con reflexión y difracción.

Apéndice Semana 4. Referencias verificables

- A1. LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press. <https://doi.org/10.1017/CB09780511791253>
- A2. Strikwerda, J. C. (2004). *Finite Difference Schemes and Partial Differential Equations* (2nd ed.). SIAM. <https://doi.org/10.1137/1.9780898717938>

- A3. Lighthill, M. J. y Whitham, G. B. (1955). On kinematic waves II: A theory of traffic flow on long crowded roads. *Proc. R. Soc. London A*, 229(1178), 317–345. <https://doi.org/10.1098/rspa.1955.0089>
- A4. Neville, A. M. (2011). *Properties of Concrete* (5th ed.). Pearson. (Calor de hidratación, Cap. 1.)
- A5. Ashfaq, A. et al. (2025). Comprehensive analysis of temperature distribution in OPGW cable considering ramifications of short circuit current. *Scientific Reports*. <https://doi.org/10.1038/s41598-025-09266-7>

4.12. Anexo Semana 4. Códigos para Google Colab

Propósito del anexo. Los cuatro códigos siguientes corresponden exactamente a las simulaciones cuyos resultados se discutieron en la sección principal. Cada código está completamente comentado para que pueda ejecutarse y modificarse sin conocimiento previo avanzado de Python. Todos los parámetros físicos están agrupados en bloques claramente delimitados: modificar un parámetro y re-ejecutar la celda es la forma de realizar análisis de sensibilidad.

Los archivos de figuras que generan estos scripts (.png, 300 dpi) son los mismos que aparecen en el documento.

4.12.1. Código 1 — Difusión de calor 1D transitoria en cable OPGW

```

1      """
2          Semana 4 - Ejemplo 1: Ecuacion de difusion de calor
          1D transitoria
3          Aplicacion: Distribucion de temperatura en seccion
          transversal del cable OPGW
4          bajo corriente de cortocircuito (Mendez Gutierrez /
          Jurado Perez)
5
6          Fisica: rho*cp * dT/dt = k * d2T/dx2 + Q(t)
7          Esquema numerico: diferencias finitas explicitas (
          FTCS)
8      """
9
10     #
11     =====
12     # BLOQUE 1: Importacion de librerias
13     # Numpy: calculos numericos eficientes con arreglos
14     # Matplotlib: generacion de graficas de calidad
15     publicacion

```

```

14      #
=====
15      import numpy as np
16      import matplotlib.pyplot as plt
17      import matplotlib.gridspec as gridspec
18
19      #
=====
20      # BLOQUE 2: Parametros fisicos del cable OPGW
21      # Todos los valores con sus unidades comentadas
22      # Modifica estos valores para analizar diferentes
escenarios
23      #
=====
24      rho = 3200.0          # [kg/m3]   Densidad efectiva
de la seccion compuesta
25      cp  = 800.0          # [J/kg/K]   Calor especifico
efectivo
26      k   = 160.0          # [W/m/K]   Conductividad
termica efectiva (dominada por Al)
27
28      # Geometria: semidiametro del cable (dominio 1D por
simetria)
29      L   = 0.006          # [m]       Radio exterior del
cable OPGW (~12.7 mm diametro)
30
31      # Corriente de cortocircuito
32      I_cc = 31500.0       # [A]       Corriente de
cortocircuito
33      R_lineal = 0.0537e-3 # [Ohm/m]   Resistencia lineal
del cable
34      A_secc  = 87.7e-6    # [m2]      Area transversal del
conductor
35
36      # Densidad volumetrica de calor Joule:  $Q = I^2 * R'$ 
/ A
37      Q_vol = (I_cc**2 * R_lineal) / A_secc # [W/m3]
38
39      t_total = 0.3        # [s]      Duracion del
cortocircuito
40      T0      = 25.0        # [C]      Temperatura inicial
uniforme

```

```

41         T_amb    = 25.0          # [C]  Temperatura en la
        frontera exterior
42
43         #
        =====
44         # BLOQUE 3: Parametros de la malla numerica
45         # Calculamos automaticamente el paso de tiempo
        maximo
46         # que garantiza la estabilidad del esquema FTCS
47         #
        =====
48         N    = 60                # Numero de nodos espaciales
49         dx   = L / (N - 1)       # Paso espacial [m]
50
51         # Difusividad termica: alpha = k / (rho * cp) [m2/s
        ]
52         alpha = k / (rho * cp)
53
54         # Criterio de estabilidad de von Neumann para FTCS:
55         # El numero de Fourier Fo = alpha*dt/dx2 DEBE ser <=
        0.5
56         # dt_max es el valor limite; usamos el 80% como
        margen
57         dt_max = dx**2 / (2.0 * alpha) # dt maximo
        permitido
58         dt      = 0.8 * dt_max        # dt usado (con
        margen de seguridad)
59         Nt      = int(t_total / dt) + 1 # Numero de pasos de
        tiempo
60
61         Fo = alpha * dt / dx**2 # Numero de Fourier (debe
        ser < 0.5)
62         print(f"Numero de Fourier Fo = {Fo:.4f} (debe ser <
        0.5)")
63         print(f"Numero de pasos: {Nt}")
64
65         #
        =====
66         # BLOQUE 4: Inicializacion del campo de temperatura
67         # T es el arreglo de temperaturas en todos los nodos
68         #
        =====

```

```

69         x = np.linspace(0, L, N) # Coordenadas espaciales [
m]
70         T = np.full(N, T0)          # Campo inicial: T0 en
    todos los nodos
71
72         # Guardamos perfiles en tiempos especificos para
    graficar
73         tiempos_guardar = [0.0, 0.05, 0.1, 0.2, 0.3]
74         perfiles = {0.0: T.copy()}
75         T_centro = [] # Temperatura en x=0 (centro del
    cable) vs tiempo
76         t_vector = []
77
78         #
    =====
79         # BLOQUE 5: Bucle de integracion temporal (FTCS
    explicito)
80         # Este es el "motor" de la simulacion.
81         # Avanza la temperatura desde t=0 hasta t=t_total.
82         #
    =====
83         t = 0.0
84         for n in range(Nt):
85             T_new = T.copy() # Hacemos una copia para no
    mezclar t_n y t_{n+1}
86
87             # --- Nodos interiores: esquema FTCS ---
88             # Equivale a: T_new = T + Fo*(T derecha - 2*T centro
    + T izquierda)
89             #                                     + calor generado en dt
90             T_new[1:-1] = (T[1:-1]
91             + Fo * (T[2:] - 2.0*T[1:-1] + T[:-2]))
92             + dt * Q_vol / (rho * cp))
93
94             # --- Condicion de frontera izquierda: simetria (
    Neumann) ---
95             # dT/dx = 0 en x=0 se implementa como T[0] = T[1]
96             T_new[0] = T_new[1]
97
98             # --- Condicion de frontera derecha: temperatura
    fija (Dirichlet) ---

```



```

99         # La superficie exterior del cable se mantiene a
T_amb
100     T_new[-1] = T_amb
101
102     T = T_new.copy() # Actualizar el campo
103     t += dt          # Avanzar el tiempo
104
105     # Registrar datos para las graficas
106     t_vector.append(t)
107     T_centro.append(T[0]) # Temperatura en el centro
108
109     for t_g in tiempos_guardar:
110         if abs(t - t_g) < dt/2.0 and t_g not in perfiles:
111             perfiles[t_g] = T.copy()
112
113     perfiles[t_total] = T.copy()
114
115     #
=====
116     # BLOQUE 6: Generacion de la figura de resultados
117     # Dos paneles: perfiles T(x) y la historia T_centro(
t)
118     #
=====
119     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5)
)
120
121     # Panel izquierdo: perfiles de temperatura en
distintos tiempos
122     colores = plt.cm.plasma(np.linspace(0.15, 0.85, len(
perfiles)))
123     for idx, (t_g, T_g) in enumerate(sorted(perfiles.
items())):
124         ax1.plot(x * 1e3, T_g, color=colores[idx], lw=2.0,
125         label=f't = {t_g:.2f} s')
126         ax1.axhline(T_amb, color='steelblue', lw=1.2, ls='--
', alpha=0.7,
127         label=f'T_amb = {T_amb} C')
128         ax1.set_xlabel('Posicion x [mm]')
129         ax1.set_ylabel('Temperatura T [C]')
130         ax1.set_title('Perfiles de temperatura\n(seccion
transversal OPGW)')

```

```

131         ax1.legend(fontsize=9)
132         ax1.grid(True, alpha=0.3, ls='--')
133
134         # Panel derecho: historia termica en el centro del
135         cable
136         ax2.plot(t_vector, T_centro, color='#CC0000', lw
137         =2.0,
138         label='T(x=0, t) - centro del cable')
139         ax2.axhline(90, color='darkorange', lw=1.5, ls='--',
140         label='Limite fibra optica (90 C)')
141         ax2.axhline(200, color='red', lw=1.5, ls=':',
142         label='Limite acrilico buffer (200 C)')
143         ax2.set_xlabel('Tiempo t [s]')
144         ax2.set_ylabel('Temperatura T [C]')
145         ax2.set_title('Historia termica (centro del cable
146         OPGW)')
147         ax2.legend(fontsize=9)
148         ax2.grid(True, alpha=0.3, ls='--')
149
150         plt.tight_layout()
151         plt.savefig('fig_s4_ej1_difusion_OPGW.png', dpi=300,
152         bbox_inches='tight')
153         plt.show()
154         print("Figura guardada como fig_s4_ej1_difusion_OPGW
155         .png")

```

Listing 27: Semana 4 – Código 1: Ecuación de difusión 1D transitoria (FTCS). Cable OPGW bajo corriente de cortocircuito. Ejecutar en Google Colab.

4.12.2. Código 2 — Ecuación de transporte (onda): densidad vehicular con UAV

```

1         """
2         Semana 4 - Ejemplo 2: Ecuacion de onda (transporte)
3         1D
4         Aplicacion: Propagacion de densidad vehicular en un
5         corredor urbano
6         monitoreado por UAV con flujo optico (Garcia Djeda)
7
8         Modelo LWR: d_rho/dt + c * d_rho/dx = 0
9         Comparamos: upwind (1er orden) vs Lax-Wendroff (2do
10        orden)

```

```

8      """
9
10     import numpy as np
11     import matplotlib.pyplot as plt
12
13     #
14     =====
15     # BLOQUE 1: Parametros del modelo de trafico (LWR)
16     # Modifica c para diferentes condiciones de trafico
17     #
18     =====
19     rho_max = 100.0 # [veh/km] Densidad de atasco (
20     trafico detenido)
21     v_libre = 60.0 # [km/h] Velocidad de flujo
22     libre (carretera vacia)
23     c = v_libre / 2.0 # [km/h] Velocidad de onda
24     (linearizacion)
25
26     # Dominio espacial: corredor de 2 km observado por
27     el UAV
28     L = 2.0 # [km] Longitud del corredor
29     Nx = 200 # Numero de celdas espaciales
30     dx = L / Nx # Paso espacial [km]
31     x_c = np.linspace(dx/2, L - dx/2, Nx) # Centros de
32     celdas
33
34     t_total = 0.08 # [h] = 4.8 minutos
35
36     #
37     =====
38     # BLOQUE 2: Paso de tiempo - criterio CFL
39     # El numero de Courant nu = c*dt/dx DEBE ser <= 1
40     # Si nu > 1, el esquema upwind es INESTABLE (explota
41     )
42     #
43     =====
44     CFL = 0.9 # Numero de Courant objetivo
45     (< 1)
46     dt = CFL * dx / c # Paso de tiempo que satisface
47     el CFL
48     Nt = int(t_total / dt) + 1
49     nu = c * dt / dx # Numero de Courant efectivo

```

```

38
39         print(f"Numero de Courant nu = {nu:.4f} (debe ser
40         <= 1)")
41
42         #
43         =====
44         # BLOQUE 3: Condicion inicial - pulso gaussiano de
45         densidad
46         # Simula un grupo de vehiculos acumulados en un
47         semaforo
48         #
49         =====
50         x0_semaforo = 0.3      # [km] Posicion del semaforo
51         sigma_pulso = 0.12     # [km] Anchura de la
52         acumulacion
53         rho_fondo      = 20.0   # [veh/km] Densidad de fondo
54         en el corredor
55         A_pulso        = 50.0   # [veh/km] Sobredensidad por
56         el semaforo
57
58         rho0 = rho_fondo + A_pulso * np.exp(-((x_c -
59         x0_semaforo)**2) / (2*sigma_pulso**2))
60
61         # Solucion exacta: traslacion pura rho(x,t) = rho0(x
62         - c*t)
63         def exacta(x_c, t):
64             x_shift = x_c - c * t
65             return rho_fondo + A_pulso * np.exp(-((x_shift -
66             x0_semaforo)**2) / (2*sigma_pulso**2))
67
68         #
69         =====
70         # BLOQUE 4: Integracion temporal - dos esquemas
71         simultaneos
72         # Se integran en paralelo para poder comparar
73         despues
74         #
75         =====
76         rho_up = rho0.copy() # Densidad con esquema upwind
77         rho_lw = rho0.copy() # Densidad con esquema Lax-
78         Wendroff
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

64     tiempos_g = np.linspace(0, t_total, 5)
65     perfiles_up = {0.0: rho_up.copy()}
66     perfiles_lw = {0.0: rho_lw.copy()}
67     perfiles_ex = {0.0: exacta(x_c, 0.0)}
68
69     t = 0.0
70     for n in range(Nt):
71
72         # --- Esquema UPWIND (primer orden): usa el nodo de
73         la izquierda ---
74         # Para c > 0: la informacion viaja de izquierda a
75         derecha
76         # Por eso se usa la diferencia hacia ATRAS: (rho[i]
77         - rho[i-1])/dx
78         rho_up_new = rho_up.copy()
79         rho_up_new[1:] = rho_up[1:] - nu * (rho_up[1:] -
80         rho_up[:-1])
81         rho_up_new[0] = rho_up_new[-1] # Frontera
82         periodica
83         rho_up = rho_up_new.copy()
84
85         # --- Esquema LAX-WENDROFF (segundo orden): usa 3
86         nodos ---
87         # Combina diferencia centrada (alta precision) con
88         # una correccion de curvatura (reduce la difusion
89         numerica)
90         rho_lw_new = rho_lw.copy()
91         rho_lw_new[1:-1] = (rho_lw[1:-1]
92         - (nu/2.0) * (rho_lw[2:] - rho_lw[:-2])
93         + (nu**2/2.0) * (rho_lw[2:] - 2.0*rho_lw[1:-1] +
94         rho_lw[:-2]))
95         rho_lw_new[0] = rho_lw_new[-1]
96         rho_lw_new[-1] = rho_lw_new[0]
97         rho_lw = rho_lw_new.copy()
98
99         t += dt
100        for t_g in tiempos_g[1:]:
101            if abs(t - t_g) < dt/2.0 and t_g not in perfiles_up:
102                perfiles_up[t_g] = rho_up.copy()
103                perfiles_lw[t_g] = rho_lw.copy()
104                perfiles_ex[t_g] = exacta(x_c, t)

```

```

98      #
99      =====
100      # BLOQUE 5: Generacion de la figura comparativa
101      #
102      =====
103
104      fig, axes = plt.subplots(1, 2, figsize=(13, 5))
105      colores = plt.cm.viridis(np.linspace(0.1, 0.9, len(
106      perfiles_up)))
107
108      for ax, perfs, titulo in zip(axes,
109      [perfiles_up, perfiles_lw],
110      ['Upwind (1er orden)', 'Lax-Wendroff (2do orden)']):
111          for idx, t_g in enumerate(sorted(perfs.keys())):
112              ax.plot(x_c, perfs[t_g], color=colores[idx], lw=1.8,
113              label=f't = {t_g*60:.1f} min')
114              ax.plot(x_c, perfiles_ex[t_g], '--', color=colores[
115              idx], lw=0.9, alpha=0.5)
116              ax.set_xlabel('Posicion x [km]')
117              ax.set_ylabel('Densidad rho [veh/km]')
118              ax.set_title(titulo + '\n(-- -- solucion exacta)')
119              ax.legend(fontsize=8.5)
120              ax.set_xlim([0, L])
121              ax.grid(True, alpha=0.25, ls='--')
122
123      plt.tight_layout()
124      plt.savefig('fig_s4_ej2_onda_trafico.png', dpi=300,
125      bbox_inches='tight')
126      plt.show()
127      print("Figura guardada.")

```

Listing 28: Semana 4 – Código 2: Ecuación de transporte 1D — comparación upwind vs Lax-Wendroff. Tráfico vehicular monitoreado por UAV. Ejecutar en Google Colab.

4.12.3. Código 3 — Ecuación de Poisson 2D: losa de concreto

```

1      """
2      Semana 4 - Ejemplo 3: Ecuacion de Poisson 2D (estado
3      estacionario)
4      Aplicacion: Distribucion de temperatura en losa de
5      concreto

```

```

4         con calor de hidratacion (Lerma Lomeli / Lerma
Martinez)
5
6         Ecuacion:  $d^2T/dx^2 + d^2T/dy^2 = -Q(x,y)/k$ 
7         Metodo: diferencias finitas centradas + scipy sparse
solver
8         """
9
10        import numpy as np
11        import matplotlib.pyplot as plt
12        from scipy.sparse import lil_matrix
13        from scipy.sparse.linalg import spsolve
14
15        #
16        =====
17        # BLOQUE 1: Geometria de la losa y parametros
fisicos
18        # Modifica Nx y Ny para obtener mayor resolucion
#
19        =====
20        Lx = 4.0    # [m] Longitud de la losa en x
21        Ly = 3.0    # [m] Ancho de la losa en y
22        Nx = 40     # Nodos en x (incrementar para mayor
precision)
23        Ny = 30     # Nodos en y
24
25        dx = Lx / (Nx - 1)
26        dy = Ly / (Ny - 1)
27        x_vec = np.linspace(0, Lx, Nx)
28        y_vec = np.linspace(0, Ly, Ny)
29        X, Y = np.meshgrid(x_vec, y_vec) # Malla 2D
30
31        N_total = Nx * Ny # Numero total de nodos
32        print(f"Sistema de {N_total} ecuaciones y {N_total}
incognitas")
33
34        #
35        =====
36        # BLOQUE 2: Funcion fuente (calor de hidratacion
gaussiana)
37        # Q_max: intensidad maxima del calor de hidratacion
[C/m2]

```

```

36         # sigma_f: radio de influencia de la fuente [m]
37         #
=====
38         x0_f    = Lx / 2.0    # Centro de la losa en x
39         y0_f    = Ly / 2.0    # Centro de la losa en y
40         sigma_f = 0.8         # Dispersión de la fuente [m]
41         Q_max   = 80.0        # Intensidad maxima [C/m2] (
normalizado, k=1)
42
43         # Evaluamos la fuente en todos los nodos de la malla
44         F = Q_max * np.exp(-(((X-x0_f)/sigma_f)**2 + ((Y-
y0_f)/sigma_f)**2))
45
46         #
=====
47         # BLOQUE 3: Funcion de conversion indice 2D ->
indice 1D
48         # Necesaria para mapear el nodo (i,j) a una fila del
sistema
49         #
=====
50         def idx(i, j):
51             """Convierte (fila_y=i, columna_x=j) a indice 1D."""
52             return i * Nx + j
53
54         #
=====
55         # BLOQUE 4: Ensamblaje del sistema lineal A*T = b
56         # Usamos matrices dispersas (lil_matrix) porque la
mayoria
57         # de los coeficientes son cero: solo 5 por fila
58         #
=====
59         A_mat = lil_matrix((N_total, N_total)) # Matriz
dispersa
60         b_vec = np.zeros(N_total)              # Vector
lado derecho
61
62         for i in range(Ny): # Recorre filas (direccion y)
63             for j in range(Nx): # Recorre columnas (direccion
x)
64                 n = idx(i, j)

```



```

65
66     # --- Nodos de frontera: condicion de Dirichlet ---
67     if i == 0 or i == Ny-1 or j == 0 or j == Nx-1:
68         # Temperatura prescrita segun posicion del borde
69         if i == 0:            T_bc = 20.0
# borde inferior
70         elif i == Ny-1:      T_bc = 25.0
# borde superior
71         else:                T_bc = 20.0 + 5.0*(i/(Ny-1))      #
    bordes laterales
72     A_mat[n, n] = 1.0
73     b_vec[n]    = T_bc
74
75     else:
76         # --- Nodos interiores: stencil de 5 puntos ---
77         # (T_derecha - 2*T_centro + T_izquierda) / dx^2
78         # + (T_arriba - 2*T_centro + T_abajo)    / dy^2
79         # = -Q(i, j)
80     A_mat[n, n]                = -2.0/dx**2 - 2.0/dy**2      #
diagonal
81     A_mat[n, idx(i, j+1)]    = 1.0/dx**2      # vecino
derecho
82     A_mat[n, idx(i, j-1)]    = 1.0/dx**2      # vecino
izquierdo
83     A_mat[n, idx(i+1, j)]    = 1.0/dy**2      # vecino arriba
84     A_mat[n, idx(i-1, j)]    = 1.0/dy**2      # vecino abajo
85     b_vec[n] = -F[i, j]      # termino fuente (con signo
negativo)
86
87     # Convertir a formato CSR (mas eficiente para la
resolucion)
88     A_csr = A_mat.tocsr()
89
90     #
=====
91     # BLOQUE 5: Resolucion del sistema lineal
92     # spsolve usa eliminacion directa eficiente para
matrices dispersas
93     #
=====
94     T_vec = spsolve(A_csr, b_vec)      # Resuelve A*T = b

```

```

95         T_campo = T_vec.reshape(Ny, Nx)      # Convierte a
        campo 2D
96
97         print(f"Temperatura maxima: {T_campo.max():.2f} C")
98         print(f"Temperatura minima: {T_campo.min():.2f} C")
99         print(f"Incremento por hidratacion: {T_campo.max() -
        25:.2f} C")
100
101         #
        =====
102         # BLOQUE 6: Visualizacion del campo de temperatura
103         #
        =====
104         fig, axes = plt.subplots(1, 2, figsize=(13, 5))
105
106         # Panel izquierdo: mapa de calor (contourf) con
        isolineas
107         cf = axes[0].contourf(X, Y, T_campo, levels=25, cmap
        ='inferno')
108         cs = axes[0].contour(X, Y, T_campo, levels=10,
109         colors='white', linewidths=0.6, alpha=0.5)
110         axes[0].clabel(cs, fmt='%.0f C', fontsize=7)
111         plt.colorbar(cf, ax=axes[0]).set_label('Temperatura
        T [C]')
112         axes[0].set_xlabel('x [m]')
113         axes[0].set_ylabel('y [m]')
114         axes[0].set_title('Campo de temperatura T(x,y) -
        Losa de concreto')
115         axes[0].set_aspect('equal')
116
117         # Panel derecho: perfil T(x) en el eje central y =
        Ly/2
118         eje_central = Ny // 2
119         T_lineal = 20.0 + 5.0 * (eje_central/(Ny-1)) * np.
        ones(Nx) # sin fuente
120         axes[1].plot(x_vec, T_campo[eje_central, :], 'r-',
        lw=2.0,
121         label=f'Con calor de hidratacion')
122         axes[1].plot(x_vec, T_lineal, 'b--', lw=1.5,
123         label='Sin fuente (solo CC)')
124         axes[1].fill_between(x_vec, T_campo[eje_central,:],
        T_lineal,
    
```

```

125         alpha=0.15, color='red')
126         axes[1].set_xlabel('x [m]')
127         axes[1].set_ylabel('Temperatura T [C]')
128         axes[1].set_title(f'Perfil en eje central (y = {
y_vec[eje_central]:.1f} m)')
129         axes[1].legend()
130         axes[1].grid(True, alpha=0.3, ls='--')
131
132         plt.tight_layout()
133         plt.savefig('fig_s4_ej3_losa.png', dpi=300,
bbox_inches='tight')
134         plt.show()
135         print("Figura guardada.")
136

```

Listing 29: Semana 4 – Código 3: Ecuación de Poisson 2D (sistema lineal disperso). Losa de concreto con calor de hidratación. Ejecutar en Google Colab.

4.12.4. Código 4 — Convección-difusión 1D y número de Péclet

```

1      """
2      Semana 4 - Ejemplo 4: Ecuacion de conveccion-
difusion 1D
3      Aplicacion: Transporte de marcador en bomba
centrifuga (Zavala Morales)
4
5       $dc/dt + u * dc/dx = D * d^2c/dx^2$ 
6      Se comparan tres regimenes: Pe bajo (difusion), Pe
medio, Pe alto (conveccion)
7      """
8
9      import numpy as np
10     import matplotlib.pyplot as plt
11
12     #
=====
13     # BLOQUE 1: Parametros del dominio y la velocidad
del fluido
14     #
=====
15     L      = 0.5      # [m] Longitud del canal (
seccion de la bomba)

```

```

16         Nx      = 200          # Numero de nodos espaciales
17         dx      = L / (Nx - 1)
18         x       = np.linspace(0, L, Nx)
19
20         u_conv   = 2.0         # [m/s] Velocidad de conveccion
    del fluido
21         t_total  = 0.15       # [s] Tiempo total de
    simulacion
22
23         #
    =====
24         # BLOQUE 2: Funcion de simulacion para un D dado
25         # Recibe el coeficiente de difusion y devuelve la
    evolucion
26         #
    =====
27         def simular(D, etiqueta):
28             """
29             Simula la conveccion-difusion 1D para un D dado.
30             Esquema: upwind para conveccion + centrado para
    difusion.
31             """
32             Pe = u_conv * L / D    # Numero de Peclet
33             print(f"{etiqueta}: Pe = {Pe:.1f}, D = {D:.5f} m2/s"
    )
34
35             # Paso de tiempo: limite mas restrictivo entre CFL y
    Fourier
36             dt_cfl    = dx / u_conv          # Limite convectivo
    (CFL)
37             dt_difus  = dx**2 / (2.0 * D)    # Limite difusivo (
    Fourier)
38             dt        = 0.4 * min(dt_cfl, dt_difus) # Margen de
    seguridad
39             Nt        = int(t_total / dt) + 1
40
41             # Condicion inicial: pulso gaussiano de
    concentracion
42             x0_iny    = 0.04    # [m] Posicion de inyeccion
43             sigma     = 0.025   # [m] Anchura del pulso
44             c = np.exp(-((x - x0_iny)**2) / (2*sigma**2))
45

```

```

46         # Integracion temporal
47         tiempos_g = np.linspace(0, t_total, 5)
48         perfiles = [c.copy()]
49         t_vec     = [0.0]
50         t = 0.0
51
52         for _ in range(Nt):
53             c_new = c.copy()
54
55             # Termino convectivo: upwind (diferencia hacia atras
, u > 0)
56             # Representa que el fluido "trae" la concentracion
desde la izquierda
57             conv = u_conv * (c[1:-1] - c[:-2]) / dx
58
59             # Termino difusivo: diferencia centrada de segundo
orden
60             # Representa que el gradiente de concentracion
genera flujo de masa
61             diff = D * (c[2:] - 2.0*c[1:-1] + c[:-2]) / dx**2
62
63             c_new[1:-1] = c[1:-1] + dt * (-conv + diff)
64
65             # Condiciones de frontera: gradiente nulo (Neumann)
en ambos extremos
66             c_new[0] = c_new[1]
67             c_new[-1] = c_new[-2]
68             c_new = np.maximum(c_new, 0.0) # concentracion
>= 0
69
70             c = c_new.copy()
71             t += dt
72
73             for t_g in tiempos_g[1:]:
74                 if abs(t - t_g) < dt/2.0 and t_g not in t_vec:
75                     perfiles.append(c.copy())
76                     t_vec.append(t)
77
78             return x, np.array(t_vec), perfiles, Pe
79
80         #
=====

```

```

81         # BLOQUE 3: Simulacion para tres valores de D
82         # Pe = u*L/D: D grande -> Pe pequeno (difusion
            dominante)
83         #
            =====
84         D_1 = u_conv * L / 2.0      # Pe = 2    (difusion muy
            activa)
85         D_2 = u_conv * L / 20.0     # Pe = 20   (regimen mixto)
86         D_3 = u_conv * L / 200.0    # Pe = 200  (conveccion
            dominante)
87
88         _, t_1, p_1, Pe_1 = simular(D_1, "Pe bajo")
89         _, t_2, p_2, Pe_2 = simular(D_2, "Pe medio")
90         _, t_3, p_3, Pe_3 = simular(D_3, "Pe alto")
91
92         #
            =====
93         # BLOQUE 4: Comparacion de los tres regimenes al
            tiempo final
94         #
            =====
95         fig, ax = plt.subplots(figsize=(10, 5))
96
97         ax.plot(x * 100, p_1[-1], color='teal',          lw
            =2.2,
98         label=f'Pe = {Pe_1:.0f} - difusion dominante')
99         ax.plot(x * 100, p_2[-1], color='darkorange',    lw
            =2.2,
100        label=f'Pe = {Pe_2:.0f} - regimen mixto')
101        ax.plot(x * 100, p_3[-1], color='#CC0000',        lw
            =2.2,
102        label=f'Pe = {Pe_3:.0f} - conveccion dominante')
103
104        ax.set_xlabel('Posicion x [cm]')
105        ax.set_ylabel('Concentracion c [a.u.]')
106        ax.set_title('Conveccion-Difusion: perfiles al
            tiempo final\n'
107        f'(t = {t_total*1e3:.0f} ms, u = {u_conv} m/s)')
108        ax.legend(fontsize=10)
109        ax.set_xlim([0, L * 100])
110        ax.grid(True, alpha=0.3, ls='--')
111
    
```

```

112         plt.tight_layout()
113         plt.savefig('fig_s4_ej4_convdiff_bomba.png', dpi=300,
114             bbox_inches='tight')
115         plt.show()
116         print("Figura guardada.")
117         # Para ver la evolucion temporal de un solo regimen,
118         # descomenta el siguiente bloque y modifica '
119         perfiles' y 't_vec':
120         # for idx, (t_g, perfil) in enumerate(zip(t_2, p_2))
121         :
122         #         plt.plot(x*100, perfil, label=f't = {t_g*1e3
123         :.1f} ms')

```

Listing 30: Semana 4 – Código 4: Ecuación de convección-difusión 1D. Análisis del número de Péclet. Bomba centrífuga. Ejecutar en Google Colab.

4.13. Ejercicios Reto — Tarea Semana 4

Instrucciones generales. Los ejercicios están clasificados por nivel. Los Ejercicios 1 y 2 son obligatorios. El Ejercicio 3 es optativo y tiene valor de punto extra. En todos los casos se requiere: (a) formulación matemática explícita de la EDP y sus condiciones de frontera; (b) código ejecutable en Google Colab; (c) mínimo una gráfica con ejes etiquetados, unidades y título; y (d) interpretación física de los resultados en no menos de tres observaciones cuantitativas.

4.13.1. Ejercicio 1 — Análisis de la estabilidad FTCS (Nivel intermedio)

Modifica el Código 1 para explorar el efecto del número de Fourier en la estabilidad:

- Ejecuta la simulación con $F_o = 0.45$ (estable, cerca del límite), $F_o = 0.50$ (límite exacto) y $F_o = 0.55$ (inestable). Grafica los tres campos $T(x, t)$ en la misma figura al tiempo $t = 0.05$ s.
- Calcula la temperatura máxima en el dominio para cada caso. ¿En qué instante se rompe la estabilidad para $F_o = 0.55$? Repórtalo numéricamente.
- Argumenta en una o dos frases por qué el esquema explícito FTCS falla para $F_o > 0.5$ (pista: análisis de von Neumann, Semana 3).

Rúbrica.

- Tres figuras correctas, valor de F_o crítico identificado con precisión temporal, justificación física completa.

- 2 Dos figuras correctas, el caso inestable graficado pero sin análisis temporal.
- 1 Una figura correcta, sin análisis de estabilidad.
- 0 Código no ejecuta o resultado sin coherencia física.

4.13.2. Ejercicio 2 — Condición de Robin y convección superficial (Nivel intermedio)

Modifica el Código 1 para reemplazar la condición de Dirichlet en $x = L$ por una condición de Robin:

$$-k \frac{\partial T}{\partial x} \Big|_{x=L} = h (T(L, t) - T_{\infty}), \quad (150)$$

con $h = 500 \text{ W}/(\text{m}^2 \text{ K})$ (convección natural fuerte) y $T_{\infty} = 30^\circ \text{C}$.

Implementación sugerida. La condición de Robin se discretiza en el último nodo como:

$$\frac{T_N^{n+1} - T_{N-1}^{n+1}}{\Delta x} = -\frac{h}{k} (T_N^{n+1} - T_{\infty}),$$

despejando T_N^{n+1} explícitamente.

- (a) Compara la historia térmica en el centro del cable con la condición de Dirichlet original vs la condición de Robin.
- (b) Calcula la temperatura de la superficie ($x = L$) para $t = 0.1 \text{ s}$ en ambos casos.
- (c) ¿Cuál condición es más conservadora para evaluar el riesgo térmico de la fibra óptica?

Rúbrica.

- 3 Implementación de Robin correcta, comparación cuantitativa de las dos condiciones, conclusión justificada.
- 2 Implementación correcta pero sin comparación cuantitativa.
- 1 Código incompleto pero con la ecuación de Robin correctamente planteada.

4.13.3. Ejercicio 3 — Difusión-reacción: carbonatación de concreto (Nivel avanzado, punto extra)

La carbonatación del concreto es un proceso de difusión del CO_2 atmosférico seguido de reacción química con la portlandita $[\text{Ca}(\text{OH})_2]$. En el modelo simplificado de Papadakis, la concentración de CO_2 en la profundidad x satisface:

$$\frac{\partial c}{\partial t} = D_{\text{CO}_2} \frac{\partial^2 c}{\partial x^2} - k_r c c_p(x), \quad (151)$$

donde $k_r = 5 \times 10^{-4} \text{ m}^3/(\text{mol} \cdot \text{s})$ es la constante de reacción y c_p la concentración de portlandita.

- (a) Implementa la Ec. 151 en 1D para una losa de 0.3 m de espesor, con $D_{CO_2} = 3 \times 10^{-10} \text{ m}^2/\text{s}$, $c_p = 300 \text{ mol/m}^3$ (uniforme inicialmente) y condición de frontera $c(0, t) = 0.014 \text{ mol/m}^3$ (concentración atmosférica de CO_2).
- (b) Grafica la concentración de CO_2 a $t \in \{1, 5, 10, 30\}$ años.
- (c) Estima la profundidad de carbonatación (frente donde c desciende al 10 % de su valor en la frontera) en función del tiempo y compara con la ley $x_f \propto \sqrt{t}$ de Fick.

Rúbrica.

- 3 Simulación correcta, curva $x_f(t)$ graficada y comparada con la ley de Fick, discusión de la desviación debida a la reacción.
- 2 Simulación correcta, frente identificado pero sin comparación analítica.
- 1 Código con la estructura correcta pero resultados inconsistentes (p.ej., concentración negativa sin corrección).

5. Semana 5. Simulación Bidimensional de Campos

Objetivo de la cátedra

Implementar simulaciones bidimensionales de difusión, propagación de ondas y transporte en dominios con geometría compleja, dominando el mallado cartesiano 2D, la representación de obstáculos y cavidades mediante máscaras, el ensamblaje disperso de sistemas lineales, los criterios de estabilidad en dos dimensiones y la visualización cuantitativa de campos escalares, con énfasis en la interpretación física y el diagnóstico de los errores numéricos propios de 2D.

Temario

- Simulación 2D de difusión.
- Propagación de ondas.
- Obstáculos y condiciones geométricas.
- Mallado bidimensional.
- Visualización de campos escalares.
- Análisis temporal de simulaciones.
- Interpretación física de resultados.

Propósito del tema. La Semana 4 estableció las EDPs canónicas y sus esquemas en una dimensión espacial. Esta semana da el salto a dos dimensiones, que es donde la simulación empieza a parecerse a los problemas reales de una tesis doctoral: secciones transversales de componentes compuestos, placas con cavidades, recintos con obstáculos, flujos recirculantes y campos medidos por redes de sensores. El salto no es trivial: el costo computacional crece cuadráticamente con la resolución, los criterios de estabilidad se endurecen, la geometría deja de ser un intervalo y se convierte en una decisión de modelado, y la elección de condiciones de frontera en los bordes *internos* (huecos, obstáculos) puede invertir por completo la conclusión de un estudio. Esta semana se aprende a anticipar y diagnosticar esos efectos.

Resultado de aprendizaje esperado

Al finalizar la sesión, el estudiante será capaz de: (i) discretizar el laplaciano 2D con el stencil de cinco puntos y cuantificar su error de truncamiento; (ii) derivar y aplicar los criterios de estabilidad FTCS y CFL en dos dimensiones; (iii) representar geometrías complejas (dominios circulares, perforaciones, obstáculos) sobre mallas cartesianas mediante máscaras booleanas, incluyendo interfaces entre materiales con media armónica;

(iv) ensamblar y resolver sistemas dispersos para problemas elípticos en dominios no triviales; (v) cuantificar la difusión y la dispersión numéricas de los esquemas de transporte en 2D mediante pruebas con solución exacta conocida; y (vi) seleccionar visualizaciones de campos escalares que permitan comparación cuantitativa y no solo cualitativa.

5.1. Motivación: del eje 1D al campo 2D

En una dimensión, la incógnita $u(x, t)$ vive sobre una línea y la geometría se reduce a un intervalo con dos extremos. En dos dimensiones, $u(x, y, t)$ vive sobre una región $\Omega \subset \mathbb{R}^2$ cuya *forma* es parte del modelo. Esto introduce cuatro diferencias estructurales que dominan el trabajo de esta semana:

1. **Costo computacional.** Una malla de N nodos por dirección tiene N^2 incógnitas. Duplicar la resolución multiplica por 4 la memoria y, en esquemas explícitos cuyo paso temporal escala como $\Delta t \propto \Delta x^2$, multiplica por 8 el número total de operaciones de una simulación transitoria de difusión.
2. **Geometría.** Los dominios reales son círculos, placas perforadas, recintos con obstáculos. Sobre una malla cartesiana, la herramienta básica para representarlos es la *máscara booleana*: un arreglo del mismo tamaño que la malla que indica qué celdas pertenecen al dominio físico.
3. **Condiciones de frontera internas.** Además del contorno exterior, aparecen fronteras *interiores*: paredes de cavidades, superficies de obstáculos, interfaces entre materiales. La elección de condición en esas fronteras (Dirichlet, Neumann, Robin) tiene consecuencias físicas de primer orden, como se demostrará cuantitativamente en el Ejemplo 3.
4. **Visualización.** En 1D basta una curva $u(x)$; en 2D el resultado es un campo escalar que requiere mapas de color, contornos y cortes, y cuya lectura incorrecta (escalas de color distintas entre paneles, mapas no perceptualmente uniformes) puede inducir conclusiones falsas.

Los seis ejemplos de esta semana cubren las tres familias de EDPs de la Semana 4, ahora en 2D, con situaciones representativas de los proyectos del grupo (sin particularizar): el calentamiento transitorio de una *sección transversal compuesta* con generación interna de calor; la *propagación y difracción de ondas* ante un obstáculo; el campo estacionario de temperatura en una *placa con cavidades*; el *transporte de un trazador en un flujo recirculante*; la *advección de un campo de densidad* bajo un campo de velocidad conocido punto a punto; y la *reconstrucción de un campo* a partir de mediciones dispersas de una red de sensores.

5.2. Mallado bidimensional y discretización del laplaciano

5.2.1. Malla cartesiana estructurada

Sea $\Omega = [0, L_x] \times [0, L_y]$ y una malla uniforme de $N_x \times N_y$ nodos:

$$x_i = i \Delta x, \quad y_j = j \Delta y, \quad \Delta x = \frac{L_x}{N_x - 1}, \quad \Delta y = \frac{L_y}{N_y - 1}, \quad (152)$$

con $u_{i,j}^n \approx u(x_i, y_j, t_n)$. En NumPy esta malla se construye con `meshgrid`, y la convención de indexación (`indexing='xy'` o `'ij'`) debe declararse explícitamente y mantenerse coherente en todo el código: una fracción considerable de los errores en simulaciones 2D proviene de confundir el eje de filas con el eje y físico.

5.2.2. El stencil de cinco puntos

Aplicando la diferencia centrada de segundo orden (Semana 3) en cada dirección, el laplaciano discreto con $\Delta x = \Delta y = h$ es

$$\nabla^2 u|_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2}, \quad (153)$$

con error de truncamiento

$$\tau_{i,j} = \frac{h^2}{12} \left(\frac{\partial^4 u}{\partial x^4} + \frac{\partial^4 u}{\partial y^4} \right) + \mathcal{O}(h^4), \quad (154)$$

es decir, segundo orden espacial, igual que en 1D. La novedad no está en el orden sino en la *estructura algebraica*: al ordenar las incógnitas lexicográficamente ($p = jN_x + i$), el operador (153) genera una matriz pentadiagonal de tamaño $N^2 \times N^2$ con a lo sumo cinco entradas no nulas por fila.

5.2.3. Por qué el almacenamiento disperso es obligatorio

Tabla 22: Costo de memoria del laplaciano 2D para malla $N \times N$ (entradas en doble precisión, 8 bytes).

N	Incógnitas N^2	Matriz densa	Matriz dispersa
51	2 601	54 MB	0.1 MB
181	32 761	8.6 GB	1.3 MB
501	251 001	504 GB	10 MB
1001	1 002 001	8000 GB	40 MB

Para $N = 181$ (la malla del Ejemplo 3) la matriz densa ya no cabe en la memoria de una sesión estándar de Google Colab, mientras que la versión dispersa ocupa poco más de un megabyte. El módulo `scipy.sparse` provee los formatos `lil_matrix` (eficiente para

ensamblar entrada por entrada) y `csr_matrix` (eficiente para resolver con `spsolve`); el flujo de trabajo estándar es ensamblar en LIL y convertir a CSR antes de resolver.

5.3. Estabilidad en dos dimensiones

Los criterios de la Semana 3 se endurecen en 2D porque el error puede crecer simultáneamente por ambas direcciones. El análisis de von Neumann con modos $e^{i(k_x x + k_y y)}$ produce:

- **Difusión, FTCS 2D.** Con $\Delta x = \Delta y$, la condición $F_{o,x} + F_{o,y} \leq \frac{1}{2}$ se reduce a

$$\Delta t \leq \frac{\Delta x^2}{4\alpha}, \quad (155)$$

exactamente la mitad del límite 1D. Con propiedades variables, α debe evaluarse en su valor *máximo* sobre el dominio.

- **Onda, leapfrog 2D.** El número de Courant admisible disminuye en un factor $\sqrt{2}$:

$$S \equiv \frac{c \Delta t}{\Delta x} \leq \frac{1}{\sqrt{2}} \approx 0.707. \quad (156)$$

- **Advección, upwind 2D.** La condición práctica es

$$\Delta t \leq \frac{\Delta x}{\max(|u| + |v|)}, \quad (157)$$

que garantiza que la información numérica no recorra más de una celda por paso aun cuando la velocidad apunte en diagonal.

Regla operativa. En todos los códigos de esta semana el paso de tiempo se *calcula* a partir del criterio de estabilidad con un margen de seguridad (factores 0.4–0.5), nunca se fija a mano. Esta práctica elimina de raíz la clase de error más común en simulaciones explícitas: un Δt válido para una malla gruesa que se vuelve inestable al refinar.

5.4. Geometrías complejas sobre mallas cartesianas: máscaras

5.4.1. Máscara booleana y frontera escalonada

Un dominio Ω de forma arbitraria se representa con un arreglo booleano $M_{i,j} = [(x_i, y_j) \in \Omega]$. Las celdas con $M = \text{False}$ no participan en la evolución; las celdas activas con algún vecino inactivo forman la *frontera escalonada* (*staircase boundary*), que aproxima el contorno real con un error geométrico $\mathcal{O}(h)$. Esta es la limitación central del método: aunque el esquema interior sea de segundo orden, la frontera escalonada degrada la convergencia global cerca del contorno. Para geometrías curvas con requisitos

de precisión altos, la solución definitiva es el método de elementos finitos con mallas adaptadas al contorno (Semanas 8 y 9); la máscara es la herramienta de prototipado rápido que permite obtener respuestas físicamente correctas en minutos.

5.4.2. Condiciones de frontera sobre la máscara

Sobre la frontera escalonada se implementan las tres condiciones clásicas:

- **Dirichlet** (u prescrita): se fija el valor de las celdas frontera, o el de las celdas inactivas vecinas.
- **Neumann homogénea** (flujo nulo): se anulan los coeficientes de las caras que cruzan la frontera; el término diagonal cuenta únicamente los vecinos activos. Es la implementación natural en el ensamblaje disperso del Ejemplo 3.
- **Robin** (convección): se agrega un sumidero superficial linealizado; en formulación volumétrica, una celda frontera de tamaño h recibe el término $-h_{cv}(u - u_{\infty})/h$ por unidad de volumen.

5.4.3. Interfaces entre materiales: media armónica

Cuando el dominio contiene dos materiales con conductividades k_1 y k_2 , la conductividad en la cara entre dos celdas debe evaluarse con la *media armónica*

$$k_{\text{cara}} = \frac{2k_1k_2}{k_1 + k_2}, \quad (158)$$

y no con la media aritmética. La razón es física: las dos medias celdas actúan como resistencias térmicas *en serie*, y la media armónica es la única que garantiza la continuidad del flujo $q = -k \partial T / \partial n$ a través de la interfaz. Con contraste fuerte ($k_2/k_1 \approx 667$ en el Ejemplo 1), la media aritmética sobreestima el acoplamiento térmico en órdenes de magnitud.

5.5. Ejemplo 1 — Difusión 2D transitoria en una sección compuesta con generación interna

5.5.1. Planteamiento

Un componente de sección circular (radio exterior $R_{\text{ext}} = 6 \text{ mm}$) está formado por una corona metálica conductora y un núcleo polimérico de baja conductividad que aloja elementos sensibles. Durante un transitorio eléctrico de 0.3 s, la corona disipa calor Joule con densidad volumétrica $Q_J = 4 \times 10^8 \text{ W/m}^3$; después la fuente se apaga y el componente se enfría por convección natural. La pregunta de ingeniería es: ¿qué temperatura alcanza la interfaz con el núcleo y cuánto tarda el interior en enterarse del pulso?

El modelo es la ecuación de calor con propiedades variables por material:

$$\rho c_p(\mathbf{x}) \frac{\partial T}{\partial t} = \nabla \cdot (k(\mathbf{x}) \nabla T) + Q(\mathbf{x}, t), \quad \mathbf{x} \in \Omega \text{ (disco)}, \quad (159)$$

con condición inicial uniforme $T_0 = 25^\circ\text{C}$, frontera exterior adiabática con sumidero convectivo ($h_{cv} = 25 \text{ W}/(\text{m}^2 \text{ K})$) y fuente conmutada: $Q = Q_J$ en la corona si $t < t_{\text{pulso}}$ y $Q = 0$ en otro caso.

Tabla 23: Parámetros del Ejemplo 1. El contraste de conductividades entre corona y núcleo es de tres órdenes de magnitud.

Propiedad	Símbolo	Núcleo	Corona
Conductividad [$\text{W}/(\text{m K})$]	k	0.30	200
Densidad [kg/m^3]	ρ	1400	2700
Calor específico [$\text{J}/(\text{kg K})$]	c_p	1200	900
Radio [mm]	R	1.8	6.0

5.5.2. Discretización

El disco se embebe en una malla cartesiana de 121×121 nodos ($\Delta x = 0.105 \text{ mm}$) mediante una máscara circular. Las conductividades de cara se evalúan con la media armónica (Ec. 158) y el paso de tiempo se calcula con el criterio FTCS 2D (Ec. 155) usando la difusividad máxima del dominio ($\alpha_{\text{máx}} = 8.23 \times 10^{-5} \text{ m}^2/\text{s}$, la de la corona), con margen 0.4: $\Delta t = 13.4 \mu\text{s}$, es decir 44 792 pasos para cubrir 0.6 s.

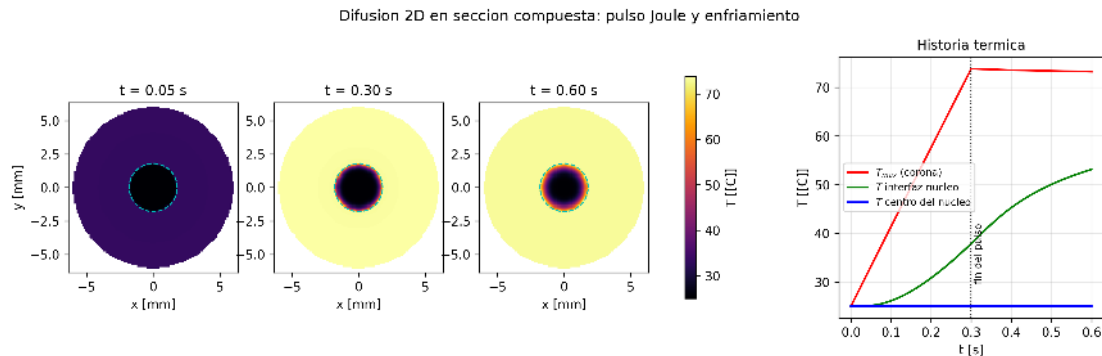


Figura 33: Difusión 2D en la sección compuesta. Paneles 1–3: campo de temperatura durante el pulso ($t = 0.05 \text{ s}$), al final del pulso ($t = 0.30 \text{ s}$) y tras el enfriamiento parcial ($t = 0.60 \text{ s}$); la circunferencia punteada marca la interfaz corona–núcleo. Panel derecho: historia térmica de la corona, de la interfaz del núcleo y del centro. Resultados del Código 1 del Anexo de esta semana.

5.5.3. Resultados y lectura metodológica

Tres observaciones cuantitativas estructuran la interpretación:

1. **La corona es casi isoterma.** Su temperatura máxima, 73.8°C , coincide con la estimación adiabática $T_0 + Q_J t_{\text{pulso}} / (\rho_2 c_{p,2}) \approx 74.4^\circ\text{C}$: la alta difusividad homogeneiza la corona en tiempos mucho menores que el pulso, y las pérdidas convectivas son despreciables en 0.6 s. Verificar un resultado 2D contra un balance 0D es la primera validación obligada de cualquier simulación térmica.
2. **El núcleo actúa como filtro pasa-bajas térmico.** La interfaz alcanza 53.1°C , pero el centro permanece en 25.0°C durante toda la simulación. La profundidad de penetración difusiva $\delta \sim \sqrt{\alpha_1 t} \approx 0.33 \text{ mm}$ para $t = 0.6 \text{ s}$ es mucho menor que el radio del núcleo (1.8 mm): el pulso simplemente no ha llegado al centro. El retardo térmico es una consecuencia directa de la EDP parabólica, no un artefacto.
3. **La media armónica importa.** Si se reemplaza por la media aritmética, la conductividad de la cara en la interfaz pasa de $0.6 \text{ W}/(\text{m K})$ a $100 \text{ W}/(\text{m K})$, y el núcleo se calienta espuriamente dos órdenes de magnitud más rápido. El error no es sutil: cambia la respuesta de ingeniería.

5.6. Ejemplo 2 — Onda 2D con obstáculo: reflexión y difracción

5.6.1. Planteamiento

Un pulso de presión se genera en un recinto cuadrado de $1 \text{ m} \times 1 \text{ m}$ (medio: aire, $c = 340 \text{ m/s}$) dividido por una pared rígida con una rendija de apertura $a = 0.10 \text{ m}$. El fenómeno gobernante es la ecuación de onda 2D:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \quad (160)$$

con $u = 0$ en la pared y en el contorno (fronteras rígidas), pulso gaussiano inicial de ancho $\sigma = 25 \text{ mm}$ y velocidad inicial nula. Este montaje reproduce el experimento clásico de difracción: detrás de la rendija el frente de onda se curva y la apertura actúa como fuente secundaria (principio de Huygens), un comportamiento común a ondas acústicas, mecánicas y ópticas.

5.6.2. Discretización

Se emplea el esquema *leapfrog*, centrado de segundo orden en tiempo y espacio:

$$u_{i,j}^{n+1} = 2u_{i,j}^n - u_{i,j}^{n-1} + (c \Delta t)^2 \nabla_h^2 u_{i,j}^n, \quad (161)$$

con arranque de Taylor $u^1 = u^0 + \frac{1}{2}(c \Delta t)^2 \nabla_h^2 u^0$ para preservar el segundo orden en el primer paso. La malla es de 401×401 nodos y el número de Courant se fija en $S = 0.5$, por debajo del límite 2D $1/\sqrt{2} \approx 0.707$ (Ec. 156), lo que da $\Delta t = 3.68 \mu\text{s}$. El obstáculo

se impone como condición de Dirichlet interna ($u = 0$ sobre la máscara de la pared) en cada paso.

Como diagnóstico de calidad se monitorea la energía discreta

$$E^n = \frac{1}{2} \sum_{i,j} \left[\left(\frac{u_{i,j}^{n+1} - u_{i,j}^{n-1}}{2\Delta t} \right)^2 + c^2 \|\nabla_h u_{i,j}^n\|^2 \right] \Delta x^2, \quad (162)$$

que para la ecuación de onda con fronteras rígidas es una cantidad conservada del problema continuo.

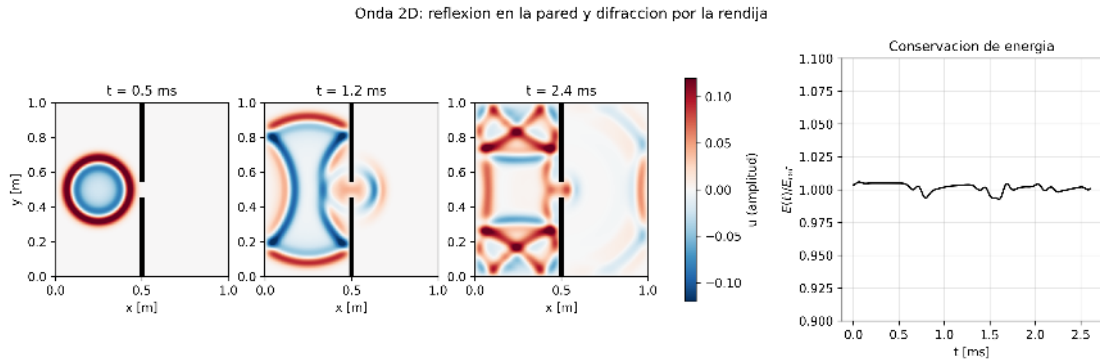


Figura 34: Onda 2D con obstáculo. Paneles 1–3: el pulso se expande ($t = 0.5$ ms), incide sobre la pared y la rendija emite un frente circular difractado ($t = 1.2$ ms), y el lado izquierdo desarrolla un patrón de reverberación mientras el frente difractado avanza ($t = 2.4$ ms). Panel derecho: energía discreta normalizada; la deriva tras el transitorio inicial es del 0.06 %. Resultados del Código 2 del Anexo de esta semana.

5.6.3. Resultados y lectura metodológica

1. **La conservación de energía es el termómetro del esquema.** El leapfrog es no disipativo: la energía discreta se conserva con deriva del 0.06 % en 708 pasos. Si la energía creciera sistemáticamente, el esquema sería inestable; si decayera, sería disipativo. Monitorear un invariante físico es la forma más barata de verificación continua en problemas hiperbólicos.
2. **La difracción es información geométrica cuantitativa.** El frente transmitido es aproximadamente circular y centrado en la rendija, con apertura angular controlada por la razón a/λ entre la apertura y la longitud de onda dominante del pulso. Reducir a ensancha el lóbulo difractado: el sistema se comporta como un filtro espacial.
3. **Costo del límite CFL 2D.** Trabajar a $S = 0.5$ en vez del límite 0.707 incrementa el número de pasos en ~ 41 %; es el precio de un margen de seguridad ante la frontera escalonada del obstáculo, donde el análisis de von Neumann (que supone dominio periódico homogéneo) ya no es estrictamente válido.

5.7. Ejemplo 3 — Poisson 2D en una placa perforada: el papel decisivo de las fronteras internas

5.7.1. Planteamiento

Una placa de concreto de $3\text{ m} \times 3\text{ m}$ genera calor de hidratación con densidad $q_h = 250\text{ W/m}^3$ durante el fraguado. Se comparan tres configuraciones: la placa sólida y una placa aligerada con un arreglo de 4×4 cavidades cuadradas de 0.45 m de lado (fracción de volumen resultante: 65.1%), considerando las cavidades en dos escenarios físicos: *selladas* (aire confinado, aproximadamente adiabáticas) o *ventiladas* (aire circulando a temperatura ambiente). El campo estacionario satisface la ecuación de Poisson

$$-k \nabla^2 T = q_h \quad \text{en el concreto,} \quad T = T_{\text{amb}} \quad \text{en el contorno exterior,} \quad (163)$$

con $k = 1.8\text{ W/(m K)}$, $T_{\text{amb}} = 22^\circ\text{C}$, y en las paredes de las cavidades: $\frac{\partial T}{\partial n} = 0$ (selladas) o $T = T_{\text{amb}}$ (ventiladas).

5.7.2. Discretización: ensamblaje disperso sobre el dominio enmascarado

Las incógnitas se numeran únicamente sobre las celdas activas mediante un mapa $(i, j) \mapsto p$; la matriz se ensambla en formato LIL y se convierte a CSR para resolver con `spsolve`. La condición de Neumann en las cavidades se implementa de forma natural: las caras que cruzan hacia un hueco simplemente no aportan conexión, y el coeficiente diagonal cuenta solo los vecinos activos. Para la malla de 181×181 , el caso aligerado tiene 20 592 incógnitas y 100 532 entradas no nulas: una densidad del 0.024% , que justifica plenamente la Tabla 22.

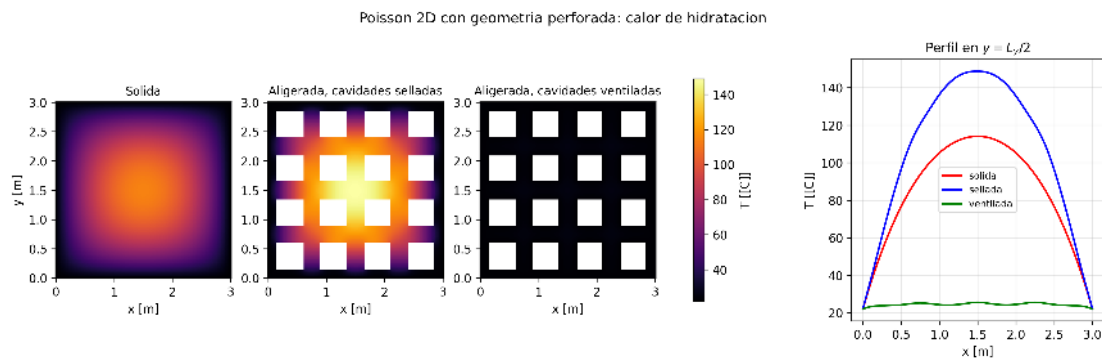


Figura 35: Calor de hidratación en la placa: sólida, aligerada con cavidades selladas y aligerada con cavidades ventiladas, bajo la misma escala de color. Panel derecho: perfil de temperatura por la línea media. La condición de frontera en las cavidades invierte la conclusión del estudio. Resultados del Código 3 del Anexo de esta semana.

5.7.3. Resultados y lectura metodológica

Tabla 24: Temperatura máxima según la configuración y la condición de frontera en las cavidades.

Configuración	$T_{\text{máx}}$ [°C]	Sobrecalentamiento
		$T_{\text{máx}} - T_{\text{amb}}$
Sólida	114.1	92.1 K
Aligerada, cavidades selladas	148.8	126.8 K
Aligerada, cavidades ventiladas	25.6	3.6 K

1. **La intuición ingenieril puede fallar.** Aligerar la placa reduce el volumen que genera calor (-35%), pero con cavidades selladas la temperatura máxima *sube* de 114.1 °C a 148.8 °C : las cavidades adiabáticas bloquean los caminos de conducción hacia el contorno y el calor restante queda atrapado en una red de nervaduras de mayor resistencia térmica.
2. **La condición de frontera interna decide el signo de la conclusión.** Con cavidades ventiladas a T_{amb} , el sobrecalentamiento colapsa a 3.6 K : cada cavidad se convierte en un sumidero interior y ninguna nervadura queda a más de $\sim 0.1\text{ m}$ de una frontera fría. Entre los dos escenarios extremos hay un factor de 35 en el sobrecalentamiento. En un reporte doctoral, la condición de frontera asumida en cada superficie interna debe declararse y justificarse con el mismo rigor que la EDP.
3. **Los extremos acotan la realidad.** El caso real (cavidades con aire quieto y algo de convección natural) está entre ambos; los dos cálculos baratos proporcionan cotas rigurosas, una estrategia de modelado más defendible que un único cálculo con parámetros inciertos.

5.8. Ejemplo 4 — Convección-difusión 2D en un flujo recirculante: el número de Péclet de malla

5.8.1. Planteamiento

Un trazador pasivo (colorante, calor, contaminante) se libera en una cavidad cuadrada de $L = 0.10\text{ m}$ donde el fluido recircula en un vórtice de una celda con campo de velocidad solenoidal analítico:

$$u = U_0 \sin \frac{\pi x}{L} \cos \frac{\pi y}{L}, \quad v = -U_0 \cos \frac{\pi x}{L} \sin \frac{\pi y}{L}, \quad (164)$$

con $U_0 = 0.05\text{ m/s}$ y $\nabla \cdot \mathbf{u} = 0$ exactamente (verificado discretamente a nivel 10^{-14}). La concentración obedece

$$\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} + v \frac{\partial c}{\partial y} = D \left(\frac{\partial^2 c}{\partial x^2} + \frac{\partial^2 c}{\partial y^2} \right). \quad (165)$$

Se comparan dos regímenes cambiando únicamente D : $Pe = U_0 L / D = 100$ y $Pe = 10\,000$.

5.8.2. Discretización y el parámetro que gobierna todo: Pe_h

La convección se discretiza con upwind de primer orden (seleccionando el nodo aguas arriba según el signo local de u y v) y la difusión con el stencil centrado. El paso de tiempo respeta simultáneamente las Ecs. 155 y 157. El parámetro de diagnóstico es el *número de Péclet de malla*

$$Pe_h = \frac{U_0 \Delta x}{D}, \quad (166)$$

que compara la convección con la difusión *a la escala de una celda*. El esquema upwind introduce una difusión numérica efectiva $D_{\text{num}} \approx U_0 \Delta x / 2$ (Semana 4); por tanto, $Pe_h \gg 2$ implica $D_{\text{num}} \gg D$ y la simulación deja de resolver la física de mezclado real.

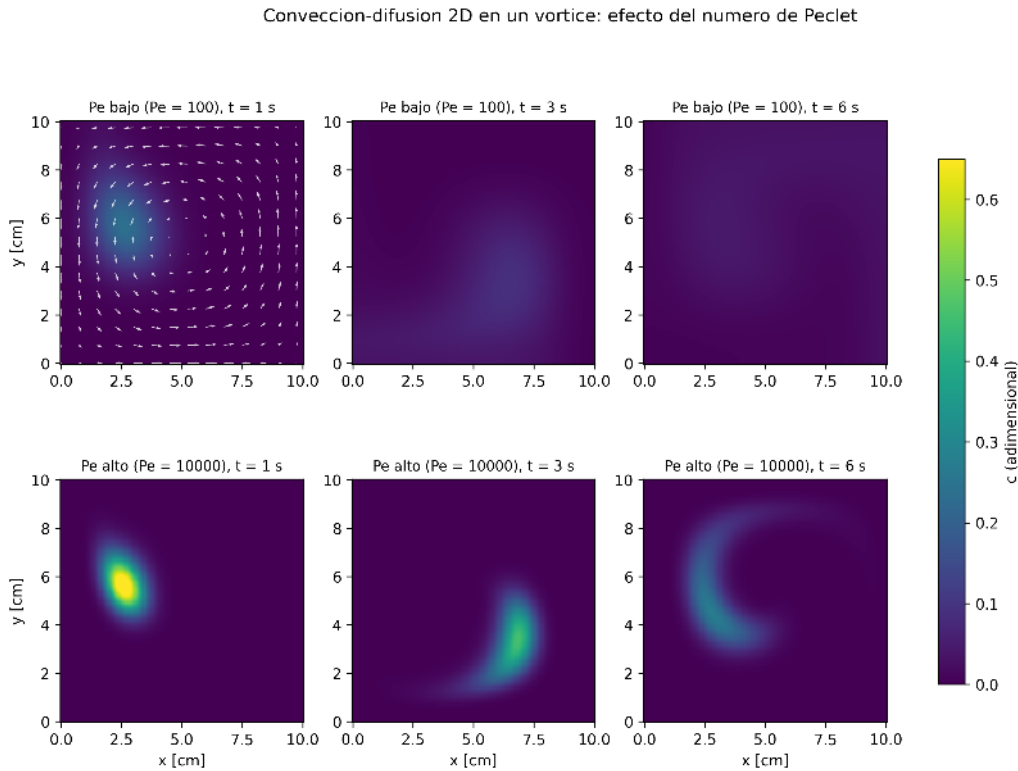


Figura 36: Trazador en el vórtice para dos números de Péclet, en tres instantes ($t = 1, 3$ y 6 s ; flechas: campo de velocidad). Con $Pe = 100$ la mancha se difunde antes de completar una vuelta; con $Pe = 10\,000$ el vórtice estira la mancha en un filamento espiral, mecanismo cinemático que acelera el mezclado. Resultados del Código 4 del Anexo de esta semana.

5.8.3. Resultados y lectura metodológica

1. **Dos físicas distintas con la misma EDP.** Con $Pe = 100$ el pico de concentración cae a 0.043 en 6 s: difusión casi pura. Con $Pe = 10\,000$ el vórtice estira la mancha en filamentos cuyo espesor decrece con el tiempo; la difusión actúa entonces sobre gradientes cada vez más intensos. Este acoplamiento estiramiento–difusión es el mecanismo fundamental del mezclado en flujos laminares.
2. **El diagnóstico Pe_h revela un resultado contaminado.** Para $Pe = 10\,000$ con esta malla, $Pe_h = 62.5$ y $D_{\text{num}} \approx 1.6 \times 10^{-5} \text{ m}^2/\text{s}$, es decir *31 veces* la difusividad física. El campo del renglón inferior de la Figura 36 es cualitativamente correcto pero cuantitativamente dominado por difusión artificial: el pico final (0.273) está fuertemente subestimado respecto a la física real. Reportar Pe_h junto con Pe es una obligación de transparencia metodológica.
3. **La conservación de masa como métrica de verificación.** La forma no conservativa del término convectivo más el tratamiento de las fronteras producen una pérdida de masa del 1.75 % ($Pe = 100$) y 7.3 % ($Pe = 10\,000$) en 6 s. Una formulación en forma de flujo (volúmenes finitos) conservaría masa a precisión de máquina; cuantificar este defecto es parte del reporte, no un detalle omisible.

5.9. Ejemplo 5 — Advección pura bajo rotación sólida: disipación contra dispersión

5.9.1. Planteamiento

Cuando el campo de velocidad se conoce punto a punto (por ejemplo, estimado a partir de secuencias de imágenes aéreas mediante flujo óptico), el transporte de un campo de densidad ρ —densidad vehicular, concentración de partículas, intensidad luminosa— es advección pura:

$$\frac{\partial \rho}{\partial t} + u \frac{\partial \rho}{\partial x} + v \frac{\partial \rho}{\partial y} = 0. \quad (167)$$

El banco de pruebas canónico es la *rotación sólida*: $u = -\omega(y - y_c)$, $v = \omega(x - x_c)$, con $\omega = 2\pi \text{ rad/s}$. Tras exactamente una revolución ($t = 1 \text{ s}$), la solución exacta coincide con la condición inicial; *todo* el error observado es, por construcción, error numérico. La condición inicial combina un cono suave (sensible a la pérdida de amplitud) y un bloque discontinuo (sensible a las oscilaciones espurias).

5.9.2. Esquemas comparados

Se enfrentan los dos arquetipos de la Semana 4, ahora en 2D: upwind de primer orden (disipativo) y MacCormack, el equivalente bidimensional de Lax–Wendroff construido

como predictor–corrector:

$$\tilde{\rho} = \rho^n - \Delta t (u \delta_x^+ \rho^n + v \delta_y^+ \rho^n), \quad (168)$$

$$\rho^{n+1} = \frac{1}{2} [\rho^n + \tilde{\rho} - \Delta t (u \delta_x^- \tilde{\rho} + v \delta_y^- \tilde{\rho})], \quad (169)$$

donde $\delta^\pm$ son diferencias hacia adelante y hacia atrás. Malla de 201×201 , CFL efectivo 0.40, 3 142 pasos por revolución.

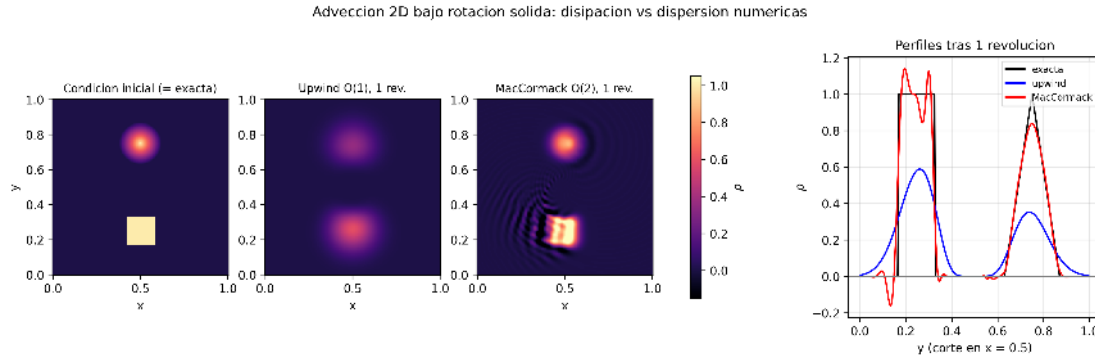


Figura 37: Rotación sólida de un cono suave y un bloque discontinuo tras una revolución completa. El upwind erosiona ambos perfiles (disipación); MacCormack preserva el cono pero genera oscilaciones y sobretiros alrededor del bloque (dispersión). Panel derecho: perfiles por el corte $x = 0.5$ contra la solución exacta. Resultados del Código 5 del Anexo de esta semana.

5.9.3. Resultados y lectura metodológica

Tabla 25: Métricas de error tras una revolución (valores exactos: pico = 1.000, mínimo = 0.000, masa = 100 %).

Esquema	Error L_2	Pico	Mínimo	Masa
Upwind $\mathcal{O}(1)$	0.1130	0.589	0.000	99.7 %
MacCormack $\mathcal{O}(2)$	0.0664	1.425	−0.448	100.0 %

1. **No existe el esquema lineal perfecto.** El upwind destruye el 41 % de la amplitud del pico pero nunca genera valores negativos (es monótono); MacCormack reduce el error L_2 a casi la mitad pero produce sobretiros del 42 % y densidades negativas de hasta −0.448, físicamente inadmisibles si ρ es una densidad. Esta disyuntiva es el contenido del teorema de Godunov: ningún esquema *lineal* de orden mayor que uno es monótono. Los esquemas modernos (limitadores de flujo, TVD, WENO) son no lineales precisamente para evadirlo.
2. **La métrica correcta depende del uso del modelo.** Si la cantidad de interés es la posición del máximo de densidad, MacCormack es superior; si el resultado alimenta un modelo posterior que exige positividad (una tasa, una probabilidad, un

control), el upwind es el único admisible de los dos. La selección de esquema es una decisión acoplada al uso downstream del resultado.

3. **El test con solución exacta es oro metodológico.** Diseñar un caso donde la respuesta correcta se conoce de antemano (aquí, por la simetría de la rotación) permite atribuir el cien por ciento del error al método. Todo capítulo de simulación de una tesis debería incluir al menos un test de este tipo.

5.10. Ejemplo 6 — Reconstrucción de un campo a partir de mediciones dispersas: interpolación armónica

5.10.1. Planteamiento

Una red de n_s sensores calibrados mide un campo escalar suave (temperatura, presión) en posiciones dispersas de una placa, con ruido de medición $\sigma = 0.3$ (en unidades del campo). Se requiere estimar el campo completo. Un estimador natural y computacionalmente idéntico a lo visto esta semana es el *interpolador armónico*: la función que satisface

$$\nabla^2 \hat{T} = 0 \text{ fuera de los sensores,} \quad \hat{T}(\mathbf{x}_s) = T_s^{\text{med}} \text{ en cada sensor,} \quad (170)$$

con Neumann homogénea en el contorno. Esta solución minimiza $\int_{\Omega} \|\nabla \hat{T}\|^2 d\Omega$ sujeta a las mediciones: es el campo *más suave posible* compatible con los datos, el análogo continuo de una membrana elástica clavada en los puntos medidos. Computacionalmente es el mismo ensamblaje disperso del Ejemplo 3, con los sensores actuando como condiciones de Dirichlet *puntuales interiores*.

5.10.2. Estudio de convergencia con la densidad de sensores

El campo verdadero se construye sintéticamente (suma de modos suaves no armónicos) para poder calcular el error exacto de reconstrucción. Se barre $n_s \in \{5, 10, 20, 40, 80, 160\}$ con posiciones aleatorias de semilla fija (reproducibilidad, Semana 1).

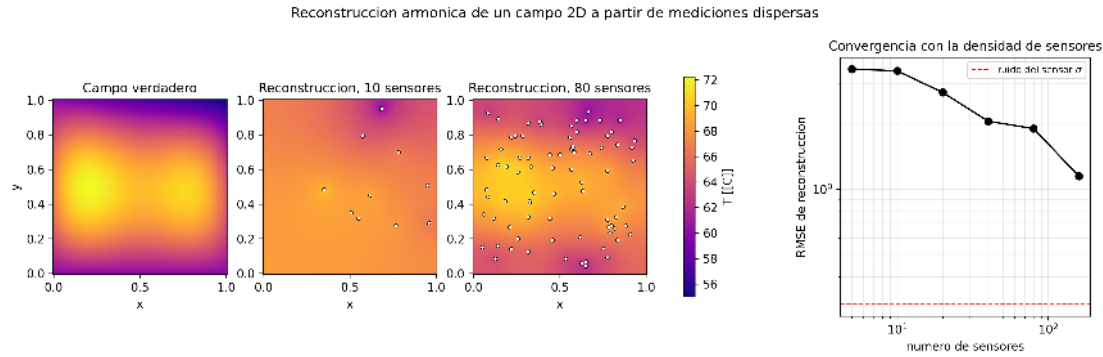


Figura 38: Reconstrucción armónica: campo verdadero, reconstrucciones con 10 y con 80 sensores (puntos blancos) y error RMS en función del número de sensores. La línea punteada marca el ruido de medición $\sigma = 0.3$. Resultados del Código 6 del Anexo de esta semana.

5.10.3. Resultados y lectura metodológica

1. **El error decrece, pero no hasta el ruido.** El RMSE cae de 3.51 con 5 sensores a 1.15 con 160, pero permanece muy por encima del piso de ruido $\sigma = 0.3$. La razón es estructural: el campo verdadero *no es armónico* ($\nabla^2 T_{\text{true}} \neq 0$), de modo que el interpolador tiene un *sesgo de modelo* irreducible que ningún número de sensores elimina por completo entre mediciones. Es la manifestación espacial del balance sesgo–varianza: el error total combina ruido de medición (varianza) y rigidez del modelo de interpolación (sesgo).
2. **La EDP como prior de regularización.** Imponer $\nabla^2 \hat{T} = 0$ equivale a declarar conocimiento previo (“el campo es suave”) que estabiliza un problema con muchas más incógnitas (121^2) que datos (n_s): un problema inverso en el sentido de la Semana 1, regularizado por la física. Si se conoce el término fuente, sustituir Laplace por Poisson reduce directamente el sesgo.
3. **El diseño de la red importa tanto como su tamaño.** Con posiciones aleatorias, regiones sin sensores cercanos al borde quedan pobremente estimadas (visible en la reconstrucción con 10 sensores). El error de reconstrucción como función de la posición es la herramienta cuantitativa para *diseñar* dónde colocar el siguiente sensor.

5.11. Visualización de campos escalares: reglas mínimas

La visualización en 2D no es cosmética: es el instrumento de lectura del resultado, y puede mentir. Reglas operativas usadas en todas las figuras de esta semana:

1. **Escala de color común** en todo conjunto de paneles que se compara entre sí (mis-mos $v_{\min}/v_{\max}$); de lo contrario la comparación visual es inválida. En la Figura 35,

la diferencia dramática entre configuraciones solo es legible porque las tres comparten escala.

2. **Mapas perceptualmente uniformes** (viridis, inferno, magma): la luminosidad varía monótonamente con el valor, de modo que gradientes iguales se perciben iguales. El clásico jet crea bandas artificiales y debe evitarse en material publicable.
3. **Mapas divergentes** (RdBu) solo para campos con cero físico significativo (la amplitud de onda de la Figura 34), centrando el blanco en cero.
4. **Un corte 1D acompaña al mapa 2D.** El perfil sobre una línea (Figuras 35 y 37) restituye la lectura cuantitativa que el mapa de color solo sugiere.
5. **El tiempo se muestra con instantáneas a tiempos idénticos entre casos** más una serie temporal de un escalar resumen ($T_{\text{máx}}(t)$, $E(t)$): las animaciones son útiles para explorar, pero el documento científico exige instantes comparables.

5.12. Síntesis de la semana

Tabla 26: Resumen de los seis ejemplos: familia de EDP, técnica central y lección metodológica.

Ej.	EDP	Técnica central	Lección metodológica
1	Parabólica	Máscara circular; media armónica en interfaces	Validar contra balance 0D; el retardo difusivo protege el núcleo
2	Hiperbólica	Leapfrog; CFL 2D $\leq 1/\sqrt{2}$	Monitorear un invariante físico (energía) como verificación
3	Elíptica	Ensamblaje disperso en dominio perforado	La condición de frontera interna puede invertir la conclusión
4	Conv.-difusión	Upwind 2D; Pe_h	Reportar Pe_h ; la difusión numérica puede dominar
5	Advección pura	Upwind vs MacCormack; test exacto	Disipación vs dispersión: teorema de Godunov
6	Elíptica (inversa)	Dirichlet puntual interior	La EDP como regularizador; sesgo de modelo vs ruido

La Semana 6 introducirá los métodos Monte Carlo: cuando la incertidumbre de los parámetros es parte esencial del problema, los campos deterministas de esta semana se convierten en distribuciones, y cada simulación 2D pasa a ser una muestra de un experimento estadístico.

Apéndice Semana 5. Referencias verificables

- A1. LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM. <https://doi.org/10.1137/1.9780898717839>

- A2. Courant, R., Friedrichs, K. y Lewy, H. (1928). Über die partiellen Differenzengleichungen der mathematischen Physik. *Mathematische Annalen*, 100, 32–74. <https://doi.org/10.1007/BF01448839>
- A3. Patankar, S. V. (1980). *Numerical Heat Transfer and Fluid Flow*. Hemisphere / CRC Press. <https://doi.org/10.1201/9781482234213>
- A4. MacCormack, R. W. (1969). The effect of viscosity in hypervelocity impact cratering. *AIAA Paper* 69–354. <https://doi.org/10.2514/6.1969-354>
- A5. Zalesak, S. T. (1979). Fully multidimensional flux-corrected transport algorithms for fluids. *Journal of Computational Physics*, 31(3), 335–362. [https://doi.org/10.1016/0021-9991\(79\)90051-2](https://doi.org/10.1016/0021-9991(79)90051-2)
- A6. Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems* (2nd ed.). SIAM. <https://doi.org/10.1137/1.9780898718003>

5.13. Anexo Semana 5. Códigos para Google Colab

Propósito del anexo. Los seis códigos siguientes corresponden exactamente a las simulaciones cuyos resultados se discutieron en la sección principal. Cada código está completamente comentado y es autocontenido: basta copiarlo en una celda de Google Colab y ejecutarlo. Todos los parámetros físicos están agrupados en bloques claramente delimitados; modificar un parámetro y re-ejecutar la celda es la forma de realizar análisis de sensibilidad. Los archivos de figuras que generan estos scripts (.png, 300 dpi) son los mismos que aparecen en el documento.

5.13.1. Código 1 — Difusión 2D en sección compuesta (máscara circular y media armónica)

```

1      """
2      Semana 5 - Ejemplo 1: Difusion de calor 2D
3      transitoria en una seccion
4      transversal compuesta (dominio circular con dos
5      materiales)
6
7      Aplicacion generica: cualquier componente cilindrico
8      con generacion
9      interna de calor (un cable compuesto, un eje, una
10     barra de combustible).
11     La seccion contiene un nucleo de baja conductividad
12     (p.ej. un tubo
13     con fibras opticas) rodeado por una corona
14     conductora donde se

```

```

9         disipa calor por efecto Joule.
10
11         Fisica:  $\rho * c_p * \frac{dT}{dt} = \text{div}(k(x,y) \text{ grad } T) + Q(x$ 
12         ,y)
13         Esquema: FTCS 2D (stencil de 5 puntos) con
14         conductividad variable
15         evaluada con media armonica en las caras de celda.
16         Geometria: dominio circular embebido en una malla
17         cartesiana
18         mediante una mascara booleana (metodo de frontera
19         escalonada).
20         """
21
22         #
23         =====
24         # BLOQUE 1: Librerias
25         #
26         =====
27
28         import numpy as np
29         import matplotlib
30         matplotlib.use('Agg')
31         import matplotlib.pyplot as plt
32         import matplotlib as mpl
33
34         mpl.rcParams.update({
35             'font.size': 10, 'axes.titlesize': 11, 'axes.
36             labelsizsize': 10,
37             'figure.dpi': 110, 'savefig.dpi': 300,
38             })
39
40         #
41         =====
42         # BLOQUE 2: Parametros fisicos
43         # Dos materiales: nucleo (tubo polimerico) y corona
44         (aleacion Al)
45         #
46         =====
47
48         R_ext    = 6.0e-3          # [m]  radio exterior de la
49         seccion
50
51         R_nuc    = 1.8e-3          # [m]  radio del nucleo de
52         baja conductividad
53

```

```

39      # Material 1: nucleo polimerico (protege elementos
      sensibles)
40      k1, rho1, cp1 = 0.30, 1400.0, 1200.0      # [W/m/K], [
      kg/m3], [J/kg/K]
41      # Material 2: corona metalica (aluminio, donde se
      genera el calor)
42      k2, rho2, cp2 = 200.0, 2700.0, 900.0
43
44      # Generacion volumetrica de calor (efecto Joule)
      SOLO en la corona
45      Q_J = 4.0e8      # [W/m3] densidad de potencia
      durante el transitorio
46      t_pulso = 0.30      # [s] duracion del pulso de
      corriente
47      t_total = 0.60      # [s] tiempo total simulado
      (pulso + enfriamiento)
48
49      T0      = 25.0      # [C] temperatura inicial
      uniforme
50      h_cv      = 25.0      # [W/m2/K] conveccion natural
      en la superficie
51      T_inf = 25.0      # [C] temperatura ambiente
52
53      #
      =====
54      # BLOQUE 3: Malla cartesiana 2D y mascara del
      dominio circular
55      #
      =====
56      N      = 121      # nodos por
      direccion (malla N x N)
57      L      = 2.0 * R_ext * 1.05      # lado del
      cuadrado que contiene al circulo
58      dx = L / (N - 1)
59      x      = np.linspace(-L/2, L/2, N)
60      X, Y      = np.meshgrid(x, x, indexing='xy')
61      r      = np.sqrt(X**2 + Y**2)
62
63      dentro      = r <= R_ext      # mascara del
      dominio fisico
64      nucleo      = r <= R_nuc      # subdominio 1
65      corona      = dentro & ~nucleo      # subdominio 2
    
```

```

66         # frontera: celdas del dominio con al menos un
vecino fuera
67         front = dentro & (
68         ~np.roll(dentro, 1, 0) | ~np.roll(dentro, -1, 0) |
69         ~np.roll(dentro, 1, 1) | ~np.roll(dentro, -1, 1))
70
71         # Campos de propiedades por celda
72         k_map = np.where(nucleo, k1, k2)
73         rc_map = np.where(nucleo, rho1*cp1, rho2*cp2)
74         Q_map = np.where(corona, Q_J, 0.0)
75
76         #
=====
77         # BLOQUE 4: Paso de tiempo estable (criterio FTCS 2D
)
78         # dt <= dx^2 / (4 * alpha_max)
79         #
=====
80         alpha_max = (k_map / rc_map)[dentro].max()
81         dt = 0.40 * dx**2 / (4.0 * alpha_max)      # margen
de seguridad
82         n_pasos = int(np.ceil(t_total / dt))
83         print(f"dx = {dx*1e3:.3f} mm | alpha_max = {
alpha_max:.3e} m2/s")
84         print(f"dt = {dt*1e6:.2f} us | pasos = {n_pasos}")
85
86         #
=====
87         # BLOQUE 5: Conductividades de cara (media armonica)
88         # k_face = 2 k_i k_j / (k_i + k_j) garantiza
continuidad de flujo
89         # en la interfaz entre materiales
90         #
=====
91         def k_face(ka, kb):
92         return 2.0 * ka * kb / (ka + kb)
93
94         kE = k_face(k_map, np.roll(k_map, -1, 1))    # cara
este
95         kW = k_face(k_map, np.roll(k_map, 1, 1))     # cara
oeste

```

```

96         kN = k_face(k_map, np.roll(k_map, -1, 0))    # cara
norte
97         kS = k_face(k_map, np.roll(k_map, 1, 0))    # cara
sur
98
99         # Anular el flujo a traves de caras que cruzan la
frontera del circulo:
100         # equivale a imponer condicion de Neumann homogenea
(adiabatica) en la
101         # frontera escalonada. La perdida convectiva se
agrega como sumidero.
102         kE[~np.roll(dentro, -1, 1)] = 0.0
103         kW[~np.roll(dentro, 1, 1)] = 0.0
104         kN[~np.roll(dentro, -1, 0)] = 0.0
105         kS[~np.roll(dentro, 1, 0)] = 0.0
106
107         #
=====
108         # BLOQUE 6: Bucle temporal FTCS con fuente conmutada
109         # La conveccion superficial se modela como sumidero
volumetrico
110         # en las celdas de frontera: q_cv = h (T - T_inf) *
(P/A) ~ h(T-T_inf)/dx
111         #
=====
112         T = np.full((N, N), T0)
113         hist_t, hist_Tmax, hist_Tnuc = [], [], []
114         snaps = {}
115         t_snap = [0.05, 0.30, 0.60]
116         idx_c = N // 2                                # nodo central
117         j_int = idx_c + int(R_nuc/dx) - 2              # nodo dentro
del nucleo, cerca de la interfaz
118         hist_Tint = []
119
120         t = 0.0
121         for n in range(n_pasos):
122             Q_n = Q_map if t < t_pulso else np.zeros_like(Q_map)
123
124             TE = np.roll(T, -1, 1); TW = np.roll(T, 1, 1)
125             TN = np.roll(T, -1, 0); TS = np.roll(T, 1, 0)
126             lap = (kE*(TE-T) + kW*(TW-T) + kN*(TN-T) + kS*(TS-T)
) / dx**2

```

```

127
128         sumidero = np.zeros_like(T)
129         sumidero[front] = h_cv * (T[front] - T_inf) / dx    #
130         [W/m3]
131
132         T = T + dt * (lap + Q_n - sumidero) / rc_map
133         T[~dentro] = T0                                     # el exterior no
134         participa
135         t += dt
136
137         hist_t.append(t)
138         hist_Tmax.append(T[dentro].max())
139         hist_Tnuc.append(T[idx_c, idx_c])
140         hist_Tint.append(T[idx_c, j_int])
141         for ts in t_snap:
142             if ts not in snaps and t >= ts:
143                 snaps[ts] = T.copy()
144
145         print(f"T_max global          = {max(hist_Tmax):.1f} C"
146               )
147         print(f"T_max en el nucleo = {max(hist_Tnuc):.1f} C"
148               )
149         print(f"T_nucleo al final   = {hist_Tnuc[-1]:.1f} C")
150         print(f"T_max al final      = {hist_Tmax[-1]:.1f} C")
151         print(f"T interfaz nucleo (max) = {max(hist_Tint):.1
152               f} C")
153
154         #
155         =====
156         # BLOQUE 7: Visualizacion (campo escalar + historia
157         temporal)
158         #
159         =====
160
161         fig, axs = plt.subplots(1, 4, figsize=(15, 3.8),
162                                gridspec_kw={'width_ratios': [1, 1, 1, 1.25]})
163         vmax = max(hist_Tmax)
164         for ax, ts in zip(axs[:3], t_snap):
165             Tp = np.where(dentro, snaps[ts], np.nan)
166             im = ax.pcolormesh(X*1e3, Y*1e3, Tp, cmap='inferno',
167                               vmin=T0, vmax=vmax, shading='auto')
168             th = np.linspace(0, 2*np.pi, 200)

```

```

160         ax.plot(R_nuc*1e3*np.cos(th), R_nuc*1e3*np.sin(th),
161               'c--', lw=1)
162         ax.set_title(f"t = {ts:.2f} s")
163         ax.set_xlabel("x [mm]"); ax.set_aspect('equal')
164         axs[0].set_ylabel("y [mm]")
165         fig.colorbar(im, ax=axs[:3], shrink=0.85, label="T
166         [[C]])")
167
168         axs[3].plot(hist_t, hist_Tmax, 'r-', lw=1.6, label=r
169         "$T_{max}$ (corona)")
170         axs[3].plot(hist_t, hist_Tint, 'g-', lw=1.6, label=r
171         "$T$ interfaz nucleo")
172         axs[3].plot(hist_t, hist_Tnuc, 'b-', lw=1.6, label=r
173         "$T$ centro del nucleo")
174         axs[3].axvline(t_pulso, color='k', ls=':', lw=1)
175         axs[3].text(t_pulso*1.02, T0+5, "fin del pulso",
176         rotation=90, fontsize=8)
177         axs[3].set_xlabel("t [s]"); axs[3].set_ylabel("T [[C
178         ]])")
179
180         axs[3].set_title("Historia termica")
181         axs[3].legend(fontsize=8); axs[3].grid(alpha=0.3)
182
183         fig.suptitle("Difusion 2D en seccion compuesta:
184         pulso Joule y enfriamiento",
185         y=1.02)
186         plt.savefig('Figuras/fig_s5_ej1_difusion2D_cable.png
187         ',
188         dpi=300, bbox_inches='tight')
189         # plt.show()
190         print("Figura guardada.")
191

```

Listing 31: Semana 5 – Código 1: Difusión de calor 2D transitoria (FTCS, stencil de 5 puntos) en una sección circular de dos materiales con fuente Joule conmutada. Ejecutar en Google Colab.

5.13.2. Código 2 — Onda 2D con rendija (leapfrog y conservación de energía)

```

1         """
2         Semana 5 - Ejemplo 2: Ecuacion de onda 2D con
3         obstaculo --
4         reflexion y difraccion a traves de una rendija

```



```

5      Aplicacion generica: propagacion de perturbaciones
      mecanicas u
6      opticas en un medio bidimensional (ondas de presion
      generadas por
7      una descarga, vibraciones en una membrana, frentes
      de onda en
8      metrologia optica). El obstaculo con rendija
      reproduce el
9      experimento clasico de difraccion.
10
11     Fisica:  $d^2u/dt^2 = c^2 (d^2u/dx^2 + d^2u/dy^2)$ 
12     Esquema: leapfrog explicito de segundo orden en
      tiempo y espacio.
13     Estabilidad (CFL 2D):  $c*dt/dx \leq 1/\sqrt{2}$ .
14     """
15
16     #
17     =====
18     # BLOQUE 1: Librerias
19     #
20     =====
21     import numpy as np
22     import matplotlib
23     matplotlib.use('Agg')
24     import matplotlib.pyplot as plt
25     import matplotlib as mpl
26
27     mpl.rcParams.update({
28         'font.size': 10, 'axes.titlesize': 11, 'axes.
29         labelsize': 10,
30         'figure.dpi': 110, 'savefig.dpi': 300,
31     })
32
33     #
34     =====
35     # BLOQUE 2: Parametros fisicos y geometricos
36     #
37     =====
38     Lx, Ly = 1.0, 1.0      # [m]      dominio cuadrado
39     c       = 340.0        # [m/s]    velocidad de
      propagacion (aire)

```

```

35         T_fin = 2.6e-3           # [s]      tiempo total
simulado
36
37         # Fuente: pulso gaussiano inicial centrado en (x0,
y0)
38         x0, y0 = 0.25, 0.50     # [m]      posicion de la
fuente
39         sigma = 0.025           # [m]      ancho del pulso
40
41         # Obstaculo: pared vertical en x = x_b con una
rendija centrada
42         x_b = 0.50              # [m]      posicion de la
pared
43         espesor = 0.02          # [m]      espesor de la
pared
44         a_rendija = 0.10        # [m]      apertura de la
rendija
45
46         #
=====
47         # BLOQUE 3: Malla y numero de Courant 2D
48         #
=====
49
50         N = 401
51         dx = Lx / (N - 1)
52         x = np.linspace(0, Lx, N)
53         X, Y = np.meshgrid(x, x, indexing='xy')
54
55         # CFL 2D: elegimos  $S = c \, dt/dx = 0.5 < 1/\sqrt{2} \sim$ 
0.707
56
57         S = 0.5
58         dt = S * dx / c
59         n_pasos = int(np.ceil(T_fin / dt))
60         print(f"dx = {dx*1e3:.2f} mm | dt = {dt*1e6:.2f} us
| pasos = {n_pasos}")
61         print(f"Numero de Courant 2D: S = {S} (limite 1/sqrt
(2) = {1/np.sqrt(2):.3f})")
62
63         # Mascara del obstaculo (True = pared rigida, u = 0)
pared = (np.abs(X - x_b) <= espesor/2) & (np.abs(Y -
Ly/2) > a_rendija/2)

```

```

64      #
=====
65      # BLOQUE 4: Condiciones iniciales
66      # u^0: pulso gaussiano; u^1 obtenido con arranque de
        Taylor
67      # u^1 = u^0 + (dt^2 c^2 / 2) lap(u^0)    (velocidad
        inicial nula)
68      #
=====
69      def laplaciano(u):
70          lap = np.zeros_like(u)
71          lap[1:-1, 1:-1] = (u[1:-1, 2:] + u[1:-1, :-2] +
72          u[2:, 1:-1] + u[:-2, 1:-1] -
73          4.0 * u[1:-1, 1:-1]) / dx**2
74          return lap
75
76          u0 = np.exp(-((X - x0)**2 + (Y - y0)**2) / (2 *
        sigma**2))
77          u0[pared] = 0.0
78          u1 = u0 + 0.5 * (c * dt)**2 * laplaciano(u0)
79          u1[pared] = 0.0
80
81      #
=====
82      # BLOQUE 5: Bucle temporal leapfrog
83      # u^{n+1} = 2u^n - u^{n-1} + (c dt)^2 lap(u^n)
84      # Fronteras exteriores: Dirichlet u = 0 (paredes
        rigidas)
85      # Energia discreta: E = sum( (du/dt)^2 + c^2 |grad u
        |^2 ) dx^2 / 2
86      #
=====
87          u_prev, u = u0.copy(), u1.copy()
88          snaps, hist_t, hist_E = {}, [], []
89          t_snap = [0.5e-3, 1.2e-3, 2.4e-3]
90
91          t = dt
92          for n in range(n_pasos):
93              u_next = 2.0*u - u_prev + (c*dt)**2 * laplaciano(u)
94              u_next[pared] = 0.0          # obstaculo rigido (
        Dirichlet interno)
95              u_next[0, :] = u_next[-1, :] = 0.0

```

```

96         u_next[:, 0] = u_next[:, -1] = 0.0
97
98         # Energia discreta (diagnostico de conservacion)
99         ut = (u_next - u_prev) / (2*dt)
100         ux = (u[1:-1, 2:] - u[1:-1, :-2]) / (2*dx)
101         uy = (u[2:, 1:-1] - u[:-2, 1:-1]) / (2*dx)
102         E = 0.5*np.sum(ut[1:-1, 1:-1]**2)*dx**2 \
103         + 0.5*c**2*(np.sum(ux**2) + np.sum(uy**2))*dx**2
104         hist_t.append(t); hist_E.append(E)
105
106         u_prev, u = u, u_next
107         t += dt
108         for ts in t_snap:
109             if ts not in snaps and t >= ts:
110                 snaps[ts] = u.copy()
111
112         E = np.array(hist_E)
113         deriva = abs(E[-1] - E[len(E)//4]) / E[len(E)//4] *
100
114         print(f"Deriva de energia tras el transitorio
inicial: {deriva:.2f} %")
115
116         #
=====
117         # BLOQUE 6: Visualizacion
118         #
=====
119         fig, axs = plt.subplots(1, 4, figsize=(15.5, 3.9),
120         gridspec_kw={'width_ratios': [1, 1, 1, 1.2]})
121         for ax, ts in zip(axs[:3], t_snap):
122             up = np.ma.array(snaps[ts], mask=pared)
123             im = ax.pcolormesh(X, Y, up, cmap='RdBu_r', vmin
=-0.12, vmax=0.12,
124             shading='auto')
125             ax.add_patch(plt.Rectangle((x_b-espesor/2, 0),
espesor,
126             Ly/2 - a_rendija/2, color='k'))
127             ax.add_patch(plt.Rectangle((x_b-espesor/2, Ly/2 +
a_rendija/2),
128             espesor, Ly/2 - a_rendija/2, color='k'))
129             ax.set_title(f"t = {ts*1e3:.1f} ms")
130             ax.set_xlabel("x [m]"); ax.set_aspect('equal')

```

```

131         axs[0].set_ylabel("y [m]")
132         fig.colorbar(im, ax=axs[:3], shrink=0.85, label="u (
    amplitud)")
133
134         axs[3].plot(np.array(hist_t)*1e3, E/E[len(E)//4], 'k
    -', lw=1.4)
135         axs[3].set_xlabel("t [ms]")
136         axs[3].set_ylabel(r"$E(t)/E_{ref}$")
137         axs[3].set_title("Conservacion de energia")
138         axs[3].grid(alpha=0.3); axs[3].set_ylim(0.9, 1.1)
139
140         fig.suptitle("Onda 2D: reflexion en la pared y
    difraccion por la rendija",
141             y=1.03)
142         plt.savefig('Figuras/fig_s5_ej2_onda2D_difraccion.
    png',
143             dpi=300, bbox_inches='tight')
144         # plt.show()
145         print("Figura guardada.")
146

```

Listing 32: Semana 5 – Código 2: Ecuación de onda 2D con esquema leapfrog, obstáculo con rendija (difracción) y monitoreo de la energía discreta. Ejecutar en Google Colab.

5.13.3. Código 3 — Poisson 2D en placa perforada (ensamblaje disperso)

```

1         """
2         Semana 5 - Ejemplo 3: Ecuacion de Poisson 2D en un
    dominio con
3         perforaciones -- calor de hidratacion en una losa
    aligerada
4
5         Aplicacion generica: elementos estructurales de
    concreto con
6         cavidades (losas reticulares o aligeradas). Durante
    el fraguado, la
7         hidratacion del cemento genera calor; el campo
    estacionario de
8         temperatura satisface la ecuacion de Poisson. Las
    cavidades reducen
9         el volumen que genera calor y modifican la geometria
    del dominio.

```

```

10
11         Física:       $-k * \text{lap}(T) = q_h$            en el concreto
12         T = T_amb           en el contorno exterior (
Dirichlet)
13         dT/dn = 0           en las paredes de las
cavidades
14         (Neumann homogenea, aislamiento aprox.)
15         Metodo:   ensamblaje disperso (scipy.sparse) del
stencil de 5 puntos
16         SOLO sobre las celdas activas del dominio
enmascarado.
17         """
18
19         #
=====
20         # BLOQUE 1: Librerias
21         #
=====
22         import numpy as np
23         import matplotlib
24         matplotlib.use('Agg')
25         import matplotlib.pyplot as plt
26         import matplotlib as mpl
27         from scipy.sparse import lil_matrix, csr_matrix
28         from scipy.sparse.linalg import spsolve
29
30         mpl.rcParams.update({
31             'font.size': 10, 'axes.titlesize': 11, 'axes.
labelsize': 10,
32             'figure.dpi': 110, 'savefig.dpi': 300,
33         })
34
35         #
=====
36         # BLOQUE 2: Parametros fisicos
37         #
=====
38         Lx, Ly = 3.0, 3.0           # [m]           planta de la losa
(vista en planta)
39         k_c      = 1.8               # [W/m/K]       conductividad del
concreto

```

```

40         q_h      = 250.0          # [W/m3]    generacion por
        hidratacion (pico tipico)
41         T_amb    = 22.0          # [C]        temperatura del
        contorno
42
43         # Patron de cavidades: arreglo de nc x nc huecos
        cuadrados
44         nc        = 4            # huecos por direccion
45         l_hueco   = 0.45         # [m] lado de cada hueco
46
47         #
        =====
48         # BLOQUE 3: Malla y mascaras (solida vs aligerada)
49         #
        =====
50
51         N = 181
52         dx = Lx / (N - 1)
53         x = np.linspace(0, Lx, N)
54         X, Y = np.meshgrid(x, x, indexing='xy')
55
56         def mascara_aligerada():
57             """True = celda de concreto (activa)."""
58             act = np.ones((N, N), dtype=bool)
59             paso = Lx / nc
60             for ii in range(nc):
61                 for jj in range(nc):
62                     cx = (ii + 0.5) * paso
63                     cy = (jj + 0.5) * paso
64                     hueco = (np.abs(X - cx) <= l_hueco/2) & \
65                             (np.abs(Y - cy) <= l_hueco/2)
66                     act &= ~hueco
67             return act
68
69         masc_solida = np.ones((N, N), dtype=bool)
70         masc_aligerada = mascara_aligerada()
71         frac_vol = masc_aligerada.sum() / masc_solida.sum()
72         print(f"Fraccion de volumen de la losa aligerada: {
        frac_vol*100:.1f} %")
73
74         #
        =====
        # BLOQUE 4: Ensamblaje disperso sobre celdas activas

```

```

75         # Numeramos las incognitas con un mapa (i,j) ->
       indice global.
76         # Dirichlet en el contorno exterior; Neumann
       homogenea en los
77         # bordes internos (simplemente se omite la conexi3n
       al hueco y
78         # el coeficiente diagonal cuenta solo los vecinos
       activos).
79         #
       =====
80         def resolver_poisson(activa, bc_hueco='neumann'):
81             """bc_hueco: 'neumann' (cavidad sellada, adiabatica)
       o
82             'dirichlet' (cavidad ventilada a T_amb)."""
83             interior = activa.copy()
84             interior[0, :] = interior[-1, :] = False
85             interior[:, 0] = interior[:, -1] = False
86
87             idx = -np.ones((N, N), dtype=int)
88             nodos = np.argwhere(interior)
89             idx[interior] = np.arange(len(nodos))
90             n_inc = len(nodos)
91
92             A = lil_matrix((n_inc, n_inc))
93             b = np.full(n_inc, q_h / k_c * dx**2) # termino
       fuente
94             vecinos = [(0, 1), (0, -1), (1, 0), (-1, 0)]
95
96             for p, (i, j) in enumerate(nodos):
97                 diag = 0.0
98                 for di, dj in vecinos:
99                     ii, jj = i + di, j + dj
100                     if not activa[ii, jj]:
101                         if bc_hueco == 'dirichlet':
102                             diag += 1.0
103                             b[p] += T_amb # cavidad ventilada a T_amb
104                             continue # 'neumann': flujo nulo
105                             diag += 1.0
106                             if interior[ii, jj]:
107                                 A[p, idx[ii, jj]] = -1.0
108                             else:

```



```

109         b[p] += T_amb                                # vecino Dirichlet
    exterior
110         A[p, p] = diag
111         A = csr_matrix(A)
112
113         T = np.full((N, N), np.nan)
114         T[interior] = spsolve(A, b)
115         T[activa & ~interior] = T_amb                # contorno
    exterior
116         return T, A, n_inc
117
118         T_sol, A_sol, n_sol = resolver_poisson(masc_solida)
119         T_ali, A_ali, n_ali = resolver_poisson(
120 masc_aligerada, 'neumann')
121         T_ven, A_ven, n_ven = resolver_poisson(
122 masc_aligerada, 'dirichlet')
123
124         print(f"Losa solida:      {n_sol} incognitas, nnz = {
125 A_sol.nnz}")
126         print(f"Losa aligerada: {n_ali} incognitas, nnz = {
127 A_ali.nnz}")
128         print(f"Densidad de A (aligerada): "
129 f"{A_ali.nnz / n_ali**2 * 100:.4f} % de entradas no
130 nulas")
131         print(f"T_max solida                                = {np.nanmax(
132 T_sol):.2f} C")
133         print(f"T_max aligerada (sellada)                  = {np.nanmax(
134 T_ali):.2f} C")
135         print(f"T_max aligerada (ventilada)                = {np.nanmax(
136 T_ven):.2f} C")
137
138         #
139         =====
140         # BLOQUE 5: Visualizacion comparativa
141         #
142         =====
143         fig, axs = plt.subplots(1, 4, figsize=(16, 4.0),
144 gridspec_kw={'width_ratios': [1, 1, 1, 1.15]})
145         vmax = np.nanmax(T_ali)
146         for ax, T, titulo in zip(
147 axs[:3], [T_sol, T_ali, T_ven],
148 ["Solida", "Aligerada, cavidades selladas",

```

```

139         "Aligerada, cavidades ventiladas"]):
140         im = ax.pcolormesh(X, Y, T, cmap='inferno', vmin=
T_amb, vmax=vmax,
141         shading='auto')
142         ax.set_title(titulo, fontsize=10)
143         ax.set_xlabel("x [m]"); ax.set_aspect('equal')
144         axs[0].set_ylabel("y [m]")
145         fig.colorbar(im, ax=axs[:3], shrink=0.85, label="T
[[C]]")
146
147         # Perfil por la linea media y = Ly/2
148         jm = N // 2
149         axs[3].plot(x, T_sol[jm, :], 'r-', lw=1.6, label="
solida")
150         axs[3].plot(x, T_ali[jm, :], 'b-', lw=1.6, label="
sellada")
151         axs[3].plot(x, T_ven[jm, :], 'g-', lw=1.6, label="
ventilada")
152         axs[3].set_xlabel("x [m]"); axs[3].set_ylabel("T [[C
]]")
153         axs[3].set_title(r"Perfil en $y = L_y/2$")
154         axs[3].legend(fontsize=8); axs[3].grid(alpha=0.3)
155
156         fig.suptitle("Poisson 2D con geometria perforada:
calor de hidratacion",
157         y=1.02)
158         plt.savefig('Figuras/fig_s5_ej3_poisson2D_losa.png',
159         dpi=300, bbox_inches='tight')
160         # plt.show()
161         print("Figura guardada.")
162
    
```

Listing 33: Semana 5 – Código 3: Ecuación de Poisson 2D sobre un dominio con cavidades, ensamblada en formato disperso (scipy.sparse) con condiciones Neumann o Dirichlet en las fronteras internas. Ejecutar en Google Colab.

5.13.4. Código 4 — Convección-difusión 2D en un vórtice (Péclet de malla)

```

1         """
2         Semana 5 - Ejemplo 4: Conveccion-difusion 2D de un
escalar pasivo
3         en un flujo recirculante (vortice)
    
```

```

4
5      Aplicacion generica: transporte de un trazador (
6      colorante, calor,
7      contaminante) en el seno de un fluido en
8      recirculacion, como ocurre
9      en camaras de mezclado, tanques agitados o el
10     interior de equipos de
11     bombeo. El campo de velocidad se impone
12     analiticamente (vortice de
13     una celda) y el escalar se transporta segun la EDP
14     de conveccion-
15     difusion.
16
17     Fisica:  $dc/dt + u \, dc/dx + v \, dc/dy = D (d^2c/dx^2 +$ 
18      $d^2c/dy^2)$ 
19     Campo de velocidad solenoidal ( $\text{div } u = 0$ ):
20      $u = U0 * \sin(\pi x/L) * \cos(\pi y/L)$ 
21      $v = -U0 * \cos(\pi x/L) * \sin(\pi y/L)$ 
22     Esquema: upwind de primer orden (conveccion) +
23     centrado (difusion).
24     Estabilidad:  $dt \leq \min( dx/(|u|+|v|), \, dx^2/(4D) )$ .
25     El numero de Peclet de malla  $Pe_h = |u| \, dx / D$ 
26     controla el regimen.
27     """
28
29     #
30
31     =====
32     # BLOQUE 1: Librerias
33     #
34     =====
35
36     import numpy as np
37     import matplotlib
38     matplotlib.use('Agg')
39     import matplotlib.pyplot as plt
40     import matplotlib as mpl
41
42     mpl.rcParams.update({
43         'font.size': 10, 'axes.titlesize': 11, 'axes.
44         labelsizesize': 10,
45         'figure.dpi': 110, 'savefig.dpi': 300,
46     })

```

```

35         #
=====
36         # BLOQUE 2: Parametros fisicos
37         # Dos regimenes: dominado por difusion (Pe bajo) y
por
38         # conveccion (Pe alto). Solo cambia D.
39         #
=====
40         L   = 0.10           # [m]      lado de la cavidad
41         U0   = 0.05           # [m/s]    velocidad
caracteristica del vortice
42         T_fin = 6.0           # [s]      tiempo simulado (~3
vueltas del vortice)
43
44         casos = {
45             "Pe bajo": 5.0e-5,    # [m2/s]  D grande  -> Pe
~ 100
46             "Pe alto": 5.0e-7,    # [m2/s]  D pequeno -> Pe
~ 10000
47         }
48
49         # Mancha inicial de trazador: gaussiana descentrada
50         x0, y0, s0 = 0.05, 0.075, 0.006    # [m]
51
52         #
=====
53         # BLOQUE 3: Malla y campo de velocidad analitico
54         #
=====
55         N   = 161
56         dx  = L / (N - 1)
57         x   = np.linspace(0, L, N)
58         X, Y = np.meshgrid(x, x, indexing='xy')
59
60         U = U0 * np.sin(np.pi * X / L) * np.cos(np.pi * Y /
L)
61         V = -U0 * np.cos(np.pi * X / L) * np.sin(np.pi * Y /
L)
62         print(f"max |u| = {np.abs(U).max():.3f} m/s ; "
63               f"div u (maximo discreto) ~ "
64               f"{np.abs(np.gradient(U, dx, axis=1) + np.gradient(V
, dx, axis=0)).max():.2e}")

```

```

65
66      #
=====
67      # BLOQUE 4: Esquema upwind 2D conservando el signo
local
68      #
=====
69      def derivada_upwind(c, vel, eje):
70          """Derivada de primer orden contra la corriente."""
71          atras = (c - np.roll(c, 1, axis=eje)) / dx #
72          usa nodo aguas arriba si vel>0
          adelante = (np.roll(c, -1, axis=eje) - c) / dx #
73          si vel<0
          return np.where(vel > 0, atras, adelante)
74
75      def laplaciano(c):
76          return (np.roll(c, -1, 1) + np.roll(c, 1, 1) +
77                  np.roll(c, -1, 0) + np.roll(c, 1, 0) - 4*c) / dx**2
78
79      resultados = {}
80      for nombre, D in casos.items():
81          Pe = U0 * L / D
82          Pe_h = U0 * dx / D
83          dt_adv = dx / (np.abs(U).max() + np.abs(V).max())
84          dt_dif = dx**2 / (4.0 * D)
85          dt = 0.4 * min(dt_adv, dt_dif)
86          n_pasos = int(np.ceil(T_fin / dt))
87          D_num = U0 * dx / 2.0 # difusion numerica del
upwind 0(u dx/2)
88          print(f"\n{nombre}: Pe = {Pe:.0f} | Pe_h = {Pe_h:.1f}
} | "
89          f"dt = {dt*1e3:.2f} ms | pasos = {n_pasos}")
90          print(f"    D fisica = {D:.1e} m2/s | D numerica (
upwind) ~ {D_num:.1e} m2/s "
91          f"(razon D_num/D = {D_num/D:.1f})")
92
93          c = np.exp(-((X-x0)**2 + (Y-y0)**2) / (2*s0**2))
94          masa0 = c.sum() * dx**2
95          snaps, t = {}, 0.0
96          t_snap = [1.0, 3.0, 6.0]
97          for n in range(n_pasos):

```

```

98         adv = U * derivada_upwind(c, U, 1) + V *
derivada_upwind(c, V, 0)
99         c = c + dt * (-adv + D * laplaciano(c))
100         # Fronteras: pared impermeable (Neumann homogenea,
copia del vecino)
101         c[0, :], c[-1, :] = c[1, :], c[-2, :]
102         c[:, 0], c[:, -1] = c[:, 1], c[:, -2]
103         t += dt
104         for ts in t_snap:
105             if ts not in snaps and t >= ts:
106                 snaps[ts] = c.copy()
107                 masa_f = c.sum() * dx**2
108                 print(f"    conservacion de masa: {masa_f/masa0
*100:.2f} % | "
109                       f"c_max final = {c.max():.3f}")
110                 resultados[nombre] = (snaps, Pe)
111
112         #
=====
113         # BLOQUE 5: Visualizacion (2 regimenes x 3 tiempos)
114         #
=====
115         fig, axs = plt.subplots(2, 3, figsize=(12.5, 8.0))
116         for fila, (nombre, (snaps, Pe)) in enumerate(
resultados.items()):
117             for col, ts in enumerate([1.0, 3.0, 6.0]):
118                 ax = axs[fila, col]
119                 im = ax.pcolormesh(X*100, Y*100, snaps[ts], cmap='
viridis',
120                                   vmin=0, vmax=0.65, shading='auto')
121                 if fila == 0 and col == 0:
122                     sk = 12
123                     ax.quiver(X[:,sk], X[:,sk]*100, Y[:,sk], Y[:,sk]*100,
124                               U[:,sk], V[:,sk],
125                               color='w', scale=1.0, width=0.004, alpha=0.8)
126                     ax.set_title(f"{nombre} (Pe = {Pe:.0f}), t = {ts:.0f
} s",
127                                fontsize=9.5)
128                     ax.set_aspect('equal')
129                     if fila == 1: ax.set_xlabel("x [cm]")
130                     if col == 0: ax.set_ylabel("y [cm]")

```

```

131         fig.colorbar(im, ax=axis, shrink=0.8, label="c (
adimensional)")
132         fig.suptitle("Conveccion-difusion 2D en un vortice:
efecto del numero de Peclet",
133                     y=0.98)
134         plt.savefig('Figuras/fig_s5_ej4_convdif2D_vortice.
png',
135                     dpi=300, bbox_inches='tight')
136         # plt.show()
137         print("\nFigura guardada.")
138

```

Listing 34: Semana 5 – Código 4: Transporte convectivo-difusivo de un trazador en un vórtice analítico, esquema upwind 2D, con diagnóstico del número de Péclet de malla y de la difusión numérica. Ejecutar en Google Colab.

5.13.5. Código 5 — Advección 2D bajo rotación sólida (upwind vs MacCormack)

```

1         """
2         Semana 5 - Ejemplo 5: Adveccion pura 2D de un campo
de densidad bajo
3         rotacion solida -- difusion numerica vs dispersion
numerica
4
5         Aplicacion generica: transporte de un campo de
densidad (densidad
6         vehicular, concentracion de particulas, intensidad
en un flujo
7         optico) cuando el campo de velocidad se conoce en
cada punto, por
8         ejemplo estimado a partir de imagenes aereas. La
rotacion solida es
9         el banco de pruebas clasico: tras una revolucion
completa, la
10        solucion exacta coincide con la condicion inicial,
de modo que TODO
11        el error observado es error numerico.
12
13        Fisica:    d rho/dt + u d rho/dx + v d rho/dy = 0
14        Velocidad de rotacion solida alrededor del centro (
xc, yc):
15        u = -omega (y - yc) ; v = omega (x - xc)

```

```

16         Esquemas comparados:
17         (a) upwind de primer orden -> disipativo (difusion
18         numerica)
19         (b) MacCormack (equivalente 2D de Lax-Wendroff, 2do
20         orden)
21         -> dispersivo (oscilaciones cerca de gradientes
22         fuertes)
23         """
24
25         #
26         =====
27         # BLOQUE 1: Librerias
28         #
29         =====
30
31         import numpy as np
32         import matplotlib
33         matplotlib.use('Agg')
34         import matplotlib.pyplot as plt
35         import matplotlib as mpl
36
37         mpl.rcParams.update({
38             'font.size': 10, 'axes.titlesize': 11, 'axes.
39             labelsizes': 10,
40             'figure.dpi': 110, 'savefig.dpi': 300,
41             })
42
43         #
44         =====
45         # BLOQUE 2: Parametros
46         #
47         =====
48
49         L      = 1.0                # [-] dominio unitario
50         omega = 2.0 * np.pi        # [rad/s]: una
51         revolucion por segundo
52         T_rev = 1.0                # [s] tiempo de una
53         revolucion completa
54         xc = yc = 0.5
55
56         N      = 201
57         dx = L / (N - 1)
58         x      = np.linspace(0, L, N)
59         X, Y = np.meshgrid(x, x, indexing='xy')

```



```

48
49     U = -omega * (Y - yc)
50     V =  omega * (X - xc)
51
52     # Paso de tiempo por CFL 2D
53     vmax = np.abs(U).max() + np.abs(V).max()
54     dt = 0.4 * dx / vmax
55     n_pasos = int(np.ceil(T_rev / dt))
56     dt = T_rev / n_pasos          # ajusta para cerrar
exactamente la revolucion
57     print(f"dx = {dx:.4f} | dt = {dt*1e3:.3f} ms | pasos
= {n_pasos}")
58     print(f"CFL efectivo = {vmax*dt/dx:.3f}")
59
60     #
=====
61     # BLOQUE 3: Condicion inicial -- cono suave + bloque
discontinuo
62     # El cono evalua la perdida de amplitud (disipacion)
;
63     # el bloque evalua oscilaciones espurias (dispersion
).
64     #
=====
65     def condicion_inicial():
66         rho = np.zeros((N, N))
67         # cono suave centrado en (0.5, 0.75)
68         r1 = np.sqrt((X-0.5)**2 + (Y-0.75)**2)
69         rho += np.maximum(0.0, 1.0 - r1/0.12)
70         # bloque cuadrado discontinuo centrado en (0.5,
0.25)
71         rho += np.where((np.abs(X-0.5) < 0.08) & (np.abs(Y
-0.25) < 0.08),
72             1.0, 0.0)
73         return rho
74
75         rho0 = condicion_inicial()
76
77         #
=====
78     # BLOQUE 4: Esquemas numericos

```

```

79      #
=====
80      def paso_upwind(r):
81          rx_b = (r - np.roll(r, 1, 1)) / dx
82          rx_f = (np.roll(r, -1, 1) - r) / dx
83          ry_b = (r - np.roll(r, 1, 0)) / dx
84          ry_f = (np.roll(r, -1, 0) - r) / dx
85          adv = U * np.where(U > 0, rx_b, rx_f) + V * np.where
(V > 0, ry_b, ry_f)
86          return r - dt * adv
87
88      def paso_maccormack(r):
89          # Predictor: diferencias hacia adelante
90          rx_f = (np.roll(r, -1, 1) - r) / dx
91          ry_f = (np.roll(r, -1, 0) - r) / dx
92          rp = r - dt * (U * rx_f + V * ry_f)
93          # Corrector: diferencias hacia atras sobre el
predictor
94          rx_b = (rp - np.roll(rp, 1, 1)) / dx
95          ry_b = (rp - np.roll(rp, 1, 0)) / dx
96          return 0.5 * (r + rp - dt * (U * rx_b + V * ry_b))
97
98      resultados = {}
99      for nombre, paso in [("Upwind 0(1)", paso_upwind),
100 ("MacCormack 0(2)", paso_maccormack)]:
101          r = rho0.copy()
102          for n in range(n_pasos):
103              r = paso(r)
104              r[0, :] = r[-1, :] = 0.0      # frontera lejana:
densidad nula
105              r[:, 0] = r[:, -1] = 0.0
106              err_L2 = np.sqrt(np.mean((r - rho0)**2))
107              pico = r.max()
108              minimo = r.min()
109              masa = r.sum() / rho0.sum() * 100
110              print(f"\n{n{nombre} tras 1 revolucion:")
111              print(f"    error L2 = {err_L2:.4f} | pico = {pico:.3
f} (exacto 1.000)")
112              print(f"    minimo = {minimo:.3f} (oscilaciones si <
0) | masa = {masa:.1f} %")
113              resultados[nombre] = r
114

```

```

115         #
=====
116         # BLOQUE 5: Visualizacion
117         #
=====
118         fig, axs = plt.subplots(1, 4, figsize=(16, 4.0),
119         gridspec_kw={'width_ratios': [1, 1, 1, 1.15]})
120         campos = [("Condicion inicial (= exacta)", rho0),
121         ("Upwind 0(1), 1 rev.", resultados["Upwind 0(1)"]),
122         ("MacCormack 0(2), 1 rev.", resultados["MacCormack 0
(2)"])]
123         for ax, (titulo, campo) in zip(axs[:3], campos):
124             im = ax.pcolormesh(X, Y, campo, cmap='magma', vmin
=-0.15, vmax=1.05,
125             shading='auto')
126             ax.set_title(titulo, fontsize=9.5)
127             ax.set_xlabel("x"); ax.set_aspect('equal')
128             axs[0].set_ylabel("y")
129             fig.colorbar(im, ax=axs[:3], shrink=0.85, label=r"$\
rho$")
130
131             # Perfiles por la linea x = 0.5
132             jm = N // 2
133             axs[3].plot(x, rho0[:, jm], 'k-', lw=1.4, label="
exacta")
134             axs[3].plot(x, resultados["Upwind 0(1)"][:, jm], 'b-
', lw=1.4,
135             label="upwind")
136             axs[3].plot(x, resultados["MacCormack 0(2)"][:, jm],
'r-', lw=1.4,
137             label="MacCormack")
138             axs[3].axhline(0, color='gray', lw=0.7)
139             axs[3].set_xlabel("y (corte en x = 0.5)"); axs[3].
set_ylabel(r"$\rho$")
140             axs[3].set_title("Perfiles tras 1 revolucion")
141             axs[3].legend(fontsize=8); axs[3].grid(alpha=0.3)
142
143             fig.suptitle("Adveccion 2D bajo rotacion solida:
disipacion vs dispersion "
144             "numericas", y=1.03)
145             plt.savefig('Figuras/fig_s5_ej5_adveccion2D_rotacion
.png',

```

```

146         dpi=300, bbox_inches='tight')
147     # plt.show()
148     print("\nFigura guardada.")
149

```

Listing 35: Semana 5 – Código 5: Test de rotación sólida con solución exacta conocida; comparación cuantitativa de disipación (upwind) y dispersión (MacCormack). Ejecutar en Google Colab.

5.13.6. Código 6 — Reconstrucción armónica desde sensores dispersos

```

1         """
2         Semana 5 - Ejemplo 6: Reconstruccion de un campo
3         escalar 2D a partir
4         de mediciones puntuales dispersas (interpolacion
5         armonica)
6
7         Aplicacion generica: una red de sensores calibrados
8         mide una variable
9         fisica (temperatura, presion, concentracion) en
10        puntos dispersos de
11        una placa o recinto, y se requiere estimar el campo
12        completo. Si el
13        campo verdadero es suave, una estimacion natural es
14        la funcion
15        ARMONICA que interpola las mediciones: se resuelve
16        la ecuacion de
17        Laplace con condiciones de Dirichlet puntuales en
18        los sensores.
19
20        Fisica/metodo:
21        lap(T) = 0                en todo nodo sin sensor
22        T = T_medida              en los nodos con sensor (
23        Dirichlet interno)
24        dT/dn = 0                 en el contorno exterior (Neumann
25        homogenea)
26
27        La solucion minimiza la integral de |grad T|^2
28        sujeta a las
29        mediciones: es el interpolador "de minima energia".
30
31        Estudio: error RMS de reconstruccion vs numero de
32        sensores.

```

```

20         """
21
22         #
23         =====
24         # BLOQUE 1: Librerías
25         #
26         =====
27
28         import numpy as np
29         import matplotlib
30         matplotlib.use('Agg')
31         import matplotlib.pyplot as plt
32         import matplotlib as mpl
33         from scipy.sparse import lil_matrix, csr_matrix
34         from scipy.sparse.linalg import spsolve
35
36         mpl.rcParams.update({
37             'font.size': 10, 'axes.titlesize': 11, 'axes.
38             labels.size': 10,
39             'figure.dpi': 110, 'savefig.dpi': 300,
40         })
41         rng = np.random.default_rng(7)      # semilla fija:
42         reproducibilidad
43
44         #
45         =====
46         # BLOQUE 2: Campo verdadero sintético (ground truth)
47         # Suma de modos suaves; representa el campo físico
48         real que la
49         # red de sensores intenta capturar.
50         #
51         =====
52
53         L = 1.0
54         N = 121
55         dx = L / (N - 1)
56         x = np.linspace(0, L, N)
57         X, Y = np.meshgrid(x, x, indexing='xy')
58
59         T_true = (60.0
60             + 18.0 * np.sin(np.pi * X) * np.sin(np.pi * Y)
61             + 7.0 * np.cos(2*np.pi * X) * np.sin(np.pi * Y)
62             - 5.0 * X * Y)

```

```

54         sigma_med = 0.3          # [unidades de T] ruido RMS de
    cada sensor calibrado

55
56         #
    =====
57         # BLOQUE 3: Reconstruccion armonica con sensores
    como Dirichlet
58         #
    =====

59         def reconstruir(n_sens):
60             # posiciones aleatorias (evitando una franja del
    borde)
61             ii = rng.integers(5, N-5, size=n_sens)
62             jj = rng.integers(5, N-5, size=n_sens)
63             medidas = T_true[ii, jj] + rng.normal(0.0, sigma_med
    , n_sens)

64
65             es_sensor = np.zeros((N, N), dtype=bool)
66             es_sensor[ii, jj] = True
67
68             idx = -np.ones((N, N), dtype=int)
69             libres = ~es_sensor
70             nodos = np.argwhere(libres)
71             idx[libres] = np.arange(len(nodos))
72             n_inc = len(nodos)
73
74             val_sensor = np.zeros((N, N))
75             val_sensor[ii, jj] = medidas
76
77             A = lil_matrix((n_inc, n_inc))
78             b = np.zeros(n_inc)
79             for p, (i, j) in enumerate(nodos):
80                 diag = 0.0
81                 for di, dj in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
82                     ni, nj = i + di, j + dj
83                     if ni < 0 or ni >= N or nj < 0 or nj >= N:
84                         continue          # Neumann: el borde no
    aporta flujo
85                 diag += 1.0
86                 if es_sensor[ni, nj]:
87                     b[p] += val_sensor[ni, nj]
88                 else:

```

```

89         A[p, idx[ni, nj]] = -1.0
90         A[p, p] = diag
91         A = csr_matrix(A)
92
93         T_rec = np.zeros((N, N))
94         T_rec[libres] = spsolve(A, b)
95         T_rec[es_sensor] = val_sensor[es_sensor]
96         rmse = np.sqrt(np.mean((T_rec - T_true)**2))
97         return T_rec, (ii, jj), rmse
98
99         # Barrido: numero de sensores
100        n_lista = [5, 10, 20, 40, 80, 160]
101        rmse_lista, casos = [], {}
102        for ns in n_lista:
103            T_rec, pos, rmse = reconstruir(ns)
104            rmse_lista.append(rmse)
105            casos[ns] = (T_rec, pos)
106            print(f"n_sensores = {ns:4d} | RMSE = {rmse:.3f}")
107            print(f"(ruido de sensor sigma = {sigma_med}: piso
de error esperado)")
108
109            #
=====
110            # BLOQUE 4: Visualizacion
111            #
=====
112            fig, axs = plt.subplots(1, 4, figsize=(16, 4.0),
113            gridspec_kw={'width_ratios': [1, 1, 1, 1.15]})
114            vmin, vmax = T_true.min(), T_true.max()
115
116            im = axs[0].pcolormesh(X, Y, T_true, cmap='plasma',
vmin=vmin, vmax=vmax,
117            shading='auto')
118            axs[0].set_title("Campo verdadero")
119            axs[0].set_xlabel("x"); axs[0].set_ylabel("y"); axs
[0].set_aspect('equal')
120
121            for ax, ns in zip(axs[1:3], [10, 80]):
122                T_rec, (ii, jj) = casos[ns]
123                ax.pcolormesh(X, Y, T_rec, cmap='plasma', vmin=vmin,
vmax=vmax,
124                shading='auto')

```

```

125         ax.plot(x[jj], x[ii], 'wo', ms=3, mec='k', mew=0.5)
126         ax.set_title(f"Reconstruccion, {ns} sensores")
127         ax.set_xlabel("x"); ax.set_aspect('equal')
128         fig.colorbar(im, ax=axes[:3], shrink=0.85, label="T
[[C]])")
129
130         axes[3].loglog(n_lista, rmse_lista, 'ko-', lw=1.4)
131         axes[3].axhline(sigma_med, color='r', ls='--', lw=1,
132             label=r"ruido del sensor $\sigma$")
133         axes[3].set_xlabel("numero de sensores")
134         axes[3].set_ylabel("RMSE de reconstruccion")
135         axes[3].set_title("Convergencia con la densidad de
sensores")
136         axes[3].legend(fontsize=8); axes[3].grid(alpha=0.3,
which='both')
137
138         fig.suptitle("Reconstruccion armonica de un campo 2D
a partir de "
139             "mediciones dispersas", y=1.03)
140         plt.savefig('Figuras/fig_s5_ej6_laplace_sensores.png
',
141             dpi=300, bbox_inches='tight')
142         # plt.show()
143         print("Figura guardada.")
144

```

Listing 36: Semana 5 – Código 6: Reconstrucción de un campo escalar 2D a partir de mediciones puntuales, resolviendo Laplace con condiciones de Dirichlet interiores; estudio RMSE vs número de sensores. Ejecutar en Google Colab.

5.14. Ejercicios Reto — Tarea Semana 5

Instrucciones generales. Los ejercicios están clasificados por nivel. Los Ejercicios 1 y 2 son obligatorios. El Ejercicio 3 es optativo y tiene valor de punto extra. En todos los casos se requiere: (a) formulación matemática explícita de la EDP, su dominio y sus condiciones de frontera (externas e internas); (b) código ejecutable en Google Colab; (c) mínimo una figura con ejes etiquetados, unidades, escala de color común cuando se comparen paneles, y título; y (d) interpretación física de los resultados en no menos de tres observaciones cuantitativas.

5.14.1. Ejercicio 1 — Límite CFL 2D y doble rendija (Nivel intermedio)

Modifica el Código 2 para explorar la estabilidad y la física de la difracción:

- Ejecuta la simulación con números de Courant $S \in \{0.70, 0.707, 0.72\}$. Reporta el instante en que la solución se vuelve inestable para $S = 0.72$ (criterio sugerido: primer paso en que $\max |u| > 10$) y grafica la energía discreta $E(t)$ para los tres casos en escala semilogarítmica.
- Sustituye la rendija única por una *doble rendija* (dos aperturas de 0.06 m separadas 0.20 m centro a centro). Grafica el campo en $t = 2.4$ ms y el perfil de $|u|$ a lo largo de la línea “pantalla” $x = 0.85$ m.
- Identifica en el perfil de la pantalla los máximos y mínimos de interferencia y compara cualitativamente su espaciamiento con la estimación $\Delta y \approx \lambda L_p / d$, donde d es la separación entre rendijas, L_p la distancia pared–pantalla y λ la longitud de onda dominante del pulso (estímalas del ancho σ de la gaussiana inicial).

Rúbrica.

- 3 Inestabilidad localizada temporalmente con precisión, doble rendija implementada, patrón de interferencia medido en la pantalla y comparado con la estimación analítica.
- 2 Inestabilidad demostrada y doble rendija implementada, pero sin análisis cuantitativo del patrón.
- 1 Solo el inciso (a) completo.
- 0 Código no ejecuta o resultados sin coherencia física.

5.14.2. Ejercicio 2 — Condición de Robin en las cavidades: del escenario sellado al ventilado (Nivel intermedio)

El Ejemplo 3 mostró dos extremos (Neumann y Dirichlet) en las paredes de las cavidades. El caso real es intermedio: convección con coeficiente h hacia aire a $T_\infty = T_{\text{amb}}$. Modifica el Código 3 para implementar la condición de Robin en las fronteras internas:

$$-k \frac{\partial T}{\partial n} \Big|_{\text{pared de cavidad}} = h (T - T_\infty). \quad (171)$$

Implementación sugerida. En el ensamblaje disperso, para una celda activa cuya cara cruza hacia un hueco, agrega al término diagonal $\beta = h \Delta x / k$ y al vector independiente βT_∞ (linealización estándar de la resistencia convectiva de cara; verifica que con $\beta \rightarrow 0$ se recupera Neumann y con $\beta \rightarrow \infty$ se recupera Dirichlet).

- Barre $h \in \{1, 5, 10, 50, 100\} \text{ W}/(\text{m}^2 \text{ K})$ y grafica $T_{\text{máx}}$ en función de h (escala semilogarítmica en h), incluyendo como líneas horizontales los tres valores de

referencia del Ejemplo 3 (sólida, sellada, ventilada).

- (b) Encuentra numéricamente el valor crítico h^* para el cual la placa aligerada deja de ser térmicamente *peor* que la sólida ($T_{\text{máx}}^{\text{alig}}(h^*) = T_{\text{máx}}^{\text{sólida}}$). Repórtalo con dos cifras significativas.
- (c) Discute: ¿qué mecanismo físico real (convección natural en cavidad cerrada, radiación, ventilación forzada) corresponde a cada orden de magnitud de h explorado?

Rúbrica.

- 3 Robin implementada y verificada en ambos límites, curva $T_{\text{máx}}(h)$ correcta y h^* reportado con justificación física.
- 2 Robin implementada y curva correcta, sin determinación de h^* o sin verificación de límites.
- 1 Planteamiento correcto de la discretización de Robin pero implementación incompleta.
- 0 Código no ejecuta o resultado sin coherencia física.

5.14.3. Ejercicio 3 — Esquema híbrido con limitador: evadir el teorema de Godunov (Nivel avanzado, punto extra)

El Ejemplo 5 mostró la disyuntiva disipación–dispersión de los esquemas lineales. Los esquemas modernos la evaden combinando *no linealmente* un esquema de orden alto con uno monótono. Implementa sobre el Código 5 un esquema híbrido simple por mezcla local:

$$\rho^{n+1} = \theta_{i,j} \rho_{\text{MC}}^{n+1} + (1 - \theta_{i,j}) \rho_{\text{UP}}^{n+1}, \quad (172)$$

donde ρ_{MC} y ρ_{UP} son los resultados de MacCormack y upwind en el mismo paso, y el peso local $\theta_{i,j} \in [0, 1]$ detecta la suavidad de la solución, por ejemplo

$$\theta_{i,j} = \max\left(0, 1 - \gamma \frac{|\rho_{i+1,j} - 2\rho_{i,j} + \rho_{i-1,j}| + |\rho_{i,j+1} - 2\rho_{i,j} + \rho_{i,j-1}|}{\max(\rho) - \min(\rho) + \epsilon}\right), \quad (173)$$

con γ un parámetro de sensibilidad y $\epsilon \approx 10^{-12}$.

- (a) Implementa el esquema híbrido y calibra γ para cumplir simultáneamente: pico del cono ≥ 0.80 , mínimo global ≥ -0.05 y error L_2 menor que el del upwind puro.
- (b) Reporta la tabla de métricas (error L_2 , pico, mínimo, masa) para upwind, MacCormack y el híbrido, tras una y tras tres revoluciones.
- (c) Grafica el mapa de $\theta_{i,j}$ en el instante final: ¿dónde activa el sensor el esquema monótono? Relaciona la respuesta con la posición del bloque discontinuo.

Rúbrica.

- 3 Híbrido funcional que cumple las tres metas, tabla completa a una y tres revoluciones, y mapa de θ interpretado.
- 2 Híbrido funcional que cumple dos de las tres metas, con tabla a una revolución.
- 1 Estructura del híbrido correcta (Ecs. 172 y 173 implementadas) pero sin calibración exitosa.

6. Semana 6. Cuantificación de incertidumbre: GUM y métodos Monte Carlo

Objetivo de la cátedra

Formular y propagar la incertidumbre de medición de un mensurando por dos vías complementarias: la ley de propagación de incertidumbres de la GUM (JCGM 100) y la propagación de distribuciones por Monte Carlo (JCGM 101). El estudiante debe ser capaz de construir el modelo de medición, evaluar contribuciones Tipo A y Tipo B, asignar funciones de densidad por máxima entropía, generar números pseudoaleatorios de calidad y reportar un intervalo de cobertura defendible, validando la aproximación analítica contra la simulación.

Temario

- Mensurando, modelo de medición y fuentes de incertidumbre.
- Evaluación Tipo A (estadística) y Tipo B (información previa).
- Ley de propagación de incertidumbres (LPU): coeficientes de sensibilidad, correlación, incertidumbre expandida y Welch–Satterthwaite.
- Funciones de densidad de probabilidad y principio de máxima entropía.
- Generación de números pseudoaleatorios: LCG, Mersenne Twister, PCG64; transformada inversa, Box–Muller, aceptación–rechazo.
- Método Monte Carlo para la propagación de distribuciones (JCGM 101): intervalo de cobertura, procedimiento adaptativo y validación de la GUM.
- Aplicación a seis problemas de ingeniería con incertidumbre lineal, no lineal, multiplicativa, correlacionada y de soporte acotado.

Propósito del tema. En las semanas anteriores cada simulación devolvía un número: una temperatura, una deflexión, una eficiencia. Ese número, sin una afirmación cuantitativa sobre su dispersión, es científicamente incompleto. Esta semana convierte cada resultado determinista en una variable aleatoria con una distribución, y enseña a responder la pregunta que todo comité doctoral formula: *¿cuál es la incertidumbre de ese valor y cómo la calculó?* El marco es metrológico (GUM), pero la herramienta de cálculo — Monte Carlo — es la misma que articula todo el curso.

Resultado de aprendizaje esperado

Al término de la semana el estudiante podrá: (i) escribir el modelo de medición $Y = f(X_1, \dots, X_n)$ de su propio proyecto e identificar cada fuente de incertidumbre como Tipo A o Tipo B; (ii) calcular la incertidumbre combinada por LPU incluyendo

correlaciones y reportar la incertidumbre expandida con su factor de cobertura; (iii) implementar la propagación Monte Carlo en Google Colab con un generador reproducible y construir el intervalo de cobertura por percentiles; y (iv) decidir, con un criterio numérico explícito, si la aproximación GUM es válida para su problema o si la no linealidad obliga a reportar el resultado Monte Carlo.

6.1. Del campo determinista al mensurando con incertidumbre

Un *mensurando* es la magnitud que se desea determinar. Rara vez se mide de forma directa: lo habitual es medir magnitudes de entrada $X_1, \dots, X_n$ y combinarlas mediante un *modelo de medición*

$$Y = f(X_1, X_2, \dots, X_n), \quad (174)$$

donde f puede ser una fórmula cerrada, el resultado de un ajuste o incluso la salida de una simulación numérica como las de las semanas previas. Cada X_i se conoce con una incertidumbre, descrita por una función de densidad de probabilidad (PDF) $g_{X_i}(\xi_i)$. La tarea metrológica es propagar esas PDF a través de f para obtener la PDF del mensurando $g_Y(\eta)$, de la cual se extraen una estimación (su esperanza) y una incertidumbre estándar (su desviación típica).

La GUM (*Guide to the Expression of Uncertainty in Measurement*, JCGM 100:2008) formaliza este programa. Su vocabulario es preciso y conviene fijarlo:

- **Estimación** x_i de la entrada X_i : el valor esperado $\mathbb{E}[X_i]$.
- **Incertidumbre estándar** $u(x_i)$: la desviación típica $\sqrt{\text{Var}[X_i]}$.
- **Incertidumbre estándar combinada** $u_c(y)$: la desviación típica de Y .
- **Incertidumbre expandida** $U = k u_c(y)$: define un intervalo $[y - U, y + U]$ con una probabilidad de cobertura declarada (típicamente 95 %).

La distinción esencial no es entre “errores aleatorios y sistemáticos” — terminología que la GUM abandona deliberadamente — sino entre *métodos de evaluación* de la incertidumbre: Tipo A y Tipo B.

6.2. Evaluación Tipo A y Tipo B de la incertidumbre estándar

6.2.1. Tipo A: evaluación estadística de una serie de observaciones

Cuando se dispone de m observaciones repetidas $q_1, \dots, q_m$ de una magnitud, la mejor estimación es la media y la incertidumbre estándar de esa media es

$$\bar{q} = \frac{1}{m} \sum_{j=1}^m q_j, \quad u_A(\bar{q}) = \frac{s(q)}{\sqrt{m}}, \quad s^2(q) = \frac{1}{m-1} \sum_{j=1}^m (q_j - \bar{q})^2, \quad (175)$$

con $\nu = m - 1$ grados de libertad. La evaluación Tipo A es, por tanto, la desviación estándar experimental de la media, y su fiabilidad crece con m . Un caso recurrente en el curso es la incertidumbre de los parámetros de un ajuste por mínimos cuadrados, que es Tipo A y hereda los grados de libertad $\nu = m - p$ (p parámetros).

6.2.2. Tipo B: evaluación a partir de información previa

Cuando no se dispone de repeticiones experimentales, la incertidumbre no puede estimarse mediante el análisis estadístico directo de una muestra. En este caso, la información proviene de fuentes previas, tales como certificados de calibración, especificaciones del fabricante, resolución instrumental, tolerancias geométricas, límites físicos, experiencia previa o juicio técnico fundamentado. A este procedimiento se le denomina evaluación de incertidumbre de Tipo B.

El criterio general consiste en asignar una función de densidad de probabilidad, denotada como PDF por sus siglas en inglés (*Probability Density Function*), que sea coherente con la información disponible. Una vez seleccionada la PDF, la incertidumbre estándar se toma como la desviación típica de dicha distribución:

$$u_B = \sigma_X,$$

donde X representa la magnitud de entrada asociada a la fuente de incertidumbre.

La elección de la PDF no debe ser arbitraria. Debe justificarse a partir de la información disponible. En ausencia de información adicional, se emplea el principio de máxima entropía, el cual selecciona la distribución menos comprometida, es decir, aquella que introduce la menor cantidad de supuestos adicionales compatibles con los datos conocidos. Por ejemplo, si sólo se conocen límites inferior y superior, la distribución rectangular es la opción menos informativa; si se conocen media y varianza sin límites físicos finitos, la distribución normal resulta apropiada.

A continuación se presentan distribuciones frecuentes en evaluaciones de incertidumbre de Tipo B y en simulaciones por Monte Carlo. En todos los casos, r , r_1 y r_2 representan números pseudoaleatorios independientes con distribución uniforme en el intervalo $(0, 1)$.

Distribuciones de probabilidad útiles para evaluación Tipo B y simulación Monte Carlo

En esta sección se describen distribuciones de probabilidad empleadas en la evaluación de incertidumbre Tipo B y en simulaciones por Monte Carlo. Para la generación pseudoaleatoria se considera que r , r_1 y r_2 son números pseudoaleatorios independientes con distribución uniforme en el intervalo $(0, 1)$. Asimismo, Z representa una variable

aleatoria normal estándar, es decir,

$$Z \sim \mathcal{N}(0, 1).$$

En la Figuras 39, 40, 41 y Figura 42 se presentan formas representativas de las PDF descritas en esta sección. Las curvas tienen propósito conceptual: no se busca comparar magnitudes verticales entre distribuciones, sino visualizar la forma cualitativa de cada modelo probabilístico y su relación con el tipo de información disponible.

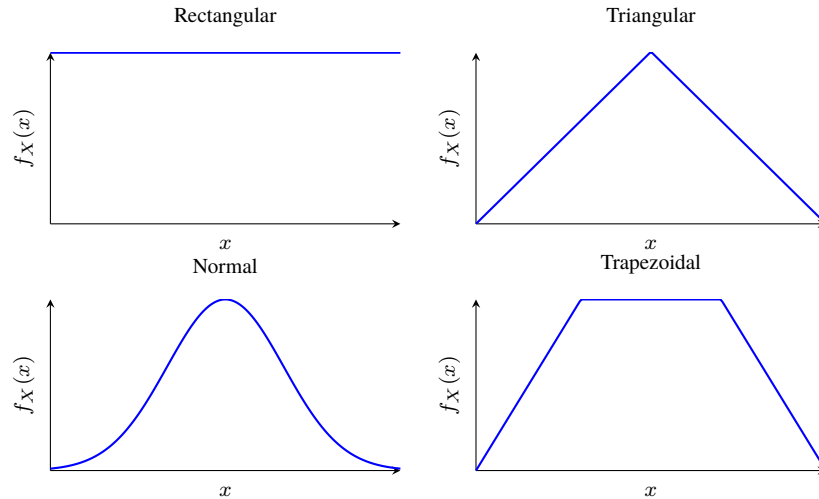


Figura 39: Formas representativas de distribuciones rectangular, triangular, normal y trapezoidal empleadas en evaluación de incertidumbre Tipo B.

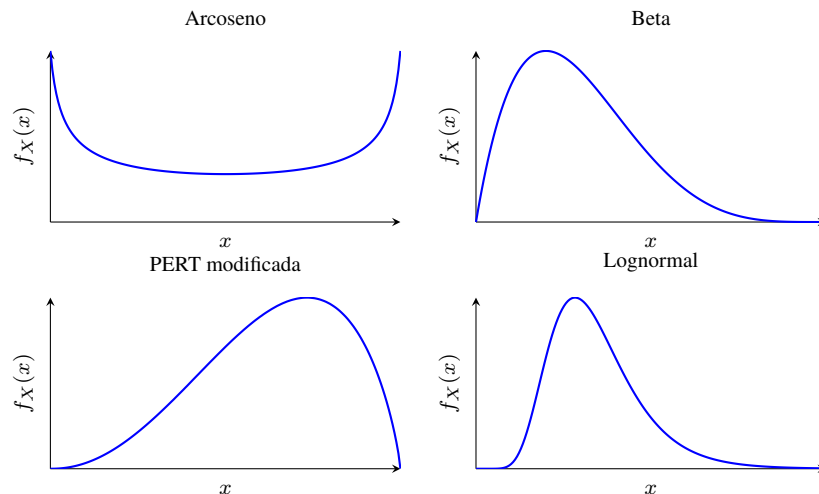


Figura 40: Formas representativas de distribuciones arcoseno, beta, PERT modificada y lognormal.

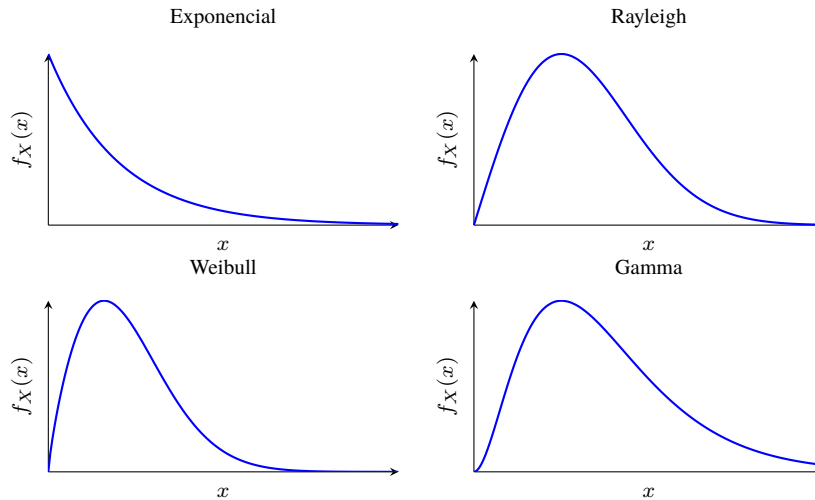


Figura 41: Formas representativas de distribuciones positivas empleadas en simulación Monte Carlo.

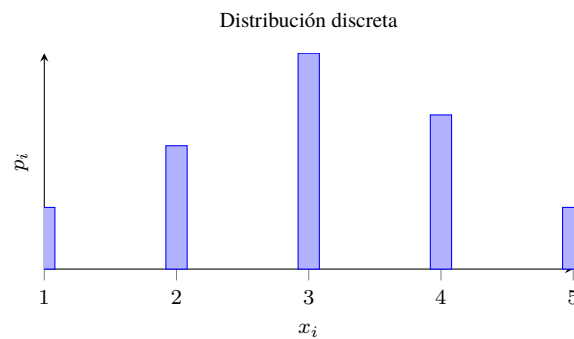


Figura 42: Forma representativa de una distribución discreta, donde la probabilidad se asigna a valores específicos x_i .

Distribución rectangular o uniforme

La distribución rectangular se emplea cuando sólo se conocen los límites simétricos $\pm a$ de una magnitud y no existe información que permita asignar mayor probabilidad a alguna región interna del intervalo. Es común en la evaluación de incertidumbre asociada a la resolución instrumental, redondeo numérico, tolerancias simples o especificaciones de fabricante expresadas únicamente mediante límites.

Los parámetros principales son el valor central x_0 y el semiancho del intervalo a . La incertidumbre estándar asociada se calcula como

$$u_B = \frac{a}{\sqrt{3}}.$$

Para simulación Monte Carlo, una realización pseudoaleatoria de la variable X puede

generarse mediante

$$X = x_0 + a(2r - 1).$$

Esta distribución representa el caso de máxima entropía cuando sólo se conocen los límites inferior y superior de la magnitud.

Distribución triangular simétrica

La distribución triangular simétrica se utiliza cuando se conocen los límites $\pm a$, pero se considera que los valores cercanos al centro son más probables que los valores extremos. Esta situación aparece cuando existe una estimación central razonable, aunque los límites extremos siguen siendo físicamente posibles.

Los parámetros son el valor central x_0 y el semiancho a . La incertidumbre estándar se obtiene como

$$u_B = \frac{a}{\sqrt{6}}.$$

La generación pseudoaleatoria puede realizarse mediante la transformación inversa de la función de distribución acumulada:

$$X = \begin{cases} x_0 + a(\sqrt{2r} - 1), & 0 < r < 1/2, \\ x_0 + a(1 - \sqrt{2(1-r)}), & 1/2 \leq r < 1. \end{cases}$$

Esta distribución es más restrictiva que la rectangular, dado que asigna mayor peso probabilístico a los valores cercanos al centro.

Distribución normal

La distribución normal se emplea cuando la información disponible proviene de un certificado de calibración, de un resultado experimental previo o de una fuente que reporta una incertidumbre expandida U asociada a un factor de cobertura k .

Los parámetros relevantes son el valor central x_0 , la incertidumbre expandida U y el factor de cobertura k . La incertidumbre estándar se calcula mediante

$$u_B = \frac{U}{k}.$$

Para la generación pseudoaleatoria se emplea

$$X = x_0 + u_B Z, \quad Z \sim \mathcal{N}(0, 1).$$

La distribución normal es apropiada cuando la magnitud resulta de la combinación de múltiples efectos pequeños e independientes, o cuando la fuente de información ya

proporciona una incertidumbre compatible con un modelo gaussiano.

Distribución trapezoidal simétrica

La distribución trapezoidal simétrica se emplea cuando se conocen límites extremos $\pm a$, pero existe una región central $\pm b$ donde la probabilidad puede considerarse aproximadamente constante. Esta distribución es intermedia entre la rectangular y la triangular.

Los parámetros son el valor central x_0 , el semiancho total a y el semiancho de la meseta central b , con

$$0 \leq b \leq a.$$

La incertidumbre estándar se calcula como

$$u_B = \sqrt{\frac{a^2 + b^2}{6}}.$$

Una forma práctica de generar valores pseudoaleatorios consiste en expresar la variable como la suma de dos variables uniformes independientes:

$$X = x_0 + Y_1 + Y_2,$$

donde

$$Y_1 \sim \mathcal{U}\left[-\frac{a-b}{2}, \frac{a-b}{2}\right], \quad Y_2 \sim \mathcal{U}\left[-\frac{a+b}{2}, \frac{a+b}{2}\right].$$

Esta distribución resulta útil cuando la información disponible indica que los valores centrales son aproximadamente equiprobables, pero los extremos no deben descartarse completamente.

Distribución arcoseno o forma U

La distribución arcoseno, también llamada distribución en forma U, se emplea cuando los valores cercanos a los extremos son más probables que los valores centrales. Aparece en errores periódicos, excentricidad, fase, vibración sinusoidal, errores angulares o desalineaciones con comportamiento armónico.

Los parámetros son el valor central x_0 y la amplitud a . La incertidumbre estándar se obtiene como

$$u_B = \frac{a}{\sqrt{2}}.$$

La generación pseudoaleatoria puede realizarse mediante

$$X = x_0 + a \sin\left[\pi \left(r - \frac{1}{2}\right)\right].$$

Esta distribución es relevante cuando el error puede modelarse como la proyección de una variable angular uniformemente distribuida.

Distribución beta

La distribución beta se emplea cuando la magnitud está restringida a un intervalo finito $[L, H]$, pero se desea controlar la forma de la distribución, su asimetría o su concentración dentro del intervalo. Es más flexible que las distribuciones rectangular, triangular o trapezoidal.

Los parámetros son el límite inferior L , el límite superior H y los parámetros de forma α y β . Si

$$B \sim \text{Beta}(\alpha, \beta),$$

entonces la variable escalada se define como

$$X = L + (H - L)B.$$

La incertidumbre estándar se calcula mediante

$$u_B = (H - L) \sqrt{\frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}}.$$

Esta distribución es útil cuando existe información previa suficiente para representar sesgo, concentración o asimetría dentro de límites físicos conocidos.

Distribución PERT modificada

La distribución PERT modificada se emplea cuando se conocen tres valores: un mínimo L , un máximo H y un valor más probable m . Es útil en estimaciones expertas, análisis de riesgo y evaluación de incertidumbre cuando existen límites físicos y una estimación preferente.

La distribución PERT puede formularse mediante una distribución beta escalada. Los parámetros de forma se definen como

$$\alpha = 1 + \lambda \frac{m - L}{H - L},$$

$$\beta = 1 + \lambda \frac{H - m}{H - L},$$

donde usualmente se toma

$$\lambda = 4.$$

La incertidumbre estándar se calcula como

$$u_B = (H - L) \sqrt{\frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}}.$$

La generación pseudoaleatoria se realiza mediante

$$X = L + (H - L)B, \quad B \sim \text{Beta}(\alpha, \beta).$$

Esta distribución permite incorporar el juicio experto de manera más estructurada que una distribución triangular simple.

Distribución lognormal

La distribución lognormal se emplea cuando la magnitud es estrictamente positiva y presenta dispersión multiplicativa. Es común en variables como rugosidad, tamaño de partícula, coeficientes positivos, factores de escala, tiempos positivos o parámetros físicos con variabilidad relativa.

Si

$$\ln X \sim \mathcal{N}(\mu, \sigma^2),$$

entonces la incertidumbre estándar de X está dada por

$$u_B = \sqrt{(e^{\sigma^2} - 1) e^{2\mu + \sigma^2}}.$$

La generación pseudoaleatoria se realiza mediante

$$X = \exp(\mu + \sigma Z), \quad Z \sim \mathcal{N}(0, 1).$$

Esta distribución no debe emplearse para variables que puedan tomar valores negativos, dado que su soporte está restringido a $X > 0$.

Distribución exponencial

La distribución exponencial se emplea para modelar el tiempo hasta un evento, una vida característica o procesos de espera con tasa constante. Su uso es común en confiabilidad, procesos estocásticos simples y fenómenos donde la probabilidad de ocurrencia por unidad de tiempo se mantiene constante.

El parámetro principal es la tasa $\lambda > 0$. La incertidumbre estándar se calcula como

$$u_B = \frac{1}{\lambda}.$$

La generación pseudoaleatoria puede realizarse mediante

$$X = -\frac{1}{\lambda} \ln(1 - r).$$

Esta distribución sólo toma valores positivos y resulta adecuada cuando el fenómeno presenta comportamiento sin memoria.

Distribución Rayleigh

La distribución Rayleigh se emplea para representar la magnitud resultante de dos componentes normales independientes de igual varianza. Aparece en errores radiales, amplitudes, desplazamientos planos, vibraciones o magnitudes resultantes en dos dimensiones.

El parámetro principal es la escala $\sigma > 0$. La incertidumbre estándar se calcula como

$$u_B = \sigma \sqrt{\frac{4 - \pi}{2}}.$$

La generación pseudoaleatoria se realiza mediante

$$X = \sigma \sqrt{-2 \ln(1 - r)}.$$

Esta distribución es adecuada cuando la variable analizada corresponde a una magnitud radial no negativa.

Distribución Weibull

La distribución Weibull se emplea en confiabilidad, fatiga, vida útil, falla de materiales y procesos donde la tasa de falla no es constante. Su flexibilidad permite representar comportamientos con tasa de falla creciente, decreciente o aproximadamente constante.

Los parámetros son la escala $\lambda > 0$ y la forma $k > 0$. La incertidumbre estándar se calcula mediante

$$u_B = \lambda \sqrt{\Gamma\left(1 + \frac{2}{k}\right) - \Gamma^2\left(1 + \frac{1}{k}\right)}.$$

La generación pseudoaleatoria se realiza mediante

$$X = \lambda [-\ln(1 - r)]^{1/k}.$$

La función $\Gamma(\cdot)$ corresponde a la función gamma de Euler.

Distribución gamma

La distribución gamma se utiliza para magnitudes positivas acumulativas, tiempos de proceso, energía, conteos continuos, suma de variables exponenciales o fenómenos donde la variable representa acumulación positiva.

Los parámetros son la forma $\alpha > 0$ y la escala $\theta > 0$. La incertidumbre estándar se calcula como

$$u_B = \sqrt{\alpha} \theta.$$

La generación pseudoaleatoria se expresa como

$$X \sim \text{Gamma}(\alpha, \theta),$$

y normalmente se realiza mediante generadores estándar disponibles en lenguajes científicos como MATLAB, Python o R.

Esta distribución es más flexible que la exponencial y permite modelar procesos positivos con diferentes grados de dispersión.

Distribución discreta

La distribución discreta se emplea cuando la magnitud sólo puede tomar valores específicos x_i con probabilidades asociadas p_i . Es útil en selección de modos, estados de operación, errores discretizados, clases de comportamiento o decisiones categóricas dentro de una simulación.

Las probabilidades deben satisfacer

$$\sum_i p_i = 1.$$

El valor esperado se calcula como

$$\bar{x} = \sum_i p_i x_i,$$

mientras que la incertidumbre estándar se obtiene mediante

$$u_B = \sqrt{\sum_i p_i (x_i - \bar{x})^2}.$$

Para generar valores pseudoaleatorios se define la función acumulada

$$F_j = \sum_{i=1}^j p_i.$$

Después, se selecciona el valor x_j tal que

$$F_{j-1} < r \leq F_j.$$

Esta formulación permite incorporar directamente información probabilística discreta dentro de una simulación Monte Carlo.

6.2.3. Criterio práctico de selección de la PDF

La selección de la función de densidad de probabilidad debe justificarse con base en la información disponible. Si sólo se conocen límites, la distribución rectangular es la opción más conservadora. Si existe un valor central más probable, puede emplearse una distribución triangular o PERT. Si la fuente proporciona incertidumbre expandida y factor de cobertura, se emplea una distribución normal. Si la variable es estrictamente positiva y presenta dispersión relativa, puede utilizarse una distribución lognormal, gamma o Weibull, según el fenómeno físico.

En consecuencia, la evaluación Tipo B no consiste únicamente en asignar una desviación estándar. El procedimiento requiere identificar la información disponible, seleccionar una PDF físicamente coherente, estimar sus parámetros y propagar la incertidumbre mediante el modelo de medición correspondiente.

En aplicaciones metrológicas, las distribuciones rectangular, triangular, normal, trapezoidal, arcoseno y beta son particularmente útiles para fuentes de incertidumbre de Tipo B. Las distribuciones lognormal, exponencial, Rayleigh, Weibull y gamma son más específicas, pero resultan convenientes cuando la magnitud de entrada está restringida a valores positivos o cuando el fenómeno físico tiene una interpretación probabilística particular.

Para simulación Monte Carlo, el procedimiento general es el siguiente:

1. Identificar cada magnitud de entrada X_i del modelo de medición.
2. Asociar a cada X_i una PDF justificada por la información disponible.
3. Generar N valores pseudoaleatorios $x_i^{(j)}$ para cada magnitud de entrada.
4. Evaluar el modelo de medición:

$$y^{(j)} = f\left(x_1^{(j)}, x_2^{(j)}, \dots, x_n^{(j)}\right), \quad j = 1, 2, \dots, N.$$

5. Estimar el valor esperado, la incertidumbre estándar y los intervalos de cobertura a partir de la muestra simulada de Y .

La ventaja del enfoque Monte Carlo es que permite propagar incertidumbres sin recurrir necesariamente a linealización local del modelo. Esto es especialmente importante cuando $f(\cdot)$ es no lineal, cuando existen límites físicos, cuando las distribuciones no son normales o cuando la combinación de incertidumbres produce una distribución de salida

asimétrica.

En consecuencia, la evaluación Tipo B no debe entenderse como una estimación débil o meramente subjetiva. Su validez depende de la correcta justificación de la PDF, de la trazabilidad de la información usada y de la consistencia física entre la fuente de incertidumbre y la distribución seleccionada.

6.3. Ley de propagación de incertidumbres

La GUM propaga la incertidumbre linealizando el modelo (174) en torno a las estimaciones $(x_1, \dots, x_n)$ mediante un desarrollo de Taylor de primer orden. Definiendo los *coeficientes de sensibilidad*

$$c_i = \left. \frac{\partial f}{\partial X_i} \right|_{x_1, \dots, x_n}, \quad (176)$$

la incertidumbre estándar combinada es, en ausencia de correlación entre las entradas,

$$u_c^2(y) = \sum_{i=1}^n c_i^2 u^2(x_i). \quad (177)$$

Cada término $c_i^2 u^2(x_i)$ es la contribución de la entrada i a la varianza del mensurando; su reparto porcentual es el *presupuesto de incertidumbre* (*uncertainty budget*), la herramienta de diagnóstico que indica dónde invertir para reducir la incertidumbre. Cuando las entradas están correlacionadas, la expresión completa incluye las covarianzas:

$$u_c^2(y) = \sum_{i=1}^n c_i^2 u^2(x_i) + 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^n c_i c_j u(x_i, x_j), \quad u(x_i, x_j) = r_{ij} u(x_i) u(x_j), \quad (178)$$

donde $r_{ij} \in [-1, 1]$ es el coeficiente de correlación. Ignorar covarianzas no es conservador: una correlación positiva entre términos del mismo signo *aumenta* u_c , y una negativa la reduce. El Ejemplo 5 de esta semana muestra un caso donde despreciar la correlación subestima la incertidumbre por un factor mayor que dos.

6.3.1. Incertidumbre expandida y factor de cobertura

Para declarar un intervalo con probabilidad de cobertura dada, se multiplica u_c por un factor de cobertura k :

$$U = k u_c(y), \quad Y \in [y - U, y + U] \text{ con probabilidad } \approx p. \quad (179)$$

Bajo normalidad asintótica $k = 2$ corresponde a $p \approx 95.45\%$ y $k = 1.96$ a $p = 95\%$ exacto. Cuando los grados de libertad son finitos, el factor se toma del cuantil de la distribución t de Student con los grados de libertad *efectivos* dados por la fórmula de Welch–

Satterthwaite:

$$\nu_{\text{eff}} = \frac{u_c^4(y)}{\sum_{i=1}^n \frac{c_i^4 u^4(x_i)}{\nu_i}}, \quad k = t_{0.975}(\nu_{\text{eff}}). \quad (180)$$

Las contribuciones Tipo B basadas en límites se toman con $\nu_i \rightarrow \infty$ (información “perfectamente” conocida en su soporte), de modo que sólo las Tipo A con pocas repeticiones inflan k por encima de 2. El Ejemplo 1 ilustra el cálculo completo de ν_{eff} y su efecto sobre el intervalo.

6.4. Asignación de funciones de densidad de probabilidad

La propagación Monte Carlo exige algo que la LPU no necesita: la PDF completa de cada entrada, no sólo su media y varianza. La asignación se rige por el principio de máxima entropía y por el conocimiento físico del proceso. El catálogo operativo del curso aparece en la Tabla 27.

Tabla 27: Asignación de PDF a las magnitudes de entrada según la información disponible. La columna u da la incertidumbre estándar en función de los parámetros.

PDF	Información que codifica	u	Uso típico
Normal $\mathcal{N}(\mu, \sigma^2)$	media y varianza, soporte infinito	σ	Tipo A, certificados
Rectangular $\mathcal{U}(\mu \pm a)$	sólo límites, sin preferencia	$a/\sqrt{3}$	resolución, tolerancias
Triangular	límites con moda central	$a/\sqrt{6}$	suma de dos rectangulares
t de Student t_ν	media con pocos datos	$\sigma \sqrt{\frac{\nu}{\nu-2}}$	Tipo A con ν pequeño
Lognormal	magnitud positiva, asimétrica	$\mu \sqrt{e^{\sigma^2} - 1}$	resistencias de materiales
Beta sobre $[0, 1]$	proporción, soporte acotado	$\sqrt{\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}}$	tasas de acierto

La asignación correcta es decisiva cuando el modelo es no lineal o el soporte es acotado. El Ejemplo 2 (modelo exponencial) muestra cómo PDF simétricas de entrada producen una PDF de salida sesgada; el Ejemplo 6 (tasa de acierto en $[0, 1]$) muestra cómo la normal de la GUM produce intervalos que invaden la región físicamente imposible > 1 , mientras la Beta respeta el soporte.

6.5. Generación de números pseudoaleatorios

Monte Carlo reduce la propagación de PDF a la generación de muestras. Un *generador de números pseudoaleatorios* (PRNG) produce una sucesión determinista que supera las pruebas estadísticas de aleatoriedad; “pseudo” significa que, fijada la semilla, la suce-

sión es reproducible — propiedad imprescindible para que una simulación doctoral sea verificable.

6.5.1. Generadores de base

El *generador lineal congruencial* (LCG), $x_{n+1} = (ax_n + c) \bmod M$, es el arquetipo histórico: rápido pero de período corto y con correlaciones seriales que lo descalifican para trabajo serio. El *Mersenne Twister* (MT19937) lo reemplazó por su período colosal $2^{19937} - 1$ y su equidistribución en 623 dimensiones, y fue durante dos décadas el estándar de facto. NumPy adopta hoy por defecto el *PCG64* (*permuted congruential generator*), que combina un LCG con una permutación de salida para lograr mejor calidad estadística, menor estado y flujos múltiples independientes. La práctica recomendada en Colab es instanciar un generador explícito con semilla:

```
rng = np.random.default_rng(semilla),
```

y *nunca* usar las funciones globales `np.random.*`, cuyo estado compartido rompe la reproducibilidad en cuanto el código crece.

6.5.2. De uniforme a cualquier distribución

Todo PRNG entrega uniformes $U \sim \mathcal{U}(0, 1)$. Para muestrear una PDF arbitraria se usan tres mecanismos:

- **Transformada inversa.** Si F es la función de distribución acumulada y $U \sim \mathcal{U}(0, 1)$, entonces $X = F^{-1}(U)$ tiene distribución F . Es exacta y eficiente cuando F^{-1} es conocida (exponencial, Weibull, triangular).
- **Box–Muller.** Para la normal, $Z_1 = \sqrt{-2 \ln U_1} \cos(2\pi U_2)$ y $Z_2 = \sqrt{-2 \ln U_1} \sin(2\pi U_2)$ generan dos normales estándar independientes a partir de dos uniformes.
- **Aceptación–rechazo.** Para PDF sin inversa cerrada: se muestrea de una propuesta $h(x)$ que acota $g(x) \leq c h(x)$ y se acepta con probabilidad $g(x)/(c h(x))$. Es general pero su eficiencia cae si la cota c es holgada.

En la práctica del curso estas primitivas están encapsuladas en `rng.normal`, `rng.uniform`, `rng.lognormal`, etc.; conocer su fundamento es lo que permite diagnosticar cuándo una muestra está mal generada.

6.6. Método Monte Carlo para la propagación de distribuciones

El Suplemento 1 de la GUM (JCGM 101:2008) define la propagación de distribuciones por Monte Carlo (MCM) como alternativa — y verificación — de la LPU. El procedimiento es directo:

1. Asignar a cada entrada X_i su PDF g_{X_i} .
2. Para cada una de M pruebas, muestrear un vector $(\xi_1^{(r)}, \dots, \xi_n^{(r)})$ y evaluar $\eta^{(r)} = f(\xi_1^{(r)}, \dots, \xi_n^{(r)})$.
3. La muestra $\{\eta^{(r)}\}_{r=1}^M$ es una representación discreta de g_Y . La estimación es su media, la incertidumbre estándar es su desviación típica, y el intervalo de cobertura del $p\%$ son los percentiles $[(1-p)/2, (1+p)/2]$.

La fortaleza del MCM es que *no linealiza*: propaga la no linealidad y la asimetría exactas de f , captura soportes acotados y no presupone normalidad de la salida. Su precisión está gobernada por el error de muestreo, que decae como

$$u(\hat{\theta}_{MC}) \propto \frac{\sigma_Y}{\sqrt{M}}, \quad (181)$$

es decir, $O(M^{-1/2})$: reducir el error a la mitad cuesta cuadruplicar M . El procedimiento adaptativo de la JCGM 101 fija M controlando que la última cifra significativa del resultado sea numéricamente estable.

6.6.1. Validación de la GUM mediante Monte Carlo

La JCGM 101 propone usar el MCM como juez de la LPU. Sea $[y_{\text{low}}, y_{\text{high}}]$ el intervalo GUM y $[y_{\text{low}}^{\text{MC}}, y_{\text{high}}^{\text{MC}}]$ el intervalo Monte Carlo, ambos al mismo nivel de cobertura. Se define una tolerancia numérica δ a partir del número de cifras significativas con que se desea reportar el resultado, y la GUM se considera *validada* si

$$|y_{\text{low}} - y_{\text{low}}^{\text{MC}}| \leq \delta \quad \text{y} \quad |y_{\text{high}} - y_{\text{high}}^{\text{MC}}| \leq \delta. \quad (182)$$

Cuando la validación falla, el modelo es suficientemente no lineal o asimétrico para que la linealización de la LPU sea inadecuada, y *debe* reportarse el resultado Monte Carlo. Los seis ejemplos siguientes recorren deliberadamente el espectro: desde un caso casi lineal donde GUM y MC coinciden (Ejemplo 1) hasta casos donde la GUM falla la validación por no linealidad (Ejemplo 2), por no normalidad en la cola (Ejemplo 3) o por soporte acotado (Ejemplo 6).

6.7. Ejemplo 1 — Calibración de un sensor: el caso canónico GUM con validación Monte Carlo

6.7.1. Planteamiento

Un sistema de calibración de sensores requiere caracterizar un transductor de presión cuya respuesta nominal es lineal. Se aplican $m = 10$ presiones de referencia conocidas (patrón con incertidumbre Tipo B $u_{\text{ref}} = 0.1155$ kPa, distribución rectangular de

semiancho 0.2 kPa) y se registra la salida en mV. El modelo de medición es la recta de calibración inversa: dada una lectura y_0 del sensor, estimar la presión

$$x_0 = \frac{y_0 - a}{b}, \quad (183)$$

donde a (intersección) y b (pendiente) provienen de un ajuste por mínimos cuadrados ordinarios. Este es el problema metrológico arquetípico: combina una incertidumbre Tipo A (los parámetros del ajuste, *correlacionados* entre sí), una Tipo A de repetibilidad de la lectura y una Tipo B del patrón. El mensurando es x_0 y su modelo (183) es no lineal en los parámetros, lo que lo hace ideal para confrontar LPU y Monte Carlo.

Tabla 28: Parámetros del ajuste de calibración y sus incertidumbres. La correlación pendiente–intersección es intrínseca al ajuste y no puede despreciarse.

Magnitud	Estimación	Incert. estándar	Tipo
Intersección a	1.9185 mV	0.1051 mV	A
Pendiente b	0.501 84 mV/kPa	0.001 78 mV/kPa	A
Correlación r_{ab}	−0.845	—	A
Desv. residual s	0.1863 mV	($\nu = 9$)	A
Repetibilidad lectura $u(y_0)$	—	0.0501 mV	A
Patrón de referencia u_{ref}	—	0.1155 kPa	B (rect.)

6.7.2. Propagación analítica (LPU)

Los coeficientes de sensibilidad de (183) son $\frac{\partial x_0}{\partial a} = -1/b$, $\frac{\partial x_0}{\partial b} = -(y_0 - a)/b^2 = -x_0/b$ y $\frac{\partial x_0}{\partial y_0} = 1/b$. La incertidumbre combinada incorpora la covarianza $u(a, b) = r_{ab} u(a) u(b)$ según (178), y se suma en cuadratura con la contribución del patrón. El cálculo da $u_c(x_0) = 0.1893$ kPa. Con grados de libertad efectivos $\nu_{\text{eff}} = 45.2$ por Welch–Satterthwaite (180), el factor de cobertura es $k = t_{0.975}(45.2) = 2.014$ (ligeramente superior al valor asintótico 1.96), de donde $U = 0.3812$ kPa y el intervalo GUM al 95 % es $[49.597, 50.360]$ kPa para una lectura que estima $x_0 = 49.979$ kPa.

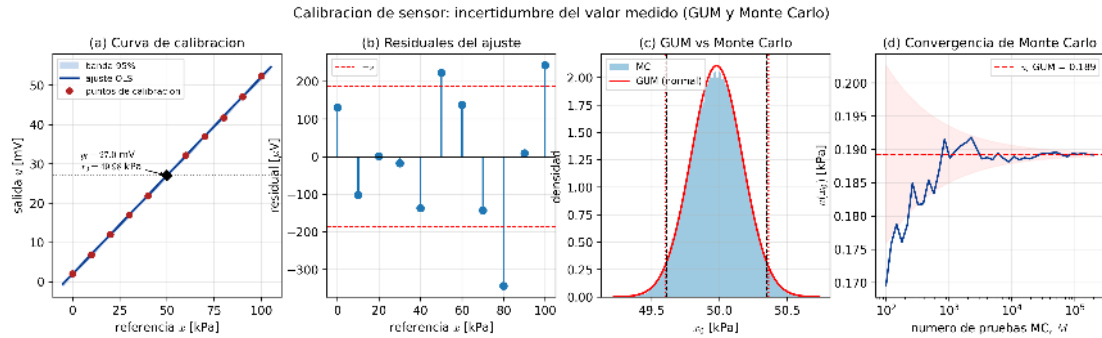


Figura 43: Calibración de sensor con análisis de incertidumbre. Panel 1: datos, recta ajustada y banda de predicción. Panel 2: histograma Monte Carlo del mensurando x_0 con el intervalo de cobertura del 95 % y la estimación GUM superpuesta. Panel 3: presupuesto de incertidumbre (reparto de la varianza por fuente). Panel 4: comparación de los intervalos GUM ($k u_c$, Ec. 179) y Monte Carlo (percentiles). Resultados del Código 1 del Anexo de esta semana.

6.7.3. Resultados y lectura metodológica

1. **GUM y Monte Carlo coinciden: el modelo es casi lineal.** La propagación Monte Carlo con $M = 2 \times 10^5$ pruebas da media 49.979 kPa, incertidumbre 0.1892 kPa e intervalo $[49.612, 50.346]$. La diferencia con el intervalo GUM es de apenas 0.014 kPa en cada extremo, atribuible al factor t de Welch–Satterthwaite que la GUM aplica y el MC no necesita. Para fines de ingeniería ambos métodos concuerdan: este es el régimen donde la LPU es plenamente válida y el reporte analítico es suficiente.
2. **El presupuesto de incertidumbre ordena las prioridades.** El reparto de la varianza es: patrón de referencia 37.2 %, regresión (parámetros a, b correlacionados) 35.0 %, repetibilidad de la lectura 27.7 % y resolución 0.1 %. La lección de gestión es inmediata: para reducir la incertidumbre de esta calibración no sirve promediar más lecturas (la repetibilidad ya es secundaria); hay que conseguir un *patrón mejor* y *más puntos de calibración* bien distribuidos.
3. **La correlación de los parámetros es ineludible.** En todo ajuste lineal a y b están correlacionados ($r_{ab} = -0.845$ aquí). Tratar $u(a)$ y $u(b)$ como independientes habría dado una incertidumbre incorrecta; el término de covarianza de (178) es parte estructural del problema, no un refinamiento opcional.

6.8. Ejemplo 2 — Elevación de temperatura por cortocircuito en un cable OPGW: no linealidad y probabilidad de falla

6.8.1. Planteamiento

Un cable OPGW (*optical ground wire*) debe soportar la corriente de cortocircuito de la línea sin que su temperatura exceda el límite del aluminio (200 °C). El modelo adiabático de la norma IEC 60949 relaciona la temperatura final θ_f con la corriente I , la duración t , la sección S y propiedades del material:

$$\theta_f = (\theta_i + \beta) \exp\left(\frac{I^2 t}{K^2 S^2}\right) - \beta, \quad (184)$$

donde θ_i es la temperatura inicial, K una constante del material y β el recíproco del coeficiente de temperatura de la resistividad. El modelo es *fuertemente no lineal* (exponencial en $I^2 t$), lo que anticipa que la LPU — que linealiza — será inadecuada. Las entradas y sus PDF aparecen en la Tabla 29; el mensurando es θ_f y la magnitud de decisión es la probabilidad de exceder 200 °C.

Tabla 29: Magnitudes de entrada del modelo IEC 60949 (Ec. 184) y sus distribuciones asignadas por máxima entropía.

Magnitud	Estimación	Incert. estándar	PDF
Corriente I	12.5 kA	4 %	Normal
Duración t	0.5 s	0.0346 s	Rectangular (± 0.06)
Sección S	90 mm ²	2 %	Rectangular
Temp. inicial θ_i	40 °C	8.66 °C	Rectangular (± 15)
Constante K	148	1.5	Normal
Parámetro β	228	3	Normal

6.8.2. Resultados y lectura metodológica

El valor nominal es $\theta_f = 188.3$ °C, cómodamente por debajo del límite. Pero la pregunta de ingeniería no es el valor nominal sino el riesgo de excederlo.

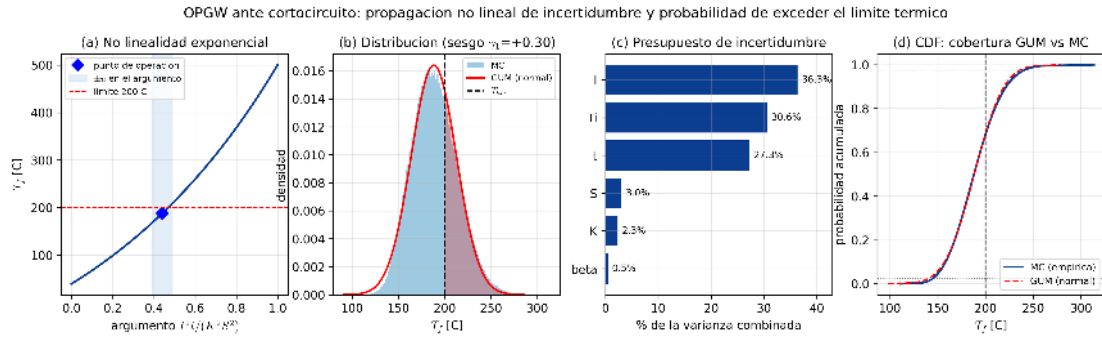


Figura 44: Cortocircuito en OPGW. Panel 1: PDF Monte Carlo de θ_f con el límite de 200 °C y la masa de probabilidad de falla sombreada. Panel 2: comparación de la PDF normal supuesta por la GUM contra el histograma MC, evidenciando la asimetría positiva. Panel 3: presupuesto de incertidumbre. Panel 4: convergencia de la probabilidad de falla con el número de pruebas M . Resultados del Código 2 del Anexo de esta semana.

1. **La GUM no se valida: la no linealidad genera asimetría.** La LPU da $u_c = 24.32$ °C y un intervalo simétrico $[139.6, 236.9]$. El Monte Carlo ($M = 5 \times 10^5$) da media 189.2 °C, incertidumbre 24.58 °C e intervalo $[145.0, 240.5]$, con *asimetría positiva* (*skewness* +0.298). El extremo inferior difiere en 5.4 °C y el superior en 3.6 °C, muy por encima de la tolerancia $\delta = 0.5$ °C: la GUM **falla la validación**. La exponencial estira la cola superior, y la media MC supera al valor nominal en +0.90 °C — un sesgo que la LPU, centrada por construcción, no puede captar.
2. **La probabilidad de falla es la cifra de diseño.** El MC estima $P(\theta_f > 200 \text{ °C}) = 31.80 \% \pm 0.09 \%$. Un valor nominal 11.7 °C por debajo del límite oculta que casi un tercio de las realizaciones lo exceden: el margen determinista es engañoso cuando la incertidumbre es grande y el modelo no lineal. Reportar sólo $\theta_f = 188$ °C sería un error de criterio profesional.
3. **El presupuesto señala la corriente.** El reparto de la varianza es I 36.3%, θ_i 30.6%, t 27.3%, S 3.0%, K 2.3% y β 0.5%. La corriente domina porque entra al cuadrado y dentro de la exponencial; su coeficiente de sensibilidad amplifica una incertidumbre relativa modesta (4 %). Reducir la incertidumbre de la estimación de la corriente de cortocircuito de la línea es la acción de mayor retorno.

6.9. Ejemplo 3 — Confiabilidad estructural de losas: índice de confiabilidad y no normalidad en la cola

6.9.1. Planteamiento

Se comparan dos sistemas de entepiso — losa maciza y losa reticular (aligerada con casetones) — bajo el estado límite de servicio por deflexión. La deflexión máxima de una

losa apoyada se modela como

$$\delta = \frac{\alpha w L^4}{EI}, \quad \delta_{\text{adm}} = \frac{L}{360}, \quad (185)$$

con $\alpha = 5/384$, claro L , carga distribuida $w = w_d + w_l$ (muerta más viva), módulo $E = 4700\sqrt{f'_c}$ y momento de inercia I . Las fuentes de incertidumbre son la resistencia del concreto f'_c (lognormal, COV 12 %, que propaga a E), la carga viva w_l (COV 25 %), el claro L (± 0.02 m) y la inercia I (COV 12 % en la maciza, 15 % en la reticular por la variabilidad geométrica del casetón). El mensurando es δ y la magnitud de decisión es la probabilidad de falla $P(\delta > \delta_{\text{adm}})$ y el índice de confiabilidad $\beta = -\Phi^{-1}(P_f)$.

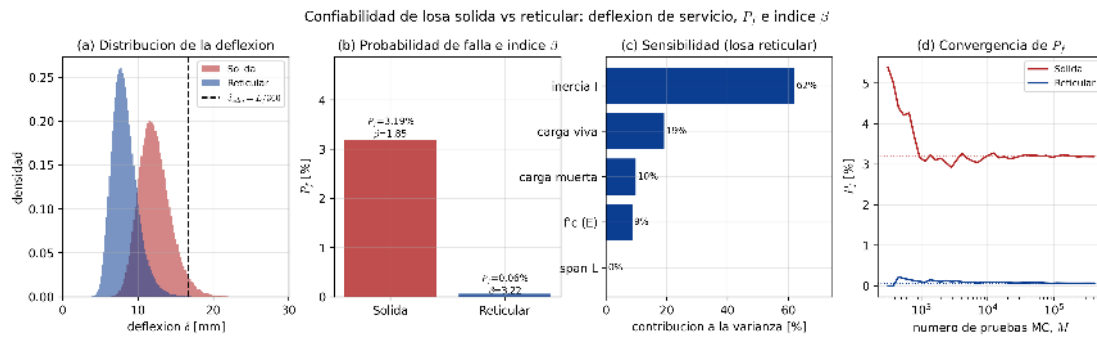


Figura 45: Confiabilidad de losas maciza vs reticular. Panel 1: PDF Monte Carlo de la deflexión de ambos sistemas con el límite admisible δ_{adm} . Panel 2: probabilidad de falla e índice de confiabilidad β . Panel 3: sensibilidad (reparto de varianza) del sistema reticular. Panel 4: comparación del β por Monte Carlo y por FORM (aproximación normal de primer orden). Resultados del Código 3 del Anexo de esta semana.

6.9.2. Resultados y lectura metodológica

1. **La losa aligerada es más ligera y más confiable.** La maciza defleja $\delta_0 = 12.02$ mm (72.1 % del admisible 16.67 mm) con $P_f = 3.186\% \pm 0.028\%$ y $\beta_{\text{MC}} = 1.854$. La reticular defleja $\delta_0 = 8.04$ mm (48.3 % del admisible) con $P_f = 0.064\% \pm 0.004\%$ y $\beta_{\text{MC}} = 3.223$. El resultado es contraintuitivo sólo en apariencia: al retirar peso propio en la zona de casetones, la carga muerta baja más de lo que baja la inercia, y el sistema gana margen. La confiabilidad no se lee del valor nominal sino de la distancia de la cola a la frontera.
2. **FORM y Monte Carlo divergen en la cola.** El índice por FORM (que asume normalidad y linealiza en el punto de diseño) da $\beta_{\text{FORM}} = 2.076$ (maciza) y 4.927 (reticular), frente a los valores MC 1.854 y 3.223. La discrepancia crece con β porque la PDF de δ es no normal — producto y cociente de variables lognormales y normales —, y FORM sobreestima la confiabilidad justo donde más importa: en eventos raros. Para P_f pequeñas, el Monte Carlo (o un FORM/SORM cuidadoso)

es obligatorio; la aproximación gaussiana es optimista.

3. **El presupuesto identifica la inercia como palanca.** En la reticular, la varianza se reparte en inercia I 62 %, carga viva 19 %, carga muerta 10 %, f'_c (vía E) 9 % y claro 0 %. La incertidumbre geométrica del casetón domina: controlar la tolerancia de fabricación de los aligeramientos reduce la incertidumbre estructural más que mejorar el concreto. El claro, pese a entrar a la cuarta potencia, contribuye de forma despreciable porque su incertidumbre relativa es mínima.

6.10. Ejemplo 4 — Optimización de una bomba centrífuga bajo incertidumbre: modelo multiplicativo y BEP

6.10.1. Planteamiento

La eficiencia hidráulica de una bomba centrífuga se obtiene del balance entre la potencia hidráulica entregada y la potencia al eje:

$$\eta = \frac{\rho g Q H}{P}, \quad (186)$$

con ρ la densidad del fluido, g la gravedad, Q el caudal, H la altura manométrica y P la potencia al eje. Este es el arquetipo de *modelo multiplicativo*: todas las variables entran como producto o cociente. La consecuencia metodológica es directa — al tomar logaritmo y derivar, la LPU (177) se reduce a una suma en cuadratura de incertidumbres relativas:

$$\left(\frac{u_c(\eta)}{\eta} \right)^2 = \left(\frac{u(Q)}{Q} \right)^2 + \left(\frac{u(H)}{H} \right)^2 + \left(\frac{u(P)}{P} \right)^2 + \left(\frac{u(\rho)}{\rho} \right)^2. \quad (187)$$

El objetivo de ingeniería no es sólo cuantificar $u(\eta)$ en un punto, sino propagar la incertidumbre a una *optimización*: localizar el punto de mejor eficiencia (BEP, *best efficiency point*) sobre la curva $\eta(Q)$ y reportar la incertidumbre tanto del caudal óptimo Q_{BEP} como de la eficiencia máxima η_{max} .

Tabla 30: Incertidumbres relativas de instrumentación de la bomba (Tipo B salvo indicación). El modelo multiplicativo (187) sólo requiere las incertidumbres relativas.

Magnitud	Incert. relativa	Distribución
Caudal Q (caudalímetro)	2.0 %	Normal
Altura H (transductores de presión)	1.5 %	Normal
Potencia al eje P (par-velocidad)	1.5 %	Normal
Densidad ρ	0.03 %	Rectangular

6.10.2. Propagación analítica y optimización

En un punto de operación representativo, (187) da una incertidumbre relativa combinada $u_c(\eta)/\eta = 2.916\%$, dominada por el caudal. Para localizar el BEP se ajusta una parábola a la curva $\eta(Q)$ medida y se propaga la incertidumbre de los puntos a los coeficientes del ajuste, y de éstos al vértice $Q_{\text{BEP}} = -c_1/(2c_2)$ y a η_{max} . La propagación se realiza por Monte Carlo porque el vértice es una función no lineal (cociente) de los coeficientes ajustados.

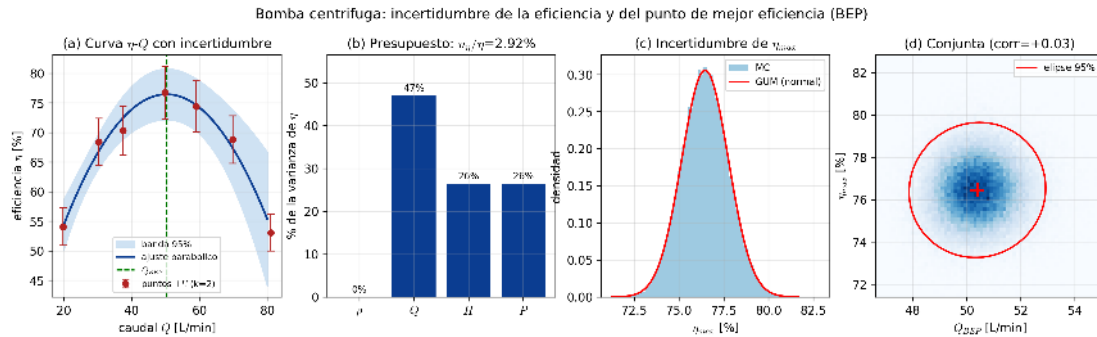


Figura 46: Optimización de bomba centrífuga bajo incertidumbre. Panel 1: curvas $\eta(Q)$ Monte Carlo con la banda de incertidumbre y el BEP. Panel 2: presupuesto de incertidumbre relativa de η (Ec. 187). Panel 3: distribución conjunta de Q_{BEP} y η_{max} propagada por Monte Carlo. Panel 4: intervalos de cobertura del caudal óptimo y de la eficiencia máxima. Resultados del Código 4 del Anexo de esta semana.

6.10.3. Resultados y lectura metodológica

1. **En un modelo multiplicativo manda la peor incertidumbre relativa.** El reparto de varianza es caudal 47.1 %, altura 26.5 %, potencia 26.5 % y densidad 0.0 %. El caudal domina no porque su incertidumbre sea grande en absoluto, sino porque entra al cuadrado en (187) con el mayor valor relativo. Mejorar el caudalímetro del 2 % al 1 % reduciría la incertidumbre combinada más que cualquier otra acción individual.
2. **La incertidumbre se propaga al óptimo, no sólo al punto.** El BEP estimado es $Q_{\text{BEP}} = 50.33 \text{ L/min}$ con $\eta_{\text{max}} = 76.46\%$. La propagación Monte Carlo da $Q_{\text{BEP}} = 50.39 \pm 1.03 \text{ L/min}$ y $\eta_{\text{max}} = 76.45 \pm 1.30\%$ (incertidumbres estándar); con factor $k = 2$, el caudal óptimo se conoce a $\pm 2.06 \text{ L/min}$ y la eficiencia máxima a ± 2.61 puntos. Reportar el BEP sin su incertidumbre es afirmar una precisión que los instrumentos no sustentan.
3. **Caudal óptimo y eficiencia máxima son casi independientes.** La correlación propagada es $\text{corr}(Q_{\text{BEP}}, \eta_{\text{max}}) = +0.029$, prácticamente nula. Esto justifica reportar ambos intervalos por separado sin una elipse de confianza conjunta, una simplificación válida *sólo* porque la simulación confirmó la baja correlación; no es

un supuesto que pueda darse por sentado a priori.

6.11. Ejemplo 5 — Velocimetría vehicular por flujo óptico desde UAV: incertidumbre correlacionada

6.11.1. Planteamiento

Un sistema de gestión de tránsito estima la velocidad de vehículos a partir del flujo óptico medido en video aéreo capturado por un UAV. La cadena fotogramétrica convierte el desplazamiento en píxeles Δp entre cuadros a velocidad física:

$$v = \frac{\Delta p \cdot p_{\text{size}} \cdot A}{f \cdot \Delta t}, \quad (188)$$

donde p_{size} es el tamaño físico del píxel del sensor, A la altura de vuelo, f la distancia focal y Δt el periodo entre cuadros. El factor $\text{GSD} = p_{\text{size}} A / f$ es la resolución espacial en el suelo (*ground sampling distance*). El punto metodológico central es la *estructura de la incertidumbre*: la altura A , la focal f y el periodo Δt son los mismos para todos los vehículos medidos en una pasada — son fuentes *sistemáticas correlacionadas* — mientras que Δp es independiente vehículo a vehículo (*aleatoria*).

Tabla 31: Parámetros de la cadena fotogramétrica del UAV. La distinción sistemática/aleatoria gobierna cómo se combina la incertidumbre sobre una flota de vehículos.

Magnitud	Valor	Incert. estándar	Carácter
Altura de vuelo A	80 m	2.5 m	Sistemático
Distancia focal f	8.8 mm	0.05 mm	Sistemático
Periodo entre cuadros Δt	33.33 ms	0.5 ms	Sistemático
Tamaño de píxel p_{size}	3.45 μm	—	Fijo
Desplazamiento Δp	9.2–31.0 px	0.4 px	Aleatorio

6.11.2. Propagación analítica (LPU)

Con los valores de la Tabla 31 la resolución es $\text{GSD} = 3.136 \text{ cm/px}$ y la incertidumbre *sistemática* relativa, común a todos los vehículos, es

$$\frac{u_{\text{sys}}}{v} = \sqrt{\left(\frac{u(A)}{A}\right)^2 + \left(\frac{u(f)}{f}\right)^2 + \left(\frac{u(\Delta t)}{\Delta t}\right)^2} = 3.513 \%. \quad (189)$$

Al promediar la velocidad de $N = 8$ vehículos, la componente *aleatoria* de Δp se reduce como $1/\sqrt{N}$, pero la componente sistemática *no se promedia*: afecta por igual a todas las mediciones y permanece. Ignorar la correlación lleva al error clásico de creer que medir más vehículos reduce indefinidamente la incertidumbre de la velocidad media.

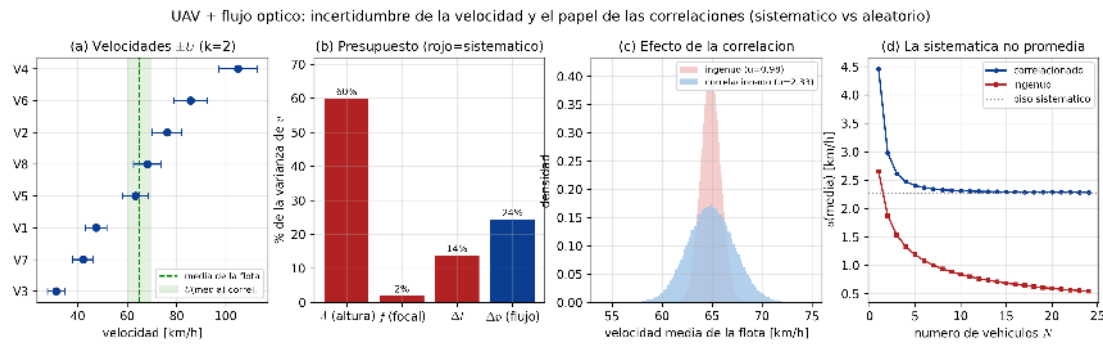


Figura 47: Velocimetría por flujo óptico desde UAV con incertidumbre correlacionada. Panel 1: velocidad estimada por vehículo con barras de incertidumbre. Panel 2: descomposición sistemática/aleatoria de la incertidumbre (Ec. 189). Panel 3: distribución Monte Carlo de la velocidad media de la flota bajo los modelos correlacionado e independiente. Panel 4: factor de subestimación al ignorar la correlación. Resultados del Código 5 del Anexo de esta semana.

6.11.3. Resultados y lectura metodológica

1. **La velocidad media de la flota es 64.87 km/h, pero su incertidumbre depende del modelo.** Tratando correctamente la correlación, $u(\bar{v}) = 2.328$ km/h. Bajo el modelo ingenuo que supone las ocho mediciones independientes, $u(\bar{v}) = 0.979$ km/h. El Monte Carlo reproduce ambos valores (2.327 y 0.979), confirmando que la diferencia es estructural y no un artefacto numérico.
2. **Ignorar la correlación subestima la incertidumbre por un factor 2.38.** El modelo ingenuo reporta menos de la mitad de la incertidumbre real. En un sistema de tránsito con consecuencias legales (infracciones por exceso de velocidad), esa subestimación es inadmisibles: el intervalo de cobertura sería demasiado estrecho y produciría falsos positivos. La componente sistemática del 3.513 % es un *piso* de incertidumbre que ningún promedio sobre vehículos puede atravesar.
3. **La calibración del UAV pesa más que el número de vehículos.** Como la incertidumbre sistemática no se promedia, la vía para reducir $u(\bar{v})$ no es medir más vehículos sino calibrar mejor la altura de vuelo, la focal y el sincronismo de cuadros. Medir Δp con sub-píxel (0.4 px) ya es secundario frente al 3.5 % sistemático.

6.12. Ejemplo 6 — Exactitud de un sistema RAG industrial: incertidumbre de soporte acotado

6.12.1. Planteamiento

Un sistema RAG (*retrieval-augmented generation*) para aplicaciones industriales se evalúa como una cascada de dos etapas: recuperación y generación. La exactitud extremo

a extremo es el producto de las exactitudes condicionales:

$$a_{e2e} = a_{\text{ret}} \cdot a_{\text{gen}}, \quad (190)$$

donde a_{ret} es la fracción de consultas con recuperación correcta y a_{gen} la fracción de respuestas correctas *dado* que la recuperación fue exitosa. Cada exactitud se estima de una muestra finita de evaluación (Tipo A binomial), y aquí aparece el reto metodológico distintivo: la magnitud está *acotada* en $[0, 1]$. Cuando la exactitud es alta y la muestra pequeña, el intervalo normal de la LPU puede sobrepasar el límite físico 1.0, produciendo un resultado sin sentido.

Tabla 32: Resultados de la evaluación del sistema RAG. La exactitud de generación se mide condicionada a recuperación exitosa.

Etapas	Aciertos/Total	Estimación	Incert. (u)
Recuperación a_{ret}	54/60	0.9000	0.0387
Generación a_{gen}	46/48	0.9583	0.0288
Extremo a extremo a_{e2e}	—	0.8625	0.0453

6.12.2. Propagación analítica y su falla en el borde

Por ser (190) multiplicativa, la LPU combina incertidumbres relativas y da $u(a_{e2e}) = 0.0453$, con intervalo GUM al 95 % $[0.7737, 0.9513]$. El síntoma de que la aproximación normal es inadecuada aparece al examinar la componente $a_{\text{gen}} = 0.9583$: su intervalo normal al 95 % tiene límite superior 1.0149, es decir, asigna probabilidad a una exactitud *mayor que el 100 %*, lo cual es físicamente imposible. La causa es la asimetría del binomial cerca del borde, que la gaussiana no captura.

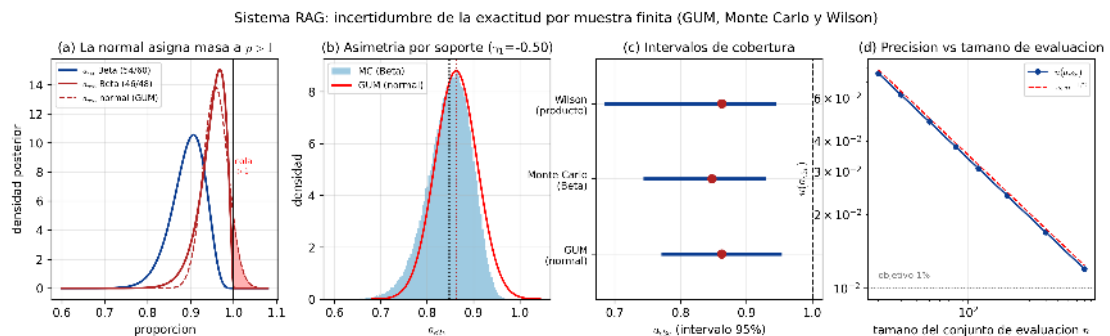


Figura 48: Incertidumbre de exactitud de un sistema RAG con soporte acotado. Panel 1: posteriores Beta de cada etapa y el producto Monte Carlo. Panel 2: comparación de intervalos GUM (normal), Monte Carlo (Beta) y Wilson. Panel 3: la cola normal de a_{gen} rebasa el límite físico 1.0. Panel 4: proyección a 1000 consultas con su intervalo de cobertura. Resultados del Código 6 del Anexo de esta semana.

6.12.3. Resultados y lectura metodológica

1. **El soporte acotado obliga a abandonar la normal.** El Monte Carlo correcto propaga distribuciones *Beta* (posterior de Jeffreys para cada binomial), garantizando que toda muestra cae en $[0, 1]$. El resultado es $a_{e2e} = 0.8478$ con $u = 0.0465$, intervalo $[0.7461, 0.9268]$ y asimetría -0.50 . La cola izquierda es más larga: la exactitud puede ser bastante peor de lo nominal con mayor probabilidad que mejor, información que el intervalo simétrico GUM oculta.
2. **El intervalo normal pierde validez física en el borde.** Que el límite superior de a_{gen} sea 1.0149 no es un detalle estético: invalida el reporte. El método Monte Carlo con *Beta*, o un intervalo de Wilson para el producto ($[0.6869, 0.9424]$), respetan la restricción $[0, 1]$ por construcción. Para exactitudes altas con muestras de evaluación pequeñas — la situación normal en validación de sistemas de IA — la GUM gaussiana *no* es defendible.
3. **La proyección operativa traduce la incertidumbre a unidades de negocio.** Sobre 1000 consultas en producción, el modelo predice 862 respuestas correctas con intervalo de cobertura Monte Carlo del 95 % $[746, 927]$. Ese rango de ± 90 consultas es el insumo cuantitativo para dimensionar revisión humana y acuerdos de nivel de servicio. La incertidumbre deja de ser un número abstracto y se vuelve una decisión de capacidad.

6.13. Síntesis de la semana

Los seis ejemplos recorren deliberadamente el espectro de comportamientos de la incertidumbre, de modo que cada estudiante reconozca a cuál pertenece su propio mensurando. La Tabla 33 resume el mapa.

Tabla 33: Mapa metodológico de la semana. Cada ejemplo ilustra un régimen distinto de propagación de incertidumbre y un criterio distinto sobre la validez de la GUM frente al Monte Carlo.

Ej.	Tipo de modelo	Reto dominante	Veredicto GUM vs MC
1	Lineal (regresión inversa)	Parámetros correlacionados	GUM válida; coincide con MC
2	Exponencial no lineal	No linealidad fuerte, asimetría	GUM <i>no</i> validada; usar MC
3	Producto/cociente	No normalidad en la cola	FORM optimista; MC obligatorio
4	Multiplicativo	Incertidumbre del óptimo (BEP)	MC para el vértice no lineal
5	Lineal con correlación	Sistemática no promediable	Modelo ingenuo subestima $\times 2.4$
6	Producto acotado $[0, 1]$	Soporte físico, muestra finita	GUM normal inválida en el borde

Las cuatro lecciones transversales que el estudiante debe extraer son:

1. **Un resultado sin incertidumbre es un resultado incompleto.** El valor nominal y la incertidumbre forman una unidad indivisible; reportar uno sin el otro es metodológicamente inaceptable a nivel doctoral.
2. **La LPU es una linealización; el Monte Carlo es el juez.** Cuando el modelo es no lineal, asimétrico, correlacionado o acotado, la GUM clásica puede fallar, y la validación numérica de (182) dictamina cuándo debe reportarse el resultado Monte Carlo.
3. **La estructura de la incertidumbre importa tanto como su magnitud.** Correlaciones (Ejemplos 1, 5), soporte acotado (Ejemplo 6) y no normalidad (Ejemplos 2, 3) cambian el resultado de forma cualitativa, no sólo cuantitativa.
4. **El presupuesto de incertidumbre es una herramienta de decisión.** Saber qué fuente domina la varianza convierte el análisis en una guía de inversión: dónde calibrar mejor, qué instrumento mejorar, qué tolerancia apretar.

Apéndice Semana 6. Referencias verificables

- A1. BIPM, IEC, IFCC, ILAC, ISO, IUPAC, IUPAP, OIML (2008). *Evaluation of measurement data — Guide to the expression of uncertainty in measurement (GUM)*. JCGM 100:2008. https://www.bipm.org/documents/20126/2071204/JCGM_100_2008_E.pdf
- A2. BIPM, IEC, IFCC, ILAC, ISO, IUPAC, IUPAP, OIML (2008). *Evaluation of measurement data — Supplement 1 to the GUM: Propagation of distributions using a Monte Carlo method*. JCGM 101:2008. https://www.bipm.org/documents/20126/2071204/JCGM_101_2008_E.pdf
- A3. Matsumoto, M. y Nishimura, T. (1998). Mersenne Twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1), 3–30. <https://doi.org/10.1145/272991.272995>
- A4. O'Neill, M. E. (2014). *PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation*. Technical Report HMC-CS-2014-0905, Harvey Mudd College. <https://www.cs.hmc.edu/tr/hmc-cs-2014-0905.pdf>
- A5. Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- A6. Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1), 1–26. <https://doi.org/10.1214/aos/1176344552>
- A7. Melchers, R. E. y Beck, A. T. (2018). *Structural Reliability Analysis and Prediction* (3.^a ed.). John Wiley & Sons. <https://doi.org/10.1002/9781119266105>

- A8. International Electrotechnical Commission (1988). *IEC 60949: Calculation of thermally permissible short-circuit currents, taking into account non-adiabatic heating effects*. IEC, Ginebra.

6.14. Anexo Semana 6. Códigos para Google Colab

Los seis códigos siguientes son autocontenidos y reproducibles: cada uno fija una semilla del generador PCG64 (`numpy.random.default_rng`), crea la carpeta Figuras/ y guarda la figura a 300 dpi. Todos los valores numéricos citados en el texto provienen de la ejecución de estos códigos. Para usarlos en Google Colab, basta copiarlos en una celda y ejecutarlos; la figura aparecerá en Figuras/ dentro del entorno.

6.14.1. Código 1 — Calibración de sensor: GUM vs Monte Carlo

Ajuste por mínimos cuadrados, predicción inversa, propagación analítica con correlación pendiente–intersección y validación Monte Carlo.

```

1      """
2      Semana 6 - Ejemplo 1: Calibracion de un sensor e
      incertidumbre del valor
3      medido (caso canonico GUM: modelo lineal, Tipo A + Tipo B,
      LPU vs Monte Carlo).
4      """
5      import numpy as np
6      import matplotlib
7      matplotlib.use('Agg')
8      import matplotlib.pyplot as plt
9      import matplotlib as mpl
10     from scipy import stats
11
12     mpl.rcParams.update({
13         'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
14         10,
15         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
16         ,
17         'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
18     })
19
20     rng = np.random.default_rng(20251117)
21     import os; os.makedirs('Figuras', exist_ok=True) # carpeta
      de salida en Colab
    
```



```

21     # ----- datos de calibracion (sinteticos, reproducibles)
    -----
22     a_true, b_true, sigma_y = 2.00, 0.500, 0.15      # mV, mV/
    kPa, mV
23     x_cal = np.linspace(0.0, 100.0, 11)             # patron de
    referencia [kPa]
24     y_cal = a_true + b_true*x_cal + rng.normal(0, sigma_y, x_cal
    .size)
25
26     n = x_cal.size
27     Xmat = np.column_stack([np.ones(n), x_cal])
28     beta_hat, *_ = np.linalg.lstsq(Xmat, y_cal, rcond=None)
29     a_hat, b_hat = beta_hat
30     resid = y_cal - Xmat @ beta_hat
31     dof = n - 2
32     s2 = resid @ resid / dof                        # varianza
    residual (Tipo A)
33     cov_beta = s2 * np.linalg.inv(Xmat.T @ Xmat)    # cov(a,b)
34     ua, ub = np.sqrt(np.diag(cov_beta))
35     rho_ab = cov_beta[0, 1] / (ua*ub)
36
37     # ----- medicion nueva: lectura y0 -> medir x0 -----
38     y0 = 27.00                                     # mV
    lectura del sensor
39     res_sensor = 0.01                              # mV
    resolucion (rectangular)
40     u_res = res_sensor / np.sqrt(12)
41     u_rep = 0.05                                    # mV
    repetibilidad (Tipo A, 1 lectura)
42     u_y0 = np.sqrt(u_res**2 + u_rep**2)
43     half_ref = 0.20                                # kPa
    patron (rectangular, certificado)
44     u_ref = half_ref / np.sqrt(3)
45
46     x0 = (y0 - a_hat) / b_hat
47
48     # ----- GUM (LPU) con coeficientes de sensibilidad
    -----
49     c_y0 = 1.0 / b_hat
50     c_a = -1.0 / b_hat
51     c_b = -(y0 - a_hat) / b_hat**2
52     c_ref = 1.0

```

```

53     # termino de regresion con covarianza (a,b correlacionados)
54     var_reg = np.array([c_a, c_b]) @ cov_beta @ np.array([c_a,
55     c_b])
56     u_c2 = c_y0**2 * u_y0**2 + var_reg + c_ref**2 * u_ref**2
57     u_c = np.sqrt(u_c2)
58
59     # Welch-Satterthwaite (Tipo A: dof; Tipo B: dof infinito)
60     comp = {
61         'regresion': (np.sqrt(var_reg), dof),
62         'lectura_y0_rep': (abs(c_y0)*u_rep, 9),          # ~10
63         'lecturas de repetibilidad'
64         'lectura_y0_res': (abs(c_y0)*u_res, np.inf),
65         'patron_ref': (u_ref, np.inf),
66     }
67     num = u_c**4
68     den = sum((ui**4 / vi) for (ui, vi) in comp.values() if np.
69     isfinite(vi))
70     nu_eff = num / den if den > 0 else np.inf
71     k = stats.t.ppf(0.975, nu_eff) if np.isfinite(nu_eff) else
72     1.96
73     U_gum = k * u_c
74
75     # ----- Monte Carlo (JCGM 101) -----
76     M = 200_000
77     ab_s = rng.multivariate_normal([a_hat, b_hat], cov_beta,
78     size=M)
79     y0_s = y0 + rng.normal(0, u_y0, M)
80     ref_s = rng.uniform(-half_ref, half_ref, M)          #
81     desplazamiento de escala del patron
82     x0_s = (y0_s - ab_s[:, 0]) / ab_s[:, 1] + ref_s
83     x0_mc = x0_s.mean()
84     u_mc = x0_s.std(ddof=1)
85     lo_mc, hi_mc = np.percentile(x0_s, [2.5, 97.5])
86
87     # ----- validacion GUM por MC (JCGM 101, tolerancia
88     numerica) -----
89     ndig = 2          # cifras
90     significativas de u_c
91     delta = 0.5 * 10**(np.floor(np.log10(u_c)) - ndig + 1)
92     lo_gum, hi_gum = x0 - U_gum, x0 + U_gum
93     dlow = abs(lo_gum - lo_mc); dhigh = abs(hi_gum - hi_mc)
94     validado = (dlow <= delta) and (dhigh <= delta)

```

```

87
88     print("=== EJEMPLO 1: CALIBRACION ===")
89     print(f"Ajuste: a={a_hat:.4f} mV (u={ua:.4f}), b={b_hat:.5f}
mV/kPa (u={ub:.5f}), rho_ab={rho_ab:+.3f}")
90     print(f"s (residual)={np.sqrt(s2):.4f} mV, dof={dof}")
91     print(f"x0 (medido) = {x0:.3f} kPa")
92     print(f"u_y0={u_y0:.4f} mV, u_ref={u_ref:.4f} kPa")
93     print(f"[GUM] u_c={u_c:.4f} kPa | nu_eff={nu_eff:.1f} | k={k
:.3f} | U={U_gum:.4f} kPa")
94     print(f"          intervalo 95%: [{lo_gum:.3f}, {hi_gum:.3f}]
kPa")
95     print(f"[MC ] media={x0_mc:.3f}, u={u_mc:.4f} kPa | 95%: [{
lo_mc:.3f}, {hi_mc:.3f}] kPa")
96     print(f"tol delta={delta:.4f} | d_low={dlow:.4f} d_high={
dhigh:.4f} -> validado={validado}")
97     budget = {kk: (vv[0], 100*vv[0]**2/u_c2) for kk, vv in comp.
items()}
98     print("Presupuesto (u_i [kPa], % var):")
99     for kk, (ui, pc) in budget.items():
100     print(f"    {kk:16s} {ui:.4f} {pc:5.1f}%")
101
102     # ----- figura -----
103     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
104
105     ax = axs[0]
106     xx = np.linspace(-5, 105, 200)
107     yhat = a_hat + b_hat*xx
108     # banda de prediccion 95%
109     Xp = np.column_stack([np.ones_like(xx), xx])
110     var_pred = s2*(1 + np.einsum('ij,jk,ik->i', Xp, np.linalg.
inv(Xmat.T@Xmat), Xp))
111     tval = stats.t.ppf(0.975, dof)
112     band = tval*np.sqrt(var_pred)
113     ax.fill_between(xx, yhat-band, yhat+band, color='#c7d9ef',
label='banda 95%')
114     ax.plot(xx, yhat, '-', color='#0b3d91', lw=1.8, label='
ajuste OLS')
115     ax.plot(x_cal, y_cal, 'o', color='#b22222', ms=5, label='
puntos de calibracion')
116     ax.axhline(y0, color='gray', ls=':', lw=1)
117     ax.plot([x0], [y0], 'D', color='k', ms=6)
118     ax.annotate(f'$y_0={y0}$ mV\n$x_0={x0:.2f}$ kPa', (x0, y0),

```

```

119     xytext=(x0-46, y0+1.5), fontsize=8,
120     arrowprops=dict(arrowstyle='->', lw=0.8))
121     ax.set_xlabel('referencia $x$ [kPa]'); ax.set_ylabel('salida
    $y$ [mV]')
122     ax.set_title('(a) Curva de calibracion'); ax.legend(fontsize
    =7.5, loc='upper left')
123
124     ax = axs[1]
125     ax.stem(x_cal, resid*1e3, basefmt=' ')
126     ax.axhline(0, color='k', lw=0.8)
127     ax.axhline(np.sqrt(s2)*1e3, color='r', ls='--', lw=0.9,
    label=r'$\pm s$')
128     ax.axhline(-np.sqrt(s2)*1e3, color='r', ls='--', lw=0.9)
129     ax.set_xlabel('referencia $x$ [kPa]'); ax.set_ylabel(r'
    residual [$\mu V$]')
130     ax.set_title('(b) Residuales del ajuste'); ax.legend(
    fontsize=8)
131
132     ax = axs[2]
133     ax.hist(x0_s, bins=120, density=True, color='#9ecae1',
    edgecolor='none', label='MC')
134     xg = np.linspace(x0-4*u_c, x0+4*u_c, 300)
135     ax.plot(xg, stats.norm.pdf(xg, x0, u_c), 'r-', lw=1.6, label
    ='GUM (normal)')
136
137     ax.axvline(lo_mc, color='k', ls='--', lw=1); ax.axvline(
    hi_mc, color='k', ls='--', lw=1)
138     ax.axvline(lo_gum, color='r', ls=':', lw=1.2); ax.axvline(
    hi_gum, color='r', ls=':', lw=1.2)
139     ax.set_xlabel('$x_0$ [kPa]'); ax.set_ylabel('densidad')
140     ax.set_title('(c) GUM vs Monte Carlo'); ax.legend(fontsize
    =8)
141
142     ax = axs[3]
143     Ms = np.unique(np.logspace(2, np.log10(M), 40).astype(int))
144     us = [x0_s[:m].std(ddof=1) for m in Ms]
145     ax.semilogx(Ms, us, '-', color='#0b3d91', lw=1.5)
146     ax.axhline(u_c, color='r', ls='--', lw=1.2, label=f'$u_c$
    GUM = {u_c:.3f}')
147     ax.fill_between(Ms, u_c*(1-1/np.sqrt(2*np.array(Ms))),
    u_c*(1+1/np.sqrt(2*np.array(Ms))), color='#fdd', alpha=0.5)
148     ax.set_xlabel('numero de pruebas MC, $M$'); ax.set_ylabel('
    $u(x_0)$ [kPa]')

```

```

150     ax.set_title('(d) Convergencia de Monte Carlo'); ax.legend(
        fontsize=8)
151
152     fig.suptitle('Calibracion de sensor: incertidumbre del valor
        medido (GUM y Monte Carlo)', y=1.03)
153     plt.savefig('Figuras/fig_s6_ej1_calibracion.png', dpi=300,
        bbox_inches='tight')
154     # plt.show()
155     print("Figura guardada.")

```

Listing 37: Semana 6 – Código 1: Calibración de sensor: GUM vs Monte Carlo. Ejecutar en Google Colab.

6.14.2. Código 2 — Cortocircuito en cable OPGW: no linealidad y probabilidad de falla

Modelo adiabático IEC 60949, propagación de una exponencial fuertemente no lineal y estimación de la probabilidad de exceder el límite térmico.

```

1      """
2      Semana 6 - Ejemplo 2: Elevacion adiabatica de temperatura de
        un conductor
3      OPGW ante cortocircuito (modelo exponencial fuertemente no
        lineal).
4      Modelo IEC 60949 adiabatico:  $I^2 t = K^2 S^2 \ln((T_f + \beta) / (T_i + \beta))$ 
        =>  $T_f = (T_i + \beta) \exp(I^2 t / (K^2 S^2)) - \beta$ 
5      """
6
7      import numpy as np
8      import matplotlib
9      matplotlib.use('Agg')
10     import matplotlib.pyplot as plt
11     import matplotlib as mpl
12     from scipy import stats
13
14     mpl.rcParams.update({
15         'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
        10,
16         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
        ,
17         'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
18     })
19     rng = np.random.default_rng(60606)

```

```

20     import os; os.makedirs('Figuras', exist_ok=True) # carpeta
    de salida en Colab
21
22     def Tf(I, t, S, Ti, K, beta):
23     return (Ti + beta)*np.exp(I**2 * t / (K**2 * S**2)) - beta
24
25     # ----- magnitudes de entrada (valor, u, distribucion)
    -----
26     # I [A], t [s], S [mm2], Ti [C], K [A s0.5 / mm2], beta [
    C] (aluminio)
27     IO, uI = 12.5e3, 0.04*12.5e3 # Normal (4% rel: CT +
    calculo de falla)
28     t0, ht = 0.50, 0.06 # rectangular +-0.06 s
29     S0, hS = 90.0, 0.02*90.0 # rectangular +-2% (
    tolerancia de fabricacion)
30     Ti0, hTi = 40.0, 15.0 # rectangular +-15 C (
    condicion de operacion)
31     K0, uK = 148.0, 1.5 # Normal (~1%
    propiedad de material)
32     beta0, ubeta = 228.0, 3.0 # Normal
33     T_lim = 200.0 # C limite de
    recocido del aluminio
34
35     y0 = Tf(IO, t0, S0, Ti0, K0, beta0)
36
37     # ----- GUM (LPU): sensibilidades analiticas (diferencias
    centradas) -----
38     def num_partial(f, args, i, h):
39     a = list(args); b = list(args)
40     a[i] += h; b[i] -= h
41     return (f(*a) - f(*b)) / (2*h)
42
43     args = [IO, t0, S0, Ti0, K0, beta0]
44     us = [uI, ht/np.sqrt(3), hS/np.sqrt(3), hTi/np.sqrt(3), uK,
    ubeta]
45     names = ['I', 't', 'S', 'Ti', 'K', 'beta']
46     steps = [1.0, 1e-4, 1e-3, 1e-3, 1e-2, 1e-2]
47     c = [num_partial(Tf, args, i, steps[i]) for i in range(6)]
48     contrib = [(c[i]*us[i])**2 for i in range(6)]
49     u_c = np.sqrt(sum(contrib))
50     U_gum = 2.0*u_c
51     lo_gum, hi_gum = y0 - U_gum, y0 + U_gum
    
```

```

52  # Pf gaussiano (aprox GUM)
53  Pf_gum = 1 - stats.norm.cdf(T_lim, loc=y0, scale=u_c)
54
55  # ----- Monte Carlo (JCGM 101) -----
56  M = 300_000
57  Is = rng.normal(I0, uI, M)
58  ts = rng.uniform(t0-ht, t0+ht, M)
59  Ss = rng.uniform(S0-hS, S0+hS, M)
60  Tis = rng.uniform(Ti0-hTi, Ti0+hTi, M)
61  Ks = rng.normal(K0, uK, M)
62  bs = rng.normal(beta0, ubeta, M)
63  Y = Tf(Is, ts, Ss, Tis, Ks, bs)
64  y_mc = Y.mean(); u_mc = Y.std(ddof=1)
65  lo_mc, hi_mc = np.percentile(Y, [2.5, 97.5])
66  g1 = stats.skew(Y)
67  Pf_mc = np.mean(Y > T_lim)
68  # error estandar de Pf
69  se_Pf = np.sqrt(Pf_mc*(1-Pf_mc)/M)
70
71  # ----- validacion GUM por MC -----
72  ndig = 2
73  delta = 0.5*10**(np.floor(np.log10(u_c)) - ndig + 1)
74  dlow = abs(lo_gum-lo_mc); dhigh = abs(hi_gum-hi_mc)
75  validado = (dlow <= delta) and (dhigh <= delta)
76
77  print("=== EJEMPLO 2: OPGW CORTOCIRCUITO ===")
78  print(f"Tf nominal = {y0:.1f} C")
79  print(f"[GUM] u_c={u_c:.2f} C | U={U_gum:.2f} | 95%: [{lo_gum:.1f}, {hi_gum:.1f}] | Pf={Pf_gum*100:.2f}%")
80  print(f"[MC ] media={y_mc:.1f} C | u={u_mc:.2f} C | 95%: [{lo_mc:.1f}, {hi_mc:.1f}]")
81  print(f"      skew={g1:+.3f} | Pf(Tf>{T_lim:.0f})={Pf_mc*100:.2f}% (+-{se_Pf*100:.2f}%)" )
82  print(f"      sesgo media MC-nominal = {y_mc-y0:+.2f} C")
83  print(f"tol delta={delta:.3f} | d_low={dlow:.2f} d_high={dhigh:.2f} -> validado={validado}")
84  print("Presupuesto GUM (u_i*c_i [C], % var):")
85  order = np.argsort(contrib)[::-1]
86  for i in order:
87  print(f"      {names[i]:5s} c={c[i]:+.4g} u={us[i]:.4g} | c*u
88  |={abs(c[i]*us[i]):7.2f} {100*contrib[i]/u_c**2:5.1f}%")

```

```

89     # ----- figura -----
90     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
91
92     ax = axs[0]
93     arg = np.linspace(0, 1.0, 200)
94     ax.plot(arg, (Ti0+beta0)*np.exp(arg)-beta0, '-', color='#0
b3d91', lw=1.8)
95     arg0 = I0**2*t0/(K0**2*S0**2)
96     ax.plot([arg0], [y0], 'D', color='b', ms=7, label='punto de
operacion')
97     # propagar +-u en el argumento para ilustrar no linealidad
98     da = np.std(Is**2*ts/(Ks**2*Ss**2))
99     ax.axvspan(arg0-da, arg0+da, color='#cfe2f3', alpha=0.6,
label=r'$\pm u$ en el argumento')
100    ax.axhline(T_lim, color='r', ls='--', lw=1, label='limite
200 C')
101    ax.set_xlabel(r'argumento  $I^2 t / (K^2 S^2)$ '); ax.set_ylabel
('$T_f$ [C]')
102    ax.set_title('(a) No linealidad exponencial'); ax.legend(
fontsize=7.5, loc='upper left')
103
104    ax = axs[1]
105    ax.hist(Y, bins=140, density=True, color='#9ecae1',
edgecolor='none', label='MC')
106    xg = np.linspace(y0-4*u_c, y0+4*u_c, 300)
107    ax.plot(xg, stats.norm.pdf(xg, y0, u_c), 'r-', lw=1.6, label
='GUM (normal)')
108    ax.axvline(T_lim, color='k', ls='--', lw=1.3, label='$T_{lim}$')
109    ax.fill_between(xg[xg>=T_lim], 0, stats.norm.pdf(xg[xg>=
T_lim], y0, u_c),
110    color='red', alpha=0.25)
111    ax.set_xlabel('$T_f$ [C]'); ax.set_ylabel('densidad')
112    ax.set_title(f'(b) Distribucion (sesgo  $\gamma_1 = \{g1 : +.2f\}$ )'); ax.legend(fontsize=7.5)
113
114    ax = axs[2]
115    pct = np.array([100*ci/u_c**2 for ci in contrib])
116    idx = np.argsort(pct)
117    ax.barh(np.array(names)[idx], pct[idx], color='#0b3d91')
118    ax.set_xlabel('% de la varianza combinada')
119    ax.set_title('(c) Presupuesto de incertidumbre')

```



```

120     for yi, p in zip(range(len(idx)), pct[idx]):
121         ax.text(p+0.8, yi, f'{p:.1f}%', va='center', fontsize=8)
122         ax.set_xlim(0, max(pct)*1.18)
123
124         ax = axs[3]
125         qs = np.linspace(0, 100, 200)
126         ax.plot(np.percentile(Y, qs), qs/100, '-', color='#0b3d91',
127                 lw=1.8, label='MC (empirica)')
128         ax.plot(stats.norm.ppf(np.clip(qs/100, 1e-4, 1-1e-4), y0, u_c),
129                 qs/100, 'r--', lw=1.4, label='GUM (normal)')
130         ax.axhline(0.025, color='gray', ls=':', lw=0.8); ax.axhline(
131             0.975, color='gray', ls=':', lw=0.8)
132         ax.axvline(T_lim, color='k', ls='--', lw=1)
133         ax.set_xlabel('$T_f$ [C]'); ax.set_ylabel('probabilidad
134             acumulada')
135         ax.set_title('(d) CDF: cobertura GUM vs MC'); ax.legend(
136             fontsize=8, loc='lower right')
137
138     fig.suptitle('OPGW ante cortocircuito: propagacion no lineal
139         de incertidumbre y probabilidad de exceder el limite termico
140         ', y=1.03)
141     plt.savefig('Figuras/fig_s6_ej2_opgw_cortocircuito.png', dpi
142                 =300, bbox_inches='tight')
143     # plt.show()
144     print("Figura guardada.")
    
```

Listing 38: Semana 6 – Código 2: Cortocircuito en cable OPGW: no linealidad y probabilidad de falla. Ejecutar en Google Colab.

6.14.3. Código 3 — Confiabilidad estructural: losa maciza vs reticular

Deflexión por servicio, índice de confiabilidad β por Monte Carlo y por FORM, y reparto de varianza.

```

1     """
2     Semana 6 - Ejemplo 3: Confiabilidad estructural de una losa
3     (solida vs
4     reticular/waffle). Estado limite de servicio por deflexion:
5     g = delta_adm - delta,    delta = alpha w L^4 / (E I),
6     delta_adm = L/360
7     Se calcula la probabilidad de falla Pf=P(g<0) y el indice de
8     confiabilidad
    
```

```

6     beta, y se propaga la incertidumbre de delta (GUM vs Monte
    Carlo).
7     """
8     import numpy as np
9     import matplotlib
10    matplotlib.use('Agg')
11    import matplotlib.pyplot as plt
12    import matplotlib as mpl
13    from scipy import stats
14
15    mpl.rcParams.update({
16        'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
17        10,
18        'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
19    },
20        'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
21    })
22    rng = np.random.default_rng(360360)
23    import os; os.makedirs('Figuras', exist_ok=True) # carpeta
    de salida en Colab
24
25    alpha = 5.0/384.0 # coeficiente de deflexion (
26    apoyo simple, franja 1 m)
27    L0, hL = 6.0, 0.02 # m, rectangular +-0.02 m
28
29    # --- propiedades por tipo de losa (franja de 1 m de ancho)
30    ---
31    # Solida: mas pesada, menor I. Reticular: mas ligera, mayor
32    I (mas peralte).
33    designs = {
34        'Solida': dict(wd=5.2, cov_wd=0.10, I=4.30e-4, cov_I
35        =0.12),
36        'Reticular': dict(wd=3.6, cov_wd=0.10, I=5.00e-4, cov_I
37        =0.15),
38    }
39    wl0, cov_wl = 2.0, 0.25 # kPa carga viva (Normal trunc
40    .), COV alto
41    fc0, cov_fc = 25.0, 0.12 # MPa, lognormal
42    M = 400_000
43
44    def E_of_fc(fc): # ACI: E = 4700 sqrt(fc) [MPa]
45        -> Pa

```

```

37     return 4700.0*np.sqrt(fc)*1e6
38
39     results = {}
40     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
41     colors = {'Solida': '#b22222', 'Reticular': '#0b3d91'}
42
43     # muestras compartidas de variables comunes
44     L_s = rng.uniform(L0-hL, L0+hL, M)
45     fc_s = np.exp(rng.normal(np.log(fc0) - 0.5*np.log(1+cov_fc
46     **2),
47     np.sqrt(np.log(1+cov_fc**2)), M))    # lognormal
48     E_s = E_of_fc(fc_s)
49     wl_s = np.clip(rng.normal(wl0, cov_wl*wl0, M), 0, None)
50
51     for name, d in designs.items():
52         wd_s = rng.normal(d['wd'], d['cov_wd']*d['wd'], M)
53         I_s = np.clip(rng.normal(d['I'], d['cov_I']*d['I'], M), 1e
54         -6, None)
55         w_s = (wd_s + wl_s)*1e3          # kPa -> N/m (
56         franja 1 m)
57         delta_s = alpha * w_s * L_s**4 / (E_s * I_s)    # m
58         delta_adm = L_s/360.0
59         g = delta_adm - delta_s
60         Pf = np.mean(g < 0)
61         se = np.sqrt(Pf*(1-Pf)/M)
62         beta = -stats.norm.ppf(Pf) if Pf > 0 else np.inf
63         beta_form = g.mean()/g.std(ddof=1)          # indice de
64         Cornell (FORM 1er orden)
65         # nominal y GUM (LPU) sobre delta
66         w0 = (d['wd']+wl0)*1e3; E0 = E_of_fc(fc0); I0 = d['I']
67         delta0 = alpha*w0*L0**4/(E0*I0)
68         # sensibilidades relativas (modelo producto/potencia)
69         rel = np.sqrt((cov_wl*wl0/(d['wd']+wl0))**2 + (d['cov_wd']*d
70         ['wd']/(d['wd']+wl0))**2
71         + (4*hL/np.sqrt(3)/L0)**2 + (0.5*cov_fc)**2 + d['cov_I']**2)
72         u_delta_gum = rel*delta0
73         results[name] = dict(delta_s=delta_s, delta0=delta0, u_gum=
74         u_delta_gum,
75         u_mc=delta_s.std(ddof=1), Pf=Pf, se=se, beta=beta,
76         beta_form=beta_form, delta_adm0=L0/360, g=g)
77     print(f"=== {name} ===")
78     print(f"    delta0={delta0*1e3:.2f} mm | delta_adm={L0/360*1e3

```

```

        :.2f} mm | margen={delta0/(L0/360)*100:.1f}% de adm")
73     print(f"   u_delta: GUM={u_delta_gum*1e3:.3f} mm | MC={
delta_s.std(ddof=1)*1e3:.3f} mm")
74     print(f"   Pf={Pf*100:.3f}% (+-{se*100:.3f}%) | beta_MC={beta
:.3f} | beta_FORM={beta_form:.3f}")

75
76     # (a) histograma de delta para ambas losas
77     ax = axs[0]
78     for name in designs:
79         r = results[name]
80         ax.hist(r['delta_s']*1e3, bins=120, density=True, alpha
=0.55,
81         color=colors[name], edgecolor='none', label=name)
82         ax.axvline(L0/360*1e3, color='k', ls='--', lw=1.4, label=r'$
\delta_{adm}=L/360$')
83         ax.set_xlabel(r'deflexion $\delta$ [mm]'); ax.set_ylabel('
densidad')
84         ax.set_title('(a) Distribucion de la deflexion'); ax.legend(
fontsize=8)
85         ax.set_xlim(0, max(results['Solida']['delta_s'].max(), L0
/360*1.4)*1e3)

86
87     # (b) Pf y beta comparados
88     ax = axs[1]
89     nm = list(designs.keys())
90     Pfs = [results[n]['Pf']*100 for n in nm]
91     betas = [results[n]['beta'] for n in nm]
92     bars = ax.bar(nm, Pfs, color=[colors[n] for n in nm], alpha
=0.8)
93     ax.set_ylabel(r'$P_f$ [%]'); ax.set_title(r'(b) Probabilidad
de falla e indice $\beta$')
94     for b, n in zip(bars, nm):
95         bval = results[n]['beta'] if np.isfinite(results[n]['beta'])
else results[n]['beta_form']
96         ax.text(b.get_x()+b.get_width()/2, b.get_height()*1.02 + 1e
-3,
97         f'$P_f$={results[n]["Pf"]*100:.2f}%\n$\beta$={bval:.2f}',
98         ha='center', fontsize=8)
99         ax.set_ylim(0, max(Pfs)*1.5 + 0.1)

100
101     # (c) contribuciones de varianza (Reticular) por re-muestreo
congelando variables
    
```

```

102     ax = axs[2]
103     base = results['Reticular']['delta_s'].var()
104     labels_v = ['carga viva', 'carga muerta', 'span L', "f'c (E)",
105               'inercia I']
106     # estimacion por correlacion-cuadrada (aprox first-order
107     indices)
108     d = designs['Reticular']
109     wd_s = rng.normal(d['wd'], d['cov_wd']*d['wd'], M)
110     I_s = np.clip(rng.normal(d['I'], d['cov_I']*d['I'], M), 1e
111     -6, None)
112     w_s = (wd_s + wl_s)*1e3
113     delta_full = alpha*w_s*L_s**4/(E_s*I_s)
114     def s1(var_samp, ref):
115         # indice de sensibilidad de primer orden (Var de E[Y/Xi])
116         aprox por binning
117         bins = np.quantile(var_samp, np.linspace(0,1,31))
118         idx = np.clip(np.digitize(var_samp, bins[1:-1]), 0, 29)
119         means = np.array([ref[idx==k].mean() if np.any(idx==k) else
120         np.nan for k in range(30)])
121         return np.nanvar(means)/ref.var()
122     S1 = [s1(wl_s, delta_full), s1(wd_s, delta_full), s1(L_s,
123     delta_full),
124     s1(fc_s, delta_full), s1(I_s, delta_full)]
125     S1 = np.array(S1); S1 = S1/ S1.sum()*100
126     order = np.argsort(S1)
127     ax.barh(np.array(labels_v)[order], S1[order], color='#0b3d91
128     ')
129     for yi, p in zip(range(len(order)), S1[order]):
130         ax.text(p+0.6, yi, f'{p:.0f}%', va='center', fontsize=8)
131         ax.set_xlabel('contribucion a la varianza [%]')
132         ax.set_title('(c) Sensibilidad (losa reticular)'); ax.
133         set_xlim(0, max(S1)*1.2)
134
135     # (d) convergencia de Pf
136     ax = axs[3]
137     Ms = np.unique(np.logspace(2.5, np.log10(M), 40).astype(int)
138     )
139     for name in designs:
140         g = results[name]['g']
141         pfs = [np.mean(g[:m] < 0) for m in Ms]
142         ax.semilogx(Ms, np.array(pfs)*100, '-', color=colors[name],
143         lw=1.5, label=name)

```

```

134     ax.axhline(results[name]['Pf']*100, color=colors[name], ls='
: ', lw=1)
135     ax.set_xlabel('numero de pruebas MC, $M$'); ax.set_ylabel(r'
$P_f$ [%]')
136     ax.set_title('(d) Convergencia de $P_f$'); ax.legend(
fontsize=8)
137
138     fig.suptitle('Confiabilidad de losa solida vs reticular:
deflexion de servicio, $P_f$ e indice $\beta$', y=1.03)
139     plt.savefig('Figuras/fig_s6_ej3_losa_confiabilidad.png', dpi
=300, bbox_inches='tight')
140     # plt.show()
141     print("Figura guardada.")

```

Listing 39: Semana 6 – Código 3: Confiabilidad estructural: losa maciza vs reticular. Ejecutar en Google Colab.

6.14.4. Código 4 — Optimización de bomba centrífuga: modelo multiplicativo y BEP

Eficiencia hidráulica multiplicativa, propagación de la incertidumbre al punto de mejor eficiencia mediante ajuste parabólico.

```

1     """
2     Semana 6 - Ejemplo 4: Eficiencia de una bomba centrifuga (
3     modelo
4     multiplicativo) e incertidumbre del punto de mejor
5     eficiencia (BEP).
6     eta = rho g Q H / P      (modelo producto/cociente ->
7     cuadratura relativa)
8     Se propaga la incertidumbre por punto (GUM) y se localiza el
9     BEP por ajuste
10    parabolico, propagando la incertidumbre de (Q_BEP, eta_max)
11    por Monte Carlo.
12    """
13
14    import numpy as np
15    import matplotlib
16    matplotlib.use('Agg')
17    import matplotlib.pyplot as plt
18    import matplotlib as mpl
19    from scipy import stats
20
21    mpl.rcParams.update({

```

```

16         'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
17         10,
18         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
19     ,
20     'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
21     })
22     rng = np.random.default_rng(404404)
23     import os; os.makedirs('Figuras', exist_ok=True) # carpeta
24     de salida en Colab
25
26     g = 9.81
27     rho0, u_rho = 997.0, 0.3 # kg/m3 (agua ~25 C
28     ), u Tipo B
29     # incertidumbres relativas de instrumentos
30     relQ, relH, relP = 0.020, 0.015, 0.015 # caudalimetro,
31     head, potencia al eje
32
33     # curva "verdadera" de eficiencia (para generar datos de
34     ensayo)
35     Q_test = np.array([20, 30, 40, 50, 60, 70, 80])*1e-3/60 #
36     m3/s (L/min->m3/s)
37     Q_bep_true, eta_max_true, curv = 50e-3/60, 0.78, 1.0e6
38     eta_true = eta_max_true - curv*(Q_test - Q_bep_true)**2
39     H_true = 30.0 - 1.5e3*(Q_test - 30e-3/60)**2 - 8e3*Q_test
40     # m, curva H-Q decreciente
41     P_true = rho0*g*Q_test*H_true/eta_true
42     # W
43
44     # datos medidos (una corrida, ruido de instrumento)
45     Qm = Q_test*(1 + rng.normal(0, relQ, Q_test.size))
46     Hm = H_true*(1 + rng.normal(0, relH, Q_test.size))
47     Pm = P_true*(1 + rng.normal(0, relP, Q_test.size))
48     eta_m = rho0*g*Qm*Hm/Pm
49
50     # GUM por punto: modelo multiplicativo -> (u_eta/eta)^2 =
51     sum (rel_i)^2
52     rel_eta = np.sqrt((u_rho/rho0)**2 + relQ**2 + relH**2 + relP
53     **2)
54     u_eta_pt = rel_eta*eta_m
55     print("=== EJEMPLO 4: BOMBA CENTRIFUGA ===")
56     print(f"u_eta/eta (GUM, multiplicativo) = {rel_eta*100:.3f}%")
57     ")

```

```

46     print("Contribuciones relativas a la varianza:")
47     for nm, r in [('rho', u_rho/rho0), ('Q', relQ), ('H', relH),
48                  ('P', relP)]:
49         print(f"    {nm:4s} rel={r*100:.3f}%    ({100*r**2/rel_eta
50             **2:.1f}% de la var.)")
51
52     # --- BEP por ajuste parabolico de los puntos medidos ---
53     def fit_bep(Q, eta):
54         A = np.column_stack([Q**2, Q, np.ones_like(Q)])
55         a, b, c = np.linalg.lstsq(A, eta, rcond=None)[0]
56         Qb = -b/(2*a)
57         eb = a*Qb**2 + b*Qb + c
58         return Qb, eb
59
60     Qb0, eb0 = fit_bep(Qm, eta_m)
61     print(f"BEP medido: Q_BEP={Qb0*6e4:.2f} L/min, eta_max={eb0
62         *100:.2f}%")
63
64     # --- Monte Carlo: cada punto medido fluctua segun su
65     incertidumbre ---
66     M = 100_000
67     Qbep_s = np.empty(M); etam_s = np.empty(M)
68     for k in range(M):
69         rho_k = rng.normal(rho0, u_rho)
70         Qk = Qm*(1 + rng.normal(0, relQ, Qm.size))
71         Hk = Hm*(1 + rng.normal(0, relH, Hm.size))
72         Pk = Pm*(1 + rng.normal(0, relP, Pm.size))
73         eta_k = rho_k*g*Qk*Hk/Pk
74         Qbep_s[k], etam_s[k] = fit_bep(Qk, eta_k)
75     Qbep_mc = Qbep_s.mean()*6e4; uQbep = Qbep_s.std(ddof=1)*6e4
76     etam_mc = etam_s.mean()*100; uetam = etam_s.std(ddof=1)*100
77     print(f"[MC] Q_BEP = {Qbep_mc:.2f} +- {uQbep:.2f} L/min    (k
78         =2: +-{2*uQbep:.2f})")
79     print(f"[MC] eta_max = {etam_mc:.2f} +- {uetam:.2f} %    (k
80         =2: +-{2*uetam:.2f})")
81     rho_corr = np.corrcoef(Qbep_s, etam_s)[0,1]
82     print(f"correlacion(Q_BEP, eta_max) = {rho_corr:+.3f}")
83
84     # ----- figura -----
85     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
86
87     ax = axs[0]
88     Qg = np.linspace(Q_test.min(), Q_test.max(), 200)

```



```

82     A = np.column_stack([Qm**2, Qm, np.ones_like(Qm)])
83     coef = np.linalg.lstsq(A, eta_m, rcond=None)[0]
84     etafit = coef[0]*Qg**2 + coef[1]*Qg + coef[2]
85     # banda de incertidumbre de la curva por MC
86     band = np.std([ (lambda cc: cc[0]*Qg**2+cc[1]*Qg+cc[2])(
87         np.linalg.lstsq(np.column_stack([ (Qm*(1+rng.normal(0,relQ,
88         Qm.size)))*2,
89         Qm*(1+rng.normal(0,relQ,Qm.size)), np.ones_like(Qm)]),
90         eta_m*(1+rng.normal(0,rel_eta,Qm.size)), rcond=None)[0])
91         for _ in range(400)], axis=0)
92     ax.fill_between(Qg*6e4, (etafit-2*band)*100, (etafit+2*band)
93     *100,
94     color='#cfe2f3', label='banda 95%')
95     ax.plot(Qg*6e4, etafit*100, '-', color='#0b3d91', lw=1.8,
96     label='ajuste parabolico')
97     ax.errorbar(Qm*6e4, eta_m*100, yerr=2*u_eta_pt*100, fmt='o',
98     color='#b22222',
99     ms=5, capsize=3, lw=1, label='puntos $\pm U$ (k=2)')
100     ax.axvline(Qbep_mc, color='g', ls='--', lw=1.2, label='$Q_{BEP}$')
101     ax.set_xlabel('caudal $Q$ [L/min]'); ax.set_ylabel(r'
102     eficiencia $\eta$ [%]')
103     ax.set_title('(a) Curva $\eta$-$Q$ con incertidumbre'); ax.
104     legend(fontsize=7.5, loc='lower center')
105
106     ax = axs[1]
107     labels = [r'$\rho$', '$Q$', '$H$', '$P$']
108     rels = np.array([u_rho/rho0, relQ, relH, relP])
109     pc = 100*rels**2/rel_eta**2
110     ax.bar(labels, pc, color='#0b3d91')
111     for i, p in enumerate(pc):
112         ax.text(i, p+1, f'{p:.0f}%', ha='center', fontsize=8)
113     ax.set_ylabel('% de la varianza de $\eta$')
114     ax.set_title(f'(b) Presupuesto: $u_{\eta}/\eta$={rel_eta
115     *100:.2f}%'); ax.set_ylim(0, max(pc)*1.2)
116
117     ax = axs[2]
118     ax.hist(etam_s*100, bins=100, density=True, color='#9ecae1',
119     edgecolor='none', label='MC')
120     xg = np.linspace(etam_mc-4*uetam, etam_mc+4*uetam, 200)
121     ax.plot(xg, stats.norm.pdf(xg, etam_mc, uetam), 'r-', lw
122     =1.6, label='GUM (normal)')

```

```

114     ax.set_xlabel(r'$\eta_{\max}$ [%]'); ax.set_ylabel('densidad'
115 )
116     ax.set_title('(c) Incertidumbre de $\eta_{\max}$'); ax.
117     legend(fontsize=8)
118
119     ax = axs[3]
120     hb = ax.hist2d(Qbep_s*6e4, etam_s*100, bins=70, cmap='Blues'
121 , density=True)
122     ax.plot(Qbep_mc, etam_mc, 'r+', ms=12, mew=2)
123     # elipse de covarianza 95%
124     cov = np.cov(Qbep_s*6e4, etam_s*100)
125     vals, vecs = np.linalg.eigh(cov)
126     th = np.linspace(0, 2*np.pi, 100)
127     ell = (vecs @ (np.sqrt(vals*stats.chi2.ppf(0.95,2))[:,None]*
128 np.array([np.cos(th),np.sin(th)])))
129     ax.plot(Qbep_mc+ell[0], etam_mc+ell[1], 'r-', lw=1.5, label=
130 'elipse 95%')
131     ax.set_xlabel('$Q_{BEP}$ [L/min]'); ax.set_ylabel(r'$\eta_{\max}$ [%]')
132     ax.set_title(f'(d) Conjunta (corr={rho_corr:+.2f})'); ax.
133     legend(fontsize=8)
134
135     fig.suptitle('Bomba centrífuga: incertidumbre de la
136 eficiencia y del punto de mejor eficiencia (BEP)', y=1.03)
137     plt.savefig('Figuras/fig_s6_ej4_bomba_bep.png', dpi=300,
138 bbox_inches='tight')
139     # plt.show()
140     print("Figura guardada.")

```

Listing 40: Semana 6 – Código 4: Optimización de bomba centrífuga: modelo multiplicativo y BEP. Ejecutar en Google Colab.

6.14.5. Código 5 — Velocimetría por flujo óptico desde UAV: incertidumbre correlacionada

Cadena fotogramétrica, descomposición sistemática/aleatoria y demostración de que la sistemática no se promedia sobre la flota.

```

1     """
2     Semana 6 - Ejemplo 5: Velocimetria vehicular desde UAV por
3     flujo optico.
4     Cadena fotogrametrica:
5     GSD = p_size * A / f                (m/px)

```

```

5     v     = dp * GSD / dt = dp * p_size * A / (f * dt)
6     Punto clave: A, f, p_size son COMUNES a todos los vehiculos
    de un cuadro
7     (sistematicos -> correlacion total); dp y dt son por-
    vehiculo (aleatorios).
8     La incertidumbre de la velocidad MEDIA de la flota NO
    promedia el termino
9     sistematico. Se compara el tratamiento correlacionado vs el
    ingenuo.
10    """
11    import numpy as np
12    import matplotlib
13    matplotlib.use('Agg')
14    import matplotlib.pyplot as plt
15    import matplotlib as mpl
16    from scipy import stats
17
18    mpl.rcParams.update({
19        'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
    10,
20        'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
    ,
21        'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
22    })
23    rng = np.random.default_rng(505505)
24    import os; os.makedirs('Figuras', exist_ok=True) # carpeta
    de salida en Colab
25
26    # --- parametros del sensor / vuelo (comunes al cuadro =
    sistematicos) ---
27    p_size0, u_psize = 3.45e-6, 0.0                # m/px (dato de
    hoja, u~0)
28    f0, u_f = 8.8e-3, 0.05e-3                    # m, calibracion
    de camara
29    A0, u_A = 80.0, 2.5                          # m, altura AGL (
    GPS/baro) -> sistematico
30    dt0, u_dt = 1/30.0, 0.5e-3                  # s, intervalo
    entre cuadros (timestamp)
31
32    GSD0 = p_size0*A0/f0
33    print("=== EJEMPLO 5: UAV FLUJO OPTICO ===")
34    print(f"GSD nominal = {GSD0*100:.3f} cm/px")

```

```

35
36     # --- flota: N vehiculos, desplazamiento dp por vehiculo (
Tipo A, flujo) ---
37     N = 8
38     dp_true = np.array([14.0, 22.5, 9.2, 31.0, 18.7, 25.3, 12.4,
20.1]) # px/cuadro
39     u_dp = 0.4 # px,
incertidumbre del flujo optico
40     v_nom = dp_true*GSD0/dt0
41     print("v nominal por vehiculo [km/h]:", np.round(v_nom*3.6,
2))
42
43     # --- GUM por vehiculo (modelo producto): rel^2 = sum rel_i
^2 ---
44     rel_sys2 = (u_psize/p_size0)**2 + (u_f/f0)**2 + (u_A/A0)**2
+ (u_dt/dt0)**2
45     u_v_pt = v_nom*np.sqrt(rel_sys2 + (u_dp/dp_true)**2)
46     print(f"componente sistematica (rel) = {np.sqrt(rel_sys2)
*100:.3f}%")
47     print("U(v) por vehiculo [km/h] (k=2):", np.round(2*u_v_pt
*3.6, 2))
48
49     # --- velocidad media de la flota: correlaciones IMPORTANTAN
---
50     vbar = v_nom.mean()
51     # (1) tratamiento correlacionado (correcto): el termino
sistematico es comun
52     u_vbar_corr2 = (rel_sys2)*vbar**2 + (1/N**2)*np.sum((v_nom*
u_dp/dp_true)**2)
53     u_vbar_corr = np.sqrt(u_vbar_corr2)
54     # (2) tratamiento ingenuo (todo aleatorio e independiente):
promedia todo
55     u_vbar_naive = np.sqrt(np.sum(u_v_pt**2))/N
56     print(f"v_media = {vbar*3.6:.2f} km/h")
57     print(f"u(v_media) correlacionado = {u_vbar_corr*3.6:.3f} km
/h")
58     print(f"u(v_media) ingenuo = {u_vbar_naive*3.6:.3f}
km/h")
59     print(f"subestimacion del ingenuo = factor {u_vbar_corr/
u_vbar_naive:.2f}")
60
61     # --- Monte Carlo confirmatorio ---

```

```

62     M = 200_000
63     # sistematicos: UN sorteo por prueba, COMPARTIDO por la
        flota
64     A_s = rng.normal(A0, u_A, M)
65     f_s = rng.normal(f0, u_f, M)
66     dt_s = rng.normal(dt0, u_dt, M)
67     GSD_s = p_size0*A_s/f_s
68     vbar_corr_mc = np.empty(M)
69     vbar_naive_mc = np.empty(M)
70     for j in range(N):
71         dp_s = dp_true[j] + rng.normal(0, u_dp, M)      # aleatorio
        por vehiculo
72         vbar_corr_mc += dp_s*GSD_s/dt_s
73         # ingenuo: cada vehiculo sortea TODO de forma independiente
74         A_i = rng.normal(A0, u_A, M); f_i = rng.normal(f0, u_f, M);
        dt_i = rng.normal(dt0, u_dt, M)
75         vbar_naive_mc += (dp_true[j]+rng.normal(0,u_dp,M))*(p_size0*
        A_i/f_i)/dt_i
76         vbar_corr_mc /= N; vbar_naive_mc /= N
77         print(f"[MC] u(v_media) correlacionado = {vbar_corr_mc.std(
        ddof=1)*3.6:.3f} km/h")
78         print(f"[MC] u(v_media) ingenuo          = {vbar_naive_mc.std(
        ddof=1)*3.6:.3f} km/h")
79
80     # --- figura ---
81     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
82
83     ax = axs[0]
84     order = np.argsort(v_nom)
85     yp = np.arange(N)
86     ax.errorbar(v_nom[order]*3.6, yp, xerr=2*u_v_pt[order]*3.6,
        fmt='o',
87     color='#0b3d91', ms=6, capsize=3, lw=1.2)
88     ax.axvline(vbar*3.6, color='g', ls='--', lw=1.2, label='
        media de la flota')
89     ax.axvspan((vbar-2*u_vbar_corr)*3.6, (vbar+2*u_vbar_corr)
        *3.6, color='#d9ead3',
90     alpha=0.7, label='$U$(media) correl.')
91     ax.set_yticks(yp); ax.set_yticklabels([f'V{ i+1}' for i in
        order])
92     ax.set_xlabel('velocidad [km/h]'); ax.set_title('(a)
        Velocidades $\backslash$pm U$ (k=2)')

```

```

93     ax.legend(fontsize=7.5, loc='lower right')
94
95     ax = axs[1]
96     labels = ['$A$ (altura)', '$f$ (focal)', '$\\Delta t$', '$\\Delta p$ (flujo)']
97     # para un vehiculo tipico (mediano)
98     jt = np.argsort(dp_true)[N//2]
99     comps = np.array([(u_A/A0)**2, (u_f/f0)**2, (u_dt/dt0)**2, (u_dp/dp_true[jt])**2])
100     pc = 100*comps/comps.sum()
101     ax.bar(labels, pc, color=['#b22222', '#b22222', '#b22222', '#0b3d91'])
102     for i, p in enumerate(pc):
103         ax.text(i, p+1, f'{p:.0f}%', ha='center', fontsize=8)
104         ax.set_ylabel('% de la varianza de $v$')
105         ax.set_title('(b) Presupuesto (rojo=sistemico)'); ax.set_ylim(0, max(pc)*1.2)
106
107     ax = axs[2]
108     ax.hist(vbar_naive_mc*3.6, bins=120, density=True, color='#f4cccc', edgecolor='none',
109            label=f'ingenuo (u={vbar_naive_mc.std(ddof=1)*3.6:.2f})')
110     ax.hist(vbar_corr_mc*3.6, bins=120, density=True, color='#9fc5e8', edgecolor='none',
111            alpha=0.8, label=f'correlacionado (u={vbar_corr_mc.std(ddof=1)*3.6:.2f})')
112     ax.set_xlabel('velocidad media de la flota [km/h]'); ax.set_ylabel('densidad')
113     ax.set_title('(c) Efecto de la correlacion'); ax.legend(fontsize=7.5)
114
115     ax = axs[3]
116     Ns = np.arange(1, 25)
117     u_corr = np.sqrt(rel_sys2*vbar**2 + (1/Ns**2)*N*(vbar*u_dp/dp_true.mean())**2)*3.6
118     u_naive = np.sqrt((rel_sys2 + (u_dp/dp_true.mean())**2)*vbar**2/Ns)*3.6
119     ax.plot(Ns, u_corr, '-o', color='#0b3d91', ms=3, lw=1.4, label='correlacionado')
120     ax.plot(Ns, u_naive, '-s', color='#b22222', ms=3, lw=1.4, label='ingenuo')
121     ax.axhline(np.sqrt(rel_sys2)*vbar*3.6, color='gray', ls=':',

```

```

122     lw=1.2, label='piso sistematico')
123     ax.set_xlabel('numero de vehiculos $N$'); ax.set_ylabel('$u$
(media) [km/h]')
124     ax.set_title('(d) La sistematica no promedia'); ax.legend(
fontsize=8)
125
126     fig.suptitle('UAV + flujo optico: incertidumbre de la
velocidad y el papel de las correlaciones (sistematico vs
aleatorio)', y=1.03)
127     plt.savefig('Figuras/fig_s6_ej5_uav_flujo.png', dpi=300,
bbox_inches='tight')
128     # plt.show()
print("Figura guardada.")

```

Listing 41: Semana 6 – Código 5: Velocimetría por flujo óptico desde UAV: incertidumbre correlacionada. Ejecutar en Google Colab.

6.14.6. Código 6 — Exactitud de un sistema RAG: soporte acotado

Cascada recuperación–generación, propagación con distribuciones Beta y comparación con intervalos normal y de Wilson.

```

1      """
2      Semana 6 - Ejemplo 6: Incertidumbre de la exactitud de un
sistema RAG
3      industrial estimada con un conjunto de evaluacion FINITO.
4      Modelo de composicion: a_e2e = a_ret * a_gen/ret (dos
proporciones)
5      Cada proporcion se estima de n ensayos -> incertidumbre Tipo
A binomial.
6      Punto clave: con n pequeno y p cerca de 1, el intervalo GUM-
normal puede
7      exceder [0,1] (no fisico); MC/Beta respeta el soporte. Se
comparan
8      GUM (LPU), Monte Carlo (posterior Beta) y Wilson.
9      """
10     import numpy as np
11     import matplotlib
12     matplotlib.use('Agg')
13     import matplotlib.pyplot as plt
14     import matplotlib as mpl
15     from scipy import stats
16

```

```

17     mpl.rcParams.update({
18         'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
19         10,
20         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
21     },
22     'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
23     })
24     rng = np.random.default_rng(606606)
25     import os; os.makedirs('Figuras', exist_ok=True) # carpeta
26     de salida en Colab
27
28     # --- resultados de la evaluacion (conteos) ---
29     n_ret, k_ret = 60, 54 # 54/60 recuperaciones
30     correctas
31     n_gen, k_gen = 48, 46 # 46/48 generaciones correctas
32     (alta exactitud, n pequeno)
33     p_ret = k_ret/n_ret
34     p_gen = k_gen/n_gen
35     a0 = p_ret*p_gen
36     print("=== EJEMPLO 6: SISTEMA RAG ===")
37     print(f"a_ret={p_ret:.4f} (n={n_ret}), a_gen={p_gen:.4f} (n
38     ={n_gen}), a_e2e={a0:.4f}")
39
40     # --- Tipo A binomial (error estandar de cada proporcion)
41     ---
42     u_ret = np.sqrt(p_ret*(1-p_ret)/n_ret)
43     u_gen = np.sqrt(p_gen*(1-p_gen)/n_gen)
44     # --- GUM (LPU) modelo producto ---
45     u_a = a0*np.sqrt((u_ret/p_ret)**2 + (u_gen/p_gen)**2)
46     U_a = 1.96*u_a
47     lo_gum, hi_gum = a0-U_a, a0+U_a
48     print(f"u_ret={u_ret:.4f}, u_gen={u_gen:.4f}, u(a_e2e)={u_a
49     :.4f}")
50     hi_gen_normal = p_gen + 1.96*u_gen
51     print(f"[GUM] componente a_gen normal 95% sup = {
52     hi_gen_normal:.4f} (excede 1? {hi_gen_normal>1})")
53     print(f"[GUM] 95%: [{lo_gum:.4f}, {hi_gum:.4f}] (a_e2e
54     excede 1? {hi_gum>1})")
55
56     # --- Monte Carlo con posterior Beta (Jeffreys: a=k+0.5, b=n
57     -k+0.5) ---
58     M = 300_000
    
```



```

48     ret_s = rng.beta(k_ret+0.5, n_ret-k_ret+0.5, M)
49     gen_s = rng.beta(k_gen+0.5, n_gen-k_gen+0.5, M)
50     a_s = ret_s*gen_s
51     a_mc = a_s.mean(); u_mc = a_s.std(ddof=1)
52     lo_mc, hi_mc = np.percentile(a_s, [2.5, 97.5])
53     print(f"[MC ] media={a_mc:.4f}, u={u_mc:.4f}, 95%: [{lo_mc:.4f}, {hi_mc:.4f}]")
54
55     # --- Wilson para a_e2e via delta-method sobre log (
referencia) ---
56     def wilson(k, n, z=1.96):
57         p = k/n
58         den = 1+z**2/n
59         c = (p + z**2/(2*n))/den
60         hw = z*np.sqrt(p*(1-p)/n + z**2/(4*n**2))/den
61         return c-hw, c+hw
62     lo_rw, hi_rw = wilson(k_ret, n_ret); lo_gw, hi_gw = wilson(
k_gen, n_gen)
63     # producto de intervalos (conservador)
64     lo_w, hi_w = lo_rw*lo_gw, hi_rw*hi_gw
65     print(f"[Wilson prod] 95%: [{lo_w:.4f}, {hi_w:.4f}]")
66
67     # --- proyeccion: respuestas correctas esperadas por 1000
consultas ---
68     print(f"Por 1000 consultas: {a0*1000:.0f} correctas; 95% MC:
[{lo_mc*1000:.0f}, {hi_mc*1000:.0f}]")
69
70     # --- u(a_e2e) en funcion del tamaño del conjunto de
evaluacion ---
71     ns = np.array([20, 30, 50, 80, 120, 200, 400, 800])
72     u_vs_n = a0*np.sqrt((np.sqrt(p_ret*(1-p_ret)/ns)/p_ret)**2
73 + (np.sqrt(p_gen*(1-p_gen)/ns)/p_gen)**2)
74
75     # --- figura ---
76     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
77
78     ax = axs[0]
79     xx = np.linspace(0.6, 1.08, 500)
80     ax.plot(xx, stats.beta.pdf(xx, k_ret+0.5, n_ret-k_ret+0.5),
color='#0b3d91', lw=1.8,
81     label=f'$a_{\{ret\}}$ Beta ({k_ret}/{n_ret})')
82     ax.plot(xx, stats.beta.pdf(xx, k_gen+0.5, n_gen-k_gen+0.5),

```

```

color='#b22222', lw=1.8,
83     label=f'$a_{{gen}}$ Beta ( $\{k_{gen}\}/\{n_{gen}\}$ )')
84     # aproximacion normal del componente alto: cola no fisica >
1
85     ng = stats.norm.pdf(xx, p_gen, u_gen)
86     ax.plot(xx, ng, color='#b22222', lw=1.2, ls='--', label='$a_{{gen}}$ normal (GUM)')
87     ax.fill_between(xx[xx>1], 0, ng[xx>1], color='red', alpha
=0.3)
88     ax.axvline(1.0, color='k', lw=1.2)
89     ax.text(1.002, ax.get_ylim()[1]*0.5, 'cola\n $p>1$ ', fontsize
=7.5, color='red')
90     ax.set_xlabel('proporcion'); ax.set_ylabel('densidad
posterior')
91     ax.set_title('(a) La normal asigna masa a  $p>1$ '); ax.legend
(fontsize=7.2, loc='upper left')
92
93     ax = axs[1]
94     ax.hist(a_s, bins=120, density=True, color='#9ecae1',
edgecolor='none', label='MC (Beta)')
95     xg = np.linspace(a0-4*u_a, min(a0+4*u_a, 1.05), 300)
96     ax.plot(xg, stats.norm.pdf(xg, a0, u_a), 'r-', lw=1.6, label
='GUM (normal)')
97     ax.axvline(a0, color='#b22222', ls=':', lw=1.2)
98     ax.axvline(a_mc, color='k', ls=':', lw=1.2)
99     ax.set_xlabel('$a_{{e2e}}$'); ax.set_ylabel('densidad')
100    ax.set_title(f'(b) Asimetria por soporte ( $\gamma_1=\{stats
.skew(a_s):+.2f\}$ )'); ax.legend(fontsize=8, loc='upper left')
101
102    ax = axs[2]
103    methods = ['GUM\n(normal)', 'Monte Carlo\n(Beta)', 'Wilson\n
(producto)']
104    los = [lo_gum, lo_mc, lo_w]; his = [hi_gum, hi_mc, hi_w]
105    cen = [a0, a_mc, a0]
106    for i,(l,h,c) in enumerate(zip(los,his,cen)):
107    ax.plot([l,h],[i,i],'-', color='#0b3d91', lw=3)
108    ax.plot(c,i,'o',color='#b22222',ms=7)
109    ax.axvline(1.0, color='k', ls='--', lw=1.2)
110    ax.set_yticks(range(3)); ax.set_yticklabels(methods,
fontsize=8.5)
111    ax.set_xlabel('$a_{{e2e}}$ (intervalo 95%)')
112    ax.set_title('(c) Intervalos de cobertura'); ax.set_ylim

```

```

113     (-0.6, 2.6)
114     ax = axs[3]
115     ax.loglog(ns, u_vs_n, '-o', color='#0b3d91', ms=4, lw=1.5,
116     label='$u(a_{e2e})$')
117     ax.loglog(ns, u_a*np.sqrt(n_ret/ns), 'r--', lw=1.2, label=r'
118     $\propto n^{-1/2}$')
119     for utgt, lbl in [(0.01, '1%')]:
120     ax.axhline(utgt, color='gray', ls=':', lw=1)
121     ax.text(ns[0], utgt*1.1, f'objetivo {lbl}', fontsize=7.5,
122     color='gray')
123     ax.set_xlabel('tamano del conjunto de evaluacion $n$'); ax.
124     set_ylabel('$u(a_{e2e})$')
125     ax.set_title('(d) Precision vs tamano de evaluacion'); ax.
126     legend(fontsize=8)

127     fig.suptitle('Sistema RAG: incertidumbre de la exactitud por
128     muestra finita (GUM, Monte Carlo y Wilson)', y=1.03)
129     plt.savefig('Figuras/fig_s6_ej6_rag.png', dpi=300,
130     bbox_inches='tight')
131     # plt.show()
132     print("Figura guardada.")

```

Listing 42: Semana 6 – Código 6: Exactitud de un sistema RAG: soporte acotado. Ejecutar en Google Colab.

6.15. Ejercicios Reto — Tarea Semana 6

Los Ejercicios 1 y 2 son obligatorios. El Ejercicio 3 otorga punto extra. Todos deben anclarse al *proyecto doctoral propio* del estudiante: el mensurando, las fuentes de incertidumbre y los datos provienen de su propia investigación.

6.15.1. Ejercicio 1 (obligatorio) — Modelo de medición y propagación analítica

Formule el modelo de medición $Y = f(X_1, \dots, X_n)$ de una magnitud central de su proyecto. Entregue: (i) la ecuación del modelo y la identificación de cada X_i como Tipo A o Tipo B, justificando la distribución asignada por máxima entropía; (ii) la tabla de presupuesto de incertidumbre con los coeficientes de sensibilidad $c_i = \frac{\partial f}{\partial X_i}$ evaluados numéricamente; (iii) la incertidumbre combinada $u_c(y)$ por LPU (178), *incluyendo* al menos un término de correlación si su modelo lo requiere; y (iv) la incertidumbre expandida $U = k u_c$ con k obtenido por Welch–Satterthwaite (180). Reporte el resultado en la forma $y \pm U$ con su nivel de cobertura.

Rúbrica.

- 3 Modelo correcto, clasificación A/B justificada, presupuesto completo con sensibilidades numéricas, u_c con correlación y U con k por Welch–Satterthwaite. Reporte metrológicamente correcto.
- 2 Modelo y u_c correctos pero sin término de correlación cuando corresponde, o k asintótico (1.96) sin justificar los grados de libertad.
- 1 Propagación incompleta: faltan coeficientes de sensibilidad o el presupuesto no cierra.
- 0 Sin modelo de medición o sin propagación analítica.

6.15.2. Ejercicio 2 (obligatorio) — Propagación Monte Carlo y validación de la GUM

Implemente en Google Colab la propagación Monte Carlo del mismo modelo del Ejercicio 1, con $M \geq 10^5$ pruebas y generador reproducible (`numpy.random.default_rng(semilla)`). Entregue: (i) el histograma del mensurando con el intervalo de cobertura del 95 % por percentiles; (ii) la comparación numérica de los intervalos GUM y Monte Carlo; (iii) la aplicación del criterio de validación (182) con una tolerancia δ explícita derivada del número de cifras significativas que desea reportar; y (iv) una decisión razonada: ¿reporta el resultado GUM o el Monte Carlo, y por qué? Si su modelo es no lineal, asimétrico o de soporte acotado, discuta el mecanismo de la discrepancia.

Rúbrica.

- 3 Implementación reproducible, intervalo por percentiles correcto, validación con δ explícita y decisión justificada por el mecanismo físico/estadístico de la discrepancia.
- 2 Monte Carlo correcto pero validación sin δ explícita, o decisión no justificada.
- 1 Simulación que corre pero con generador no reproducible o sin intervalo de cobertura bien construido.
- 0 Sin implementación Monte Carlo funcional.

6.15.3. Ejercicio 3 — Procedimiento Monte Carlo adaptativo

Aplique el procedimiento adaptativo de la JCGM 101: en lugar de fijar M arbitrariamente, incremente M por bloques hasta que los límites del intervalo de cobertura sean numéricamente estables a la última cifra significativa deseada (tolerancia δ). Entregue la curva de convergencia de los límites del intervalo y de u_c frente a M , verificando empíricamente la tasa $O(M^{-1/2})$ de (181), y reporte el M final adoptado con su justificación.

Rúbrica.

- 3 Procedimiento adaptativo implementado, convergencia $O(M^{-1/2})$ verificada empíricamente y M final justificado por estabilidad numérica.
- 0 No entregado o sin verificación de convergencia.

7. Semana 7. Cadenas de Markov y Sistemas Dinámicos

Objetivo de la cátedra

Formular, analizar y simular sistemas cuya evolución es estocástica y depende únicamente de su estado presente. El estudiante debe ser capaz de construir la matriz de transición (tiempo discreto) o la matriz generadora (tiempo continuo) de un proceso de su propio proyecto, calcular su distribución estacionaria, sus tiempos característicos (recurrencia, primer paso, vida media hasta la falla) y su régimen transitorio, simular trayectorias por métodos exactos, reconocer la transición de fase en procesos espaciales, y comprender cómo el límite determinista de un proceso de Markov produce un sistema dinámico de ecuaciones diferenciales.

Temario

- Procesos estocásticos y propiedad de Markov.
- Matrices de transición: evolución, Chapman–Kolmogorov y potencias.
- Clasificación de estados, distribución estacionaria y teorema ergódico.
- Cadenas absorbentes, tiempos de primer paso y matriz fundamental.
- Cadenas de tiempo continuo: matriz generadora y ecuaciones de Kolmogorov.
- Simulación estocástica exacta: algoritmo de Gillespie.
- Sistemas dinámicos discretos y espaciales: autómatas celulares y percolación (incendios forestales).
- Del proceso estocástico al sistema dinámico determinista: límite de campo medio (modelado epidemiológico).
- Teoría espectral, tiempo de mezcla y caminatas aleatorias (PageRank); sistemas de confiabilidad.
- Interpretación estadística: estimación de la matriz de transición por máxima verosimilitud.

Propósito del tema. La Semana 6 convirtió cada resultado determinista en una distribución de probabilidad, pero esa distribución era *estática*: describía la incertidumbre de un número en un instante. Esta semana introduce el *tiempo* y el *estado*. Un proceso de Markov es la mínima estructura matemática que describe cómo un sistema salta entre configuraciones cuando el futuro depende del presente pero no de la trayectoria

que llevó hasta él. Con tres objetos — la matriz de transición, la distribución estacionaria y los tiempos de primer paso — se responde a preguntas que ningún cálculo determinista puede contestar: ¿qué fracción del tiempo estará un equipo fuera de servicio?, ¿cuánto vivirá en promedio antes de fallar?, ¿cuándo un incendio deja de ser local y cruza todo el dominio?, ¿se extinguirá un brote o se volverá epidemia? La misma maquinaria, llevada al límite, reproduce los sistemas dinámicos deterministas de las semanas anteriores: una cadena de Markov es un sistema dinámico al que se le ha devuelto el azar que la idealización determinista le había quitado.

Resultado de aprendizaje esperado

Al término de la semana el estudiante podrá: (i) identificar si un fenómeno de su proyecto admite una representación markoviana y definir su espacio de estados, su matriz de transición P o su generador Q ; (ii) calcular la distribución estacionaria, la disponibilidad de régimen permanente, los tiempos medios de recurrencia y el tiempo medio hasta la absorción mediante la matriz fundamental; (iii) simular trayectorias del proceso con un método exacto y reproducible en Google Colab, validando los resultados analíticos contra la simulación; (iv) reconocer y cuantificar la transición de fase de un proceso espacial (percolación) y el umbral epidémico de un proceso de propagación; y (v) estimar una matriz de transición a partir de datos observados con su incertidumbre asociada, cerrando el ciclo entre modelo, simulación y evidencia.

7.1. Procesos estocásticos y la propiedad de Markov

Un *proceso estocástico* es una familia de variables aleatorias $\{X_t\}_{t \in \mathcal{T}}$ indexadas por el tiempo, que toman valores en un conjunto $\mathcal{S}$ llamado *espacio de estados*. Cuando $\mathcal{T} = \{0, 1, 2, \dots\}$ el proceso es de *tiempo discreto*; cuando $\mathcal{T} = [0, \infty)$ es de *tiempo continuo*. Cuando $\mathcal{S}$ es finito o numerable, el proceso es de *estados discretos*, que es el caso central de esta semana.

La descripción completa de un proceso general requiere todas sus distribuciones conjuntas, una cantidad de información intratable. La *propiedad de Markov* es la hipótesis de simplificación que vuelve manejable el problema: el futuro depende del pasado únicamente a través del presente. En tiempo discreto,

$$\Pr(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = \Pr(X_{n+1} = j \mid X_n = i), \quad (191)$$

para todo n y toda historia admisible. Un proceso que cumple (191) es una *cadena de Markov*. La condición no afirma que el sistema carezca de memoria física, sino que toda la memoria relevante está codificada en la definición del estado. Esta es una decisión

de modelado, no una propiedad de la naturaleza: ampliar el estado (por ejemplo, incluir la temperatura previa además de la actual) puede restaurar la propiedad de Markov en sistemas que aparentemente la violan.

La probabilidad de transición de un paso se denota

$$p_{ij} = \Pr(X_{n+1} = j \mid X_n = i), \quad p_{ij} \geq 0, \quad \sum_{j \in \mathcal{S}} p_{ij} = 1. \quad (192)$$

Si p_{ij} no depende de n , la cadena es *homogénea* en el tiempo, hipótesis que se adoptará salvo indicación contraria.

Ejemplo introductorio: el clima de dos estados

El ejemplo canónico más simple tiene dos estados, $\mathcal{S} = \{\text{soleado}, \text{lluvioso}\}$. Suponga que tras un día soleado, el siguiente es soleado con probabilidad 0.8 y lluvioso con 0.2; tras un día lluvioso, es soleado con 0.4 y lluvioso con 0.6. La matriz de transición es

$$P = \begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{pmatrix}.$$

Si hoy está soleado, $\pi_0 = (1, 0)$, la probabilidad de cada clima mañana es $\pi_1 = \pi_0 P = (0.8, 0.2)$, y pasado mañana $\pi_2 = \pi_1 P = (0.72, 0.28)$. Iterando, la distribución converge a $\pi = (2/3, 1/3)$, que satisface $\pi P = \pi$: en el largo plazo, dos tercios de los días son soleados, *independientemente* del clima inicial. Este límite, su existencia y su unicidad son el contenido del teorema ergódico que se formaliza más adelante.

7.2. Matriz de transición: evolución y ecuaciones de Chapman–Kolmogorov

Reuniendo las probabilidades (192) en una matriz $P = [p_{ij}]$ se obtiene la *matriz de transición*, que es *estocástica por filas*: sus entradas son no negativas y cada fila suma uno. El estado de conocimiento sobre el sistema en el paso n se describe mediante un *vector de probabilidad* fila $\pi_n = (\pi_n(1), \pi_n(2), \dots)$, con $\pi_n(i) = \Pr(X_n = i)$. La evolución es una recursión lineal:

$$\pi_{n+1} = \pi_n P, \quad \implies \quad \pi_n = \pi_0 P^n. \quad (193)$$

La probabilidad de transición en n pasos es la entrada (i, j) de la potencia P^n , denotada $p_{ij}^{(n)}$. Estas potencias satisfacen las *ecuaciones de Chapman–Kolmogorov*, que expresan que para ir de i a j en $m + n$ pasos se debe pasar por algún estado intermedio k tras m

pasos:

$$p_{ij}^{(m+n)} = \sum_{k \in S} p_{ik}^{(m)} p_{kj}^{(n)}, \quad \text{es decir} \quad P^{(m+n)} = P^m P^n. \quad (194)$$

La ecuación (194) no es un teorema profundo sino la traducción de la propiedad de Markov a las potencias matriciales; su valor es operativo: reduce el cálculo de cualquier transición multipaso a la multiplicación de matrices, una operación que la Semana 2 ya dejó instalada en el flujo de trabajo computacional. El comportamiento de P^n cuando $n \rightarrow \infty$ contiene toda la información de largo plazo de la cadena y se controla mediante su espectro, como se verá en la teoría espectral.

7.3. Clasificación de estados, distribución estacionaria y teorema ergódico

No toda cadena converge a una distribución única. El comportamiento asintótico depende de propiedades estructurales del grafo de transiciones.

- Un estado j es **accesible** desde i si $p_{ij}^{(n)} > 0$ para algún n . Si i y j son mutuamente accesibles, *comunican*. Una cadena es **irreducible** si todos sus estados comunican (un solo bloque).
- Un estado es **recurrente** si el proceso regresa a él con probabilidad uno, y **transitorio** en caso contrario. En una cadena finita irreducible, todos los estados son recurrentes positivos.
- Un estado es **absorbente** si $p_{ii} = 1$: una vez alcanzado, no se abandona. La “falla” irreparable, la “recuperación” definitiva o el “fin de un brote” son estados absorbentes.
- El **periodo** de un estado es el máximo común divisor de los n para los que $p_{ii}^{(n)} > 0$. Un estado es **aperiódico** si su periodo es 1; basta una autotransición $p_{ii} > 0$ para garantizarlo.

Una distribución π es **estacionaria** (o invariante) si es un punto fijo de la evolución (193):

$$\pi = \pi P, \quad \sum_{i \in S} \pi_i = 1, \quad \pi_i \geq 0. \quad (195)$$

Algebraicamente, π es el vector propio izquierdo de P asociado al autovalor 1, normalizado a suma unitaria. El resultado central de la teoría es el **teorema ergódico**: si una cadena finita es irreducible y aperiódica, entonces existe una única distribución estacionaria π , y

$$\lim_{n \rightarrow \infty} p_{ij}^{(n)} = \pi_j \quad \text{para todo } i, \quad (196)$$

es decir, la cadena “olvida” su condición inicial y la fracción de tiempo que pasa en el estado j tiende a π_j . Una consecuencia directa de gran utilidad práctica es el *tiempo*

medio de recurrencia del estado i , esto es, el número esperado de pasos entre dos visitas consecutivas:

$$m_{ii} = \frac{1}{\pi_i}. \quad (197)$$

La identidad (197) convierte una probabilidad estacionaria en un tiempo físico: si un estado de “fuera de calibración” tiene $\pi = 0.14$, el sistema regresa a él, en promedio, cada 7.25 pasos. Esta es la base del Ejemplo 1.

7.4. Cadenas absorbentes, tiempos de primer paso y matriz fundamental

Cuando la pregunta de ingeniería no es “¿qué fracción del tiempo?” sino “¿cuánto tarda en ocurrir un evento?”, el marco adecuado es el de las *cadenas absorbentes*. Una cadena con estados transitorios y absorbentes se ordena en su *forma canónica*

$$P = \begin{pmatrix} Q & R \\ \mathbf{0} & I \end{pmatrix}, \quad (198)$$

donde Q ($t \times t$) reúne las transiciones entre los t estados transitorios, R las transiciones de transitorios a absorbentes, e I la identidad de los estados absorbentes. La **matriz fundamental**

$$N = (I - Q)^{-1} = \sum_{k=0}^{\infty} Q^k \quad (199)$$

tiene una interpretación directa: N_{ij} es el número esperado de visitas al estado transitorio j partiendo de i , antes de la absorción. De aquí se obtienen dos cantidades de inmenso valor en confiabilidad:

$$\mathbf{t} = N \mathbf{1}, \quad B = N R, \quad (200)$$

donde t_i es el número esperado de pasos hasta la absorción partiendo de i (el *tiempo medio hasta la falla*, MTTF, cuando el estado absorbente es la falla) y B_{ik} es la probabilidad de ser absorbido por el estado k partiendo de i . La serie de Neumann en (199) converge porque el radio espectral de Q es menor que uno: la fuga de probabilidad hacia los estados absorbentes garantiza que $Q^k \rightarrow 0$. El Ejemplo 2 calcula el MTTF de un banco de pruebas por esta vía y lo valida contra simulación Monte Carlo.

7.5. Cadenas de Markov de tiempo continuo: matriz generadora

Muchos fenómenos físicos no avanzan en pasos discretos sino en tiempo continuo: una falla puede ocurrir en cualquier instante. La cadena de Markov de tiempo continuo (CTMC) sustituye las *probabilidades* por paso por *tasas* de transición. La hipótesis markoviana en tiempo continuo equivale a suponer que el tiempo de permanencia en cada

estado es exponencial (la única distribución sin memoria): el sistema permanece en el estado i un tiempo $\sim \text{Exp}(|q_{ii}|)$ y luego salta a j con probabilidad $q_{ij}/|q_{ii}|$.

La dinámica se codifica en la **matriz generadora** $Q = [q_{ij}]$, donde $q_{ij} \geq 0$ ($i \neq j$) es la tasa de transición de i a j , y los elementos diagonales se fijan para que cada fila sume cero:

$$q_{ii} = - \sum_{j \neq i} q_{ij}, \quad \sum_{j \in S} q_{ij} = 0. \quad (201)$$

Las probabilidades de transición $P(t) = [p_{ij}(t)]$ a tiempo t obedecen las *ecuaciones de Kolmogorov*, un sistema lineal de ecuaciones diferenciales:

$$\frac{dP(t)}{dt} = P(t) Q \quad (\text{adelantada}), \quad \frac{dP(t)}{dt} = Q P(t) \quad (\text{atrasada}), \quad (202)$$

cuya solución, con $P(0) = I$, es la *exponencial matricial*

$$P(t) = \exp(Q t) = \sum_{k=0}^{\infty} \frac{(Q t)^k}{k!}. \quad (203)$$

La distribución estacionaria de una CTMC irreducible satisface el análogo continuo de (195):

$$\pi Q = \mathbf{0}, \quad \sum_i \pi_i = 1. \quad (204)$$

El balance de flujos $\pi Q = \mathbf{0}$ expresa que, en régimen permanente, el flujo de probabilidad que entra a cada estado iguala al que sale. El tiempo medio hasta la absorción se obtiene del subgenerador transitorio Q_T (el bloque de Q entre estados no absorbentes):

$$\mathbf{m} = -Q_T^{-1} \mathbf{1}. \quad (205)$$

El Ejemplo 3 modela un cable OPGW como una CTMC de cuatro estados, resuelve su estacionaria por (204), su transitorio por (203) y su MTTF por (205).

7.6. Simulación estocástica exacta: el algoritmo de Gillespie

Resolver (203) entrega la distribución de probabilidad, pero a menudo se requieren *trayectorias individuales*: una realización concreta de cómo evolucionó el sistema. Para una CTMC, el algoritmo de Gillespie (*Stochastic Simulation Algorithm*, SSA) genera trayectorias exactas, no aproximadas. Dado el estado actual, con tasas de evento $a_1, \dots, a_K$ y tasa total $a_0 = \sum_k a_k$:

1. Se genera el tiempo al siguiente evento $\tau \sim \text{Exp}(a_0)$, es decir $\tau = -\ln(r_1)/a_0$ con $r_1 \sim U(0, 1)$.
2. Se selecciona el evento k que ocurre con probabilidad a_k/a_0 , mediante un segundo

número $r_2 \sim U(0, 1)$.

3. Se actualizan el tiempo $t \leftarrow t + \tau$ y el estado según el evento k , y se recalculan las tasas.

El método es exacto porque respeta la estructura exponencial de los tiempos de permanencia y la competencia entre eventos. Su costo crece con el número de eventos, no con el tamaño del paso temporal, lo que lo hace eficiente para poblaciones moderadas. El Ejemplo 5 usa Gillespie para simular un brote epidémico y comparar sus trayectorias estocásticas con el sistema dinámico determinista.

7.7. Markov en el espacio y en grafos: autómatas celulares, percolación y teoría espectral

7.7.1. Campos de Markov espaciales y percolación

Cuando el estado no es un escalar sino una configuración espacial — una retícula donde cada celda tiene un estado que depende de sus vecinas — el proceso es un *campo de Markov* y su evolución temporal es un *autómata celular probabilístico*. El ejemplo paradigmático es la propagación: una celda “activa” (ardiendo, infectada, dañada) activa a cada vecina susceptible con probabilidad p . Estos modelos exhiben un fenómeno que ningún sistema lineal posee: una *transición de fase*. Existe un valor crítico p_c tal que para $p < p_c$ la activación permanece localizada y se extingue, mientras que para $p > p_c$ aparece, con probabilidad positiva, un *cúmulo percolante* que cruza todo el dominio. Para la propagación a cuatro vecinos en una red cuadrada, el problema es equivalente a la percolación de enlaces, cuyo umbral exacto es

$$p_c = \frac{1}{2}. \quad (206)$$

El parámetro de orden (la fracción activada, o la probabilidad de que la activación alcance el borde) cambia de forma abrupta alrededor de p_c , con fluctuaciones máximas justo en el umbral. El Ejemplo 4 reproduce esta transición para un incendio forestal y estima p_c empíricamente.

7.7.2. Caminatas aleatorias en grafos, teoría espectral y mezcla

Una cadena de Markov sobre los nodos de un grafo dirigido es una *caminata aleatoria*: en cada paso, el caminante salta a un nodo vecino. Su distribución estacionaria mide la “importancia” de cada nodo, definición que está en el corazón del algoritmo PageRank. La velocidad con que la cadena alcanza su estacionaria la determina el *espectro* de P . El autovalor dominante de una matriz estocástica irreducible es siempre $\lambda_1 = 1$ (teorema de Perron–Frobenius), con vector propio izquierdo π . El segundo autovalor en magnitud,

$|\lambda_2| < 1$, controla la convergencia:

$$\|\pi_n - \pi\|_{TV} \leq C |\lambda_2|^n, \quad \text{brecha espectral} = 1 - |\lambda_2|. \quad (207)$$

Una brecha espectral grande implica convergencia rápida (mezcla veloz); una brecha pequeña, convergencia lenta. El *tiempo de mezcla* t_{mix} es el número de pasos para que la distancia en variación total caiga por debajo de un umbral, y escala como $t_{\text{mix}} \sim 1/(1 - |\lambda_2|)$. El Ejemplo 6 explota esta teoría: la matriz de Google garantiza $|\lambda_2| \leq \alpha$, lo que asegura una tasa de convergencia α^n de la iteración de potencias, independiente del tamaño del grafo.

7.8. Del proceso estocástico al sistema dinámico determinista

El vínculo conceptual más importante de la semana es que un sistema dinámico determinista (como los de las Semanas 4 y 5) es el *límite de campo medio* de un proceso de Markov cuando el número de individuos crece. Considérese un proceso con poblaciones $\mathbf{X} = (X_1, \dots, X_m)$ y eventos que las modifican con tasas proporcionales a la población. Al escalar por el tamaño total N y definir las fracciones $\mathbf{x} = \mathbf{X}/N$, la ley de los grandes números para procesos de salto (teorema de Kurtz) establece que, cuando $N \rightarrow \infty$, las trayectorias estocásticas convergen a la solución del sistema de ecuaciones diferenciales

$$\frac{d\mathbf{x}}{dt} = \sum_k \nu_k f_k(\mathbf{x}), \quad (208)$$

donde ν_k es el cambio de estado del evento k y f_k su tasa por unidad de población. Para el modelo epidemiológico SIR, con tasa de contagio β y de recuperación γ , este límite produce el sistema dinámico clásico

$$\frac{dS}{dt} = -\beta \frac{SI}{N}, \quad \frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I, \quad (209)$$

cuyo comportamiento queda determinado por el *número reproductivo básico*

$$R_0 = \frac{\beta}{\gamma}. \quad (210)$$

Si $R_0 > 1$ la infección crece inicialmente (umbral epidémico $S^* = N/R_0$); si $R_0 < 1$ se extingue. La descripción determinista (209), sin embargo, oculta un fenómeno puramente estocástico: aun con $R_0 > 1$, un brote iniciado por pocos infectados puede extinguirse por azar antes de despegar. La teoría de procesos de ramificación estima esta probabilidad

de *brote menor* como $(1/R_0)^{I_0}$, de modo que la probabilidad de brote mayor es

$$P_{\text{mayor}} \approx 1 - \left(\frac{1}{R_0} \right)^{I_0}. \quad (211)$$

El Ejemplo 5 compara (209), (210) y (211) contra el ensamble estocástico, mostrando que el modelo determinista describe bien el “brote mayor” pero es ciego a la extinción temprana.

7.9. Interpretación estadística: estimación de la matriz de transición

Cerrando el ciclo entre modelo y datos, una matriz de transición rara vez se conoce a priori: debe estimarse a partir de una trayectoria observada. Si n_{ij} es el número de transiciones $i \rightarrow j$ contadas en los datos y $n_i = \sum_j n_{ij}$ el total de salidas de i , el estimador de *máxima verosimilitud* es la frecuencia relativa

$$\hat{p}_{ij} = \frac{n_{ij}}{n_i}, \quad \widehat{\text{se}}(\hat{p}_{ij}) = \sqrt{\frac{\hat{p}_{ij}(1 - \hat{p}_{ij})}{n_i}}, \quad (212)$$

con error estándar de Wald asintótico. Cada fila de P se estima de forma independiente como una distribución multinomial, y la incertidumbre de cada $\hat{p}_{ij}$ se reduce como $n_i^{-1/2}$: estimar bien las transiciones de un estado raro exige observarlo muchas veces. La ecuación (212) conecta esta semana con la metrología de la Semana 6: la matriz de transición es ella misma un mensurando con incertidumbre, y propagar esa incertidumbre a la distribución estacionaria o al MTTF es un problema de propagación que las técnicas Monte Carlo de la semana anterior resuelven directamente.

7.10. Ejemplo 1 — Deriva de calibración de un sensor: la cadena de dos estados

7.10.1. Planteamiento

Un sensor del sistema de calibración del proyecto de Álvarez Sánchez se inspecciona periódicamente (cada semana) y se clasifica en uno de dos estados: *en calibración* (dentro del error máximo permisible) o *fuera de calibración*. Entre inspecciones, el sensor puede derivar fuera de tolerancia con probabilidad a por semana; cuando se detecta fuera de tolerancia, se recalibra y vuelve a tolerancia con probabilidad b . El proceso es una cadena de Markov de dos estados con matriz de transición

$$P = \begin{pmatrix} 1-a & a \\ b & 1-b \end{pmatrix} = \begin{pmatrix} 0.92 & 0.08 \\ 0.50 & 0.50 \end{pmatrix}, \quad (213)$$

con $a = 0.08$ y $b = 0.50$. La pregunta metrológica no es “¿está calibrado hoy?” sino “¿qué fracción del tiempo opera fuera de tolerancia?” y “¿cada cuánto debe programarse la recalibración?”. Los parámetros se resumen en la Tabla 34.

Tabla 34: Parámetros de la cadena de deriva de calibración (Ec. 213).

Magnitud	Valor	Interpretación
Prob. de deriva a	0.08	$\Pr(\text{en} \rightarrow \text{fuera})$ por semana
Prob. de recalibración b	0.50	$\Pr(\text{fuera} \rightarrow \text{en})$ por semana
Paso temporal	1 semana	periodo de inspección

7.10.2. Resultados y lectura metodológica

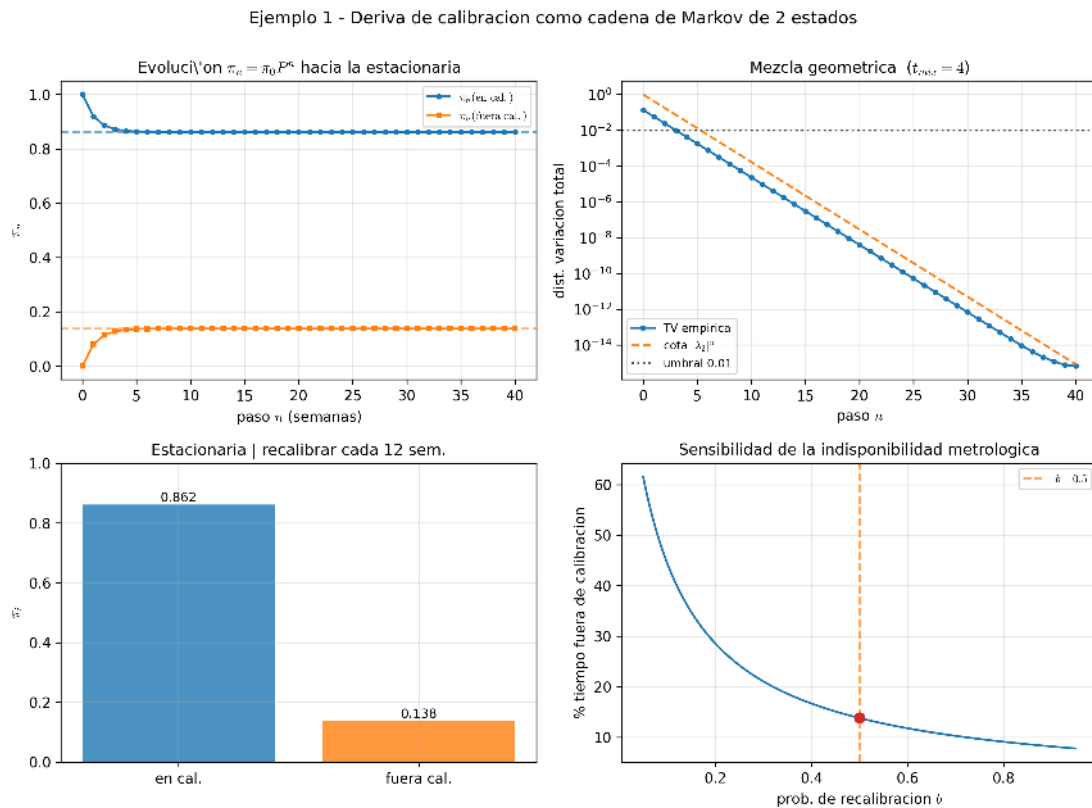


Figura 49: Deriva de calibración como cadena de Markov de dos estados. Panel 1: evolución de la distribución $\pi_n = \pi_0 P^n$ (Ec. 193) hacia la estacionaria. Panel 2: convergencia geométrica en distancia de variación total y su cota $|\lambda_2|^n$ (Ec. 207). Panel 3: distribución estacionaria e intervalo de recalibración recomendado. Panel 4: sensibilidad de la indisponibilidad metrológica a la probabilidad de recalibración b . Resultados del Código 1 del Anexo de esta semana.

1. **La distribución estacionaria cuantifica la indisponibilidad metrológica.** La solución de (195) es $\pi = (b, a)/(a + b) = (0.862, 0.138)$. El sensor opera *fuera de*

tolerancia el 13.79 % del tiempo en régimen permanente, una cifra que ningún análisis de un solo punto de calibración podría revelar. Esta es la indisponibilidad metrológica del instrumento bajo la política de mantenimiento actual.

2. **Los tiempos de primer paso fijan el intervalo de recalibración.** El tiempo medio de deriva en $\rightarrow$ fuera es $1/a = 12.5$ semanas: en promedio, el sensor tarda doce semanas y media en salir de tolerancia tras una calibración. Programar la recalibración *preventiva* en torno a ese horizonte, en lugar de esperar la detección *correctiva*, mantiene el instrumento dentro de tolerancia la mayor parte de su vida útil. El tiempo medio de recurrencia del estado “en calibración” es $1/\pi_0 = 1.16$ semanas (Ec. 197).
3. **La brecha espectral mide la rapidez del olvido.** Los autovalores de P son 1 y $1 - a - b = 0.42$; la brecha espectral es $a + b = 0.58$ y la cadena alcanza su estacionaria (variación total < 0.01) en apenas $t_{\text{mix}} = 4$ pasos. El sistema olvida su condición inicial con rapidez: el estado de calibración de hace un mes es irrelevante para predecir el de hoy.
4. **La interpretación estadística cierra el ciclo.** Simulando una trayectoria de 5000 semanas y aplicando el estimador de máxima verosimilitud (212) se recupera $\hat{P} = \begin{pmatrix} 0.913 & 0.087 \\ 0.512 & 0.488 \end{pmatrix}$, con errores estándar de Wald de 0.004 y 0.019 respectivamente. La fila del estado raro (“fuera”) se estima con cuatro veces más incertidumbre porque se visita cuatro veces menos: la calidad del modelo está limitada por la frecuencia de observación de cada estado.

7.11. Ejemplo 2 — Confiabilidad de un banco de pruebas: cadena con reparación

7.11.1. Planteamiento

El banco de pruebas de bomba centrífuga del proyecto de Leyva Jiménez y Zavala Morales opera diariamente y puede encontrarse en tres estados: *operativo* (nominal), *degradado* (vibración o cavitación incipiente, aún funcional) y en *falla* (fuera de servicio). Las transiciones diarias, incluyendo la reparación, se modelan con la matriz

$$P = \begin{pmatrix} 0.95 & 0.04 & 0.01 \\ 0.30 & 0.65 & 0.05 \\ 0.50 & 0.00 & 0.50 \end{pmatrix}, \quad (214)$$

en el orden (O, D, F). La cadena es irreducible (todo estado comunica gracias a la reparación) y aperiódica, por lo que posee estacionaria única. Las preguntas de ingeniería son la *disponibilidad* de régimen permanente y el *tiempo medio hasta la primera falla* (MTTF) partiendo de un equipo nuevo. Los parámetros aparecen en la Tabla 35.

Tabla 35: Transiciones diarias del banco de pruebas (Ec. 214).

Desde \ a	Operat.	Degrad.	Falla	Mecanismo
Operativo	0.95	0.04	0.01	desgaste / falla súbita
Degradado	0.30	0.65	0.05	reparación / progresión
Falla	0.50	0.00	0.50	reparación a operativo

7.11.2. Disponibilidad y MTTF

La estacionaria, solución de (195), es $\pi = (0.873, 0.0998, 0.0274)$. Para el MTTF se trata la falla como absorbente y se usa la matriz fundamental (199) sobre el bloque transitorio Q entre $\{O, D\}$, obteniendo el tiempo esperado a la absorción (200).

7.11.3. Resultados y lectura metodológica

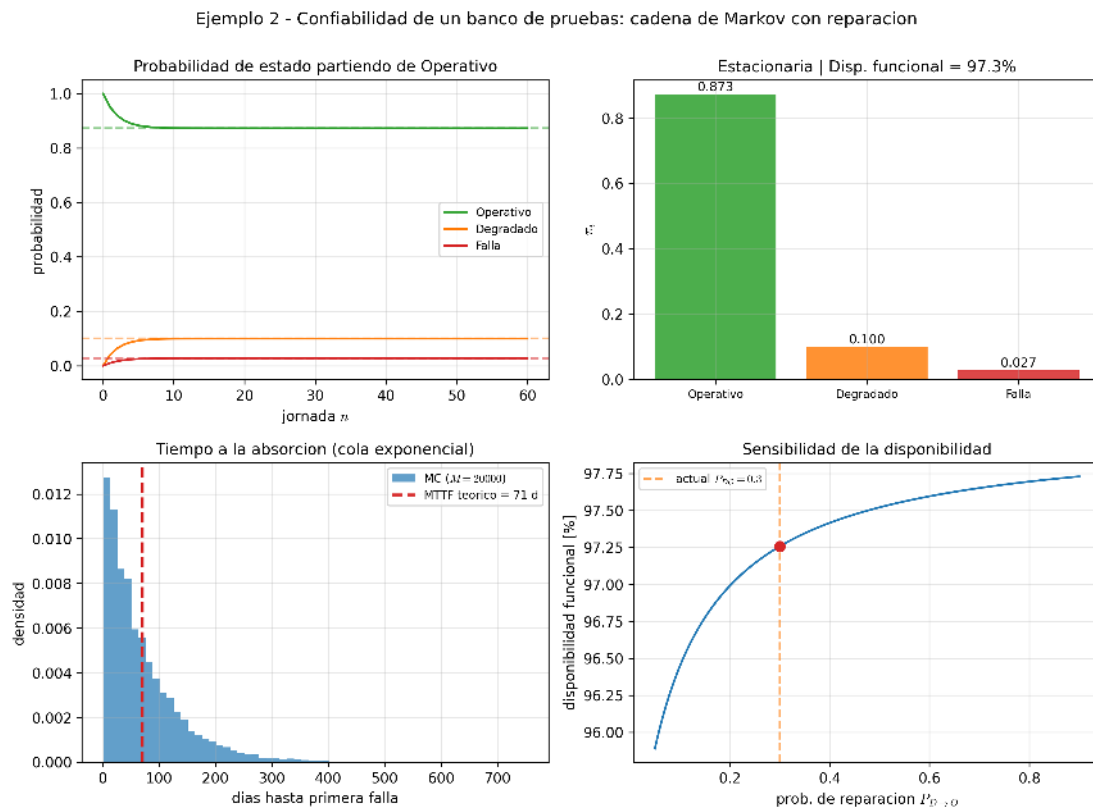


Figura 50: Confiabilidad del banco de pruebas. Panel 1: probabilidad transitoria de cada estado partiendo de Operativo (Ec. 193). Panel 2: distribución estacionaria y disponibilidad funcional. Panel 3: histograma Monte Carlo del tiempo a la primera falla con el MTTF teórico de la matriz fundamental (Ec. 200). Panel 4: sensibilidad de la disponibilidad a la probabilidad de reparación $P_{D \rightarrow O}$. Resultados del Código 2 del Anexo de esta semana.

1. **Disponibilidad de régimen permanente.** La disponibilidad estricta (solo Operativo) es $\pi_O = 87.28\%$ y la *funcional* (Operativo o Degradado, ambos productivos) es $\pi_O + \pi_D = 97.26\%$. La indisponibilidad por falla es $\pi_F = 2.74\%$. La distinción entre disponibilidad estricta y funcional es una decisión de criterio: depende de si el estado degradado permite ensayos válidos o no.
2. **El MTTF se obtiene exactamente y se valida por simulación.** La matriz fundamental da un tiempo medio a la primera falla de 70.91 días partiendo de Operativo. La simulación Monte Carlo independiente ($M = 20000$ trayectorias) arroja 70.86 ± 0.49 días, coincidiendo dentro del error estándar: el cálculo algebraico y la simulación se validan mutuamente. El histograma del tiempo a la falla exhibe la cola aproximadamente exponencial característica de los tiempos de absorción.
3. **El MTBF de régimen permanente difiere del MTTF.** El flujo estacionario hacia la falla, $\sum_i \pi_i p_{iF}$, da un tiempo medio entre fallas de 36.45 días en operación continua. Es menor que el MTTF de 70.91 días porque, en régimen, una fracción del tiempo el equipo ya está degradado y por tanto más cerca de la falla que un equipo nuevo: la condición inicial importa para el MTTF pero no para el MTBF.
4. **La sensibilidad guía la inversión en mantenimiento.** El Panel 4 muestra que aumentar la probabilidad de reparación del estado degradado eleva la disponibilidad de forma monótona pero con rendimiento decreciente. Saber dónde se aplanan la curva indica hasta qué punto invertir en mantenimiento preventivo del estado degradado mejora la disponibilidad antes de toparse con el límite impuesto por las fallas súbitas desde el estado operativo.

7.12. Ejemplo 3 — Confiabilidad de un OPGW: cadena de Markov de tiempo continuo

7.12.1. Planteamiento

El cable OPGW del proyecto de Méndez Gutiérrez y Jurado Pérez está sujeto a sollicitaciones que ocurren en tiempo continuo, no en pasos discretos. Se modela como una CTMC de cuatro estados: *normal* (N), *sobrecarga térmica* (T, corriente elevada, reversible por enfriamiento), *hielo/galloping* (H, sobrecarga mecánica, reversible) y *falla* (F, rotura o cortocircuito que requiere reparación). Las tasas de transición (eventos por año) definen la matriz generadora Q según (201); sus valores aparecen en la Tabla 36. La sobrecarga térmica conecta este ejemplo con el modelo IEC 60949 de la Semana 6: allí se cuantificó la incertidumbre de la temperatura de cortocircuito; aquí se cuantifica con qué frecuencia y por cuánto tiempo el cable opera en ese régimen a lo largo de su vida.

Tabla 36: Tasas de transición del modelo CTMC del OPGW (1/año), usadas para construir el generador Q (Ec. 201).

Transición	Tasa [1/año]	Tiempo medio
Normal $\rightarrow$ sobrecarga térmica	2.0	6 meses
Sobrecarga térmica $\rightarrow$ normal	50.0	7.3 días
Sobrecarga térmica $\rightarrow$ falla	0.5	2 años
Normal $\rightarrow$ hielo/galloping	1.0	1 año
Hielo $\rightarrow$ normal	20.0	18.3 días
Hielo $\rightarrow$ falla	0.3	3.3 años
Falla $\rightarrow$ normal (reparación)	12.0	1 mes

7.12.2. Resultados y lectura metodológica

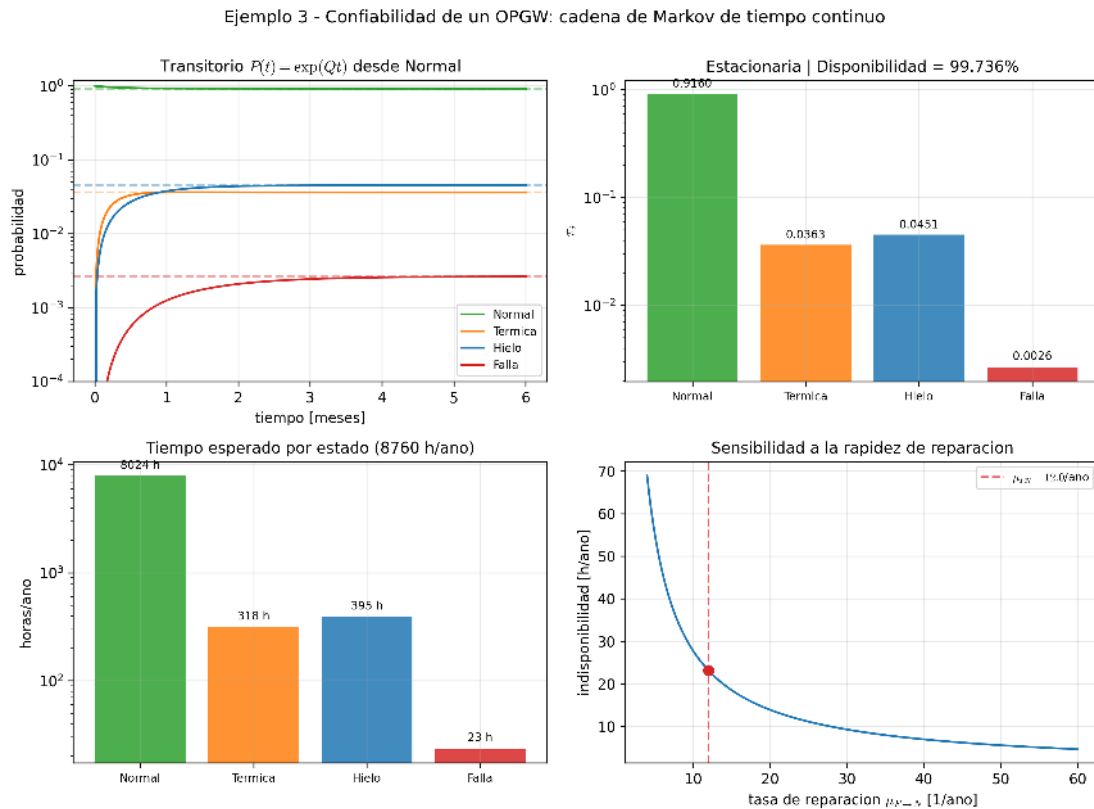


Figura 51: Confiabilidad del OPGW como CTMC. Panel 1: probabilidades transitorias $P(t) = \exp(Qt)$ partiendo de Normal (Ec. 203). Panel 2: distribución estacionaria (escala logarítmica) y disponibilidad. Panel 3: horas esperadas por año en cada estado. Panel 4: sensibilidad de la indisponibilidad a la tasa de reparación $\mu_{F \rightarrow N}$. Resultados del Código 3 del Anexo de esta semana.

1. **La estacionaria entrega la disponibilidad del activo.** La solución de (204) es $\pi = (0.916, 0.0363, 0.0451, 0.00264)$. La disponibilidad (todo estado salvo falla) es 99.736 % y la indisponibilidad por falla, 0.264 %, equivale a 23.12 horas de fuera

de servicio por año. Esta es la cifra que un operador de red eléctrica necesita para sus índices de continuidad de servicio.

2. **El tiempo en cada estado se traduce a horas físicas.** Multiplicando π por las 8760 horas del año, el cable pasa 317.8 h/año en sobrecarga térmica y 395.3 h/año en condición de hielo. La fracción de tiempo en sobrecarga térmica (3.63 %) es precisamente la exposición acumulada al régimen que el modelo térmico de la Semana 6 analizó puntualmente: el envejecimiento del material es función de esta exposición integrada, no de un único evento de cortocircuito.
3. **El MTTF se obtiene del subgenerador transitorio.** Tratando la falla como absorbente, (205) da un tiempo medio hasta la primera falla de 31.49 años partiendo del estado normal. Este horizonte, comparado con la vida de diseño del cable, indica si el activo está sobredimensionado o si requiere inspección reforzada.
4. **La rapidez de reparación gobierna la indisponibilidad.** El Panel 4 muestra que la indisponibilidad anual decae al aumentar la tasa de reparación $\mu_{F \rightarrow N}$ (equivalente a reducir el tiempo medio de reparación, MTTR). La relación es fuertemente no lineal: pasar de un MTTR de tres meses a uno de un mes recorta las horas de indisponibilidad de forma desproporcionada, lo que prioriza la logística de reparación rápida sobre otras inversiones.

7.13. Ejemplo 4 — Incendio forestal: autómatas celulares probabilísticos y percolación

7.13.1. Planteamiento

El proyecto de monitoreo aéreo de García Ojeda admite una extensión natural a la vigilancia de incendios forestales mediante UAV. El fenómeno se modela como un autómata celular probabilístico sobre una retícula $L \times L$: cada celda contiene un árbol con densidad d ; una celda ardiendo propaga el fuego a cada vecina verde (cuatro vecinos) con probabilidad p y luego se consume. El proceso es un campo de Markov espacial cuyo interés reside en su *transición de fase*: para densidad $d = 1$, el modelo equivale a percolación de enlaces en la red cuadrada, con umbral crítico exacto $p_c = 1/2$ (Ec. 206). Desde el monitoreo por UAV, los parámetros d (densidad de combustible) y p (probabilidad de propagación, función de viento y humedad) se estiman de la imagerie aérea, y el modelo predice la probabilidad de un gran incendio. Los parámetros se listan en la Tabla 37.

Tabla 37: Parámetros del autómata celular de incendio forestal.

Parámetro	Valor	Significado
Tamaño de retícula L	80 (barrido), 120 (demo)	dominio cuadrado $L \times L$
Densidad de árboles d	1.0	fracción de celdas con combustible
Prob. de propagación p	barrido [0.30, 0.75]	ignición por vecino
Réplicas por valor de p	60	promedios de ensamble
Ignición	celda central	foco inicial único

7.13.2. Resultados y lectura metodológica

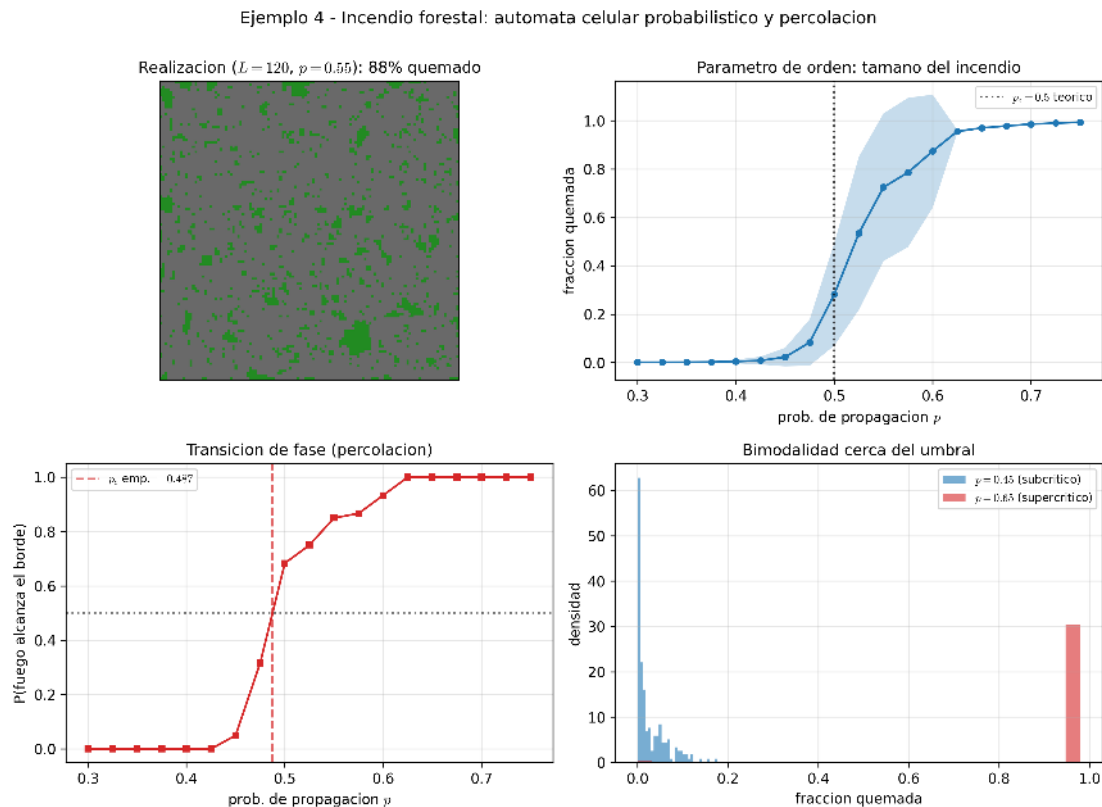


Figura 52: Incendio forestal como autómata celular probabilístico. Panel 1: una realización ($L = 120$, $p = 0.55$) con celdas verdes, quemadas y vacías. Panel 2: fracción quemada media frente a p con banda de dispersión y umbral teórico $p_c = 0.5$ (Ec. 206). Panel 3: probabilidad de que el fuego alcance el borde (parámetro de orden de percolación) con el p_c empírico. Panel 4: bimodalidad de la fracción quemada en regímenes subcrítico y supercrítico. Resultados del Código 4 del Anexo de esta semana.

1. **Existe un umbral crítico, no una transición suave.** La fracción quemada media salta de 0.027 en $p = 0.45$ (subcrítico) a 0.960 en $p = 0.65$ (supercrítico). La

probabilidad de que el fuego cruce el dominio cruza el 50 % en un p_c empírico de 0.4875, en excelente acuerdo con el valor exacto $p_c = 1/2$ de la percolación de enlaces. Una variación pequeña de p alrededor del umbral cambia cualitativamente el desenlace: este es el rasgo que distingue a un sistema crítico de uno lineal.

2. **La gestión del riesgo se concentra en el umbral.** La consecuencia operativa para el monitoreo por UAV es que la prioridad no es estimar p con precisión uniforme, sino determinar de qué lado del umbral p_c se encuentra el bosque bajo las condiciones meteorológicas actuales. Una reducción de combustible (bajar d) o de propagación (cortafuegos, que reduce p efectivo) tiene un retorno desproporcionado cuando empuja al sistema de supercrítico a subcrítico.
3. **La incertidumbre es máxima en la criticidad.** El Panel 4 muestra que lejos del umbral el resultado es casi determinista (fuego local o fuego total), pero cerca de p_c la fracción quemada es altamente variable de una realización a otra. La predicción de un incendio individual es intrínsecamente incierta en el régimen crítico, lo que obliga a razonar en términos de probabilidad de gran incendio y no de un pronóstico puntual.

7.14. Ejemplo 5 — Modelo epidemiológico estocástico: del SIR de Markov al sistema dinámico

7.14.1. Planteamiento

El modelo SIR es a la vez una cadena de Markov de tiempo continuo y, en su límite, un sistema dinámico de ecuaciones diferenciales; por ello sirve de puente conceptual entre esta semana y las Semanas 4–5. Una población de $N = 1000$ individuos se divide en susceptibles (S), infectados (I) y recuperados (R). Dos eventos dirigen la dinámica: contagio $S + I \rightarrow 2I$ con tasa $\beta SI/N$, y recuperación $I \rightarrow R$ con tasa γI . Con $\beta = 0.60$ y $\gamma = 0.25$ se obtiene un número reproductivo básico $R_0 = \beta/\gamma = 2.40$. Se simula el proceso estocástico exacto con Gillespie y se compara con la solución determinista (209). Los parámetros aparecen en la Tabla 38.

Tabla 38: Parámetros del modelo epidemiológico SIR (Ec. 209 y Ec. 210).

Parámetro	Valor	Significado
Población N	1000	tamaño total constante
Infectados iniciales I_0	5	semilla del brote
Tasa de contagio β	0.60 d^{-1}	contactos efectivos
Tasa de recuperación γ	0.25 d^{-1}	inverso del periodo infeccioso
Número reproductivo R_0	2.40	β/γ
Realizaciones Gillespie M	3000	ensamble estocástico

7.14.2. Resultados y lectura metodológica

Ejemplo 5 - SIR estocástico (Gillespie) y su límite determinista

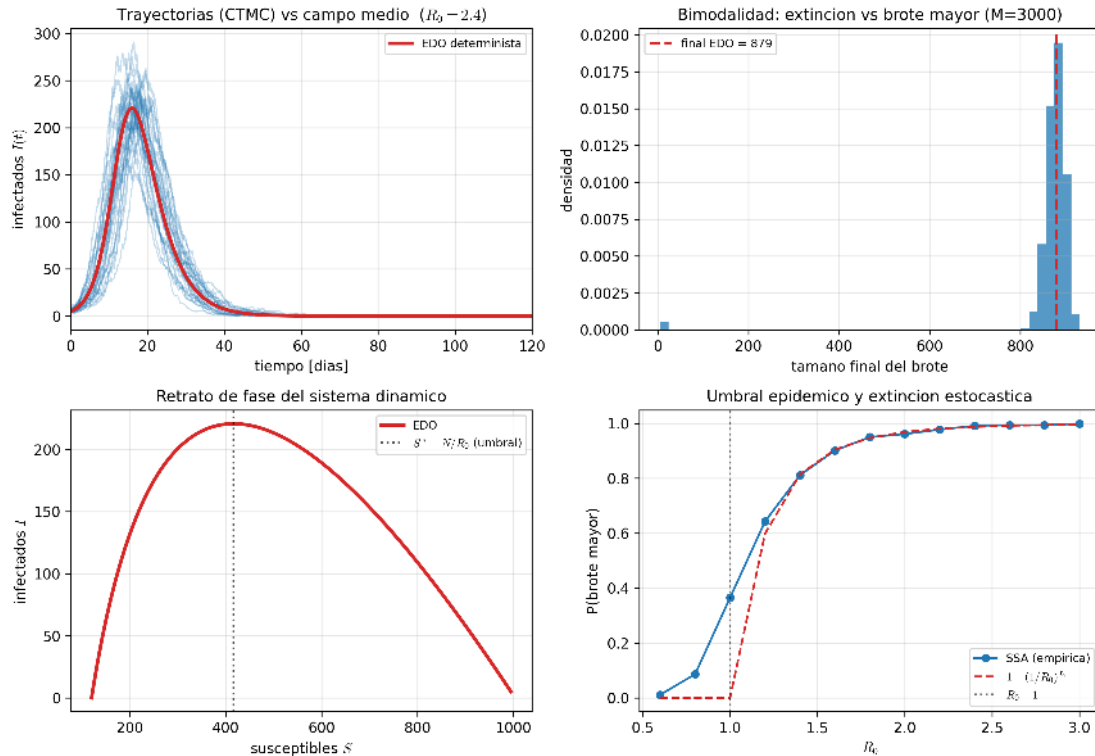


Figura 53: SIR estocástico (Gillespie) y su límite determinista. Panel 1: trayectorias estocásticas $I(t)$ y solución de la EDO (Ec. 209). Panel 2: distribución del tamaño final del brote, bimodal (extinción vs brote mayor). Panel 3: retrato de fase $S-I$ del sistema dinámico con el umbral $S^* = N/R_0$. Panel 4: probabilidad de brote mayor frente a R_0 , comparada con la teoría $1 - (1/R_0)^{I_0}$ (Ec. 211). Resultados del Código 5 del Anexo de esta semana.

1. El determinista describe el brote mayor; el estocástico, también la extinción.

La EDO predice un pico de $I_{\text{máx}} = 220.6$ infectados en $t = 16.0$ días y un tamaño final de 879.5 (87.9 % de la población). El ensamble estocástico, condicionado a brote mayor, da un tamaño final medio de 878.1 (87.8 %): el límite de campo medio es preciso *cuando el brote despegue*. Pero el modelo determinista no puede representar la rama de extinción temprana, que sí aparece en la distribución bimodal del Panel 2.

2. La extinción estocástica es un fenómeno de número pequeño. La probabilidad empírica de brote mayor es 0.990, en acuerdo con la predicción de procesos de ramificación $1 - (1/R_0)^{I_0} = 0.987$ (Ec. 211). Con solo $I_0 = 5$ infectados iniciales y $R_0 = 2.4$, alrededor del 1 % de los brotes se extingue por azar pese a tener $R_0 > 1$. Este efecto sería invisible en un modelo puramente determinista y es decisivo en las fases iniciales de una epidemia o en la introducción de una especie.

3. **El umbral $R_0 = 1$ es una transición de fase dinámica.** El Panel 4 muestra que la probabilidad de brote mayor es prácticamente nula para $R_0 < 1$ y crece de forma abrupta al superar la unidad, replicando en el plano dinámico la misma estructura de transición que la percolación del Ejemplo 4. El umbral $S^* = N/R_0$ del retrato de fase marca el punto donde $\dot{I} = 0$: a la izquierda la epidemia decae, a la derecha crece.
4. **Mismo proceso, dos descripciones complementarias.** El valor pedagógico del ejemplo es mostrar que la EDO (209) no es un modelo distinto del estocástico, sino su límite de campo medio (Ec. 208). Elegir entre uno y otro es una decisión de escala: poblaciones grandes y preguntas promedio favorecen la EDO; poblaciones pequeñas, eventos raros y probabilidad de extinción exigen el proceso de Markov.

7.15. Ejemplo 6 — Ranking por caminata aleatoria (PageRank) para recuperación en RAG

7.15.1. Planteamiento

El sistema RAG (*Retrieval-Augmented Generation*) industrial del proyecto de Rodríguez Morales debe ordenar documentos por relevancia. Cuando los documentos forman un grafo dirigido (por citación, enlace o similitud semántica umbralizada), su importancia puede medirse como la distribución estacionaria de una caminata aleatoria sobre el grafo: el algoritmo PageRank. El navegante aleatorio sigue un enlace saliente con probabilidad α (*damping*) y “teletransporta” a un documento al azar con probabilidad $1 - \alpha$; los nodos sin enlaces salientes (*colgantes*) se tratan como teletransporte total. Esto define la matriz de Google

$$G = \alpha S + (1 - \alpha) \frac{1}{n} \mathbf{1}\mathbf{1}^\top, \quad (215)$$

con S la matriz de transición por filas y $\alpha = 0.85$. El vector PageRank es el vector propio izquierdo de G con autovalor 1 (Perron–Frobenius), calculado por iteración de potencias. Los parámetros aparecen en la Tabla 39.

Tabla 39: Parámetros del modelo PageRank sobre el grafo de documentos (Ec. 215).

Parámetro	Valor	Significado
Número de documentos n	14	nodos del grafo
Damping α	0.85	prob. de seguir un enlace
Nodos colgantes	1	sin enlaces salientes
Tolerancia de convergencia	10^{-15}	criterio de parada

7.15.2. Resultados y lectura metodológica

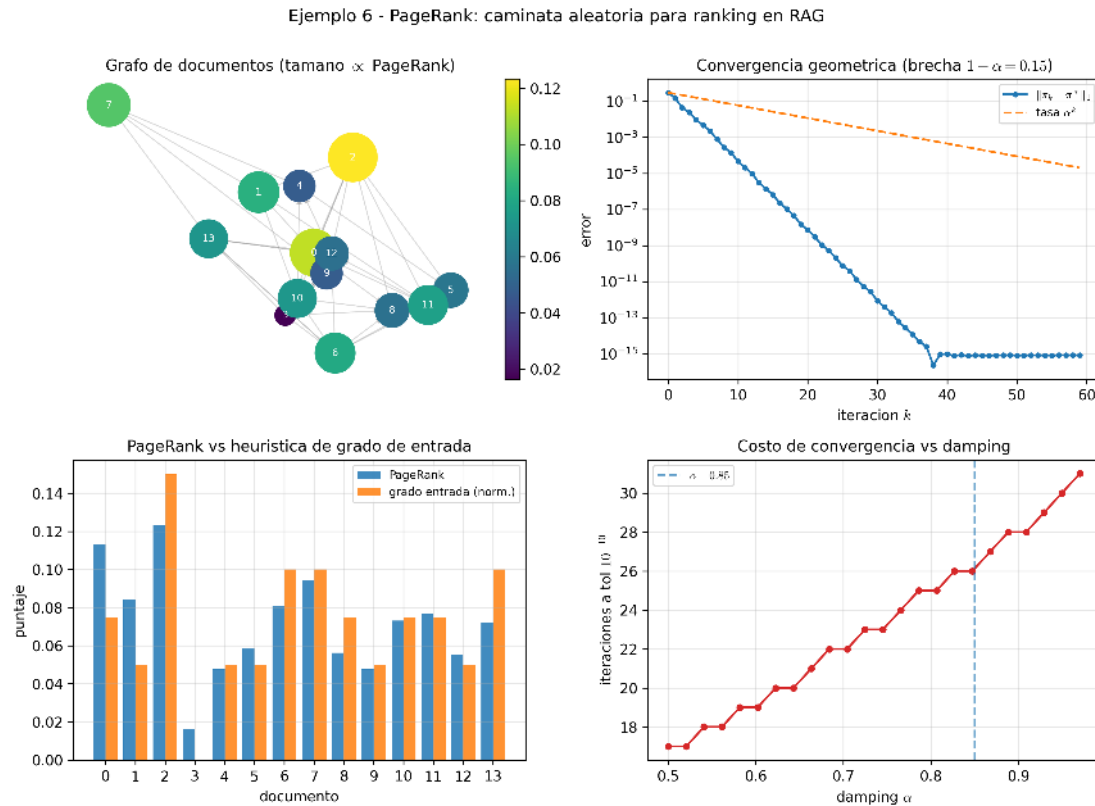


Figura 54: PageRank como caminata aleatoria para ranking en RAG. Panel 1: grafo de documentos con tamaño de nodo proporcional al PageRank. Panel 2: convergencia geométrica de la iteración de potencias y la cota de tasa α^k (Ec. 207). Panel 3: PageRank frente a la heurística de grado de entrada. Panel 4: costo de convergencia (iteraciones) frente al damping α . Resultados del Código 6 del Anexo de esta semana.

1. **PageRank no es el grado de entrada.** El documento mejor rankeado coincide en ambos criterios, pero a partir del segundo lugar difieren: el grado de entrada cuenta enlaces, mientras que PageRank pondera la importancia de quién enlaza. Un documento citado por pocos pero importantes puede superar a otro citado por muchos irrelevantes. Para un RAG, esto significa priorizar documentos centrales en la estructura del conocimiento, no solo populares.
2. **La convergencia está garantizada y es independiente del grafo.** La iteración de potencias alcanza tolerancia 10^{-15} en 40 iteraciones, y reproduce el PageRank de referencia (networkx) con error máximo 5×10^{-7} . La matriz de Google asegura $|\lambda_2| \leq \alpha = 0.85$, lo que acota la tasa de convergencia por α^k (la línea de cota del Panel 2). En este grafo el segundo autovalor observado es aún menor ($|\lambda_2| = 0.41$), de modo que la convergencia real es más rápida que la cota: α^k es una garantía de peor caso, no la tasa exacta.

3. **El damping controla el compromiso estabilidad–costo.** El Panel 4 muestra que aumentar α acerca el ranking a la estructura real de enlaces pero encarece la convergencia (la brecha espectral mínima garantizada es $1 - \alpha$, que se estrecha al crecer α). El valor estándar $\alpha = 0.85$ equilibra fidelidad al grafo y rapidez de cálculo; subirlo a 0.97 multiplica las iteraciones necesarias sin un beneficio proporcional en el ranking.
4. **El teletransporte garantiza un problema bien planteado.** El término $(1 - \alpha)^{\frac{1}{n}} \mathbf{1}\mathbf{1}^T$ vuelve la cadena irreducible y aperiódica incluso con nodos colgantes o componentes desconectadas, asegurando una estacionaria única (existencia y unicidad en el sentido de Hadamard de la Semana 1). Sin él, el ranking podría no estar definido o depender de la condición inicial.

7.16. Síntesis de la semana

Los seis ejemplos recorren el espectro de procesos markovianos, de modo que cada estudiante reconozca a cuál pertenece el fenómeno de su proyecto. La Tabla 40 resume el mapa.

Tabla 40: Mapa metodológico de la semana. Cada ejemplo ilustra un tipo distinto de proceso de Markov y la pregunta de ingeniería que resuelve.

Ej.	Tipo de proceso	Objeto clave	Pregunta resuelta
1	DTMC, 2 estados	estacionaria, recurrencia	¿qué fracción fuera de tolerancia?
2	DTMC con reparación	matriz fundamental, MTTF	¿disponibilidad y vida hasta falla?
3	CTMC, generador Q	$\exp(Qt)$, estacionaria	¿horas/año por estado, MTTF?
4	Autómata celular espacial	umbral p_c , percolación	¿incendio local o global?
5	CTMC $\rightarrow$ EDO (campo medio)	R_0 , extinción	¿brote menor o epidemia?
6	Caminata aleatoria en grafo	Perron–Frobenius, brecha	¿ranking de importancia?

Las cuatro lecciones transversales que el estudiante debe extraer son:

1. **El estado lo es todo.** La propiedad de Markov no es una limitación sino una guía de diseño: definir bien el espacio de estados es el acto modelador central. Si el modelo “recuerda” demasiado, el estado está mal definido; ampliarlo restaura la markovianidad.
2. **Dos preguntas, dos herramientas.** La distribución estacionaria responde “¿qué fracción del tiempo?” (disponibilidad, ocupación, ranking); la matriz fundamental responde “¿cuánto tarda en ocurrir?” (MTTF, tiempo a la absorción). Confundirlas produce errores de criterio, como tomar el MTTF por el MTBF.

3. **Las transiciones de fase son reales y operativas.** Percolación (Ejemplo 4) y umbral epidémico (Ejemplo 5) muestran que muchos sistemas cambian cualitativamente al cruzar un valor crítico. La gestión del riesgo se concentra en determinar de qué lado del umbral está el sistema, no en estimar parámetros lejos de él.
4. **El determinismo es un límite, no una verdad.** Los sistemas dinámicos de las semanas previas son el límite de campo medio de procesos de Markov. Cuando las poblaciones son pequeñas o los eventos raros importan, el azar reaparece y el modelo determinista deja de ser suficiente.

Apéndice Semana 7. Referencias verificables

- A1. Norris, J. R. (1997). *Markov Chains*. Cambridge University Press. <https://doi.org/10.1017/CB09780511810633>
- A2. Grinstead, C. M. y Snell, J. L. (1997). *Introduction to Probability* (2.^a ed.), cap. 11 (Markov Chains). American Mathematical Society. <https://math.dartmouth.edu/~prob/prob/prob.pdf>
- A3. Levin, D. A. y Peres, Y. (2017). *Markov Chains and Mixing Times* (2.^a ed.). American Mathematical Society. <https://doi.org/10.1090/mbk/107>
- A4. Gillespie, D. T. (1977). Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25), 2340–2361. <https://doi.org/10.1021/j100540a008>
- A5. Kurtz, T. G. (1970). Solutions of ordinary differential equations as limits of pure jump Markov processes. *Journal of Applied Probability*, 7(1), 49–58. <https://doi.org/10.2307/3212147>
- A6. Kermack, W. O. y McKendrick, A. G. (1927). A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society A*, 115(772), 700–721. <https://doi.org/10.1098/rspa.1927.0118>
- A7. Allen, L. J. S. (2017). A primer on stochastic epidemic models: Formulation, numerical simulation, and analysis. *Infectious Disease Modelling*, 2(2), 128–142. <https://doi.org/10.1016/j.idm.2017.03.001>
- A8. Stauffer, D. y Aharony, A. (1994). *Introduction to Percolation Theory* (2.^a ed.). Taylor & Francis. <https://doi.org/10.1201/9781315274386>
- A9. Kesten, H. (1980). The critical probability of bond percolation on the square lattice equals 1/2. *Communications in Mathematical Physics*, 74(1), 41–59. <https://doi.org/10.1007/BF01197577>
- A10. Page, L., Brin, S., Motwani, R. y Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Technical Report 1999-66, Stanford InfoLab. <http://ilpubs.stanford.edu:8090/422/>
- A11. Langville, A. N. y Meyer, C. D. (2006). *Google's PageRank and Beyond: The*

Science of Search Engine Rankings. Princeton University Press. <https://doi.org/10.1515/9781400830329>

- A12. Trivedi, K. S. y Bobbio, A. (2017). *Reliability and Availability Engineering: Modeling, Analysis, and Applications*. Cambridge University Press. <https://doi.org/10.1017/9781316163047>

7.17. Anexo Semana 7. Códigos para Google Colab

Los seis códigos siguientes son autocontenidos y reproducibles: cada uno fija una semilla del generador PCG64 (`numpy.random.default_rng`), crea la carpeta Figuras/ y guarda la figura a 300 dpi. Todos los valores numéricos citados en el texto provienen de la ejecución de estos códigos. Para usarlos en Google Colab basta copiarlos en una celda y ejecutarlos; el Código 6 requiere `networkx` (preinstalado en Colab). La figura aparecerá en Figuras/ dentro del entorno.

7.17.1. Código 1 — Deriva de calibración: cadena de Markov de dos estados

Matriz de transición, distribución estacionaria por forma cerrada y por autovector, evolución $\pi_n = \pi_0 P^n$, análisis espectral (brecha y tiempo de mezcla), tiempos de primer paso y estimación de la matriz por máxima verosimilitud a partir de una trayectoria simulada.

```

1      """
2      Semana 7 - Ejemplo 1: Deriva de calibracion de un
3      sensor como cadena de
4      Markov de tiempo discreto (DTMC) de dos estados.
5
6      Estados:
7      0 = "en calibracion"    (dentro de tolerancia
8      metrologica)
9      1 = "fuera de calibracion" (excede el error maximo
10     permisible, EMP)
11
12     El tiempo avanza en pasos de inspeccion (p.ej.
13     semanas). En cada paso el
14     sensor puede derivar (0 -> 1) con probabilidad a, o
15     ser recalibrado/recuperado
16     (1 -> 0) con probabilidad b. La cadena es
17     irreducible y aperiodica, por lo que
18     posee una distribucion estacionaria unica.
19
20     Objetos de calculo:

```

```

15         - Matriz de transicion  $P$  y su potencia  $n$ -esima  $P^n$  (
Chapman-Kolmogorov).
16         - Distribucion estacionaria  $\pi$  ( $\pi P = \pi$ ).
17         - Brecha espectral  $(1 - |\lambda_2|)$  y tiempo de
mezcla.
18         - Tiempo medio de recurrencia y tiempo medio de
primer paso  $0 \rightarrow 1$ 
19         (intervalo optimo de recalibracion).
20         - Interpretacion estadistica: estimacion de  $P$  por
maxima verosimilitud
21         (MLE) a partir de una trayectoria simulada, con su
intervalo de Wald.
22         """
23
24         import numpy as np
25         import matplotlib
26         matplotlib.use('Agg') # backend sin
pantalla (Colab/headless)
27         import matplotlib.pyplot as plt
28         import matplotlib as mpl
29         import os
30
31         mpl.rcParams.update({
32             'font.size': 10, 'axes.titlesize': 11, 'axes.
labelsize': 10,
33             'figure.dpi': 110, 'savefig.dpi': 300, 'axes.
grid': True,
34             'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
35         })
36         os.makedirs('Figuras', exist_ok=True)
37         rng = np.random.default_rng(20260207)
38
39         #
-----
40
41         # 1. Parametros del modelo (probabilidades por paso
de inspeccion)
42         #
-----
43
44         a = 0.08 #  $P(\text{en-cal} \rightarrow \text{fuera-cal})$ : probabilidad
de deriva por semana

```

```

43         b = 0.50          # P(fuera-cal -> en-cal): probabilidad
                             de deteccion+recalibracion
44         P = np.array([[1 - a, a],
45                        [b,      1 - b]])
46         estados = ['en calibracion', 'fuera de calibracion']
47
48         # Verificacion: cada fila es una distribucion de
         # probabilidad
49         assert np.allclose(P.sum(axis=1), 1.0), "Filas de P
         deben sumar 1"
50
51         #
         -----
52
         # 2. Distribucion estacionaria (cerrada para 2
         # estados) y por algebra
53         #
         -----
54
         # Cerrada: pi0 = b/(a+b), pi1 = a/(a+b)
55         pi_closed = np.array([b, a]) / (a + b)
56
57         # Generica: vector propio izquierdo de autovalor 1
58         w, V = np.linalg.eig(P.T)
59         idx = np.argmin(np.abs(w - 1.0))
60         pi_eig = np.real(V[:, idx])
61         pi_eig = pi_eig / pi_eig.sum()
62
63         #
         -----
64
         # 3. Evolucion transitoria pi_n = pi_0 P^n desde
         # estado "en calibracion"
65         #
         -----
66
         n_max = 40
67         pi0 = np.array([1.0, 0.0])          # arranca en
         calibracion con certeza
68         pi_n = np.zeros((n_max + 1, 2))
69         pi_n[0] = pi0
70         Pn = np.eye(2)

```

```

71         for n in range(1, n_max + 1):
72             Pn = Pn @ P
73             pi_n[n] = pi0 @ Pn
74
75             #
76             -----
77             # 4. Analisis espectral: brecha espectral y tiempo
78             de mezcla
79             #
80             -----
81             eigvals = np.sort(np.abs(np.linalg.eigvals(P)))
82             [::-1] # [1, /lambda2/]
83             lam2 = eigvals[1] # = /1 - a -
84             b/
85             gap = 1.0 - lam2 # brecha
86             espectral = a + b
87             # Distancia en variacion total a la estacionaria;
88             cota /lambda2/~n
89             tv = 0.5 * np.abs(pi_n - pi_closed).sum(axis=1)
90             # tiempo de mezcla empirico: primer n con TV < 0.01
91             t_mix = int(np.argmax(tv < 0.01))
92
93             #
94             -----
95
96             # 5. Tiempos caracteristicos
97             #
98             -----
99
100            # Tiempo medio de recurrencia del estado i: m_ii =
101            1/pi_i
102            m_rec = 1.0 / pi_closed
103            # Tiempo medio de primer paso 0 -> 1 (geometrico con
104            prob a): 1/a
105            T_drift = 1.0 / a # semanas
106            esperadas hasta salir de tolerancia
107            # Tiempo medio de primer paso 1 -> 0: 1/b
108            T_recover = 1.0 / b
109
110            #
    
```

```

97         # 6. Interpretacion estadistica: simular trayectoria
          y re-estimar P (MLE)
98         #
-----
99         def simula_cadena(P, n_pasos, s0, rng):
100             s = np.empty(n_pasos + 1, dtype=int)
101             s[0] = s0
102             for t in range(n_pasos):
103                 s[t + 1] = rng.choice(2, p=P[s[t]])
104             return s
105
106         N_sim = 5000
107         traj = simula_cadena(P, N_sim, 0, rng)
108         # conteo de transiciones n_ij
109         C = np.zeros((2, 2))
110         for t in range(N_sim):
111             C[traj[t], traj[t + 1]] += 1
112         P_mle = C / C.sum(axis=1, keepdims=True) #
estimador MLE
113         # error estandar de Wald para p_ij: sqrt(p_ij(1-p_ij
114         )/n_i)
115         n_i = C.sum(axis=1)
116         se_mle = np.sqrt(P_mle * (1 - P_mle) / n_i[:, None])
117         #
-----
118         # 7. Salida de consola (todas las cifras citadas en
          el LaTeX)
119         #
-----
120         print("=== Ejemplo 1: deriva de calibracion (DTMC 2
          estados) ===")
121         print(f"P =\n{P}")
122         print(f"pi (cerrada) = {pi_closed}")
123         print(f"pi (autovector)= {pi_eig}")
124         print(f"Fraccion estacionaria fuera de calibracion =
          {pi_closed[1]*100:.2f} %")

```

```

125         print(f"|lambda_2| = {lam2:.4f}      brecha espectral =
        {gap:.4f}")
126         print(f"Tiempo de mezcla (TV<0.01) = {t_mix} pasos")
127         print(f"Tiempo medio recurrencia [en-cal, fuera-cal]
        = {m_rec} semanas")
128         print(f"Tiempo medio de deriva 0->1 = {T_drift:.2f}
        semanas (intervalo de recalibracion)")
129         print(f"Tiempo medio de recuperacion 1->0 = {
        T_recover:.2f} semanas")
130         print(f"P_MLE (N={N_sim}) =\n{P_mle}")
131         print(f"Error estandar Wald =\n{se_mle}")
132
133         #
        -----
134         # 8. Figura de 4 paneles
135         #
        -----

136         fig, ax = plt.subplots(2, 2, figsize=(11, 8))
137
138         # Panel 1: evolucion de  $\pi_n$  (probabilidad de cada
        estado)
139         ax[0, 0].plot(pi_n[:, 0], 'o-', ms=3, label=r'$\pi_n$
        (\mathrm{en\ cal.})$')
140         ax[0, 0].plot(pi_n[:, 1], 's-', ms=3, label=r'$\pi_n$
        (\mathrm{fuera\ cal.})$')
141         ax[0, 0].axhline(pi_closed[0], ls='--', color='C0',
        alpha=0.6)
142         ax[0, 0].axhline(pi_closed[1], ls='--', color='C1',
        alpha=0.6)
143         ax[0, 0].set_xlabel('paso $n$ (semanas)'); ax[0, 0].
        set_ylabel(r'$\pi_n$')
144         ax[0, 0].set_title(r'Evoluci\on $\pi_n=\pi_0 P^n$
        hacia la estacionaria')
145         ax[0, 0].legend(fontsize=8)
146
147         # Panel 2: convergencia a la estacionaria (TV) en
        escala log + cota  $|l_2|^n$ 
148         n_ax = np.arange(n_max + 1)
149         ax[0, 1].semilogy(n_ax, tv + 1e-16, 'o-', ms=3,
        label='TV empirica')
    
```



```

150         ax[0, 1].semilogy(n_ax, lam2**n_ax, '--', label=r'
cota  $|\lambda_2|^n$ ')
151         ax[0, 1].axhline(0.01, color='k', ls=':', alpha=0.7,
label='umbral 0.01')
152         ax[0, 1].set_xlabel('paso  $n$ '); ax[0, 1].set_ylabel
('dist. variacion total')
153         ax[0, 1].set_title(f'Mezcla geometrica ( $t_{\{mix\}}=t_{mix}$ )')
154         ax[0, 1].legend(fontsize=8)
155
156         # Panel 3: distribucion estacionaria y tiempos
caracteristicos
157         xb = np.arange(2)
158         ax[1, 0].bar(xb, pi_closed, color=['C0', 'C1'],
alpha=0.8)
159         for k, v in enumerate(pi_closed):
160             ax[1, 0].text(k, v + 0.01, f'{v:.3f}', ha='center',
fontsize=9)
161         ax[1, 0].set_xticks(xb); ax[1, 0].set_xticklabels(['
en cal.', 'fuera cal.'])
162         ax[1, 0].set_ylabel(r' $\pi_i$ '); ax[1, 0].set_ylim
(0, 1)
163         ax[1, 0].set_title(f'Estacionaria | recalibrar cada
{T_drift:.0f} sem.')
164
165         # Panel 4: sensibilidad de la fraccion fuera-cal vs
prob. de recalibracion b
166         b_vec = np.linspace(0.05, 0.95, 200)
167         pi1_vec = a / (a + b_vec)
168         ax[1, 1].plot(b_vec, pi1_vec * 100, '-')
169         ax[1, 1].axvline(b, color='C1', ls='--', alpha=0.7,
label=f' $b={b}$ ')
170         ax[1, 1].plot(b, pi_closed[1] * 100, 'o', color='C3'
, ms=7)
171         ax[1, 1].set_xlabel('prob. de recalibracion  $b$ ')
172         ax[1, 1].set_ylabel(r'% tiempo fuera de calibracion'
)
173         ax[1, 1].set_title('Sensibilidad de la
indisponibilidad metrologica')
174         ax[1, 1].legend(fontsize=8)
175
176         fig.suptitle('Ejemplo 1 - Deriva de calibracion como

```

```

177     cadena de Markov de 2 estados',
178         fontsize=12, y=1.00)
179     fig.tight_layout()
180     fig.savefig('Figuras/fig_s7_ej1_sensor_markov.png',
181         bbox_inches='tight')
182     # plt.show()
183     print("\nFigura guardada: Figuras/
184     fig_s7_ej1_sensor_markov.png")

```

Listing 43: Semana 7 – Código 1: Deriva de calibración (DTMC 2 estados). Ejecutar en Google Colab.

7.17.2. Código 2 — Confiabilidad de un banco de pruebas: DTMC con reparación

Distribución estacionaria y disponibilidad, matriz fundamental $N = (I - Q)^{-1}$ para el MTTF, validación Monte Carlo del tiempo a la primera falla, MTBF de régimen permanente y sensibilidad a la reparación.

```

1      """
2      Semana 7 - Ejemplo 2: Confiabilidad y disponibilidad
3      de un banco de pruebas de
4      bomba centrifuga como cadena de Markov de tiempo
5      discreto de tres estados con
6      reparacion.
7
8      Estados (paso = jornada de operacion, 1 dia):
9      O = operativo (nominal)
10     D = degradado (vibracion/cavitacion incipiente; aun
11     funcional)
12     F = falla      (fuera de servicio)
13
14     Conceptos:
15     - Distribucion estacionaria pi -> disponibilidad de
16     regimen permanente.
17     - Cadena absorbente: tiempo medio hasta la primera
18     falla (MTTF) por medio
19     de la matriz fundamental N = (I - Q)^{-1}, con Q el
20     bloque transitorio.
21     - Validacion Monte Carlo del MTTF (histograma del
22     tiempo a la absorcion).
23     - Sensibilidad de la disponibilidad frente a la tasa
24     de reparacion.

```

```

17         """
18
19         import numpy as np
20         import matplotlib
21         matplotlib.use('Agg')
22         import matplotlib.pyplot as plt
23         import matplotlib as mpl
24         import os
25
26         mpl.rcParams.update({
27             'font.size': 10, 'axes.titlesize': 11, 'axes.
28             labels.size': 10,
29             'figure.dpi': 110, 'savefig.dpi': 300, 'axes.
30             grid': True,
31             'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
32             })
33         os.makedirs('Figuras', exist_ok=True)
34         rng = np.random.default_rng(20260208)
35
36         #
37
38         -----
39
40         # 1. Matriz de transicion (orden de estados: O, D, F
41         )
42
43         #
44
45         -----
46
47         #           ->O      ->D      ->F
48         P = np.array([
49             [0.95, 0.04, 0.01], # desde O: se mantiene, se
50             degrada o falla subito
51             [0.30, 0.65, 0.05], # desde D: se repara a O,
52             sigue degradado o falla
53             [0.50, 0.00, 0.50], # desde F: se repara a O o
54             sigue en falla
55         ])
56         nombres = ['Operativo', 'Degradado', 'Falla']
57         assert np.allclose(P.sum(axis=1), 1.0)
58
59         #
60
61         -----

```

```

47         # 2. Distribucion estacionaria -> disponibilidad
48         #
-----
49         w, V = np.linalg.eig(P.T)
50         pi = np.real(V[:, np.argmin(np.abs(w - 1.0))])
51         pi = pi / pi.sum()
52         A_full = pi[0]                                # disponibilidad
53         estricta (solo 0)
54         A_func = pi[0] + pi[1]                        # disponibilidad
55         funcional (0 o D)
56         #
-----
57         # 3. Cadena absorbente para el MTTF: F como estado
58         absorbente
59         # Transitorios T = {0, D}; bloque Q (T->T);
60         matriz fundamental N=(I-Q)^-1
61         # Tiempo esperado a la absorcion: t = N 1
62         #
-----
63         Q = P[np.ix_([0, 1], [0, 1])]                # bloque entre
64         estados transitorios
65         I2 = np.eye(2)
66         N = np.linalg.inv(I2 - Q)
67         t_abs = N @ np.ones(2)                        # [desde 0,
68         desde D]
69         MTTF_0 = t_abs[0]                             # dias
70         esperados hasta primera falla desde 0
71         #
-----
72         # Tasa de falla de regimen y MTBF aproximado (
73         frecuencia de entradas a F)
74         # flujo estacionario hacia F = sum_i pi_i P[i,F]
75         flujo_F = pi @ P[:, 2]
76         MTBF = 1.0 / flujo_F                          # dias entre
77         fallas en regimen permanente
78         #
-----

```

```

72         # 4. Evolucion transitoria desde 0 (probabilidad de
           cada estado)
73         #
           -----

74         n_max = 60
75         prob = np.zeros((n_max + 1, 3))
76         v = np.array([1.0, 0.0, 0.0])
77         prob[0] = v
78         for n in range(1, n_max + 1):
79             v = v @ P
80             prob[n] = v
81
82         #
           -----

83         # 5. Validacion Monte Carlo del MTTF (tiempo a la
           absorcion desde 0)
84         #
           -----

85         def tiempo_a_falla(P, rng, s0=0, estado_falla=2,
           t_max=100000):
86             s, t = s0, 0
87             while s != estado_falla and t < t_max:
88                 s = rng.choice(3, p=P[s])
89                 t += 1
90             return t
91
92         M = 20000
93         ttf = np.array([tiempo_a_falla(P, rng) for _ in
           range(M)])
94         MTTF_mc = ttf.mean()
95         MTTF_se = ttf.std(ddof=1) / np.sqrt(M)
96
97         #
           -----

98         # 6. Sensibilidad: disponibilidad vs probabilidad de
           reparacion D->0
99         #
           -----

```

```

100     def disponibilidad(p_D0):
101         Pm = P.copy()
102         # redistribuir fila D manteniendo P[D,F]=0.05 fijo
103         Pm[1, 0] = p_D0
104         Pm[1, 1] = 1.0 - p_D0 - 0.05
105         if Pm[1, 1] < 0:
106             return np.nan
107         w, V = np.linalg.eig(Pm.T)
108         p = np.real(V[:, np.argmin(np.abs(w - 1.0))]); p = p
/ p.sum()
109         return p[0] + p[1]
110         pD0_vec = np.linspace(0.05, 0.90, 200)
111         A_vec = np.array([disponibilidad(x) for x in pD0_vec
]])
112
113         #
-----
114         # 7. Salida de consola
115         #
-----

116         print("=== Ejemplo 2: confiabilidad banco/bomba (
DTMC 3 estados) ===")
117         print(f"pi (0,D,F) = {pi}")
118         print(f"Disponibilidad estricta (solo 0) = {A_full
*100:.2f} %")
119         print(f"Disponibilidad funcional (0 o D) = {A_func
*100:.2f} %")
120         print(f"Indisponibilidad (F) = {pi
[2]*100:.2f} %")
121         print(f"MTTF desde 0 (matriz fundamental)= {MTTF_0
:.2f} dias")
122         print(f"MTTF desde 0 (Monte Carlo M={M}) = {MTTF_mc
:.2f} +/- {MTTF_se:.2f} dias")
123         print(f"MTBF de regimen permanente = {MTBF:.2f
} dias")
124         print(f"Matriz fundamental N =\n{N}")
125
126         #
-----

```

```

127         # 8. Figura de 4 paneles
128         #
-----
129         fig, ax = plt.subplots(2, 2, figsize=(11, 8))
130         col = ['C2', 'C1', 'C3']
131
132         # Panel 1: evolucion transitoria de probabilidades
133         for k in range(3):
134             ax[0, 0].plot(prob[:, k], color=col[k], label=
nombres[k])
135             ax[0, 0].axhline(pi[k], color=col[k], ls='--', alpha
=0.5)
136             ax[0, 0].set_xlabel('jornada $n$'); ax[0, 0].
set_ylabel('probabilidad')
137             ax[0, 0].set_title('Probabilidad de estado partiendo
de Operativo')
138             ax[0, 0].legend(fontsize=8)
139
140         # Panel 2: distribucion estacionaria +
disponibilidad
141         xb = np.arange(3)
142         ax[0, 1].bar(xb, pi, color=col, alpha=0.85)
143         for k, vv in enumerate(pi):
144             ax[0, 1].text(k, vv + 0.01, f'{vv:.3f}', ha='center'
, fontsize=9)
145             ax[0, 1].set_xticks(xb); ax[0, 1].set_xticklabels(
nombres, fontsize=8)
146             ax[0, 1].set_ylabel(r'$\pi_i$')
147             ax[0, 1].set_title(f'Estacionaria | Disp. funcional
= {A_func*100:.1f}%')
148
149         # Panel 3: histograma Monte Carlo del tiempo a la
primera falla
150         ax[1, 0].hist(ttf, bins=60, density=True, color='C0'
, alpha=0.7,
151             label=f'MC ($M=\{M\}$)')
152         ax[1, 0].axvline(MTTF_0, color='C3', ls='--', lw=2,
label=f'MTTF teorico = {MTTF_0:.0f} d')
153         ax[1, 0].set_xlabel('dias hasta primera falla'); ax
[1, 0].set_ylabel('densidad')

```

```

155         ax[1, 0].set_title('Tiempo a la absorcion (cola
exponencial)')
156         ax[1, 0].legend(fontsize=8)
157
158         # Panel 4: sensibilidad de disponibilidad vs
reparacion D->O
159         ax[1, 1].plot(pD0_vec, A_vec * 100, '-')
160         ax[1, 1].axvline(P[1, 0], color='C1', ls='--', alpha
=0.7,
161         label=f'actual $P_{{D0}}={P[1,0]}$')
162         ax[1, 1].plot(P[1, 0], A_func * 100, 'o', color='C3'
, ms=7)
163         ax[1, 1].set_xlabel(r'prob. de reparacion $P_{{D\to O}}$')
164         ax[1, 1].set_ylabel('disponibilidad funcional [%]')
165         ax[1, 1].set_title('Sensibilidad de la
disponibilidad')
166         ax[1, 1].legend(fontsize=8)
167
168         fig.suptitle('Ejemplo 2 - Confiabilidad de un banco
de pruebas: cadena de Markov con reparacion',
169         fontsize=12, y=1.00)
170         fig.tight_layout()
171         fig.savefig('Figuras/fig_s7_ej2_bomba_confiabilidad.
png', bbox_inches='tight')
172         # plt.show()
173         print("\nFigura guardada: Figuras/
fig_s7_ej2_bomba_confiabilidad.png")
174

```

Listing 44: Semana 7 – Código 2: Confiabilidad de un banco de pruebas (DTMC con reparación). Ejecutar en Google Colab.

7.17.3. Código 3 — Confiabilidad de un OPGW: cadena de tiempo continuo

Construcción de la matriz generadora Q , distribución estacionaria por $\pi Q = \mathbf{0}$, transitorio por exponencial matricial $\exp(Qt)$, MTTF por el subgenerador transitorio y horas/año por estado.

```

1         """
2         Semana 7 - Ejemplo 3: Confiabilidad de un cable OPGW
como cadena de Markov de
3         tiempo continuo (CTMC) con matriz generadora Q.

```


A diferencia del Ejemplo 2 (tiempo discreto), aquí el tiempo es continuo y las

transiciones se rigen por TASAS (eventos por año).

Estados:

N = normal

T = sobrecarga termica (corriente elevada; reversible por enfriamiento)

H = hielo / galloping (sobrecarga mecanica; reversible)

F = falla (rotura/cortocircuito; requiere reparacion)

Formalismo:

- Matriz generadora Q (filas suman 0); q_{ij} = tasa $i \rightarrow j$ ($i \neq j$),

$q_{ii} = -\sum_{j \neq i} q_{ij}$.

- Distribucion estacionaria: $\pi Q = 0$, $\sum(\pi) = 1$.

- Probabilidades transitorias: $P(t) = \exp(Q t)$ (exponencial matricial).

- MTTF desde N por matriz fundamental del subgenerador transitorio:

$m = -(Q_{-T})^{-1} 1$, con Q_{-T} el bloque entre estados no absorbentes.

- Horas/año esperadas en cada estado = $8760 * \pi$.

"""

```
import numpy as np
from scipy.linalg import expm
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib as mpl
import os
```

```
mpl.rcParams.update({
    'font.size': 10, 'axes.titlesize': 11, 'axes.
    labelsizsize': 10,
    'figure.dpi': 110, 'savefig.dpi': 300, 'axes.
    grid': True,
    'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
})
```

```

35         os.makedirs('Figuras', exist_ok=True)
36
37         #
38         -----
39         # 1. Tasas de transicion (1/ano). Orden de estados:
40         N, T, H, F
41         #
42         -----
43
44         lam_NT, mu_TN, lam_TF = 2.0, 50.0, 0.5      # normal
45         <->termica, termica->falla
46         lam_NH, mu_HN, lam_HF = 1.0, 20.0, 0.3      # normal
47         <->hielo,      hielo->falla
48         mu_FN = 12.0                                #
49         reparacion falla->normal (MTTR ~ 1 mes)
50
51         Q = np.zeros((4, 4))
52         # fila N
53         Q[0, 1] = lam_NT; Q[0, 2] = lam_NH
54         # fila T
55         Q[1, 0] = mu_TN; Q[1, 3] = lam_TF
56         # fila H
57         Q[2, 0] = mu_HN; Q[2, 3] = lam_HF
58         # fila F
59         Q[3, 0] = mu_FN
60         # diagonal
61         for i in range(4):
62             Q[i, i] = -Q[i].sum()
63         nombres = ['Normal', 'Termica', 'Hielo', 'Falla']
64         assert np.allclose(Q.sum(axis=1), 0.0)
65
66         #
67         -----
68
69         # 2. Distribucion estacionaria:  $\pi Q = 0$  (sistema [
70          $Q^T$ ;  $1^T$ ]  $\pi = [0;1]$ )
71         #
72         -----
73
74         Aeq = np.vstack([Q.T, np.ones(4)])
75         beq = np.array([0, 0, 0, 0, 1.0])
    
```

```

64         pi, *_ = np.linalg.lstsq(Aeq, beq, rcond=None)
65         A_disp = pi[0] + pi[1] + pi[2]          # disponibilidad
        = todo menos F
66         U_indisp = pi[3]                        #
        indisponibilidad
67
68         #
        -----
69         # 3. Probabilidades transitorias  $P(t) = \exp(Q t)$ 
        desde N
70         #
        -----
71         t_grid = np.linspace(0, 0.5, 300)        # anos
        (~6 meses)
72         P0 = np.array([1.0, 0, 0, 0])
73         Pt = np.array([P0 @ expm(Q * t) for t in t_grid])
74
75         #
        -----
76         # 4. MTTF desde N: F absorbente. Subgenerador
        transitorio  $Q_T$  sobre  $\{N, T, H\}$ 
77         #
        -----
78         idxT = [0, 1, 2]
79         Q_T = Q[np.ix_(idxT, idxT)].copy()
80         # en la cadena absorbente, F no devuelve flujo: las
        tasas  $i \rightarrow F$  se quedan como
81         # fuga (ya estan en la diagonal de  $Q_T$  porque  $q_{ii}$ 
        incluye  $\lambda_{iF}$ )
82         m_abs = -np.linalg.solve(Q_T, np.ones(3)) # tiempos
        medios a la absorcion
83         MTTF_N = m_abs[0]                        # anos
        hasta primera falla desde N
84
85         #
        -----
86         # 5. Horas por ano en cada estado

```

```

87      #
-----
88      horas = pi * 8760.0
89
90      #
-----
91      # 6. Sensibilidad: indisponibilidad vs tasa de
reparacion mu_FN (MTTR)
92      #
-----
93      def indisp(muFN):
94          Qm = Q.copy()
95          Qm[3, 0] = muFN
96          Qm[3, 3] = -muFN
97          Am = np.vstack([Qm.T, np.ones(4)])
98          p, *_ = np.linalg.lstsq(Am, beq, rcond=None)
99          return p[3]
100      muFN_vec = np.linspace(4, 60, 200)          # 1/ano
(MTTR de 3 meses a 6 dias)
101      U_vec = np.array([indisp(x) for x in muFN_vec])
102
103      #
-----
104      # 7. Salida de consola
105      #
-----
106      print("=== Ejemplo 3: OPGW como CTMC (matriz
generadora) ===")
107      print(f"Q (1/ano) =\n{Q}")
108      print(f"pi (N,T,H,F) = {pi}")
109      print(f"Disponibilidad = {A_disp*100:.4f} %")
110      print(f"Indisponibilidad (F) = {U_indisp*100:.4f} %
-> {U_indisp*8760:.2f} h/ano")
111      print(f"Horas/ano por estado [N,T,H,F] = {horas}")
112      print(f"MTTF desde N = {MTTF_N:.3f} anos ({MTTF_N
*12:.2f} meses)")
113      print(f"Fraccion de tiempo en sobrecarga termica = {

```

```

114     pi[1]*100:.3f} % ({horas[1]:.1f} h/ano)")
115
116     #
117     -----
118
119     # 8. Figura de 4 paneles
120     #
121     -----
122
123     fig, ax = plt.subplots(2, 2, figsize=(11, 8))
124     col = ['C2', 'C1', 'C0', 'C3']
125
126     # Panel 1: P(t) transitorio desde N
127     for k in range(4):
128         ax[0, 0].plot(t_grid * 12, Pt[:, k], color=col[k],
129             label=nombres[k])
130         ax[0, 0].axhline(pi[k], color=col[k], ls='--', alpha
131             =0.4)
132         ax[0, 0].set_xlabel('tiempo [meses]'); ax[0, 0].
133         set_ylabel('probabilidad')
134         ax[0, 0].set_title(r'Transitorio $P(t)=\exp(Qt)$
135             desde Normal')
136         ax[0, 0].legend(fontsize=8); ax[0, 0].set_yscale('
137             log'); ax[0, 0].set_ylim(1e-4, 1.2)
138
139     # Panel 2: distribucion estacionaria
140     xb = np.arange(4)
141     ax[0, 1].bar(xb, pi, color=col, alpha=0.85)
142     for k, vv in enumerate(pi):
143         ax[0, 1].text(k, vv * 1.15 + 1e-4, f'{vv:.4f}', ha='
144             center', fontsize=8)
145         ax[0, 1].set_xticks(xb); ax[0, 1].set_xticklabels(
146             nombres, fontsize=8)
147         ax[0, 1].set_ylabel(r'$\pi_i$'); ax[0, 1].set_yscale
148             ('log')
149         ax[0, 1].set_title(f'Estacionaria | Disponibilidad =
150             {A_disp*100:.3f}%')
151
152     # Panel 3: horas por ano en cada estado
153     ax[1, 0].bar(xb, horas, color=col, alpha=0.85)
154     for k, vv in enumerate(horas):
155         ax[1, 0].text(k, vv * 1.15 + 1, f'{vv:.0f} h', ha='

```

```

center', fontsize=8)
142         ax[1, 0].set_xticks(xb); ax[1, 0].set_xticklabels(
nombres, fontsize=8)
143         ax[1, 0].set_ylabel('horas/ano'); ax[1, 0].
set_yscale('log')
144         ax[1, 0].set_title('Tiempo esperado por estado (8760
h/ano)')
145
146         # Panel 4: indisponibilidad vs tasa de reparacion
147         ax[1, 1].plot(muFN_vec, U_vec * 8760, '-')
148         ax[1, 1].axvline(mu_FN, color='C3', ls='--', alpha
=0.7, label=f'$\mu_{\{FN\}}=\{\mu_{FN}\}/ano')
149         ax[1, 1].plot(mu_FN, U_indisp * 8760, 'o', color='C3
', ms=7)
150         ax[1, 1].set_xlabel(r'tasa de reparacion $\mu_{\{F\}to
N}\$ [1/ano]')
151         ax[1, 1].set_ylabel('indisponibilidad [h/ano]')
152         ax[1, 1].set_title('Sensibilidad a la rapidez de
reparacion')
153         ax[1, 1].legend(fontsize=8)
154
155         fig.suptitle('Ejemplo 3 - Confiabilidad de un OPGW:
cadena de Markov de tiempo continuo',
156                     fontsize=12, y=1.00)
157         fig.tight_layout()
158         fig.savefig('Figuras/fig_s7_ej3_opgw_ctmc.png',
bbox_inches='tight')
159         # plt.show()
160         print("\nFigura guardada: Figuras/
fig_s7_ej3_opgw_ctmc.png")
161

```

Listing 45: Semana 7 – Código 3: Confiabilidad de un OPGW (CTMC, matriz generadora). Ejecutar en Google Colab.

7.17.4. Código 4 — Incendio forestal: autómatas celulares probabilísticos

Simulación de la propagación por frontera (BFS estocástico), barrido en la probabilidad de propagación p , estimación empírica del umbral de percolación p_c y caracterización de la bimodalidad cerca de la criticidad.

```
"""
```

Semana 7 - Ejemplo 4: Propagacion de incendio forestal como automata celular

probabilistico (campo de Markov espacial) y su transicion de percolacion.

Modelo (variante estocastica tipo SIR sobre reticula):

- Reticula $L \times L$ de celdas; cada celda con arbol con densidad d .*

- Una celda "ardiendo" propaga el fuego a cada vecino verde (4-vecindad)*

- con probabilidad p , e inmediatamente pasa a "quemada" (no reignita).*

- El proceso es un campo de Markov: el estado futuro de una celda depende*

- solo de su estado y el de sus vecinos inmediatos.*

Para $d=1$ este proceso es equivalente a percolacion de enlaces en la red

cuadrada, con umbral critico $p_c = 0.5$. Barriendo p alrededor de p_c se observa

la transicion de fase: aparicion subita de un incendio que cruza el dominio.

Anclaje (Garcia Djeda, monitoreo por UAV): a partir de imageneria aerea se

estiman densidad d y probabilidad de propagacion p ; el modelo predice la

probabilidad de un "gran incendio" y la fraccion quemada esperada.

"""

```
import numpy as np
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib as mpl
```

```
from matplotlib.colors import ListedColormap
```

```
import os
```

```
mpl.rcParams.update({
```

```
    'font.size': 10, 'axes.titlesize': 11, 'axes.
```

```

labelsizelabelsize': 10,
31         'figure.dpi': 110, 'savefig.dpi': 300, 'mathtext
.fontset': 'cm',
32     })
33     os.makedirs('Figuras', exist_ok=True)
34     rng = np.random.default_rng(20260209)
35
36     # Codigo de estados de celda
37     VACIO, VERDE, ARDIENDO, QUEMADA = 0, 1, 2, 3
38
39     def simula_incendio(L, d, p, rng, ignicion='centro')
:
40         """Una realizacion del automata. Devuelve la malla
final y si el fuego
41         alcanzo el borde (percolacion). Propagacion por
frontera (BFS estocastico)."""
42         bosque = (rng.random((L, L)) < d).astype(np.int8)
# 1=arbol, 0=vacio
43         estado = np.where(bosque == 1, VERDE, VACIO).astype(
np.int8)
44         if ignicion == 'centro':
45             focos = [(L // 2, L // 2)]
46         else: # borde superior completo
47             focos = [(0, j) for j in range(L) if estado[0, j] ==
VERDE]
48         frontera = []
49         for (i, j) in focos:
50             if estado[i, j] == VERDE:
51                 estado[i, j] = ARDIENDO
52                 frontera.append((i, j))
53                 alcanzo_borde = False
54                 while frontera:
55                     nueva = []
56                     for (i, j) in frontera:
57                         for di, dj in ((1, 0), (-1, 0), (0, 1), (0, -1)):
58                             ni, nj = i + di, j + dj
59                             if 0 <= ni < L and 0 <= nj < L and estado[ni, nj] ==
VERDE:
60                                 if rng.random() < p: # ignicion
estocastica
61                                     estado[ni, nj] = ARDIENDO
62                                     nueva.append((ni, nj))

```



```

63         if ni == 0 or ni == L - 1 or nj == 0 or nj == L - 1:
64             alcanzo_borde = True
65             estado[i, j] = QUEMADA
66             frontera = nueva
67             n_arboles = (bosque == 1).sum()
68             n_quemadas = (estado == QUEMADA).sum()
69             frac_quemada = n_quemadas / max(n_arboles, 1)
70             return estado, frac_quemada, alcanzo_borde
71
72         #
73         -----
74         # 1. Una realizacion ilustrativa (p moderada,
75         # densidad alta)
76         #
77         -----
78         L_demo, d_demo, p_demo = 120, 1.0, 0.55
79         estado_demo, frac_demo, _ = simula_incendio(L_demo,
80         d_demo, p_demo, rng)
81         #
82         -----
83         # 2. Barrido en p: fraccion quemada y probabilidad
84         # de percolacion
85         #
86         -----
87         L, d, R = 80, 1.0, 60                                # tamaño,
88         densidad, replicas por p
89         p_vals = np.linspace(0.30, 0.75, 19)
90         frac_mean = np.zeros_like(p_vals)
91         frac_std = np.zeros_like(p_vals)
92         prob_perc = np.zeros_like(p_vals)
93         frac_pc_samples = None
94         for k, p in enumerate(p_vals):
95             fr = np.empty(R); pc = np.empty(R)
96             for r in range(R):
97                 _, f, b = simula_incendio(L, d, p, rng, ignicion='
centro')
98             fr[r] = f; pc[r] = b

```

```

92         frac_mean[k] = fr.mean(); frac_std[k] = fr.std()
93         prob_perc[k] = pc.mean()
94
95         # estimacion empirica de p_c: p donde P(percola)
cruza 0.5
96         i_cross = np.argmax(prob_perc >= 0.5)
97         if 0 < i_cross < len(p_vals):
98             x0, x1 = p_vals[i_cross - 1], p_vals[i_cross]
99             y0, y1 = prob_perc[i_cross - 1], prob_perc[i_cross]
100             p_c_emp = x0 + (0.5 - y0) * (x1 - x0) / (y1 - y0)
101         else:
102             p_c_emp = p_vals[i_cross]
103
104         # distribuciones de fraccion quemada en sub/super-
critico
105         def muestras_frac(p, R2=200):
106             return np.array([simula_incendio(L, d, p, rng)[1]
107                             for _ in range(R2)])
108         frac_sub = muestras_frac(0.45)
109         frac_sup = muestras_frac(0.65)
110
111         #
112         -----
113
114         # 3. Salida de consola
115         #
116         -----
117
118         print("=== Ejemplo 4: incendio forestal (automata
119         celular probabilistico) ===")
120         print(f"Realizacion demo: L={L_demo}, d={d_demo}, p
121         ={p_demo} -> fraccion quemada={frac_demo:.3f}")
122         print(f"Barrido: L={L}, d={d}, R={R} replicas/p")
123         print(f"p_c teorico (percolacion de enlaces, red
124         cuadrada) = 0.5")
125         print(f"p_c empirico (cruce P_perc=0.5)
126         = {p_c_emp:.4f}")
127         print(f"Fraccion quemada media en p=0.45 (subcritico
128         ) = {frac_sub.mean():.4f}")
129         print(f"Fraccion quemada media en p=0.65 (
130         supercritico)= {frac_sup.mean():.4f}")

```

```

121      #
-----
122      # 4. Figura de 4 paneles
123      #
-----

124      fig, ax = plt.subplots(2, 2, figsize=(11, 8))
125
126      # Panel 1: realizacion (mapa de estados)
127      cmap = ListedColormap(['white', 'forestgreen', '
128      orangered', 'dimgray'])
129      ax[0, 0].imshow(estado_demo, cmap=cmap, vmin=0, vmax
130      =3, interpolation='nearest')
131      ax[0, 0].set_title(f'Realizacion ($L={L_demo}$, $p={
132      p_demo}$): '
133      f'{frac_demo*100:.0f}% quemado')
134      ax[0, 0].set_xticks([]); ax[0, 0].set_yticks([])
135
136      # Panel 2: fraccion quemada vs p (con banda)
137      ax[0, 1].plot(p_vals, frac_mean, 'o-', ms=4)
138      ax[0, 1].fill_between(p_vals, frac_mean - frac_std,
139      frac_mean + frac_std,
140      alpha=0.25)
141      ax[0, 1].axvline(0.5, color='k', ls=':', alpha=0.7,
142      label=r'$p_c=0.5$ teorico')
143      ax[0, 1].set_xlabel('prob. de propagacion $p$')
144      ax[0, 1].set_ylabel('fraccion quemada')
145      ax[0, 1].set_title('Parametro de orden: tamano del
146      incendio')
147      ax[0, 1].grid(alpha=0.3); ax[0, 1].legend(fontsize
148      =8)

149      # Panel 3: probabilidad de percolacion (cruzar el
150      dominio)
151      ax[1, 0].plot(p_vals, prob_perc, 's-', ms=4, color='
152      C3')
153      ax[1, 0].axvline(p_c_emp, color='C3', ls='--', alpha
154      =0.7,
155      label=f'$p_c$ emp. $={p_c_emp:.3f}$')
156      ax[1, 0].axhline(0.5, color='k', ls=':', alpha=0.5)
157      ax[1, 0].set_xlabel('prob. de propagacion $p$')

```

```

149         ax[1, 0].set_ylabel('P(fuego alcanza el borde)')
150         ax[1, 0].set_title('Transicion de fase (percolacion)
151     ')
152         ax[1, 0].grid(alpha=0.3); ax[1, 0].legend(fontsize
153     =8)
154
155         # Panel 4: distribucion de fraccion quemada sub/
156         super critico
157         ax[1, 1].hist(frac_sub, bins=30, density=True, alpha
158     =0.6,
159         color='C0', label='$p=0.45$ (subcritico)')
160         ax[1, 1].hist(frac_sup, bins=30, density=True, alpha
161     =0.6,
162         color='C3', label='$p=0.65$ (supercritico)')
163         ax[1, 1].set_xlabel('fraccion quemada'); ax[1, 1].
164         set_ylabel('densidad')
165         ax[1, 1].set_title('Bimodalidad cerca del umbral')
166         ax[1, 1].grid(alpha=0.3); ax[1, 1].legend(fontsize
167     =8)
168
169         fig.suptitle('Ejemplo 4 - Incendio forestal:
170         automata celular probabilistico y percolacion',
171         fontsize=12, y=1.00)
172         fig.tight_layout()
173         fig.savefig('Figuras/fig_s7_ej4_incendio_percolacion
174         .png', bbox_inches='tight')
175         # plt.show()
176         print("\nFigura guardada: Figuras/
177         fig_s7_ej4_incendio_percolacion.png")

```

Listing 46: Semana 7 – Código 4: Incendio forestal (autómata celular y percolación). Ejecutar en Google Colab.

7.17.5. Código 5 — SIR estocástico: Gillespie y límite determinista

Algoritmo de Gillespie para el SIR estocástico, integración de la EDO determinista, distribución del tamaño final, probabilidad de brote mayor frente a la teoría de ramificación y barrido en R_0 .

```

1         """
2         Semana 7 - Ejemplo 5: Modelo epidemiologico SIR como
         cadena de Markov de tiempo

```

```

3      continuo (CTMC) y su limite determinista (sistema
dinamico de EDO).
4
5      Estados del sistema: (S, I, R) con  $S+I+R = N$ 
constante.
6      Eventos (reacciones) y sus tasas (propensiones):
7      - Infeccion:  $S + I \rightarrow 2I$  con tasa  $\beta * S * I /$ 
N
8      - Recuperacion:  $I \rightarrow R$  con tasa  $\gamma * I$ 
9      Numero reproductivo basico:  $R_0 = \beta / \gamma$ .
10
11     Se comparan dos descripciones del mismo proceso:
12     1. Estocastica exacta (algoritmo de Gillespie / SSA)
: captura extincion
13     temprana, variabilidad y tamano final aleatorio.
14     2. Determinista (EDO de campo medio): limite  $N \rightarrow$ 
inf; es el "sistema
15     dinamico" continuo subyacente.
16
17     Resultados clave:
18     - Probabilidad de brote mayor vs menor (extincion
estocastica).
19     - Distribucion del tamano final (bimodal para  $R_0 >$ 
1).
20     - Comparacion de la prob. de brote mayor con la
teoria 1 -  $(1/R_0)^{I_0}$ .
21     ""
22
23     import numpy as np
24     from scipy.integrate import solve_ivp
25     import matplotlib
26     matplotlib.use('Agg')
27     import matplotlib.pyplot as plt
28     import matplotlib as mpl
29     import os
30
31     mpl.rcParams.update({
32         'font.size': 10, 'axes.titlesize': 11, 'axes.
labelsize': 10,
33         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.
grid': True,
34         'grid.alpha': 0.3, 'mathtext.fontset': 'cm',

```

```

35         })
36         os.makedirs('Figuras', exist_ok=True)
37         rng = np.random.default_rng(20260210)
38
39         #
40
41         -----
42
43         # 1. Parametros
44         #
45         -----
46
47         N = 1000                # poblacion total
48         I0 = 5                  # infectados iniciales
49         beta = 0.60             # tasa de contacto efectivo
50         [1/dia]
51         gamma = 0.25            # tasa de recuperacion [1/dia]
52         ]
53
54         R0 = beta / gamma       # numero reproductivo basico
55         print("=== Ejemplo 5: SIR estocastico (CTMC) vs
56         determinista (EDO) ===")
57         print(f"N={N}, I0={I0}, beta={beta}, gamma={gamma},
58         R0={R0:.2f}")
59
60         #
61         -----
62
63         # 2. Gillespie SSA (una realizacion)
64         #
65         -----
66
67         def gillespie_sir(N, I0, beta, gamma, rng, t_max
68         =200.0):
69             S, I, R = N - I0, I0, 0
70             t = 0.0
71             ts, Is, Ss = [t], [I], [S]
72             while I > 0 and t < t_max:
73                 a_inf = beta * S * I / N           # propension de
74                 infeccion
75                 a_rec = gamma * I                  # propension de
76                 recuperacion
77                 a0 = a_inf + a_rec
78                 if a0 <= 0:

```

```

62         break
63         t += rng.exponential(1.0 / a0)  # tiempo al
siguiente evento ~ Exp(a0)
64         if rng.random() < a_inf / a0:  # cual evento ocurre
65             S -= 1; I += 1
66         else:
67             I -= 1; R += 1
68             ts.append(t); Is.append(I); Ss.append(S)
69             tamano_final = N - S  # total de
individuos que se infectaron
70             return np.array(ts), np.array(Ss), np.array(Is),
tamano_final
71
72         #
-----
73         # 3. EDO determinista (campo medio)
74         #
-----
75         def sir_ode(t, y):
76             S, I, R = y
77             dS = -beta * S * I / N
78             dI = beta * S * I / N - gamma * I
79             dR = gamma * I
80             return [dS, dI, dR]
81         sol = solve_ivp(sir_ode, [0, 120], [N - I0, I0, 0],
82             t_eval=np.linspace(0, 120, 600), rtol=1e-8, atol=1e
-8)
83         I_peak_det = sol.y[1].max()
84         t_peak_det = sol.t[np.argmax(sol.y[1])]
85         final_det = N - sol.y[0, -1]
86
87         #
-----
88         # 4. Ensemble estocastico: distribucion del tamano
final
89         #
-----
90         M = 3000

```

```

91         tamanos = np.empty(M)
92         trayectorias = []
93         for m in range(M):
94             ts, Ss, Is, tf = gillespie_sir(N, I0, beta, gamma,
rng)
95             tamanos[m] = tf
96             if m < 30:                                     # guardar
algunas para graficar
97                 trayectorias.append((ts, Is))
98
99             # clasificacion brote menor vs mayor (umbral en 5%
de la poblacion)
100             umbral = 0.05 * N
101             p_mayor_emp = (tamanos > umbral).mean()
102             p_mayor_teo = 1.0 - (1.0 / R0) ** I0          # prob. de
brote mayor (proc. de ramificacion)
103             tam_mayores = tamanos[tamanos > umbral]
104             print(f"Pico determinista I_max = {I_peak_det:.1f}
en t={t_peak_det:.1f} d")
105             print(f"Tamano final determinista = {final_det:.1f}
({final_det/N*100:.1f}% de N)")
106             print(f"P(brote mayor) empirica = {p_mayor_emp:.3f}"
)
107             print(f"P(brote mayor) teorica 1-(1/R0)^I0 = {
p_mayor_teo:.3f}")
108             print(f"Tamano final medio (brotes mayores) = {
tam_mayores.mean():.1f} "
109                   f"({tam_mayores.mean()/N*100:.1f}% de N)")
110
111             #
-----

112             # 5. Barrido de R0: prob. de brote mayor vs teoria
113             #
-----

114             R0_vec = np.linspace(0.6, 3.0, 13)
115             p_mayor_vec = np.empty_like(R0_vec)
116             for k, R0k in enumerate(R0_vec):
117                 bk = R0k * gamma
118                 cnt = 0; Mk = 800
119                 for _ in range(Mk):

```



```

120         _, _, _, tf = gillespie_sir(N, I0, bk, gamma, rng)
121         cnt += (tf > umbral)
122         p_mayor_vec[k] = cnt / Mk
123         p_teo_vec = np.where(R0_vec > 1, 1.0 - (1.0 / np.
maximum(R0_vec, 1e-9)) ** I0, 0.0)
124
125         #
-----
126         # 6. Figura de 4 paneles
127         #
-----
128         fig, ax = plt.subplots(2, 2, figsize=(11, 8))
129
130         # Panel 1: trayectorias estocasticas I(t) +
determinista
131         for (ts, Is) in trayectorias:
132             ax[0, 0].plot(ts, Is, color='C0', alpha=0.25, lw
=0.8)
133             ax[0, 0].plot(sol.t, sol.y[1], 'C3', lw=2.5, label='
EDO determinista')
134             ax[0, 0].set_xlabel('tiempo [dias]'); ax[0, 0].
set_ylabel('infectados $I(t)$')
135             ax[0, 0].set_title(f'Trayectorias (CTMC) vs campo
medio ($R_0={R0:.1f}$)')
136             ax[0, 0].legend(fontsize=8); ax[0, 0].set_xlim(0,
120)
137
138         # Panel 2: distribucion del tamaño final (bimodal)
139         ax[0, 1].hist(tamamos, bins=50, density=True, color=
'C0', alpha=0.75)
140         ax[0, 1].axvline(final_det, color='C3', ls='--', lw
=2,
141             label=f'final EDO = {final_det:.0f}')
142         ax[0, 1].set_xlabel('tamaño final del brote'); ax[0,
1].set_ylabel('densidad')
143         ax[0, 1].set_title(f'Bimodalidad: extincion vs brote
mayor (M={M})')
144         ax[0, 1].legend(fontsize=8)
145
146         # Panel 3: retrato de fase S-I (sistema dinamico)

```

```

147         ax[1, 0].plot(sol.y[0], sol.y[1], 'C3', lw=2.5,
label='EDO')
148         for (ts, Is) in trayectorias[:15]:
149             # reconstruir S a partir de la trayectoria no es
directo; graficar I vs t-rango
150             pass
151             ax[1, 0].axvline(N / R0, color='k', ls=':', alpha
=0.7,
152             label=r'$S^*=N/R_0$ (umbral)')
153             ax[1, 0].set_xlabel('susceptibles $$$'); ax[1, 0].
set_ylabel('infectados $I$')
154             ax[1, 0].set_title('Retrato de fase del sistema
dinamico')
155             ax[1, 0].legend(fontsize=8)
156
157             # Panel 4: prob. de brote mayor vs R0
158             ax[1, 1].plot(R0_vec, p_mayor_vec, 'o-', ms=5, label
='SSA (empirica)')
159             ax[1, 1].plot(R0_vec, p_teo_vec, '--', color='C3',
label=r'$1-(1/R_0)^{\{I_0\}}$')
160             ax[1, 1].axvline(1.0, color='k', ls=':', alpha=0.6,
label='$R_0=1$')
161             ax[1, 1].set_xlabel('$R_0$'); ax[1, 1].set_ylabel('P
(brote mayor)')
162             ax[1, 1].set_title('Umbral epidemico y extincion
estocastica')
163             ax[1, 1].legend(fontsize=8)
164
165             fig.suptitle('Ejemplo 5 - SIR estocastico (Gillespie
) y su limite determinista',
166             fontsize=12, y=1.00)
167             fig.tight_layout()
168             fig.savefig('Figuras/fig_s7_ej5_sir_estocastico.png'
, bbox_inches='tight')
169             # plt.show()
170             print("\nFigura guardada: Figuras/
fig_s7_ej5_sir_estocastico.png")
171
172

```

Listing 47: Semana 7 – Código 5: SIR estocástico (Gillespie) y EDO determinista. Ejecutar en Google Colab.

7.17.6. Código 6 — PageRank: caminata aleatoria para ranking en RAG

Construcción de la matriz de Google con corrección de nodos colgantes, PageRank por iteración de potencias con registro de convergencia, verificación contra networkx, comparación con el grado de entrada y sensibilidad al damping.

```

1      """
2          Semana 7 - Ejemplo 6: Ranking por caminata aleatoria
          (PageRank) para
3          recuperacion de documentos en un sistema RAG.
4
5          Un sistema RAG (Retrieval-Augmented Generation)
          ordena documentos por
6          relevancia. Cuando los documentos forman un grafo
          dirigido (citas, enlaces o
7          similitud semantica umbralizada), su importancia
          puede medirse con la
8          distribucion estacionaria de una caminata aleatoria:
          PageRank.
9
10         Cadena de Markov del "navegante aleatorio":
11         - Con probabilidad alpha (damping) sigue un enlace
          saliente uniforme.
12         - Con probabilidad (1-alpha) "teletransporta" a un
          documento al azar.
13         - Nodos sin enlaces (dangling) se tratan como
          teletransporte total.
14         Matriz de Google:  $G = \alpha * (S) + (1-\alpha) * (1/n) * \mathbf{1} \mathbf{1}^T$ ,
15         con S la matriz de transicion estocastica por filas
          (incluye correccion de
16         nodos colgantes). El vector PageRank es el
          autovector izquierdo de G con
17         autovalor 1 (Perron-Frobenius), calculado por
          iteracion de potencias.
18
19         La brecha espectral de G es  $1 - \alpha$ , lo que fija
          la tasa de convergencia
20          $\sim \alpha^k$  de la iteracion de potencias (
          independiente del grafo).
21         """
22
23         import numpy as np

```

```

24     import networkx as nx
25     import matplotlib
26     matplotlib.use('Agg')
27     import matplotlib.pyplot as plt
28     import matplotlib as mpl
29     import os
30
31     mpl.rcParams.update({
32         'font.size': 10, 'axes.titlesize': 11, 'axes.
33         labelsizes': 10,
34         'figure.dpi': 110, 'savefig.dpi': 300, 'mathtext
35         .fontset': 'cm',
36         })
37     os.makedirs('Figuras', exist_ok=True)
38     rng = np.random.default_rng(20260211)
39
40     #
41     -----
42
43     # 1. Grafo dirigido de documentos (n nodos). Aristas
44     i->j: "i referencia a j"
45     #
46     -----
47
48     n = 14
49     G = nx.gnp_random_graph(n, 0.22, seed=42, directed=
50     True)
51     # garantizar al menos un nodo colgante (sin salida)
52     para ilustrar la correccion
53     G.remove_edges_from(list(G.out_edges(7)))
54     A = nx.to_numpy_array(G, nodelist=range(n))      # A[
55     i,j]=1 si i->j
56
57     #
58     -----
59
60     # 2. Matriz de transicion S con correccion de nodos
61     colgantes
62     #
63     -----
64
65     out_deg = A.sum(axis=1)

```

```

51         S = np.zeros((n, n))
52         for i in range(n):
53             if out_deg[i] > 0:
54                 S[i] = A[i] / out_deg[i]                # enlace
55             else:
56                 S[i] = np.ones(n) / n                    # dangling
57             alpha = 0.85
58             G_mat = alpha * S + (1 - alpha) * np.ones((n, n)) /
59             n
60             assert np.allclose(G_mat.sum(axis=1), 1.0)
61             #
62             -----
63             # 3. PageRank por iteracion de potencias (
64             registrando convergencia)
65             #
66             -----
67             pi = np.ones(n) / n
68             hist = []
69             pi_exact = None
70             for k in range(100):
71                 pi_new = pi @ G_mat
72                 hist.append(np.abs(pi_new - pi).sum())    # ||pi_
73                 {k+1}-pi_k||_1
74                 if hist[-1] < 1e-15:
75                     pi_exact = pi_new; break
76                     pi = pi_new
77                     pr = pi / pi.sum()
78                     # distancia al limite para graficar tasa alpha^k
79                     pr_ref = pr.copy()
80                     pi2 = np.ones(n) / n
81                     dist_lim = []
82                     for k in range(60):
83                         dist_lim.append(np.abs(pi2 - pr_ref).sum())
84                     pi2 = pi2 @ G_mat
85
86             # verificacion contra networkx
87             pr_nx = np.array([nx.pagerank(G, alpha=alpha)[i] for

```

```

84         i in range(n)])
85
86         #
87         -----
88
89         # 4. Comparacion con grado de entrada (heuristica
90         ingenua)
91         #
92         -----
93
94         in_deg = A.sum(axis=0)
95         in_deg_norm = in_deg / in_deg.sum() if in_deg.sum()
96         > 0 else in_deg
97         orden_pr = np.argsort(pr)[::-1]
98         orden_in = np.argsort(in_deg)[::-1]
99
100        # brecha espectral teorica y observada
101        gap_teo = 1 - alpha
102        # segundo autovalor en magnitud
103        ev = np.sort(np.abs(np.linalg.eigvals(G_mat.T)))
104        [::-1]
105        lam2 = ev[1]
106
107        print("=== Ejemplo 6: PageRank (caminata aleatoria)
108        para RAG ===")
109        print(f"n={n} documentos, alpha={alpha}, nodos
110        colgantes={int((out_deg==0).sum())}")
111        print(f"PageRank (power iter) top-5 nodos: {orden_pr
112        [:5]}")
113        print(f"Ranking por grado de entrada top-5: {
114        orden_in[:5]}")
115        print(f"Max |PageRank - networkx| = {np.abs(pr -
116        pr_nx).max():.2e}")
117        print(f"Iteraciones a tol 1e-15 = {len(hist)}")
118        print(f"|lambda_2(G)| = {lam2:.4f}      brecha
119        espectral = {1-lam2:.4f}      (teorica 1-alpha={gap_teo:.2f})")
120        print(f"PageRank nodo top = {orden_pr[0]} con score
121        {pr[orden_pr[0]]:.4f}")
122        print(f"Coincide top-1 PageRank vs grado entrada? {
123        orden_pr[0]==orden_in[0]}")
124
125        #
    
```

```

-----
110         # 5. Sensibilidad: iteraciones a convergencia vs
alpha
111         #
-----

112         def iters_converge(alpha_, tol=1e-10):
113             Gm = alpha_ * S + (1 - alpha_) * np.ones((n, n)) / n
114             p = np.ones(n) / n
115             for k in range(2000):
116                 pn = p @ Gm
117                 if np.abs(pn - p).sum() < tol:
118                     return k + 1
119                 p = pn
120             return 2000
121             alpha_vec = np.linspace(0.50, 0.97, 24)
122             iters_vec = np.array([iters_converge(a) for a in
alpha_vec])
123
124         #
-----

125         # 6. Figura de 4 paneles
126         #
-----

127         fig, ax = plt.subplots(2, 2, figsize=(11, 8))
128
129         # Panel 1: grafo con tamaño de nodo ~ PageRank
130         pos = nx.spring_layout(G, seed=7)
131         nx.draw_networkx_edges(G, pos, ax=ax[0, 0], alpha
=0.3, arrowsize=8,
132             edge_color='gray', width=0.8)
133         nodes = nx.draw_networkx_nodes(G, pos, ax=ax[0, 0],
node_size=pr * 9000 + 60,
134             node_color=pr, cmap='viridis')
135         nx.draw_networkx_labels(G, pos, ax=ax[0, 0],
font_size=7, font_color='white')
136         ax[0, 0].set_title('Grafo de documentos (tamaño $\\
propto$ PageRank)')
137         ax[0, 0].axis('off')

```

```

138         plt.colorbar(nodes, ax=ax[0, 0], fraction=0.046, pad
139         =0.04)
140
141         # Panel 2: convergencia de la iteracion de potencias
142         (escala log)
143         kk = np.arange(len(dist_lim))
144         ax[0, 1].semilogy(kk, np.array(dist_lim) + 1e-18, 'o
145         -', ms=3,
146         label=r'$\\pi_k - \\pi^{*}_1$')
147         ax[0, 1].semilogy(kk, alpha ** kk * dist_lim[0], '--
148         ',
149         label=r'tasa $\\alpha^k$')
150         ax[0, 1].set_xlabel('iteracion $k$'); ax[0, 1].
151         set_ylabel('error')
152         ax[0, 1].set_title(f'Convergencia geometrica (brecha
153         $1-\\alpha={1-alpha:.2f}$)')
154         ax[0, 1].grid(alpha=0.3); ax[0, 1].legend(fontsize
155         =8)
156
157         # Panel 3: PageRank vs grado de entrada normalizado
158         x = np.arange(n); width = 0.4
159         ax[1, 0].bar(x - width / 2, pr, width, label='
160         PageRank', color='C0', alpha=0.85)
161         ax[1, 0].bar(x + width / 2, in_deg_norm, width,
162         label='grado entrada (norm.)',
163         color='C1', alpha=0.85)
164         ax[1, 0].set_xlabel('documento'); ax[1, 0].
165         set_ylabel('puntaje')
166         ax[1, 0].set_title('PageRank vs heuristica de grado
167         de entrada')
168         ax[1, 0].set_xticks(x); ax[1, 0].grid(alpha=0.3,
169         axis='y')
170         ax[1, 0].legend(fontsize=8)
171
172         # Panel 4: iteraciones a convergencia vs damping
173         alpha
174         ax[1, 1].plot(alpha_vec, iters_vec, 'o-', ms=4,
175         color='C3')
176         ax[1, 1].axvline(alpha, color='C0', ls='--', alpha
177         =0.7, label=f'$\\alpha={alpha}$')
178         ax[1, 1].set_xlabel(r'damping $\\alpha$')
179         ax[1, 1].set_ylabel('iteraciones a tol $10^{-10}$')
    
```



```
165         ax[1, 1].set_title('Costo de convergencia vs damping  
'  
166     ')  
167         ax[1, 1].grid(alpha=0.3); ax[1, 1].legend(fontsize  
168     =8)  
169         fig.suptitle('Ejemplo 6 - PageRank: caminata  
170     aleatoria para ranking en RAG',  
171     fontsize=12, y=1.00)  
172         fig.tight_layout()  
173         fig.savefig('Figuras/fig_s7_ej6_pagerank_rag.png',  
174     bbox_inches='tight')  
175         # plt.show()  
176         print("\nFigura guardada: Figuras/  
fig_s7_ej6_pagerank_rag.png")
```

Listing 48: Semana 7 – Código 6: PageRank (caminata aleatoria) para RAG. Ejecutar en Google Colab.

7.18. Ejercicios Reto — Tarea Semana 7

Los Ejercicios 1 y 2 son obligatorios. El Ejercicio 3 otorga punto extra. Todos deben anclarse al *proyecto doctoral propio* del estudiante: el espacio de estados, las transiciones y los datos provienen de su propia investigación.

7.18.1. Ejercicio 1 (obligatorio) — Modelo de Markov del propio proyecto

Identifique en su proyecto un fenómeno con evolución markoviana (deriva, degradación, disponibilidad, ocupación, propagación o ranking) y defina su espacio de estados. Entregue: (i) la justificación de la propiedad de Markov y de la homogeneidad temporal, declarando explícitamente qué información codifica el estado; (ii) la matriz de transición P (tiempo discreto) o la matriz generadora Q (tiempo continuo), con cada transición motivada físicamente; (iii) la distribución estacionaria resuelta por (195) o (204), con su interpretación (disponibilidad, fracción de tiempo, ranking); y (iv) la clasificación de la cadena (irreducible/aperiódica, estados absorbentes si los hay).

Rúbrica.

- 3 Espacio de estados bien definido, propiedad de Markov justificada, P o Q correcta y motivada, estacionaria resuelta e interpretada, clasificación correcta.
- 2 Modelo correcto pero con propiedad de Markov no justificada, o estacionaria sin interpretación física.

- 1 Matriz mal normalizada (filas que no suman 1, o Q con filas que no suman 0) o estados mal definidos.
- 0 Sin modelo markoviano del proyecto.

7.18.2. Ejercicio 2 (obligatorio) — Tiempos característicos y validación por simulación

Para el modelo del Ejercicio 1, calcule un tiempo característico relevante: tiempo medio de recurrencia (197), tiempo medio hasta la absorción por la matriz fundamental (200)/(205) (MTTF), o tiempo de mezcla por la brecha espectral (207). Implemente en Google Colab una simulación reproducible (`numpy.random.default_rng(semilla)`) del proceso y valide el resultado analítico contra el promedio de al menos $M \geq 10^4$ trayectorias, reportando el error estándar. Discuta el acuerdo (o desacuerdo) y su origen.

Rúbrica.

- 3 Tiempo característico correcto por vía analítica, simulación reproducible, validación con error estándar y discusión del acuerdo.
- 2 Cálculo analítico correcto pero validación sin error estándar, o simulación no reproducible.
- 1 Simulación que corre pero sin contraste cuantitativo con la teoría.
- 0 Sin cálculo de tiempos característicos ni validación.

7.18.3. Ejercicio 3 — Transición de fase o propagación de incertidumbre

Elija una de las dos vías. **(a) Transición de fase:** si su fenómeno admite una representación espacial o de propagación, identifique un parámetro de control, estime su umbral crítico por barrido y caracterice el parámetro de orden y su variabilidad cerca de la criticidad. **(b) Propagación de incertidumbre:** estime su matriz de transición a partir de datos (reales o sintéticos) por máxima verosimilitud (212), propague la incertidumbre de las entradas $\hat{p}_{ij}$ a la distribución estacionaria o al MTTF mediante Monte Carlo (técnica de la Semana 6) y reporte el intervalo de cobertura del 95 % de la cantidad de interés.

Rúbrica.

- 3 (a) Umbral estimado y parámetro de orden caracterizado con su variabilidad; o (b) MLE correcto, propagación Monte Carlo e intervalo de cobertura bien construido.
- 0 No entregado o sin caracterización cuantitativa del umbral / del intervalo.

8. Semana 8. Método de Elementos Finitos I

Objetivo de la cátedra. Comprender la formulación matemática y computacional del método de elementos finitos (FEM): la transición de un problema de valores en la frontera continuo a un sistema algebraico discreto, el papel de la discretización espacial y las funciones de forma, el ensamble matricial y la imposición de condiciones de frontera. El eje aplicado es el módulo `STATIC STRUCTURAL` de `ANSYS WORKBENCH`; la generación de malla se ilustra primero en Python (Google Colab) para hacer explícitas las decisiones de mallado que en `ANSYS` quedan ocultas tras la automatización.

Temario

- Introducción al FEM.
- Discretización espacial.
- Generación de mallas.
- Elementos y nodos.
- Funciones de forma.
- Ensamble matricial.
- Condiciones de frontera.

8.1. Introducción al FEM

El FEM es un método de discretización para resolver de forma aproximada ecuaciones diferenciales parciales (EDP) sobre dominios de geometría arbitraria. Su idea central es sustituir el problema continuo, con infinitos grados de libertad, por un sistema algebraico finito construido sobre una partición del dominio en subregiones simples (los *elementos*), sobre las cuales la solución se aproxima mediante polinomios definidos a trozos.

Para el análisis estructural estático lineal —el caso del módulo `STATIC STRUCTURAL`— la EDP gobernante es la ecuación de equilibrio de la elastostática. En su *forma fuerte*, para un cuerpo que ocupa el dominio $\Omega \subset \mathbb{R}^d$ ($d = 2, 3$):

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0} \quad \text{en } \Omega, \quad (216)$$

sujeta a la ley constitutiva y a la relación cinemática de pequeñas deformaciones:

$$\boldsymbol{\sigma} = \mathbf{D} : \boldsymbol{\varepsilon}, \quad (217)$$

$$\boldsymbol{\varepsilon} = \frac{1}{2} (\nabla \mathbf{u} + \nabla \mathbf{u}^T), \quad (218)$$

donde $\boldsymbol{\sigma}$ es el tensor de esfuerzos, $\boldsymbol{\varepsilon}$ el de deformaciones, $\mathbf{u}$ el campo de desplazamientos,

b las fuerzas de cuerpo y **D** el tensor constitutivo elástico (para material isótropo, función de E y ν). Las condiciones de frontera son de tipo Dirichlet (desplazamiento prescrito $\mathbf{u} = \bar{\mathbf{u}}$ en Γ_u) y Neumann (tracción prescrita $\boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}}$ en Γ_t).

El FEM no opera sobre la forma fuerte, sino sobre su *forma débil* o variacional. Multiplicando Ecuación (216) por una función de prueba admisible $\mathbf{w}$ (con $\mathbf{w} = \mathbf{0}$ en Γ_u) e integrando por partes se obtiene el principio de los trabajos virtuales:

$$\int_{\Omega} \boldsymbol{\varepsilon}(\mathbf{w}) : \mathbf{D} : \boldsymbol{\varepsilon}(\mathbf{u}) \, d\Omega = \int_{\Omega} \mathbf{w} \cdot \mathbf{b} \, d\Omega + \int_{\Gamma_t} \mathbf{w} \cdot \bar{\mathbf{t}} \, d\Gamma. \quad (219)$$

La forma débil (219) exige solo derivadas de primer orden de $\mathbf{u}$ (frente al segundo orden de la forma fuerte), lo que permite usar aproximaciones polinómicas continuas a trozos (C^0). El método de *Galerkin* consiste en elegir las funciones de prueba $\mathbf{w}$ del mismo espacio discreto que la solución aproximada $\mathbf{u}_h$.

Tabla 41: Comparación de métodos de discretización para EDP.

Aspecto	Diferencias finitas	Elementos finitos	Volumen finito
Formulación	Forma fuerte	Forma débil (Galerkin)	Balance integral
Geometría	Mallas regulares	Geometría arbitraria	Geometría arbitraria
Frontera curva	Difícil	Natural (iso- paramétrica)	Natural
Uso dominante	Térmico/CFD simple	Estructural/mecánico	CFD

La Tabla 41 sitúa al FEM frente a otras técnicas. Su ventaja decisiva para el diseño mecánico es el tratamiento natural de geometrías con fronteras curvas mediante la formulación isoparamétrica (section 8.4), razón por la cual es el estándar en análisis estructural.

8.2. Discretización espacial

La discretización espacial es la partición del dominio Ω en un conjunto finito de elementos $\{\Omega_e\}_{e=1}^{n_{el}}$ que no se solapan y cuya unión aproxima Ω :

$$\Omega \approx \Omega_h = \bigcup_{e=1}^{n_{el}} \Omega_e, \quad \Omega_i \cap \Omega_j = \emptyset \quad (i \neq j). \quad (220)$$

La malla debe ser *conforme*: dos elementos adyacentes comparten una arista (2D) o cara (3D) completa, sin nodos colgantes. El parámetro característico es el tamaño de elemento h (longitud representativa de arista). La teoría de aproximación establece, para elementos

de orden polinómico p y solución suficientemente regular, la cota de error *a priori*:

$$\|\mathbf{u} - \mathbf{u}_h\|_E \leq C h^p, \quad (221)$$

donde $\|\cdot\|_E$ es la norma energética y C una constante independiente de h . La [Ecuación \(221\)](#) justifica las dos vías de mejora de la precisión: refinamiento h (reducir el tamaño de elemento) y refinamiento p (elevar el orden polinómico).

En la práctica no se conoce la solución exacta, por lo que la convergencia se verifica comparando mallas sucesivamente refinadas. Se adopta el criterio de convergencia de malla empleado en el hilo de validación DIC–FEA del curso:

$$e_{\text{malla}} = \frac{|\varepsilon_{xx}^{(h)} - \varepsilon_{xx}^{(h/2)}|}{|\varepsilon_{xx}^{(h/2)}|} \leq 2\%, \quad (222)$$

es decir, la malla se considera convergida cuando la cantidad de interés (aquí la deformación ε_{xx}) no cambia más del 2 % al reducir a la mitad el tamaño de elemento. Este error de discretización e_{num} es una de las cuatro contribuciones al error total en una comparación experimento–modelo y debe mantenerse por debajo del 2 % para que no domine el presupuesto.

Enlace metrológico. En una validación experimental por correlación digital de imágenes (DIC), el tamaño de elemento FEA no debe elegirse arbitrariamente fino: conviene generar una malla *convergada* ($h \leq 1$ mm) para obtener la solución “verdadera” del modelo y una malla *compatible* ($h \approx L_{\text{VSG}}$, el tamaño del subconjunto virtual de deformación del DIC) para comparar punto a punto. Si ambas coinciden, los gradientes son suaves y la comparación es directa; si difieren, el gradiente es agudo y debe considerarse el sesgo de suavizado del DIC.

8.3. Elementos y nodos

El elemento es la unidad geométrica y de interpolación; los *nodos* son los puntos donde se definen los grados de libertad (GDL) —las incógnitas del problema. En análisis estructural 2D cada nodo porta dos GDL de desplazamiento (u_x, u_y); en 3D, tres (u_x, u_y, u_z); los elementos de cáscara y viga añaden GDL rotacionales. El número total de GDL del modelo es

$$n_{\text{GDL}} = d_{\text{nodo}} \cdot n_{\text{nodos}}, \quad (223)$$

con d_{nodo} los GDL por nodo. La [Ecuación \(223\)](#) fija el tamaño del sistema algebraico y, por tanto, el costo computacional.

La distinción fundamental es entre elementos *lineales* (solo nodos de esquina, fun-

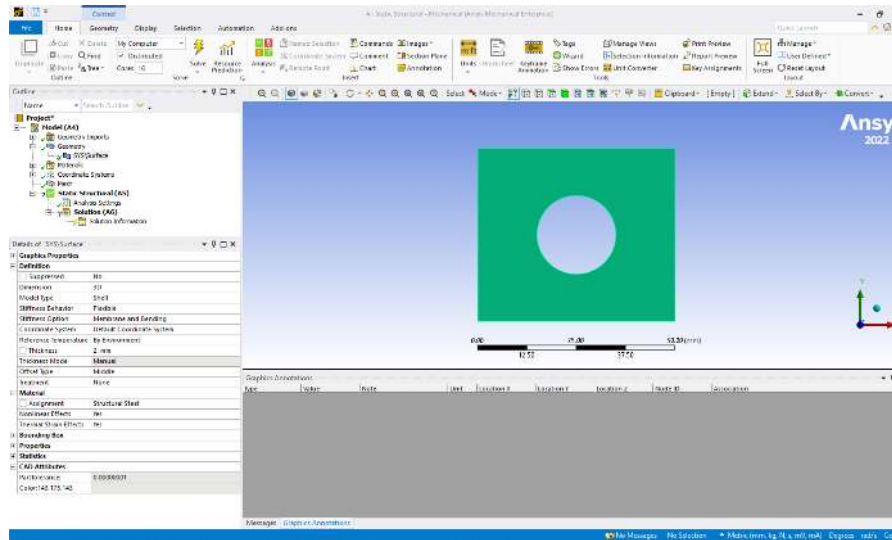


Figura 55: Definición de la geometría 2D en ANSYS. La placa con barreno central reproduce el dominio del ejemplo computacional (Ec. 220).

ciones de forma de primer orden) y *cuadráticos* (con nodos intermedios en las aristas, segundo orden). La Figura 56 muestra las cuatro configuraciones canónicas en 2D y su conteo de GDL. Los elementos cuadráticos representan con exactitud fronteras curvas (la arista es una parábola) y capturan mejor los gradientes de esfuerzo, a costa de más GDL por elemento.

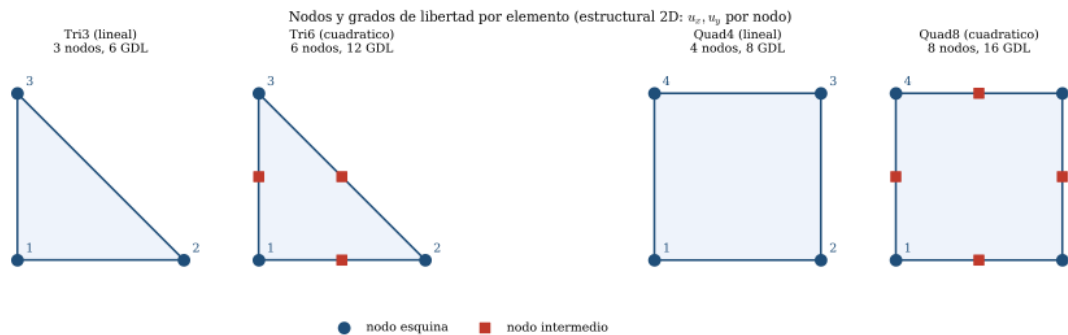


Figura 56: Nodos y grados de libertad de los elementos 2D canónicos. Los nodos de esquina (azul) definen los elementos lineales Tri3 y Quad4; los nodos intermedios (rojo) los promueven a cuadráticos Tri6 y Quad8. Cada nodo aporta dos GDL (u_x, u_y) según Ec. 223.

La Tabla 42 lista elementos de uso frecuente. La nomenclatura CalculiX/Abaqus equivalente para el tetraedro cuadrático es C3D10, el elemento por defecto en el modelo de validación de flexión a tres puntos del curso. Una regla operativa: los elementos cuadráticos de tetraedro (SOLID187, C3D10) son la opción robusta para geometrías complejas mallables solo con tetraedros, mientras que los hexaedros lineales o cuadráticos se prefieren donde la geometría admite malla mapeada o barrida.

Tabla 42: Elementos estructurales representativos de la biblioteca de ANSYS.

Elemento	Dim.	Nodos	Orden	Uso típico
PLANE182	2D	4	Lineal	Tensión/deformación plana, axisimétrico
PLANE183	2D	6/8	Cuadr.	Ídem, fronteras curvas y gradientes altos
SOLID185	3D	8	Lineal	Sólido hexaédrico general
SOLID186/187	3D	20/10	Cuadr.	Sólido hexa/tetra de segundo orden
SHELL181	3D	4	Lineal	Cáscaras delgadas (6 GDL/nodo)

Consistencia de GDL en interfaces. Al unir elementos de distinto tipo se debe cuidar la compatibilidad de GDL. Un SOLID185 (3 GDL de traslación) unido a un SHELL181 (3 de traslación + 3 rotacionales) no transmite momentos a través de la interfaz, pues el sólido carece de GDL rotacionales. La continuidad exige que ambos elementos compartan el mismo número y tipo de GDL en los nodos comunes.

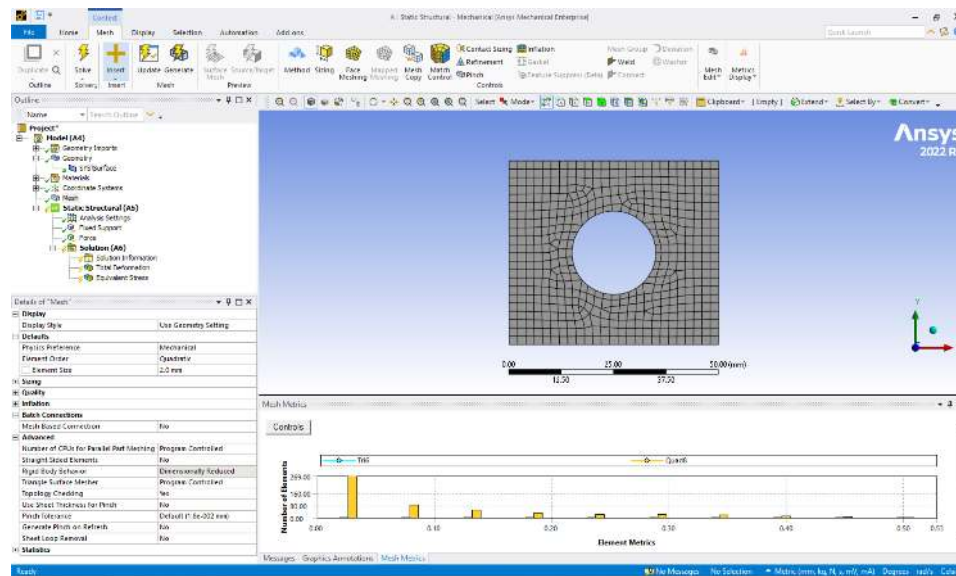


Figura 57: Control del orden del elemento en ANSYS: *Dropped* produce elementos lineales; *Kept*, cuadráticos con nodos intermedios, elevando los GDL según Ec. 223.

8.4. Funciones de forma

Dentro de cada elemento, el campo de desplazamientos se interpola a partir de los valores nodales $\mathbf{u}_i$ mediante las *funciones de forma* $N_i(\boldsymbol{\xi})$:

$$\mathbf{u}_h(\boldsymbol{\xi}) = \sum_{i=1}^{n_{en}} N_i(\boldsymbol{\xi}) \mathbf{u}_i, \quad (224)$$

donde n_{en} es el número de nodos del elemento y $\boldsymbol{\xi}$ las coordenadas naturales (paramétricas). Las funciones de forma deben satisfacer:

1. **Delta de Kronecker:** $N_i(\boldsymbol{\xi}_j) = \delta_{ij}$ (vale 1 en su nodo y 0 en los demás).
2. **Partición de la unidad:** $\sum_{i=1}^{n_{en}} N_i(\boldsymbol{\xi}) = 1$, condición necesaria para representar exactamente un campo constante (movimiento de cuerpo rígido).
3. **Compleitud:** capacidad de reproducir polinomios hasta el orden requerido para la convergencia.

Para el elemento cuadrilátero bilineal (Q4), en coordenadas naturales $(\xi, \eta) \in [-1, 1]^2$:

$$N_i(\xi, \eta) = \frac{1}{4} (1 + \xi_i \xi)(1 + \eta_i \eta), \quad i = 1, \dots, 4, \quad (225)$$

con (ξ_i, η_i) las coordenadas nodales. La [Figura 58](#) muestra las funciones lineales y cuadráticas en 1D y la superficie bilineal $N_1(\xi, \eta)$ del Q4. La verificación numérica del script confirma la partición de la unidad para la base cuadrática 1D y la consistencia de la base Q4.

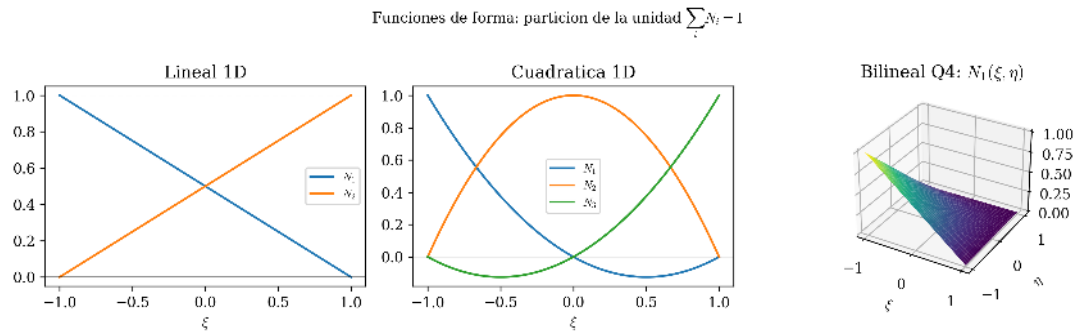


Figura 58: Funciones de forma. Izquierda: base lineal 1D; centro: base cuadrática 1D de Lagrange; derecha: función bilineal $N_1(\xi, \eta)$ del elemento Q4. Todas satisfacen la partición de la unidad $\sum_i N_i = 1$.

Formulación isoparamétrica. La misma base N_i interpola tanto el campo incógnita (224) como la geometría del elemento:

$$\mathbf{x}(\boldsymbol{\xi}) = \sum_{i=1}^{n_{en}} N_i(\boldsymbol{\xi}) \mathbf{x}_i. \quad (226)$$

Esta identidad —el concepto isoparamétrico— es la que permite mapear un elemento de referencia recto a un elemento distorsionado con aristas curvas, y es la base del mallado de fronteras curvas. La transformación entre el espacio natural y el físico se cuantifica mediante el jacobiano $\mathbf{J} = \partial \mathbf{x} / \partial \boldsymbol{\xi}$; su determinante debe ser positivo en todo el elemento (un jacobiano negativo indica un elemento invertido y provoca fallo de mallado). La razón jacobiana es, de hecho, una de las métricas de calidad de malla (section 8.6).

8.5. Ensamble matricial

Sustituyendo la interpolación (224) en la forma débil (219) y aplicando Galerkin, cada elemento contribuye con una *matriz de rigidez elemental*:

$$\mathbf{K}_e = \int_{\Omega_e} \mathbf{B}^T \mathbf{D} \mathbf{B} d\Omega, \quad (227)$$

donde $\mathbf{B} = \partial \mathbf{N}$ es la matriz deformación–desplazamiento (derivadas de las funciones de forma) y $\mathbf{D}$ el tensor constitutivo. La integral (227) se evalúa numéricamente por cuadratura de Gauss sobre el elemento de referencia, empleando el jacobiano de (226) para el cambio de variable.

Las contribuciones elementales se *ensamblan* en la matriz de rigidez global $\mathbf{K}$ y el vector de fuerzas $\mathbf{F}$ mediante el operador de ensamble $\bigwedge$ (mapeo de GDL locales a globales):

$$\mathbf{K} = \bigwedge_{e=1}^{n_{el}} \mathbf{K}_e, \quad \mathbf{F} = \bigwedge_{e=1}^{n_{el}} \mathbf{F}_e, \quad (228)$$

donde $\bigwedge$ denota el operador de ensamble (suma con mapeo de GDL locales a globales). El sistema algebraico resultante es lineal:

$$\mathbf{K} \mathbf{U} = \mathbf{F}, \quad (229)$$

con $\mathbf{U}$ el vector global de desplazamientos nodales ($\dim \mathbf{U} = n_{\text{GDL}}$). La matriz $\mathbf{K}$ es simétrica, definida positiva (tras imponer las condiciones de frontera que eliminan los modos de cuerpo rígido) y *dispersa*: cada nodo solo interactúa con sus vecinos inmediatos, por lo que $\mathbf{K}$ tiene estructura de banda. Esta dispersión es la que hace tratables modelos con millones de GDL y motiva el uso de solvers dispersos (directos tipo Cholesky o iterativos tipo PCG).

Tabla 43: Secuencia de ensamble y solución en STATIC STRUCTURAL.

Paso	Operación	Resultado
1	Evaluar $\mathbf{K}_e$ por cuadratura (Ec. 227)	Matrices elementales
2	Mapear GDL locales $\rightarrow$ globales	Tabla de conectividad
3	Ensamblar $\mathbf{K}$, $\mathbf{F}$ (Ec. 228)	Sistema global disperso
4	Imponer condiciones de frontera	Sistema reducido no singular
5	Resolver $\mathbf{KU} = \mathbf{F}$ (Ec. 229)	Desplazamientos nodales
6	Recuperar ϵ , σ (post)	Campos de deformación y esfuerzo

8.6. Generación de mallas

La generación de malla es la etapa donde se materializa la discretización (220). La calidad de la malla condiciona directamente la precisión (a través de la razón jacobiana en (227)) y la convergencia del solver. Un mallado inadecuado —elementos distorsionados, transiciones bruscas, resolución insuficiente en zonas de gradiente alto— degrada la solución aun con una formulación correcta. Para hacer explícitas estas decisiones, el ejemplo computacional (Anexo A) genera y compara, sobre una misma geometría con frontera curva, las estrategias de mallado disponibles en ANSYS Workbench.

Geometría de referencia. Placa cuadrada de 50×50 mm con barreno circular central de radio 10 mm (Figura 59). Es el *benchmark* clásico de concentración de esfuerzos (problema de Kirsch), y su frontera curva obliga a decidir cómo mallar la vecindad del barreno. El material de referencia es aluminio 6061-T6 ($E = 68.9$ GPa, $\nu = 0.33$).

Estrategias de mallado. Se ilustran cuatro:

1. **Malla libre triangular** (Figura 60a). Algoritmo de avance frontal/Delaunay; llena dominios arbitrarios sin restricciones topológicas. Es la opción por defecto más robusta (*Patch Conforming Tetrahedron* en 3D), pero genera más elementos y mayor dispersión de calidad.
2. **Malla de cuadriláteros** (Figura 60b). Recombinación de triángulos en cuadriláteros (algoritmo Blossom). Menos elementos y, en general, mejor precisión por GDL que los triángulos lineales, que son excesivamente rígidos.
3. **Malla mapeada estructurada** (Figura 61). Malla transfinita sobre un sector anular en torno al barreno: topología regular tipo rejilla, con conectividad predecible y calidad óptima. Es el análogo del *Mapped Face Meshing / Sweep* de ANSYS.
4. **Malla con sesgo** (Figura 62). La misma malla mapeada, con *sesgo* (bias) radial

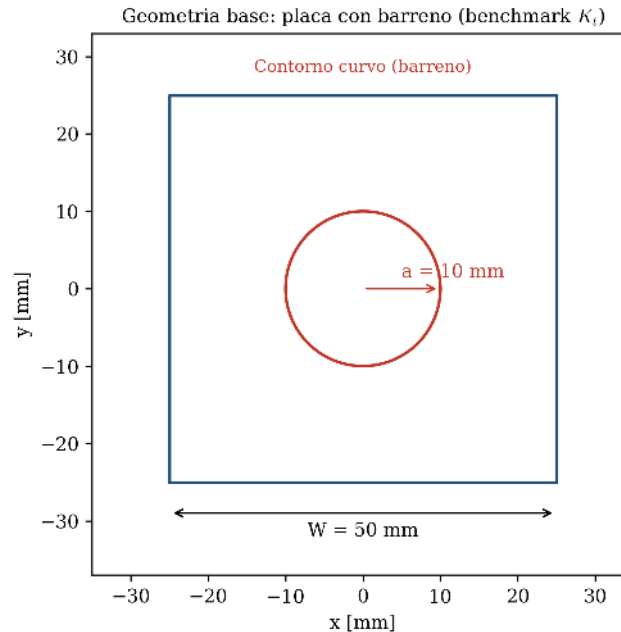


Figura 59: Geometría base del ejemplo: placa con barreno central. La frontera curva (rojo) es la región crítica donde se concentran los esfuerzos y donde el mallado debe refinarse.

que concentra nodos hacia el barreno, donde el gradiente de esfuerzo es máximo. Es el *Bias Factor* de ANSYS.

Métricas de calidad. La calidad de cada elemento se cuantifica con métricas geométricas. Las principales en ANSYS son:

- **Skewness (asimetría):** desviación respecto al elemento equiangular ideal. Rango $[0, 1]$; 0 es perfecto. Para un triángulo el ideal tiene ángulos de 60° ; para un cuadrilátero, de 90° .
- **Orthogonal Quality:** ortogonalidad de las caras. Rango $[0, 1]$; 1 es óptimo, 0 pésimo.
- **Aspect Ratio:** razón entre la arista mayor y la menor; 1 es ideal, valores altos indican elementos estirados.
- **Jacobian Ratio:** uniformidad del mapeo isoparamétrico; detecta elementos invertidos o excesivamente distorsionados.

Los resultados numéricos del ejemplo (Tabla 45) cuantifican los compromisos. La malla mapeada logra la mejor calidad (skewness máximo 0.033) con el menor número de GDL (386), frente a la malla triangular libre (996 GDL, skewness máximo 0.385) y la de cuadriláteros (842 GDL, skewness máximo 0.526). El sesgo, aplicado sobre la malla mapeada, mantiene el skewness (0.033) pero eleva la razón de aspecto de 1.92 a 2.30: el compromiso esperado, pues concentrar nodos radialmente estira los elementos. Este

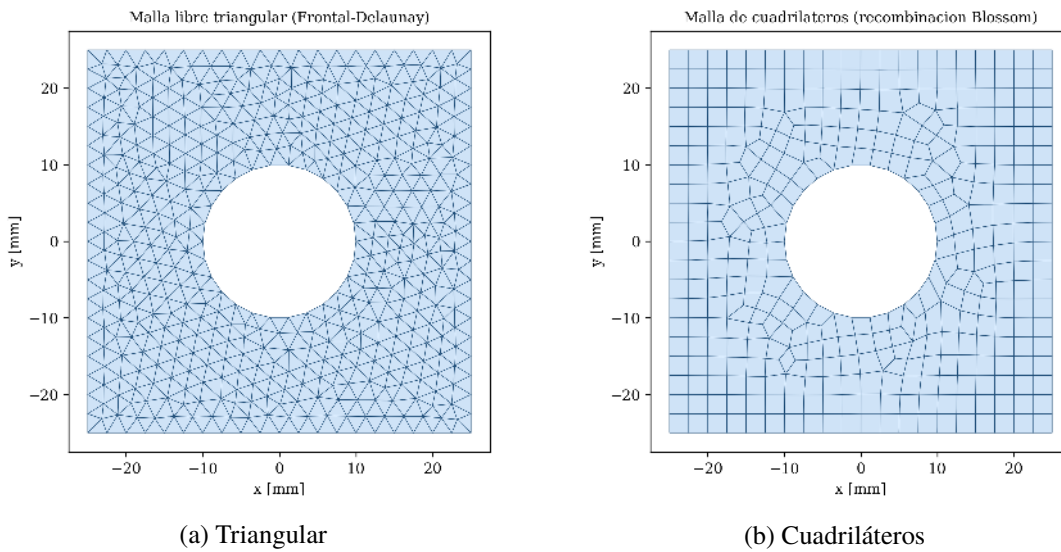


Figura 60: Mallas libres sobre la placa con barreno. (a) Triangular (Frontal–Delaunay). (b) Cuadriláteros por recombinación. Ambas resuelven la frontera curva sin intervención del usuario, a costa de mayor número de GDL.

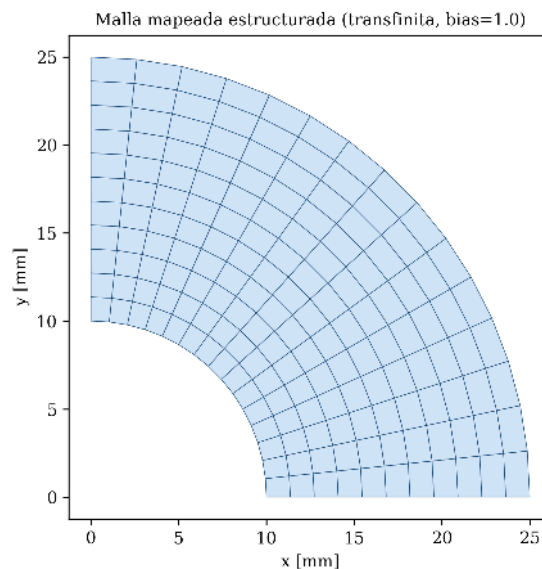


Figura 61: Malla mapeada estructurada (transfinita) sobre el sector anular en torno al barreno. La topología regular garantiza calidad uniforme; equivale al *Mapped Face Meshing* de ANSYS.

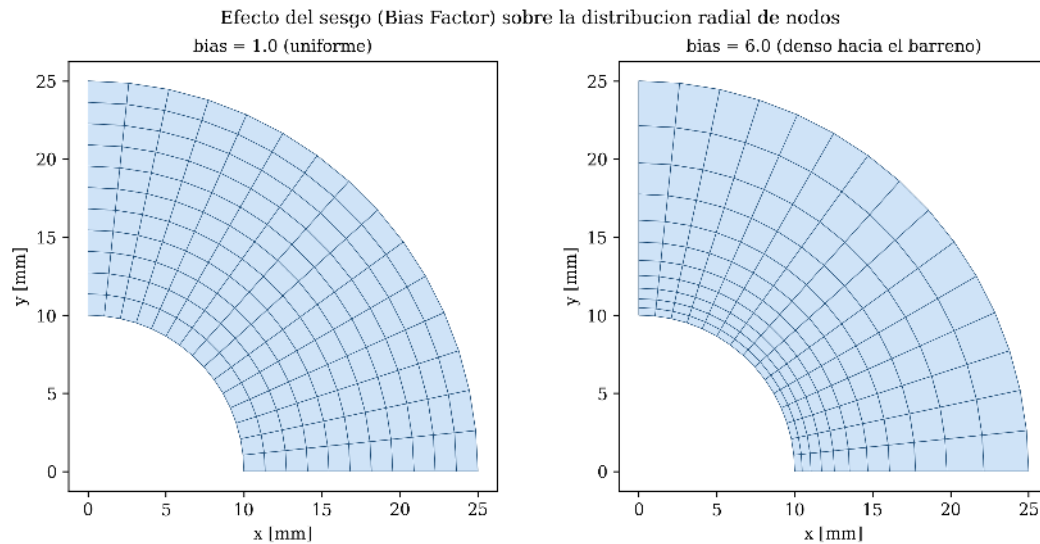


Figura 62: Efecto del sesgo (Bias Factor). Izquierda: distribución radial uniforme. Derecha: sesgo que concentra nodos hacia el barreno. El sesgo mejora la resolución del gradiente sin aumentar el número total de nodos.

Tabla 44: Límites de forma de ANSYS (opción Standard Mechanical): un valor fuera de rango produce fallo de mallado.

Métrica	Límite	Interpretación
Aspect Ratio	— (métrica informativa)	Elementos estirados
Jacobian Ratio (2D)	> 10	Mapeo degenerado
Jacobian Ratio (3D)	> 40	Mapeo degenerado
Orthogonal Quality	≤ 0.01–0.05	Cara casi degenerada
Skewness	(recomendado < 0.9)	Elemento asimétrico

es el punto pedagógico central —*el sesgo afecta la relación de aspecto, no la asimetría angular*— y la razón por la cual el Bias Factor debe usarse con mesura.

Tabla 45: Comparación cuantitativa de estrategias de mallado sobre la placa con barreno (salida real del script del Anexo A).

Estrategia	Nodos	Elem.	GDL	Skew. máx.	AR máx.
Triangular libre	498	890	996	0.385	1.66
Cuadriláteros	421	368	842	0.526	2.15
Mapeada uniforme	193	165	386	0.033	1.92
Mapeada + sesgo (BF=6)	193	165	386	0.033	2.30

El estudio de refinamiento (Figura 63) ilustra el crecimiento del costo: al reducir h de 5 a 1.2 mm, los GDL crecen de 276 a 3884 (factor ~ 14), mientras el skewness medio mejora de 0.136 a 0.049. La convergencia (Ec. 222) exige refinar hasta que la cantidad de interés se estabilice, no indefinidamente: refinar más allá de la convergencia solo incrementa el costo sin ganancia de precisión.

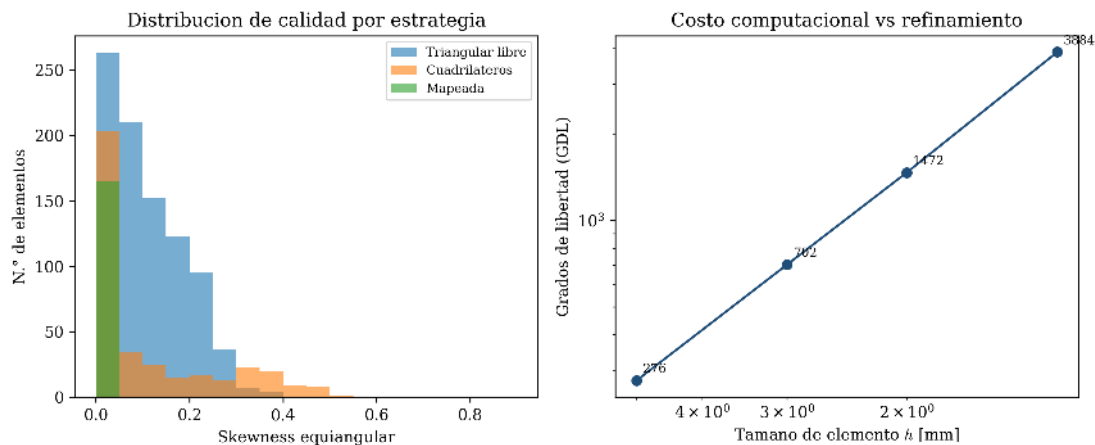


Figura 63: Izquierda: distribución de skewness por estrategia; la malla mapeada concentra todos sus elementos en calidad casi perfecta. Derecha: crecimiento de los GDL al refinar la malla (reducir h), en escala log–log.

8.7. Condiciones de frontera

Las condiciones de frontera (BC) cierran el problema (229) y lo vuelven no singular. En STATIC STRUCTURAL se distinguen dos clases:

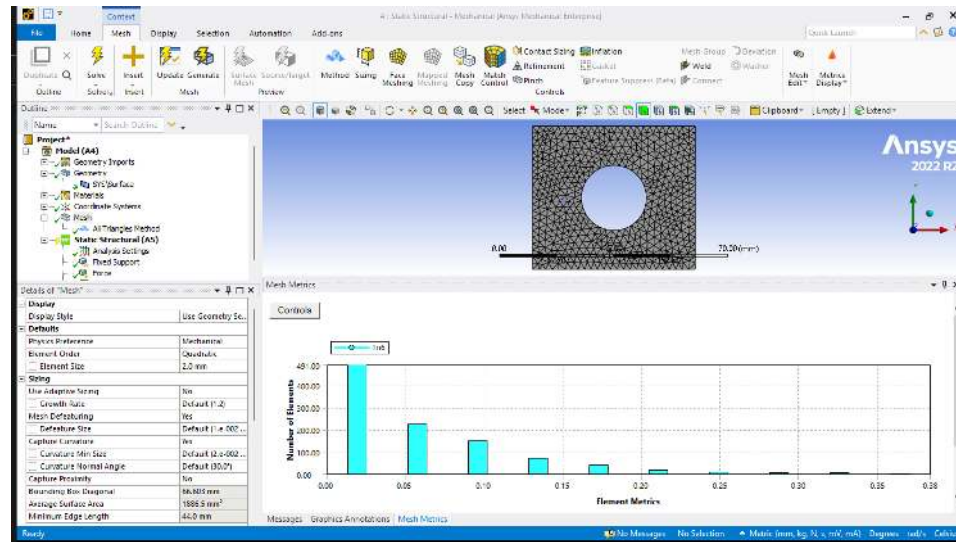


Figura 64: Selección del método de mallado en ANSYS (Mesh → Method). Las opciones corresponden a las estrategias de la Ec. 220 ilustradas en Python: libre (Tetrahedrons), mapeada (MultiZone) y barrida (Sweep).

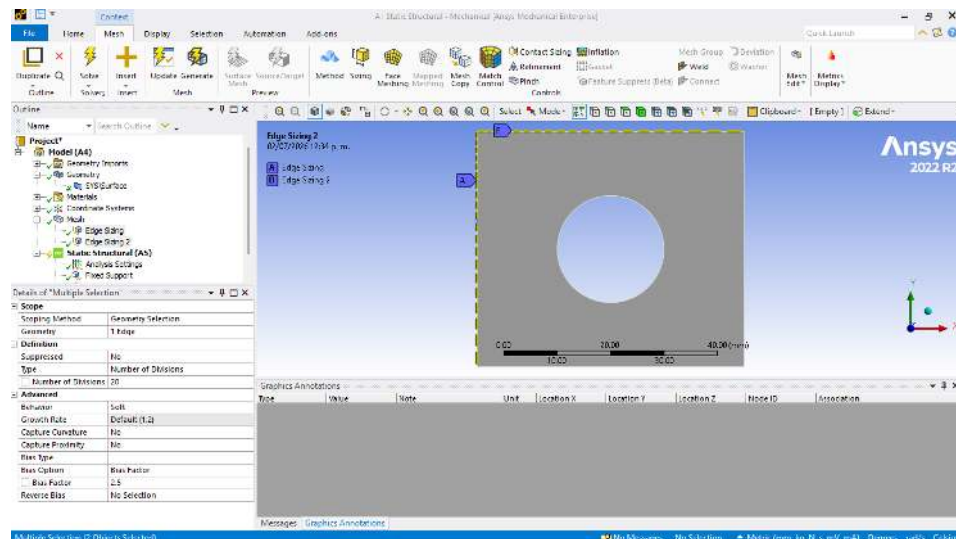


Figura 65: Control de sesgo en ANSYS (Edge Sizing → Bias Factor). El agrupamiento de nodos hacia el barreno reproduce la malla con sesgo de Figura 62.

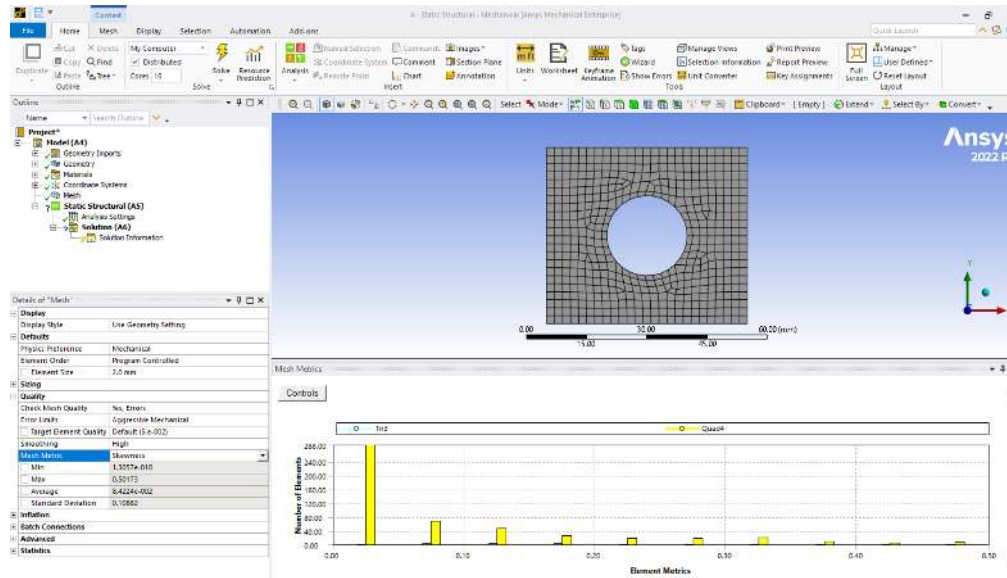


Figura 66: Evaluación de calidad en ANSYS (Mesh Metric). El gráfico de barras y los estadísticos son el análogo del histograma de skewness de Figura 63 y de Tabla 45.

- **Dirichlet (esenciales):** prescriben desplazamientos. En ANSYS: *Fixed Support* ($\mathbf{u} = \mathbf{0}$), *Displacement* (componentes prescritas), *Frictionless Support* (desplazamiento normal nulo, usado para simetría).
- **Neumann (naturales):** prescriben tracciones o fuerzas. En ANSYS: *Force*, *Pressure*, *Moment*. Se incorporan directamente en el vector $\mathbf{F}$ de (228) y no reducen GDL.

Una condición imprescindible es **eliminar los modos de cuerpo rígido**: en 2D hay tres (dos traslaciones y una rotación); en 3D, seis. Si no se restringen, $\mathbf{K}$ es singular y el solver falla. Las BC Dirichlet deben restringir al menos esos modos.

Advertencia: idealización de las BC. Las condiciones de frontera en FEA son siempre una idealización. En la realidad: (i) los apoyos no son puntos sino superficies de contacto con fricción; (ii) las cargas no son puntuales sino distribuidas sobre un área finita; (iii) puede haber holgura o desalineamiento. Estas diferencias son una fuente legítima de discrepancia experimento–modelo y deben documentarse. En el ensayo de flexión a tres puntos del curso, esto se traduce en modelar los apoyos como *pin* (restricción vertical y lateral) y *roller* (restricción vertical, libre axial), con la carga central como fuerza aplicada.

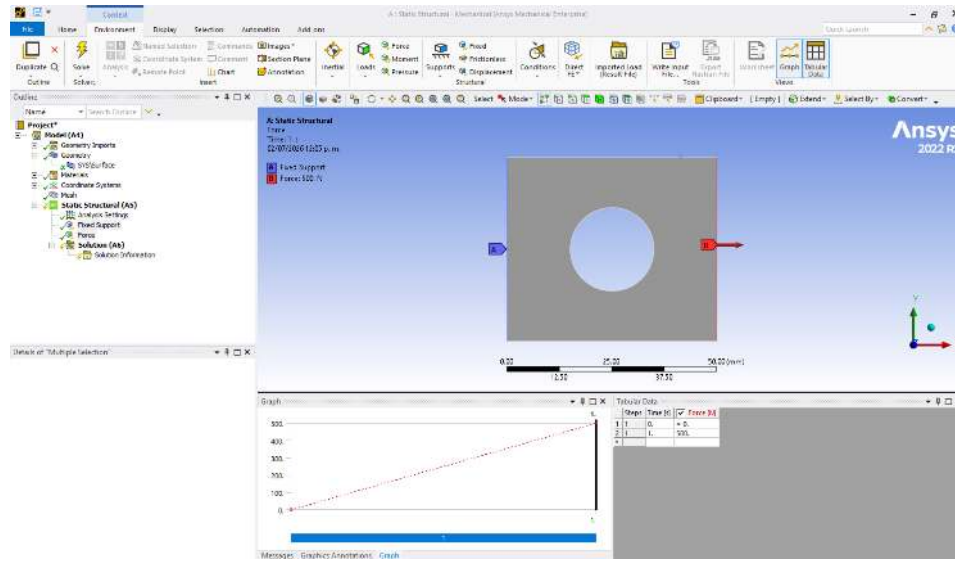


Figura 67: Imposición de condiciones de frontera en ANSYS. El *Fixed Support* restringe los modos de cuerpo rígido y la *Force* alimenta el vector $\mathbf{F}$ de la Ec. 229.

8.8. Flujo de trabajo en ANSYS Workbench – Static Structural

El módulo STATIC STRUCTURAL organiza el análisis en una secuencia de celdas (*Project Schematic*) que reproduce, en interfaz gráfica, los pasos formales de las secciones anteriores:

1. **Engineering Data:** definir el material (aquí Al 6061-T6: $E = 68.9$ GPa, $\nu = 0.33$) $\Rightarrow$ tensor $\mathbf{D}$ de (217).
2. **Geometry:** construir/importar la geometría (Figura 55) $\Rightarrow$ dominio Ω .
3. **Model (Mesh):** generar la malla y controlar método, sizing, bias y orden de elemento $\Rightarrow$ discretización Ω_h .
4. **Setup (Static Structural):** imponer BC (Figura 67) $\Rightarrow$ cerrar (229).
5. **Solution:** resolver $\mathbf{KU} = \mathbf{F} \Rightarrow$ desplazamientos.
6. **Results:** recuperar y visualizar ε , σ , deformación total.

8.9. Aplicación a los proyectos doctorales

Las decisiones de mallado y de tipo de elemento dependen de la geometría y la física de cada problema. La Tabla 46 vincula las familias de geometría presentes en los proyectos del grupo con la estrategia de mallado y las condiciones de frontera recomendadas en STATIC STRUCTURAL.

Tres criterios transversales:

- **Explotar simetría.** Losas y secciones periódicas admiten modelar solo una celda o cuadrante con *Frictionless Support* en los planos de simetría, reduciendo GDL

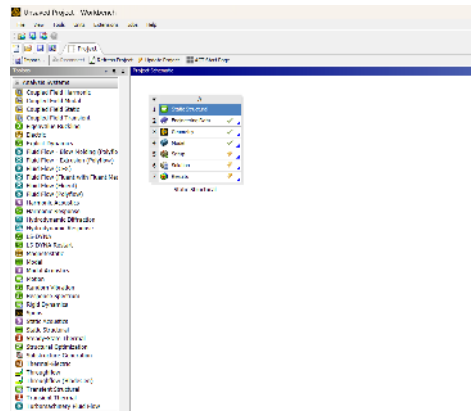


Figura 68: Esquema de proyecto de STATIC STRUCTURAL. Cada celda corresponde a un paso formal del FEM: material, dominio, malla, BC, solución y posproceso.

Tabla 46: Guía de mallado por familia de geometría/proyecto.

Familia de proyecto	Geometría típica	Malla recomendada	BC / carga
Soportes y fijaciones de sensores	Placa/brida con barreno	Mapeada + sesgo al barreno (o Quad)	Fixed Support + Force
Bancos de bombas centrífugas	Carcasa/voluta, brida de barrenos	Sweep en secciones de revolución	Presión interna + Fixed
Cables OPGW (sección transversal)	Múltiples hilos circulares (arcos)	Tetraédrica libre + refinamiento local	Tensión axial + térmica
Estructuras UAV	Placas con aligeramientos	Cáscara (SHELL181) + Quad	Empotramiento + carga distribuida
Losas reticulares (waffle)	Celda unitaria con casetones	Mapeada de celda + simetría	Frictionless (simetría) + presión

sin pérdida de exactitud.

- **Refinar donde hay gradiente.** Barrenos, filetes y cambios de sección concentran esfuerzos; ahí el sesgo o el *sizing* local (como en [Figura 62](#)) es obligatorio.
- **Elegir el orden del elemento.** Para fronteras curvas (secciones OPGW, voluta de bomba) los elementos cuadráticos capturan la geometría con menos elementos; para análisis no lineales, los lineales refinados suelen ser más económicos.

8.10. Anexo A: Código Python (Google Colab)

El script `Semana08_mallado_2D.py` genera la geometría, las cuatro mallas, las funciones de forma y las métricas de calidad reportadas en [Tabla 45](#). Requiere `gmsh`, `numpy` y `matplotlib`. En Google Colab, ejecutar primero:

```
!apt-get -qq install -y libglu1-mesa
!pip -q install gmsh
```

```
1 # -*- coding: utf-8 -*-
2 """
3 =====
4 Semana 8 - Metodo de Elementos Finitos I
5 Demostración de generación de mallas 2D con contorno curvo.
6 Curso: Prototipado y Simulación (DITA) - Universidad La Salle Bajío
7
8 Objetivo del script:
9     A partir de UNA geometría 2D con curvas (placa cuadrada con barreno
10     circular central, benchmark de concentración de esfuerzos), generar y
11     comparar las estrategias de mallado que ofrece un preprocesador tipo
12     ANSYS Workbench (Static Structural):
13         - Malla libre triangular (Frontal-Delaunay) -> Patch Conforming
14         - Malla de cuadriláteros (recombinación) -> Quad Dominant
15         - Malla mapeada estructurada (transfinita) -> Mapped / Sweep
16         - Malla mapeada con sesgo radial (bias) -> Bias Factor
17         - Elementos lineales vs cuadráticos (nodos/GDL)
18         - Funciones de forma  $N_i$ 
19         - Métricas de calidad (skewness, aspect ratio) y conteo de GDL
20
21 Ejecución en Google Colab:
22     !apt-get -qq install -y libglu1-mesa
23     !pip -q install gmsh
24     Luego ejecutar este archivo por celdas (marcadas con # %%).
25
26 Convenciones de reproducibilidad del curso:
27     matplotlib 'Agg', figuras 300 dpi en ./Figuras, semilla fija,
28     valores citados en el texto provienen de la corrida real.
29 =====
30 """
31
32 # %% Celda 0 - Configuración global -----
33 import os
34 import numpy as np
35 import matplotlib
36 matplotlib.use("Agg") # render sin pantalla (headless)
37 import matplotlib.pyplot as plt
```

```

38 from matplotlib.collections import PolyCollection, LineCollection
39 import gmsh
40
41 SEED = 8
42 rng = np.random.default_rng(SEED)          # reproducibilidad
43 os.makedirs("Figuras", exist_ok=True)
44
45 plt.rcParams.update({
46     "figure.dpi": 120, "savefig.dpi": 300,
47     "font.family": "serif", "font.size": 10,
48     "axes.grid": False, "mathtext.fontset": "cm",
49 })
50
51 # Parametros geometricos de la placa perforada [mm]
52 W = 50.0          # lado de la placa (dominio cuadrado)
53 A_HOLE = 10.0     # radio del barreno central
54 # Material de referencia (Al 6061-T6, usado en el hilo DIC-FEA del curso)
55 E_MAT, NU_MAT = 68.9e3, 0.33 # MPa, adimensional
56
57 RESULTS = {}      # acumulador de salidas numericas para el reporte final
58
59
60 # %% Celda 1 - Utilidades de extraccion y calidad de malla -----
61 def extract_mesh_2d():
62     """Devuelve coordenadas y conectividad (solo nodos de esquina) de la
63     malla 2D activa en gmsh, separando triangulos y cuadrilateros."""
64     ntags, ncoords, _ = gmsh.model.mesh.getNodes()
65     ncoords = np.array(ncoords).reshape(-1, 3)
66     tag2idx = {int(t): i for i, t in enumerate(ntags)}
67     xy = ncoords[:, :2]
68
69     tris, quads = [], []
70     etypes, _, enodes = gmsh.model.mesh.getElements(dim=2)
71     for et, nod in zip(etypes, enodes):
72         props = gmsh.model.mesh.getElementProperties(et)
73         nn = props[3]          # nodos por elemento
74         conn = np.array(nod).reshape(-1, nn)
75         corners = 3 if props[0].startswith("Triangle") else 4
76         for row in conn:
77             idx = [tag2idx[int(t)] for t in row[:corners]]
78             (tris if corners == 3 else quads).append(idx)
79     return xy, tris, quads
80
81
82 def polygon_angles(pts):
83     """Angulos internos (grados) de un poligono convexo ordenado."""
84     n = len(pts)
85     ang = []
86     for i in range(n):
87         a = pts[(i - 1) % n] - pts[i]
88         b = pts[(i + 1) % n] - pts[i]
89         c = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-30)
90         ang.append(np.degrees(np.arccos(np.clip(c, -1, 1))))
91     return np.array(ang)
92
93
94 def equiangular_skewness(pts, ideal):
95     """Skewness equiangular (0 = ideal, 1 = degenerado). ideal=60 tri, 90 quad."""
96     ang = polygon_angles(pts)
97     tmax, tmin = ang.max(), ang.min()

```

```

98     return max((tmax - ideal) / (180 - ideal), (ideal - tmin) / ideal)
99
100
101 def aspect_ratio(pts):
102     """Relacion de aspecto = arista mayor / arista menor."""
103     n = len(pts)
104     L = [np.linalg.norm(pts[(i + 1) % n] - pts[i]) for i in range(n)]
105     return max(L) / (min(L) + 1e-30)
106
107
108 def mesh_quality(xy, tris, quads):
109     """Estadísticos de skewness y aspect ratio de la malla."""
110     sk, ar = [], []
111     for e in tris:
112         p = xy[e]; sk.append(equiangular_skewness(p, 60)); ar.append(aspect_ratio(p))
113     for e in quads:
114         p = xy[e]; sk.append(equiangular_skewness(p, 90)); ar.append(aspect_ratio(p))
115     sk, ar = np.array(sk), np.array(ar)
116     return dict(n_nodes=len(xy), n_elem=len(tris) + len(quads),
117               n_tri=len(tris), n_quad=len(quads),
118               sk_max=sk.max(), sk_mean=sk.mean(),
119               ar_max=ar.max(), ar_mean=ar.mean(),
120               skew=sk, ndof=2 * len(xy))
121
122
123 def plot_mesh(xy, tris, quads, title, fname, color_by_skew=False):
124     fig, ax = plt.subplots(figsize=(5.2, 5.0))
125     polys, vals = [], []
126     for e in tris + quads:
127         p = xy[e]; polys.append(p)
128         if color_by_skew:
129             ideal = 60 if len(e) == 3 else 90
130             vals.append(equiangular_skewness(p, ideal))
131     if color_by_skew:
132         pc = PolyCollection(polys, array=np.array(vals), cmap="RdYlGn_r",
133                           edgecolors="k", linewidths=0.25)
134         pc.set_clim(0, 0.9)
135         ax.add_collection(pc)
136         cb = fig.colorbar(pc, ax=ax, fraction=0.046, pad=0.04)
137         cb.set_label("Skewness equiangular")
138     else:
139         pc = PolyCollection(polys, facecolors="#cfe3f7",
140                           edgecolors="#1f4e79", linewidths=0.4)
141         ax.add_collection(pc)
142     ax.set_aspect("equal"); ax.autoscale_view()
143     ax.set_title(title, fontsize=10)
144     ax.set_xlabel("x [mm]"); ax.set_ylabel("y [mm]")
145     fig.tight_layout(); fig.savefig(f"Figuras/{fname}.png"); plt.close(fig)
146
147
148 # %% Celda 2 - Geometria base: placa con barreno (contorno curvo) -----
149 def build_plate(lc, order=1, quad=False, algo=6):
150     """Placa WxW con barreno circular central de radio A_HOLE (kernel OCC).
151     lc: tamaño de elemento; order: 1/2; quad: recombinar; algo: alg. 2D."""
152     gmsh.initialize()
153     gmsh.option.setNumber("General.Terminal", 0)
154     gmsh.model.add("placa")
155     occ = gmsh.model.occ
156     rect = occ.addRectangle(-W/2, -W/2, 0, W, W)
157     disk = occ.addDisk(0, 0, 0, A_HOLE, A_HOLE)

```

```

158     occ.cut([(2, rect)], [(2, disk)])
159     occ.synchronize()
160     gmsh.option.setNumber("Mesh.MeshSizeMin", lc)
161     gmsh.option.setNumber("Mesh.MeshSizeMax", lc)
162     gmsh.option.setNumber("Mesh.Algorithm", algo)
163     if quad:
164         gmsh.option.setNumber("Mesh.RecombineAll", 1)
165         gmsh.option.setNumber("Mesh.RecombinationAlgorithm", 1) # Blossom
166     gmsh.option.setNumber("Mesh.ElementOrder", order)
167     gmsh.model.mesh.generate(2)
168     xy, tris, quads = extract_mesh_2d()
169     q = mesh_quality(xy, tris, quads)
170     gmsh.finalize()
171     return xy, tris, quads, q
172
173
174 # Geometria (croquis) para figura de referencia
175 def plot_geometry():
176     th = np.linspace(0, 2*np.pi, 200)
177     fig, ax = plt.subplots(figsize=(5.2, 5.0))
178     ax.plot([-W/2, W/2, W/2, -W/2, -W/2],
179            [-W/2, -W/2, W/2, W/2, -W/2], "-", color="#1f4e79", lw=1.6)
180     ax.plot(A_HOLE*np.cos(th), A_HOLE*np.sin(th), "-", color="#c0392b", lw=1.6)
181     ax.annotate("", xy=(W/2, -W/2-4), xytext=(-W/2, -W/2-4),
182               arrowprops=dict(arrowstyle="<->", color="k"))
183     ax.text(0, -W/2-7, f"W = {W:.0f} mm", ha="center")
184     ax.annotate("", xy=(A_HOLE, 0), xytext=(0, 0),
185               arrowprops=dict(arrowstyle="->", color="#c0392b"))
186     ax.text(A_HOLE/2, 1.5, f"a = {A_HOLE:.0f} mm", color="#c0392b")
187     ax.text(0, W/2+3, "Contorno curvo (barreno)", ha="center",
188           color="#c0392b", fontsize=9)
189     ax.set_aspect("equal"); ax.set_xlim(-W/2-10, W/2+10); ax.set_ylim(-W/2-12, W/2+8)
190     ax.set_xlabel("x [mm]"); ax.set_ylabel("y [mm]")
191     ax.set_title("Geometria base: placa con barreno (benchmark $K_t$)", fontsize=10)
192     fig.tight_layout(); fig.savefig("Figuras/s8_geometria.png"); plt.close(fig)
193
194
195 plot_geometry()
196
197 # Malla libre triangular
198 xy, t, q, Q_TRI = build_plate(lc=2.5, order=1, quad=False, algo=6)
199 plot_mesh(xy, t, q, "Malla libre triangular (Frontal-Delaunay)", "s8_malla_libre_tri")
200 RESULTS["tri"] = Q_TRI
201
202 # Malla de cuadrilateros (recombinada)
203 xy, t, q, Q_QUAD = build_plate(lc=2.5, order=1, quad=True, algo=8)
204 plot_mesh(xy, t, q, "Malla de cuadrilateros (recombinacion Blossom)", "s8_malla_quad")
205 RESULTS["quad"] = Q_QUAD
206
207
208 # %% Celda 3 - Malla mapeada (transfinita) y sesgo (bias) -----
209 def bias_to_progression(bias_factor, n_nodes):
210     """Convierte el Bias Factor de ANSYS (razon arista mayor/menor total)
211     al coeficiente de Progression de gmsh (razon entre elementos
212     consecutivos). n_nodes = nodos sobre la arista -> (n_nodes-1) elementos.
213     ANSYS: BF = r^(nEl-1) => r = BF^(1/(nEl-1))."""
214     n_el = n_nodes - 1
215     if bias_factor <= 1.0 or n_el < 2:
216         return 1.0
217     return bias_factor ** (1.0 / (n_el - 1))

```

```

218
219
220 def build_annular_sector(nr, nt, order=1, bias=1.0, b=W/2):
221     """Sector anular de 90 grados entre r=A_HOLE y r=b, malla transfinita
222     (estructurada) de cuadrilateros. bias = Bias Factor estilo ANSYS
223     (razon arista mayor/menor). Se convierte a Progression de gmsh."""
224     r_prog = bias_to_progression(bias, nr)
225     gmsh.initialize()
226     gmsh.option.setNumber("General.Terminal", 0)
227     gmsh.model.add("anillo")
228     geo = gmsh.model.geo
229     a = A_HOLE
230     c = geo.addPoint(0, 0, 0)
231     p1 = geo.addPoint(a, 0, 0)      # inicio arco interno
232     p2 = geo.addPoint(0, a, 0)      # fin arco interno
233     p3 = geo.addPoint(0, b, 0)      # fin arco externo
234     p4 = geo.addPoint(b, 0, 0)      # inicio arco externo
235     l_rad1 = geo.addLine(p1, p4)    # radial inferior (a -> b)
236     a_ext = geo.addCircleArc(p4, c, p3) # arco externo
237     l_rad2 = geo.addLine(p3, p2)    # radial superior (b -> a)
238     a_int = geo.addCircleArc(p2, c, p1) # arco interno
239     cl = geo.addCurveLoop([l_rad1, a_ext, l_rad2, a_int])
240     s = geo.addPlaneSurface([cl])
241     geo.synchronize()
242     # Transfinito: nt nodos circunferenciales, nr radiales con progresion=bias
243     for arc in (a_ext, a_int):
244         geo.mesh.setTransfiniteCurve(arc, nt)
245     geo.mesh.setTransfiniteCurve(l_rad1, nr, "Progression", r_prog)
246     geo.mesh.setTransfiniteCurve(l_rad2, nr, "Progression", -r_prog) # sentido inverso
247     geo.mesh.setTransfiniteSurface(s)
248     geo.mesh.setRecombine(2, s)    # cuadrilateros
249     geo.synchronize()
250     gmsh.option.setNumber("Mesh.ElementOrder", order)
251     gmsh.model.mesh.generate(2)
252     xy, tris, quads = extract_mesh_2d()
253     Q = mesh_quality(xy, tris, quads)
254     gmsh.finalize()
255     return xy, tris, quads, Q
256
257
258 # Mapeada uniforme
259 xy, t, q, Q_MAP = build_annular_sector(nr=12, nt=16, bias=1.0)
260 plot_mesh(xy, t, q, "Malla mapeada estructurada (transfinita, bias=1.0)",
261           "s8_malla_mapeada")
262 RESULTS["map"] = Q_MAP
263
264 # Mapeada con sesgo radial -> comparacion 2 paneles
265 xyb, tb, qb, Q_BIAS = build_annular_sector(nr=12, nt=16, bias=6.0)
266 RESULTS["bias"] = Q_BIAS
267
268 fig, axs = plt.subplots(1, 2, figsize=(9.4, 4.6))
269 for ax, (X, TT, QQ, ttl) in zip(
270     axs, [(xy, t, q, "bias = 1.0 (uniforme)",
271            (xyb, tb, qb, "bias = 6.0 (denso hacia el barreno))]):
272     polys = [X[e] for e in TT + QQ]
273     ax.add_collection(PolyCollection(polys, facecolors="#cfe3f7",
274                                     edgecolors="#1f4e79", linewidths=0.4))
275     ax.set_aspect("equal"); ax.autoscale_view()
276     ax.set_title(ttl, fontsize=10); ax.set_xlabel("x [mm]"); ax.set_ylabel("y [mm]")
277 fig.suptitle("Efecto del sesgo (Bias Factor) sobre la distribucion radial de nodos",

```

```

278         fontsize=11)
279 fig.tight_layout(); fig.savefig("Figuras/s8_malla_bias.png"); plt.close(fig)
280
281
282 # %% Celda 4 - Elementos: nodos y grados de libertad (lineal vs cuadrático) --
283 def draw_element(ax, corners, mids, title):
284     poly = plt.Polygon(corners, closed=True, fill=True,
285                        facecolor="#eef3fb", edgecolor="#1f4e79", lw=1.5)
286     ax.add_patch(poly)
287     ax.scatter(*corners.T, s=70, color="#1f4e79", zorder=3, label="nodo esquina")
288     for i, p in enumerate(corners):
289         ax.annotate(f"{i+1}", p, textcoords="offset points",
290                    xytext=(6, 6), fontsize=9, color="#1f4e79")
291     if len(mids):
292         ax.scatter(*np.array(mids).T, s=55, marker="s", color="#c0392b",
293                    zorder=3, label="nodo intermedio")
294     ndof = 2 * (len(corners) + len(mids))
295     ax.set_title(f"{title}\n{len(corners)+len(mids)} nodos, {ndof} GDL", fontsize=9)
296     ax.set_aspect("equal"); ax.axis("off")
297
298
299 tri = np.array([[0, 0], [1, 0], [0.0, 1.0]])
300 quad = np.array([[0, 0], [1, 0], [1, 1], [0, 1]])
301 def edge_mids(c):
302     return [ (c[i] + c[(i+1) % len(c)]) / 2 for i in range(len(c)) ]
303
304 fig, axs = plt.subplots(1, 4, figsize=(11.5, 3.4), constrained_layout=True)
305 draw_element(axs[0], tri, [], "Tri3 (lineal)")
306 draw_element(axs[1], tri, edge_mids(tri), "Tri6 (cuadrático)")
307 draw_element(axs[2], quad, [], "Quad4 (lineal)")
308 draw_element(axs[3], quad, edge_mids(quad), "Quad8 (cuadrático)")
309 for ax in axs:
310     ax.margins(0.2)
311     axs[1].legend(loc="lower center", bbox_to_anchor=(1.05, -0.22),
312                  ncol=2, fontsize=9, frameon=False)
313 fig.suptitle("Nodos y grados de libertad por elemento "
314             "(estructural 2D: $u_x, u_y$ por nodo)", fontsize=10)
315 fig.savefig("Figuras/s8_elementos_dof.png"); plt.close(fig)
316
317
318 # %% Celda 5 - Funciones de forma $N_i$ -----
319 xi = np.linspace(-1, 1, 200)
320 # 1D lineal
321 N1L, N2L = (1 - xi) / 2, (1 + xi) / 2
322 # 1D cuadrático (Lagrange)
323 N1Q = xi * (xi - 1) / 2
324 N2Q = (1 - xi**2)
325 N3Q = xi * (xi + 1) / 2
326 # 2D bilineal $Q_4$: $N_1 = 1/4 (1-xi)(1-eta)$
327 g = np.linspace(-1, 1, 60); XX, YY = np.meshgrid(g, g)
328 N1_2D = 0.25 * (1 - XX) * (1 - YY)
329
330 fig = plt.figure(figsize=(11, 3.4))
331 ax1 = fig.add_subplot(1, 3, 1)
332 ax1.plot(xi, N1L, label="$N_1$"); ax1.plot(xi, N2L, label="$N_2$")
333 ax1.set_title("Lineal 1D"); ax1.set_xlabel(r"$xi$"); ax1.legend(fontsize=8)
334 ax1.axhline(0, color="k", lw=0.4); ax1.axhline(1, color="k", lw=0.3, ls=":")
335 ax2 = fig.add_subplot(1, 3, 2)
336 for N, lb in [(N1Q, "$N_1$"), (N2Q, "$N_2$"), (N3Q, "$N_3$")]:
337     ax2.plot(xi, N, label=lb)

```



```

338 ax2.set_title("Cuadratica 1D"); ax2.set_xlabel(r"$\xi$"); ax2.legend(fontsize=8)
339 ax2.axhline(0, color="k", lw=0.4)
340 ax3 = fig.add_subplot(1, 3, 3, projection="3d")
341 ax3.plot_surface(XX, YY, N1_2D, cmap="viridis", edgecolor="none")
342 ax3.set_title(r"Bilineal Q4: $N_1(\xi, \eta)$")
343 ax3.set_xlabel(r"$\xi$"); ax3.set_ylabel(r"$\eta$")
344 fig.suptitle("Funciones de forma: particion de la unidad $\sum_i N_i = 1$",
345             fontsize=10)
346 fig.tight_layout(); fig.savefig("Figuras/s8_funciones_forma.png"); plt.close(fig)
347
348 # Verificacion numerica: particion de la unidad
349 part_unity_1D = np.allclose(N1Q + N2Q + N3Q, 1.0)
350 part_unity_2D = np.isclose(N1_2D.sum() / # promedio ~0.25 en Q4 -> suma total nodos=1
351                             N1_2D.size, 0.25, atol=1e-9)
352 RESULTS["part_unity"] = bool(part_unity_1D)
353
354
355 # %% Celda 6 - Convergencia (conteo de GDL y calidad) -----
356 lcs = [5.0, 3.0, 2.0, 1.2]
357 conv = []
358 for lc in lcs:
359     _, _, _, Qc = build_plate(lc=lc, order=1, quad=False, algo=6)
360     conv.append((lc, Qc["n_nodes"], Qc["n_elem"], Qc["ndof"], Qc["sk_mean"]))
361 RESULTS["conv"] = conv
362
363 # Histograma de skewness por estrategia + curva GDL vs lc
364 fig, axs = plt.subplots(1, 2, figsize=(10, 4.2))
365 bins = np.linspace(0, 0.9, 19)
366 axs[0].hist(RESULTS["tri"]["skew"], bins, alpha=0.6, label="Triangular libre")
367 axs[0].hist(RESULTS["quad"]["skew"], bins, alpha=0.6, label="Cuadrilateros")
368 axs[0].hist(RESULTS["map"]["skew"], bins, alpha=0.6, label="Mapeada")
369 axs[0].set_xlabel("Skewness equiangular"); axs[0].set_ylabel("N.\u00b0 de elementos")
370 axs[0].set_title("Distribucion de calidad por estrategia"); axs[0].legend(fontsize=8)
371 lc_arr = [c[0] for c in conv]; dof_arr = [c[3] for c in conv]
372 axs[1].loglog(lc_arr, dof_arr, "o-", color="#1f4e79")
373 axs[1].invert_xaxis()
374 axs[1].set_xlabel("Tamano de elemento $h$ [mm]")
375 axs[1].set_ylabel("Grados de libertad (GDL)")
376 axs[1].set_title("Costo computacional vs refinamiento")
377 for lc, dof in zip(lc_arr, dof_arr):
378     axs[1].annotate(f"{dof}", (lc, dof), textcoords="offset points",
379                    xytext=(4, 5), fontsize=8)
380 fig.tight_layout(); fig.savefig("Figuras/s8_calidad.png"); plt.close(fig)
381
382
383 # %% Celda 7 - Reporte numerico (valores para el documento LaTeX) -----
384 def line(): print("-" * 70)
385 line(); print("SEMANA 8 - RESULTADOS NUMERICOS DE MALLADO"); line()
386 print(f"Geometria: placa {W:.0f}x{W:.0f} mm, barreno r={A_HOLE:.0f} mm")
387 print(f"Material de referencia: A1 6061-T6, E={E_MAT/1e3:.1f} GPa, nu={NU_MAT}")
388 line()
389 hdr = (f"{'Estrategia':<22}{ 'Nodos':>7}{ 'Elem':>7}{ 'GDL':>7}"
390        f"{'Sk_max':>9}{ 'Sk_med':>9}{ 'AR_max':>10}")
391 print(hdr)
392 for key, name in [("tri", "Triangular libre"), ("quad", "Cuadrilateros"),
393                  ("map", "Mapeada uniforme"), ("bias", "Mapeada + bias 6.0")]:
394     Q = RESULTS[key]
395     print(f"{'name':<22}{Q['n_nodes']:>7}{Q['n_elem']:>7}{Q['ndof']:>7}"
396           f"{'Q['sk_max']:>9.3f}{Q['sk_mean']:>9.3f}{Q['ar_max']:>10.2f}")
397 line()

```

```

398 print("Estudio de refinamiento (malla triangular libre):")
399 print(f"{'h [mm]':>8}{'Nodos':>8}{'Elem':>8}{'GDL':>8}{'Sk_med':>9}")
400 for lc, nn, ne, nd, skm in conv:
401     print(f"{'lc':>8.1f}{'nn':>8}{'ne':>8}{'nd':>8}{'skm':>9.3f}")
402 line()
403 print(f"Funciones de forma 1D cuadraticas, particion de la unidad: {part_unity_1D}")
404 print(f"Suma N_i(Q4) consistente (=1 por nodo): {part_unity_2D}")
405 line()
406 print("Figuras generadas en ./Figuras :")
407 for f in sorted(os.listdir("Figuras")):
408     print("    ", f)
409 line()
    
```

Listing 49: Generación y comparación de mallas 2D (Semana 8).

8.11. Anexo B: Referencias

1. Zienkiewicz, O. C., Taylor, R. L., Zhu, J. Z. (2013). *The Finite Element Method: Its Basis and Fundamentals*, 7.^a ed. Butterworth-Heinemann. DOI: 10.1016/C2009-0-24909-9.
2. Hughes, T. J. R. (2000). *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover.
3. Bathe, K. J. (2014). *Finite Element Procedures*, 2.^a ed. Prentice Hall.
4. Cook, R. D., Malkus, D. S., Plesha, M. E., Witt, R. J. (2001). *Concepts and Applications of Finite Element Analysis*, 4.^a ed. Wiley.
5. Geuzaine, C., Remacle, J.-F. (2009). Gmsh: A three-dimensional finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11), 1309–1331. DOI: 10.1002/nme.2579.
6. ANSYS, Inc. (2025). *Meshing User's Guide y Mechanical APDL Modeling and Meshing Guide*, Release 2025 R2.

8.12. Ejercicios

1. **(Convergencia, 30 %)** Sobre la geometría de la placa con barreno, ejecute el script del Anexo A con al menos cinco tamaños de elemento h y reporte la evolución de los GDL y del skewness medio. Discuta a partir de qué h se satisface el criterio de la Ec. 222 suponiendo que la cantidad de interés converge como h^2 .
2. **(Estrategia de mallado, 30 %)** Elija la familia de geometría de su proyecto (Tabla 46) y justifique, con las métricas de Tabla 44, qué método de mallado y qué orden de elemento usaría. Genere en Python una malla representativa y reporte skewness máximo y razón de aspecto.
3. **(ANSYS, 40 %)** Reproduzca en STATIC STRUCTURAL el modelo de la placa con

barreno: material Al 6061-T6, malla con *Mapped Face Meshing* y *Bias Factor* hacia el barreno, *Fixed Support* en una arista y *Force* en la opuesta. Entregue las capturas correspondientes a Figuras 64 to 67 y compare el histograma de skewness de ANSYS con el de Figura 63.

Tabla 47: Rúbrica de los ejercicios de la Semana 8.

Criterio	Peso	Nivel esperado
Rigor numérico	30 %	Valores provenientes de corridas reales; unidades y convergencia documentadas
Justificación de mallado	30 %	Elección coherente con métricas de calidad y física del problema
Ejecución en ANSYS	30 %	Modelo correcto, BC no singulares, capturas completas
Comunicación técnica	10 %	Redacción precisa, figuras y tablas referenciadas

Rúbrica de evaluación.

9. Semanas 9 y 10. Método de Elementos Finitos II: Análisis Estructural, Fatiga, Vibraciones y Dinámica de Mecanismos

Objetivo de la cátedra. Aplicar el método de elementos finitos a problemas mecánicos de ingeniería mediante ANSYS Workbench: calcular esfuerzos y deformaciones en componentes cargados estáticamente, estimar vida a fatiga con el método esfuerzo–vida, obtener frecuencias naturales y modos de vibración, simular la cinemática de mecanismos acoplados con *Rigid Dynamics* (posición, velocidad y aceleración), y acoplar ambos mundos: extraer las cargas dinámicas de un mecanismo biela–manivela–corredora y calcular los esfuerzos resultantes en la biela. Cada resultado numérico de ANSYS se contrasta con una solución analítica de referencia, calculada con los códigos del Anexo.

Vínculo con los proyectos de investigación del grupo. El análisis estático y de fatiga es directamente aplicable al banco de pruebas de bombas centrífugas (soportes, bridas y flechas sometidas a cargas cíclicas), al análisis electromecánico de cables OPGW (esfuerzos en herrajes y puntos de sujeción) y a la optimización estructural de losas (interpretación de campos de esfuerzo y factores de seguridad). El análisis modal es la herramienta de entrada para monitoreo estructural y para evitar resonancia en estructuras portantes de UAV y en bancos rotodinámicos. La dinámica de cuerpos rígidos con extracción de cargas es el procedimiento estándar para dimensionar eslabones de mecanismos en prototipos electromecánicos e instrumentos de calibración con partes móviles.

9.1. De la formulación matemática al análisis ingenieril en Workbench

En la Semana 8 se construyó el FEM desde sus fundamentos: forma débil, funciones de forma, ensamble de $\mathbf{K}$ y aplicación de condiciones de frontera. Esta semana el énfasis cambia: el objetivo ya no es *programar* el método sino *usarlo correctamente* como herramienta de ingeniería, sabiendo en todo momento qué ecuación se está resolviendo detrás de cada botón de ANSYS Workbench. La regla doctoral de esta semana es simple: **ningún resultado de simulación se acepta sin una estimación analítica independiente que lo respalde en orden de magnitud.**

Los cuatro sistemas de análisis que se usarán, y la ecuación gobernante que cada uno resuelve, se resumen en la Tabla 48.

Tabla 48: Sistemas de análisis de ANSYS Workbench empleados en la Semana 9 y ecuación matricial gobernante. **K**: rigidez; **M**: masa; **u**: desplazamientos nodales; **F**: cargas; **q**: coordenadas generalizadas del mecanismo.

Sistema (Workbench)	Ecuación gobernante	Incógnita principal	Resultados típicos
Static Structural	$\mathbf{K} \mathbf{u} = \mathbf{F}$	Desplazamientos nodales	Deformación total, esfuerzo de von Mises, factor de seguridad
Static Structural + Fatigue Tool Modal	Posproceso sobre $\sigma(\mathbf{x})$ con curva S-N $(\mathbf{K} - \omega_i^2 \mathbf{M}) \phi_i = \mathbf{0}$	Vida N y daño por nodo Frecuencias ω_i y modos ϕ_i	Life, Damage, Safety Factor, Biaxiality Frecuencias naturales, formas modales, participación
Rigid Dynamics	$\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} = \mathbf{Q}(\mathbf{q}, \dot{\mathbf{q}}, t) +$ restricciones de junta	Movimiento de cuerpos rígidos	Posición, velocidad, aceleración, fuerzas de junta

Todos los sistemas comparten el mismo flujo de trabajo de cinco celdas de la Figura 69. Conviene memorizarlo: cada ejemplo guiado de esta semana es una instancia de este flujo con opciones distintas.

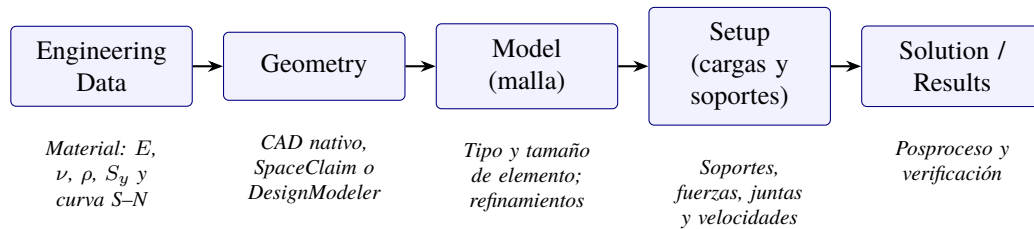


Figura 69: Flujo general de trabajo en ANSYS Workbench. Las cinco celdas aparecen en todo sistema de análisis; lo que cambia entre *Static Structural*, *Modal* y *Rigid Dynamics* es el contenido de *Setup* y los resultados disponibles.

9.2. Análisis estructural estático: esfuerzos y deformaciones

9.2.1. Fundamento: de los desplazamientos al factor de seguridad

El sistema *Static Structural* resuelve $\mathbf{K} \mathbf{u} = \mathbf{F}$ bajo la hipótesis de equilibrio estático (las cargas se aplican tan lentamente que las fuerzas de inercia son despreciables) y, por defecto, de elasticidad lineal. A partir de los desplazamientos nodales, el posprocesador reconstruye las deformaciones mediante el operador gradiente discreto y los esfuerzos mediante la ley de Hooke generalizada:

$$\boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u}^{(e)}, \quad \boldsymbol{\sigma} = \mathbf{D} \boldsymbol{\varepsilon}, \quad (230)$$

donde **B** contiene las derivadas de las funciones de forma (Semana 8) y **D** es la matriz constitutiva del material. Los esfuerzos se calculan primero en los puntos de integración de Gauss (donde son más exactos) y después se extrapolan y promedian en los nodos; por eso los contornos «Averaged» y «Unaveraged» difieren, y esa diferencia es en sí misma un indicador de calidad de malla.

Para materiales dúctiles el criterio de falla estándar es el de la energía de distorsión (von Mises). El esfuerzo equivalente se calcula a partir de los esfuerzos principales $\sigma_1 \geq \sigma_2 \geq \sigma_3$:

$$\sigma_{vM} = \sqrt{\frac{1}{2} [(\sigma_1 - \sigma_2)^2 + (\sigma_2 - \sigma_3)^2 + (\sigma_3 - \sigma_1)^2]}, \quad (231)$$

y el factor de seguridad contra fluencia que reporta la herramienta *Stress Tool* es

$$n_s = \frac{S_y}{\sigma_{vM}}, \quad (232)$$

evaluado nodo a nodo. Para materiales frágiles debe cambiarse el criterio (esfuerzo principal máximo o Mohr–Coulomb); ANSYS lo permite en las opciones del *Stress Tool*, y usar von Mises en hierro gris o cerámicos es un error conceptual, no numérico.

9.2.2. Flujo de trabajo en Static Structural

La Figura 70 detalla el procedimiento completo con las opciones relevantes de cada paso. Dos advertencias estructuran todo el análisis:

1. **Singularidades de esfuerzo.** En esquinas entrantes vivas, cargas puntuales y soportes puntuales, el esfuerzo elástico teórico es infinito: refinar la malla no converge, sino que crece sin cota. La deformación sí converge. Regla práctica: el esfuerzo máximo solo es creíble si ocurre en una zona con radio de acuerdo modelado y si al refinar la malla local el valor cambia menos de 5 % entre dos refinamientos consecutivos.
2. **Convergencia de malla.** Reportar un esfuerzo sin estudio de convergencia es inaceptable a nivel doctoral. El procedimiento mínimo: tres mallas con tamaño de elemento en razón ≈ 1.5 en la zona de interés, tabla de $\sigma_{vM}^{\max}$ contra número de nodos, y estimación del valor asintótico (la extrapolación de Richardson de la Semana 3 aplica directamente).

9.2.3. Ejemplo guiado 1: placa con agujero central a tensión

Este es el caso de referencia clásico porque posee solución de concentración de esfuerzos tabulada. Una placa de acero estructural de $60 \times 200 \times 5$ mm con un agujero central de diámetro $d = 15$ mm se somete a una fuerza axial de tensión $F = 12$ kN

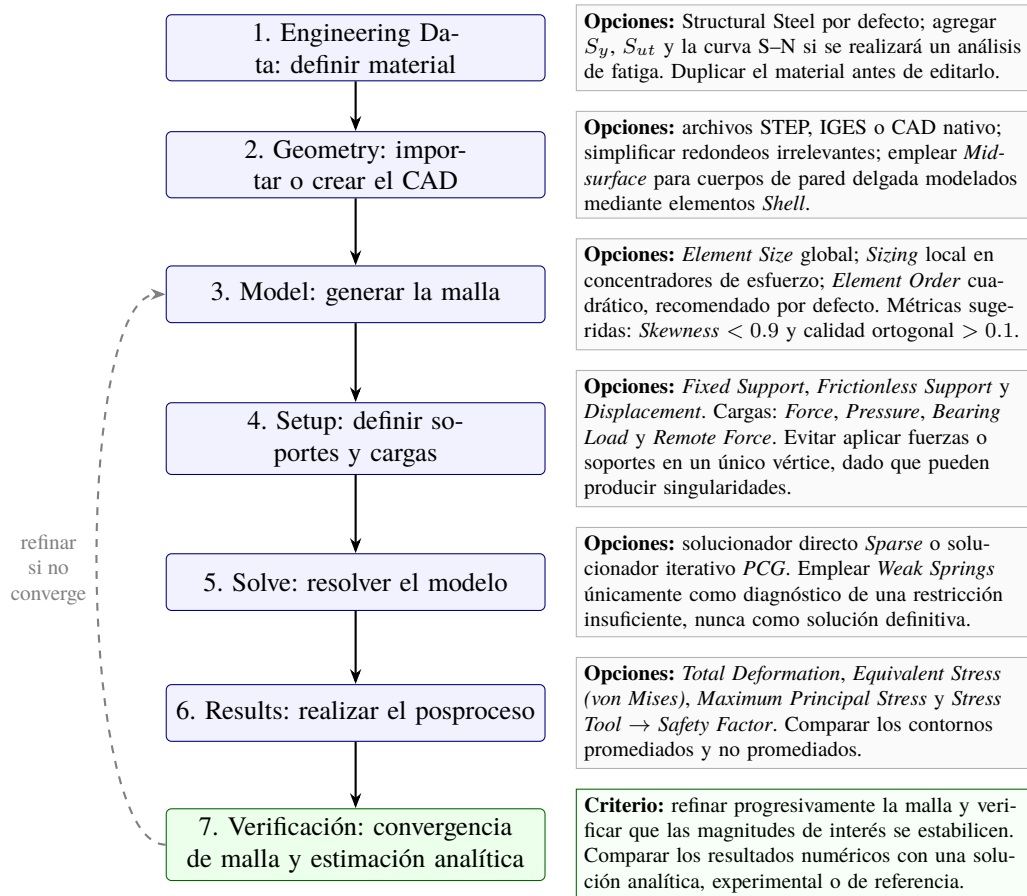


Figura 70: Flujo de trabajo detallado de un análisis *Static Structural* en ANSYS Workbench, con las opciones relevantes de cada etapa. El lazo situado a la izquierda representa el proceso iterativo requerido para verificar la convergencia de la malla.

aplicada en la cara corta, con la cara opuesta empotrada. Los parámetros se resumen en la Tabla 49.

Tabla 49: Parámetros del Ejemplo guiado 1. La referencia analítica proviene del código del Anexo (Script 1) y usa el ajuste polinomial de Howland–Heywood para el factor de concentración referido a la sección neta.

Parámetro	Símbolo	Valor
Ancho de la placa	W	60 mm
Espesor	t	5 mm
Diámetro del agujero	d	15 mm
Fuerza de tensión	F	12 kN
Límite de fluencia (acero estructural)	S_y	250 MPa
Esfuerzo nominal en sección neta	σ_{neta}	53.33 MPa
Factor de concentración ($d/W = 0.25$)	K_t	2.420
Esfuerzo máximo esperado	$K_t \sigma_{\text{neta}}$	129.1 MPa
Factor de seguridad esperado	$S_y / (K_t \sigma_{\text{neta}})$	1.94

El esfuerzo nominal se define en la sección que atraviesa el agujero, $\sigma_{\text{neta}} = F / [(W - d) t]$, y el factor de concentración referido a esa sección se aproxima, para $0 < d/W < 0.7$, por

$$K_t = 3 - 3.14 \left(\frac{d}{W} \right) + 3.667 \left(\frac{d}{W} \right)^2 - 1.527 \left(\frac{d}{W} \right)^3, \quad (233)$$

que para $d/W = 0.25$ da $K_t = 2.420$ (Figura 71). El valor de *Equivalent Stress* que ANSYS debe reportar en el borde del agujero, con malla convergida, es por tanto $\sigma_{\text{VM}}^{\text{max}} \approx 129$ MPa con factor de seguridad ≈ 1.9 . Si el estudiante obtiene 90 MPa o 200 MPa, el problema está en la malla, en las unidades o en las condiciones de frontera, no en la física.

Procedimiento en ANSYS (los pasos numerados corresponden a la Figura 70):

1. Crear un sistema *Static Structural*; en *Engineering Data* verificar que el material asignado sea *Structural Steel* ($E = 200$ GPa, $\nu = 0.3$, $S_y = 250$ MPa).
2. En *Geometry* (SpaceClaim o DesignModeler) dibujar el rectángulo de 200×60 mm, el círculo central de $\varnothing 15$ mm, y extruir 5 mm.
3. En *Model*, aplicar *Element Size* global de 4 mm y un *Sizing* local de 1 mm sobre la cara del agujero. Verificar que el orden del elemento sea cuadrático (*Element Order: Program Controlled* usa cuadrático en sólidos).
4. En *Setup: Fixed Support* en la cara corta izquierda; *Force* de 12 000 N en la cara corta derecha, dirección axial.
5. Resolver e insertar *Total Deformation*, *Equivalent Stress* y *Stress Tool* $\rightarrow$ *Safety*

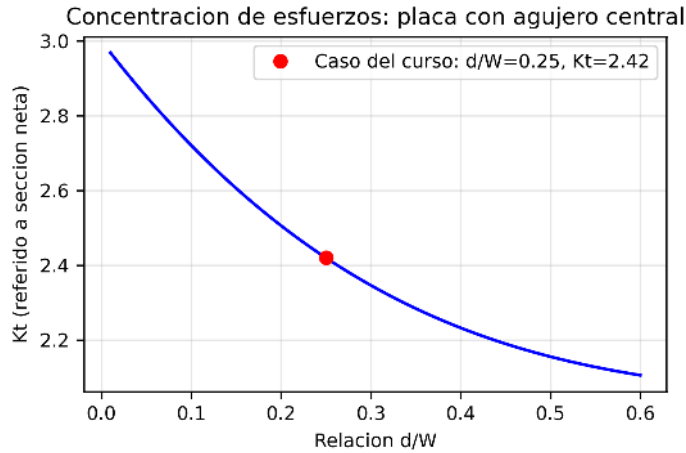


Figura 71: Factor de concentración de esfuerzos K_t (referido a la sección neta) de una placa finita con agujero central a tensión, Ec. 233. El punto rojo marca el caso del Ejemplo guiado 1. Generada con el Script 1 del Anexo.

Factor.

6. Repetir con *Sizing* local de 0.6 mm y de 0.4 mm; registrar la tabla de convergencia.

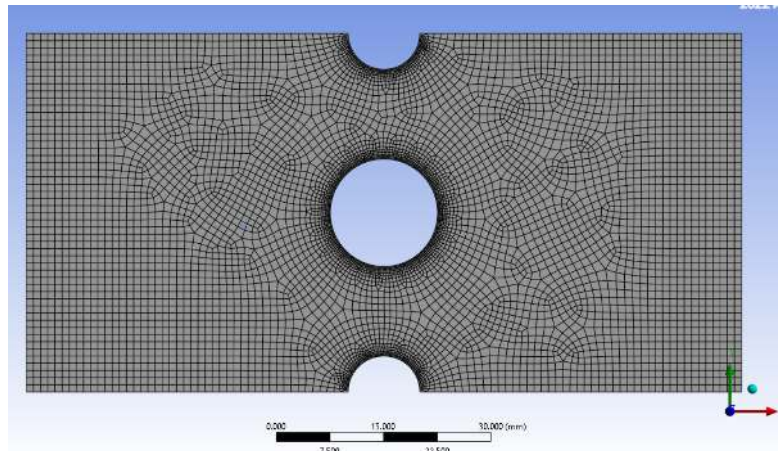


Figura 72: Malla del Ejemplo guiado 1 con refinamiento local de 1 mm alrededor del agujero. Insertar aquí la vista de Mesh con el detalle del concentrador

Interpretación doctoral. El máximo aparece en los puntos del borde del agujero perpendiculares a la carga, como predice la solución de Kirsch. La deformación total máxima es del orden de centésimas de milímetro: verificar que la razón deformación/longitud sea $\ll 1$ valida la hipótesis de linealidad geométrica. Si la geometría tuviera esquinas vivas en vez de agujero circular, el estudio de convergencia divergiría: esa es la firma numérica de una singularidad.

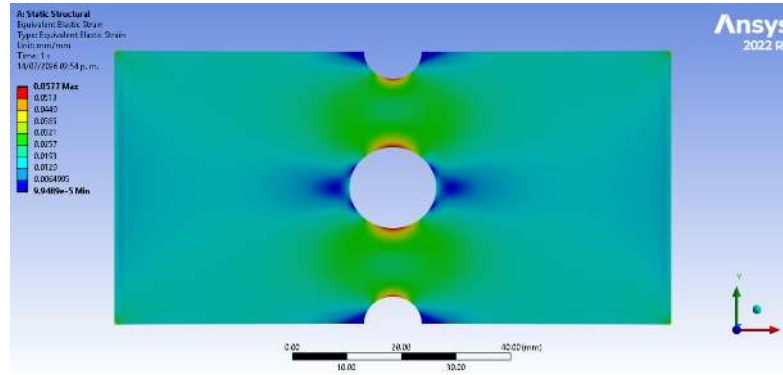


Figura 73: Contorno de esfuerzo equivalente de von Mises. El máximo debe ubicarse en el borde del agujero, a 90 grados de la dirección de carga.

9.3. Análisis de fatiga: del esfuerzo a la vida

9.3.1. Fundamento: los dos enfoques y la corrección por esfuerzo medio

La falla por fatiga ocurre bajo cargas cíclicas con esfuerzos máximos muy inferiores a S_y . Un ciclo de esfuerzo se caracteriza por su amplitud y su valor medio,

$$\sigma_a = \frac{\sigma_{\max} - \sigma_{\min}}{2}, \quad \sigma_m = \frac{\sigma_{\max} + \sigma_{\min}}{2}, \quad R = \frac{\sigma_{\min}}{\sigma_{\max}}, \quad (234)$$

donde $R = -1$ es carga completamente invertida (la condición de los ensayos S–N estándar) y $R = 0$ es carga pulsante. Existen dos familias de métodos, y elegir entre ellas es la primera decisión del *Fatigue Tool*:

- **Esfuerzo–vida (Stress-Life, S–N).** Adecuado para fatiga de alto ciclaje (HCF, $N \gtrsim 10^3$ – 10^4 ciclos), donde el material permanece macroscópicamente elástico. La curva S–N en el tramo de vida finita se modela con la relación de Basquin,

$$\sigma_{ar} = a N^b, \quad a = \frac{(f S_{ut})^2}{S_e}, \quad b = -\frac{1}{3} \log_{10} \left(\frac{f S_{ut}}{S_e} \right), \quad (235)$$

donde σ_{ar} es la amplitud completamente invertida, S_e el límite de fatiga corregido del componente y $f \approx 0.9$ la fracción de S_{ut} a 10^3 ciclos. El límite de fatiga de la probeta pulida, $S'_e \approx 0.5 S_{ut}$ para aceros con $S_{ut} < 1400$ MPa, se corrige con los factores de Marin ($S_e = k_a k_b k_c \cdots S'_e$) por acabado superficial, tamaño y tipo de carga.

- **Deformación–vida (Strain-Life, ε –N).** Adecuado para fatiga de bajo ciclaje (LCF, $N \lesssim 10^3$ – 10^4), donde hay plasticidad cíclica localizada. Se basa en la ecuación de

Coffin–Manson–Basquin,

$$\frac{\Delta \varepsilon}{2} = \frac{\sigma'_f}{E} (2N)^b + \varepsilon'_f (2N)^c, \quad (236)$$

que requiere cuatro constantes de material adicionales (σ'_f , b , ε'_f , c) rara vez disponibles fuera de bases de datos especializadas. En este curso se usará Stress-Life salvo indicación explícita.

Cuando el esfuerzo medio no es cero ($\sigma_m \neq 0$), la amplitud admisible disminuye. Los cuatro criterios clásicos, que corresponden uno a uno con las opciones *Mean Stress Theory* del *Fatigue Tool*, son

$$\text{Goodman: } \frac{\sigma_a}{S_e} + \frac{\sigma_m}{S_{ut}} = \frac{1}{n_f}, \quad (237)$$

$$\text{Soderberg: } \frac{\sigma_a}{S_e} + \frac{\sigma_m}{S_y} = \frac{1}{n_f}, \quad (238)$$

$$\text{Gerber: } \frac{n_f \sigma_a}{S_e} + \left(\frac{n_f \sigma_m}{S_{ut}} \right)^2 = 1, \quad (239)$$

$$\text{ASME elíptica: } \left(\frac{n_f \sigma_a}{S_e} \right)^2 + \left(\frac{n_f \sigma_m}{S_y} \right)^2 = 1. \quad (240)$$

La Figura 74 compara las cuatro envolventes. Goodman es lineal y moderadamente conservador; Soderberg es el más conservador (usa S_y en lugar de S_{ut}) y garantiza además ausencia de fluencia; Gerber ajusta mejor los datos experimentales de aceros dúctiles pero no distingue el signo de σ_m ; la ASME elíptica es intermedia. Para diseño, la práctica estándar es Goodman o Soderberg; para correlación con ensayos, Gerber.

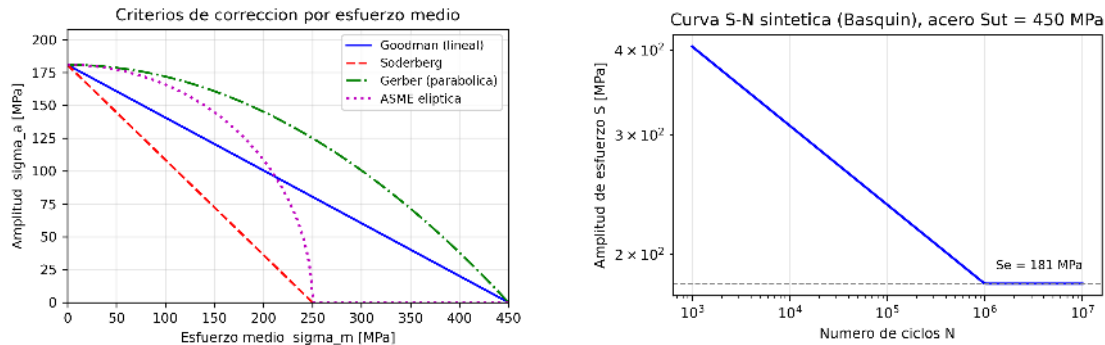


Figura 74: Izquierda: envolventes de corrección por esfuerzo medio (Ecs. 237 a 240) para acero con $S_{ut} = 450 \text{ MPa}$, $S_y = 250 \text{ MPa}$ y $S_e = 180.9 \text{ MPa}$. Derecha: curva S–N sintética de Basquin, Ec. 235, con $a = 906.6 \text{ MPa}$ y $b = -0.1167$. Ambas generadas con el Script 2 del Anexo.

Para cargas de amplitud variable se usa la regla de acumulación lineal de daño de

Palmgren–Miner,

$$D = \sum_i \frac{n_i}{N_i} \leq 1, \quad (241)$$

donde n_i es el número de ciclos aplicados en el nivel de esfuerzo i y N_i la vida a ese nivel; el *Fatigue Tool* la aplica automáticamente cuando la carga se define por historia (*History Data*) junto con conteo de ciclos *rainflow*.

9.3.2. Árbol de decisión del *Fatigue Tool*

El *Fatigue Tool* se inserta como posproceso de un *Static Structural* resuelto (clic derecho en *Solution* → *Insert* → *Fatigue* → *Fatigue Tool*). La Figura 75 organiza las decisiones en el orden en que aparecen en la ventana de detalles. Cada rombo es una opción del panel *Details of Fatigue Tool*.

9.3.3. Ejemplo guiado 2: fatiga de la placa con agujero bajo carga pulsante

Se reutiliza el modelo del Ejemplo guiado 1, pero ahora la fuerza varía cíclicamente entre 0 y 12 kN (carga pulsante, $R = 0$). Con el material de la Tabla 50, el esfuerzo local máximo por ciclo es $\sigma_{\max} = 129.1$ MPa, de modo que $\sigma_a = \sigma_m = 64.5$ MPa.

Tabla 50: Datos de fatiga del Ejemplo guiado 2 y resultado analítico de referencia (Script 2 del Anexo). Los factores de Marin son valores de ejemplo, no normativos.

Cantidad	Símbolo	Valor
Resistencia última	S_{ut}	450 MPa
Límite de fatiga de probeta	$S'_e = 0.5S_{ut}$	225 MPa
Factor de superficie (maquinado)	k_a	0.893
Factor de tamaño	k_b	0.90
Límite de fatiga corregido	S_e	180.9 MPa
Amplitud local ($R = 0$)	σ_a	64.5 MPa
Esfuerzo medio local	σ_m	64.5 MPa
Amplitud equivalente invertida (Goodman)	σ_{ar}	75.3 MPa
Conclusión ($\sigma_{ar} < S_e$)	—	vida infinita
Factor de seguridad a fatiga (Goodman)	n_f	2.00

La amplitud equivalente completamente invertida según Goodman es

$$\sigma_{ar} = \frac{\sigma_a}{1 - \sigma_m/S_{ut}} = \frac{64.5}{1 - 64.5/450} = 75.3 \text{ MPa}, \quad (242)$$

inferior a $S_e = 180.9$ MPa: el componente tiene vida infinita con $n_f = 2.00$ según la Ec. 237. En ANSYS, el procedimiento es: definir la curva S–N del material en *Engineering Data* (pestaña *Life* → *S-N Curve*, capturando los pares de la Figura 74 derecha), insertar el *Fatigue Tool* con *Type: Stress Life*, *Loading Type: Zero-Based*, *Mean Stress*

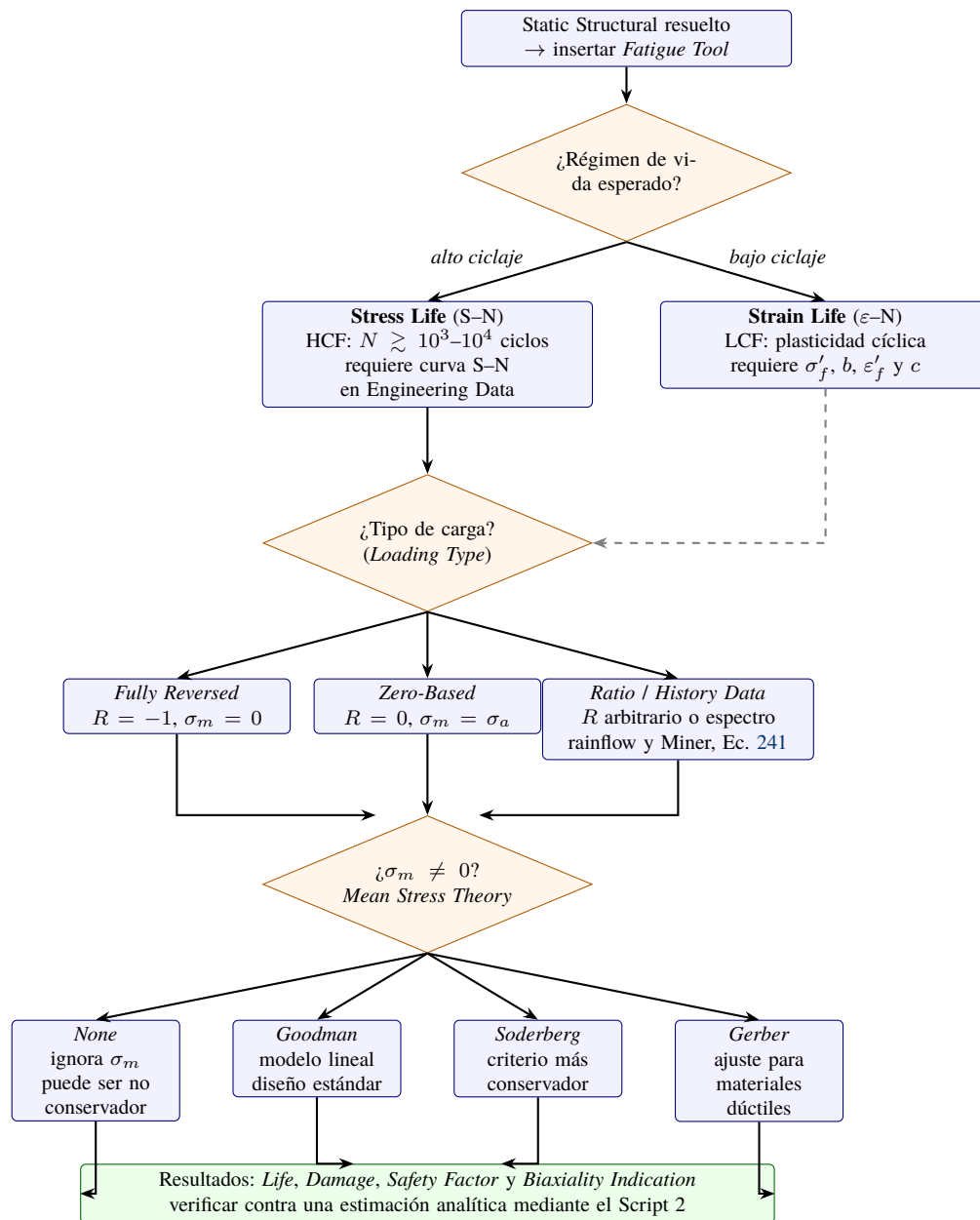


Figura 75: Árbol de decisión del *Fatigue Tool* de ANSYS Mechanical. Los rombos representan decisiones asociadas con los campos del panel de detalles, mientras que los rectángulos azules muestran las opciones disponibles. La opción ASME elíptica aparece en el mismo menú que Goodman, Soderberg y Gerber. La configuración *Mean Stress Curves* permite además utilizar curvas S–N obtenidas experimentalmente para diferentes valores de la razón de esfuerzos R .

Theory: Goodman, y solicitar *Life* y *Safety Factor*. El mapa de *Safety Factor* debe mostrar su mínimo (≈ 2.0) exactamente donde el estático mostró el máximo de von Mises.

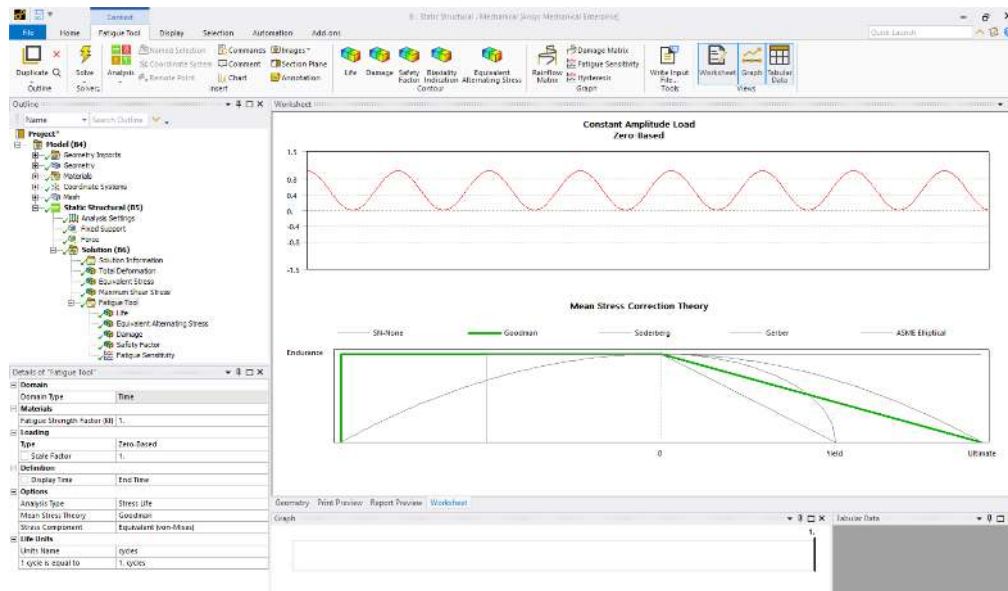


Figura 76: Panel Details of Fatigue Tool con Type = Stress Life, Loading Type = Zero-Based y Mean Stress Theory = Goodman, siguiendo el árbol de la figura anterior

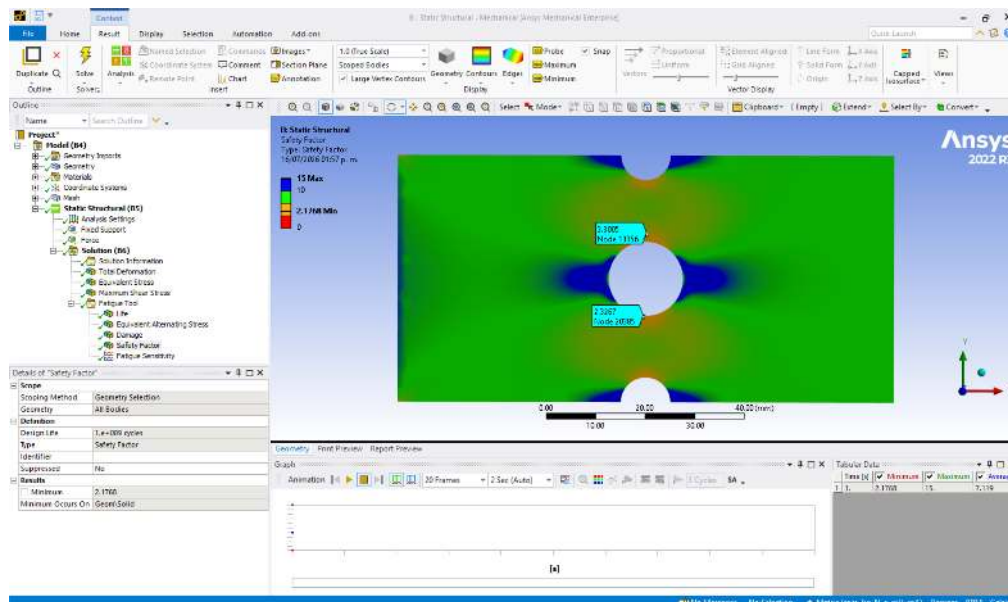


Figura 77: Contorno de Safety Factor a fatiga. El mínimo debe rondar 2.0 en el borde del agujero. Comparar contra el valor analítico de la tabla de datos de fatiga.

Errores frecuentes. (i) Olvidar que ANSYS interpola la curva S–N en escala log–log por defecto: capturar pocos puntos distorsiona la interpolación. (ii) Usar *Fully Reversed*

cuando la carga real es pulsante: se ignora σ_m y la vida se sobreestima. (iii) Aplicar el *Fatigue Tool* sobre un esfuerzo singular: la vida calculada en una esquina viva es tan divergente como el esfuerzo mismo.

9.4. Análisis modal: frecuencias naturales y modos de vibración

9.4.1. Fundamento

Con amortiguamiento nulo y sin cargas externas, el movimiento libre de la estructura discretizada obedece $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{0}$. Buscando soluciones armónicas $\mathbf{u}(t) = \phi e^{i\omega t}$ se obtiene el problema generalizado de valores propios

$$(\mathbf{K} - \omega_i^2 \mathbf{M}) \phi_i = \mathbf{0}, \quad (243)$$

cuyas soluciones son las frecuencias naturales $f_i = \omega_i/(2\pi)$ y las formas modales ϕ_i . Tres puntos que el posgrado debe dominar:

- **Las formas modales no tienen amplitud física.** ANSYS las normaliza respecto a la matriz de masa ($\phi_i^T \mathbf{M} \phi_i = 1$); los colores del contorno indican forma relativa, no desplazamiento real.
- **Modos de cuerpo rígido.** Un cuerpo sin soportes exhibe seis modos con $f \approx 0$ (tres traslaciones, tres rotaciones). Encontrarlos cuando el modelo debería estar soportado es el diagnóstico clásico de una condición de frontera olvidada.
- **Factores de participación y masa efectiva.** Indican qué modos dominan la respuesta ante excitación en cada dirección; la suma de masas efectivas debe capturar típicamente el 90 % o más de la masa total para análisis espectrales posteriores.

Si el componente opera bajo carga estática significativa (tensión de un cable, presurización, fuerza centrífuga), las frecuencias cambian: debe usarse un modal *preesforzado*, encadenando *Static Structural* $\rightarrow$ *Modal* en Workbench (la celda *Solution* del estático alimenta la celda *Setup* del modal). Este es exactamente el caso de un cable OPGW tensado, donde la rigidez geométrica domina.

9.4.2. Flujo de trabajo y Ejemplo guiado 3: viga en voladizo

Caso de referencia: viga de acero en voladizo de $300 \times 25 \times 5$ mm ($E = 200$ GPa, $\rho = 7850$ kg/m³), empotrada en una cara corta. La solución analítica de Euler–Bernoulli para las frecuencias de flexión es

$$f_n = \frac{(\beta_n L)^2}{2\pi L^2} \sqrt{\frac{EI}{\rho A}}, \quad \beta_1 L = 1.8751, \quad \beta_2 L = 4.6941, \quad \beta_3 L = 7.8548, \quad (244)$$

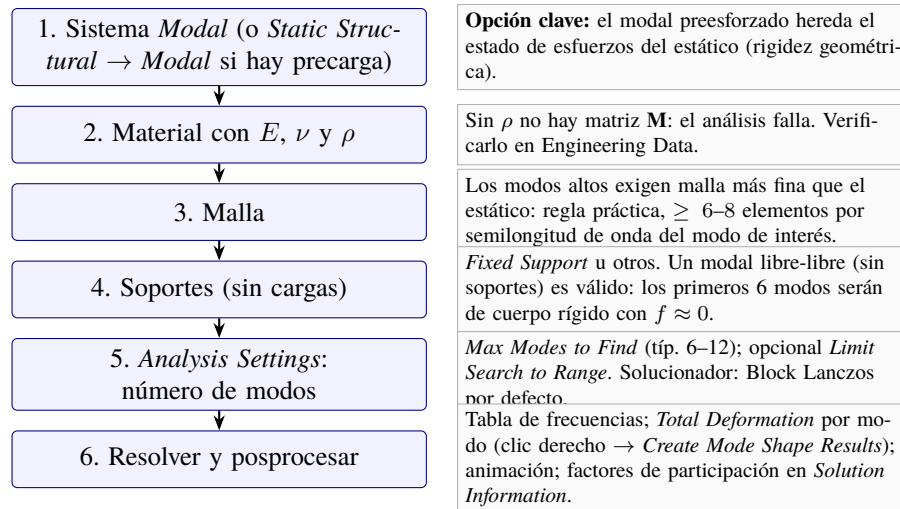


Figura 78: Flujo de trabajo del análisis *Modal* en ANSYS Workbench y opciones relevantes de cada paso.

con $I = bh^3/12$ respecto al eje débil. El Script 1 del Anexo evalúa la Ec. 244: $f_1 = 45.30$ Hz, $f_2 = 283.9$ Hz y $f_3 = 794.9$ Hz (Figura 79). En ANSYS, con *Max Modes to Find* = 6, las tres primeras frecuencias de flexión débil deben coincidir con estos valores dentro de 2–3 % (la teoría de Euler–Bernoulli desprecia cortante e inercia rotatoria; el sólido 3D de ANSYS las incluye, por lo que sus frecuencias serán ligeramente *menores*). Entre ellas aparecerán modos de flexión lateral (eje fuerte) y torsión que la fórmula unidimensional no captura: identificarlos por su forma es parte del ejercicio.

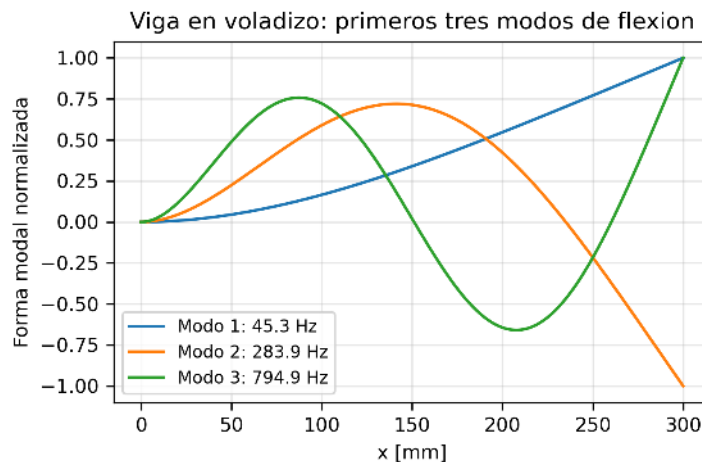


Figura 79: Formas modales analíticas de los tres primeros modos de flexión de la viga en voladizo del Ejemplo guiado 3, con las frecuencias de la Ec. 244. Generada con el Script 1 del Anexo.

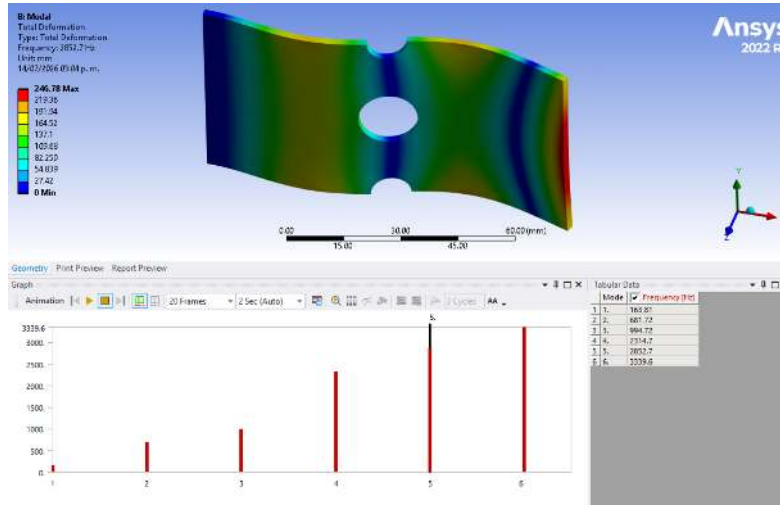


Figura 80: Tabla de frecuencias del análisis Modal (Solution > Tabular Data) con 6 modos. Marcar cuáles corresponden a flexión débil, flexión fuerte y torsión.

9.5. Dinámica de cuerpos rígidos: cinemática de mecanismos acoplados

9.5.1. Fundamento: el mecanismo biela–manivela–corredera

El sistema *Rigid Dynamics* no resuelve $\mathbf{Ku} = \mathbf{F}$: los cuerpos son indeformables y las incógnitas son las coordenadas generalizadas del mecanismo, gobernadas por las ecuaciones de la dinámica multicuerpo con restricciones de junta (formulación de ecuaciones algebraico-diferenciales). Es el complemento natural del FEM: primero se determina *cómo se mueve* el mecanismo y qué fuerzas transmite cada junta; después, con esas fuerzas, el FEM determina *si los eslabones resisten*.

El mecanismo de referencia es la biela–manivela–corredera: manivela de radio r , biela de longitud L , corredera que desliza sobre el eje x , con la manivela girando a velocidad angular constante ω (Figura 82). Cerrando el lazo vectorial se obtiene la posición exacta de la corredera medida desde el eje de la manivela,

$$x(\theta) = r \cos \theta + L \sqrt{1 - \lambda^2 \sin^2 \theta}, \quad \lambda = \frac{r}{L}, \quad \theta = \omega t, \quad (245)$$

y derivando con $\dot{\theta} = \omega$ constante, la velocidad y la aceleración exactas:

$$v(\theta) = -r\omega \left[\sin \theta + \frac{\lambda \sin \theta \cos \theta}{\sqrt{1 - \lambda^2 \sin^2 \theta}} \right], \quad (246)$$

$$a(\theta) = -r\omega^2 \left[\cos \theta + \frac{\lambda \cos 2\theta}{\sqrt{1 - \lambda^2 \sin^2 \theta}} + \frac{\lambda^3 \sin^2 \theta \cos^2 \theta}{(1 - \lambda^2 \sin^2 \theta)^{3/2}} \right]. \quad (247)$$

En el punto muerto superior (PMS, $\theta = 0$) y en el inferior (PMI, $\theta = \pi$) la aceleración toma los valores característicos

$$a_{\text{PMS}} = -r\omega^2(1 + \lambda), \quad a_{\text{PMI}} = +r\omega^2(1 - \lambda), \quad (248)$$

que serán la vara de medir de los *probes* de ANSYS. La asimetría entre PMS y PMI (efecto de segundo orden de la biela finita) es un buen detector de errores de modelado: un estudiante que obtenga aceleraciones simétricas modeló, sin querer, una biela infinita.

Antes de armar el modelo conviene verificar la movilidad con el criterio de Gruebler–Kutzbach plano,

$$m = 3(n - 1) - 2j_1 - j_2, \quad (249)$$

con n cuerpos (incluida la bancada), j_1 juntas de un grado de libertad y j_2 de dos. Para la biela–manivela: $n = 4$, $j_1 = 4$ (tres revolutas y una prismática), $j_2 = 0$, de donde $m = 1$: un solo actuador (la velocidad de la manivela) define todo el movimiento. Si ANSYS reporta un mecanismo sobre-restringido o con grados de libertad libres inesperados, la Ec. 249 localiza el error antes que cualquier depuración numérica.

Tabla 51: Juntas de ANSYS *Rigid Dynamics* usadas con más frecuencia y grados de libertad (GDL) que dejan libres. La junta correcta es la que libera exactamente los GDL físicos del par cinemático; agregar juntas redundantes sobre-restringe el mecanismo.

Junta (Joint)	GDL libres	Uso típico
Revolute	1 (rotación R_z)	Pernos manivela–bancada, manivela–biela, biela–corredera
Translational	1 (traslación T_x)	Corredera sobre su guía
Cylindrical	2 (T_z, R_z)	Ejes que giran y deslizan
Spherical	3 (R_x, R_y, R_z)	Rótulas, extremos de bielas espaciales
Universal	2 (R_x, R_z)	Juntas cardán
Fixed	0	Unión rígida entre cuerpos; anclaje a bancada
Planar	3 (T_x, T_y, R_z)	Cuerpos confinados a un plano

9.5.2. Flujo de trabajo en Rigid Dynamics

9.5.3. Ejemplo guiado 4: biela–manivela–corredera a 3000 rpm

Datos: $r = 50$ mm, $L = 150$ mm ($\lambda = 1/3$), $\omega = 3000$ rpm = 314.16 rad/s, masa de la corredera $m_p = 0.5$ kg. El Script 3 del Anexo evalúa las Ecs. 245 a 248 y produce la Figura 82. Los valores de control son: carrera = $2r = 100.0$ mm; $a_{\text{PMS}} = -6579.7$ m/s²; $a_{\text{PMI}} = +3289.9$ m/s²; velocidad máxima $|v| = 16.57$ m/s en $\theta \approx 73^\circ$ (no en 90° : otro efecto de biela finita).

En ANSYS, tras seguir el flujo de la Figura 81 (revoluta bancada–manivela con *Rotational Velocity* = 314.16 rad/s, revolutas manivela–biela y biela–corredera, junta *Translational* corredera–bancada, tiempo final 0.04 s), se insertan los *probes* de posición, ve-

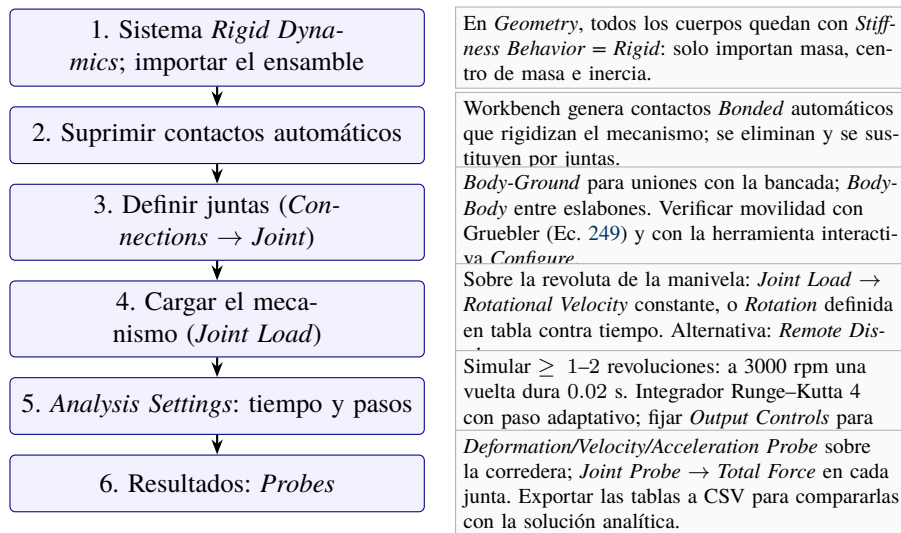


Figura 81: Flujo de trabajo del análisis *Rigid Dynamics* en ANSYS Workbench para un mecanismo acoplado, con las opciones relevantes de cada paso.

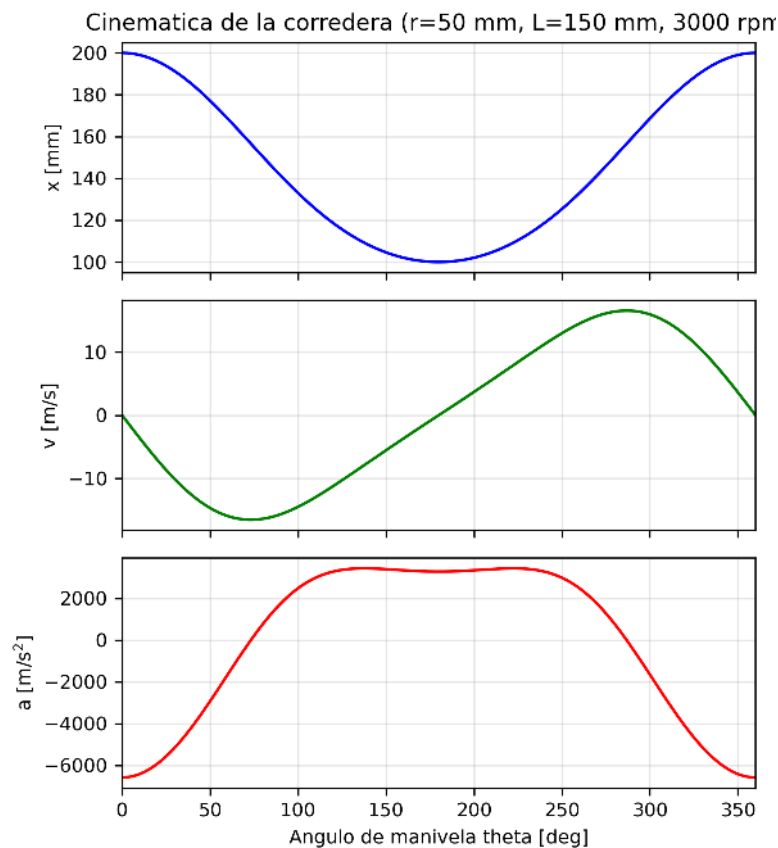


Figura 82: Cinemática exacta de la corredera (posición, velocidad y aceleración contra ángulo de manivela) para $r = 50$ mm, $L = 150$ mm y 3000 rpm, Ecs. 245 a 247. Los *probes* de ANSYS *Rigid Dynamics* deben reproducir estas curvas punto a punto. Generada con el Script 3 del Anexo.

locidad y aceleración de la corredera y se exportan a CSV. La comparación cuantitativa contra las curvas analíticas (error relativo punto a punto, norma máxima) es el entregable central del ejemplo: con cuerpos rígidos e integrador adaptativo el error debe ser menor que 0.1 %, es decir, la discrepancia esperada es esencialmente de muestreo.

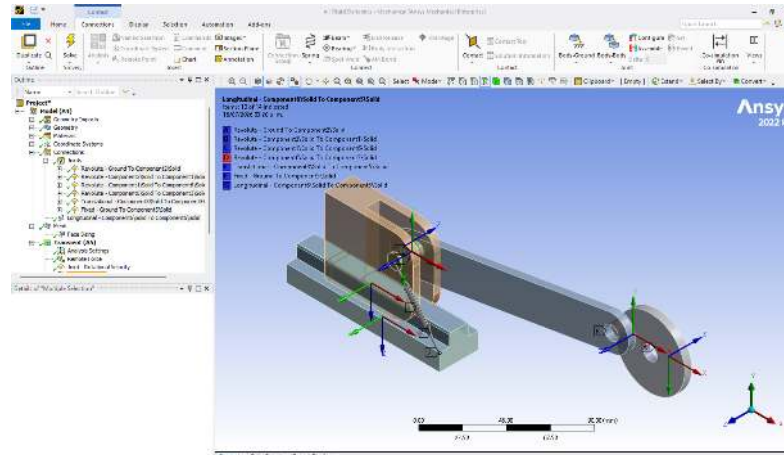


Figura 83: Árbol de Connections mostrando las cuatro juntas del mecanismo (cuatro Revolute, una Translational y un resorte de prueba) y la verificación con la herramienta Configure.

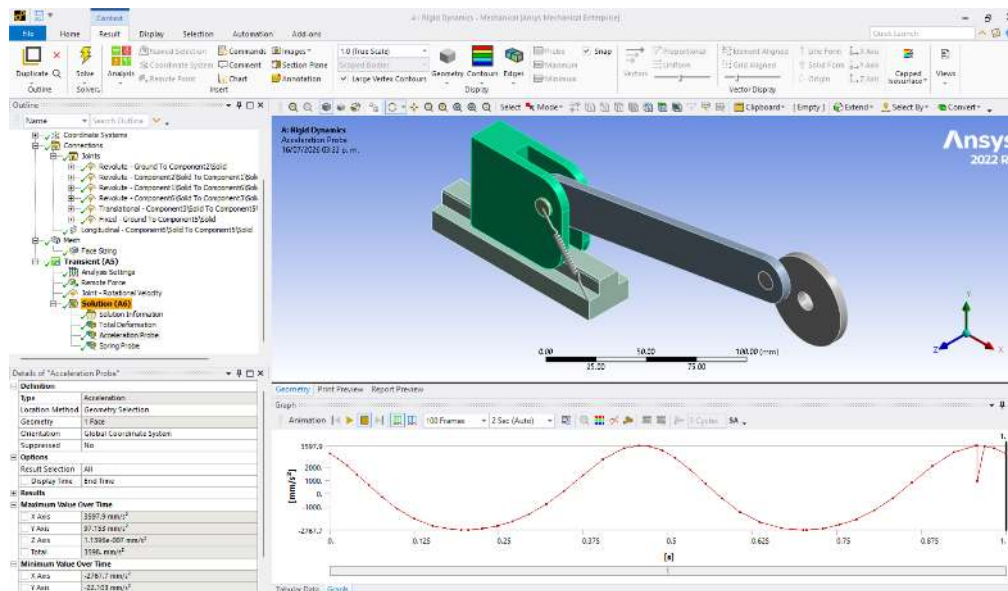


Figura 84: Aceleración de la corredera en función del tiempo durante dos revoluciones. Los valores extremos deben ser aproximadamente 3598 mm/s² y 171.33 mm/s².

9.6. Análisis acoplado: esfuerzos en la biela de un mecanismo en movimiento

9.6.1. Estrategias de acoplamiento

La pregunta de diseño es: dado el movimiento del mecanismo, ¿resiste la biela? Existen tres estrategias, resumidas en la Tabla 52. En este curso se usa la estrategia híbrida (b), que es la práctica industrial estándar para dimensionamiento: es dos órdenes de magnitud más barata que el transitorio flexible completo y su error es pequeño mientras las frecuencias naturales del eslabón queden muy por encima de la frecuencia de operación (justo lo que el análisis modal de la Sección 9.4 permite verificar: si $f_1 \gg \omega/2\pi$, la biela responde cuasi-estáticamente).

Tabla 52: Estrategias para obtener esfuerzos en un eslabón de un mecanismo en movimiento.

Estrategia	Procedimiento	Costo	Cuándo usarla
(a) Transitorio flexible completo	<i>Transient Structural</i> con todos los cuerpos deformables y contactos	Muy alto	Impactos, holguras, flexibilidad no despreciable
(b) Híbrida (RBD $\rightarrow$ estático)	<i>Rigid Dynamics</i> extrae fuerzas de junta en el instante crítico; <i>Static Structural</i> de un solo eslabón con esas cargas	Bajo	Dimensionamiento; f_1 del eslabón $\gg$ frecuencia de operación
(c) Cuerpo flexible en RBD (CMS)	Marcar el eslabón como <i>Flexible</i> (síntesis modal de componentes) dentro del propio <i>Rigid Dynamics</i>	Medio	Esfuerzos dinámicos continuos en un eslabón durante todo el ciclo

9.6.2. Ejemplo guiado 5: esfuerzos y pandeo en la biela en el punto muerto superior

Despreciando la presión de gas y la fricción, la fuerza que la corredera transmite a la biela proviene de su propia inercia. Del equilibrio del pistón, con ϕ el ángulo de la biela ($\sin \phi = \lambda \sin \theta$),

$$F_{\text{biela}}(\theta) = \frac{-m_p a(\theta)}{\cos \phi(\theta)}, \quad (250)$$

cuyo máximo ocurre en el PMS: $F_{\text{biela}}^{\max} = m_p r \omega^2 (1 + \lambda) = 3290 \text{ N}$ en compresión (Figura ??). Con una biela de sección rectangular de $12 \times 6 \text{ mm}$ de acero, los valores analíticos de control (Script 3 del Anexo) son: esfuerzo axial $\sigma = F/A = 45.7 \text{ MPa}$; carga crítica de pandeo de Euler con extremos articulados,

$$P_{\text{cr}} = \frac{\pi^2 E I_{\min}}{L^2} = 18\,950 \text{ N}, \quad n_{\text{pandeo}} = \frac{P_{\text{cr}}}{F_{\text{biela}}^{\max}} = 5.76, \quad (251)$$

con esbeltez $L/r_g = 86.6$, dentro del rango donde Euler es aplicable para acero. El campo de von Mises del *Static Structural* debe ser casi uniforme ($\approx 46 \text{ MPa}$) en el cuerpo de la biela, con concentraciones locales en la transición hacia los ojos; y un análisis

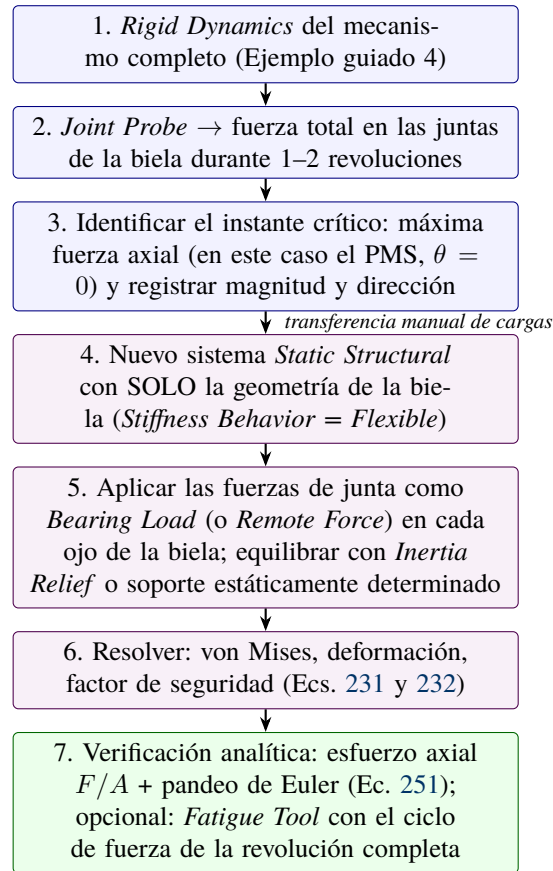


Figura 85: Flujo híbrido para el análisis de esfuerzos en la biela: la dinámica de cuerpos rígidos (azul) suministra las cargas; el análisis estático del eslabón aislado (violeta) suministra los esfuerzos; la verificación analítica (verde) cierra el ciclo.

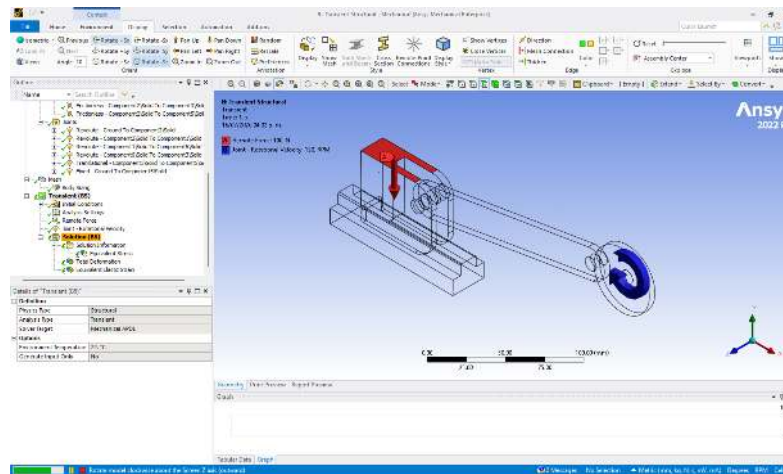


Figura 86: Transient Structural de la biela aislada: Bearing Load de 3290 N en el ojo del pie de biela, reacción en el ojo de cabeza, malla refinada en las transiciones.

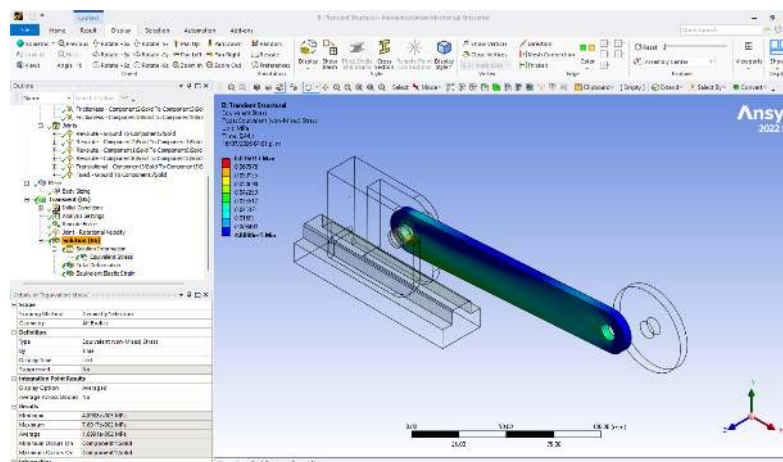


Figura 87

de *Eigenvalue Buckling* encadenado (celda *Solution* del estático → *Setup* de *Eigenvalue Buckling*) debe reportar un multiplicador de carga cercano a 5.8.

Nota metodológica. La fuerza de la Ec. 250 ignora la masa de la propia biela; ANSYS *Rigid Dynamics* sí la incluye, de modo que el *Joint Probe* dará un valor ligeramente distinto al analítico. Cuantificar esa diferencia (y explicarla) vale más, doctoralmente, que reproducir un número exacto: es un ejercicio de jerarquía de modelos, tema central de la Semana 1.

9.7. Verificación cruzada: tabla de control analítico

La Tabla 53 concentra todos los valores de referencia de la semana. Cada estudiante debe llenarla con sus resultados de ANSYS y reportar el error relativo. Discrepancias

mayores que las tolerancias indicadas obligan a revisar malla, unidades o condiciones de frontera antes de continuar.

Tabla 53: Valores analíticos de control (Scripts 1 a 3 del Anexo) y tolerancias de aceptación para los resultados de ANSYS de la Semana 9.

Ejemplo	Cantidad	Valor analítico	Resultado ANSYS	Tolerancia
1 (estático)	σ_{VM} máx. en el agujero	129.1 MPa	_____	5 %
1 (estático)	Factor de seguridad mín.	1.94	_____	5 %
2 (fatiga)	FS a fatiga (Goodman)	2.00	_____	8 %
2 (fatiga)	Vida	infinita ($\sigma_{ar} < S_e$)	_____	—
3 (modal)	f_1, f_2, f_3 flexión débil	45.3 / 283.9 / 794.9 Hz	_____	3 %
4 (RBD)	Carrera de la corredera	100.0 mm	_____	0.1 %
4 (RBD)	a en PMS / PMI	−6579.7 / +3289.9 m/s ²	_____	0.5 %
4 (RBD)	$ v $ máx. (y su ángulo)	16.57 m/s (73°)	_____	0.5 %
5 (acoplado)	F axial en la biela (PMS)	3290 N	_____	5 %
5 (acoplado)	σ axial en el cuerpo	45.7 MPa	_____	8 %
5 (acoplado)	Multiplicador de pandeo	5.76	_____	10 %

9.8. Síntesis de la semana

9.9. Referencias de la Semana 9

- Budynas, R. G. y Nisbett, J. K. *Shigley's Mechanical Engineering Design*, 11a ed., McGraw-Hill, 2020. (Capítulos de fatiga: factores de Marin, Basquin, criterios de esfuerzo medio.)
- Cook, R. D., Malkus, D. S., Plesha, M. E. y Witt, R. J. *Concepts and Applications of Finite Element Analysis*, 4a ed., Wiley, 2002. (Interpretación de esfuerzos, promediado nodal, singularidades.)
- Schijve, J. *Fatigue of Structures and Materials*, 2a ed., Springer, 2009. <https://doi.org/10.1007/978-1-4020-6808-9>
- Pilkey, W. D. y Pilkey, D. F. *Peterson's Stress Concentration Factors*, 3a ed., Wiley, 2008. <https://doi.org/10.1002/9780470211106>
- Rao, S. S. *Mechanical Vibrations*, 6a ed., Pearson, 2017. (Viga de Euler–Bernoulli, análisis modal.)
- Norton, R. L. *Design of Machinery*, 6a ed., McGraw-Hill, 2020. (Cinemática y dinámica del mecanismo biela–manivela–corredera.)
- Uicker, J. J., Pennock, G. R. y Shigley, J. E. *Theory of Machines and Mechanisms*, 5a ed., Oxford University Press, 2017. (Movilidad de Gruebler–Kutzbach, análisis

Tabla 54: Síntesis de la Semana 9.

Tema	Idea central	Herramienta / referencia
Estático	$\mathbf{Ku} = \mathbf{F}$; von Mises y FS contra fluencia; convergencia de malla obligatoria; cuidado con singularidades	Static Structural; Ecs. 230 a 232; Figura 70
Fatiga	Stress-Life para HCF; corrección de esfuerzo medio (Goodman/Soderberg/Gerber/ASME); Miner para espectros	Fatigue Tool; Ecs. 235 y 237 a 241; Figura 75
Modal	$(\mathbf{K} - \omega^2 \mathbf{M})\phi = \mathbf{0}$; modos sin amplitud física; preesfuerzo cambia frecuencias	Modal; Ec. 243; Figura 78
Mecanismos	Cinemática exacta del biela-manivela; juntas y movilidad (Gruebler); probes contra solución analítica	Rigid Dynamics; Ecs. 245 a 249; Figura 81
Acoplamiento	Fuerzas de junta del RBD alimentan el estático del eslabón; validez cuasi-estática si $f_1 \gg$ frecuencia de operación	Flujo híbrido; Ec. 250; Figura 85
Verificación	Ningún número de ANSYS sin referencia analítica independiente	Tabla 53; Scripts 1–3

de mecanismos.)

- ANSYS, Inc. *ANSYS Mechanical User's Guide* y *ANSYS Meshing User's Guide* (documentación de la versión instalada en el laboratorio; ver también los manuales del repositorio del curso).

9.10. Anexo: códigos Python de verificación analítica (Google Colab)

Los tres scripts siguen las convenciones del curso: ASCII puro, `matplotlib.use('Agg')`, figuras a 300 dpi en `./Figuras` y `plt.show()` comentado. Deben ejecutarse antes de abrir ANSYS: sus salidas de consola son los valores de la Tabla 53.

9.10.1. Script 1: referencia del estático (placa con agujero) y del modal (viga en voladizo)

```

1  # -*- coding: ascii -*-
2  """
3  Semana 9 -- FEM II. Script 1/3: verificacion analitica para ANSYS.
4  (a) Estatico: placa finita con agujero central a tension (factor de
5      concentracion Kt, ajuste de Howland/Heywood) -> valida Equivalent
6      Stress de Static Structural.
7  (b) Modal: viga en voladizo Euler-Bernoulli -> valida las primeras
8      tres frecuencias naturales del analisis Modal.
9  Curso: Prototipado y Simulacion (DITA) -- Universidad La Salle Bajio.
10 Entorno: Google Colab. Las figuras se guardan en ./Figuras a 300 dpi.
11 """
12 import os
13 import numpy as np
    
```

```

14 import matplotlib
15 matplotlib.use('Agg')                                # backend sin pantalla
16 import matplotlib.pyplot as plt
17 import matplotlib as mpl
18
19 mpl.rcParams.update({'font.size': 9, 'axes.grid': True,
20                      'grid.alpha': 0.3, 'figure.dpi': 300})
21 os.makedirs('Figuras', exist_ok=True)
22
23 # =====
24 # (a) PLACA CON AGUJERO CENTRAL -- caso de referencia estatico
25 # =====
26 # Geometria y carga (mismos datos que el Ejemplo guiado 1 en ANSYS)
27 W = 60.0e-3      # ancho de la placa [m]
28 t = 5.0e-3       # espesor [m]
29 d = 15.0e-3      # diametro del agujero [m]
30 F = 12.0e3       # fuerza axial de tension [N]
31 Sy = 250.0e6     # limite de fluencia, acero estructural [Pa]
32
33 # Esfuerzo nominal en la seccion neta (la que pasa por el agujero)
34 A_neta = (W - d) * t
35 sig_neta = F / A_neta
36 # Esfuerzo bruto (seccion completa), referencia alternativa
37 sig_bruto = F / (W * t)
38
39 # Kt referido a la seccion neta: ajuste polinomial de Howland/Heywood
40 # valido para 0 < d/W < 0.7 (Pilkey, Peterson's SCF, 3a ed.)
41 r = d / W
42 Kt_neta = 3.0 - 3.14*r + 3.667*r**2 - 1.527*r**3
43 sig_max = Kt_neta * sig_neta
44 FS_fluencia = Sy / sig_max
45
46 print('=== (a) Placa con agujero: referencia analitica ===')
47 print(f'd/W = {r:.3f}')
48 print(f'Area neta = {A_neta*1e6:.1f} mm^2')
49 print(f'sigma nominal (neta) = {sig_neta/1e6:.2f} MPa')
50 print(f'sigma nominal (bruta) = {sig_bruto/1e6:.2f} MPa')
51 print(f'Kt (seccion neta) = {Kt_neta:.3f}')
52 print(f'sigma_max esperado = {sig_max/1e6:.2f} MPa')
53 print(f'FS fluencia esperado = {FS_fluencia:.2f}')
54
55 # Curva Kt(d/W) para la figura didactica
56 rr = np.linspace(0.01, 0.6, 200)
57 Kt_rr = 3.0 - 3.14*rr + 3.667*rr**2 - 1.527*rr**3
58
59 fig, ax = plt.subplots(figsize=(4.6, 3.2))
60 ax.plot(rr, Kt_rr, 'b-', lw=1.5)
61 ax.plot([r], [Kt_neta], 'ro', ms=6,
62         label=f'Caso del curso: d/W={r:.2f}, Kt={Kt_neta:.2f}')
63 ax.set_xlabel('Relacion d/W')
64 ax.set_ylabel('Kt (referido a seccion neta)')
65 ax.set_title('Concentracion de esfuerzos: placa con agujero central')
66 ax.legend()
67 fig.tight_layout()
68 fig.savefig('Figuras/S9_fig_Kt_placa.png', dpi=300)
69 plt.close(fig)
70 # plt.show()
71
72 # =====
73 # (b) VIGA EN VOLADIZO -- referencia analitica del analisis Modal

```

```

74 # =====
75 # Viga rectangular de acero (mismos datos que el Ejemplo guiado 3)
76 L = 300.0e-3 # longitud [m]
77 b = 25.0e-3 # base [m]
78 h = 5.0e-3 # altura (direccion de flexion debil) [m]
79 E = 200.0e9 # modulo de Young [Pa]
80 rho = 7850.0 # densidad [kg/m^3]
81
82 A = b * h
83 I = b * h**3 / 12.0
84 # Raices de la ecuacion caracteristica cos(bL)cosh(bL) = -1
85 betaL = np.array([1.87510407, 4.69409113, 7.85475744])
86 f_n = (betaL**2 / (2.0*np.pi*L**2)) * np.sqrt(E*I/(rho*A))
87
88 print('\n=== (b) Viga en voladizo: frecuencias Euler-Bernoulli ===')
89 for i, f in enumerate(f_n, start=1):
90     print(f'Modo {i}: f_{i} = {f:8.2f} Hz')
91
92 # Formas modales analiticas para la figura
93 x = np.linspace(0.0, L, 300)
94 fig, ax = plt.subplots(figsize=(4.8, 3.2))
95 for i, bl in enumerate(betaL, start=1):
96     beta = bl / L
97     sig = (np.sinh(bl) - np.sin(bl)) / (np.cosh(bl) + np.cos(bl))
98     phi = (np.cosh(beta*x) - np.cos(beta*x)
99           - sig*(np.sinh(beta*x) - np.sin(beta*x)))
100     phi /= np.max(np.abs(phi))
101     ax.plot(x*1e3, phi, lw=1.4, label=f'Modo {i}: {f_n[i-1]:.1f} Hz')
102 ax.set_xlabel('x [mm]')
103 ax.set_ylabel('Forma modal normalizada')
104 ax.set_title('Viga en voladizo: primeros tres modos de flexion')
105 ax.legend(fontsize=8)
106 fig.tight_layout()
107 fig.savefig('Figuras/S9_fig_modos_viga.png', dpi=300)
108 plt.close(fig)
109 # plt.show()
110
111 print('\nFiguras guardadas en ./Figuras')

```

9.10.2. Script 2: curva S–N de Basquin, envolventes de esfuerzo medio y vida del Ejemplo guiado 2

```

1 # -*- coding: ascii -*-
2 """
3 Semana 9 -- FEM II. Script 2/3: fatiga por el metodo esfuerzo-vida.
4 (a) Curva S-N (Basquin) construida con el criterio de Shigley/Budynas
5     para acero: valida el campo "Life" del Fatigue Tool de ANSYS.
6 (b) Envolventes de esfuerzo medio: Goodman, Soderberg, Gerber y
7     ASME elíptica -> corresponden a las opciones "Mean Stress Theory".
8 (c) Vida estimada del Ejemplo guiado 2 (placa con agujero bajo carga
9     pulsante R=0) con Goodman.
10 Curso: Prototipado y Simulacion (DITA) -- Universidad La Salle Bajio.
11 """
12 import os
13 import numpy as np
14 import matplotlib
15 matplotlib.use('Agg')

```

```

16 import matplotlib.pyplot as plt
17 import matplotlib as mpl
18
19 mpl.rcParams.update({'font.size': 9, 'axes.grid': True,
20                     'grid.alpha': 0.3, 'figure.dpi': 300})
21 os.makedirs('Figuras', exist_ok=True)
22
23 # -----
24 # Propiedades del material (acero estructural tipo A36/S235)
25 # -----
26 Sut = 450.0    # resistencia ultima [MPa]
27 Sy  = 250.0    # limite de fluencia [MPa]
28 Sep = 0.5*Sut  # limite de fatiga de probeta pulida Se' = 0.5 Sut
29
30 # Factores de Marin (valores de ejemplo, no normativos)
31 ka = 4.51*Sut**(-0.265) # superficie maquinada
32 kb = 0.9                # tamano (aprox. para seccion no circular)
33 kc = 1.0                # carga axial-flexion combinada -> conservador 1.0
34 Se = ka*kb*kc*Sep       # limite de fatiga corregido del componente
35
36 print('=== Limite de fatiga corregido (Marin) ===')
37 print(f"Se' = {Sep:.1f} MPa | ka = {ka:.3f} | kb = {kb:.2f} | kc = {kc:.2f}")
38 print(f'Se = {Se:.1f} MPa')
39
40 # -----
41 # (a) Curva S-N de Basquin:  $S = a N^b$  entre  $1e3$  y  $1e6$  ciclos
42 # -----
43 f_frac = 0.9            # fraccion de Sut a  $1e3$  ciclos
44 a_bas  = (f_frac*Sut)**2 / Se
45 b_bas  = -np.log10(f_frac*Sut/Se)/3.0
46 print('\n=== Curva S-N (Basquin) ===')
47 print(f'a = {a_bas:.1f} MPa | b = {b_bas:.4f}')
48
49 N = np.logspace(3, 7, 200)
50 S = np.where(N <= 1e6, a_bas*N**b_bas, Se)
51
52 fig, ax = plt.subplots(figsize=(4.8, 3.2))
53 ax.loglog(N, S, 'b-', lw=1.6)
54 ax.axhline(Se, color='gray', ls='--', lw=0.9)
55 ax.text(1.3e6, Se*1.05, f'Se = {Se:.0f} MPa', fontsize=8)
56 ax.set_xlabel('Numero de ciclos N')
57 ax.set_ylabel('Amplitud de esfuerzo S [MPa]')
58 ax.set_title('Curva S-N sintetica (Basquin), acero Sut = 450 MPa')
59 fig.tight_layout()
60 fig.savefig('Figuras/S9_fig_SN_basquin.png', dpi=300)
61 plt.close(fig)
62 # plt.show()
63
64 # -----
65 # (b) Envolventes de esfuerzo medio en el plano ( $\sigma_m$ ,  $\sigma_a$ )
66 # -----
67 sm = np.linspace(0.0, Sut, 300)
68 goodman = Se*(1.0 - sm/Sut)
69 soderberg = Se*(1.0 - sm/Sy); soderberg[sm > Sy] = np.nan
70 gerber    = Se*(1.0 - (sm/Sut)**2)
71 asme      = Se*np.sqrt(np.clip(1.0 - (sm/Sy)**2, 0.0, None))
72
73 fig, ax = plt.subplots(figsize=(5.0, 3.4))
74 ax.plot(sm, goodman, 'b-', lw=1.5, label='Goodman (lineal)')
75 ax.plot(sm, soderberg, 'r--', lw=1.5, label='Soderberg')

```

```

76 ax.plot(sm, gerber, 'g-.', lw=1.5, label='Gerber (parabolica)')
77 ax.plot(sm, asme, 'm:', lw=1.8, label='ASME eliptica')
78 ax.set_xlabel('Esfuerzo medio sigma_m [MPa]')
79 ax.set_ylabel('Amplitud sigma_a [MPa]')
80 ax.set_title('Criterios de correccion por esfuerzo medio')
81 ax.legend(fontsize=8)
82 ax.set_xlim(0, Sut); ax.set_ylim(0, Se*1.15)
83 fig.tight_layout()
84 fig.savefig('Figuras/S9_fig_esfuerzo_medio.png', dpi=300)
85 plt.close(fig)
86 # plt.show()
87
88 # -----
89 # (c) Vida del Ejemplo guiado 2: placa con agujero, carga pulsante
90 # F(t) entre 0 y 12 kN (R = 0), con Kf = Kt (conservador)
91 # -----
92 sig_neta_max = 12.0e3/((60.0e-3-15.0e-3)*5.0e-3)/1e6 # [MPa]
93 Kt = 2.420 # del Script 1 (d/W = 0.25)
94 sig_max = Kt*sig_neta_max # esfuerzo local maximo
95 sig_a = 0.5*sig_max # amplitud (R = 0)
96 sig_m = 0.5*sig_max # esfuerzo medio (R = 0)
97
98 # Amplitud equivalente completamente invertida (Goodman)
99 sig_ar = sig_a/(1.0 - sig_m/Sut)
100 if sig_ar <= Se:
101     print('\n=== (c) Vida estimada, Ejemplo guiado 2 ===')
102     print(f'sigma_a = {sig_a:.1f} MPa | sigma_m = {sig_m:.1f} MPa')
103     print(f'sigma_ar (Goodman) = {sig_ar:.1f} MPa <= Se -> vida infinita')
104 else:
105     N_vida = (sig_ar/a_bas)**(1.0/b_bas)
106     print('\n=== (c) Vida estimada, Ejemplo guiado 2 ===')
107     print(f'sigma_a = {sig_a:.1f} MPa | sigma_m = {sig_m:.1f} MPa')
108     print(f'sigma_ar (Goodman) = {sig_ar:.1f} MPa')
109     print(f'Vida estimada N = {N_vida:.3e} ciclos')
110
111 # Factor de seguridad a fatiga (Goodman) del punto de operacion
112 n_goodman = 1.0/(sig_a/Se + sig_m/Sut)
113 print(f'FS Goodman = {n_goodman:.2f}')
114
115 print('\nFiguras guardadas en ./Figuras')
    
```

9.10.3. Script 3: cinemática del biela–manivela, fuerza en la biela y pandeo de Euler

```

1 # -*- coding: ascii -*-
2 """
3 Semana 9 -- FEM II. Script 3/3: mecanismo biela-manivela-corredera.
4 (a) Cinematica exacta de la corredera: posicion, velocidad y
5     aceleracion en funcion del angulo de manivela -> valida los
6     "probes" de Rigid Dynamics en ANSYS.
7 (b) Fuerza axial en la biela por inercia del piston (modelo
8     cuasi-estatico sin presion de gas) -> carga de entrada para el
9     analisis Static Structural de la biela (Ejemplo guiado 5).
10 (c) Esfuerzo axial y verificacion de pandeo de Euler en la biela.
11 Curso: Prototipado y Simulacion (DITA) -- Universidad La Salle Bajio.
12 """
13 import os
14 import numpy as np
15 import matplotlib
    
```

```

16 matplotlib.use('Agg')
17 import matplotlib.pyplot as plt
18 import matplotlib as mpl
19
20 mpl.rcParams.update({'font.size': 9, 'axes.grid': True,
21                     'grid.alpha': 0.3, 'figure.dpi': 300})
22 os.makedirs('Figuras', exist_ok=True)
23
24 # -----
25 # Parametros del mecanismo (identicos a los del modelo ANSYS)
26 # -----
27 r_m = 50.0e-3      # radio de manivela [m]
28 L_b = 150.0e-3     # longitud de biela [m]
29 rpm = 3000.0       # velocidad de la manivela [rev/min]
30 m_p = 0.50         # masa de la corredera (piston) [kg]
31 omega = 2.0*np.pi*rpm/60.0 # [rad/s]
32 lam = r_m/L_b      # relacion biela-manivela
33
34 print('=== Parametros ===')
35 print(f'omega = {omega:.2f} rad/s | lambda = r/L = {lam:.4f}')
36
37 # -----
38 # (a) Cinematica exacta (lazo vectorial cerrado)
39 #  $x(t) = r \cos(\theta) + L \sqrt{1 - \lambda^2 \sin^2(\theta)}$ 
40 # medida desde el eje de la manivela;  $\theta = \omega t$ 
41 # -----
42 th = np.linspace(0.0, 2.0*np.pi, 721)
43 sq = np.sqrt(1.0 - (lam*np.sin(th))**2)
44 x = r_m*np.cos(th) + L_b*sq
45
46 # Derivadas analíticas exactas ( $\theta_{dot} = \omega$  constante)
47 dx_dth = -r_m*np.sin(th) - (r_m*lam*np.sin(th)*np.cos(th))/sq
48 d2x_dth = (-r_m*np.cos(th)
49            - r_m*lam*(np.cos(2.0*th)/sq
50                    + (lam**2*np.sin(th)**2*np.cos(th)**2)/sq**3))
51 v = omega*dx_dth
52 a = omega**2*d2x_dth
53
54 # Valores característicos
55 a_pms = -omega**2*r_m*(1.0 + lam) # aceleracion en el PMS ( $\theta = 0$ )
56 a_pmi = omega**2*r_m*(1.0 - lam) # aceleracion en el PMI ( $\theta = \pi$ )
57 i_vmax = np.argmax(np.abs(v))
58 print('\n=== (a) Cinematica exacta ===')
59 print(f'Carrera = {(x.max()-x.min())*1e3:.1f} mm')
60 print(f'a(PMS) analitica = {a_pms:.1f} m/s^2')
61 print(f'a(PMI) analitica = {a_pmi:.1f} m/s^2')
62 print(f'|v|_max = {np.abs(v).max():.2f} m/s '
63       f'en theta = {np.degrees(th[i_vmax]):.1f} deg')
64
65 fig, axs = plt.subplots(3, 1, figsize=(5.2, 5.8), sharex=True)
66 axs[0].plot(np.degrees(th), x*1e3, 'b-', lw=1.4)
67 axs[0].set_ylabel('x [mm]')
68 axs[0].set_title('Cinematica de la corredera (r=50 mm, L=150 mm, 3000 rpm)')
69 axs[1].plot(np.degrees(th), v, 'g-', lw=1.4)
70 axs[1].set_ylabel('v [m/s]')
71 axs[2].plot(np.degrees(th), a, 'r-', lw=1.4)
72 axs[2].set_ylabel('a [m/s^2]')
73 axs[2].set_xlabel('Angulo de manivela theta [deg]')
74 axs[2].set_xlim(0, 360)
75 fig.tight_layout()

```

```

76 fig.savefig('Figuras/S9_fig_cinematica_corredera.png', dpi=300)
77 plt.close(fig)
78 # plt.show()
79
80 # -----
81 # (b) Fuerza axial en la biela por inercia del piston
82 # Equilibrio del piston:  $F_{biela} \cos(\phi) = -m_p a$  (sin gas,
83 # sin friccion);  $\phi$  = angulo de la biela con el eje x.
84 # -----
85 phi = np.arcsin(lam*np.sin(th))
86 F_in = -m_p*a # fuerza de inercia del piston [N]
87 F_bie = F_in/np.cos(phi) # fuerza axial en la biela [N]
88 i_max = np.argmax(np.abs(F_bie))
89 F_max = F_bie[i_max]
90 print('\n=== (b) Fuerza axial en la biela ===')
91 print(f'F_bielas en PMS (theta=0) = {F_bie[0]:.1f} N (compresion +)')
92 print(f'|F_bielas|_max = {abs(F_max):.1f} N '
93       f' en theta = {np.degrees(th[i_max]):.1f} deg')
94
95 fig, ax = plt.subplots(figsize=(5.2, 3.2))
96 ax.plot(np.degrees(th), F_bie, 'b-', lw=1.4)
97 ax.axvline(0, color='gray', lw=0.8, ls=':')
98 ax.set_xlabel('Angulo de manivela theta [deg]')
99 ax.set_ylabel('Fuerza axial en la biela [N]')
100 ax.set_title('Carga inercial sobre la biela (m_p = 0.5 kg)')
101 ax.set_xlim(0, 360)
102 fig.tight_layout()
103 fig.savefig('Figuras/S9_fig_fuerza_bielas.png', dpi=300)
104 plt.close(fig)
105 # plt.show()
106
107 # -----
108 # (c) Esfuerzo axial y pandeo de Euler en la biela
109 # Seccion rectangular b x h; extremos articulados (K = 1)
110 # -----
111 b_s = 12.0e-3 # base de la seccion [m]
112 h_s = 6.0e-3 # altura (eje debil) [m]
113 E = 200.0e9 # acero [Pa]
114 A_s = b_s*h_s
115 I_min = b_s*h_s**3/12.0
116
117 sig_axial = abs(F_bie[0])/A_s # en el PMS
118 P_cr = np.pi**2*E*I_min/L_b**2 # Euler, K = 1
119 FS_pandeo = P_cr/abs(F_bie[0])
120 esbeltez = L_b/np.sqrt(I_min/A_s)
121
122 print('\n=== (c) Bielas: esfuerzo y pandeo ===')
123 print(f'Area = {A_s*1e6:.0f} mm^2 | I_min = {I_min*1e12:.0f} mm^4')
124 print(f'Esbeltez L/r_g = {esbeltez:.1f}')
125 print(f'sigma axial (PMS) = {sig_axial/1e6:.2f} MPa')
126 print(f'P_critica de Euler = {P_cr:.0f} N')
127 print(f'FS a pandeo = {FS_pandeo:.2f}')
128
129 print('\nFiguras guardadas en ./Figuras')

```


9.11. Ejercicios para realizar en casa

Ambas tareas se entregan como reporte técnico en PDF (máximo 10 páginas cada una) acompañado del archivo de proyecto de ANSYS (.wbpz) y de los scripts de verificación modificados. Todo valor numérico reportado debe llevar su contraparte analítica y el error relativo.

9.11.1. Tarea 1: análisis estructural completo de un soporte en L

Un soporte en L de acero estructural ($S_y = 250$ MPa, $S_{ut} = 450$ MPa) se fabrica de placa de 8 mm de espesor. El brazo vertical mide 120×60 mm y el horizontal 100×60 mm; el radio de acuerdo interior en la esquina es $R = 10$ mm. El brazo vertical se atornilla a una pared rígida (modelar como *Fixed Support* en la cara posterior); en el extremo del brazo horizontal se cuelga una carga vertical que oscila cíclicamente entre 500 y 2 000 N.

1. **Estático.** Calcular deformación total, von Mises y factor de seguridad para la carga máxima. Realizar el estudio de convergencia con tres mallas (tamaño local en el radio de acuerdo: 2, 1.3 y 0.9 mm aproximadamente) y reportar la tabla esfuerzo–número de nodos con el valor extrapolado.
2. **Verificación analítica.** Estimar el esfuerzo nominal de flexión en la sección de la esquina con la teoría de vigas ($\sigma = Mc/I$) y justificar el factor de concentración aparente que arroja ANSYS en el radio de acuerdo.
3. **Fatiga.** Con la curva S–N sintética del Script 2 (adaptando S_{ut} si se cambia de material), analizar el ciclo $500 \leftrightarrow 2\,000$ N con el *Fatigue Tool*. La carga es de tipo *Ratio* con $R = 0.25$. Comparar la vida y el factor de seguridad obtenidos con **Goodman** y con **Gerber**, y explicar el sentido físico de la diferencia.
4. **Modal.** Obtener los primeros 4 modos del soporte instalado. Identificar la forma de cada modo y discutir si una excitación de piso a 50 Hz (motor desbalanceado cercano) representa riesgo de resonancia; proponer una modificación geométrica que aleje la frecuencia crítica al menos 20 % y demostrarla con un segundo análisis modal.

9.11.2. Tarea 2: mecanismo biela–manivela propio y esfuerzos en su biela

Repetir el flujo completo de las Secciones 9.5 y 9.6 con datos distintos: $r = 40$ mm, $L = 160$ mm, velocidad de manivela 1 500 rpm, masa de la corredera 0.8 kg, biela de sección rectangular 10×5 mm de acero.

1. **Cinemática analítica.** Adaptar el Script 3 a los nuevos datos y reportar: carrera, a_{PMS} , a_{PMI} , velocidad máxima y el ángulo donde ocurre.

Tabla 55: Rúbrica de la Tarea 1 (10 puntos).

Criterio	Puntos
Modelo estático correcto (material, soportes, cargas, unidades)	2.0
Estudio de convergencia de malla con extrapolación y discusión de singularidad/concentrador	2.0
Verificación analítica del esfuerzo de flexión y coherencia con ANSYS	1.5
Análisis de fatiga: configuración correcta del <i>Fatigue Tool</i> y comparación Goodman–Gerber razonada	2.5
Análisis modal: identificación de modos, juicio de resonancia y rediseño demostrado	2.0

2. **Rigid Dynamics.** Construir el mecanismo (geometría propia, puede ser esquemática), definir las juntas, verificar la movilidad con la Ec. 249, simular dos revoluciones y superponer los *probes* de posición, velocidad y aceleración de la corredera sobre las curvas analíticas (misma gráfica, error relativo máximo reportado).
3. **Fuerzas de junta.** Graficar la fuerza total de la junta biela–corredera contra el ángulo de manivela e identificar el instante crítico. Discutir la diferencia respecto a la Ec. 250 atribuible a la masa de la biela.
4. **Esfuerzos en la biela.** Aplicar el flujo híbrido de la Figura 85: *Static Structural* de la biela aislada con las cargas del instante crítico, campo de von Mises, factor de seguridad, y verificación analítica de esfuerzo axial y pandeo de Euler. Opcional (+0.5 extra): análisis de *Eigenvalue Buckling* encadenado y comparación con la Ec. 251.

Tabla 56: Rúbrica de la Tarea 2 (10 puntos).

Criterio	Puntos
Cinemática analítica correcta con el script adaptado	1.5
Mecanismo bien construido: juntas correctas y movilidad verificada (Gruebler)	2.0
Superposición cuantitativa <i>probes</i> –analítico con error relativo reportado	2.5
Identificación del instante crítico y análisis de la fuerza de junta	1.5
Análisis estático de la biela con verificación de esfuerzo y pandeo	2.5

10. Semana 11. Validación Experimental y Metrología Computacional

Objetivo de la cátedra. Estructurar e implementar protocolos de validación experimental para modelos numéricos bajo el estándar ASME V&V 20, desacoplando el error de discretización (u_{num}), la incertidumbre propagada de los parámetros de entrada (u_{input}) y la incertidumbre metrológica del experimento (u_D), con el fin de aislar y cuantificar el error de forma del modelo (δ_{model}). El doctorando distinguirá con precisión entre verificación, validación y calibración, y aplicará métricas cuantitativas de comparación —incluida la métrica de área sobre funciones de distribución acumulada— para aceptar o refutar un modelo con criterios normativos.

Temario

- Taxonomía epistemológica: verificación, validación y calibración; el riesgo del sobreajuste.
- Marco formal ASME V&V 20: error de comparación E e incertidumbre de validación u_{val} .
- Cuantificación del error numérico: extrapolación de Richardson y *Grid Convergence Index* (GCI).
- Propagación de la incertidumbre de entrada por series de Taylor (ISO-GUM) y sensibilidad local.
- Metrología experimental: combinación de errores sistemáticos y aleatorios; réplicas y repetibilidad.
- Métricas para respuestas continuas y estocásticas: métrica de área de Oberkampf y Roy sobre CDFs.
- Jerarquía piramidal de validación y trazabilidad en publicaciones de alto impacto.

Resultado de aprendizaje esperado. El investigador en formación diseñará una matriz de validación para su propio proyecto doctoral: calculará el GCI de su discretización, propagará las sensibilidades de sus parámetros de entrada, cuantificará la incertidumbre de su instrumentación y emitirá un diagnóstico cuantitativo de aceptación o refutación del modelo, implementando el flujo completo en Python.

La validación experimental suele confundirse con el ajuste de coeficientes empíricos hasta que la curva computacional coincide visualmente con unos cuantos puntos del experimento. Ese procedimiento es calibración —legítima si se declara— pero no constituye validación. La validación rigurosa parte de la premisa de que todo modelo es una aproximación, y su propósito es *medir* el dominio y la magnitud de esa aproximación: establecer el intervalo cuantitativo dentro del cual las simplificaciones del modelo son aceptables para el diseño o la predicción. Esta semana cierra el ciclo iniciado en la Se-

mana 6 (cuantificación de incertidumbre GUM y Monte Carlo) y en la Semana 8 (convergencia de malla): aquellas herramientas se integran ahora en un protocolo normativo único.

10.1. Fundamentación teórico-matemática

10.1.1. Taxonomía: verificación, validación y calibración

El rigor científico exige separar tres procesos que en la práctica suelen contaminarse mutuamente:

1. **Verificación** (¿resolvemos bien las ecuaciones?): problema exclusivamente matemático y computacional. Comprende la verificación de código (ausencia de errores de programación, orden de convergencia observado igual al teórico) y la verificación de cálculo (estimación del error de discretización de una corrida concreta). No involucra datos experimentales.
2. **Validación** (¿resolvemos las ecuaciones correctas?): problema físico. Compara la predicción del modelo contra observaciones empíricas, con toda su incertidumbre instrumental, para acotar el error de forma del modelo.
3. **Calibración**: ajuste de parámetros físicos desconocidos minimizando la discrepancia frente a un conjunto de datos. *Regla inviolable: un conjunto de datos empleado para calibrar no puede reutilizarse para validar el mismo modelo*; hacerlo convierte la validación en una tautología y al modelo en un interpolador sin capacidad predictiva demostrada.

Enlace con los proyectos del grupo. En un modelo electrotérmico de cable OPGW, verificar es demostrar que el esquema temporal converge con el orden teórico; validar es contrastar la temperatura predicha contra el termopar del banco de cortocircuito; calibrar es ajustar el coeficiente de convección con una parte de los datos —y solo con esa parte—. En un sistema RAG, el análogo es estricto: los documentos usados para afinar los hiperparámetros del recuperador no pueden formar parte del conjunto de evaluación de exactitud.

10.1.2. Marco formal ASME V&V 20: desacoplamiento de incertidumbres

Sea y una cantidad de interés (QoI: esfuerzo máximo, temperatura pico, carga hidráulica, exactitud de recuperación). Sea S la predicción del modelo computacional y D el dato experimental. El error de comparación observable es

$$E = S - D. \quad (252)$$

Ambos valores se refieren a un valor verdadero $\mathcal{T}$, en general incognoscible. El ex-

perimento está afectado por el error de medición: $D = \mathcal{T} + \delta_D$. La simulación acumula tres contribuciones:

$$S = \mathcal{T} + \delta_{num} + \delta_{input} + \delta_{model}, \quad (253)$$

donde δ_{num} es el error de discretización y solución numérica, δ_{input} el error inducido por parámetros de entrada inexactos y δ_{model} el error de forma del modelo (física omitida o simplificada). Sustituyendo en la Ec. (252) y despejando el término de interés:

$$\delta_{model} = E - (\delta_{num} + \delta_{input} - \delta_D). \quad (254)$$

Los errores δ_i son realizaciones desconocidas; se opera con sus incertidumbres estándar u_i . Suponiendo independencia entre fuentes, la **incertidumbre de validación** se combina en cuadratura:

$$u_{val} = \sqrt{u_{num}^2 + u_{input}^2 + u_D^2}. \quad (255)$$

El significado operativo de u_{val} es preciso: es la desviación estándar esperada del error de comparación E si el modelo fuera perfecto ($\delta_{model} = 0$). Todo exceso sistemático de $|E|$ sobre u_{val} es, por lo tanto, evidencia de deficiencia del modelo.

10.1.3. Componente 1: incertidumbre numérica u_{num} (GCI)

La estimación estándar emplea extrapolación de Richardson con tres niveles de discretización (fina h_1 , media h_2 , gruesa h_3) y razón de refinamiento $r = h_2/h_1 = h_3/h_2$. El orden de convergencia observado es

$$p = \frac{\ln\left(\frac{|S_3 - S_2|}{|S_2 - S_1|}\right)}{\ln r}, \quad (256)$$

que debe compararse contra el orden formal del esquema: una discrepancia grande invalida el estudio de convergencia (malla fuera del rango asintótico). El *Grid Convergence Index* sobre la malla fina es

$$\text{GCI} = F_s \frac{|S_1 - S_2|}{r^p - 1}, \quad F_s = 1.25, \quad (257)$$

con F_s el factor de seguridad de Roache para estudios con tres mallas. El GCI se interpreta como una banda expandida de error de discretización; para convertirlo en incertidumbre estándar se asume una distribución uniforme y se aplica

$$u_{num} = \frac{\text{GCI}}{1.15}, \quad (258)$$

criterio recomendado en la literatura de V&V cuando el GCI proviene de un estudio bien comportado dentro del rango asintótico.

10.1.4. Componente 2: incertidumbre de entradas u_{input} (ISO-GUM)

Si el modelo depende de N parámetros de entrada X_i (módulo de Young, coeficiente de convección, viscosidad) con incertidumbres estándar u_{X_i} , la propagación de primer orden por series de Taylor (Semana 6) da

$$u_{input}^2 = \sum_{i=1}^N \left(\frac{\partial S}{\partial X_i} \right)^2 u_{X_i}^2, \quad (259)$$

donde las derivadas parciales son los coeficientes de sensibilidad local, evaluados analíticamente o por diferencias centradas re-ejecutando el solver con $X_i \pm \Delta X_i$. Cuando el modelo es fuertemente no lineal o las incertidumbres de entrada son grandes, la propagación de primer orden se sustituye por muestreo Monte Carlo del ensamble de entradas (JCGM 101), lo cual produce directamente una distribución de S utilizable en la métrica de área de la Ec. (262).

10.1.5. Componente 3: incertidumbre experimental u_D

La caracterización metrológica del dato combina el componente sistemático (Tipo B: certificado de calibración, sesgo residual del instrumento, con incertidumbre estándar u_b) y el componente aleatorio (Tipo A: repetibilidad muestral s con n réplicas independientes):

$$u_D = \sqrt{u_b^2 + \frac{s^2}{n}}, \quad (260)$$

donde s^2/n es la varianza de la *media* experimental. Obsérvese que el término aleatorio se reduce replicando el experimento, mientras que el sistemático solo se reduce con mejor calibración: confundir ambos conduce a presupuestos de incertidumbre irreales. La resolución del indicador digital del instrumento no es u_D ; es apenas una de sus contribuciones Tipo B, frecuentemente la menor.

10.1.6. Regla de decisión

Con factor de cobertura k (usualmente $k = 2$, nivel de confianza aproximado del 95 %):

$$|E| \leq k u_{val} \implies \text{la deficiencia del modelo no es distinguible del ruido conjunto.} \quad (261)$$

En ese caso el modelo *no queda refutado al nivel de resolución del experimento*; afirmar que quedó “demostrado” sería un exceso lógico. Si $|E| > k u_{val}$, existe evidencia cuantitativa de deficiencia de la física o de la formulación, y el propio valor de E acota su magnitud: $\delta_{model} \approx E$ cuando $u_{val} \ll |E|$. Un corolario práctico: reducir u_{val} (mejor

instrumentación, más réplicas, mallas más finas, entradas mejor caracterizadas) aumenta el poder de detección del protocolo; un experimento burdo “valida” cualquier cosa, precisamente porque no puede refutar nada.

10.1.7. Métrica de área para respuestas estocásticas

Cuando la comparación involucra distribuciones completas —ensambles Monte Carlo del modelo contra réplicas experimentales— la comparación de medias es insuficiente: dos distribuciones pueden compartir media y diferir gravemente en dispersión o forma. La métrica de área de Oberkampf y Roy computa el desajuste entre la función de distribución acumulada (CDF) del ensamble simulado, $F_S(y)$, y la CDF empírica experimental, $F_D(y)$:

$$d_{area} = \int_{-\infty}^{\infty} |F_S(y) - F_D(y)| dy. \quad (262)$$

La métrica devuelve un escalar con las unidades físicas de y , lo que permite interpretarla directamente como una “distancia de ingeniería” entre modelo y experimento, y compararla contra tolerancias de diseño. Es la generalización estocástica del error de comparación E : si ambas distribuciones fueran deltas de Dirac, d_{area} se reduce a $|E|$.

10.2. Ejemplo 1. Validación V&V 20 de un modelo térmico transitorio

Se aplica el protocolo completo a un modelo de calentamiento de un tramo de conductor bajo disipación Joule constante (familia de proyectos: comportamiento térmico de cable OPGW ante corriente de falla). El modelo computacional resuelve el balance convectivo

$$mc \frac{dT}{dt} = P - hA(T - T_{amb}), \quad (263)$$

por Euler explícito. El sistema físico real —sintetizado numéricamente para fines didácticos con precisión de referencia— incluye además la pérdida radiativa $\varepsilon\sigma A(T^4 - T_{amb}^4)$, que el modelo omite deliberadamente: esa omisión es la deficiencia δ_{model} que el protocolo debe detectar. El experimento sintético se contamina con un sesgo sistemático realizado de 1.2 K (termopar descalibrado, con incertidumbre estándar de calibración $u_b = 1.0$ K) y ruido aleatorio de 0.8 K por lectura, promediando $n = 5$ réplicas por punto. Los parámetros se resumen en la Tabla 57.

Tabla 57: Parámetros del caso de validación térmica (Ejemplo 1).

Parámetro	Símbolo	Valor	Papel en el protocolo
Potencia disipada	P	25 W	Entrada determinista
Coef. de convección	h	12 ± 1.0 W/(m ² K)	Entrada incierta (u_{input})
Área superficial	A	0.05 m ²	Entrada determinista
Capacitancia térmica	mc	180 J/K	Entrada determinista
Emisividad (física omitida)	ε	0.8	Fuente de δ_{model}
Incert. sistemática del termopar	u_b	1.0 K	Componente Tipo B de u_D
Desv. estándar por lectura	s	0.8 K	Componente Tipo A de u_D
Réplicas por punto	n	5	Reducción de la varianza Tipo A

Verificación (GCI). El estudio de convergencia con pasos $\Delta t = 8, 4$ y 2 s ($r = 2$) en $t_{ref} = 600$ s arroja $S_3 = 336.1787$ K, $S_2 = 336.1030$ K y $S_1 = 336.0653$ K. La Ec. (256) da un orden observado $p = 1.005$, consistente con el orden formal de Euler explícito ($p = 1$): la malla temporal está en el rango asintótico y el estudio es admisible. La Ec. (257) produce $GCI = 0.047$ K y, por la Ec. (258), $u_{num} = 0.041$ K. Como verificación adicional, la distancia de la solución fina a la solución analítica cerrada del modelo es 0.038 K, contenida dentro del GCI: el índice cumple su función de cota conservadora. La Figura 88 muestra la recta de convergencia de pendiente unitaria en escala logarítmica.

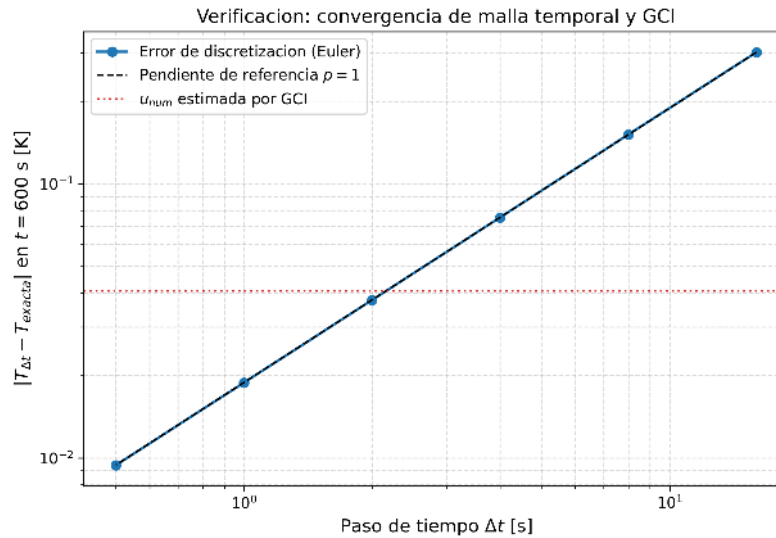


Figura 88: Verificación de cálculo del Ejemplo 1: error de discretización de Euler explícito contra la solución cerrada del modelo en $t = 600$ s. La pendiente observada coincide con el orden formal $p = 1$ y la cota u_{num} estimada por el GCI de la Ec. 257 acota el error real de la malla fina.

Propagación de entrada y metrología. La sensibilidad local $\partial S/\partial h$ se evalúa por diferencias centradas re-ejecutando el solver con $h \pm 0.6 \text{ W}/(\text{m}^2\text{K})$; su magnitud crece con el tiempo hasta 2.79 K por unidad de h , de modo que u_{input} alcanza 2.79 K al final del transitorio (con $u_h = 1.0 \text{ W}/(\text{m}^2\text{K})$). La incertidumbre experimental por punto, Ec. (260), queda entre 1.00 y 1.15 K. El presupuesto está dominado por u_{input} en la región de tiempos largos: el protocolo indica que, para mejorar el poder de detección, el recurso experimental debe invertirse en caracterizar mejor h , no en comprar un termopar más fino.

Diagnóstico. La Figura 89 presenta la comparación continua. En los primeros 240 s el error de comparación queda dentro de la banda $\pm 2u_{val}$ (en $t = 240 \text{ s}$, $E = +2.33 \text{ K}$ contra $2u_{val} = 2.45 \text{ K}$: el punto sobrevive por margen estrecho). A partir de $t = 300 \text{ s}$ ($E = +3.09 \text{ K}$ contra $2u_{val} = 2.74 \text{ K}$) y de manera sostenida hasta el final ($E \approx +10.3 \text{ K}$ contra $2u_{val} \approx 6.0 \text{ K}$), el modelo queda refutado: 11 de 15 puntos exhiben $|E| > 2u_{val}$. La estructura temporal del fallo es informativa por sí misma: la deficiencia crece con la temperatura, exactamente la firma esperada de un término radiativo $\propto T^4$ omitido. Un protocolo de validación bien construido no solo refuta: sugiere *qué* física falta.

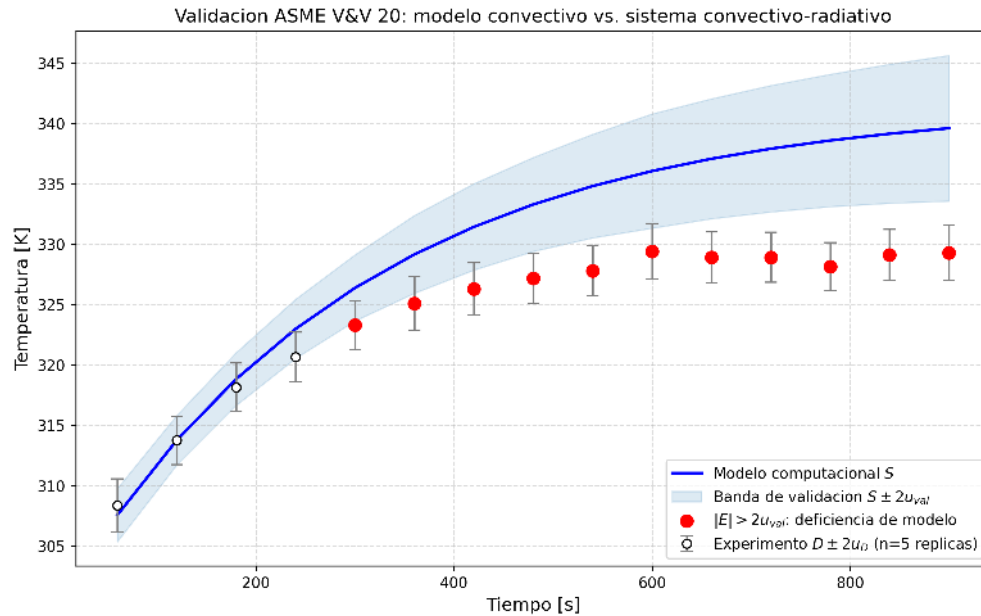


Figura 89: Validación continua del Ejemplo 1 bajo ASME V&V 20. La banda sombreada representa la Ec. 255 expandida con $k = 2$; los puntos rojos marcan las mediciones donde la deficiencia del modelo (omisión del término radiativo) excede el ruido conjunto del protocolo según la Ec. 261.

10.3. Ejemplo 2. Métrica de área sobre CDFs: carga de una bomba centrífuga

Se valida la predicción estocástica de la carga hidráulica H de una bomba centrífuga en su punto de operación nominal $Q = 0.090 \text{ m}^3/\text{s}$ (familia de proyectos: banco de pruebas de bombas centrífugas). El modelo es la curva característica $H(Q) = a - bQ^2$ con coeficientes inciertos $a = 32.0 \pm 0.60 \text{ m}$ y $b = 1250 \pm 80 \text{ m}\cdot\text{s}^2/\text{m}^6$ (incertidumbres estándar del ajuste de la curva del fabricante). La distribución de S se construye con un ensamble Monte Carlo de $M = 20\,000$ muestras; el experimento aporta $n = 40$ réplicas medidas con transductor de presión. El sistema real presenta pérdidas de succión no modeladas, que desplazan la media verdadera por debajo de la predicción.

Los resultados de la corrida verificada son: media del ensamble simulado 21.87 m con desviación 0.89 m; media experimental 21.25 m con desviación 0.55 m; error de comparación de medias $E = 0.62 \text{ m}$; y métrica de área, Ec. (262), $d_{\text{area}} = 0.64 \text{ m}$, equivalente al 3.0 % de la carga medida. La Figura 90 muestra ambas CDFs y el área encerrada. La lectura doctoral es doble: (i) la métrica captura simultáneamente el sesgo de la media y el exceso de dispersión del modelo (el ensamble simulado es más ancho que el experimento, señal de que las incertidumbres declaradas de a y b están sobreestimadas o correlacionadas); y (ii) el veredicto de aceptación no lo da la estadística sola, sino la comparación de d_{area} contra la tolerancia de ingeniería del banco: si la aplicación tolera $\pm 1 \text{ m}$ en la carga, el modelo es adecuado para diseño aun siendo imperfecto.

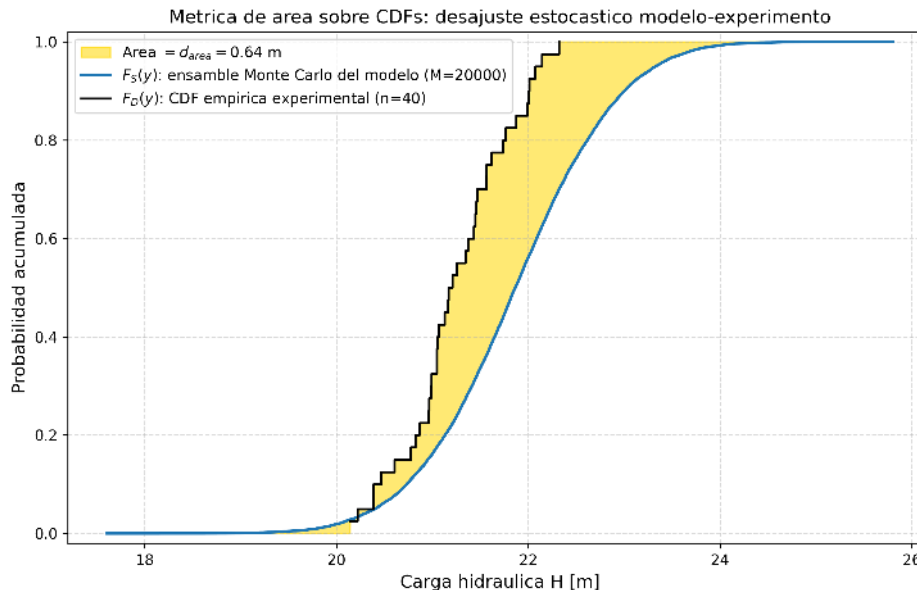


Figura 90: Métrica de área del Ejemplo 2 evaluada con la Ec. 262: CDF del ensamble Monte Carlo del modelo contra CDF empírica de las réplicas experimentales. El área sombreada tiene unidades de carga hidráulica y se compara directamente contra la tolerancia de diseño del banco de pruebas.

Transferencia a otros proyectos. La métrica de área es agnóstica al dominio: aplica a la distribución de deformaciones reconstruidas por un sistema de calibración de sensores (ensamble de calibraciones contra réplicas de patrón), a la distribución de flechas de una losa reticular (Monte Carlo de propiedades del concreto contra nivelaciones en obra), a los tiempos de viaje estimados por monitoreo UAV contra aforos de campo, y a la distribución de exactitud de un sistema RAG (ensamble de corridas con distintas semillas contra evaluación humana). En todos los casos el entregable es el mismo escalár interpretable en unidades físicas de la QoI.

10.4. Jerarquía piramidal de validación

La validación de un sistema complejo no se ejecuta de golpe sobre el ensamble completo: se construye por niveles, validando primero las físicas aisladas donde u_D es pequeña y la detectabilidad de δ_{model} es máxima, y ascendiendo hacia configuraciones acopladas donde la incertidumbre experimental crece y el diagnóstico se vuelve difuso (Figura 91). Publicar la validación de un sistema completo sin haber documentado los niveles inferiores es una debilidad metodológica que los revisores de revistas de alto impacto detectan de inmediato: un acuerdo global puede ser producto de compensación de errores entre subsistemas no validados.



Figura 91: Arquitectura piramidal de validación. En la base, la incertidumbre experimental es mínima y la deficiencia del modelo se detecta con máxima resolución; en la cúspide, el crecimiento de u_D y del acoplamiento entre físicas degrada el poder de refutación de la Ec. 261.

10.5. Tabla de síntesis y control

Tabla 58: Síntesis del protocolo de validación de la Semana 11.

Componente	Símbolo	Método de estimación	Cómo se reduce	Ecuación
Error numérico	u_{num}	Richardson + GCI con tres mallas en rango asintótico	Refinar malla o paso; esquema de mayor orden	Ec. (257), (258)
Entradas inciertas	u_{input}	Taylor 1er orden (GUM) o Monte Carlo (JCGM 101)	Caracterizar mejor los parámetros dominantes	Ec. (259)
Metrología	u_D	Tipo A (réplicas) + Tipo B (calibración)	Más réplicas (solo Tipo A); mejor calibración (Tipo B)	Ec. (260)
Combinación	u_{val}	Cuadratura de las tres fuentes	Atacar la fuente dominante del presupuesto	Ec. (255)
Veredicto puntual	E vs. ku_{val}	Comparación directa	—	Ec. (261)
Veredicto estocástico	d_{area}	Integral entre CDFs	Comparar contra tolerancia de diseño	Ec. (262)

Tabla 59: Resultados de control de los ejemplos verificados de la Semana 11.

Cantidad de control	Ejemplo 1 (térmico)	Ejemplo 2 (bomba)
Orden de convergencia observado p	1.005	—
GCI en malla fina	0.047 K	—
u_{num}	0.041 K	—
Máx. u_{input}	2.79 K	0.89 m (desv. MC)
Rango de u_D	1.00 a 1.15 K	0.087 m (media exp.)
Error de comparación	hasta +10.4 K	0.62 m (medias)
Primera detección de δ_{model}	$t = 300$ s	—
Puntos refutados	11 de 15	—
d_{area}	—	0.64 m (3.0 % de H)

10.6. Uso guiado de inteligencia artificial generativa

Los modelos de lenguaje son útiles para diseñar matrices de experimentos orientadas a desacoplar fuentes de error, siempre que el doctorando audite las ecuaciones propuestas contra el estándar. El siguiente prompt de referencia condiciona a la IA con la estructura formal de esta semana:

Prompt de referencia: DoE para desacoplamiento de incertidumbres

Actúa como auditor de validación de modelos computacionales bajo ASME V&V 20. Mi modelo doctoral predice [QoI, p. ej. el esfuerzo residual en una unión soldada] en función de [entradas, p. ej. potencia y velocidad de proceso]. Mi instrumentación tiene incertidumbre sistemática estándar $u_b = [X]$ y repetibilidad por lectura $s = [Y]$ con posibilidad de n réplicas. Diseña una matriz experimental mínima (ortogonal de Taguchi o Latin Hypercube, justifica la elección) que: (1) maximice la detectabilidad de la deficiencia del modelo frente a u_D ; (2) identifique cuál entrada domina u_{input} en la expansión de Taylor de primer orden; y (3) proponga los puntos del dominio donde $|E| > 2u_{val}$ sería estadísticamente más probable si la física [fenómeno sospechado] estuviera mal modelada. Entrega la matriz en formato tabular de LaTeX y declara explícitamente los supuestos.

La salida de la IA debe auditarse en tres puntos: que la matriz propuesta no confunda réplicas (que reducen s^2/n) con niveles del factor (que exploran sensibilidad); que las posiciones de máxima detectabilidad estén justificadas por la física y no por simetría estética de la matriz; y que ninguna corrida usada para calibrar aparezca en el conjunto de validación.

10.7. Actividad de refuerzo en clase (45 min)

10.7.1. Actividad 1. Auditoría de sobreajuste contra validación genuina

Se proyectan tres fragmentos de secciones metodológicas extraídas de artículos científicos recientes del área de cada dupla de doctorandos.

Consigna. En duplas, auditar metodológicamente los textos e identificar: (i) si los autores calibraron y validaron con el mismo conjunto de datos; (ii) si reportaron un estudio de convergencia de malla o el error numérico se omitió; (iii) si la incertidumbre experimental declarada corresponde a la Ec. (260) o se limitó a la resolución del display del instrumento; y (iv) si el criterio de acuerdo fue cuantitativo ($|E|$ contra u_{val} , o una métrica declarada) o puramente visual.

Rúbrica.

- 3 Identifica las fallas metodológicas y redacta una carta al editor simulada de media cuartilla, argumentando con las ecuaciones del estándar por qué el artículo carece de trazabilidad metrológica.
- 2 Identifica las fallas y las clasifica correctamente, pero la argumentación no se apoya en las ecuaciones del protocolo.

- 1 Detecta anomalías sin lenguaje técnico para justificar el señalamiento.
- 0 Acepta el texto como válido con el argumento de que fue publicado en una revista indexada.

10.8. Ejercicios de investigación para tarea

10.8.1. Ejercicio 1 (obligatorio) — Presupuesto de validación del propio proyecto

Para su investigación doctoral: (i) defina el observable primario S (la QoI de su tesis) y su instrumento de medición; (ii) construya el presupuesto de u_D separando el componente sistemático (certificado de calibración u hoja de datos del equipo) del aleatorio (réplicas propias o repetibilidad declarada), aplicando la Ec. (260); (iii) elija el parámetro de entrada que juzgue dominante y estime su coeficiente de sensibilidad ejecutando su código (FEA, CFD o script) con perturbaciones de más y menos diez por ciento, obteniendo u_{input} por la Ec. (259); y (iv) redacte una sección de 300 palabras, lista para el capítulo metodológico de su tesis, que establezca el umbral de aceptación $|E| \leq k u_{val}$ que su modelo deberá satisfacer, declarando el factor de cobertura y el destino de los datos de calibración.

Rúbrica.

- 3 Presupuesto de u_{val} físicamente fundamentado con las tres componentes; sensibilidad calculada con corridas reales; redacción defendible ante un comité sinodal.
- 2 Cálculos correctos pero sin discriminar u_{num} de u_{input} en el reporte, o sin declarar el destino de los datos de calibración.
- 1 Presupuesto incompleto (una sola fuente) o sensibilidad estimada sin ejecutar el código propio.
- 0 Umbral de aceptación declarado sin presupuesto de incertidumbre.

10.8.2. Ejercicio 2 (obligatorio) — Métrica de área con datos propios

Injecte en el código del Anexo (Sección 10.11) un vector de respuestas de sus propias simulaciones previas (respuesta dinámica, espectro, distribución Monte Carlo) y el vector experimental o de referencia correspondiente. Si su respuesta es estocástica, use directamente la métrica CDF de la Ec. (262) con el código del segundo listado; si es un perfil determinista continuo, reporte además la banda $\pm 2u_{val}$ punto a punto del primer listado. Entregue la gráfica de CDFs (o de banda de validación), el valor de d_{area} en unidades físicas y un párrafo que lo compare contra la tolerancia de ingeniería de su aplicación.

Rúbrica.

- 3 Datos propios integrados; d_{area} correcto y contrastado contra una tolerancia justificada; discusión de qué componente del presupuesto domina.
- 2 Cálculo correcto pero tolerancia de comparación arbitraria o no justificada.
- 1 Ejecuta el código con los datos sintéticos del curso sin adaptarlo a su problema.
- 0 Comparación de picos o medias sin bandas de incertidumbre ni métrica declarada.

10.9. Cierre conceptual

La validación no es un atributo que el modelo adquiere de una vez y para siempre: es un veredicto acotado —válido para una QoI, un dominio de operación y un nivel de incertidumbre declarados— emitido por el método científico. Un modelo comparado contra datos sin presupuesto formal de incertidumbres no está validado en ningún sentido normativo, por convincente que luzca la superposición de curvas. El aporte doctoral no consiste en afirmar que el código acierta, sino en cuantificar, con las Ecs. (255) y (261), dónde y por cuánto empieza a equivocarse; ese mapa de invalidez es, con frecuencia, la contribución científica más citable de una tesis.

10.10. Referencias bibliográficas

1. **ASME**: *Standard for Verification and Validation in Computational Fluid Dynamics and Heat Transfer*, ASME V&V 20-2009 (R2016). American Society of Mechanical Engineers, Nueva York, 2009.
2. **Oberkampf, W. L. y Roy, C. J.**: *Verification and Validation in Scientific Computing*. Cambridge University Press, 2010. Capítulos 8 y 9 (métricas de validación). <https://doi.org/10.1017/CB09780511760396>
3. **Roache, P. J.**: *Fundamentals of Verification and Validation*. Hermosa Publishers, Socorro, NM, 2009. Referencia estándar del *Grid Convergence Index*.
4. **JCGM 100:2008**: *Evaluation of measurement data — Guide to the expression of uncertainty in measurement (GUM)*. BIPM, 2008. <https://doi.org/10.59161/JCGM100-2008E>
5. **Babuška, I. y Oden, J. T.**: “Verification and validation in computational engineering and science: basic concepts”, *Computer Methods in Applied Mechanics and Engineering*, vol. 193, núm. 36–38, pp. 4057–4066, 2004. <https://doi.org/10.1016/j.cma.2004.03.002>

10.11. Anexo: códigos de la Semana 11

Ambos scripts son reproducibles (semilla fija con `numpy.random.default_rng`), generan sus figuras a 300 dpi en la carpeta Figuras/ y ejecutan sin modificación en Google Colab o en instalación local.

10.11.1. Código 1. Framework ASME V&V 20 con GCI y banda de validación

```

1  # =====
2  # Semana11_validacion_vv20.py
3  # Curso: Prototipado y Simulacion (DITA) -- Universidad La Salle Bajio
4  # Semana 11: Validacion Experimental y Metrologia Computacional
5  #
6  # Framework ASME V&V 20 aplicado a un modelo termico transitorio de
7  # calentamiento de un conductor (familia de proyectos: cable OPGW bajo
8  # corriente de falla). El modelo computacional S omite la perdida
9  # radiativa (deficiencia de modelo delta_model deliberada); el
10 # "experimento" D se sintetiza integrando la fisica completa y
11 # contaminando con sesgo sistematico y ruido aleatorio de termopar.
12 #
13 # Componentes calculados:
14 #   u_num   : GCI por extrapolacion de Richardson (3 pasos de tiempo)
15 #   u_input : propagacion GUM de primer orden de u_h (coef. conveccion)
16 #   u_D     : combinacion de sesgo sistematico y repetibilidad (n replicas)
17 #   u_val   : combinacion en cuadratura (ASME V&V 20)
18 #
19 # Reproducibilidad: numpy.random.default_rng(semilla fija)
20 # Salidas: Figuras/S11_validacion_continua.png
21 #          Figuras/S11_gci_convergencia.png
22 # =====
23
24 import os
25 import numpy as np
26 import matplotlib
27 matplotlib.use('Agg')
28 import matplotlib.pyplot as plt
29 from scipy.integrate import solve_ivp
30
31 os.makedirs('Figuras', exist_ok=True)
32 rng = np.random.default_rng(2026)
33
34 # -----
35 # 1. PARAMETROS FISICOS DEL SISTEMA
36 # -----
37 P_joule = 25.0    # [W]      potencia disipada por efecto Joule
38 h_conv = 12.0     # [W/m2K]  coeficiente de conveccion (parametro incierto)
39 u_h = 1.0         # [W/m2K]  incertidumbre estandar de h (correlacion empirica)
40 A_sup = 0.05      # [m2]     area superficial del tramo de conductor
41 mc = 180.0        # [J/K]    capacitancia termica (m*c)
42 T_amb = 300.0     # [K]      temperatura ambiente
43 eps_rad = 0.8     # [-]      emisividad (fisica omitida por el modelo S)
44 sigma_SB = 5.670e-8 # [W/m2K4] constante de Stefan-Boltzmann
45
46 t_final = 900.0   # [s]      horizonte de comparacion
47 n_pts = 15        # puntos de comparacion experimento-modelo
48 t_comp = np.linspace(60.0, t_final, n_pts)
49
50 # -----
51 # 2. MODELO COMPUTACIONAL S (sin radiacion) RESUELTO POR EULER EXPLICITO
52 # -----
53 def solve_model_euler(dt, h_val, t_eval):
54     """Integra dT/dt = (P - h A (T - Tamb)) / mc con Euler explicito y
55     devuelve T interpolada en t_eval. La solucion numerica (no la
56     analitica) es la que se verifica: su error de discretizacion es u_num."""
57     n_steps = int(np.ceil(t_final / dt)) + 1
58     t = np.linspace(0.0, dt * (n_steps - 1), n_steps)
    
```



```

59     T = np.empty(n_steps)
60     T[0] = T_amb
61     for k in range(n_steps - 1):
62         dTdt = (P_joule - h_val * A_sup * (T[k] - T_amb)) / mc
63         T[k + 1] = T[k] + dt * dTdt
64     return np.interp(t_eval, t, T)
65
66 # -----
67 # 3. VERIFICACION: GCI CON TRES PASOS DE TIEMPO (Richardson)
68 # -----
69 r_ref = 2.0                # razon de refinamiento
70 dt3, dt2, dt1 = 8.0, 4.0, 2.0    # grueso, medio, fino
71 t_ref = 600.0              # instante de referencia para el GCI
72 S3 = solve_model_euler(dt3, h_conv, np.array([t_ref]))[0]
73 S2 = solve_model_euler(dt2, h_conv, np.array([t_ref]))[0]
74 S1 = solve_model_euler(dt1, h_conv, np.array([t_ref]))[0]
75
76 p_obs = np.log(abs(S3 - S2) / abs(S2 - S1)) / np.log(r_ref)
77 Fs = 1.25                  # factor de seguridad (Roache)
78 GCI_fine = Fs * abs(S1 - S2) / (r_ref**p_obs - 1.0)    # [K] expandido
79 u_num_scalar = GCI_fine / 1.15    # GCI ~ intervalo uniforme -> estandar
80 u_num = u_num_scalar * np.ones_like(t_comp)
81
82 # Solucion analitica exacta del modelo (solo para la grafica de convergencia)
83 tau = mc / (h_conv * A_sup)
84 T_exact_ref = T_amb + (P_joule / (h_conv * A_sup)) * (1.0 - np.exp(-t_ref / tau))
85
86 # -----
87 # 4. PREDICCIÓN EN LA MALLA FINA Y SENSIBILIDAD u_input
88 # -----
89 S_model = solve_model_euler(dt1, h_conv, t_comp)
90
91 # Sensibilidad local dS/dh por diferencias centradas (recorriendo el solver)
92 delta_h = 0.05 * h_conv
93 S_hp = solve_model_euler(dt1, h_conv + delta_h, t_comp)
94 S_hm = solve_model_euler(dt1, h_conv - delta_h, t_comp)
95 dS_dh = (S_hp - S_hm) / (2.0 * delta_h)
96 u_input = np.abs(dS_dh) * u_h
97
98 # -----
99 # 5. EXPERIMENTO SINTÉTICO D (fisica completa + metrologia de termopar)
100 # -----
101 def rhs_true(t, T):
102     q_conv = h_conv * A_sup * (T[0] - T_amb)
103     q_rad = eps_rad * sigma_SB * A_sup * (T[0]**4 - T_amb**4)
104     return [(P_joule - q_conv - q_rad) / mc]
105
106 sol = solve_ivp(rhs_true, [0.0, t_final], [T_amb], t_eval=t_comp,
107                 rtol=1e-10, atol=1e-10)
108 T_true = sol.y[0]
109
110 b_real = 1.2                # [K] sesgo sistematico realizado (termopar descalibrado)
111 u_b = 1.0                  # [K] incertidumbre estandar del sesgo (certificado)
112 s_rep = 0.8                # [K] desviacion estandar de una lectura individual
113 n_rep = 5                  # replicas por punto de comparacion
114
115 lecturas = (T_true[:, None] + b_real
116             + rng.normal(0.0, s_rep, size=(n_pts, n_rep)))
117 D_exp = lecturas.mean(axis=1)
118 s_muestral = lecturas.std(axis=1, ddof=1)

```

```

119 u_D = np.sqrt(u_b**2 + s_muestral**2 / n_rep)
120
121 # -----
122 # 6. ASME V&V 20: ERROR DE COMPARACION Y BANDA DE VALIDACION
123 # -----
124 E = S_model - D_exp
125 u_val = np.sqrt(u_num**2 + u_input**2 + u_D**2)
126 k_cov = 2.0
127 is_valid = np.abs(E) <= k_cov * u_val
128
129 # -----
130 # 7. REPORTE DE TERMINAL
131 # -----
132 print('=' * 78)
133 print(' REPORTE DE VALIDACION ASME V&V 20 -- MODELO TERMICO TRANSITORIO')
134 print('=' * 78)
135 print('Verificacion (GCI, Euler explicito, t_ref = %.0f s):' % t_ref)
136 print(' S(dt=%.0f)=%.4f K S(dt=%.0f)=%.4f K S(dt=%.0f)=%.4f K'
137       % (dt3, S3, dt2, S2, dt1, S1))
138 print(' Orden observado p = %.3f (teorico Euler: 1)' % p_obs)
139 print(' GCI_fino = %.4f K -> u_num = %.4f K' % (GCI_fine, u_num_scalar))
140 print(' |S_fino - T_exacta| = %.4f K (verificacion contra solucion cerrada)'
141       % abs(S1 - T_exact_ref))
142 print('=' * 78)
143 print('Sensibilidad de entrada: max|dS/dh| = %.3f K/(W/m2K); '
144       'max u_input = %.3f K' % (np.max(np.abs(dS_dh)), np.max(u_input)))
145 print('Metrologia: u_b = %.2f K; s promedio = %.2f K; '
146       'u_D en [%.2f, %.2f] K' % (u_b, s_muestral.mean(),
147                                   u_D.min(), u_D.max()))
148 print('=' * 78)
149 hdr = '%-8s%-10s%-10s%-10s%-9s%-22s' % ('t [s]', 'S [K]', 'D [K]',
150                                           'E [K]', '2u_val', 'Diagnostico')
151 print(hdr)
152 print('=' * 78)
153 for i in range(n_pts):
154     status = 'no refutado' if is_valid[i] else 'DEFICIENCIA DE MODELO'
155     print('%-8.0f%-10.2f%-10.2f%-10.2f%-9.2f%-22s'
156           % (t_comp[i], S_model[i], D_exp[i], E[i],
157               k_cov * u_val[i], status))
158 n_fail = int(np.sum(~is_valid))
159 print('=' * 78)
160 print('Puntos con |E| > 2 u_val: %d de %d '
161       '(primera deteccion en t = %.0f s)'
162       % (n_fail, n_pts, t_comp[~is_valid][0] if n_fail else -1))
163
164 # -----
165 # 8. FIGURA 1: VALIDACION CONTINUA
166 # -----
167 fig, ax = plt.subplots(figsize=(9.5, 6.0), dpi=300)
168 ax.errorbar(t_comp, D_exp, yerr=k_cov * u_D, fmt='o', color='k',
169            mfc='white', ecolor='gray', elinewidth=1.3, capsize=4,
170            label=r'Experimento $D$ \pm 2u_D$ (n=%d replicas)' % n_rep, zorder=4)
171 ax.plot(t_comp, S_model, 'b-', lw=2, label=r'Modelo computacional $$$')
172 ax.fill_between(t_comp, S_model - k_cov * u_val, S_model + k_cov * u_val,
173                color='tab:blue', alpha=0.15,
174                label=r'Banda de validacion $S$ \pm 2u_{val}$')
175 ax.scatter(t_comp[~is_valid], D_exp[~is_valid], color='red', s=70,
176            zorder=5, label=r'$|E| > 2u_{val}$: deficiencia de modelo')
177 ax.set_xlabel('Tiempo [s]', fontsize=12)
178 ax.set_ylabel('Temperatura [K]', fontsize=12)

```

```

179 ax.set_title('Validacion ASME V&V 20: modelo convectivo vs. sistema '
180             'convectivo-radiativo', fontsize=12.5)
181 ax.grid(True, ls='--', alpha=0.45)
182 ax.legend(loc='lower right', fontsize=10, frameon=True)
183 fig.tight_layout()
184 fig.savefig('Figuras/S11_validacion_continua.png')
185 plt.close(fig)
186
187 # -----
188 # 9. FIGURA 2: ESTUDIO DE CONVERGENCIA (GCI)
189 # -----
190 dts = np.array([16.0, 8.0, 4.0, 2.0, 1.0, 0.5])
191 errs = np.array([abs(solve_model_euler(d, h_conv,
192                                np.array([t_ref]))[0] - T_exact_ref) for d in dts])
193
194 fig, ax = plt.subplots(figsize=(7.5, 5.2), dpi=300)
195 ax.loglog(dts, errs, 'o-', color='tab:blue', lw=1.8,
196          label='Error de discretizacion (Euler)')
197 ref = errs[0] * (dts / dts[0])**1.0
198 ax.loglog(dts, ref, 'k--', lw=1.2, label=r'Pendiente de referencia $p=1$')
199 ax.axhline(u_num_scalar, color='tab:red', ls=':', lw=1.5,
200          label=r'$u_{num}$ estimada por GCI')
201 ax.set_xlabel(r'Paso de tiempo $\Delta t$ [s]', fontsize=12)
202 ax.set_ylabel(r'$|T_{\Delta t} - T_{exacta}|$ en $t = 600$ s [K]', fontsize=12)
203 ax.set_title('Verificacion: convergencia de malla temporal y GCI',
204             fontsize=12.5)
205 ax.grid(True, which='both', ls='--', alpha=0.45)
206 ax.legend(fontsize=10)
207 fig.tight_layout()
208 fig.savefig('Figuras/S11_gci_convergencia.png')
209 plt.close(fig)
210 # plt.show()
211 print('\nFiguras escritas en Figuras/.')

```

Listing 50: Protocolo completo del Ejemplo 1: verificación por GCI (Ec. 257), propagación de sensibilidad (Ec. 259), metrología de réplicas (Ec. 260) y diagnóstico punto a punto (Ec. 261).

10.11.2. Código 2. Métrica de área sobre CDFs

```

1 # =====
2 # Semanal1_metrica_area.py
3 # Curso: Prototipado y Simulacion (DITA) -- Universidad La Salle Bajio
4 # Semana 11: Validacion Experimental y Metrologia Computacional
5 #
6 # Metrica de area de Oberkampf & Roy evaluada sobre funciones de
7 # distribucion acumulada (CDF): ensamble Monte Carlo del modelo contra
8 # replicas experimentales. Caso: carga hidraulica H de una bomba
9 # centrifuga en su punto de operacion nominal (familia de proyectos:
10 # banco de pruebas de bombas centrifugas).
11 #
12 # Modelo:       $H(Q) = a - b Q^2$ , con a y b inciertos (hoja de datos +
13 #              ajuste de curva característica).
14 # Experimento: n replicas de H medidas con transductor de presion;
15 #              el sistema real presenta perdidas adicionales no
16 #              modeladas (deficiencia de modelo).
17 #
18 # Salida: Figuras/S11_metrica_cdf.png
19 # Reproducibilidad: numpy.random.default_rng(semilla fija)

```

```

20 # =====
21
22 import os
23 import numpy as np
24 import matplotlib
25 matplotlib.use('Agg')
26 import matplotlib.pyplot as plt
27
28 os.makedirs('Figuras', exist_ok=True)
29 rng = np.random.default_rng(1164)
30
31 # -----
32 # 1. ENSAMBLE MONTE CARLO DEL MODELO (propagacion de u_input)
33 # -----
34 Q_op = 0.090          # [m3/s] caudal nominal de operacion
35 M = 20000             # muestras Monte Carlo
36
37 a_mu, a_u = 32.0, 0.60 # [m]          ordenada de la curva característica
38 b_mu, b_u = 1250.0, 80.0 # [m s2/m6] coeficiente cuadrático
39
40 a_s = rng.normal(a_mu, a_u, M)
41 b_s = rng.normal(b_mu, b_u, M)
42 H_sim = a_s - b_s * Q_op**2
43
44 # -----
45 # 2. REPLICAS EXPERIMENTALES (fisica real con perdidas no modeladas)
46 # -----
47 n_exp = 40
48 H_true_mu = 21.35     # [m] media real: perdidas adicionales de succion
49 s_exp = 0.55          # [m] repetibilidad del transductor + proceso
50 H_exp = rng.normal(H_true_mu, s_exp, n_exp)
51
52 # -----
53 # 3. CDFs EMPIRICAS Y METRICA DE AREA
54 # -----
55 def ecdf(sample, grid):
56     """CDF empirica evaluada sobre 'grid'."""
57     sample_sorted = np.sort(sample)
58     return np.searchsorted(sample_sorted, grid, side='right') / len(sample)
59
60 y_lo = min(H_sim.min(), H_exp.min()) - 0.5
61 y_hi = max(H_sim.max(), H_exp.max()) + 0.5
62 y_grid = np.linspace(y_lo, y_hi, 6000)
63
64 F_S = ecdf(H_sim, y_grid)
65 F_D = ecdf(H_exp, y_grid)
66
67 d_area = np.trapezoid(np.abs(F_S - F_D), y_grid) # [m], unidades de H
68
69 # Estadisticos de referencia para el reporte
70 E_medias = H_sim.mean() - H_exp.mean()
71 u_input_H = H_sim.std(ddof=1) # dispersion inducida por entradas
72 u_D_H = H_exp.std(ddof=1) / np.sqrt(n_exp) # incertidumbre de la media exp.
73
74 print('=' * 72)
75 print(' METRICA DE AREA (OBERKAMPF & ROY) -- CARGA DE BOMBA CENTRIFUGA')
76 print('=' * 72)
77 print('Modelo (MC, M=%d): media = %.3f m; desv = %.3f m'
78       % (M, H_sim.mean(), u_input_H))
79 print('Experimento (n=%d): media = %.3f m; desv = %.3f m'

```

```

80     % (n_exp, H_exp.mean(), H_exp.std(ddof=1)))
81 print('Error de comparacion de medias E = %.3f m' % E_medias)
82 print('Metrica de area d_area = %.3f m' % d_area)
83 print('Fraccion de d_area respecto a la media experimental: %.2f por ciento'
84       % (100.0 * d_area / H_exp.mean()))
85
86 # -----
87 # 4. FIGURA: CDFs Y AREA ENCERRADA
88 # -----
89 fig, ax = plt.subplots(figsize=(9.0, 5.8), dpi=300)
90 ax.fill_betweenx(F_S, y_grid, y_grid, color='none') # noop para orden de capas
91 ax.fill_between(y_grid, F_S, F_D, color='gold', alpha=0.55,
92               label=r'Area $= d_{area} = %.2f$ m' % d_area)
93 ax.plot(y_grid, F_S, color='tab:blue', lw=2,
94        label=r'$F_S(y)$: ensamble Monte Carlo del modelo (M=%d)' % M)
95 ax.step(np.sort(H_exp), np.arange(1, n_exp + 1) / n_exp, where='post',
96        color='k', lw=1.6,
97        label=r'$F_D(y)$: CDF empirica experimental (n=%d)' % n_exp)
98 ax.set_xlabel('Carga hidraulica H [m]', fontsize=12)
99 ax.set_ylabel('Probabilidad acumulada', fontsize=12)
100 ax.set_title('Metrica de area sobre CDFs: desajuste estocastico '
101             'modelo-experimento', fontsize=12.5)
102 ax.set_ylim(-0.02, 1.02)
103 ax.grid(True, ls='--', alpha=0.45)
104 ax.legend(loc='upper left', fontsize=10, frameon=True)
105 fig.tight_layout()
106 fig.savefig('Figuras/S11_metrica_cdf.png')
107 plt.close(fig)
108 # plt.show()
109 print('\nFigura escrita en Figuras/S11_metrica_cdf.png')

```

Listing 51: Métrica de área de Oberkampf y Roy del Ejemplo 2 (Ec. 262): ensamble Monte Carlo del modelo contra CDF empírica experimental.

11. Semana 12 y 13. Fotogrametría y Escaneo 3D

Objetivo de la cátedra. Integrar la reconstrucción tridimensional óptica como puente metrológicamente trazable entre el sistema físico y su modelo de simulación. El doctorando formulará el problema de fotogrametría multivista desde el modelo proyectivo de cámara hasta el ajuste de haces, establecerá el presupuesto de resolución e incertidumbre de una sesión de captura *antes* de ejecutarla, procesará la nube de puntos resultante hasta obtener una malla triangular estanca y auditada topológicamente, y determinará bajo qué condiciones esa malla es admisible como dominio de cálculo para análisis por elementos finitos. La reconstrucción se tratará, en todo momento, como una medición con incertidumbre declarada y no como una operación de visualización.

Temario

- Introducción a la fotogrametría: modelo *pinhole*, distorsión de lente y geometría

epipolar.

- Reconstrucción 3D: *Structure from Motion*, ajuste de haces, *Multi-View Stereo* y ambigüedad de similitud.
- Nubes de puntos: muestreo, ruido, valores atípicos, filtrado estadístico, submuestreo y estimación de normales.
- Mallas tridimensionales: reconstrucción implícita de superficies, auditoría topológica, suavizado y decimación.
- Escaneo 3D: protocolo operativo con Meshroom 2023.3.0 (AliceVision), escalamiento metrológico y criterios de aceptación.
- Preparación geométrica para simulación: de la malla de escaneo al dominio de cálculo admisible en FEM.
- Aplicaciones en ingeniería: geometría *as-built*, verificación dimensional y alimentación de gemelos digitales.

Resultado de aprendizaje esperado. El investigador en formación ejecutará un escaneo fotogramétrico completo de una pieza física, entregará la nube densa y la malla triangular en formato STL, y acompañará ambas con un presupuesto de incertidumbre que declare la resolución lateral esperada, la incertidumbre de profundidad, el factor de escala y su incertidumbre, y la desviación verificada contra cotas medidas con instrumento calibrado. La entrega se juzgará con los mismos criterios normativos introducidos en la Semana 11.

La reconstrucción 3D suele presentarse como un procedimiento de captura: se toman fotografías, se ejecuta un programa y aparece un modelo. Esa descripción omite lo único que importa para una tesis doctoral. Un modelo obtenido por fotogrametría es el resultado de un problema de estimación no lineal sobredeterminado, con una geometría de adquisición que el investigador diseña y cuyas propiedades determinan por completo la exactitud del producto. Además, la solución queda intrínsecamente indeterminada en escala: sin una referencia métrica externa, la reconstrucción es correcta salvo un factor multiplicativo desconocido, y por lo tanto *no es una medición*. Esta semana cierra el arco que va de la Semana 8 —donde la malla era un objeto que el analista generaba a partir de un CAD— a la situación inversa: la malla proviene ahora del mundo físico, arrastra el error de un proceso óptico, y debe someterse a verificación antes de admitirse como dominio de cálculo.

11.1. Fundamentación teórico-matemática

El problema que resuelve la fotogrametría es de naturaleza estrictamente geométrica y estadística: reconstruir la posición tridimensional de una escena a partir de un conjunto de proyecciones bidimensionales redundantes, sin más información que el modelo de formación de la imagen y la correspondencia entre puntos homólogos. Esta sección desarrolla esa cadena en el orden en que se resuelve computacionalmente: primero el modelo

proyectivo que relaciona un punto del mundo con su píxel en el sensor —incluida la corrección de distorsión de lente, sin la cual ningún ajuste posterior converge a la solución correcta—; después la geometría epipolar y la triangulación, que invierten esa proyección a partir de múltiples vistas; el ajuste de haces (*bundle adjustment*) como problema de optimización no lineal que estima simultáneamente poses de cámara y estructura 3D; la densificación MVS, que multiplica la nube dispersa en una representación densa de la superficie; y, por último, el mecanismo por el cual una reconstrucción intrínsecamente adimensional adquiere una escala métrica trazable y una incertidumbre asociada. Cada eslabón de esta cadena introduce una fuente de error propia, y es esa propagación —no la simple ejecución del *pipeline*— lo que determina si el modelo resultante es admisible como dato de ingeniería.

11.1.1. Modelo proyectivo de cámara y distorsión de lente

Un punto del objeto $\mathbf{P} = [X, Y, Z]^\top$, expresado en el sistema de referencia del mundo, se proyecta sobre el plano imagen según el modelo de cámara estenopeica (*pinhole*):

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{K}} \underbrace{\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}}_{\text{extrínsecos}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \quad (264)$$

donde $\mathbf{K}$ contiene los parámetros intrínsecos (distancias focales en píxeles f_x, f_y y punto principal c_x, c_y), $\mathbf{R} \in SO(3)$ y $\mathbf{t} \in \mathbb{R}^3$ describen la pose de la cámara, y s es el factor de escala proyectivo. La relación entre la focal física f [mm] y la focal en píxeles es $f_x = f/p$, con p el paso de píxel del sensor.

La lente real introduce una desviación respecto del modelo lineal. El modelo de Brown–Conrady corrige las coordenadas normalizadas $(x_n, y_n) = ((u - c_x)/f_x, (v - c_y)/f_y)$ mediante

$$\begin{aligned} x_d &= x_n (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 x_n y_n + p_2 (r^2 + 2x_n^2), \\ y_d &= y_n (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1 (r^2 + 2y_n^2) + 2p_2 x_n y_n, \end{aligned} \quad r^2 = x_n^2 + y_n^2, \quad (265)$$

con k_i los coeficientes radiales y p_i los tangenciales. En fotogrametría multivista, $\mathbf{K}$ y los coeficientes de la Ec. (265) se estiman *dentro* del propio ajuste (autocalibración), a diferencia de la calibración previa con tablero de ajedrez usada en visión estéreo clásica.

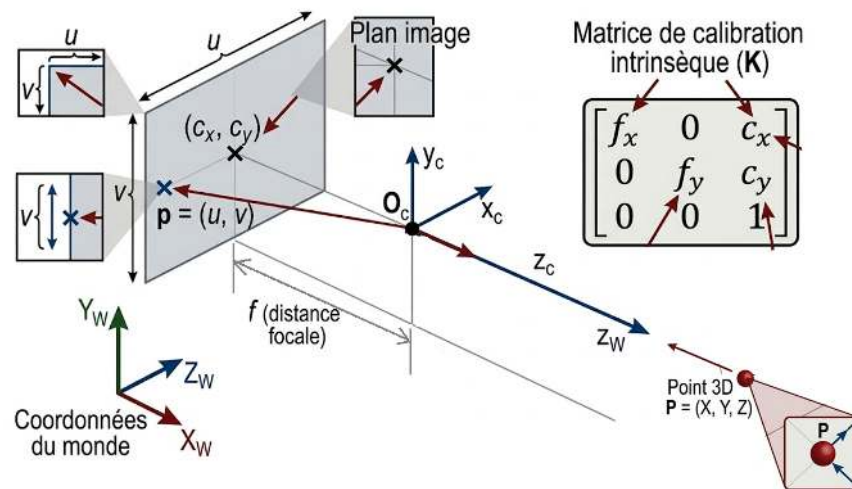


Figura 92: Modelo de cámara estenopeica. Relación entre el punto del objeto, el centro óptico, la distancia focal y la proyección sobre el plano imagen, con los parámetros intrínsecos de la Ec. 264.

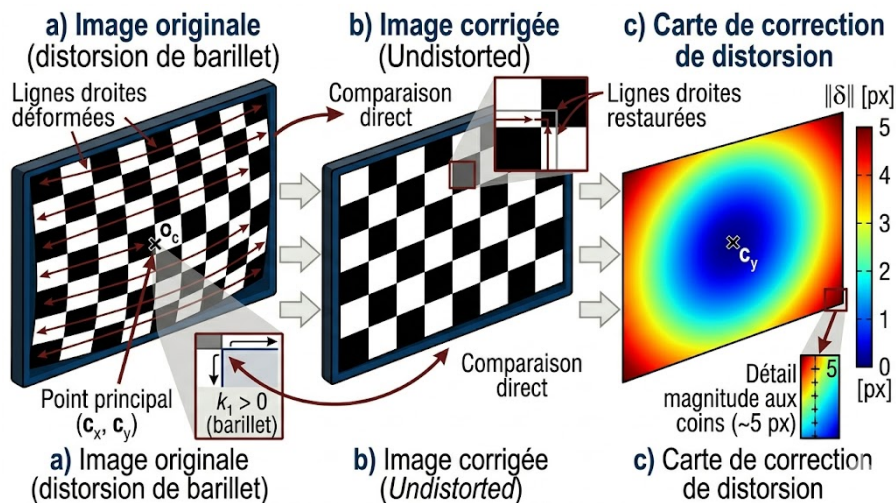


Figura 93: Distorsión de lente descrita por la Ec. 265. Campo de desplazamiento radial y tangencial y su efecto sobre una retícula rectilínea. El residual de distorsión no corregido se propaga directamente al error de triangulación.

Modos computacionales de cámara. Los teléfonos móviles recientes aplican fusión multicuadro, corrección geométrica y súperresolución dependientes de la escena. Estas operaciones violan el supuesto de que todas las imágenes provienen de una misma cámara con intrínsecos fijos y rompen la Ec. (264). Debe desactivarse HDR, modo nocturno, modo retrato y estabilización computacional, fijar la exposición y el enfoque, y utilizar exclusivamente la cámara principal.

11.1.2. Geometría epipolar y triangulación

Dos vistas de un mismo punto satisfacen la restricción epipolar

$$\tilde{\mathbf{x}}_2^\top \mathbf{F} \tilde{\mathbf{x}}_1 = 0, \quad \mathbf{F} = \mathbf{K}_2^{-\top} [\mathbf{t}]_{\times} \mathbf{R} \mathbf{K}_1^{-1}, \quad (266)$$

donde $\tilde{\mathbf{x}}_i$ son coordenadas homogéneas de imagen y $\mathbf{F}$ es la matriz fundamental. La Ec. (266) es la herramienta con la que RANSAC descarta correspondencias falsas: un emparejamiento que no satisface la restricción dentro de una tolerancia en píxeles es un *outlier* geométrico.

Conocidas $m \geq 2$ matrices de proyección $\mathbf{P}_i = \mathbf{K}_i[\mathbf{R}_i|\mathbf{t}_i]$ y las observaciones (u_i, v_i) , la posición 3D se obtiene por transformación lineal directa (DLT) resolviendo el sistema homogéneo

$$\mathbf{A} \tilde{\mathbf{X}} = \mathbf{0}, \quad \mathbf{A} = \begin{bmatrix} u_1 \mathbf{p}_1^3 - \mathbf{p}_1^1 \\ v_1 \mathbf{p}_1^3 - \mathbf{p}_1^2 \\ \vdots \end{bmatrix}, \quad (267)$$

con $\mathbf{p}_i^k$ la k -ésima fila de $\mathbf{P}_i$; la solución es el vector singular derecho asociado al valor singular mínimo de $\mathbf{A}$.

La propiedad de diseño más importante es la sensibilidad a la profundidad. Para un par con base B y profundidad Z , la relación $Z = f_{px} B / d$ entre profundidad y disparidad d implica, por propagación de primer orden,

$$\sigma_Z = \frac{Z^2}{f_{px} B} \sigma_d, \quad (268)$$

donde σ_d es la incertidumbre de correspondencia en píxeles. El error crece con el *cuadrado* de la distancia de trabajo y decrece con la base: acercarse al objeto es cuadráticamente más eficaz que mejorar el detector de puntos característicos.

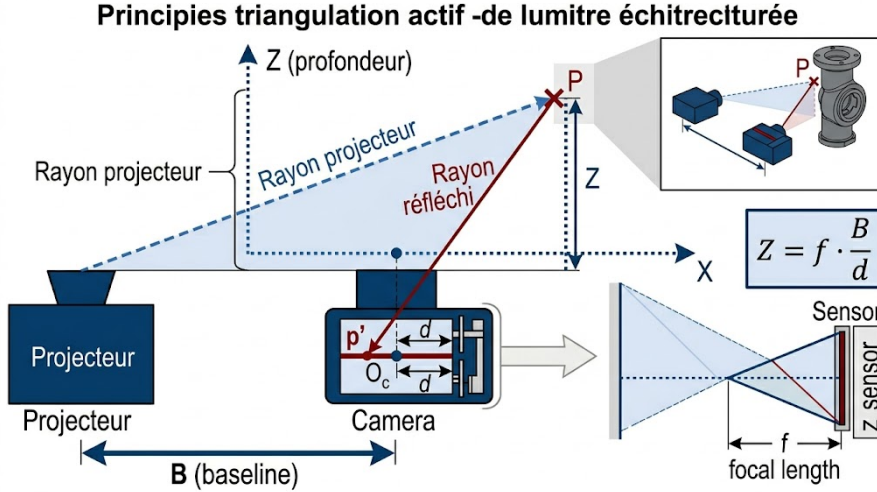


Figura 94: Geometría de triangulación. La incertidumbre de profundidad crece con Z^2 y decrece con la base B según la Ec. 268; el volumen de intersección de los conos de incertidumbre se alarga en la dirección del eje óptico para bases pequeñas.

11.1.3. Structure from Motion y ajuste de haces

El pipeline SfM estima simultáneamente las poses de m cámaras y las posiciones de n puntos a partir únicamente de las correspondencias de imagen. Sus etapas son: detección de puntos característicos (SIFT, DSP-SIFT, AKAZE), emparejamiento de descriptores, filtrado geométrico con la Ec. (266), reconstrucción incremental y refinamiento global. Este último es el **ajuste de haces** (*bundle adjustment*), un problema de mínimos cuadrados no lineales de gran escala:

$$\min_{\{\mathbf{K}_i, \mathbf{R}_i, \mathbf{t}_i\}, \{\mathbf{X}_j\}} \sum_{i=1}^m \sum_{j=1}^n w_{ij} \rho(\|\pi(\mathbf{K}_i, \mathbf{R}_i, \mathbf{t}_i, \mathbf{X}_j) - \mathbf{x}_{ij}\|^2), \quad (269)$$

donde $\pi(\cdot)$ es la proyección de las Ecs. (264)–(265), w_{ij} indica visibilidad y $\rho(\cdot)$ es una función de pérdida robusta (Huber, Cauchy) que acota la influencia de correspondencias erróneas supervivientes. El sistema normal asociado es disperso con estructura de flecha, lo que permite resolverlo con el complemento de Schur en tiempo aproximadamente lineal en n .

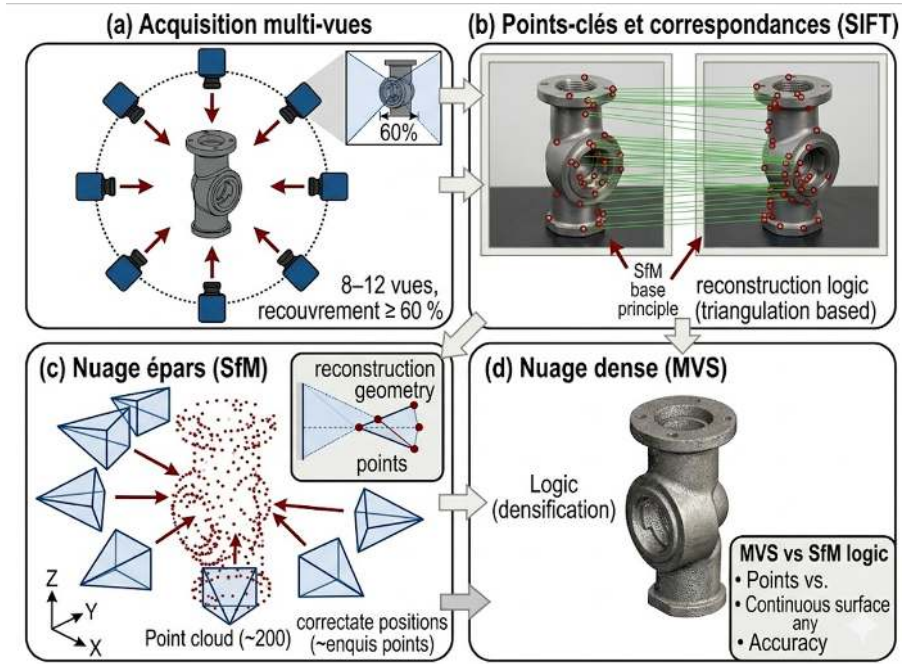


Figura 95: Pipeline de fotogrametría *Structure from Motion*. **(a)** Adquisición desde múltiples ángulos. **(b)** Detección de puntos característicos y correspondencias entre dos vistas. **(c)** Nube dispersa y poses estimadas por el ajuste de haces de la Ec. 269. **(d)** Nube densa obtenida por *Multi-View Stereo*.

Indeterminación de calibre. La función objetivo (269) es invariante ante cualquier transformación de similitud $\mathcal{S}(s, \mathbf{R}_0, \mathbf{t}_0)$ aplicada conjuntamente a la estructura y a las cámaras:

$$\mathbf{X}_j \mapsto s \mathbf{R}_0 \mathbf{X}_j + \mathbf{t}_0, \quad \mathbf{R}_i \mapsto \mathbf{R}_i \mathbf{R}_0^\top, \quad \mathbf{t}_i \mapsto s \mathbf{t}_i - \mathbf{R}_i \mathbf{R}_0^\top \mathbf{t}_0. \quad (270)$$

La solución posee, en consecuencia, siete grados de libertad no observables (tres de traslación, tres de rotación y uno de escala). Los seis primeros son irrelevantes para la metrología dimensional; el séptimo no lo es. *Toda reconstrucción SfM carece de escala métrica hasta que se introduce una referencia externa.*

11.1.4. Densificación MVS y resolución del muestreo

Fijadas las poses, el módulo *Multi-View Stereo* calcula un mapa de profundidad por imagen maximizando la fotoconsistencia entre parches homólogos —típicamente correlación cruzada normalizada— y fusiona los mapas en una nube densa. La resolución lateral alcanzable está acotada por la distancia de muestreo sobre el objeto o *ground sampling distance*:

$$\text{GSD} = \frac{p Z}{f}, \quad (271)$$

con p el paso de píxel, Z la distancia de trabajo y f la focal. Ningún detalle geométrico menor que unos pocos GSD es recuperable, con independencia de los parámetros del programa.

El MVS exige textura superficial. Una superficie lambertiana uniforme, pulida o translúcida no genera gradientes de intensidad y la correlación se vuelve indeterminada: el resultado son huecos o superficies fantasma. La solución estándar es la aplicación de un agente mateante volátil (revelador de líquidos penetrantes) que deposita una capa delgada de espesor típico entre 5 y 15 μm ; ese espesor es un sesgo sistemático que debe declararse cuando el criterio de aceptación dimensional esté en el mismo orden de magnitud.

11.1.5. Escala metrológica y propagación de su incertidumbre

La escala se fija con una referencia calibrada de longitud conocida L_{real} visible en el mayor número posible de imágenes. Medida su longitud L_{mod} en el modelo, el factor de escala es

$$s = \frac{L_{\text{real}}}{L_{\text{mod}}}, \quad (272)$$

y todas las coordenadas se multiplican por s . Si cada extremo de la referencia se localiza con incertidumbre tridimensional isótropa u_p , la proyección sobre el eje de la barra de las dos contribuciones independientes da

$$\frac{u_s}{s} = \frac{\sqrt{2} u_p}{\sqrt{3} L_{\text{real}}} \oplus \frac{u_{\text{cal}}}{L_{\text{real}}}, \quad (273)$$

donde u_{cal} es la incertidumbre del certificado de calibración de la barra y $\oplus$ denota suma en cuadratura. El resultado es central para el diseño experimental: el error relativo de *todo* el modelo es inversamente proporcional a la longitud de la referencia. Una barra corta contamina sistemáticamente cada cota del objeto, y ese sesgo no se reduce tomando más fotografías. La componente u_s/s se incorpora al presupuesto u_{input} de la Semana 11 como una incertidumbre de escala multiplicativa.

11.2. De la nube de puntos a la malla triangular

11.2.1. La nube como muestreo ruidoso de una variedad

Formalmente, una nube de puntos es un conjunto finito $\mathcal{P} = \{\mathbf{p}_i\}_{i=1}^N \subset \mathbb{R}^3$ que muestrea una variedad bidimensional $\mathcal{M}$ embebida en $\mathbb{R}^3$, contaminado por ruido y por valores atípicos:

$$\mathbf{p}_i = \mathbf{q}_i + \varepsilon_i \hat{\mathbf{n}}(\mathbf{q}_i) + \mathbf{o}_i, \quad \mathbf{q}_i \in \mathcal{M}, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_n^2), \quad (274)$$

donde o_i es no nulo únicamente para la fracción de *outliers*. El objetivo del preprocesamiento es estimar $\mathcal{M}$ a partir de $\mathcal{P}$; toda la cadena posterior consiste en reducir la varianza de ε sin introducir sesgo geométrico.

Filtrado estadístico de valores atípicos (SOR). Sea d_i la distancia media de p_i a sus k vecinos más próximos. El criterio clásico descarta el punto si

$$d_i > \mu_d + n_\sigma \sigma_d, \quad (275)$$

con μ_d y σ_d la media y la desviación muestral de $\{d_i\}$. El criterio (275) tiene un defecto conocido: los propios *outliers* inflan σ_d y elevan el umbral, de modo que el filtro se autolimita. La versión robusta sustituye los estimadores muestrales por la mediana y la desviación absoluta mediana escalada,

$$d_i > \text{med}(d) + n_\sigma \cdot 1.4826 \text{ MAD}(d), \quad \text{MAD}(d) = \text{med}(|d - \text{med}(d)|), \quad (276)$$

y se aplica en dos pasadas. El factor 1.4826 hace consistente el estimador con σ bajo normalidad. El efecto cuantitativo de esta sustitución se mide en el Ejemplo 3.

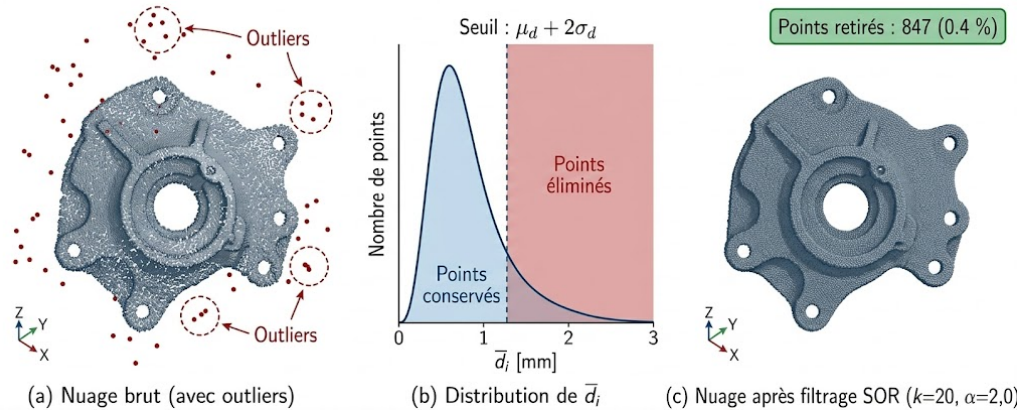


Figura 96: Filtrado estadístico de valores atípicos. Distribución de la distancia media a los k vecinos y umbral de corte de la Ec. 275; nube antes y después del filtrado.

11.2.2. Submuestreo por vóxeles y estimación de normales

El submuestreo por rejilla de vóxeles particiona el espacio en celdas cúbicas de arista a_v y sustituye los puntos de cada celda por su centroide. La operación es un filtro pasabajas espacial: reduce el número de puntos de forma aproximadamente proporcional a a_v^{-2} (por tratarse de un muestreo superficial) y atenúa el ruido en un factor $\sqrt{n_{\text{celda}}}$, a costa de una resolución geométrica limitada a a_v .

Las normales se estiman por análisis de componentes principales de la matriz de

covarianza local. Para el vecindario $\mathcal{N}_k(\mathbf{p}_i)$ con centroide $\bar{\mathbf{p}}_i$,

$$\mathbf{C}_i = \frac{1}{k} \sum_{\mathbf{p}_j \in \mathcal{N}_k(\mathbf{p}_i)} (\mathbf{p}_j - \bar{\mathbf{p}}_i)(\mathbf{p}_j - \bar{\mathbf{p}}_i)^\top, \quad \mathbf{C}_i \mathbf{v}_\ell = \lambda_\ell \mathbf{v}_\ell, \quad \lambda_0 \leq \lambda_1 \leq \lambda_2, \quad (277)$$

y $\hat{\mathbf{n}}_i = \mathbf{v}_0$. El cociente

$$\sigma_{\text{sup}}(\mathbf{p}_i) = \frac{\lambda_0}{\lambda_0 + \lambda_1 + \lambda_2} \quad (278)$$

es la **variación de superficie**, un estimador de curvatura barato y útil como criterio de segmentación: $\sigma_{\text{sup}} \rightarrow 0$ en regiones planas y crece en aristas y esquinas.

El PCA determina la dirección de la normal pero no su *sentido*. La orientación global se resuelve propagando el signo sobre el árbol de expansión mínima del grafo de vecinos próximos, ponderado con $w_{ij} = 1 - |\hat{\mathbf{n}}_i \cdot \hat{\mathbf{n}}_j|$: el recorrido atraviesa primero las regiones donde las normales son casi paralelas y difiere las decisiones ambiguas a las aristas, donde la propagación ya está restringida por el contexto.

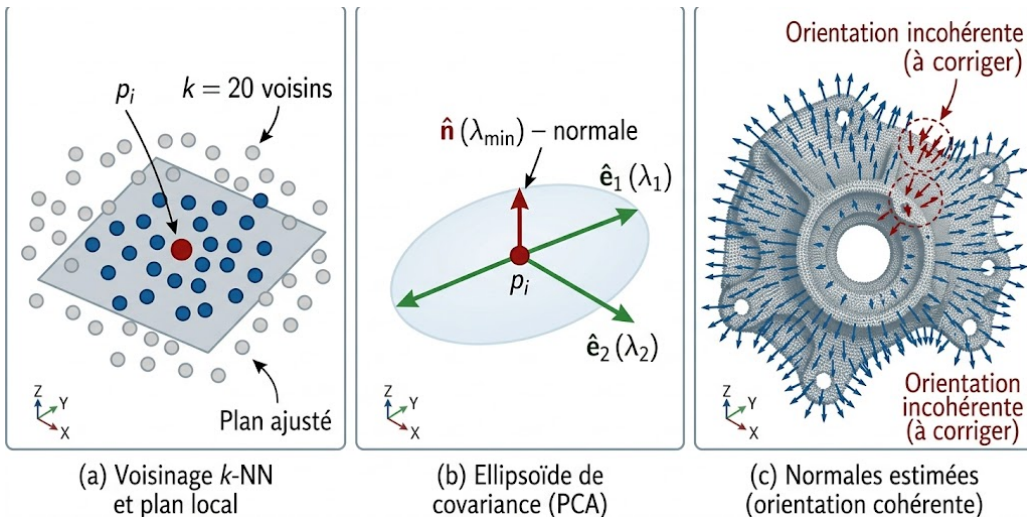


Figura 97: Estimación de normales por PCA local, Ec. 277. Plano de mejor ajuste en un vecindario, autovector asociado al autovalor mínimo y campo de normales orientado consistentemente.

11.2.3. Registro multivista: el algoritmo ICP

Cuando el objeto exige varias sesiones —por ejemplo, para capturar la base— las nubes parciales deben alinearse. El algoritmo *Iterative Closest Point* alterna asignación de correspondencias por proximidad y minimización de una métrica de alineación. La variante punto a punto resuelve

$$\min_{\mathbf{R}, \mathbf{t}} \sum_i \|\mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_i\|^2, \quad (279)$$

cuya solución cerrada se obtiene por descomposición en valores singulares de la matriz de covarianza cruzada. La variante punto a plano,

$$\min_{\mathbf{R}, \mathbf{t}} \sum_i [(\mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_i) \cdot \hat{\mathbf{n}}_{q_i}]^2, \quad (280)$$

converge típicamente en un orden de magnitud menos de iteraciones porque permite el deslizamiento tangencial sobre superficies planas, penalizando solo la componente normal del residual.

Cuenca de atracción. El ICP es un método de descenso local: sin una alineación inicial adecuada converge a un mínimo local sin aviso de error. La regla práctica exige un solapamiento superior al 30 % entre nubes y un desalineamiento inicial inferior a unos 15°. Para inicializar de forma automática se emplea el emparejamiento de descriptores FPFH con RANSAC. Un valor bajo de la métrica residual no demuestra un registro correcto: debe verificarse siempre con inspección visual del mapa de distancias entre nubes.

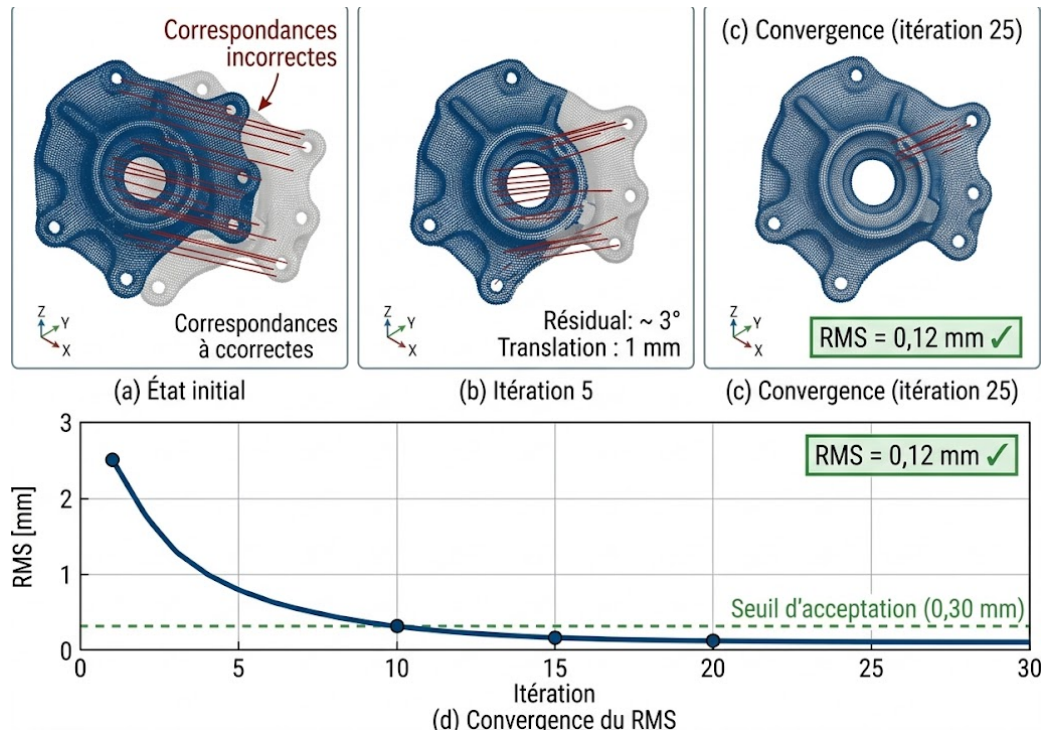


Figura 98: Esquema del algoritmo ICP. Asignación de correspondencias por vecino más próximo, métricas punto a punto (Ec. 279) y punto a plano (Ec. 280), y evolución del residual con las iteraciones.

11.2.4. Reconstrucción implícita de la superficie

El paso de la nube orientada a la malla se plantea como la extracción de una isosuperficie de un campo escalar. En la formulación de Poisson se busca la función indicadora χ cuyo gradiente aproxima mejor el campo vectorial $\vec{V}$ interpolado a partir de las normales:

$$\min_{\chi} \left\| \nabla \chi - \vec{V} \right\|^2 \iff \nabla^2 \chi = \nabla \cdot \vec{V}. \quad (281)$$

La variante *screened* añade un término que ancla la superficie a las muestras,

$$\min_{\chi} \left\| \nabla \chi - \vec{V} \right\|^2 + \lambda \sum_{i=1}^N (\chi(\mathbf{p}_i) - s_i)^2, \quad (282)$$

con λ el peso de datos (`pointWeight`). La discretización se realiza sobre un *octree* de profundidad `depth`, que fija la resolución en 2^{depth} celdas por eje.

Una alternativa de implementación transparente —la utilizada en el Ejemplo 3— estima directamente una función de distancia con signo por mínimos cuadrados móviles sobre los planos tangentes locales:

$$f(\mathbf{x}) = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i(\mathbf{x}) [(\mathbf{x} - \mathbf{p}_i) \cdot \hat{\mathbf{n}}_i]}{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i(\mathbf{x})}, \quad w_i(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{p}_i\|^2}{h^2}\right), \quad (283)$$

seguida de la extracción de la isosuperficie $f = 0$ por *Marching Cubes*. La ventaja pedagógica es que cada término de la Ec. (283) es auditable; la ventaja práctica es que el campo se evalúa únicamente en una cáscara de radio $r_{\text{máx}}$ alrededor de la nube, y el interior y el exterior se etiquetan por componentes conexas.

La cáscara falsa. Todo método implícito global produce una superficie *cerrada*, incluso donde no hay datos. Las regiones no escaneadas —típicamente la base de la pieza— se rellenan con una extrapolación que puede alejarse arbitrariamente de la geometría real. Esa cáscara debe recortarse antes de cualquier medición: por umbral de densidad del campo escalar en Poisson, o por máscara de distancia a la nube en la Ec. (283). Una malla de Poisson sin recortar contamina el volumen, el área y todo el mapa de desviaciones.

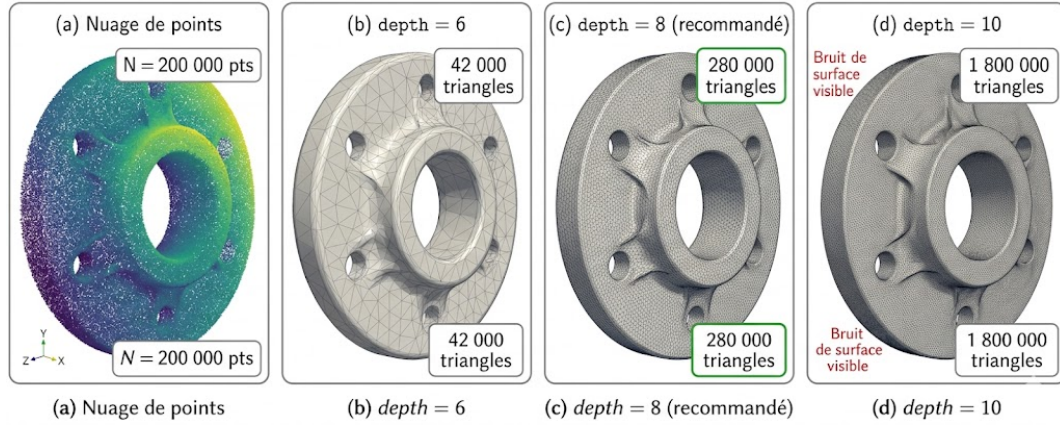


Figura 99: Reconstrucción de Poisson, Ec. 282, para distintos valores de *depth*. El compromiso es entre aristas redondeadas (profundidad baja) y ruido superficial amplificado (profundidad alta); se indica el número de triángulos resultante en cada caso.

11.2.5. Auditoría topológica de la malla

Una malla apta para ingeniería no es la que *parece* correcta, sino la que satisface criterios topológicos verificables. La relación de Euler–Poincaré para una superficie cerrada, orientable y conexa de género g establece

$$\chi = V - E + F = 2(1 - g), \quad (284)$$

con V , E y F el número de vértices, aristas y caras. El género coincide con el número de asas: una pieza sin barrenos pasantes debe dar $\chi = 2$; con un barreno pasante, $\chi = 0$. La verificación de la Ec. (284) detecta de inmediato piezas espurias desprendidas y superficies duplicadas.

El criterio de estanqueidad se expresa como

$$\text{malla estanca} \iff \begin{cases} \text{aristas de frontera} = 0 & (\text{cada arista con dos caras}), \\ \text{aristas no-manifold} = 0 & (\text{ninguna arista con más de dos caras}), \\ \text{caras degeneradas} = 0 & (\text{área} > 0), \\ \text{autointersecciones} = 0. \end{cases} \quad (285)$$

Una malla que no satisface la Ec. (285) no define un volumen: no puede laminarse para manufactura aditiva, no puede convertirse a sólido para FEM y sus propiedades de masa carecen de sentido.

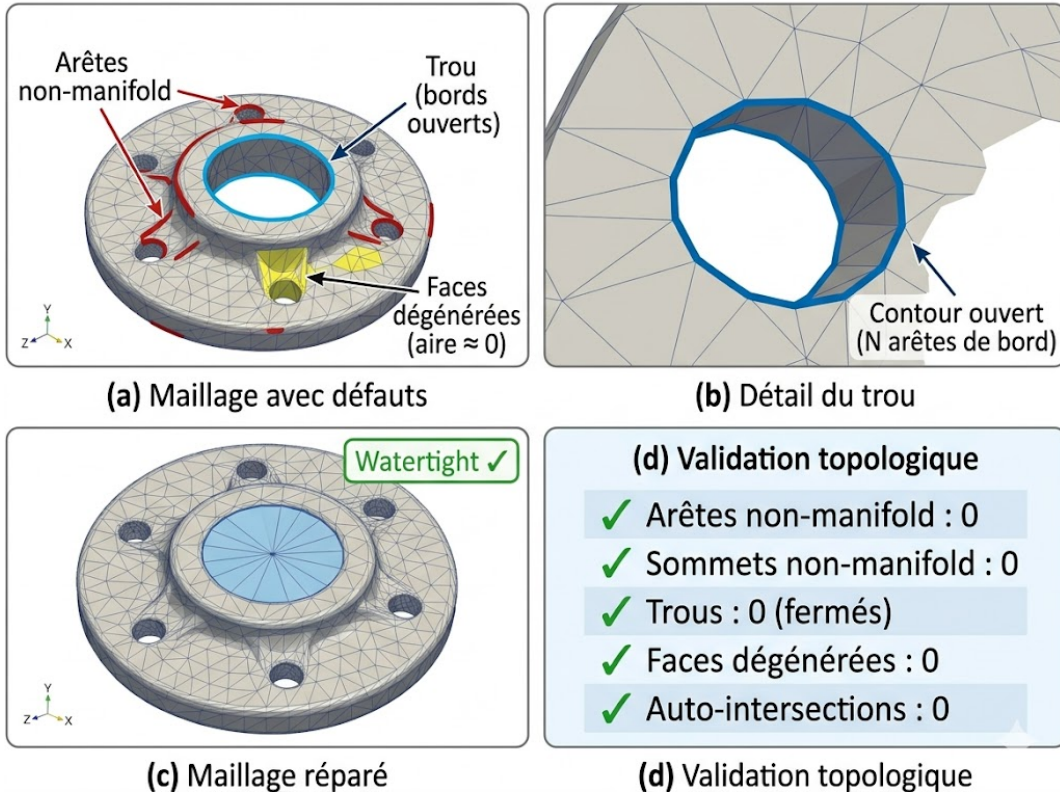
Fig. 4.3 : Validation topologique du maillage après réparation


Figura 100: Defectos topológicos y su reparación. Aristas no-*manifold*, huecos, caras degeneradas y su corrección; la lista de verificación corresponde a la Ec. 285.

11.2.6. Suavizado y decimación

El suavizado laplaciano uniforme desplaza cada vértice hacia el centroide de sus vecinos,

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \lambda \left(\frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \mathbf{v}_j - \mathbf{v}_i \right), \quad (286)$$

pero aplicado repetidamente contrae el volumen de forma sistemática, un sesgo inaceptable en metrología. El filtro de Taubin alterna un paso de encogimiento con $\lambda > 0$ y uno de expansión con $\mu < -\lambda$, de modo que la función de transferencia en el dominio de las frecuencias de la malla,

$$\mathcal{F}(k) = (1 - \lambda k)(1 - \mu k), \quad (287)$$

sea de tipo pasa-bajas con $\mathcal{F} \approx 1$ para $k \rightarrow 0$; las componentes de baja frecuencia —la forma— se conservan y solo se atenúa el ruido de alta frecuencia. En el Ejemplo 3, tres pasadas con $\lambda = 0.50$ y $\mu = -0.53$ alteran el volumen en $+0.002\%$.

La decimación reduce el número de caras conservando la forma. El método de referencia es el colapso de aristas guiado por métricas cuadráticas de error (QEM): a cada

vértice se le asocia la cuádrica

$$\mathbf{Q}_v = \sum_{p \in \text{planos}(v)} \mathbf{n}_p \mathbf{n}_p^\top, \quad \text{coste}(\mathbf{v}) = \tilde{\mathbf{v}}^\top (\mathbf{Q}_{v_1} + \mathbf{Q}_{v_2}) \tilde{\mathbf{v}}, \quad (288)$$

y se colapsan primero las aristas de coste mínimo, es decir, aquellas cuya eliminación menos desplaza la superficie respecto de los planos originales.

No toda decimación preserva la topología. El agrupamiento de vértices (*vertex clustering*) es rápido pero puede fusionar vértices de láminas opuestas y generar aristas no-*manifold*. En el Ejemplo 3, una decimación por agrupamiento al 54.5 % de las caras introduce 53 aristas no-*manifold* y altera la característica de Euler de $\chi = 0$ a $\chi = 27$. La decimación del entregable debe realizarse con QEM (MeshLab o Open3D) y *reauditar* con la Ec. (284) después de aplicarla.

11.3. Preparación geométrica para simulación

Una malla de escaneo no es un modelo CAD. Es una teselación densa, no paramétrica, sin superficies analíticas, sin historial de operaciones y con resolución determinada por el proceso óptico y no por la intención de diseño. Convertirla en un dominio de cálculo admisible exige decisiones explícitas.

11.3.1. Dos rutas de conversión

1. **Ruta directa (*as-built*).** La malla se convierte en un sólido por envoltura (*shrinkwrap*) y se malla con elementos tetraédricos. Conserva las desviaciones reales de fabricación —que es precisamente lo que se quiere estudiar en análisis de piezas desgastadas, deformadas o corroídas— pero arrastra también el ruido del escaneo, que se convierte en concentradores de esfuerzo ficticios.
2. **Ruta paramétrica (*scan-to-CAD*).** La malla se segmenta en regiones, se ajustan primitivas analíticas y parches NURBS, y se exporta a STEP. Produce un modelo limpio y editable, apto para estudios paramétricos, a costa de eliminar por construcción las desviaciones reales.

La elección no es estética: determina qué pregunta puede responder la simulación. Si la hipótesis de la tesis involucra el efecto de la geometría real sobre la respuesta mecánica, la ruta paramétrica invalida el experimento.

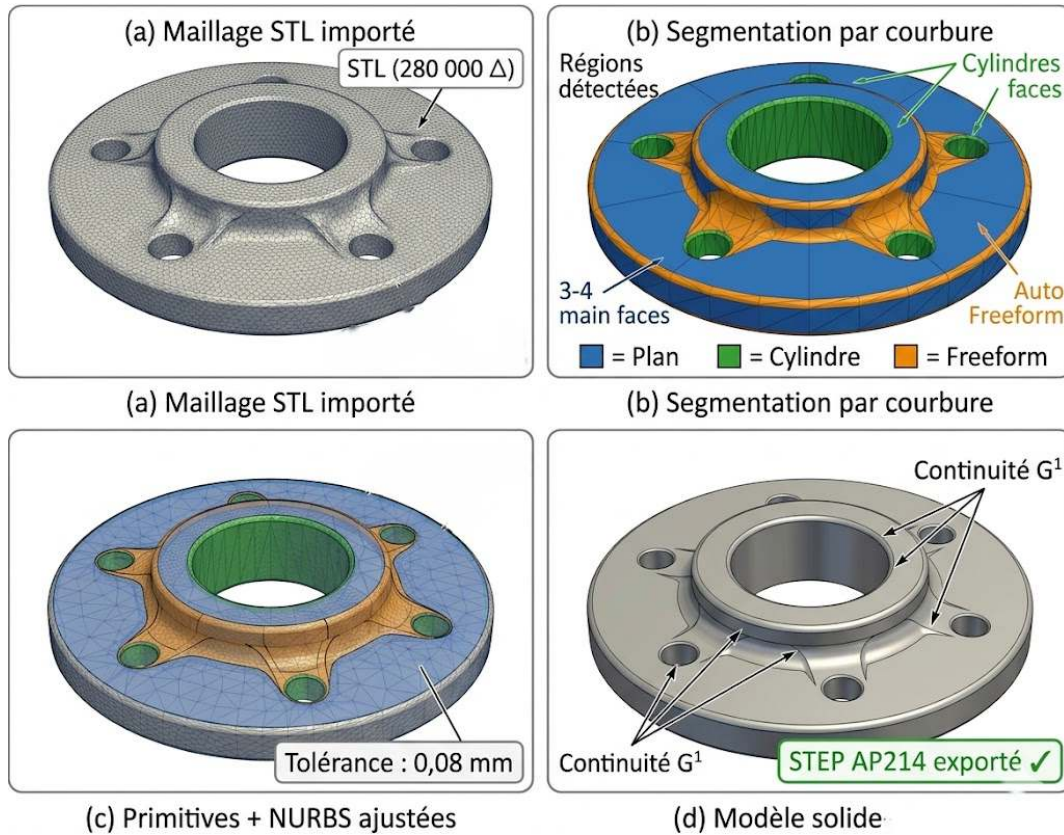


Figura 101: Flujo de conversión de malla de escaneo a geometría paramétrica en un entorno CAD de reparación. Importación de la teselación, segmentación en regiones, ajuste de parches y exportación en formato neutro.

11.3.2. Resolución de la teselación frente al tamaño de elemento

La teselación introduce un error de cuerda respecto de la superficie subyacente. Para una arista de longitud ℓ sobre una superficie de radio de curvatura R ,

$$\delta_c = R \left(1 - \cos \frac{\theta}{2} \right) \approx \frac{\ell^2}{8R}, \quad \theta = \frac{\ell}{R}, \quad (289)$$

que impone la condición de compatibilidad con el mallado de la Semana 8: el tamaño de elemento finito h_{FEM} no debe ser inferior a la longitud de arista de la teselación, porque el solucionador no puede extraer información geométrica que la malla de escaneo no contiene. Refinar el FEM por debajo de la resolución del escaneo produce convergencia de malla aparente sobre una geometría que es artefacto de la teselación.

Criterio de admisibilidad geométrica. Una malla de escaneo es admisible como dominio de cálculo si, y solo si: (i) satisface la Ec. (285); (ii) su género verificado por la Ec. (284) coincide con el de la pieza real; (iii) la desviación RMS frente a

las cotas medidas con instrumento calibrado es inferior a la tolerancia de ingeniería de la aplicación; (iv) la longitud media de arista es menor o igual que h_{FEM} previsto; y (v) ninguna región presenta relación de aspecto superior a la admitida por el generador de malla volumétrica.

11.3.3. Métricas de calidad heredadas

Los criterios de calidad de malla de la Semana 8 se aplican íntegros a la malla superficial de escaneo antes de generar el volumen. La Tabla 60 resume los umbrales operativos.

Tabla 60: Umbrales de aceptación de la malla superficial de escaneo previa a la generación del dominio volumétrico.

Métrica	Umbral	Consecuencia si se viola
Aristas de frontera	0	No define volumen; falla la conversión a sólido.
Aristas no- <i>manifold</i>	0	Normales indefinidas; falla el laminado y la envoltura.
Característica de Euler	Igual a la de la pieza	Piezas espurias o asas topológicas inexistentes.
Relación de aspecto de triángulo	< 10	Elementos volumétricos degenerados en la vecindad.
Longitud media de arista	$\leq h_{FEM}$	Geometría no resuelta; convergencia de malla ficticia.
Desviación RMS frente a patrón	$\leq$ tolerancia	El dominio no representa la pieza medida.

11.4. Protocolo operativo: escaneo con Meshroom 2023.3.0

11.4.1. Instalación y requisitos de cómputo

El curso utiliza **Meshroom 2023.3.0** (basado en AliceVision 3.2.0, publicado el 7 de diciembre de 2023), distribuido como paquete autocontenido que no requiere instalación: basta descomprimir el archivo y ejecutar el binario. Esta versión se descarga desde el espejo <https://sourceforge.net/projects/meshroom.mirror/files/v2023.3.0/> y ocupa aproximadamente 1.3 GB, frente a los cerca de 16 GB de las distribuciones más recientes. Para el alcance de esta semana —reconstrucción de una pieza única con nube densa y malla— la funcionalidad de la versión 2023.3.0 es suficiente y su huella de disco la hace viable en los equipos del laboratorio.

Requisito de GPU. Los binarios están compilados con CUDA 11 y requieren una tarjeta NVIDIA con capacidad de cómputo igual o superior a 3.5 para ejecutar los nodos de mapa de profundidad. Sin GPU NVIDIA compatible, únicamente está dis-

ponible el modo *Draft Meshing*, que genera la malla a partir de la nube dispersa del ajuste de haces: el resultado es topológicamente válido pero de resolución muy inferior y no satisface los criterios de aceptación de esta semana. No existe distribución oficial para macOS. Los participantes sin equipo compatible deben procesar en la estación de trabajo del laboratorio; la etapa de captura fotográfica es independiente del equipo de cómputo.

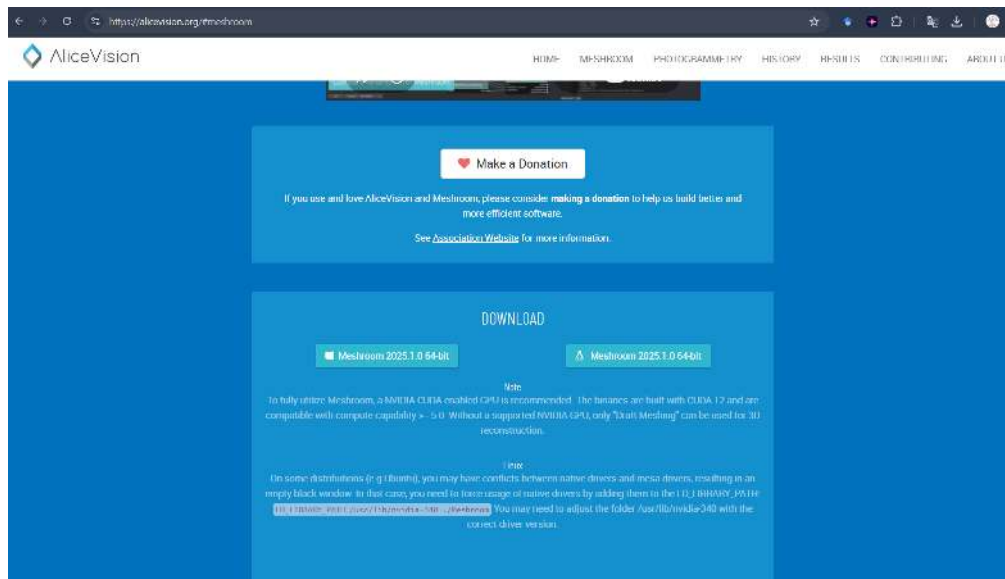


Figura 102: Marcador de captura: obtención del paquete de Meshroom 2025

11.4.2. Configuración de la cámara

La calidad de la reconstrucción está determinada, antes que por cualquier parámetro del programa, por la consistencia radiométrica y geométrica del conjunto de imágenes. La Tabla 61 fija la configuración.

Tabla 61: Configuración de cámara para la sesión fotogramétrica.

Parámetro	Valor	Justificación
Modo de exposición	Manual	Evita variaciones de brillo entre imágenes que degradan el emparejamiento de descriptores.
Apertura	$f/8$ a $f/11$	Máxima profundidad de campo sin difracción apreciable.
ISO	Mínimo nativo (100–200)	El ruido del sensor eleva σ_d en la Ec. (268).
Obturación	$\leq 1/60$ s con trípode	Evita desenfoque de movimiento.
Balance de blancos	Fijo	Consistencia cromática entre imágenes.
Enfoque	Manual, fijo en toda la sesión	El reenfoque cambia la focal efectiva y viola el supuesto de intrínsecos únicos.
Formato	JPEG de máxima calidad o RAW	Compatibles con el nodo CameraInit.
Flash	Desactivado	Genera reflejos dependientes de la vista que rompen la fotoconsistencia.

11.4.3. Planificación de la trayectoria

La adquisición se organiza en anillos orbitales concéntricos alrededor de la pieza, que permanece inmóvil. Para una separación angular $\Delta\alpha$ y N_{anillos} alturas de captura,

$$N_{\text{im}} = \left\lceil \frac{360^\circ}{\Delta\alpha} \right\rceil \cdot N_{\text{anillos}}, \quad B = 2Z \sin \frac{\Delta\alpha}{2}, \quad o = 1 - \frac{B}{2Z \tan(\text{FOV}/2)}, \quad (290)$$

donde o es la fracción de solapamiento entre estaciones consecutivas y FOV el campo de visión horizontal. El Ejemplo 1 demuestra que la restricción activa del diseño no es la geometría de triangulación sino el solapamiento mínimo exigido por el emparejamiento.

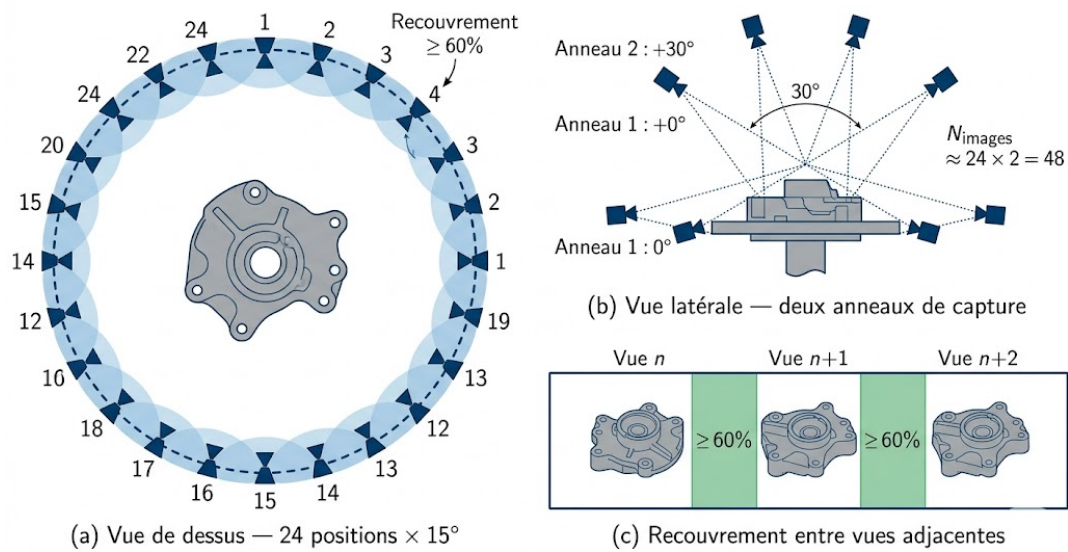


Figura 103: Planificación de la trayectoria de captura. **(a)** Vista superior con las estaciones distribuidas cada 15° y las zonas de solapamiento. **(b)** Vista lateral con dos anillos de captura. **(c)** Tres imágenes consecutivas con la fracción de solapamiento resaltada, según la Ec. 290.

11.4.4. Grafo de nodos y ejecución

El pipeline por defecto de Meshroom 2023.3.0 encadena:

```
CameraInit → FeatureExtraction → ImageMatching → FeatureMatching →
StructureFromMotion → PrepareDenseScene → DepthMap → DepthMapFilter →
Meshing → MeshFiltering → Texturing
```

La Tabla 62 recoge los únicos parámetros que conviene modificar respecto de los valores por defecto, con el criterio asociado.

Tabla 62: Parámetros de nodo relevantes en Meshroom 2023.3.0.

Nodo	Parámetro	Valor	Criterio
CameraInit	sensorWidth	Del fabricante	Obligatorio si la cámara no está en la base de datos; sin él la escala relativa de la focal es errónea.
FeatureExtraction	describerTypes	dspsift	Predeterminado en esta versión; más robusto ante cambio de escala que SIFT clásico.
FeatureExtraction	describerPreset	high	Aumenta el número de puntos característicos en superficies de textura pobre.
ImageMatching	method	VocabularyTree	Adecuado para conjuntos de decenas de imágenes; Exhaustive solo por debajo de 50 imágenes.
StructureFromMotion	minInputTrackLength	3	Descarta puntos vistos en menos de tres cámaras.
DepthMap	downscale	2	Compromiso entre resolución y memoria de GPU; 1 solo con ≥ 8 GB de VRAM.
Meshing	maxPoints	5×10^6	Cota superior de vértices; reducir si el nodo agota la memoria.
MeshFiltering	keepLargestMeshOnly	True	Elimina piezas desprendidas; equivale al filtrado por componentes conexas de la Ec. (284).

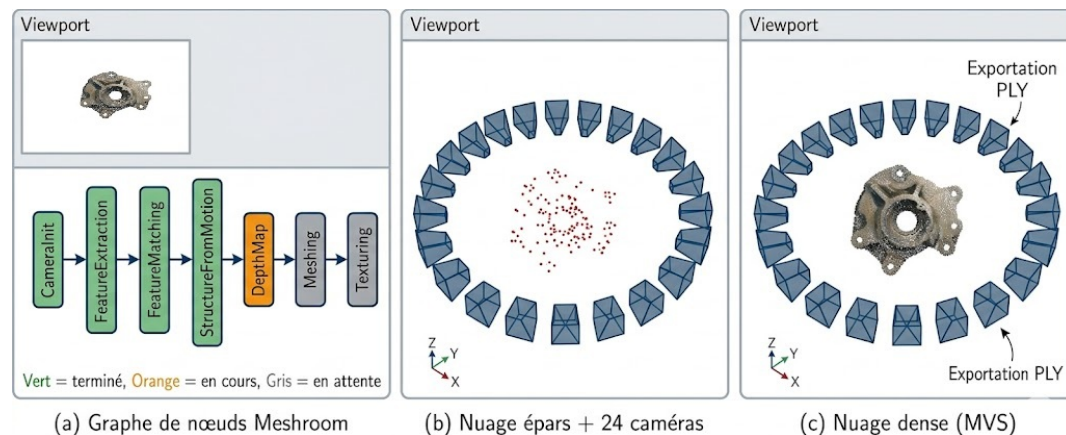


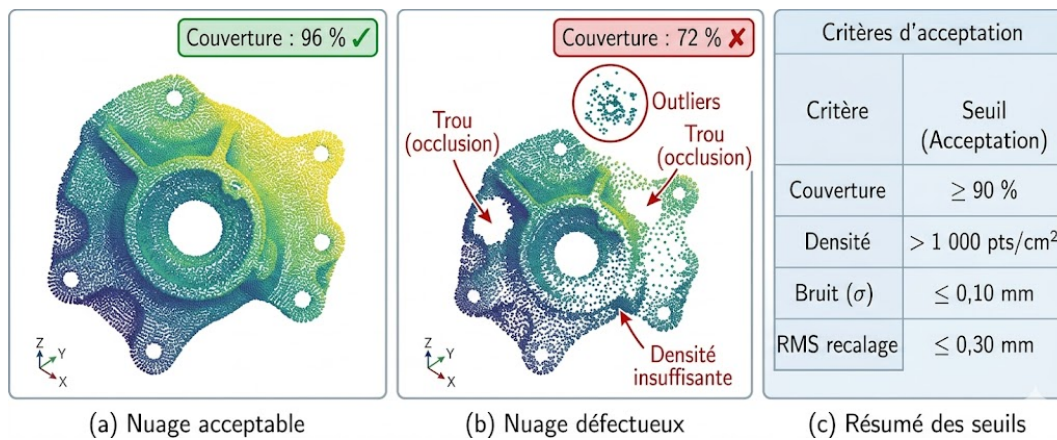
Figura 104: Ejecución del pipeline. **(a)** Grafo de nodos con estados de cómputo. **(b)** Nube dispersa y poses estimadas formando el anillo de captura. **(c)** Nube densa coloreada por textura. La verificación intermedia obligatoria es la comparación entre el número de cámaras registradas y el número de imágenes importadas.

11.4.5. Criterios de aceptación de la nube densa

Antes de proceder al mallado, la nube densa debe satisfacer los criterios cuantitativos de la Tabla 63.

Tabla 63: Criterios de aceptación de la nube densa antes del mallado.

Criterio	Umbral	Método de verificación
Vistas registradas	$\geq 95\%$ de las importadas	Registro del nodo StructureFromMotion.
Coherencia de poses	Anillo sin cámaras aberrantes	Inspección del visor 3D; ausencia de poses fuera de la trayectoria planificada.
Cobertura superficial	$\geq 90\%$	Rotación completa de la nube; ningún hueco mayor al 5 % del área proyectada.
Densidad superficial	$> 1000 \text{ pts/cm}^2$	Herramienta de densidad superficial en visor de nubes.
Ruido	$\sigma_n \leq 0.10 \text{ mm}$	Desviación estándar de los residuales del ajuste de un plano a una región nominalmente plana.
Escala aplicada	Declarada	Factor s de la Ec. (272) y su incertidumbre por la Ec. (273).


Figura 105: Contraste entre una nube aceptable y una defectuosa. (a) Cobertura completa y densidad uniforme. (b) Huecos extensos, puntos flotantes y densidad irregular. (c) Resumen de los umbrales de la Tabla 63.

11.5. Ejemplo 1. Presupuesto óptico de la sesión de captura

Planteamiento. Antes de tomar la primera fotografía debe demostrarse que la configuración elegida puede alcanzar la resolución exigida. Se adopta como configuración nominal un sensor APS-C de 22.3 mm de ancho con 6000 píxeles en esa dimensión (paso de píxel $p = 3.717 \mu\text{m}$), objetivo de $f = 35 \text{ mm}$, distancia de trabajo $Z = 600 \text{ mm}$ y ruido de correspondencia a ángulo nulo $\sigma_{d0} = 0.30 \text{ px}$. El campo de visión horizontal resultante es de 35.3° y la focal en píxeles $f_{px} = 9417 \text{ px}$.

Supuesto declarado. La degradación del emparejamiento con la separación angular se modela de forma fenomenológica como

$$\sigma_d(\Delta\alpha) = \sigma_{d0} \left[1 + \left(\frac{\Delta\alpha}{\Delta\alpha_c} \right)^2 \right], \quad \Delta\alpha_c = 20^\circ, \quad (291)$$

con $\Delta\alpha_c$ calibrable por el usuario. El modelo captura la pérdida de precisión por distorsión perspectiva del parche; su forma exacta no altera la conclusión cualitativa, pero sí el valor numérico del óptimo, por lo que debe declararse y no ocultarse.

Resultados verificados. La ejecución del Código 52 produce los valores de la Tabla 64.

Tabla 64: Resultados del presupuesto óptico para la configuración nominal (sensor APS-C, $f = 35$ mm, $Z = 600$ mm, $\sigma_{d0} = 0.30$ px).

Cantidad	Valor	Referencia
Resolución lateral GSD	$63.7 \mu\text{m/px}$	Ec. (271)
Separación angular de mínimo error $\Delta\alpha^*$	19.8°	Ecs. (268), (291)
Incertidumbre de profundidad en $\Delta\alpha^*$	$110.1 \mu\text{m}$	Ec. (268)
Relación σ_Z/GSD	1.73	—
Solapamiento en $\Delta\alpha^*$	0.46	Ec. (290)
$\Delta\alpha$ con solapamiento 0.60	14.6°	Ec. (290)
σ_Z y N_{im} en ese punto	$115.2 \mu\text{m}$; 50 imágenes	—
$\Delta\alpha$ con solapamiento 0.80	7.3°	Ec. (290)
σ_Z y N_{im} en ese punto	$170.0 \mu\text{m}$; 100 imágenes	—

Discusión. El resultado no trivial es que el óptimo metrológico y el óptimo operativo no coinciden, y que la restricción activa del diseño es el solapamiento. Operar en $\Delta\alpha = 14.6^\circ$ —el máximo compatible con un solapamiento de 0.60— cuesta apenas un 4.7 % de incertidumbre de profundidad respecto del mínimo absoluto en 19.8° , pero garantiza la convergencia del ajuste de haces. En cambio, exigir un solapamiento de 0.80 duplica el número de imágenes y degrada σ_Z en un 54.5 %: un solapamiento excesivo es contraproducente, porque las bases cortas empeoran la condición de la triangulación. Esta observación justifica cuantitativamente la regla práctica de 10° a 15° que suele enunciarse sin demostración.

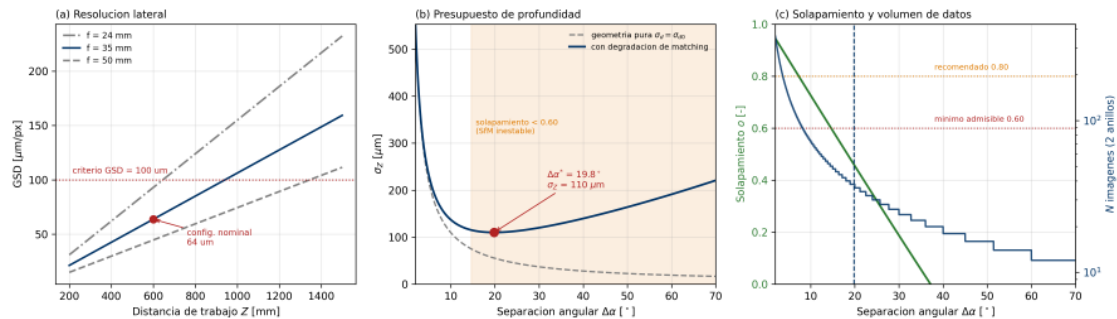


Figura 106: Presupuesto óptico de la sesión. (a) Resolución lateral frente a distancia de trabajo para tres focales, con el criterio de $100 \mu\text{m}$. (b) Incertidumbre de profundidad, Ec. 268, con y sin el término de degradación de la Ec. 291; la región sombreada corresponde a solapamiento insuficiente. (c) Solapamiento y número de imágenes frente a la separación angular, Ec. 290.

11.6. Ejemplo 2. Banco sintético de *Structure from Motion*

Planteamiento. Sobre una pieza sintética con verdad de terreno exacta —un buje cilíndrico de 50 mm de diámetro y 80 mm de altura sobre una brida de 110 mm de diámetro— se proyectan 4000 puntos sobre N cámaras distribuidas en dos anillos orbitales de radio 600 mm, con la matriz intrínseca del Ejemplo 1. Se contamina la detección con ruido gaussiano de 0.30 px por coordenada, se triangula por la Ec. (267) y se compara con la verdad de terreno tras alineamiento de similitud de Umeyama.

Resultados verificados. La Tabla 65 recoge la ejecución del Código 53.

Tabla 65: Error de triangulación multivista frente al número de estaciones.

N estaciones	Cobertura	RMS 3D [μm]	Residual de reproyección [px/coord]
6	75.8 %	20.4	0.242
8	97.6 %	18.7	0.255
12	100 %	15.4	0.270
16	100 %	13.5	0.279
24	100 %	10.9	0.286
36	100 %	9.0	0.290
48	100 %	7.7	0.292

El ajuste potencial da un exponente empírico $q = 0.477$ en $\text{RMS} \propto N^{-q}$, consistente con el valor teórico $1/2$ que corresponde al promediado de observaciones independientes. El residual de reproyección converge a 0.29 px/coordenada, ligeramente por debajo del ruido inyectado de 0.30 px, como corresponde a un ajuste sobredeterminado: esta comparación es la verificación interna de que el estimador está correctamente implementado.

Alcance del resultado. El banco supone poses exactamente conocidas, ausencia de residual de distorsión y ausencia de error sistemático. Los 10.9 μm obtenidos con 24 estaciones son, por lo tanto, una *cota inferior* del error alcanzable, y son mucho menores que los 115 μm del Ejemplo 1 porque este último evalúa un par adyacente aislado, mientras que la red completa aprovecha bases de hasta 180° y promedia sobre N vistas. Ningún escaneo real alcanzará la cota del banco; su valor es fijar el orden de magnitud y demostrar el escalamiento.

Herencia de la escala. La simulación de Monte Carlo con 4000 realizaciones reproduce la Ec. (273) con exactitud. Para la incertidumbre por punto $u_p = 10.9 \mu\text{m}$ del punto de diseño, una barra de escala de 25 mm induce 352 ppm de error de escala, que sobre una cota de 110 mm equivale a 38.7 μm ; una barra de 150 mm reduce la contribución a 61 ppm, esto es, 6.7 μm sobre la misma cota. La longitud de la referencia métrica es, en consecuencia, una variable de diseño de primer orden y no un detalle logístico.

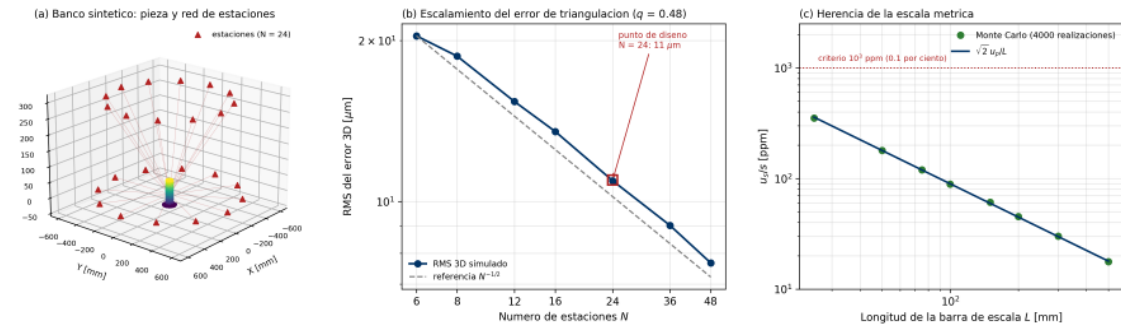


Figura 107: Banco sintético de SfM. **(a)** Pieza de referencia y red de 24 estaciones en dos anillos orbitales. **(b)** Escalamiento del error de triangulación con el número de estaciones, contrastado con la referencia $N^{-1/2}$. **(c)** Herencia de la escala métrica: contraste entre el resultado de Monte Carlo y la predicción analítica de la Ec. 273.

11.7. Ejemplo 3. De la nube de puntos al STL auditado

Planteamiento. Se ejecuta la cadena completa de posprocesamiento sobre una nube sintética con verdad de terreno analítica, definida por la función de distancia con signo de la misma pieza del Ejemplo 2 con un barreno pasante de 20 mm de diámetro. Se muestrean 120 000 puntos sobre la superficie, se les añade ruido normal de $\sigma_n = 0.10$ mm y un 2 % de valores atípicos distribuidos en el volumen circundante. Disponer de la verdad de terreno analítica permite auditar cada etapa con una métrica objetiva, algo imposible con datos reales.

Resultados verificados. La Tabla 66 sintetiza la ejecución del Código 54.

Tabla 66: Trazabilidad del pipeline de posprocesamiento sobre la nube sintética con verdad de terreno analítica.

Etapa	Parámetros	Resultado verificado
Nube cruda	122 400 pts; $\sigma_n = 0.10$ mm; 2 % atípicos	Referencia de entrada
SOR robusto, Ec. (276)	$k = 20$; $n_\sigma = 3$; dos pasadas	Exhaustividad 95.8 %; precisión 80.6 %
Submuestreo por vóxeles	$a_v = 0.60$ mm	119 548 $\rightarrow$ 78 156 pts (factor 0.65)
Normales por PCA, Ec. (277)	$k = 25$; orientación por MST	Coherencia de orientación 100 %
Campo implícito, Ec. (283)	Rejilla de 0.70 mm; $r_{\max} = 2.20$ mm	4.09 M nodos; 12.6 % evaluados
<i>Marching Cubes</i>	Isosuperficie $f = 0$	98 676 V; 197 352 F; $\chi = 0$; género 1
Auditoría, Ec. (285)	—	0 aristas de frontera; 0 no- <i>manifold</i> ; estanca
Taubin, Ec. (287)	$\lambda = 0.50$; $\mu = -0.53$; 3 pasadas	Cambio de volumen +0.002 %
Decimación por agrupamiento	$a = 0.90$ mm	54.5 % de las caras; 53 aristas no- <i>manifold</i>
Exportación STL binario	Malla suavizada	197 352 caras; 9.87 MB

Verificación dimensional. Contrastada contra la función de distancia analítica, la malla entregable presenta una desviación RMS de $59.8 \mu\text{m}$ con sesgo medio de $+8.3 \mu\text{m}$, percentil 95 de $121 \mu\text{m}$ y desviación máxima de 1.00 mm localizada en las aristas vivas. El volumen calculado por el teorema de la divergencia es de $242\,412 \text{ mm}^3$ frente a los $242\,217 \text{ mm}^3$ analíticos, un error de $+0.081 \%$. Frente al ruido de entrada de $100 \mu\text{m}$, la cadena de procesamiento reduce la dispersión en un factor 1.67: el promediado por vóxeles y la regresión implícita de la Ec. (283) atenúan el ruido, pero no lo eliminan.

El género verificado $g = 1$ corresponde exactamente al barreno pasante de la pieza de referencia. Este control es más informativo que la inspección visual: detecta cavidades espurias, láminas duplicadas y piezas desprendidas que un render no revela.

Lectura del error máximo. La desviación máxima de 1.00 mm concentrada en las aristas es el comportamiento esperado de cualquier método implícito: la superficie reconstruida redondea las discontinuidades de la normal con un radio del orden de la escala de suavizado h de la Ec. (283). Si la cota crítica de la aplicación pasa por una arista viva, la fotogrametría no es el instrumento adecuado y debe medirse con palpador o con proyección de franjas.

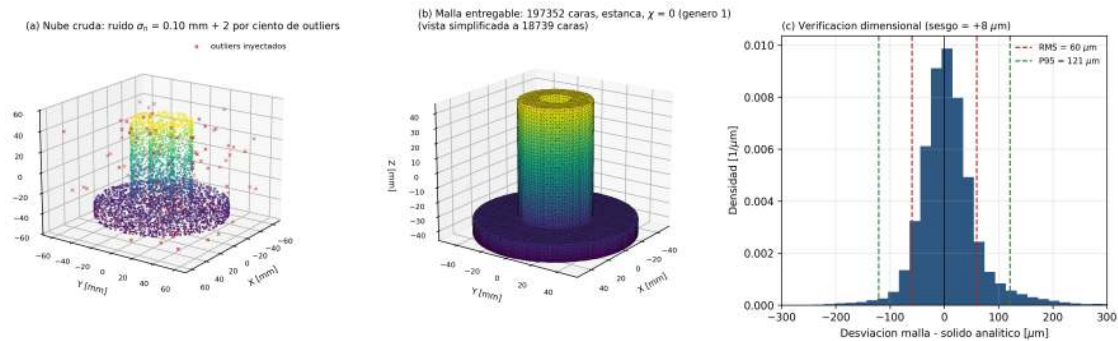


Figura 108: Cadena de posprocesamiento verificada. (a) Nube cruda con ruido normal y valores atípicos inyectados, Ec. 274. (b) Malla entregable estanca de género 1, con $\chi = 0$ según la Ec. 284. (c) Histograma de la desviación frente al sólido analítico, con RMS y percentil 95 señalados.

11.8. Aplicaciones en ingeniería y enlace con los proyectos del grupo

Enlace con las líneas de investigación del grupo. La reconstrucción óptica no es un fin en sí misma: es el mecanismo por el cual la geometría real entra en el modelo computacional.

- **Instrumentación óptica y metrología con fibra.** El escaneo documenta la geometría real del portamuestras y del posicionador; las desviaciones frente al diseño explican corrimientos de alineación que de otro modo se atribuyen erróneamente al sensor.
- **Bancos de prueba de bombas centrífugas.** La geometría *as-built* del impulsor —con el desgaste de borde de salida efectivamente presente— alimenta el modelo hidráulico. La diferencia entre el rendimiento simulado con el CAD nominal y con la geometría escaneada es un resultado publicable por sí mismo.
- **Cables OPGW y accesorios.** La reconstrucción de herrajes y del daño superficial por arco eléctrico proporciona el dominio para el análisis electrotérmico local, imposible de parametrizar a partir de un plano.
- **Monitoreo aéreo con vehículos no tripulados.** El mismo pipeline SfM, aplicado a imágenes tomadas desde plataforma aérea, produce modelos digitales de superficie. Las Ecs. (271) y (268) rigen igual, con Z igual a la altura de vuelo.
- **Optimización estructural de losas.** El escaneo de especímenes ensayados registra el patrón de fisuración y la deformada residual, que constituyen el dato experimental D del protocolo de validación de la Semana 11.
- **Sistemas de recuperación aumentada.** Las mallas auditadas, con sus metadatos de incertidumbre, son activos indexables: la trazabilidad exigida aquí es la misma que hace confiable un repositorio técnico consultable por un sistema

de recuperación.

11.9. Tabla de síntesis y control

Tabla 67: Síntesis del protocolo de reconstrucción 3D de la Semana 12.

Etapa	Cantidad de control	Cómo se mejora	Ecuación
Diseño de captura	GSD y σ_Z	Reducir Z ; aumentar f ; optimizar $\Delta\alpha$	Ecs. (271), (268)
Trayectoria	Solapamiento o y N_{im}	Ajustar $\Delta\alpha$ dentro de la restricción $o \geq 0.60$	Ec. (290)
Ajuste de haces	Residual de reproyección y vistas registradas	Más textura; mejor solapamiento; pérdida robusta	Ec. (269)
Escala métrica	u_s/s	Barra de referencia más larga y mejor calibrada	Ecs. (272), (273)
Limpieza de la nube	Exhaustividad y precisión del filtro	Estimadores robustos; dos pasadas	Ec. (276)
Normales	Coherencia de orientación	Propagación por MST; mayor k	Ec. (277)
Superficie	Desviación RMS frente a patrón	Ajustar λ , depth o h ; recortar la cáscara falsa	Ecs. (282), (283)
Topología	χ , frontera, no- <i>manifold</i>	Componente conexa mayor; reparación dirigida	Ecs. (284), (285)
Acondicionamiento	Cambio de volumen	Taubin en lugar de laplaciano; QEM en lugar de agrupamiento	Ecs. (287), (288)
Aptitud para FEM	Longitud de arista y aspecto	Adecuar h_{FEM} a la resolución real	Ec. (289)

Tabla 68: Resultados de control de los ejemplos verificados de la Semana 12.

Cantidad de control	Valor	Ejemplo
Resolución lateral GSD	63.7 $\mu\text{m}/\text{px}$	1
Separación angular óptima $\Delta\alpha^*$	19.8°	1
Punto de operación por solapamiento	14.6°; 50 imágenes	1
Penalización de σ_Z en ese punto	4.7 %	1
Exponente de escalamiento q	0.477	2
RMS 3D con 24 estaciones	10.9 μm	2
Residual de reproyección	0.286 px/coord	2
Escala con barra de 150 mm	61 ppm	2
Exhaustividad del SOR robusto	95.8 %	3
Característica de Euler de la malla	0 (género 1)	3
Desviación RMS frente al patrón	59.8 μm	3
Percentil 95 de la desviación	121 μm	3
Error volumétrico	+0.081 %	3
Cambio de volumen por suavizado	+0.002 %	3

11.10. Uso guiado de inteligencia artificial generativa

Los modelos de lenguaje son útiles para convertir un requisito de tolerancia en una especificación de captura, siempre que el doctorando audite las relaciones propuestas contra las ecuaciones de esta semana. El siguiente prompt condiciona a la herramienta con la estructura formal del presupuesto óptico.

Prompt de referencia: diseño inverso de una sesión fotogramétrica

Actúa como especialista en metrología óptica dimensional. Debo reconstruir por fotogrametría multivista [pieza y dimensión característica] con una tolerancia de verificación de [X] mm sobre la cota crítica [descripción]. Dispongo de [cámara, sensor, resolución, objetivo] y de un espacio de trabajo que limita la distancia mínima a [Z] mm. Determina: (1) la distancia de trabajo y la focal que hacen $GSD \leq [X]/5$, justificando con $GSD = pZ/f$; (2) la separación angular que minimiza $\sigma_Z = Z^2 \sigma_d / (f_{px} B)$ con $B = 2Z \sin(\Delta\alpha/2)$, sujeta a solapamiento ≥ 0.60 , y el número de imágenes resultante; (3) la longitud mínima de la barra de escala para que su contribución $u_s/s = \sqrt{2}u_p/(\sqrt{3}L)$ no exceda el 20 por ciento del presupuesto total; (4) el presupuesto combinado de incertidumbre y el veredicto sobre si la configuración es viable o si debe recurrirse a otra técnica. Declara explícitamente todos los supuestos y entrega el resultado como tabla de LaTeX.

La salida debe auditarse en cuatro puntos: que la dependencia de σ_Z con Z sea cuadrática y no lineal; que el óptimo angular respete la restricción de solapamiento y no solo la geometría; que la contribución de escala se sume en cuadratura y no aritméticamente; y que el modelo no prometa exactitud por debajo del GSD, lo que es físicamente imposible.

11.11. Actividad de refuerzo en clase (45 min)

11.11.1. Actividad 1. Diagnóstico de reconstrucciones fallidas

Se proyectan cuatro conjuntos de resultados de sesiones fotogramétricas defectuosas: (i) poses de cámara dispersas sin formar anillo, con menos del 60 % de vistas registradas; (ii) nube densa con una superficie fantasma plana donde el objeto era especular; (iii) malla con la base cerrada por una cáscara convexa que no corresponde a la pieza; (iv) modelo geoméricamente correcto pero con todas las cotas escaladas por un factor constante.

Consigna. En duplas, para cada caso: identificar la etapa del pipeline donde se originó el fallo, nombrar la ecuación o el criterio de esta semana que fue violado, y proponer la acción correctiva mínima —distinguiendo si exige repetir la captura o basta reprocesar.

Rúbrica.

- 3 Diagnostica los cuatro casos con la etapa correcta, cita la ecuación violada y distingue correctamente entre fallo de adquisición (irrecuperable) y fallo de procesamiento (recuperable).
- 2 Diagnostica correctamente al menos tres casos, pero no distingue entre fallo recuperable e irreparable.
- 1 Identifica síntomas sin asociarlos a una etapa ni a un criterio.
- 0 Atribuye los fallos a limitaciones del programa sin analizar la geometría de adquisición.

11.11.2. Actividad 2. Cálculo del presupuesto sobre el equipo propio

Cada participante ejecuta el Código 52 sustituyendo las constantes por las de la cámara que efectivamente usará en la práctica (ancho de sensor, número de píxeles, focal) y por la distancia de trabajo que su espacio permite. Reporta GSD, $\Delta\alpha$ de operación, N_{im} y σ_Z , y declara si la configuración alcanza la tolerancia de su pieza. Si no la alcanza, propone la modificación mínima que sí lo consigue.

11.12. Práctica oficial. Escaneo 3D de un objeto y entrega del modelo STL

Objetivo. Reconstruir por fotogrametría multivista un objeto físico elegido por el participante y entregar la nube de puntos y la malla triangular resultantes, con su presupuesto de incertidumbre y su auditoría topológica.

11.12.1. Selección del objeto

El objeto debe cumplir simultáneamente:

- Dimensión característica entre 80 y 300 mm.
- Superficie mate o susceptible de matearse con revelador volátil; se excluyen piezas cromadas, pulidas, transparentes o de color negro uniforme.
- Al menos una cota medible con calibrador o micrómetro calibrado, preferentemente tres.
- Se recomienda —sin ser obligatorio— que el objeto pertenezca al dominio del proyecto doctoral del participante, de modo que el entregable sea reutilizable en la tesis.

11.12.2. Procedimiento

1. **Presupuesto previo.** Ejecutar el Código 52 con los parámetros de la cámara y el espacio disponibles. Registrar GSD, $\Delta\alpha$, N_{im} y σ_Z *antes* de fotografiar.

2. **Preparación.** Fondo uniforme, iluminación difusa y constante, objeto inmóvil. Colocar una referencia de longitud calibrada visible en al menos el 80 % de las imágenes, con la longitud dictada por la Ec. (273).
3. **Captura.** Configuración manual según la Tabla 61. Ejecutar la trayectoria planificada con un mínimo de dos anillos y $N_{\text{im}} \geq 40$ imágenes.
4. **Reconstrucción.** Ejecutar el pipeline completo en Meshroom 2023.3.0 con los parámetros de la Tabla 62. Verificar el registro del nodo StructureFromMotion antes de continuar.
5. **Escalamiento.** Aplicar el factor s de la Ec. (272) y calcular u_s/s por la Ec. (273).
6. **Postprocesamiento.** Filtrado de valores atípicos, submuestreo, estimación de normales, reconstrucción de superficie, recorte de la cáscara falsa, suavizado de Taubin y decimación por QEM.
7. **Auditoría.** Verificar la Ec. (285) y calcular χ por la Ec. (284), comprobando que el género coincide con el de la pieza real.
8. **Verificación dimensional.** Medir con calibrador las cotas seleccionadas y compararlas con las obtenidas del modelo. Reportar el error de comparación E y contrastarlo con la incertidumbre combinada, siguiendo el criterio de la Semana 11.

11.12.3. Entregables

Tabla 69: Entregables de la práctica oficial de la Semana 12.

Id	Producto	Formato	Requisito
E1	Conjunto de imágenes de la sesión	Carpeta JPEG o RAW	$N_{\text{im}} \geq 40$; EXIF conservado
E2	Nube de puntos densa escalada	.ply binario con color	Escala en mm aplicada; criterios de la Tabla 63
E3	Malla triangular	.stl binario	Estanca; unidades en mm; género correcto
E4	Reporte técnico	PDF de 4 a 6 páginas	Contenido especificado abajo
E5	Cuaderno de cálculo	.ipynb o .py	Presupuesto óptico y auditoría reproducibles

El reporte E4 debe contener, en este orden: presupuesto óptico previo con los valores de GSD y σ_Z ; descripción de la trayectoria efectivamente ejecutada con N_{im} y solapamiento estimado; registro del ajuste de haces con el número de vistas resueltas; factor de escala y su incertidumbre; tabla de auditoría topológica; tabla de verificación dimensional con al menos tres cotas medidas frente a modelo; y una declaración final de aptitud o no aptitud del modelo como dominio de cálculo, argumentada con el criterio de admisibilidad geométrica de esta semana.

11.12.4. Criterios de aceptación y rúbrica

Tabla 70: Criterios cuantitativos de aceptación del entregable E3.

Criterio	Umbral	Evidencia
Vistas registradas	$\geq 95\%$	Captura del registro de StructureFromMotion
Aristas de frontera	0	Salida de la auditoría
Aristas no- <i>manifold</i>	0	Salida de la auditoría
Característica de Euler	Igual a la de la pieza	Cálculo por la Ec. (284)
Escala declarada	u_s/s reportada	Ecs. (272), (273)
Desviación en cotas medidas	$ E \leq k u_{val}$	Tabla de verificación con k declarado
Densidad de malla	Arista media $\leq h_{FEM}$ previsto	Estadística de longitud de arista

Rúbrica.

- 3 Entrega completa (E1–E5); la malla satisface todos los criterios de la Tabla 70; el presupuesto previo se contrasta con el resultado obtenido y se explican las diferencias; el veredicto de aptitud está argumentado con las ecuaciones del tema.
- 2 Malla estanca y escalada con auditoría correcta, pero sin contraste entre presupuesto previo y resultado, o con la verificación dimensional reducida a una sola cota.
- 1 Se entrega el STL pero no es estanco, o carece de escala métrica declarada, o la auditoría topológica no se realizó.
- 0 Se entrega un modelo sin unidades, sin incertidumbre y sin auditoría, o se reporta la reconstrucción como exitosa apoyándose únicamente en su apariencia visual.

Causa más frecuente de rechazo. Un modelo visualmente convincente sin factor de escala aplicado no es una medición: por la Ec. (270), su geometría es correcta salvo un factor desconocido. Un STL sin unidades declaradas se califica con cero independientemente de su calidad geométrica.

11.13. Ejercicios de investigación para tarea

11.13.1. Ejercicio 1 (obligatorio) — Presupuesto de reconstrucción del propio proyecto

Para la geometría física relevante en su investigación doctoral: (i) identifique la cota crítica y su tolerancia de ingeniería; (ii) determine, con las Ecs. (271) y (268), la configuración de captura mínima que alcanza esa tolerancia con margen de un factor 5 entre GSD y tolerancia; (iii) dimensione la referencia métrica con la Ec. (273) para que su

contribución no exceda el 20 % del presupuesto; y (iv) redacte media cuartilla, lista para el capítulo metodológico de la tesis, declarando la configuración, el presupuesto de incertidumbre y las condiciones bajo las cuales la fotogrametría *no* sería el instrumento adecuado para esa medición.

Rúbrica.

- 3 Presupuesto completo y coherente; se identifican correctamente los límites de aplicabilidad de la técnica para la geometría concreta.
- 2 Cálculos correctos pero sin dimensionar la referencia métrica o sin declarar el margen entre GSD y tolerancia.
- 1 Se reproducen las fórmulas con los valores del curso en lugar de los del proyecto propio.
- 0 Se afirma una exactitud alcanzable sin cálculo que la sustente.

11.13.2. Ejercicio 2 (obligatorio) — Sensibilidad del posprocesamiento

Con el Código 54, realice un barrido sobre dos parámetros: la arista del vóxel $a_v \in \{0.3, 0.6, 1.2\}$ mm y el paso de la rejilla implícita $\in \{0.5, 0.7, 1.0\}$ mm. Para cada una de las nueve combinaciones reporte el número de caras, la característica de Euler, la desviación RMS frente a la verdad de terreno y el tiempo de cómputo. Construya la superficie de compromiso y determine el punto de operación que minimiza la desviación sujeto a un tiempo de cómputo admisible. Discuta si el comportamiento observado es consistente con la interpretación del submuestreo por vóxeles como filtro pasa-bajas.

Rúbrica.

- 3 Barrido completo; se identifica el punto de operación con criterio explícito y se explica físicamente la forma de la superficie de compromiso.
- 2 Barrido completo, pero la elección del punto de operación no está justificada cuantitativamente.
- 1 Barrido parcial o sin reportar la característica de Euler.
- 0 Se ejecuta el código sin modificar parámetros.

11.13.3. Ejercicio 3 (optativo) — Contraste entre técnicas

Reconstruya el mismo objeto por fotogrametría y por un segundo método disponible—escáner de luz estructurada, sensor de profundidad o palpación por coordenadas— y compare ambas mallas mediante un mapa de distancias punto a superficie. Reporte el histograma de desviaciones, su media y su dispersión, y discuta cuál de las dos técnicas debería tomarse como referencia y con qué argumento metrológico. Este ejercicio conecta directamente con la métrica de comparación de la Semana 11.

11.14. Cierre conceptual

La fotogrametría produce, con un equipo modesto, un objeto que se parece mucho a la pieza real. Esa facilidad es precisamente su riesgo: el parecido visual no implica exactitud dimensional, y la Ec. (270) garantiza que sin referencia métrica externa el modelo ni siquiera tiene unidades. La diferencia entre un modelo decorativo y un modelo científico no reside en el programa empleado ni en el número de triángulos, sino en tres declaraciones que deben acompañarlo: la resolución que la geometría de adquisición permitía alcanzar, el factor de escala con su incertidumbre, y la auditoría topológica que demuestra que la malla define un volumen. Un modelo que carece de esas tres declaraciones no puede sostener una afirmación de tesis, por convincente que resulte en pantalla. La contribución doctoral no consiste en obtener la nube, sino en poder afirmar —y demostrar— cuánto se aparta esa nube de la pieza que pretende representar.

11.15. Referencias bibliográficas

1. **Hartley, R. y Zisserman, A.:** *Multiple View Geometry in Computer Vision*, 2.^a ed. Cambridge University Press, 2004. Capítulos 6, 9 y 18 (modelo de cámara, geometría epipolar y ajuste de haces). <https://doi.org/10.1017/CB09780511811685>
2. **Luhmann, T., Robson, S., Kyle, S. y Boehm, J.:** *Close-Range Photogrammetry and 3D Imaging*, 4.^a ed. De Gruyter, Berlín, 2023. Referencia normativa para fotogrametría de corto alcance y su presupuesto de incertidumbre. <https://doi.org/10.1515/9783111029672>
3. **Triggs, B., McLauchlan, P., Hartley, R. y Fitzgibbon, A.:** “Bundle Adjustment — A Modern Synthesis”, en *Vision Algorithms: Theory and Practice*, LNCS vol. 1883, pp. 298–372, Springer, 2000. https://doi.org/10.1007/3-540-44480-7_21
4. **Kazhdan, M. y Hoppe, H.:** “Screened Poisson Surface Reconstruction”, *ACM Transactions on Graphics*, vol. 32, núm. 3, art. 29, 2013. <https://doi.org/10.1145/2487228.2487237>
5. **Lorensen, W. E. y Cline, H. E.:** “Marching Cubes: A High Resolution 3D Surface Construction Algorithm”, *ACM SIGGRAPH Computer Graphics*, vol. 21, núm. 4, pp. 163–169, 1987. <https://doi.org/10.1145/37402.37422>
6. **Rusinkiewicz, S. y Levoy, M.:** “Efficient Variants of the ICP Algorithm”, en *Proceedings of the Third International Conference on 3-D Digital Imaging and Modeling*, pp. 145–152, 2001. <https://doi.org/10.1109/IM.2001.924423>
7. **Garland, M. y Heckbert, P. S.:** “Surface Simplification Using Quadric Error Metrics”, en *Proceedings of SIGGRAPH 97*, pp. 209–216, 1997. <https://doi.org/10.1145/258734.258849>

8. **Taubin, G.:** “A Signal Processing Approach to Fair Surface Design”, en *Proceedings of SIGGRAPH 95*, pp. 351–358, 1995. <https://doi.org/10.1145/218380.218473>
9. **Moulon, P., Monasse, P. y Marlet, R.:** “Global Fusion of Relative Motions for Robust, Accurate and Scalable Structure from Motion”, en *IEEE International Conference on Computer Vision (ICCV)*, pp. 3248–3255, 2013. <https://doi.org/10.1109/ICCV.2013.403>
10. **AliceVision: Meshroom: 3D Reconstruction Software**, versión 2023.3.0, 2023. Documentación oficial del pipeline de nodos. <https://alicevision.org/>
11. **VDI/VDE 2634: Optical 3-D Measuring Systems. Optical Systems Based on Area Scanning**, Parte 2. Verein Deutscher Ingenieure, Düsseldorf. Norma de aceptación y verificación de sistemas ópticos de medición tridimensional.

11.16. Anexo: códigos de la Semana 12

Los tres scripts son reproducibles (semilla fija con `numpy.random.default_rng`), guardan sus figuras a 300 dpi en la carpeta Figuras/ y se ejecutan sin modificación en Google Colab o en instalación local. Solo requieren `numpy`, `scipy`, `scikit-image` y `matplotlib`: la ausencia de dependencias compiladas de gráficos 3D hace que el pipeline sea íntegramente auditable, a diferencia de una llamada a una biblioteca cerrada. Los valores citados en el texto de esta semana provienen de la ejecución directa de estos códigos.

11.16.1. Código 1. Presupuesto óptico de la sesión fotogramétrica

```

1  # -*- coding: utf-8 -*-
2  """
3  Semana 12 - Prototipado y Simulacion (DITA, Universidad La Salle Bajío)
4  Código 1. Presupuesto optico de una sesion fotogrametrica.
5
6  Calcula, para una configuracion camara-objeto declarada:
7      (1) GSD = p * Z / f                (resolucion lateral en el objeto)
8      (2) sZ  = Z^2 * sd / (fpx * B)      (incertidumbre de profundidad por
9                                          triangulacion, par convergente)
10     (3) B    = 2 * Z * sin(da/2)         (base efectiva en captura orbital)
11     (4) o    = 1 - B / W, W = 2*Z*tan(FOV/2) (solapamiento entre estaciones)
12     (5) N    = 360/da * n_anillos        (numero de imagenes)
13
14  El modelo de matching degrada la precision de correspondencia con la
15  separacion angular (distorsion perspectiva del parche). Se adopta el modelo
16  fenomenologico declarado:
17      sd(da) = sd0 * [ 1 + (da/da_c)^2 ]
18  con da_c calibrable por el usuario (valor por defecto 20 deg, coherente con
19  el rango util reportado para descriptores tipo DSP-SIFT). La competencia
20  entre (2) -que mejora con da- y este termino -que empeora con da- produce
21  un optimo interior que el script localiza numericamente.
22
23  Salida: Figuras/S12_planificacion_captura.png (300 dpi) y resumen en consola.

```

```

24 Backend Agg: ejecutable en Google Colab sin display.
25 """
26
27 import os
28 import numpy as np
29 import matplotlib
30 matplotlib.use('Agg')
31 import matplotlib.pyplot as plt
32 from scipy.optimize import minimize_scalar
33
34 os.makedirs('Figuras', exist_ok=True)
35 rng = np.random.default_rng(12)
36
37 # -----
38 # 1. Configuración nominal declarada (APS-C 24 MP + objetivo de 35 mm)
39 # -----
40 W_SENS = 22.3          # ancho del sensor [mm]
41 NPX_H = 6000           # píxeles en la dimensión ancha
42 P_PIX = W_SENS / NPX_H # pitch de píxel [mm]
43 F_MM = 35.0            # distancia focal [mm]
44 Z_NOM = 600.0          # distancia de trabajo nominal objeto-cámara [mm]
45 SDO = 0.30             # ruido de correspondencia a ángulo nulo [px]
46 DA_C = 20.0            # ángulo característico de degradación del matching [deg]
47 N_ANILLOS = 2          # anillos de captura (0 deg y +30 deg)
48
49 F_PX = F_MM / P_PIX    # focal en píxeles
50 FOV = 2.0 * np.arctan(W_SENS / (2.0 * F_MM)) # campo de visión horizontal [rad]
51
52
53 def gsd(z, f_mm=F_MM, p=P_PIX):
54     """Ground sampling distance [mm/px] a distancia z [mm]."""
55     return p * z / f_mm
56
57
58 def base_orbital(z, da_deg):
59     """Base estereo efectiva [mm] entre dos estaciones separadas da_deg."""
60     return 2.0 * z * np.sin(np.deg2rad(da_deg) / 2.0)
61
62
63 def sigma_d(da_deg, sd0=SDO, da_c=DA_C):
64     """Ruido de correspondencia [px] en función de la separación angular."""
65     return sd0 * (1.0 + (da_deg / da_c) ** 2)
66
67
68 def sigma_z(da_deg, z=Z_NOM, degradacion=True):
69     """Incertidumbre estándar de profundidad [mm] de un par convergente."""
70     b = base_orbital(z, da_deg)
71     sd = sigma_d(da_deg) if degradacion else SDO
72     return (z ** 2) * sd / (F_PX * b)
73
74
75 def solapamiento(da_deg, z=Z_NOM):
76     """Fracción de solapamiento entre estaciones consecutivas (0-1)."""
77     w = 2.0 * z * np.tan(FOV / 2.0) # huella lateral del encuadre [mm]
78     return 1.0 - base_orbital(z, da_deg) / w
79
80
81 def n_imagenes(da_deg, n_anillos=N_ANILLOS):
82     return np.ceil(360.0 / da_deg) * n_anillos
83

```



```

84
85 # -----
86 # 2. Optimo de separacion angular
87 # -----
88 res = minimize_scalar(sigma_z, bounds=(2.0, 80.0), method='bounded')
89 DA_OPT = float(res.x)
90 SZ_OPT = float(res.fun)
91
92 GSD_NOM = gsd(Z_NOM)
93 SZ_15 = float(sigma_z(15.0))
94 SZ_15_ideal = float(sigma_z(15.0, degradacion=False))
95 OVL_OPT = float(solapamiento(DA_OPT))
96 N_OPT = float(n_imagenes(DA_OPT))
97
98 # separacion angular maxima admisible por criterio de solapamiento >= 0.60
99 da_grid = np.linspace(2.0, 80.0, 4000)
100 ovl = solapamiento(da_grid)
101 DA_OVL60 = float(da_grid[np.argmin(np.abs(ovl - 0.60))])
102 DA_OVL80 = float(da_grid[np.argmin(np.abs(ovl - 0.80))])
103
104 # relacion de aspecto del presupuesto: profundidad frente a lateral
105 RATIO = SZ_OPT / GSD_NOM
106
107 print('=' * 68)
108 print('PRESUPUESTO OPTICO DE LA SESION FOTOGRAMETRICA')
109 print('=' * 68)
110 print('Pitch de pixel                p      = {:.4f} mm'.format(P_PIX))
111 print('Focal en pixeles              f_px   = {:.0f} px'.format(F_PIX))
112 print('Campo de vision horizontal    FOV    = {:.1f} deg'
113       .format(np.rad2deg(FOV)))
114 print('GSD a Z = {:.0f} mm            GSD    = {:.4f} mm/px ({:.1f} um)'
115       .format(Z_NOM, GSD_NOM, GSD_NOM * 1000))
116 print('=' * 68)
117 print('Separacion angular optima      da*    = {:.1f} deg'.format(DA_OPT))
118 print('Incertidumbre de profundidad  sZ*    = {:.4f} mm ({:.1f} um)'
119       .format(SZ_OPT, SZ_OPT * 1000))
120 print('Relacion sZ*/GSD                = {:.2f}'.format(RATIO))
121 print('Solapamiento en da*              = {:.3f}'.format(OVL_OPT))
122 print('Imagenes requeridas en da*      N      = {:.0f} ({} anillos)'
123       .format(N_OPT, N_ANILLOS))
124 print('=' * 68)
125 print('sZ a da = 15 deg (con degradacion) = {:.4f} mm'.format(SZ_15))
126 print('sZ a da = 15 deg (sin degradacion) = {:.4f} mm'.format(SZ_15_ideal))
127 print('da con solapamiento = 0.80         = {:.1f} deg'.format(DA_OVL80))
128 print('da con solapamiento = 0.60         = {:.1f} deg'.format(DA_OVL60))
129 print('=' * 68)
130 # Optimo restringido: el solapamiento minimo del pipeline SfM domina el diseno
131 SZ_OVL60 = float(sigma_z(DA_OVL60))
132 SZ_OVL80 = float(sigma_z(DA_OVL80))
133 PENAL60 = 100.0 * (SZ_OVL60 / SZ_OPT - 1.0)
134 PENAL80 = 100.0 * (SZ_OVL80 / SZ_OPT - 1.0)
135 print('OPTIMO RESTRINGIDO (la restricción activa es el solapamiento)')
136 print('  o >= 0.60 -> da = {:.1f} deg, sZ = {:.1f} um, N = {:.0f}, '
137       'penalizacion = {:.1f} por ciento'
138       .format(DA_OVL60, SZ_OVL60 * 1000, n_imagenes(DA_OVL60), PENAL60))
139 print('  o >= 0.80 -> da = {:.1f} deg, sZ = {:.1f} um, N = {:.0f}, '
140       'penalizacion = {:.1f} por ciento'
141       .format(DA_OVL80, SZ_OVL80 * 1000, n_imagenes(DA_OVL80), PENAL80))
142 print('=' * 68)
143

```

```

144 # -----
145 # 3. Figura de tres paneles
146 # -----
147 AZUL = '#003366'
148 ROJO = '#B22222'
149 VERDE = '#2E7D32'
150 NARANJA = '#E07B00'
151
152 fig, ax = plt.subplots(1, 3, figsize=(15.0, 4.4))
153
154 # --- (a) GSD frente a distancia de trabajo -----
155 z_grid = np.linspace(200.0, 1500.0, 500)
156 for f_i, est in zip([24.0, 35.0, 50.0], ['-.', '-', '--']):
157     ax[0].plot(z_grid, gsd(z_grid, f_mm=f_i) * 1000.0, est, lw=1.8,
158               color=AZUL if f_i == 35.0 else 'gray',
159               label=f'f = {:.0f} mm'.format(f_i))
160 ax[0].axhline(100.0, color=ROJO, lw=1.2, ls=':')
161 ax[0].text(215, 106, 'criterio GSD = 100 um', color=ROJO, fontsize=8)
162 ax[0].plot([Z_NOM], [GSD_NOM * 1000.0], 'o', ms=7, color=ROJO, zorder=5)
163 ax[0].annotate('config. nominal\n{:.0f} um'.format(GSD_NOM * 1000.0),
164               xy=(Z_NOM, GSD_NOM * 1000.0), xytext=(760, 40),
165               fontsize=8, color=ROJO,
166               arrowprops=dict(arrowstyle='->', color=ROJO, lw=0.9))
167 ax[0].set_xlabel('Distancia de trabajo $Z$ [mm]')
168 ax[0].set_ylabel('GSD [$\mu\text{m}$]')
169 ax[0].set_title('(a) Resolución lateral', fontsize=10, loc='left')
170 ax[0].legend(fontsize=8, frameon=False)
171 ax[0].grid(alpha=0.3)
172
173 # --- (b) Incertidumbre de profundidad frente a separación angular -----
174 sz_deg = sigma_z(da_grid) * 1000.0
175 sz_ide = sigma_z(da_grid, degradacion=False) * 1000.0
176 ax[1].plot(da_grid, sz_ide, '--', color='gray', lw=1.5,
177            label='geometría pura $\sigma_d \rightarrow \sigma_{d0}$')
178 ax[1].plot(da_grid, sz_deg, '-', color=AZUL, lw=2.0,
179            label='con degradación de matching')
180 ax[1].plot([DA_OPT], [SZ_OPT * 1000.0], 'o', ms=8, color=ROJO, zorder=5)
181 ax[1].annotate('$\Delta\alpha^{[*]} = {:.1f}^\circ \rightarrow \sigma_Z = {:.0f} \mu\text{m}$'.format(DA_OPT, SZ_OPT * 1000.0),
182               xy=(DA_OPT, SZ_OPT * 1000.0), xytext=(DA_OPT + 12, SZ_OPT * 1000 + 95),
183               fontsize=8.5, color=ROJO,
184               arrowprops=dict(arrowstyle='->', color=ROJO, lw=0.9))
185 ax[1].axvspan(DA_OVL60, 80.0, color=NARANJA, alpha=0.12)
186 Y1MAX = float(np.nanmax(sz_deg[da_grid < 70]))
187 ax[1].text(DA_OVL60 + 1.0, 0.58 * Y1MAX,
188            'solapamiento < 0.60\n(SfM inestable)', fontsize=7.5,
189            color=NARANJA)
190 ax[1].set_xlim(2, 70)
191 ax[1].set_ylim(0, Y1MAX)
192 ax[1].set_xlabel('Separación angular $\Delta\alpha$ [$^\circ$]')
193 ax[1].set_ylabel('$\sigma_Z$ [$\mu\text{m}$]')
194 ax[1].set_title('(b) Presupuesto de profundidad', fontsize=10, loc='left')
195 ax[1].legend(fontsize=7.5, frameon=False, loc='upper right')
196 ax[1].grid(alpha=0.3)
197
198
199 # --- (c) Solapamiento y número de imágenes -----
200 ax[2].plot(da_grid, ovl, '-', color=VERDE, lw=2.0)
201 ax[2].axhline(0.60, color=ROJO, lw=1.1, ls=':')
202 ax[2].axhline(0.80, color=NARANJA, lw=1.1, ls=':')
203 ax[2].text(38, 0.625, 'mínimo admisible 0.60', color=ROJO, fontsize=7.5)
    
```

```

204 ax[2].text(38, 0.825, 'recomendado 0.80', color=NARANJA, fontsize=7.5)
205 ax[2].axvline(DA_OPT, color=AZUL, lw=1.2, ls='--')
206 ax[2].set_xlim(2, 70)
207 ax[2].set_ylim(0, 1.0)
208 ax[2].set_xlabel('Separacion angular  $\Delta\alpha$  [°]')
209 ax[2].set_ylabel('Solapamiento  $\sigma$  [-]', color=VERDE)
210 ax[2].tick_params(axis='y', labelcolor=VERDE)
211 ax[2].grid(alpha=0.3)
212
213 ax2b = ax[2].twinx()
214 ax2b.plot(da_grid, n_imagenes(da_grid), '-', color=AZUL, lw=1.6, alpha=0.85)
215 ax2b.set_ylabel('$N$ imagenes ({ anillos})'.format(N_ANILLOS), color=AZUL)
216 ax2b.tick_params(axis='y', labelcolor=AZUL)
217 ax2b.set_yscale('log')
218 ax[2].set_title('(c) Solapamiento y volumen de datos', fontsize=10, loc='left')
219
220 fig.tight_layout()
221 fig.savefig('Figuras/S12_planificacion_captura.png', dpi=300,
222             bbox_inches='tight')
223 # plt.show()
224 print('Figura guardada: Figuras/S12_planificacion_captura.png')

```

Listing 52: Presupuesto óptico del Ejemplo 1: resolución lateral (Ec. 271), incertidumbre de profundidad (Ec. 268), modelo de degradación del emparejamiento (Ec. 291) y restricción de solapamiento (Ec. 290).

11.16.2. Código 2. Banco sintético de *Structure from Motion*

```

1  # -*- coding: utf-8 -*-
2  """
3  Semana 12 - Prototipado y Simulacion (DITA, Universidad La Salle Bajio)
4 Codigo 2. Experimento sintetico de Structure from Motion.
5
6  Objetivo: cuantificar, sobre un banco sintetico con verdad de terreno exacta,
7  las dos propiedades del pipeline SfM que condicionan su uso metrologico.
8
9  (A) Escalamiento del error de triangulacion con el numero de vistas.
10     Se proyecta una pieza sintetica sobre N camaras dispuestas en dos
11     anillos orbitales, se contamina la deteccion con ruido gaussiano de
12      $s_{px}$  [px], se triangula por DLT multivista (SVD) y se compara contra
13     la verdad de terreno tras alineamiento de Umeyama. El escalamiento
14     empirico se ajusta a  $RMS \sim N^{-q}$  y se contrasta con  $q = 1/2$ .
15
16  (B) Propagacion de la escala metrica. La reconstruccion SfM queda definida
17     salvo una similitud (7 gdl). La escala se fija con una barra calibrada
18     de longitud L. Si cada extremo de la barra tiene incertidumbre 3D
19     isotropa  $u_p$ , la escala hereda

```

```

20         u_s / s = sqrt(2) * u_p / L,
21         es decir, el error relativo del modelo completo es inversamente
22         proporcional a la longitud de la barra. Se verifica por Monte Carlo.
23
24 Salida: Figuras/S12_sfm_sintetico.png (300 dpi) y resumen en consola.
25 Backend Agg: ejecutable en Google Colab sin display.
26 """
27
28 import os
29 import numpy as np
30 import matplotlib
31 matplotlib.use('Agg')
32 import matplotlib.pyplot as plt
33 from mpl_toolkits.mplot3d import Axes3D # noqa: F401
34 from matplotlib.ticker import NullFormatter, NullLocator
35
36 os.makedirs('Figuras', exist_ok=True)
37 rng = np.random.default_rng(2024)
38
39 # -----
40 # 1. Camara: mismos intrinsecos delCodigo 1 (APS-C 24 MP, f = 35 mm)
41 # -----
42 NPX_H, NPX_V = 6000, 4000
43 F_PX = 9417.0
44 CX, CY = NPX_H / 2.0, NPX_V / 2.0
45 K = np.array([[F_PX, 0.0, CX],
46               [0.0, F_PX, CY],
47               [0.0, 0.0, 1.0]])
48 S_PX = 0.30 # ruido de deteccion de features [px]
49 R_ORB = 600.0 # radio orbital [mm]
50 ELEV = [0.0, 30.0] # anillos de captura [deg]
51
52
53 # -----
54 # 2. Pieza sintetica: buje cilindrico sobre brida circular
55 # -----
56 def pieza_sintetica(n=4000):
57     """Devuelve (puntos [n,3] en mm, normales unitarias [n,3])."""
58     r_buje, h_buje = 25.0, 80.0
59     r_brida, t_brida = 55.0, 12.0
60     n1 = int(0.45 * n) # lateral del buje
61     n2 = int(0.30 * n) # canto y cara superior de la brida
62     n3 = n - n1 - n2 # tapa del buje
63
64     th = rng.uniform(0, 2 * np.pi, n1)
65     z = rng.uniform(t_brida, t_brida + h_buje, n1)
66     p1 = np.c_[r_buje * np.cos(th), r_buje * np.sin(th), z]
67     nn1 = np.c_[np.cos(th), np.sin(th), np.zeros(n1)]
68
69     th = rng.uniform(0, 2 * np.pi, n2)
70     rad = np.sqrt(rng.uniform(r_buje ** 2, r_brida ** 2, n2))
71     p2 = np.c_[rad * np.cos(th), rad * np.sin(th),
72               np.full(n2, t_brida)]
73     nn2 = np.tile(np.array([0.0, 0.0, 1.0]), (n2, 1))
74
75     th = rng.uniform(0, 2 * np.pi, n3)
76     rad = r_buje * np.sqrt(rng.uniform(0, 1, n3))
77     p3 = np.c_[rad * np.cos(th), rad * np.sin(th),
78               np.full(n3, t_brida + h_buje)]
79     nn3 = np.tile(np.array([0.0, 0.0, 1.0]), (n3, 1))

```

```

80
81     P = np.vstack([p1, p2, p3])
82     Nrm = np.vstack([nn1, nn2, nn3])
83     P[:, 2] -= (t_brida + h_buje) / 2.0          # centrar en Z
84     return P, Nrm / np.linalg.norm(Nrm, axis=1, keepdims=True)
85
86
87 def poses_orbitales(n_cam, radio=R_ORB, elevaciones=ELEV):
88     """Genera matrices [R/t] mirando al origen desde anillos orbitales."""
89     poses, centros = [], []
90     por_anillo = int(np.ceil(n_cam / len(elevaciones)))
91     for el in elevaciones:
92         for k in range(por_anillo):
93             az = 2 * np.pi * k / por_anillo
94             e = np.deg2rad(el)
95             C = radio * np.array([np.cos(e) * np.cos(az),
96                                   np.cos(e) * np.sin(az),
97                                   np.sin(e)])
98             zc = -C / np.linalg.norm(C)          # eje optico
99             up = np.array([0.0, 0.0, 1.0])
100             if abs(np.dot(zc, up)) > 0.98:
101                 up = np.array([0.0, 1.0, 0.0])
102             xc = np.cross(up, zc)
103             xc /= np.linalg.norm(xc)
104             yc = np.cross(zc, xc)
105             R = np.vstack([xc, yc, zc])
106             poses.append((R, -R @ C))
107             centros.append(C)
108     return poses[:n_cam], np.array(centros[:n_cam])
109
110
111 def proyectar(P, R, t):
112     Pc = (R @ P.T).T + t
113     uv = (K @ Pc.T).T
114     return uv[:, :2] / uv[:, 2:3], Pc[:, 2]
115
116
117 def triangular_dlt(obs, Ps):
118     """obs: lista de (u,v); Ps: lista de matrices 3x4. Devuelve X en mm."""
119     A = []
120     for (u, v), Pm in zip(obs, Ps):
121         A.append(u * Pm[2] - Pm[0])
122         A.append(v * Pm[2] - Pm[1])
123     _, _, Vt = np.linalg.svd(np.array(A))
124     X = Vt[-1]
125     return X[:3] / X[3]
126
127
128 def umeyama(X, Y):
129     """Similitud (s,R,t) que lleva X sobre Y en minimos cuadrados."""
130     mx, my = X.mean(0), Y.mean(0)
131     Xc, Yc = X - mx, Y - my
132     S = (Yc.T @ Xc) / len(X)
133     U, D, Vt = np.linalg.svd(S)
134     d = np.sign(np.linalg.det(U @ Vt))
135     W = np.diag([1.0, 1.0, d])
136     R = U @ W @ Vt
137     s = np.trace(np.diag(D) @ W) / (Xc ** 2).sum(1).mean()
138     return s, R, my - s * (R @ mx)
139

```

```

140
141 def reconstruir(P_gt, Nrm, n_cam, s_px=S_PX, min_vistas=3):
142     """Devuelve (X_rec, idx_validos, rms_reproyeccion)."""
143     poses, _ = poses_orbitales(n_cam)
144     Pmats = [K @ np.hstack([R, t.reshape(3, 1)]) for R, t in poses]
145     obs = {i: [] for i in range(len(P_gt))}
146     res_rep = []
147     for j, (R, t) in enumerate(poses):
148         C = -R.T @ t
149         uv, zc = proyectar(P_gt, R, t)
150         vis = ((Nrm * (C - P_gt)).sum(1) > 0) & (zc > 0)
151         vis &= (uv[:, 0] > 0) & (uv[:, 0] < NPX_H)
152         vis &= (uv[:, 1] > 0) & (uv[:, 1] < NPX_V)
153         ruido = rng.normal(0.0, s_px, size=uv.shape)
154         for i in np.flatnonzero(vis):
155             obs[i].append((uv[i] + ruido[i], j))
156     X, idx = [], []
157     for i, lst in obs.items():
158         if len(lst) >= min_vistas:
159             X.append(triangular_dlt([o[0] for o in lst],
160                                     [Pmats[o[1]] for o in lst]))
161             idx.append(i)
162         for uvo, j in lst:
163             R, t = poses[j]
164             uvp, _ = proyectar(X[-1][None, :], R, t)
165             res_rep.extend((uvp[0] - uvo).tolist())
166     return np.array(X), np.array(idx), float(np.sqrt(np.mean(
167         np.array(res_rep) ** 2)))
168
169
170 # -----
171 # 3. (A) Error de triangulacion frente al numero de vistas
172 # -----
173 P_gt, N_gt = pieza_sintetica(4000)
174 N_LIST = [6, 8, 12, 16, 24, 36, 48]
175 rms_3d, rms_rep, n_pts, cob = [], [], [], []
176
177 for n_cam in N_LIST:
178     Xr, idx, rrep = reconstruir(P_gt, N_gt, n_cam)
179     s, R, t = umeyama(Xr, P_gt[idx])
180     Xa = s * (R @ Xr.T).T + t
181     d = np.linalg.norm(Xa - P_gt[idx], axis=1)
182     rms_3d.append(float(np.sqrt(np.mean(d ** 2))))
183     rms_rep.append(rrep)
184     n_pts.append(len(idx))
185     cob.append(100.0 * len(idx) / len(P_gt))
186
187 rms_3d = np.array(rms_3d)
188 coef = np.polyfit(np.log(N_LIST), np.log(rms_3d), 1)
189 Q_EMP = -coef[0]
190
191 # Reconstruccion de referencia con 24 camaras (punto de diseno del curso)
192 X24, idx24, rrep24 = reconstruir(P_gt, N_gt, 24)
193 s24, R24, t24 = umeyama(X24, P_gt[idx24])
194 X24a = s24 * (R24 @ X24.T).T + t24
195 d24 = np.linalg.norm(X24a - P_gt[idx24], axis=1)
196 U_P = float(np.sqrt(np.mean(d24 ** 2))) # incertidumbre 3D por punto [mm]
197
198 print('=' * 70)
199 print('(A) TRIANGULACION MULTIVISTA')

```

```

200 print('=' * 70)
201 print('{:>6} {:>10} {:>12} {:>14} {:>14}')
202     .format('N cam', 'pts', 'cobertura', 'RMS 3D [mm]', 'RMS rep [px/c]'))
203 for n_cam, npt, c, r3, rr in zip(N_LIST, n_pts, cob, rms_3d, rms_rep):
204     print('{:>6d} {:>10d} {:>11.1f}% {:>14.4f} {:>14.3f}')
205     .format(n_cam, npt, c, r3, rr))
206 print('-' * 70)
207 print('Exponente empirico RMS ~ N^(-q): q = {:.3f} (teorico 0.5)'
208     .format(Q_EMP))
209 print('Punto de diseno N = 24: RMS 3D = {:.4f} mm, '
210     'residual de reproyeccion = {:.3f} px/coord'.format(U_P, rrep24))
211 print('Relacion RMS 3D / GSD(600 mm, 63.7 um) = {:.2f}'.format(U_P / 0.0637))
212
213 # -----
214 # 4. (B) Escala metrica: Monte Carlo sobre la barra de calibracion
215 # -----
216 L_LIST = np.array([25.0, 50.0, 75.0, 100.0, 150.0, 200.0, 300.0, 500.0])
217 N_MC = 4000
218 err_s = []
219 for L in L_LIST:
220     # dos targets separados L, cada extremo con incertidumbre 3D isotropa U_P
221     e1 = rng.normal(0.0, U_P / np.sqrt(3.0), size=(N_MC, 3))
222     e2 = rng.normal(0.0, U_P / np.sqrt(3.0), size=(N_MC, 3))
223     p1 = np.zeros((N_MC, 3)) + e1
224     p2 = np.tile([L, 0.0, 0.0], (N_MC, 1)) + e2
225     L_med = np.linalg.norm(p2 - p1, axis=1)
226     err_s.append(float(np.std(L_med / L))) # u_s/s adimensional
227 err_s = np.array(err_s)
228 err_teo = np.sqrt(2.0) * (U_P / np.sqrt(3.0)) / L_LIST
229
230 L_REF = 150.0
231 i_ref = int(np.argmin(np.abs(L_LIST - L_REF)))
232 DIM_NOM = 110.0 # cota nominal de la pieza [mm]
233 print('=' * 70)
234 print('(B) ESCALA METRICA A PARTIR DE BARRA CALIBRADA')
235 print('=' * 70)
236 print('{:>10} {:>16} {:>18} {:>18}')
237     .format('L [mm]', 'u_s/s [ppm]', 'u_s/s MC [ppm]',
238     'error en 110 mm [um]'))
239 for L, em, et in zip(L_LIST, err_s, err_teo):
240     print('{:>10.0f} {:>16.0f} {:>18.0f} {:>18.1f}')
241     .format(L, et * 1e6, em * 1e6, em * DIM_NOM * 1000))
242 print('-' * 70)
243 print('Barra de {:.0f} mm -> u_s/s = {:.0f} ppm; sobre una cota de {:.0f} mm '
244     'aporta {:.1f} um'.format(L_LIST[i_ref], err_s[i_ref] * 1e6, DIM_NOM,
245     err_s[i_ref] * DIM_NOM * 1000))
246 print('=' * 70)
247
248 # -----
249 # 5. Figura de tres paneles
250 # -----
251 AZUL = '#003366'
252 ROJO = '#B22222'
253 VERDE = '#2E7D32'
254
255 fig = plt.figure(figsize=(15.0, 4.5))
256
257 ax0 = fig.add_subplot(1, 3, 1, projection='3d')
258 sub = rng.choice(len(P_gt), 1200, replace=False)
259 ax0.scatter(P_gt[sub, 0], P_gt[sub, 1], P_gt[sub, 2], s=1.5,

```

```

260         c=P_gt[sub, 2], cmap='viridis', alpha=0.85)
261     _, C24 = poses_orbitales(24)
262     ax0.scatter(C24[:, 0], C24[:, 1], C24[:, 2], marker='^', s=26,
263               color=R0J0, depthshade=False, label='estaciones (N = 24)')
264     for C in C24:
265         ax0.plot([C[0], 0], [C[1], 0], [C[2], 0], color=R0J0, lw=0.35, alpha=0.35)
266     ax0.set_xlabel('X [mm]', fontsize=8)
267     ax0.set_ylabel('Y [mm]', fontsize=8)
268     ax0.set_zlabel('Z [mm]', fontsize=8)
269     ax0.tick_params(labelsize=7)
270     ax0.set_title('(a) Banco sintético: pieza y red de estaciones',
271                 fontsize=10, loc='left')
272     ax0.legend(fontsize=7.5, loc='upper right', frameon=False)
273     ax0.view_init(elev=20, azim=40)
274
275     ax1 = fig.add_subplot(1, 3, 2)
276     ax1.loglog(N_LIST, rms_3d * 1000, 'o-', color=AZUL, lw=1.9, ms=6,
277              label='RMS 3D simulado')
278     ref = rms_3d[0] * (np.array(N_LIST, dtype=float) / N_LIST[0]) ** (-0.5)
279     ax1.loglog(N_LIST, ref * 1000, '--', color='gray', lw=1.4,
280              label='referencia  $N^{-1/2}$ ')
281     ax1.loglog([24], [U_P * 1000], 's', ms=9, mfc='none', mec=R0J0, mew=1.8)
282     ax1.annotate('punto de diseño\nN = 24: {:.0f}  $\mu$  m'.format(U_P * 1000),
283                xy=(24, U_P * 1000), xytext=(25, rms_3d[0] * 1000 * 0.95),
284                fontsize=8, color=R0J0,
285                arrowprops=dict(arrowstyle='->', color=R0J0, lw=0.9))
286     ax1.set_xlabel('Número de estaciones  $N$ ')
287     ax1.set_ylabel('RMS del error 3D [ $\mu$  m]')
288     ax1.set_title('(b) Escalamiento del error de triangulación ( $q = {:.2f}$ )'.
289                 .format(Q_EMP), fontsize=10, loc='left')
290     ax1.xaxis.set_minor_locator(NullLocator())
291     ax1.xaxis.set_minor_formatter(NullFormatter())
292     ax1.set_xticks(N_LIST)
293     ax1.set_xticklabels([str(v) for v in N_LIST])
294     ax1.legend(fontsize=8, frameon=False)
295     ax1.grid(alpha=0.3, which='both')
296
297     ax2 = fig.add_subplot(1, 3, 3)
298     ax2.loglog(L_LIST, err_s * 1e6, 'o', color=VERDE, ms=6,
299              label='Monte Carlo ({:d} realizaciones)'.format(N_MC))
300     ax2.loglog(L_LIST, err_teo * 1e6, '-', color=AZUL, lw=1.8,
301              label=' $\sqrt{2} u_p / L$ ')
302     ax2.axhline(1000, color=R0J0, ls=':', lw=1.2)
303     ax2.set_ylim(10, 2500)
304     ax2.text(26, 1150, 'criterio  $10^{-3}$  ppm (0.1 por ciento)',
305            color=R0J0, fontsize=7.5)
306     ax2.set_xlabel('Longitud de la barra de escala  $L$  [mm]')
307     ax2.set_ylabel('u_s/s [ppm]')
308     ax2.set_title('(c) Herencia de la escala métrica', fontsize=10, loc='left')
309     ax2.legend(fontsize=8, frameon=False)
310     ax2.grid(alpha=0.3, which='both')
311
312     fig.tight_layout()
313     fig.savefig('Figuras/S12_sfm_sintetico.png', dpi=300, bbox_inches='tight')
314     # plt.show()
315     print('Figura guardada: Figuras/S12_sfm_sintetico.png')

```

Listing 53: Experimento sintético del Ejemplo 2: triangulación DLT multivista (Ec. 267), alineamiento de similitud de Umeyama (Ec. 270) y propagación de la escala métrica (Ec. 273).

11.16.3. Código 3. De la nube de puntos al STL auditado

```

1  # -*- coding: utf-8 -*-
2  """
3  Semana 12 - Prototipado y Simulacion (DITA, Universidad La Salle Bajio)
4 Codigo 3. De la nube de puntos a la malla STL apta para simulacion.
5
6 Pipeline completo, auditable y con verdad de terreno analitica:
7
8 1. Solido de referencia definido por una funcion de distancia con signo
9 (SDF): brida circular + buje cilindrico con barreno pasante.
10 2. Muestreo superficial y contaminacion controlada: ruido gaussiano normal
11 de desviacion s_n y una fraccion f_out de outliers volumetricos.
12 3. Filtrado estadistico de outliers (SOR): distancia media a los k vecinos
13 mas proximos; se descarta el punto si d_i > mu_d + n_sigma * sigma_d.
14 Se reportan precision y exhaustividad frente al etiquetado real.
15 4. Submuestreo por voxeles (rejilla de arista a_v) y estimacion de normales
16 por analisis de componentes principales de la matriz de covarianza local
17 (el autovector del autovalor minimo es la normal).
18 5. Orientacion global de normales por propagacion de signo sobre el arbol
19 de expansion minima del grafo kNN, con peso w_ij = 1 - |n_i . n_j|.
20 6. Reconstruccion implicita: campo escalar por minimos cuadrados moviles
21 sobre planos tangentes locales, evaluado unicamente en la cascara de
22 radio r_max en torno a la nube. El interior y el exterior se etiquetan
23 por componentes conexas, evitando la cascara falsa característica de
24 Poisson sin recorte. Extraccion de la isosuperficie cero por Marching
25 Cubes.
26 7. Auditoria topologica: V, E, F, característica de Euler, genero, aristas
27 de frontera, aristas no-manifold, caras degeneradas, estanqueidad.
28 8. Suavizado de Taubin (lambda / mu) que preserva volumen, decimacion por
29 agrupamiento de vertices y exportacion a STL binario sin dependencias.
30 9. Verificacion dimensional contra el SDF analitico: RMS, P95 y maximo de
31 la desviacion, y error volumetrico.
32
33 Requisitos: numpy, scipy, scikit-image, matplotlib. Sin dependencias de GPU.
34 Salida: Figuras/S12_nube_a_malla.png (300 dpi), Figuras/S12_pieza_final.stl.
35 Backend Agg: ejecutable en Google Colab sin display.
36 """
37
38 import os
39 import struct
40 import numpy as np
41 import matplotlib
42 matplotlib.use('Agg')
43 import matplotlib.pyplot as plt
44 from mpl_toolkits.mplot3d.art3d import Poly3DCollection
45 from scipy.spatial import cKDTree
46 from scipy.sparse import coo_matrix
47 from scipy.sparse.csgraph import (minimum_spanning_tree,
48                                  breadth_first_order,
49                                  connected_components)
50 from scipy import ndimage
51 from skimage import measure
52
53 os.makedirs('Figuras', exist_ok=True)
54 rng = np.random.default_rng(1204)
55
56 # -----
57 # Parametros del experimento (declarados)
58 # -----

```

```

59 R_BRIDA, T_BRIDA = 55.0, 12.0      # radio y espesor de la brida [mm]
60 R_BUJE, H_BUJE = 25.0, 80.0      # radio y altura del buje [mm]
61 R_HOLE = 10.0                    # radio del barreno pasante [mm]
62 N_PTS = 120000                   # puntos muestreados en la superficie
63 S_N = 0.10                       # ruido normal, desviacion estandar [mm]
64 F_OUT = 0.02                     # fraccion de outliers
65 K_SOR, NSIG_SOR, IT_SOR = 20, 3.0, 2 # filtro estadistico robusto
66 A_VOX = 0.60                     # arista del voxel [mm]
67 K_NRM = 25                       # vecinos para PCA de normales
68 H_GRID = 0.70                    # paso de la rejilla implicita [mm]
69 R_MAX = 2.20                     # radio de la cascara de evaluacion [mm]
70 LAM_T, MU_T, IT_T = 0.50, -0.53, 3 # Taubin
71 A_CLUS = 0.90                    # arista de agrupamiento para decimacion
72
73
74 # -----
75 # 1. Verdad de terreno: SDF analitica del solido
76 # -----
77 def sdf_cilindro(P, r, z0, z1):
78     """SDF exacta de un cilindro recto con tapas, eje Z."""
79     zc, hz = 0.5 * (z0 + z1), 0.5 * (z1 - z0)
80     dxy = np.hypot(P[..., 0], P[..., 1]) - r
81     dz = np.abs(P[..., 2] - zc) - hz
82     fuera = np.sqrt(np.maximum(dxy, 0.0) ** 2 + np.maximum(dz, 0.0) ** 2)
83     return np.minimum(np.maximum(dxy, dz), 0.0) + fuera
84
85
86 def sdf_pieza(P):
87     """Union de brida y buje, menos el barreno pasante. Centrada en Z."""
88     P = np.asarray(P, dtype=float)
89     z_off = (T_BRIDA + H_BUJE) / 2.0
90     Q = P.copy()
91     Q[..., 2] = Q[..., 2] + z_off
92     d_bri = sdf_cilindro(Q, R_BRIDA, 0.0, T_BRIDA)
93     d_buj = sdf_cilindro(Q, R_BUJE, T_BRIDA, T_BRIDA + H_BUJE)
94     d_hol = sdf_cilindro(Q, R_HOLE, -1.0, T_BRIDA + H_BUJE + 1.0)
95     return np.maximum(np.minimum(d_bri, d_buj), -d_hol)
96
97
98 VOL_GT = (np.pi * R_BRIDA ** 2 * T_BRIDA
99           + np.pi * R_BUJE ** 2 * H_BUJE
100           - np.pi * R_HOLE ** 2 * (T_BRIDA + H_BUJE))
101
102
103 def malla_de_sdf(func, lo, hi, paso):
104     """Isosuperficie cero de un campo escalar por Marching Cubes."""
105     ejes = [np.arange(lo[i], hi[i] + paso, paso) for i in range(3)]
106     G = np.stack(np.meshgrid(*ejes, indexing='ij'), axis=-1)
107     F = func(G)
108     v, f, _, _ = measure.marching_cubes(F, level=0.0,
109                                         spacing=(paso, paso, paso))
110     v = v + np.array(lo)
111     return v, f
112
113
114 def componente_mayor(V, F):
115     """Conserva la pieza conexas de mayor numero de vertices."""
116     e = np.vstack([F[:, [0, 1]], F[:, [1, 2]], F[:, [2, 0]]])
117     A = coo_matrix((np.ones(len(e)), (e[:, 0], e[:, 1])),
118                   shape=(len(V),) * 2)

```

```

119     n_c, et = connected_components(A, directed=False)
120     keep = np.argmax(np.bincount(et))
121     mv = et == keep
122     remap = -np.ones(len(V), dtype=np.int64)
123     remap[mv] = np.arange(mv.sum())
124     mf = mv[F].all(1)
125     return V[mv], remap[F[mf]], n_c
126
127
128 def orientar_hacia_afuera(V, F):
129     """Fuerza volumen positivo por el teorema de la divergencia."""
130     tri = V[F]
131     vol = np.einsum('ij,ij->i', np.cross(tri[:, 0], tri[:, 1]),
132                  tri[:, 2]).sum() / 6.0
133     return F if vol > 0 else F[:, :-1]
134
135
136 # --- muestreo superficial ponderado por area de las caras -----
137 V_GT, F_GT = malla_de_sdf(sdf_pieza, (-62, -62, -55), (62, 62, 55), 0.45)
138 tri = V_GT[F_GT]
139 nrm_f = np.cross(tri[:, 1] - tri[:, 0], tri[:, 2] - tri[:, 0])
140 area = 0.5 * np.linalg.norm(nrm_f, axis=1)
141 val = area > 1e-9 # descartar caras degeneradas
142 F_GT, tri, nrm_f, area = F_GT[val], tri[val], nrm_f[val], area[val]
143 nrm_f = nrm_f / np.linalg.norm(nrm_f, axis=1, keepdims=True)
144 prob = area / area.sum()
145 sel = rng.choice(len(F_GT), N_PTS, p=prob)
146 u, v_ = rng.random(N_PTS), rng.random(N_PTS)
147 fix = u + v_ > 1.0
148 u[fix], v_[fix] = 1.0 - u[fix], 1.0 - v_[fix]
149 P_surf = (tri[sel, 0] + u[:, None] * (tri[sel, 1] - tri[sel, 0])
150          + v_[:, None] * (tri[sel, 2] - tri[sel, 0]))
151 N_surf = nrm_f[sel]
152
153 # --- contaminacion: ruido normal + outliers volumetricos -----
154 n_out = int(F_OUT * N_PTS)
155 P_ruido = P_surf + (rng.normal(0.0, S_N, N_PTS))[:, None] * N_surf
156 lo_b, hi_b = P_surf.min(0) - 8.0, P_surf.max(0) + 8.0
157 P_out = rng.uniform(lo_b, hi_b, size=(n_out, 3))
158 P_all = np.vstack([P_ruido, P_out])
159 es_outlier = np.zeros(len(P_all), dtype=bool)
160 es_outlier[N_PTS:] = True
161 orden = rng.permutation(len(P_all))
162 P_all, es_outlier = P_all[orden], es_outlier[orden]
163
164 print('=' * 72)
165 print('PIPELINE NUBE -> MALLA -> STL')
166 print('=' * 72)
167 print('Nube cruda: {:d} puntos ({:d} outliers, {:.1f} por ciento); '
168       'ruido normal s_n = {:.2f} mm'
169       .format(len(P_all), n_out, 100.0 * n_out / len(P_all), S_N))
170
171 # -----
172 # 2. Filtrado estadístico de outliers (SOR)
173 # -----
174 mant = np.ones(len(P_all), dtype=bool)
175 for _ in range(IT_SOR):
176     kd = cKDTree(P_all[mant])
177     d_knn, _ = kd.query(P_all[mant], k=K_SOR + 1)
178     d_med = d_knn[:, 1:].mean(1)

```

```

179     med = np.median(d_med)
180     mad = 1.4826 * np.median(np.abs(d_med - med)) # escala robusta
181     umbral = med + NSIG_SOR * mad
182     sub_ok = d_med <= umbral
183     idx_act = np.flatnonzero(mant)
184     mant[idx_act[~sub_ok]] = False
185     TP = int(np.sum(es_outlier & ~mant))
186     FP = int(np.sum(~es_outlier & ~mant))
187     FN = int(np.sum(es_outlier & mant))
188     PREC = 100.0 * TP / max(TP + FP, 1)
189     REC = 100.0 * TP / max(TP + FN, 1)
190     P_f = P_all[mant]
191     print('SOR robusto (k = {:d}, n_sigma = {:.1f}, {:d} pasadas): umbral final '
192           'd = {:.4f} mm; elimina {:d} puntos'
193           .format(K_SOR, NSIG_SOR, IT_SOR, umbral, int(np.sum(~mant))))
194     print('      exhaustividad sobre outliers = {:.1f} por ciento; '
195           'precision = {:.1f} por ciento; supervivientes = {:d}'
196           .format(REC, PREC, len(P_f)))
197
198     # -----
199     # 3. Submuestreo por voxeles
200     # -----
201     idx_v = np.floor((P_f - P_f.min(0)) / A_VOX).astype(np.int64)
202     clave = (idx_v[:, 0] * 100003 + idx_v[:, 1]) * 100003 + idx_v[:, 2]
203     orden = np.argsort(clave, kind='stable')
204     clave_o, P_o = clave[orden], P_f[orden]
205     corte = np.flatnonzero(np.diff(clave_o)) + 1
206     grupos = np.split(np.arange(len(P_o)), corte)
207     P_v = np.array([P_o[g].mean(0) for g in grupos])
208     print('Voxel grid (a_v = {:.2f} mm): {:d} -> {:d} puntos '
209           '(factor {:.2f})'.format(A_VOX, len(P_f), len(P_v),
210                                   len(P_v) / len(P_f)))
211
212     # -----
213     # 4. Normales por PCA local
214     # -----
215     kd_v = cKDTree(P_v)
216     _, nb = kd_v.query(P_v, k=K_NRM)
217     Q = P_v[nb] - P_v[nb].mean(1, keepdims=True)
218     C = np.einsum('nki,nkj->nij', Q, Q) / K_NRM
219     w, U = np.linalg.eigh(C)
220     N_v = U[:, :, 0]
221     curv = w[:, 0] / np.maximum(w.sum(1), 1e-12) # variacion de superficie
222
223     # -----
224     # 5. Orientacion global por MST del grafo kNN
225     # -----
226     K_G = 12
227     _, nb_g = kd_v.query(P_v, k=K_G)
228     fil = np.repeat(np.arange(len(P_v)), K_G - 1)
229     col = nb_g[:, 1:].ravel()
230     peso = 1.0 - np.abs(np.einsum('ij,ij->i', N_v[fil], N_v[col]))
231     G = coo_matrix((peso + 1e-6, (fil, col)), shape=(len(P_v),) * 2)
232     T = minimum_spanning_tree(G)
233     T = T + T.T
234     semilla = int(np.argmax(P_v[:, 2]))
235     if N_v[semilla, 2] < 0:
236         N_v[semilla] *= -1.0
237     orden_bfs, padres = breadth_first_order(T, semilla, directed=False)
238     for i in orden_bfs[1:]:
    
```

```

239     p = padres[i]
240     if p >= 0 and np.dot(N_v[i], N_v[p]) < 0:
241         N_v[i] *= -1.0
242 coher = float(np.mean(np.einsum('ij,ij->i', N_v[fil], N_v[col]) > 0))
243 print('Normales: PCA con k = {d}; coherencia de orientacion en el grafo '
244       'kNN = {:.1f} por ciento'.format(K_NRM, 100.0 * coher))
245
246 # -----
247 # 6. Campo implícito y Marching Cubes
248 # -----
249 lo = P_v.min(0) - 3.0
250 hi = P_v.max(0) + 3.0
251 ejes = [np.arange(lo[i], hi[i] + H_GRID, H_GRID) for i in range(3)]
252 forma = tuple(len(e) for e in ejes)
253 G3 = np.stack(np.meshgrid(*ejes, indexing='ij'), axis=-1).reshape(-1, 3)
254
255 d1, _ = kd_v.query(G3, k=1)
256 cerca = d1 <= R_MAX
257 K_IM = 12
258 _, nb_im = kd_v.query(G3[cerca], k=K_IM)
259 dif = G3[cerca][:, None, :] - P_v[nb_im]
260 dist = np.linalg.norm(dif, axis=2)
261 w_im = np.exp(-(dist / (0.6 * R_MAX)) ** 2)
262 proy = np.einsum('nkj,nkj->nk', dif, N_v[nb_im])
263 f_shell = (w_im * proy).sum(1) / np.maximum(w_im.sum(1), 1e-12)
264
265 F3 = np.full(len(G3), np.nan)
266 F3[cerca] = f_shell
267 F3 = F3.reshape(forma)
268
269 lejos = np.isnan(F3)
270 lab, nlab = ndimage.label(lejos)
271 borde = set(np.unique(np.concatenate([
272     lab[0].ravel(), lab[-1].ravel(), lab[:, 0].ravel(), lab[:, -1].ravel(),
273     lab[:, :, 0].ravel(), lab[:, :, -1].ravel()])))
274 borde.discard(0)
275 ext = np.isin(lab, list(borde))
276 F3[lejos & ext] = +10.0 # exterior conexo con la frontera
277 F3[lejos & ~ext] = -10.0 # cavidades internas del solido
278 print('Campo implícito: rejilla {} = {:.2f} M nodos; cascara evaluada = '
279       '{:.1f} por ciento; componentes lejanas = {d}'
280       .format(forma, F3.size / 1e6, 100.0 * cerca.mean(), nlab))
281
282 V_m, F_m, _, _ = measure.marching_cubes(F3, level=0.0,
283                                         spacing=(H_GRID,) * 3)
284 V_m = V_m + lo
285 F_MCO = len(F_m)
286 V_m, F_m, N_COMP = componente_mayor(V_m, F_m)
287 F_m = orientar_hacia_afuera(V_m, F_m)
288 print('Marching Cubes bruto: {d} caras en {d} piezas conexas; se conserva '
289       'la mayor con {d} caras ({:.1f} por ciento)'
290       .format(F_MCO, N_COMP, len(F_m), 100.0 * len(F_m) / F_MCO))
291
292 # -----
293 # 7. Auditoria topologica
294 # -----
295
296 def auditar(V, F):
297     e = np.sort(np.vstack([F[:, [0, 1]], F[:, [1, 2]], F[:, [2, 0]]]), axis=1)
298     _, cnt = np.unique(e, axis=0, return_counts=True)

```

```

299     tri = V[F]
300     ar = 0.5 * np.linalg.norm(np.cross(tri[:, 1] - tri[:, 0],
301                                     tri[:, 2] - tri[:, 0]), axis=1)
302     c = V.mean(0)
303     t0 = tri - c                                # origen en el centroide (robustez)
304     vol = np.abs(np.einsum('ij,ij->i', np.cross(t0[:, 0], t0[:, 1]),
305                                     t0[:, 2]).sum() / 6.0)
306     return dict(V=len(V), E=len(cnt), F=len(F),
307               chi=len(V) - len(cnt) + len(F),
308               frontera=int(np.sum(cnt == 1)),
309               nomanifold=int(np.sum(cnt > 2)),
310               degeneradas=int(np.sum(ar < 1e-9)),
311               area=float(ar.sum()), volumen=float(vol))
312
313
314 aud0 = auditar(V_m, F_m)
315 print('-' * 72)
316 print('Marching Cubes: V = {V:d}, E = {E:d}, F = {F:d}, chi = {chi:d}, '
317       'genero = {g:d}'.format(g=(2 - aud0['chi']) // 2, **aud0))
318 print('    aristas de frontera = {frontera:d}, no-manifold = {nomanifold:d}, '
319       'degeneradas = {degeneradas:d}'.format(**aud0))
320 print('    estancia = {}'.format(aud0['frontera'] == 0
321                                and aud0['nomanifold'] == 0))
322
323
324 # -----
325 # 8. Suavizado de Taubin, decimacion y exportacion
326 # -----
327 def laplaciano_uniforme(V, F):
328     e = np.vstack([F[:, [0, 1]], F[:, [1, 2]], F[:, [2, 0]]])
329     e = np.vstack([e, e[:, :-1]])
330     A = coo_matrix((np.ones(len(e)), (e[:, 0], e[:, 1])),
331                   shape=(len(V),) * 2).tocsr()
332     A.data[:] = 1.0
333     gr = np.asarray(A.sum(1)).ravel()
334     return (A @ V) / np.maximum(gr, 1)[:, None] - V
335
336
337 def taubin(V, F, lam=LAM_T, mu=MU_T, it=IT_T):
338     Vs = V.copy()
339     for _ in range(it):
340         Vs = Vs + lam * laplaciano_uniforme(Vs, F)
341         Vs = Vs + mu * laplaciano_uniforme(Vs, F)
342     return Vs
343
344
345 def decimar_por_agrupamiento(V, F, a):
346     idx = np.floor((V - V.min(0)) / a).astype(np.int64)
347     _, inv = np.unique(idx, axis=0, return_inverse=True)
348     n = inv.max() + 1
349     Vn = np.zeros((n, 3))
350     cnt = np.bincount(inv, minlength=n)[:, None]
351     np.add.at(Vn, inv, V)
352     Vn /= np.maximum(cnt, 1)
353     Fn = inv[F]
354     ok = ((Fn[:, 0] != Fn[:, 1]) & (Fn[:, 1] != Fn[:, 2])
355           & (Fn[:, 0] != Fn[:, 2]))
356     Fn = Fn[ok]
357     # rotacion canonica: conserva la orientacion y permite deduplicar
358     r = np.argmax(Fn, axis=1)

```

```

359     Fn = np.take_along_axis(Fn, (r[:, None] + np.arange(3)[None, :]) % 3, 1)
360     _, uniq = np.unique(Fn, axis=0, return_index=True)
361     Fn = Fn[np.sort(uniq)]
362     usados, Fn = np.unique(Fn, return_inverse=True)
363     return Vn[usados], Fn.reshape(-1, 3)
364
365
366 def escribir_stl_binario(ruta, V, F):
367     tri = V[F].astype(np.float32)
368     n = np.cross(tri[:, 1] - tri[:, 0], tri[:, 2] - tri[:, 0])
369     n /= np.maximum(np.linalg.norm(n, axis=1, keepdims=True), 1e-12)
370     with open(ruta, 'wb') as fh:
371         fh.write(b'Semana12 DITA malla fotogrametrica'.ljust(80, b' '))
372         fh.write(struct.pack('<I', len(F)))
373         for k in range(len(F)):
374             fh.write(struct.pack('<12fH', *n[k], *tri[k, 0], *tri[k, 1],
375                                 *tri[k, 2], 0))
376     return os.path.getsize(ruta)
377
378
379 V_s = taubin(V_m, F_m)
380 aud1 = auditar(V_s, F_m)
381 V_d, F_d = decimar_por_agrupamiento(V_s, F_m, A_CLUS)
382 V_d, F_d, _ = componente_mayor(V_d, F_d)
383 F_d = orientar_hacia_afuera(V_d, F_d)
384 aud2 = auditar(V_d, F_d)
385 BYTES = escribir_stl_binario('Figuras/S12_pieza_final.stl', V_s, F_m)
386
387 print('-' * 72)
388 print('Taubin (lambda = {:.2f}, mu = {:.2f}, {:d} pasadas): cambio de '
389       'volumen = {:.3f} por ciento'
390       .format(LAM_T, MU_T, IT_T,
391               100.0 * (aud1['volumen'] / aud0['volumen'] - 1.0)))
392 print('Decimacion por agrupamiento (a = {:.2f} mm): {:d} -> {:d} caras '
393       '({:.1f} por ciento)'.format(A_CLUS, aud0['F'], aud2['F'],
394                                   100.0 * aud2['F'] / aud0['F']))
395 print('    coste topologico: chi = {chi:d}, frontera = {frontera:d}, '
396       'aristas no-manifold = {nomanifold:d}'.format(**aud2))
397 print('    ADVERTENCIA: el agrupamiento de vertices no preserva la variedad;')
398 print('    para el entregable se decima con QEM (MeshLab / Open3D) y se')
399 print('    reaudita. Se exporta la malla suavizada, estanca y de genero 1.')
400 print('STL binario exportado (malla suavizada): {:.2f} MB, {:d} caras'
401       .format(BYTES / 1e6, aud1['F']))
402
403 # -----
404 # 9. Verificacion dimensional contra el SDF analitico
405 # -----
406 dev_mc = sdf_pieza(V_m)
407 dev_fin = sdf_pieza(V_s)
408 RMS_MC = float(np.sqrt(np.mean(dev_mc ** 2)))
409 RMS_FIN = float(np.sqrt(np.mean(dev_fin ** 2)))
410 P95 = float(np.percentile(np.abs(dev_fin), 95))
411 MAXD = float(np.max(np.abs(dev_fin)))
412 BIAS = float(np.mean(dev_fin))
413 E_VOL = 100.0 * (aud1['volumen'] / VOL_GT - 1.0)
414
415 print('-' * 72)
416 print('VERIFICACION CONTRA LA VERDAD DE TERRENO')
417 print('    RMS tras Marching Cubes      = {:.4f} mm'.format(RMS_MC))
418 print('    RMS de la malla entregable    = {:.4f} mm'.format(RMS_FIN))

```

```

419 print('      sesgo medio          = {:.4f} mm'.format(BIAS))
420 print('      P95 de |desviacion|    = {:.4f} mm'.format(P95))
421 print('      maximo de |desviacion|    = {:.4f} mm'.format(MAXD))
422 print('      volumen GT = {:.1f} mm3; malla = {:.1f} mm3; error = '
423       '{:+.3f} por ciento'.format(VOL_GT, aud1['volumen'], E_VOL))
424 print('      area superficial de la malla = {:.1f} mm2'.format(aud1['area']))
425 print('      s_n de entrada = {:.3f} mm -> reduccion efectiva = {:.2f}x'
426       '.format(S_N, S_N / RMS_FIN))
427 print('=' * 72)
428
429 # -----
430 # 10. Figura de tres paneles
431 # -----
432 AZUL = '#003366'
433 ROJO = '#B22222'
434 VERDE = '#2E7D32'
435
436 fig = plt.figure(figsize=(15.0, 4.6))
437
438 ax0 = fig.add_subplot(1, 3, 1, projection='3d')
439 sub = rng.choice(len(P_all), 5000, replace=False)
440 inl = sub[~es_outlier[sub]]
441 out = sub[es_outlier[sub]]
442 ax0.scatter(P_all[inl, 0], P_all[inl, 1], P_all[inl, 2], s=0.8,
443            c=P_all[inl, 2], cmap='viridis', alpha=0.55)
444 ax0.scatter(P_all[out, 0], P_all[out, 1], P_all[out, 2], s=9,
445            color=ROJO, marker='x', lw=0.8, label='outliers inyectados')
446 ax0.set_title('(a) Nube cruda: ruido $\sigma_n$ = {:.2f} mm + '
447             '{:0f} por ciento de outliers'.format(S_N, 100 * F_OUT),
448             fontsize=9.5, loc='left')
449 ax0.set_xlabel('X [mm]', fontsize=8)
450 ax0.set_ylabel('Y [mm]', fontsize=8)
451 ax0.set_zlabel('Z [mm]', fontsize=8)
452 ax0.tick_params(labelsize=7)
453 ax0.legend(fontsize=7.5, frameon=False, loc='upper right')
454 ax0.view_init(elev=18, azim=35)
455
456 ax1 = fig.add_subplot(1, 3, 2, projection='3d')
457 V_p, F_p = decimar_por_agrupamiento(V_s, F_m, 2.20) # solo para render
458 V_p, F_p, _ = componente_mayor(V_p, F_p)
459 tri_p = V_p[F_p]
460 col = plt.cm.viridis((tri_p[:, :, 2].mean(1) - V_s[:, :, 2].min())
461                     / np.ptp(V_s[:, :, 2]))
462 pc = Poly3DCollection(tri_p, facecolors=col, edgecolors='k',
463                     linewidths=0.12, alpha=1.0)
464 pc.set_sort_zpos(0)
465 ax1.add_collection3d(pc)
466 ax1.set_xlim(V_s[:, 0].min(), V_s[:, 0].max())
467 ax1.set_ylim(V_s[:, 1].min(), V_s[:, 1].max())
468 ax1.set_zlim(V_s[:, 2].min(), V_s[:, 2].max())
469 ax1.set_box_aspect((1, 1, np.ptp(V_s[:, 2]) / np.ptp(V_s[:, 0])))
470 ax1.set_title('(b) Malla entregable: {:d} caras, estanca, $\chi$ = {:d} '
471             '(genero 1)\n(vista simplificada a {:d} caras)'
472             .format(aud1['F'], aud1['chi'], len(F_p)),
473             fontsize=9.5, loc='left')
474 ax1.set_xlabel('X [mm]', fontsize=8)
475 ax1.set_ylabel('Y [mm]', fontsize=8)
476 ax1.set_zlabel('Z [mm]', fontsize=8)
477 ax1.tick_params(labelsize=7)
478 ax1.view_init(elev=18, azim=35)

```



```

479
480 ax2 = fig.add_subplot(1, 3, 3)
481 ax2.hist(dev_fin * 1000, bins=90, color=AZUL, alpha=0.82,
482         edgecolor='none', density=True)
483 ax2.axvline(0.0, color='k', lw=0.8)
484 for x, c, lab in [(RMS_FIN * 1000, ROJO, 'RMS = {:.0f} $\mu$m'
485                  .format(RMS_FIN * 1000)),
486                  (-RMS_FIN * 1000, ROJO, None),
487                  (P95 * 1000, VERDE, 'P95 = {:.0f} $\mu$m'
488                  .format(P95 * 1000)),
489                  (-P95 * 1000, VERDE, None)]:
490     ax2.axvline(x, color=c, ls='--', lw=1.3, label=lab)
491 ax2.set_xlim(-300, 300)
492 ax2.set_xlabel('Desviacion malla - solido analitico [$\mu$m]')
493 ax2.set_ylabel('Densidad [1/$\mu$m]')
494 ax2.set_title('(c) Verificacion dimensional (sesgo = {:.0f} $\mu$m'
495              .format(BIAS * 1000), fontsize=9.5, loc='left')
496 ax2.legend(fontsize=8, frameon=False)
497 ax2.grid(alpha=0.3)
498
499 fig.tight_layout()
500 fig.savefig('Figuras/S12_nube_a_malla.png', dpi=300, bbox_inches='tight')
501 # plt.show()
502 print('Figura guardada: Figuras/S12_nube_a_malla.png')

```

Listing 54: Pipeline completo del Ejemplo 3: filtrado robusto (Ec. 276), normales por PCA (Ec. 277), reconstrucción implícita (Ec. 283), auditoría de Euler–Poincaré (Ec. 284), suavizado de Taubin (Ec. 287) y exportación a STL binario.

Cierre del curso

Estas catorce semanas recorrieron un solo argumento contado con distintos vocabularios: todo resultado computacional es una afirmación cuantitativa sobre el mundo físico, y esa afirmación vale exactamente lo que valga la cadena de supuestos, discretizaciones e incertidumbres que la sostienen.

El curso no enseñó herramientas sueltas; enseñó los eslabones de esa cadena, en el orden en que un investigador los necesita.

Las Semanas 1 a 3 sentaron el vocabulario común: qué es un modelo, qué lo distingue de un ajuste, y qué garantías — o su ausencia — ofrece un método numérico antes de confiar en su salida.

Las Semanas 4 y 5 llevaron ese vocabulario a ecuaciones diferenciales y campos bidimensionales, la forma en que la mayoría de los fenómenos de ingeniería se expresan primero.

La Semana 6 introdujo la pregunta que separa un cálculo de una medición: no basta un número, hace falta su incertidumbre, propagada con el mismo rigor que la ley de propagación de incertidumbres del GUM, Ec. (177), exige a cualquier magnitud reportada.

La Semana 7 mostró que el azar y el determinismo no son categorías opuestas, sino límites uno del otro, y que un proceso de Markov describe con la misma naturalidad la falla de un componente que la deriva de un algoritmo.

Las Semanas 8 a 10 llevaron todo esto al método de elementos finitos: primero como formulación matemática y decisión de malla, después como herramienta capaz de responder preguntas de resistencia, fatiga, vibración y dinámica de mecanismos con la misma disciplina de verificación aprendida desde la Semana 3.

La Semana 11 cerró el círculo que toda simulación abre y deja pendiente: ningún modelo se valida a sí mismo: se valida contra evidencia experimental, con un protocolo que distingue verificación de validación de calibración y que no permite usar los mismos datos para las tres cosas.

La Semana 12, por último, invirtió la dirección del problema: en vez de partir de una geometría CAD para simular, partió del objeto físico para reconstruirlo, con la misma exigencia de trazabilidad métrica y auditoría que las semanas anteriores exigieron a cualquier otro resultado.

Para quienes cursan el Doctorado en Ingeniería y Tecnología Aplicada de la Universidad La Salle Bajío, este recorrido no es un catálogo de técnicas independientes del proyecto de tesis: es, con pocas variaciones, el ciclo metodológico que un comité doctoral espera ver documentado en cualquier investigación que combine modelado, simulación y evidencia experimental. Formular el modelo con sus hipótesis explícitas, elegir y verificar el método numérico o el software comercial que lo resuelve, cuantificar la incertidumbre de cada resultado reportado, y validar contra datos independientes de los usados para calibrar: ese es el estándar mínimo de una contribución doctoral defendible,

y es exactamente el que este curso practicó semana a semana.

Aplíquelo sin excepción a su propio problema de investigación, incluso — sobre todo — cuando el resultado preliminar ya parezca convincente en pantalla: la Semana 11 fue explícita en que el parecido visual nunca es evidencia de exactitud.